GUIDE DU DÉVELOPPEUR EXHAUSTIF

SOLVEUR DIPHASIQUE ET THERMOCONVECTIF

COVOF 2D

Architecture Logicielle, Formulations Mathématiques Discrètes, Algorithmes Numériques et Analyse Intégrale des Codes Sources

---

PARTIE I : Implémentation Orientée Objet C++17 (`COVOF_2D-C++`)

PARTIE II : Implémentation Modulaire Fortran 2018 (`COVOF_2D-FORTRAN`)

Méthodes Numériques Documentées :
Volumes Finis sur Grille Décalée MAC • Lagrangien Augmenté Monolithique (r_{al}) et Algorithme d'Uzawa
Solveurs de Krylov Parallèles OpenMPI : BiCGSTAB(2), BiCGSTAB, PCG, GMRES(m), MINRES
Capture d'Interface VOF : Reconstruction Géométrique PLIC, ELVIRA, LVIRA, SLIC
Advection Conservatrice : Weymouth & Yue (2010), CIAM (Li 1995), TVD (Superbee, MC, Van Leer, Minmod, HRIC)
Tension Superficielle CSF Balanced-Force • Pénalisation Volumique de Brinkman • Thermoconvection de Boussinesq

Version 3.0.0 — Octobre 2026

Table des matières

I Guide du Développeur C++17 (`COVOF_2D-C++`) 2

1 Architecture Globale C++17, Système de Build CMake et Types Fondamentaux (Types.hpp, main.cpp) 3
1.1 Philosophie Architecturale et Standard C++17 . . . 3
1.2 Structure des Blocs de Configuration dans `Types.hpp` . . . 3
1.3 Cycle de Vie du Programme Principal (`src/main.cpp`) . . . 4
1.4 Code Source Intégral Annoté : `CMakeLists.txt` . . . 4
1.5 Répertoire des Classes, Structures et Méthodes de `include/covof/Types.hpp` . . . 5
1.6 Code Source Intégral Annoté : `include/covof/Types.hpp` . . . 5
1.7 Répertoire des Classes, Structures et Méthodes de `src/main.cpp` . . . 7
1.8 Code Source Intégral Annoté : `src/main.cpp` . . . 7

2 Analyseur Syntaxique JSON et Gestion de la Configuration (Config.hpp, Config.cpp) 9
2.1 Conception de l'Analyseur JSON Autonome Sans Dépendance Externe . . . 9
2.1.1 Grammaire Reconnue et Extraction par Sous-Blocs . . . 9
2.2 Compatibilité Ascendante et Alias de Clés JSON . . . 9
2.3 Répertoire des Classes, Structures et Méthodes de `include/covof/Config.hpp` . . . 10
2.4 Code Source Intégral Annoté : `include/covof/Config.hpp` . . . 10
2.5 Répertoire des Classes, Structures et Méthodes de `src/Config.cpp` . . . 11
2.6 Code Source Intégral Annoté : `src/Config.cpp` . . . 11

3 Topologie du Maillage Cartésien MAC 2D et Décomposition Spatiale (Mesh.hpp, Mesh.cpp) 18
3.1 Géométrie de la Grille Décalée MAC (*Marker-And-Cell*) . . . 18
3.2 Systèmes de Coordonnées Décalées (x_c, y_c), (x_u, y_c) et (x_c, y_v) . . . 18
3.3 Répertoire des Classes, Structures et Méthodes de `include/covof/Mesh.hpp` . . . 18
3.4 Code Source Intégral Annoté : `include/covof/Mesh.hpp` . . . 19
3.5 Code Source Intégral Annoté : `src/Mesh.cpp` . . . 20

4 Conteneurs de Champs Eulériens 2D et Opérations Vectorielles (Field.hpp) 22
4.1 Conception du Template `Field2D<T>` . . . 22
4.1.1 Adressage Avec Cellules Fantômes (*Ghost Cells*) . . . 22
4.2 Répertoire des Classes, Structures et Méthodes de `include/covof/Field.hpp` . . . 22
4.3 Code Source Intégral Annoté : `include/covof/Field.hpp` . . . 22

5 Parallélisme Distribué OpenMPI et Échanges de Halos Cartésiens (MpiComm.hpp, MpiComm.cpp) 24
5.1 Topologie Cartésienne 2D et Découverte des Voisins . . . 24
5.2 Protocole d'Échange de Halos Sans Interblocage (*Deadlock-Free Halo Exchange*) . . . 24
5.3 Répertoire des Classes, Structures et Méthodes de `include/covof/MpiComm.hpp` . . . 24
5.4 Code Source Intégral Annoté : `include/covof/MpiComm.hpp` . . . 25
5.5 Répertoire des Classes, Structures et Méthodes de `src/MpiComm.cpp` . . . 25
5.6 Code Source Intégral Annoté : `src/MpiComm.cpp` . . . 26

6 Algèbre Linéaire Creuse au Format CSR (CSRMatrix.hpp, CSRMatrix.cpp) 35
6.1 Représentation Compressed Sparse Row (CSR) . . . 35
6.2 Produit Matrice-Vecteur Creux (`multiply` / SpMV) . . . 35
6.3 Répertoire des Classes, Structures et Méthodes de `include/covof/CSRMatrix.hpp` . . . 35
6.4 Code Source Intégral Annoté : `include/covof/CSRMatrix.hpp` . . . 35
6.5 Répertoire des Classes, Structures et Méthodes de `src/CSRMatrix.cpp` . . . 36
6.6 Code Source Intégral Annoté : `src/CSRMatrix.cpp` . . . 36

7 Préconditionneurs Algébriques : Jacobi, SSOR et ILU(0) (Preconditioner.hpp, Preconditioner.cpp) 39
7.1 Rôle du Préconditionnement dans les Méthodes de Krylov . . . 39
7.1.1 Préconditionneur Diagonal de Jacobi (`JacobiPreconditioner`) . . . 39
7.1.2 Préconditionneur SSOR et Factorisation Incomplète `ILU(0)` . . . 39
7.2 Répertoire des Classes, Structures et Méthodes de `include/covof/Preconditioner.hpp` . . . 39
7.3 Code Source Intégral Annoté : `include/covof/Preconditioner.hpp` . . . 40
7.4 Répertoire des Classes, Structures et Méthodes de `src/Preconditioner.cpp` . . . 42
7.5 Code Source Intégral Annoté : `src/Preconditioner.cpp` . . . 42

8 Solveurs de Sous-Espaces de Krylov : BiCGSTAB(2), PCG, GMRES (LinearSolver.hpp, LinearSolver.cpp) 46
8.1 Architecture des Solveurs de Krylov Parallèles . . . 46
8.2 Algorithme Mathématique Complet de BiCGSTAB(2) (Sleijpen & Fokkema 1993) . . . 46
8.3 Répertoire des Classes, Structures et Méthodes de `include/covof/LinearSolver.hpp` . . . 47
8.4 Code Source Intégral Annoté : `include/covof/LinearSolver.hpp` . . . 47
8.5 Répertoire des Classes, Structures et Méthodes de `src/LinearSolver.cpp` . . . 50
8.6 Code Source Intégral Annoté : `src/LinearSolver.cpp` . . . 50

9 Solveur Monolithique de Navier-Stokes par Lagrangien Augmenté (NavierStokesSolver.hpp, NavierStokesSolver.cpp) 60
9.1 Formulation Monolithique et Numérotation Globale des Inconnues (u, v) . . . 60
9.2 Discrétisation Exacte de l'Opérateur du Lagrangien Augmenté $-r_{al}\nabla_h(\nabla_h \cdot \mathbf{u})$ . . . 60
9.3 Tenseur Visqueux à Viscosité Variable et Moyenne Harmonique Nodale . . . 60
9.4 Répertoire des Classes, Structures et Méthodes de `include/covof/NavierStokesSolver.hpp` . . . 61
9.5 Code Source Intégral Annoté : `include/covof/NavierStokesSolver.hpp` . . . 61
9.6 Répertoire des Classes, Structures et Méthodes de `src/NavierStokesSolver.cpp` . . . 63
9.7 Code Source Intégral Annoté : `src/NavierStokesSolver.cpp` . . . 63

10 Reconstruction Géométrique d'Interface VOF : PLIC, ELVIRA, LVIRA, SLIC (InterfaceReconstruction.hpp/.cpp) 77
10.1 Représentation Linéaire par Morceaux PLIC (*Piecewise Linear Interface Calculation*) . . . 77
10.2 Relations Analytiques Directe et Inverse de Scardovelli & Zaleski (2000) . . . 77
10.3 Estimation des Normales : Parker-Youngs, ELVIRA et LVIRA . . . 77
10.4 Répertoire des Classes, Structures et Méthodes de `include/covof/InterfaceReconstruction.hpp` 78
10.5 Code Source Intégral Annoté : `include/covof/InterfaceReconstruction.hpp` . . . 78
10.6 Répertoire des Classes, Structures et Méthodes de `src/InterfaceReconstruction.cpp` . . . 79
10.7 Code Source Intégral Annoté : `src/InterfaceReconstruction.cpp` . . . 79

11 Transport et Advection Conservatrice VOF : Weymouth-Yue, CIAM et TVD (InterfaceTracker.hpp/.cpp) 87
11.1 Séparation Directionnelle d'Ordre 2 de Strang (*Strang Operator Splitting*) . . . 87
11.2 Schéma de Weymouth & Yue (2010) à Conservation de Masse Exacte . . . 87
11.3 Répertoire des Classes, Structures et Méthodes de `include/covof/InterfaceTracker.hpp` . . . 88
11.4 Code Source Intégral Annoté : `include/covof/InterfaceTracker.hpp` . . . 88
11.5 Répertoire des Classes, Structures et Méthodes de `src/InterfaceTracker.cpp` . . . 89
11.6 Code Source Intégral Annoté : `src/InterfaceTracker.cpp` . . . 89

12 Exportation Graphique VTK XML Parallèle (.vtr / .pvtr) (VTKWriter.hpp, VTKWriter.cpp) 99
12.1 Architecture des Fichiers VTK XML `RectilinearGrid` et `PRectilinearGrid` . . . 99
12.2 Répertoire des Classes, Structures et Méthodes de `include/covof/VTKWriter.hpp` . . . 99
12.3 Code Source Intégral Annoté : `include/covof/VTKWriter.hpp` . . . 99
12.4 Répertoire des Classes, Structures et Méthodes de `src/VTKWriter.cpp` . . . 100
12.5 Code Source Intégral Annoté : `src/VTKWriter.cpp` . . . 100

13 Orchestration de la Simulation, Pas de Temps Adaptatif et Diagnostics VV (Simulation.hpp, Simulation.cpp) 105
13.1 Orchestration Globale dans `covof::Simulation` . . . 105
13.2 Répertoire des Classes, Structures et Méthodes de `include/covof/Simulation.hpp` . . . 105
13.3 Code Source Intégral Annoté : `include/covof/Simulation.hpp` . . . 105
13.4 Répertoire des Classes, Structures et Méthodes de `src/Simulation.cpp` . . . 107
13.5 Code Source Intégral Annoté : `src/Simulation.cpp` . . . 107

II Guide du Développeur Fortran 2018 (`COVOF_2D-FORTRAN`) 118

14 Architecture Modulaire Fortran 2018, Précision IEEE-754 et Programme Principal (CMake-Lists.txt, module$_{precision}$.f90, module$_{constants}$.f90, covof2d$_{main}$.f90) 119
14.1 Conception Modulaire selon la Norme Fortran 2018 (ISO/IEC 1539-1 :2018) . . . 119
14.1.1 Gestion de la Précision Flottante (`module_precision.f90`) . . . 119
14.1.2 Table de Correspondance Bijective C++17 ↔ Fortran 2018 . . . 119
14.2 Code Source Intégral Annoté : `CMakeLists.txt` . . . 119
14.3 Répertoire des Types Dérivés, Sous-Routines et Fonctions de `src/module_precision.f90` . . . 120
14.4 Code Source Intégral Annoté : `src/module_precision.f90` . . . 120
14.5 Répertoire des Types Dérivés, Sous-Routines et Fonctions de `src/module_constants.f90` . . . 121
14.6 Code Source Intégral Annoté : `src/module_constants.f90` . . . 121
14.7 Code Source Intégral Annoté : `src/covof2d_main.f90` . . . 122

15 Analyseur Syntaxique JSON Natif en Fortran 2018 et Module de Configuration (module$_{json}$.f90, module$_{confi}$
15.1 Architecture du Parseur JSON Pur Fortran 2018 (`module_json.f90`) . . . 124
15.2 Structure de Configuration `simulation_config_t` (`module_config.f90`) . . . 124
15.3 Répertoire des Types Dérivés, Sous-Routines et Fonctions de `src/module_json.f90` . . . 124
15.4 Code Source Intégral Annoté : `src/module_json.f90` . . . 125
15.5 Répertoire des Types Dérivés, Sous-Routines et Fonctions de `src/module_config.f90` . . . 130
15.6 Code Source Intégral Annoté : `src/module_config.f90` . . . 130

16 Maillage MAC 2D et Tableaux Column-Major à Bornes Fantômes en Fortran 2018 (module$_{mesh}$.f90, module$_{fi}$
16.1 Indiçage Naturel à Base 1 et Bornes Fantômes (`1-ng:nx+ng`) en Fortran 2018 . . . 136
16.2 Ordre Mémoire *Column-Major* et Ordre Optimal des Boucles Imbriquées . . . 136
16.3 Répertoire des Types Dérivés, Sous-Routines et Fonctions de `src/module_mesh.f90` . . . 137
16.4 Code Source Intégral Annoté : `src/module_mesh.f90` . . . 137
16.5 Répertoire des Types Dérivés, Sous-Routines et Fonctions de `src/module_fields.f90` . . . 139
16.6 Code Source Intégral Annoté : `src/module_fields.f90` . . . 139

17 Communications Parallèles MPI et Échanges de Halos en Fortran 2018 (module$_{mpi_comm}$.f90) 142
17.1 Gestion de la Topologie Cartésienne et des Halos dans `module_mpi_comm.f90` . . . 142
17.2 Répertoire des Types Dérivés, Sous-Routines et Fonctions de `src/module_mpi_comm.f90` . . . 142
17.3 Code Source Intégral Annoté : `src/module_mpi_comm.f90` . . . 142

18 Matrices Creuses CSR, Préconditionneurs et Solveurs de Krylov en Fortran 2018 (module$_{linear_solvers}$.f90) 15
18.1 Implémentation Vectorisée des Solveurs de Krylov dans `module_linear_solvers.f90` . . . 152
18.2 Répertoire des Types Dérivés, Sous-Routines et Fonctions de `src/module_linear_solvers.f90` . . . 152
18.3 Code Source Intégral Annoté : `src/module_linear_solvers.f90` . . . 152

19 Solveur Monolithique de Navier-Stokes par Lagrangien Augmenté en Fortran 2018 (module$_{navier_stokes}$.f90) 1
19.1 Architecture des 1540 Lignes du Module `module_navier_stokes.f90` . . . 163
19.2 Répertoire des Types Dérivés, Sous-Routines et Fonctions de `src/module_navier_stokes.f90` . . . 163
19.3 Code Source Intégral Annoté : `src/module_navier_stokes.f90` . . . 164

20 Reconstruction Géométrique d'Interface VOF en Fortran 2018 (module$_{interface_reconstruction}$.f90) 182
20.1 Algorithmes Géométriques dans `module_interface_reconstruction.f90` . . . 182
20.2 Répertoire des Types Dérivés, Sous-Routines et Fonctions de `src/module_interface_reconstruction.f90` 182
20.3 Code Source Intégral Annoté : `src/module_interface_reconstruction.f90` . . . 182

21 Advection Géométrique et Algébrique VOF en Fortran 2018 (module$_{interface_vof}$.f90) 190
21.1 Architecture des 1034 Lignes de `module_interface_vof.f90` . . . 190
21.2 Répertoire des Types Dérivés, Sous-Routines et Fonctions de `src/module_interface_vof.f90` . . . 190
21.3 Code Source Intégral Annoté : `src/module_interface_vof.f90` . . . 190

22 Générateur VTK XML Parallèle en Fortran 2018 (module$_{vtk_writer}$.f90) 203
22.1 Écriture Conforme des Fichiers `.vtr` et `.pvtr` (`module_vtk_writer.f90`) . . . 203
22.2 Répertoire des Types Dérivés, Sous-Routines et Fonctions de `src/module_vtk_writer.f90` . . . 203
22.3 Code Source Intégral Annoté : `src/module_vtk_writer.f90` . . . 203

23 Boucle Temporelle Principale, Diagnostics Intégraux et Vérification en Fortran 2018 (module_*simulation.f*90) 2
23.1 Orchestration Complète dans `module_simulation.f90` (1004 Lignes) 210
23.2 Répertoire des Types Dérivés, Sous-Routines et Fonctions de `src/module_simulation.f90` 210
23.3 Code Source Intégral Annoté : `src/module_simulation.f90` 210

Première partie

Guide du Développeur C++17 (COVOF_2D-C++)

Chapitre 1

Architecture Globale C++17, Système de Build CMake et Types Fondamentaux (Types.hpp, main.cpp)

1.1 Philosophie Architecturale et Standard C++17

L'implémentation **C++17** du solveur **COVOF 2D** (`COVOF_2D-C++`) repose sur une architecture orientée objet modulaire à couplage faible et forte cohésion interne, encapsulée dans l'espace de noms `covof`. Les principes directeurs d'ingénierie logicielle retenus sont les suivants :

1. **Zéro allocation dynamique dans la boucle temporelle** : tous les champs eulériens (`Field2D<double>`), les tampons d'échange de halos MPI et la structure de la matrice creuse CSR (`CSRMatrix`) sont pré-alloués lors de la phase d'initialisation (`Simulation::initialize`).
2. **Contiguïté mémoire stricte (*Row-Major Order*)** : chaque champ 2D de dimension locale $N_x \times N_y$ entouré de n_g couches de cellules fantômes (*ghost cells*) est stocké dans un unique `std::vector<double>` unidimensionnel de taille $(N_x + 2n_g)(N_y + 2n_g)$. L'indice linéaire correspondant à la maille (i, j) est donné par :
$$k_{\mathrm{C++}}(i, j) = (j + n_g) \times (N_x + 2n_g) + (i + n_g) \tag{1.1}$$
Ainsi, une boucle interne parcourant l'indice horizontal $i \in \{0, \ldots, N_x - 1\}$ à ordonnée j fixée accède à des adresses mémoire strictement consécutives ($\Delta k = 1$), maximisant l'utilisation des lignes de cache L1/L2 de 64 octets et permettant la vectorisation automatique SIMD (AVX2 / AVX-512 / NEON Apple Silicon).
3. **Typage fort des modèles physiques et numériques (`Types.hpp`)** : l'ensemble des choix algorithmiques (type de condition aux limites `BoundaryType`, schéma d'advection VOF `AdvectionMethod`, méthode de reconstruction géométrique `ReconstructionMethod`, limiteur de flux `TVDLimiter`, solveur de Krylov `SolverType`, préconditionneur `PreconditionerType`, mode d'initialisation `InitialConditionType`) est représenté par des énumérations fortement typées (`enum class`) interdisant toute conversion implicite erronée.

1.2 Structure des Blocs de Configuration dans `Types.hpp`

L'en-tête `include/covof/Types.hpp` définit également l'intégralité des structures de données de paramétrage (*Plain Old Data structs*) qui reflètent bijectivement l'arborescence des fichiers JSON du répertoire `cases/*.json` :

- `MeshConfig` : dimensions entières globales `nx`, `ny` et dimensions physiques `lx`, `ly` [m].
- `PhysicsConfig` : masses volumiques `rho1`, `rho2` [kg/m^3], viscosités dynamiques `mu1`, `mu2` [Pa · s], gravité (`gx`, `gy`) [m/s^2], tension superficielle `sigma` [N/m] et paramètres de viscosité variable exponentielle `variable_viscosity`, `visc_law`, `visc_a`, `visc_e`.
- `ThermalConfig` : drapeau `enable_energy`, diffusivité thermique α (`alpha_th`), coefficient d'expansion thermique de Boussinesq β (`beta_th`), température de référence T_0 (`t_ref`) et températures imposées (`t_hot`, `t_cold`).
- `VofConfig` : méthode de reconstruction géométrique (`PLIC`, `ELVIRA`, `LVIRA`, `SLIC`, `NONE`), schéma d'advection (`WEYMOUTH_YUE`, `CIAM`, `TVD`), limiteur TVD (`SUPERBEE`, `MC`, `VAN_LEER`, `MINMOD`, `HRIC`) et nombre de Courant interfacial maximal `cfl_vof`.
- `SolverConfig` : solveur de Krylov (`BICGSTAB2`, `BICGSTAB`, `PCG`, `GMRES`, `MINRES`), préconditionneur (`JACOBI`, `SSOR`, `ILU0`), tolérance relative `tol`, nombre maximal d'itérations `max_iter`, paramètre de pénalisation du Lagrangien Augmenté r_{al} (`r_al`) et nombre de sous-itérations d'Uzawa (`uzawa_iter`).

1.3 Cycle de Vie du Programme Principal (`src/main.cpp`)

Le point d'entrée `main(int argc, char* argv[])` orchestre le cycle de vie MPI et l'exécution complète d'un cas test en quatre étapes :

1. Initialisation de l'environnement parallèle via `MPI_Init(&argc, &argv)` et interrogation du rang global `rank` et du nombre de processus `size`.
2. Lecture et validation du fichier JSON passé en argument de ligne de commande via `Config::load_from_file(json_pat`
3. Instanciation de l'objet maître `Simulation sim(cfg, MPI_COMM_WORLD)`, appel à `sim.initialize()` puis exécution de la boucle temporelle `sim.run()` chronométrée par `MPI_Wtime()`.
4. Libération ordonnée des ressources MPI via `MPI_Finalize()`.

1.4 Code Source Intégral Annoté : `CMakeLists.txt`

Le listing ci-dessous présente l'intégralité des 48 lignes du fichier `CMakeLists.txt` de l'implémentation C++17 :

```
cmake_minimum_required(VERSION 3.16)
project(COVOF_2D_CPP CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_EXPORT_COMPILE_COMMANDS ON)

# Optimization and warning flags
set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} -O3 -Wall -Wextra -Wno-unused-parameter")

# Find OpenMPI / MPI
find_package(MPI REQUIRED)
include_directories(SYSTEM ${MPI_INCLUDE_PATH})

# Include directory
include_directories(${CMAKE_CURRENT_SOURCE_DIR}/include)

# Source files
set(SOURCES
    src/Config.cpp
    src/Mesh.cpp
    src/MpiComm.cpp
    src/CSRMatrix.cpp
    src/Preconditioner.cpp
    src/LinearSolver.cpp
    src/InterfaceReconstruction.cpp
    src/InterfaceTracker.cpp
    src/NavierStokesSolver.cpp
    src/VTKWriter.cpp
    src/Simulation.cpp
    src/main.cpp
)

# Executable target: covof2d
add_executable(covof2d ${SOURCES})
target_link_libraries(covof2d PRIVATE MPI::MPI_CXX)

set_target_properties(covof2d PROPERTIES
    RUNTIME_OUTPUT_DIRECTORY "${CMAKE_CURRENT_SOURCE_DIR}/bin"
)

# Create compatibility symlink/target for fugu2d
add_custom_command(TARGET covof2d POST_BUILD
    COMMAND ${CMAKE_COMMAND} -E create_symlink
        ${CMAKE_CURRENT_SOURCE_DIR}/bin/covof2d
        ${CMAKE_CURRENT_SOURCE_DIR}/bin/fugu2d
    COMMENT "Creating compatibility symlink bin/fugu2d -> bin/covof2d"
)
```

1.5 Répertoire des Classes, Structures et Méthodes de `include/covof/Types.h`

Ligne	Catégorie	Identifiant C++17	Signature / Rôle
14	enum class	`BCType`	Déclaration de type / classe / structure
26	struct	`BoundaryCondition`	Déclaration de type / classe / structure
52	struct	`Obstacle`	Déclaration de type / classe / structure
73	struct	`DomainBC`	Déclaration de type / classe / structure
87	struct	`FluidProperties`	Déclaration de type / classe / structure
103	enum class	`InitialConditionType`	Déclaration de type / classe / structure
119	struct	`InitialConditionConfig`	Déclaration de type / classe / structure
149	struct	`SolverOptions`	Déclaration de type / classe / structure
169	struct	`OutputConfig`	Déclaration de type / classe / structure
185	struct	`VofConfig`	Déclaration de type / classe / structure

1.6 Code Source Intégral Annoté : `include/covof/Types.hpp`

Le listing ci-dessous présente l'intégralité des 196 lignes du fichier `include/covof/Types.hpp` de l'implémentation C++17 :

```cpp
#pragma once

#include <string>
#include <vector>
#include <iostream>
#include <cmath>

namespace covof {
namespace fugu = covof; // Alias rétrocompatibilité

/**
 * @brief Types de conditions aux limites physiques (Annexe B thèse M. El Ouafa)
 */
enum class BCType {
    // Calcul et assignation numérique
    DIRICHLET,  ///< Vitesse imposée (adhérence u=0, paroi mobile u=U_wall, ou profil d'entrée)
    // Calcul de norme euclidienne ou résidu de convergence
    NEUMANN,    ///< Dérivée normale nulle (sortie libre dU/dn = 0)
    FREE_SLIP,  ///< Glissement libre (composante normale nulle, contrainte tangentielle nulle)
    PERIODIC    ///< Conditions aux limites périodiques
};

/**
 * @brief Spécification des conditions aux limites sur les 4 frontières du domaine 2D
 */
struct BoundaryCondition {
    // Calcul et assignation numérique
    BCType type = BCType::DIRICHLET;
    // Déclaration de variable typée
    double u_val = 0.0;  ///< Valeur de u imposée si Dirichlet
    // Déclaration de variable typée
    double v_val = 0.0;  ///< Valeur de v imposée si Dirichlet
    // Déclaration de variable typée
    std::string profile = "UNIFORM"; ///< "UNIFORM", "PARABOLIC", "POISEUILLE_STEP"
    // Déclaration de variable typée
    double u_mean = 0.0; ///< Vitesse moyenne pour profils paraboliques
    // Déclaration de variable typée
    double y_step = 0.0; ///< Hauteur de la marche pour POISEUILLE_STEP
    // Déclaration de variable typée pour la largeur de l'injecteur (Cas 3.3 Fluids 2021)
    double jet_width = 0.08; ///< Largeur de l'injecteur pour profil JET_INLET [m]
    // Déclaration de variable typée pour le centre de l'injecteur
    double jet_center = 0.5; ///< Position centrale de l'injecteur selon X [m]
    // Calcul et assignation numérique
    BCType T_type = BCType::NEUMANN; ///< Type de CL pour l'énergie (DIRICHLET ou NEUMANN)
    // Déclaration de variable typée
    double T_val = 0.0;              ///< Température imposée si DIRICHLET
};

/**
 * @brief Obstacle immergé avec pénalisation de Brinkman (Standard CFD)
 */
struct Obstacle {
    // Déclaration de variable typée
    std::string type = "BOX"; ///< Forme géométrique ("BOX", "CYLINDER")
    // Déclaration de variable typée
    double x0 = 0.0;          ///< Borne inférieure X [m]
    // Déclaration de variable typée
    double y0 = 0.0;          ///< Borne inférieure Y [m]
    // Déclaration de variable typée
    double x1 = 0.0;          ///< Borne supérieure X [m]
    // Déclaration de variable typée
    double y1 = 0.0;          ///< Borne supérieure Y [m]
    // Déclaration de variable typée
    double xc = 0.0;          ///< Centre cylindre X [m]
    // Déclaration de variable typée
```

```
66      double yc = 0.0;          ///< Centre cylindre Y [m]
67      // Déclaration de variable typée
68      double radius = 0.0;      ///< Rayon cylindre [m]
69      // Déclaration de variable typée
70      double permeability = 1.0e-14; ///< Perméabilité de Darcy-Brinkman K
71  };
72
73  struct DomainBC {
74      // Calcul et assignation numérique
75      BoundaryCondition left;    ///< Frontière x = 0
76      // Calcul et assignation numérique
77      BoundaryCondition right;   ///< Frontière x = Lx
78      // Calcul et assignation numérique
79      BoundaryCondition bottom;  ///< Frontière y = 0
80      // Calcul et assignation numérique
81      BoundaryCondition top;     ///< Frontière y = Ly
82  };
83
84  /**
85   * @brief Propriétés physiques d'une phase fluide
86   */
87  struct FluidProperties {
88      // Déclaration de variable typée
89      double rho = 1.0;          ///< Masse volumique [kg/m^3]
90      // Déclaration de variable typée
91      double mu  = 1.0;          ///< Viscosité dynamique [Pa.s]
92      // Déclaration de variable typée
93      double Cp = 1006.0;        ///< Capacité calorifique à pression constante [J/(kg.K)]
94      // Déclaration de variable typée
95      double k_thermal = 0.026; ///< Conductivité thermique [W/(m.K)]
96      // Déclaration de variable typée
97      double beta = 3.33e-3;     ///< Coefficient de dilatation thermique volumique [1/K]
98  };
99
100 /**
101  * @brief Types de configurations initiales de la fraction volumique C
102  */
103 enum class InitialConditionType {
104     // Calcul et assignation numérique
105     SINGLE_PHASE,     ///< Écoulement monophasique (C = 1 partout)
106     DROPLET,          ///< Goutte (Fluide 1 à l'intérieur, Fluide 2 à l'extérieur)
107     BUBBLE,           ///< Bulle (Fluide 2 à l'intérieur, Fluide 1 à l'extérieur)
108     DAM_BREAK,        ///< Colonne rectangulaire d'eau (rupture de barrage)
109     SHEAR_LAYER,      ///< Nappe liquide cisaillée avec perturbation
110     LIQUID_SHEET,     ///< Nappe liquide cisaillée périodique (Cas 3.2 Fluids 2021)
111     VISCOUS_JET,      ///< Jet liquide visqueux injecté / flambement (Cas 3.3 Fluids 2021)
112     RAYLEIGH_TAYLOR, ///< Interface perturbée avec fluide lourd au-dessus
113     VORTEX_SHEAR      ///< Tâche circulaire déformée par vortex avec cisaillement
114 };
115
116 /**
117  * @brief Paramètres géométriques de la condition initiale diphasique
118  */
119 struct InitialConditionConfig {
120     // Calcul et assignation numérique
121     InitialConditionType type = InitialConditionType::SINGLE_PHASE;
122     // Déclaration de variable typée
123     double center_x = 0.5;    ///< Centre de la goutte en X [m]
124     // Déclaration de variable typée
125     double center_y = 0.5;    ///< Centre de la goutte en Y [m]
126     // Déclaration de variable typée
127     double radius   = 0.25;   ///< Rayon de la goutte [m]
128     // Déclaration de variable typée
129     double box_x0   = 0.0;    ///< Coin inf-gauche du rectangle (Dam break)
130     // Déclaration de variable typée
131     double box_y0   = 0.0;
132     // Déclaration de variable typée
133     double box_x1   = 1.0;    ///< Coin sup-droit du rectangle (Dam break)
134     // Déclaration de variable typée
135     double box_y1   = 1.0;
136     // Déclaration de variable typée
137     double pert_amp = 0.01;   ///< Amplitude de perturbation (Rayleigh-Taylor)
138     // Déclaration de variable typée
139     double pert_k   = 1.0;    ///< Nombre d'onde de perturbation
140     // Déclaration de variable typée pour l'épaisseur de nappe liquide
141     double thickness = 0.0003; ///< Épaisseur de la nappe liquide (Cas 3.2 Fluids 2021) [m]
142     // Déclaration de variable typée pour la largeur du jet liquide
143     double jet_width = 0.08;   ///< Largeur du jet liquide injecté (Cas 3.3 Fluids 2021) [m]
144 };
145
146 /**
147  * @brief Options de paramétrage du solveur linéaire de Krylov BiCGSTAB(2)
148  */
149 struct SolverOptions {
150     // Déclaration de variable typée
151     std::string type = "AUGMENTED_LAGRANGIAN"; ///< AUGMENTED_LAGRANGIAN ou MONOLITHIC
152     // Déclaration de variable typée
153     double tol = 1.0e-7;                       ///< Résidu relatif d'arrêt ||r_k|| / ||r_0||
154     // Déclaration de variable typée
155     int max_iter = 1000;                       ///< Nombre maximum d'itérations
156     // Déclaration de variable typée
157     double r_al = 500.0;                       ///< Paramètre de pénalité de divergence r [Pa.s ou kg/(m.s)]
```

```
    // Déclaration de variable typée
    int uzawa_iter = 1;                         ///< Nombre d'itérations d'Uzawa-Lagrangien augmenté
    // Déclaration de variable typée
    bool use_precond = true;                    ///< Activer le préconditionneur
    // Déclaration de variable typée
    std::string precond_type = "JACOBI";        ///< JACOBI, BLOCK_TRIANGULAR, ou NONE
};

/**
 * @brief Options de gestion des sorties et de la visualisation ParaView
 */
struct OutputConfig {
    // Déclaration de variable typée
    std::string output_dir = "output"; ///< Répertoire d'écriture
    // Déclaration de variable typée
    std::string case_name  = "covof";   ///< Préfixe des fichiers VTK
    // Déclaration de variable typée
    int vtk_interval = 10;              ///< Pas d'export VTK (tous les N pas de temps)
    // Déclaration de variable typée
    int log_interval = 1;               ///< Pas d'affichage console et CSV
    // Déclaration de variable typée
    bool write_csv_history = true;      ///< Écrire l'historique temporel (centre de masse, vitesse...)
};

/**
 * @brief Options de capture d'interface et d'advection VOF
 */
struct VofConfig {
    // Déclaration de variable typée
    std::string reconstruction_method = "PLIC";       ///< "PLIC", "ELVIRA", "LVIRA", "SLIC", "NONE"
    // Déclaration de variable typée
    std::string advection_method = "WEYMOUTH_YUE";    ///< "WEYMOUTH_YUE", "CIAM", "VOF_TVD"
    // Déclaration de variable typée
    std::string tvd_limiter = "NONE";                 ///< "SUPERBEE", "VAN_LEER", "MUSCL", "HRIC", "NONE"
    // Déclaration de variable typée
    double cfl_vof = 0.5;                             ///< Nombre de CFL maximum pour le transport VOF
};

} // namespace covof
```

1.7 Répertoire des Classes, Structures et Méthodes de `src/main.cpp`

Ligne	Catégorie	Identifiant C++17	Signature / Rôle
18	Méthode / Fonction	`main`	Type retour : int

1.8 Code Source Intégral Annoté : `src/main.cpp`

Le listing ci-dessous présente l'intégralité des 106 lignes du fichier `src/main.cpp` de l'implémentation C++17 :

```
// ================================================================================
// Projet         : COVOF 2D (Convection & Volume-Of-Fluid 2D CFD Solver)
// Fichier        : main.cpp
// Description    : Point d'entrée principal de l'exécutable COVOF 2D C++17 / OpenMPI.
//                  Initialise l'environnement parallèle MPI, analyse la ligne
//                  de commande, charge la configuration JSON, orchestre l'exécution
//                  de la simulation numérique et gère la libération des ressources.
// Auteur         : Elhadji Ibrahima Thiam
// Standard       : C++17 / MPI 3+
// ================================================================================

#include "covof/Simulation.hpp" // Orchestrateur principal de simulation (grille, solveur, temps)
#include <mpi.h>                // Bibliothèque Message Passing Interface pour le calcul parallèle
#include <iostream>             // Flux d'entrée/sortie standard pour l'affichage console
#include <string>               // Gestion des chaînes de caractères pour les chemins de fichiers

// Point d'entrée standard du programme en ligne de commande
int main(int argc, char** argv) {
    // 1. Initialisation de l'environnement d'exécution parallèle MPI
    // Distribue les arguments de la ligne de commande à tous les processus participants
    MPI_Init(&argc, &argv);

    // Récupère l'identifiant unique (rang) du processus courant dans le communicateur global
    int rank = 0;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    // Récupère le nombre total de processus alloués pour l'exécution parallèle
    int size = 1;
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // Fichier de configuration par défaut si aucun argument n'est fourni par l'utilisateur
    std::string config_file = "cases/cavity.json";

    // 2. Traitement des arguments de la ligne de commande
    if (argc > 1) {
```

```
        std::string arg = argv[1];
        // Détection de la demande d'aide (--help ou -h)
        if (arg == "-h" || arg == "--help") {
            // Seul le processus maître (rang 0) affiche le message d'aide pour éviter la duplication
            if (rank == 0) {
                std::cout << "
                    ================================================================================\n"
                          << "   COVOF 2D -- Solveur CFD Navier-Stokes Incompressible Parallèle (C++17/MPI)   \
                              n"
                          << "
                              ================================================================================\
                              n"
                          << "Usage :\n"
                          << "  mpirun -np <N> ./bin/covof2d [fichier_configuration.json]\n\n"
                          << "Exemples d'exécution :\n"
                          << "  mpirun -np 4 ./bin/covof2d cases/cavity_re1000.json\n"
                          << "  mpirun -np 4 ./bin/covof2d cases/rising_bubble.json\n"
                          << "  mpirun -np 2 ./bin/covof2d cases/static_droplet.json\n"
                          << "  mpirun -np 4 ./bin/covof2d cases/dam_break.json\n"
                          << "
                              ================================================================================\
                              n";
            }
            // Clôture propre de l'environnement MPI avant de quitter
            MPI_Finalize();
            return 0;
        }
        // Utilisation du chemin spécifié en premier argument
        config_file = arg;
    }

    // Affichage de la bannière d'accueil par le processus racine
    if (rank == 0) {
        std::cout << "================================================================================\n"
                  << "   COVOF 2D -- Solveur CFD Navier-Stokes Incompressible Parallèle (C++17/MPI)   \n"
                  << "================================================================================\n"
                  << "  Nombre de processus MPI  : " << size << "\n"
                  << "  Fichier de configuration : " << config_file << "\n"
                  << "================================================================================\n";
    }

    // 3. Bloc d'exécution sécurisé sous surveillance des exceptions C++
    try {
        // Instanciation de l'objet Simulation encapsulant maillage, champs, solveurs et exports
        covof::Simulation sim;

        // Initialisation de la simulation : lecture du JSON, partitionnement spatial, allocation
        if (!sim.init(MPI_COMM_WORLD, config_file)) {
            // En cas d'erreur de parsing ou de cohérence des données, signalement sur le rang 0
            if (rank == 0) {
                std::cerr << "[ERREUR CRITIQUE] Échec de l'initialisation de la simulation COVOF 2D.\n"
                          << "                  Vérifiez la validité du fichier JSON : " << config_file << "\n
                              ";
            }
            // Interruption d'urgence coordonnée de tous les processus du communicateur MPI
            MPI_Abort(MPI_COMM_WORLD, 1);
            return 1;
        }

        // Exécution de la boucle temporelle principale de calcul Navier-Stokes / VOF
        sim.run();

    } catch (const std::exception& e) {
        // Capture de toute exception standard (dépassement mémoire, assertion, I/O)
        if (rank == 0) {
            std::cerr << "[EXCEPTION INTERCEPTÉE] " << e.what() << "\n";
        }
        // Interruption immédiate de l'ensemble de la grappe de calcul
        MPI_Abort(MPI_COMM_WORLD, 1);
        return 1;
    }

    // 4. Clôture normale et libération de l'environnement parallèle OpenMPI
    MPI_Finalize();

    // Code de retour standard indiquant le succès de l'exécution
    return 0;
}
```

Chapitre 2

Analyseur Syntaxique JSON et Gestion de la Configuration (Config.hpp, Config.cpp)

2.1 Conception de l'Analyseur JSON Autonome Sans Dépendance Externe

Afin de garantir une portabilité immédiate sur tout supercalculateur ou station de travail sans nécessiter le téléchargement de bibliothèques tierces (telles que `nlohmann/json` ou `Boost.PropertyTree`), `COVOF_2D-C++` intègre dans `src/Config.cpp` un analyseur syntaxique JSON déterministe et léger.

2.1.1 Grammaire Reconnue et Extraction par Sous-Blocs

Le parseur opère en trois phases :

1. **Prétraitement et nettoyage lexical** : lecture intégrale du flux fichier dans une chaîne `std::string`, gestion des espaces, retours chariot et commentaires éventuels.
2. **Isolation des objets JSON de premier niveau (`extract_section`)** : pour une clé de section donnée (par exemple `"mesh"`, `"physics"`, `"vof"`, `"solver"`), l'automate recherche l'ouverture d'accolade `{` correspondante et maintient un compteur de profondeur d'imbrication depth $= \sum(\texttt{'\{'} - \texttt{'\}'})$ tout en ignorant les accolades situées à l'intérieur de littéraux chaînes entre guillemets. Cela permet d'isoler exactement la sous-chaîne JSON du bloc, y compris lorsqu'il contient des tableaux d'objets imbriqués comme `"obstacles": [ ... ]`.
3. **Désérialisation typée avec valeurs par défaut sécurisées** :
   — `get_string(section, key, default_val)` : extrait les chaînes de caractères et normalise la casse pour la résolution des énumérations ;
   — `get_double(section, key, default_val)` : parse les nombres flottants en notation décimale ou scientifique (`std::stod`) ;
   — `get_int(section, key, default_val)` et `get_bool(section, key, default_val)` : extraient les entiers et les booléens (`true`/`false`).

2.2 Compatibilité Ascendante et Alias de Clés JSON

Pour assurer une interopérabilité totale entre les différentes générations de fichiers de cas tests et entre les dépôts C++17 et Fortran 2018, `Config::load_from_file` implémente une résolution d'alias intelligente :

— Pour le solveur linéaire : reconnaît aussi bien `"tol"` que `"tolerance"`, et `"r_al"` que `"augmented_lagrangian_r"`.
— Pour le module VOF : reconnaît la section dédiée `"vof"` (`reconstruction_method`, `advection_method`, `tvd_limiter`, `cfl_vof`) ainsi que les clés historiques éventuellement placées dans `"solver"` ou `"physics"`.
— Pour les obstacles immergés de Brinkman : parse dynamiquement le tableau JSON `"obstacles"` en distinguant les primitives géométriques rectangulaires (`type = "box"`, définies par $[x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}]$) et circulaires (`type = "circle"`, définies par le centre (x_c, y_c) et le rayon R).

2.3 Répertoire des Classes, Structures et Méthodes de `include/covof/Config.`

Ligne	Catégorie	Identifiant C++17	Signature / Rôle
12	class	`Config`	Déclaration de type / classe / structure
92	Méthode / Fonction	`load_from_json`	Type retour : bool
97	Méthode / Fonction	`print`	Type retour : void

2.4 Code Source Intégral Annoté : `include/covof/Config.hpp`

Le listing ci-dessous présente l'intégralité des 100 lignes du fichier `include/covof/Config.hpp` de l'implémentation C++17 :

```cpp
#pragma once

#include "Types.hpp"
#include <string>

namespace covof {
namespace fugu = covof; // Alias rétrocompatibilité

/**
 * @brief Structure complète regroupant tous les paramètres de simulation
 */
class Config {
public:
    // Paramètres de maillage
    int nx = 64;                ///< Nombre de mailles globales en X
    // Déclaration de variable typée
    int ny = 64;                ///< Nombre de mailles globales en Y
    // Déclaration de variable typée
    double lx = 1.0;            ///< Longueur physique du domaine en X [m]
    // Déclaration de variable typée
    double ly = 1.0;            ///< Hauteur physique du domaine en Y [m]

    // Équations résolues et modèles physiques
    std::string mode = "NAVIER_STOKES";  ///< "NAVIER_STOKES", "LAPLACIAN", "VORTEX_ADVECTION"
    // Déclaration de variable typée
    bool enable_two_phase = false;       ///< Modèle diphasique (true) ou monophasique (false)
    // Déclaration de variable typée
    bool enable_surface_tension = false; ///< Activer la force capillaire CSF
    // Déclaration de variable typée
    bool enable_convection = true;       ///< Activer le transport convectif (Navier-Stokes vs Stokes)
    // Déclaration de variable typée
    bool enable_energy = false;          ///< Activer l'équation de l'énergie (température T)
    // Déclaration de variable typée
    double steady_state_tol = 0.0;       ///< Tolérance d'arrêt sur l'état stationnaire ||u^{n+1}-u^n||

    // Convection thermique Boussinesq (Cas 6.1)
    double Ra = 1.0e5;                   ///< Nombre de Rayleigh
    // Déclaration de variable typée
    double Pr = 0.71;                    ///< Nombre de Prandtl
    // Déclaration de variable typée
    double T_ref = 0.0;                  ///< Température de référence Boussinesq
    // Déclaration de variable typée
    double beta = 3.33e-3;               ///< Coefficient de dilatation volumique [1/K]

    // Modèle de viscosité spatiale (Cas 3.9)
    std::string viscosity_model = "CONSTANT"; ///< "CONSTANT", "EXPONENTIAL"
    // Déclaration de variable typée
    double visc_a = 0.02;
    // Déclaration de variable typée
    double visc_e = 0.01;
    // Déclaration de variable typée
    double visc_mu0 = 1.85e-5;

    // Obstacles immergés (Pénalisation de Brinkman)
    std::vector<Obstacle> obstacles;

    // Propriétés physiques
    FluidProperties fluid1;  ///< Fluide 1 (ex: liquide ou fluide lourd)
    FluidProperties fluid2;  ///< Fluide 2 (ex: gaz ou fluide léger)
    // Déclaration de variable typée
    double sigma = 0.0;      ///< Tension de surface [N/m]
    // Déclaration de variable typée
    double gx = 0.0;         ///< Gravité selon X [m/s^2]
    // Déclaration de variable typée
    double gy = 0.0;         ///< Gravité selon Y [m/s^2]

    // Intégration temporelle
    double dt = 0.001;       ///< Pas de temps initial [s]
    // Déclaration de variable typée
    double t_final = 1.0;    ///< Temps physique final [s]
    // Déclaration de variable typée
    double cfl_max = 0.4;    ///< Nombre de CFL maximum pour le pas de temps adaptatif
    // Déclaration de variable typée
    bool adaptive_dt = false;///< Activer le pas de temps automatique selon CFL
```

```
75
76      // Conditions aux limites et conditions initiales
77      DomainBC bc;
78      InitialConditionConfig init_cond;
79
80      // Solveur linéaire et sorties
81      SolverOptions solver;
82      OutputConfig output;
83
84      // Capture d'interface et transport VOF
85      VofConfig vof;
86
87      /**
88       * @brief Charge la configuration depuis un fichier JSON
89       * @param filepath Chemin vers le fichier .json du cas test
90       * @return true si le chargement a réussi
91       */
92      bool load_from_json(const std::string& filepath);
93
94      /**
95       * @brief Affiche la configuration synthétique dans la console
96       */
97      void print() const;
98  };
99
100 } // namespace covof
```

2.5 Répertoire des Classes, Structures et Méthodes de `src/Config.cpp`

Ligne	Catégorie	Identifiant C++17	Signature / Rôle
12	Méthode / Fonction	`trim`	Type retour : static std : :string
23	Méthode / Fonction	`extract_value`	Type retour : static std : :string
78	Méthode / Fonction	`get_double`	Type retour : static double
86	Méthode / Fonction	`get_int`	Type retour : static int
94	Méthode / Fonction	`get_bool`	Type retour : static bool
105	Méthode / Fonction	`get_string`	Type retour : static std : :string
112	Méthode / Fonction	`parse_bc_type`	Type retour : static BCType
125	Méthode / Fonction	`parse_single_bc`	Type retour : static void
149	Méthode / Fonction	`Config::load_from_json`	Type retour : bool
150	Méthode / Fonction	`file`	Type retour : std : :ifstream
447	Méthode / Fonction	`Config::print`	Type retour : void

2.6 Code Source Intégral Annoté : `src/Config.cpp`

Le listing ci-dessous présente l'intégralité des 503 lignes du fichier `src/Config.cpp` de l'implémentation C++17 :

```
1   #include "covof/Config.hpp"
2   #include <fstream>
3   #include <sstream>
4   #include <iostream>
5   #include <algorithm>
6   #include <cctype>
7
8   namespace covof {
9   namespace fugu = covof; // Alias rétrocompatibilité
10
11  // Fonctions utilitaires locales pour parser le JSON simplement
12  static std::string trim(const std::string& str) {
13      // Déclaration de variable typée
14      size_t first = str.find_first_not_of(" \t\n\r");
15      // Évaluation de la condition de contrôle
16      if (first == std::string::npos) return "";
17      // Déclaration de variable typée
18      size_t last = str.find_last_not_of(" \t\n\r");
19      // Retour de fonction
20      return str.substr(first, (last - first + 1));
21  }
22
23  static std::string extract_value(const std::string& content, const std::string& key) {
24      // Déclaration de variable typée
25      std::string search = "\"" + key + "\"";
26      // Déclaration de variable typée
27      size_t pos = content.find(search);
28      // Évaluation de la condition de contrôle
29      if (pos == std::string::npos) return "";
30      // Déclaration de variable typée
31      size_t colon = content.find(':', pos + search.length());
32      // Évaluation de la condition de contrôle
33      if (colon == std::string::npos) return "";
34
35      // Déclaration de variable typée
```

```cpp
    size_t start = content.find_first_not_of(" \t\n\r", colon + 1);
    // Évaluation de la condition de contrôle
    if (start == std::string::npos) return "";

    // Évaluation de la condition de contrôle
    if (content[start] == '{' || content[start] == '[') {
        // Calcul et assignation numérique
        char open_c = content[start];
        // Calcul et assignation numérique
        char close_c = (open_c == '{') ? '}' : ']';
        // Déclaration de variable typée
        int depth = 1;
        // Déclaration de variable typée
        size_t end = start + 1;
        // Boucle itérative conditionnelle
        while (end < content.length() && depth > 0) {
            // Évaluation de la condition de contrôle
            if (content[end] == open_c) depth++;
            // Branche alternative conditionnelle
            else if (content[end] == close_c) depth--;
            end++;
        }
        // Retour de fonction
        return content.substr(start, end - start);
    // Calcul et assignation numérique
    } else if (content[start] == '"') {
        // Déclaration de variable typée
        size_t end = content.find('"', start + 1);
        // Évaluation de la condition de contrôle
        if (end == std::string::npos) return "";
        // Retour de fonction
        return content.substr(start + 1, end - start - 1);
    } else {
        // Déclaration de variable typée
        size_t end = content.find_first_of(",}\n\r", start);
        // Évaluation de la condition de contrôle
        if (end == std::string::npos) end = content.length();
        // Retour de fonction
        return trim(content.substr(start, end - start));
    }
}

static double get_double(const std::string& block, const std::string& key, double default_val) {
    // Déclaration de variable typée
    std::string val = extract_value(block, key);
    // Évaluation de la condition de contrôle
    if (val.empty()) return default_val;
    try { return std::stod(val); } catch (...) { return default_val; }
}

static int get_int(const std::string& block, const std::string& key, int default_val) {
    // Déclaration de variable typée
    std::string val = extract_value(block, key);
    // Évaluation de la condition de contrôle
    if (val.empty()) return default_val;
    try { return std::stoi(val); } catch (...) { return default_val; }
}

static bool get_bool(const std::string& block, const std::string& key, bool default_val) {
    // Déclaration de variable typée
    std::string val = extract_value(block, key);
    // Évaluation de la condition de contrôle
    if (val == "true" || val == "1") return true;
    // Évaluation de la condition de contrôle
    if (val == "false" || val == "0") return false;
    // Retour de fonction
    return default_val;
}

static std::string get_string(const std::string& block, const std::string& key, const std::string& default_val
    ) {
    // Déclaration de variable typée
    std::string val = extract_value(block, key);
    // Retour de fonction
    return val.empty() ? default_val : val;
}

static BCType parse_bc_type(const std::string& str) {
    // Évaluation de la condition de contrôle
    if (str == "DIRICHLET") return BCType::DIRICHLET;
    // Évaluation de la condition de contrôle
    if (str == "NEUMANN") return BCType::NEUMANN;
    // Évaluation de la condition de contrôle
    if (str == "FREE_SLIP") return BCType::FREE_SLIP;
    // Évaluation de la condition de contrôle
    if (str == "PERIODIC") return BCType::PERIODIC;
    // Retour de fonction
    return BCType::DIRICHLET;
}

static void parse_single_bc(const std::string& block, BoundaryCondition& bc) {
    // Calcul et assignation numérique
```

```
    bc.type = parse_bc_type(get_string(block, "type", "DIRICHLET"));
    // Calcul et assignation numérique
    bc.u_val = get_double(block, "u", 0.0);
    // Calcul et assignation numérique
    bc.v_val = get_double(block, "v", 0.0);
    // Calcul et assignation numérique
    bc.profile = get_string(block, "profile", "UNIFORM");
    // Calcul et assignation numérique
    bc.u_mean = get_double(block, "u_mean", 0.0);
    // Calcul et assignation numérique
    bc.y_step = get_double(block, "y_step", 0.0);
    // Calcul et assignation de la largeur d'injecteur (Cas 3.3 Fluids 2021)
    bc.jet_width = get_double(block, "jet_width", bc.jet_width);
    // Calcul et assignation de la position centrale de l'injecteur
    bc.jet_center = get_double(block, "jet_center", bc.jet_center);
    // Calcul et assignation numérique
    bc.T_type = parse_bc_type(get_string(block, "T_type", "NEUMANN"));
    // Calcul et assignation numérique
    bc.T_val = get_double(block, "T", 0.0);
}

// Déclaration de variable typée
bool Config::load_from_json(const std::string& filepath) {
    std::ifstream file(filepath);
    // Évaluation de la condition de contrôle
    if (!file.is_open()) {
        std::cerr << "[Config] Erreur: Impossible d'ouvrir le fichier JSON : " << filepath << std::endl;
        // Retour de fonction
        return false;
    }
    std::stringstream buffer;
    buffer << file.rdbuf();
    // Déclaration de variable typée
    std::string json = buffer.str();

    // 0. Mode global et arrêt stationnaire
    mode = get_string(json, "mode", mode);
    // Calcul de la résolution géométrique de cellule
    steady_state_tol = get_double(json, "steady_state_tol", steady_state_tol);

    // 1. Maillage
    std::string mesh_block = extract_value(json, "mesh");
    // Évaluation de la condition de contrôle
    if (!mesh_block.empty()) {
        // Calcul et assignation numérique
        nx = get_int(mesh_block, "nx", nx);
        // Calcul et assignation numérique
        ny = get_int(mesh_block, "ny", ny);
        // Calcul et assignation numérique
        lx = get_double(mesh_block, "lx", lx);
        // Calcul et assignation numérique
        ly = get_double(mesh_block, "ly", ly);
    }

    // 2. Modèle physique
    std::string model_block = extract_value(json, "physics");
    // Évaluation de la condition de contrôle
    if (!model_block.empty()) {
        // Calcul et assignation numérique
        enable_two_phase = get_bool(model_block, "enable_two_phase", enable_two_phase);
        // Calcul et assignation numérique
        enable_surface_tension = get_bool(model_block, "enable_surface_tension", enable_surface_tension);
        // Calcul et assignation numérique
        enable_convection = get_bool(model_block, "enable_convection", enable_convection);
        // Calcul et assignation numérique
        enable_energy = get_bool(model_block, "enable_energy", enable_energy);
        // Calcul et assignation numérique
        sigma = get_double(model_block, "sigma", sigma);
        // Calcul et assignation numérique
        gx = get_double(model_block, "gx", gx);
        // Calcul et assignation numérique
        gy = get_double(model_block, "gy", gy);

        // Calcul et assignation numérique
        Ra = get_double(model_block, "Ra", Ra);
        // Calcul et assignation numérique
        Pr = get_double(model_block, "Pr", Pr);
        // Calcul et assignation numérique
        T_ref = get_double(model_block, "T_ref", T_ref);
        // Calcul et assignation numérique
        beta = get_double(model_block, "beta", beta);

        // Calcul et assignation numérique
        viscosity_model = get_string(model_block, "viscosity_model", viscosity_model);
        // Calcul et assignation numérique
        visc_a = get_double(model_block, "visc_a", visc_a);
        // Calcul et assignation numérique
        visc_e = get_double(model_block, "visc_e", visc_e);
        // Calcul et assignation numérique
        visc_mu0 = get_double(model_block, "visc_mu0", visc_mu0);

        // Déclaration de variable typée
```

```
        std::string f1_block = extract_value(model_block, "fluid1");
        // Évaluation de la condition de contrôle
        if (!f1_block.empty()) {
            // Calcul et assignation numérique
            fluid1.rho = get_double(f1_block, "rho", fluid1.rho);
            // Calcul et assignation numérique
            fluid1.mu  = get_double(f1_block, "mu", fluid1.mu);
            // Calcul et assignation numérique
            fluid1.Cp  = get_double(f1_block, "Cp", fluid1.Cp);
            // Calcul et assignation numérique
            fluid1.k_thermal = get_double(f1_block, "k_thermal", fluid1.k_thermal);
            // Calcul et assignation numérique
            fluid1.beta = get_double(f1_block, "beta", fluid1.beta);
        }

        // Déclaration de variable typée
        std::string f2_block = extract_value(model_block, "fluid2");
        // Évaluation de la condition de contrôle
        if (!f2_block.empty()) {
            // Calcul et assignation numérique
            fluid2.rho = get_double(f2_block, "rho", fluid2.rho);
            // Calcul et assignation numérique
            fluid2.mu  = get_double(f2_block, "mu", fluid2.mu);
            // Calcul et assignation numérique
            fluid2.Cp  = get_double(f2_block, "Cp", fluid2.Cp);
            // Calcul et assignation numérique
            fluid2.k_thermal = get_double(f2_block, "k_thermal", fluid2.k_thermal);
            // Calcul et assignation numérique
            fluid2.beta = get_double(f2_block, "beta", fluid2.beta);
        }
    }

    // 3. Intégration temporelle
    std::string time_block = extract_value(json, "time");
    // Évaluation de la condition de contrôle
    if (!time_block.empty()) {
        // Calcul et assignation numérique
        dt = get_double(time_block, "dt", dt);
        // Calcul et assignation numérique
        t_final = get_double(time_block, "t_final", t_final);
        // Calcul et assignation numérique
        cfl_max = get_double(time_block, "cfl_max", cfl_max);
        // Calcul et assignation numérique
        adaptive_dt = get_bool(time_block, "adaptive_dt", adaptive_dt);
    }

    // 4. Conditions aux limites
    std::string bc_block = extract_value(json, "boundary_conditions");
    // Évaluation de la condition de contrôle
    if (!bc_block.empty()) {
        // Déclaration de variable typée
        std::string b_left = extract_value(bc_block, "left");
        // Évaluation de la condition de contrôle
        if (!b_left.empty()) parse_single_bc(b_left, bc.left);

        // Déclaration de variable typée
        std::string b_right = extract_value(bc_block, "right");
        // Évaluation de la condition de contrôle
        if (!b_right.empty()) parse_single_bc(b_right, bc.right);

        // Déclaration de variable typée
        std::string b_bot = extract_value(bc_block, "bottom");
        // Évaluation de la condition de contrôle
        if (!b_bot.empty()) parse_single_bc(b_bot, bc.bottom);

        // Déclaration de variable typée
        std::string b_top = extract_value(bc_block, "top");
        // Évaluation de la condition de contrôle
        if (!b_top.empty()) parse_single_bc(b_top, bc.top);
    }

    // 5. Condition initiale
    std::string ic_block = extract_value(json, "initial_condition");
    // Évaluation de la condition de contrôle
    if (!ic_block.empty()) {
        // Déclaration de variable typée
        std::string type_str = get_string(ic_block, "type", "SINGLE_PHASE");
        // Évaluation de la condition de contrôle
        if (type_str == "DROPLET") init_cond.type = InitialConditionType::DROPLET;
        // Branche alternative conditionnelle pour bulle ou cylindre immergé
        else if (type_str == "BUBBLE" || type_str == "CYLINDER") init_cond.type = InitialConditionType::BUBBLE
            ;
        // Branche alternative conditionnelle
        else if (type_str == "DAM_BREAK") init_cond.type = InitialConditionType::DAM_BREAK;
        // Branche alternative conditionnelle pour nappe liquide (Cas 3.2 Fluids 2021)
        else if (type_str == "LIQUID_SHEET") init_cond.type = InitialConditionType::LIQUID_SHEET;
        // Branche alternative conditionnelle pour jet liquide visqueux (Cas 3.3 Fluids 2021)
        else if (type_str == "VISCOUS_JET" || type_str == "JET_BUCKLING") init_cond.type =
            InitialConditionType::VISCOUS_JET;
        // Branche alternative conditionnelle
        else if (type_str == "SHEAR_LAYER") init_cond.type = InitialConditionType::SHEAR_LAYER;
        // Branche alternative conditionnelle
```

```
        else if (type_str == "VORTEX_SHEAR") init_cond.type = InitialConditionType::VORTEX_SHEAR;
        // Branche alternative conditionnelle
        else if (type_str == "RAYLEIGH_TAYLOR") init_cond.type = InitialConditionType::RAYLEIGH_TAYLOR;
        // Branche par défaut
        else init_cond.type = InitialConditionType::SINGLE_PHASE;

        // Calcul et assignation numérique
        init_cond.center_x = get_double(ic_block, "center_x", init_cond.center_x);
        // Calcul et assignation numérique
        init_cond.center_y = get_double(ic_block, "center_y", init_cond.center_y);
        // Calcul et assignation numérique
        init_cond.radius   = get_double(ic_block, "radius", init_cond.radius);
        // Calcul et assignation numérique
        init_cond.box_x0   = get_double(ic_block, "box_x0", init_cond.box_x0);
        // Calcul et assignation numérique
        init_cond.box_y0   = get_double(ic_block, "box_y0", init_cond.box_y0);
        // Calcul et assignation numérique
        init_cond.box_x1   = get_double(ic_block, "box_x1", init_cond.box_x1);
        // Calcul et assignation numérique
        init_cond.box_y1   = get_double(ic_block, "box_y1", init_cond.box_y1);
        // Calcul et assignation numérique
        init_cond.pert_amp = get_double(ic_block, "pert_amp", init_cond.pert_amp);
        // Calcul et assignation numérique
        init_cond.pert_k   = get_double(ic_block, "pert_k", init_cond.pert_k);
        // Calcul et assignation de l'épaisseur de nappe liquide
        init_cond.thickness = get_double(ic_block, "thickness", init_cond.thickness);
        // Calcul et assignation de la largeur du jet liquide
        init_cond.jet_width = get_double(ic_block, "jet_width", init_cond.jet_width);
    }

    // 6. Solveur linéaire
    std::string sol_block = extract_value(json, "solver");
    // Évaluation de la condition de contrôle
    if (!sol_block.empty()) {
        // Calcul et assignation numérique
        solver.type = get_string(sol_block, "type", solver.type);
        // Calcul et assignation numérique
        solver.tol = get_double(sol_block, "tol", solver.tol);
        // Calcul et assignation numérique
        solver.max_iter = get_int(sol_block, "max_iter", solver.max_iter);
        // Calcul et assignation numérique
        solver.r_al = get_double(sol_block, "r_al", solver.r_al);
        // Calcul et assignation numérique
        solver.uzawa_iter = get_int(sol_block, "uzawa_iter", solver.uzawa_iter);
        // Calcul et assignation numérique
        solver.use_precond = get_bool(sol_block, "use_precond", solver.use_precond);
        // Calcul et assignation numérique
        solver.precond_type = get_string(sol_block, "precond_type", solver.precond_type);
    }

    // 7. Sorties et visualisation
    std::string out_block = extract_value(json, "output");
    // Évaluation de la condition de contrôle
    if (!out_block.empty()) {
        // Calcul et assignation numérique
        output.output_dir = get_string(out_block, "output_dir", output.output_dir);
        // Calcul et assignation numérique
        output.case_name  = get_string(out_block, "case_name", output.case_name);
        // Calcul et assignation numérique
        output.vtk_interval = get_int(out_block, "vtk_interval", output.vtk_interval);
        // Calcul et assignation numérique
        output.log_interval = get_int(out_block, "log_interval", output.log_interval);
        // Calcul et assignation numérique
        output.write_csv_history = get_bool(out_block, "write_csv_history", output.write_csv_history);
    }

    // 8. Capture d'interface et transport VOF
    std::string vof_block = extract_value(json, "vof");
    // Évaluation de la condition de contrôle
    if (!vof_block.empty()) {
        // Calcul et assignation numérique
        vof.advection_method      = get_string(vof_block, "advection_method", vof.advection_method);
        // Calcul et assignation numérique
        vof.reconstruction_method = get_string(vof_block, "reconstruction_method", vof.reconstruction_method);
        // Calcul et assignation numérique
        vof.tvd_limiter           = get_string(vof_block, "tvd_limiter", vof.tvd_limiter);
        // Calcul et assignation numérique
        vof.cfl_vof               = get_double(vof_block, "cfl_vof", vof.cfl_vof);
    }
    // Cohérence stricte : le schéma algébrique TVD ne s'accompagne d'aucune reconstruction géométrique,
    // tandis que les schémas géométriques (WEYMOUTH_YUE, CIAM) n'utilisent pas de limiteur TVD.
    if (vof.advection_method == "VOF_TVD" || vof.advection_method == "TVD") {
        vof.reconstruction_method = "NONE";
        if (vof.tvd_limiter == "NONE" || vof.tvd_limiter.empty()) {
            vof.tvd_limiter = "SUPERBEE";
        }
    } else {
        if (vof.reconstruction_method == "NONE" || vof.reconstruction_method.empty()) {
            vof.reconstruction_method = "PLIC";
        }
        vof.tvd_limiter = "NONE";
    }
```

```
401
402     // 9. Obstacles immergés (Pénalisation de Brinkman)
403     obstacles.clear();
404     // Déclaration de variable typée
405     std::string obs_array = extract_value(json, "obstacles");
406     // Évaluation de la condition de contrôle
407     if (!obs_array.empty()) {
408         // Déclaration de variable typée
409         size_t pos = 0;
410         // Boucle itérative conditionnelle
411         while ((pos = obs_array.find('{', pos)) != std::string::npos) {
412             // Déclaration de variable typée
413             size_t end_pos = obs_array.find('}', pos);
414             // Évaluation de la condition de contrôle
415             if (end_pos == std::string::npos) break;
416             // Déclaration de variable typée
417             std::string obs_block = obs_array.substr(pos, end_pos - pos + 1);
418             Obstacle obs;
419             // Calcul et assignation numérique
420             obs.type = get_string(obs_block, "type", "BOX");
421             // Calcul et assignation numérique
422             obs.x0 = get_double(obs_block, "x0", 0.0);
423             // Calcul et assignation numérique
424             obs.y0 = get_double(obs_block, "y0", 0.0);
425             // Calcul et assignation numérique
426             obs.x1 = get_double(obs_block, "x1", 0.0);
427             // Calcul et assignation numérique
428             obs.y1 = get_double(obs_block, "y1", 0.0);
429             // Calcul et assignation numérique
430             obs.xc = get_double(obs_block, "xc", 0.0);
431             // Calcul et assignation numérique
432             obs.yc = get_double(obs_block, "yc", 0.0);
433             // Calcul et assignation numérique
434             obs.radius = get_double(obs_block, "radius", 0.0);
435             // Calcul et assignation numérique
436             obs.permeability = get_double(obs_block, "permeability", 1.0e-14);
437             obstacles.push_back(obs);
438             // Calcul et assignation numérique
439             pos = end_pos + 1;
440         }
441     }
442
443     // Retour de fonction
444     return true;
445 }
446
447 void Config::print() const {
448     // Calcul et assignation numérique
449     std::cout << "============================================================\n"
450               << "       COVOF 2D PARALLELE - CONFIGURATION DU CAS          \n"
451               // Calcul et assignation numérique
452               << "============================================================\n"
453               << " Mode calcul : " << mode << "\n"
454               // Calcul et assignation numérique
455               << " Maillage : " << nx << " x " << ny << " cellules (Lx = " << lx << " m, Ly = " << ly << " m)\
                      n"
456               // Calcul de la résolution géométrique de cellule
457               << " dx = " << lx / nx << " m, dy = " << ly / ny << " m\n"
458               << " Physique : " << (enable_two_phase ? "Diphasique" : "Monophasique")
459               << " | Convection : " << (enable_convection ? "Oui" : "Non")
460               << " | Tension de surface : " << (enable_surface_tension ? "Oui" : "Non") << "\n";
461     // Évaluation de la condition de contrôle
462     if (!obstacles.empty()) {
463         std::cout << " Obstacles immerges : " << obstacles.size() << " defini(s) [Brinkman]\n";
464         // Boucle d'itération algorithmique
465         for (size_t i = 0; i < obstacles.size(); ++i) {
466             std::cout << "   - Obstacle " << i+1 << " : [" << obstacles[i].x0 << ", " << obstacles[i].x1
467                       // Calcul et assignation numérique
468                       << "] x [" << obstacles[i].y0 << ", " << obstacles[i].y1 << "], K=" << obstacles[i].
                              permeability << "\n";
469         }
470     }
471     // Calcul et assignation numérique
472     std::cout << " Fluide 1 : rho=" << fluid1.rho << " kg/m3, mu=" << fluid1.mu << " Pa.s\n";
473     // Évaluation de la condition de contrôle
474     if (enable_two_phase) {
475         // Calcul et assignation numérique
476         std::cout << " Fluide 2 : rho=" << fluid2.rho << " kg/m3, mu=" << fluid2.mu << " Pa.s\n"
477                   // Calcul et assignation numérique
478                   << " Tension superficielle sigma = " << sigma << " N/m\n";
479         if (vof.advection_method == "VOF_TVD" || vof.advection_method == "TVD") {
480             std::cout << " VOF : Advection=" << vof.advection_method
481                       << " (Algebrique, sans reconstruction) | Limiteur=" << vof.tvd_limiter << "\n";
482         } else {
483             std::cout << " VOF : Advection=" << vof.advection_method
484                       << " | Reconstruction=" << vof.reconstruction_method << "\n";
485         }
486     }
487     // Calcul et assignation numérique
488     std::cout << " Gravite : g = (" << gx << ", " << gy << ") m/s^2\n"
489               // Calcul et assignation numérique
490               << " Temps : dt = " << dt << " s, t_final = " << t_final << " s\n"
```

```
491                 // Calcul et assignation numérique
492                 << " Solveur : " << solver.type << " | BiCGSTAB(2), tol=" << solver.tol
493                 // Calcul et assignation numérique
494                 << ", max_iter=" << solver.max_iter << ", r_al=" << solver.r_al
495                 // Calcul et assignation numérique
496                 << ", uzawa_iter=" << solver.uzawa_iter << "\n"
497                 // Calcul et assignation numérique
498                 << " Sorties : Dossier '" << output.output_dir << "', intervalle VTK=" << output.vtk_interval <<
                        "\n"
499                 // Calcul et assignation numérique
500                 << "============================================================\n";
501 }
502
503 } // namespace covof
```

Chapitre 3

Topologie du Maillage Cartésien MAC 2D et Décomposition Spatiale (Mesh.hpp, Mesh.cpp)

3.1 Géométrie de la Grille Décalée MAC (*Marker-And-Cell*)

La classe `covof::Mesh` (`include/covof/Mesh.hpp`, `src/Mesh.cpp`) encapsule la géométrie discrète du sous-domaine attribué au processus MPI courant. Soit un domaine global $[0, L_x] \times [0, L_y]$ discrétisé par $N_{x,\text{glob}} \times N_{y,\text{glob}}$ cellules uniformes de pas :

$$\Delta x = \frac{L_x}{N_{x,\text{glob}}}, \qquad \Delta y = \frac{L_y}{N_{y,\text{glob}}} \tag{3.1}$$

Lorsque le domaine est partitionné sur une grille cartésienne de $P_x \times P_y$ processus MPI, le processus de coordonnées cartésiennes $(p_x, p_y) \in \{0, \ldots, P_x - 1\} \times \{0, \ldots, P_y - 1\}$ reçoit un sous-domaine local de $N_x \times N_y$ cellules débutant aux indices globaux $(i_{\text{start}}, j_{\text{start}})$:

$$N_x = \left\lfloor \frac{N_{x,\text{glob}}}{P_x} \right\rfloor + \mathbb{I}(p_x < N_{x,\text{glob}} \bmod P_x) \tag{3.2}$$

Ce partitionnement à reste distribué garantit un déséquilibre de charge maximal d'au plus une colonne ou une ligne de mailles entre deux processus quelconques.

3.2 Systèmes de Coordonnées Décalées (x_c, y_c), (x_u, y_c) et (x_c, y_v)

En indiçage local C++17 à base zéro ($i \in \{0, \ldots, N_x - 1\}$, $j \in \{0, \ldots, N_y - 1\}$), les trois familles de points de collocation de la grille MAC sont évaluées par les méthodes en ligne (`inline`) de `Mesh` :

1. **Centre de la maille** (i, j) (support de $p, T, C, \rho, \mu, \nabla \cdot \mathbf{u}$) :

$$x_c(i) = x_{\min,\text{loc}} + \left(i + \frac{1}{2}\right) \Delta x, \qquad y_c(j) = y_{\min,\text{loc}} + \left(j + \frac{1}{2}\right) \Delta y \tag{3.3}$$

2. **Face verticale** (i, j) (support de la vitesse horizontale $u_{i,j}$ située sur la face Ouest de la maille (i, j), avec $i \in \{0, \ldots, N_x\}$) :

$$x_u(i) = x_{\min,\text{loc}} + i\,\Delta x, \qquad y_u(j) = y_c(j) \tag{3.4}$$

3. **Face horizontale** (i, j) (support de la vitesse verticale $v_{i,j}$ située sur la face Sud de la maille (i, j), avec $j \in \{0, \ldots, N_y\}$) :

$$x_v(i) = x_c(i), \qquad y_v(j) = y_{\min,\text{loc}} + j\,\Delta y \tag{3.5}$$

3.3 Répertoire des Classes, Structures et Méthodes de `include/covof/Mesh.hp`

Ligne	Catégorie	Identifiant C++17	Signature / Rôle
12	class	`Mesh2D`	Déclaration de type / classe / structure
80	Méthode / Fonction	`cell_idx`	Type retour : inline int
85	Méthode / Fonction	`u_idx`	Type retour : inline int
90	Méthode / Fonction	`v_idx`	Type retour : inline int
95	Méthode / Fonction	`node_idx`	Type retour : inline int

3.4 Code Source Intégral Annoté : `include/covof/Mesh.hpp`

Le listing ci-dessous présente l'intégralité des 109 lignes du fichier `include/covof/Mesh.hpp` de l'implémentation C++17 :

```cpp
#pragma once

#include <mpi.h>
#include <vector>

namespace covof {
namespace fugu = covof; // Alias rétrocompatibilité

/**
 * @brief Maillage 2D cartésien décalé (MAC) partitionné en sous-domaines MPI
 */
class Mesh2D {
public:
    // Paramètres globaux
    int nx_global = 0;
    // Déclaration de variable typée
    int ny_global = 0;
    // Déclaration de variable typée
    double lx = 1.0;
    // Déclaration de variable typée
    double ly = 1.0;
    // Déclaration de variable typée
    double dx = 0.0;
    // Déclaration de variable typée
    double dy = 0.0;

    // Décomposition de domaine MPI
    MPI_Comm comm = MPI_COMM_NULL;
    // Déclaration de variable typée
    int rank = 0;
    // Déclaration de variable typée
    int num_procs = 1;
    // Déclaration de variable typée
    int dims[2] = {1, 1};       ///< Nombre de processeurs {px, py}
    // Déclaration de variable typée
    int coords[2] = {0, 0};     ///< Coordonnées du processus dans la topologie cartésienne

    // Voisins MPI
    int rank_left   = MPI_PROC_NULL;
    // Déclaration de variable typée
    int rank_right  = MPI_PROC_NULL;
    // Déclaration de variable typée
    int rank_bottom = MPI_PROC_NULL;
    // Déclaration de variable typée
    int rank_top    = MPI_PROC_NULL;

    // Indicateurs de frontière physique globale
    bool is_boundary_left   = false;
    // Déclaration de variable typée
    bool is_boundary_right  = false;
    // Déclaration de variable typée
    bool is_boundary_bottom = false;
    // Déclaration de variable typée
    bool is_boundary_top    = false;

    // Dimensions locales (sans mailles fantômes)
    int nx_local = 0;
    // Déclaration de variable typée
    int ny_local = 0;
    // Déclaration de variable typée
    int i_offset = 0; ///< Décalage d'indice global selon X
    // Déclaration de variable typée
    int j_offset = 0; ///< Décalage d'indice global selon Y

    // Dimensions allouées avec mailles fantômes (1 couche de chaque côté)
    int nx_ghost = 0; ///< nx_local + 2
    // Déclaration de variable typée
    int ny_ghost = 0; ///< ny_local + 2

    /**
     * @brief Initialise le maillage et la topologie cartésienne MPI
     */
    void init(MPI_Comm parent_comm, int nx_g, int ny_g, double length_x, double length_y,
              // Déclaration de variable typée
              bool periodic_x = false, bool periodic_y = false);

    /**
     * @brief Fonctions d'indexation 1D locale avec mailles fantômes
     */
    inline int cell_idx(int i, int j) const {
        // Retour de fonction
        return j * nx_ghost + i;
    }

    inline int u_idx(int i, int j) const {
        // Retour de fonction
```

```
        return j * (nx_ghost + 1) + i;
    }

    inline int v_idx(int i, int j) const {
        // Retour de fonction
        return j * nx_ghost + i;
    }

    inline int node_idx(int i, int j) const {
        // Retour de fonction
        return j * (nx_ghost + 1) + i;
    }

    // Coordonnées physiques locales
    inline double xc(int i) const { return (i_offset + i - 0.5) * dx; }
    inline double yc(int j) const { return (j_offset + j - 0.5) * dy; }
    inline double xu(int i) const { return (i_offset + i - 1.0) * dx; }
    inline double xv(int i) const { return xu(i); } ///< Coordonnée X de la face verticale / sommet
    inline double yv(int j) const { return (j_offset + j - 1.0) * dy; }
    inline double yu(int j) const { return yv(j); } ///< Coordonnée Y de la face horizontale / sommet
};

} // namespace covof
```

3.5 Code Source Intégral Annoté : `src/Mesh.cpp`

Le listing ci-dessous présente l'intégralité des 94 lignes du fichier `src/Mesh.cpp` de l'implémentation C++17 :

```
#include "covof/Mesh.hpp"
#include <iostream>
#include <cassert>

namespace covof {
namespace fugu = covof; // Alias rétrocompatibilité

void Mesh2D::init(MPI_Comm parent_comm, int nx_g, int ny_g, double length_x, double length_y,
                  // Déclaration de variable typée
                  bool periodic_x, bool periodic_y) {
    // Calcul et assignation numérique
    nx_global = nx_g;
    // Calcul et assignation numérique
    ny_global = ny_g;
    // Calcul et assignation numérique
    lx = length_x;
    // Calcul et assignation numérique
    ly = length_y;
    // Calcul de la résolution géométrique de cellule
    dx = lx / nx_global;
    // Calcul de la résolution géométrique de cellule
    dy = ly / ny_global;

    // Déclaration de variable typée
    int parent_size, parent_rank;
    // Appel de communication ou synchronisation parallèle OpenMPI
    MPI_Comm_size(parent_comm, &parent_size);
    // Appel de communication ou synchronisation parallèle OpenMPI
    MPI_Comm_rank(parent_comm, &parent_rank);

    // 1. Détermination de la topologie cartésienne 2D optimale
    dims[0] = 0;
    // Calcul et assignation numérique
    dims[1] = 0;
    // Appel de communication ou synchronisation parallèle OpenMPI
    MPI_Dims_create(parent_size, 2, dims);

    // Déclaration de variable typée
    int periods[2] = {periodic_x ? 1 : 0, periodic_y ? 1 : 0};
    // Déclaration de variable typée
    int reorder = 1;
    // Appel de communication ou synchronisation parallèle OpenMPI
    MPI_Cart_create(parent_comm, 2, dims, periods, reorder, &comm);
    // Appel de communication ou synchronisation parallèle OpenMPI
    MPI_Comm_rank(comm, &rank);
    // Appel de communication ou synchronisation parallèle OpenMPI
    MPI_Comm_size(comm, &num_procs);
    // Appel de communication ou synchronisation parallèle OpenMPI
    MPI_Cart_coords(comm, rank, 2, coords);

    // 2. Découverte des voisins MPI
    MPI_Cart_shift(comm, 0, 1, &rank_left, &rank_right);
    // Appel de communication ou synchronisation parallèle OpenMPI
    MPI_Cart_shift(comm, 1, 1, &rank_bottom, &rank_top);

    // Calcul et assignation numérique
    is_boundary_left   = (!periodic_x && (coords[0] == 0));
    // Calcul et assignation numérique
    is_boundary_right  = (!periodic_x && (coords[0] == dims[0] - 1));
    // Calcul et assignation numérique
```

```
    is_boundary_bottom = (!periodic_y && (coords[1] == 0));
    // Calcul et assignation numérique
    is_boundary_top    = (!periodic_y && (coords[1] == dims[1] - 1));

    // 3. Répartition équitable des mailles locales
    auto partition_dim = [](int n_global, int n_procs, int coord, int& n_local, int& offset) {
        // Déclaration de variable typée
        int base = n_global / n_procs;
        // Déclaration de variable typée
        int rem  = n_global % n_procs;
        // Évaluation de la condition de contrôle
        if (coord < rem) {
            // Calcul et assignation numérique
            n_local = base + 1;
            // Calcul et assignation numérique
            offset  = coord * (base + 1);
        } else {
            // Calcul et assignation numérique
            n_local = base;
            // Calcul et assignation numérique
            offset  = rem * (base + 1) + (coord - rem) * base;
        }
    };

    partition_dim(nx_global, dims[0], coords[0], nx_local, i_offset);
    partition_dim(ny_global, dims[1], coords[1], ny_local, j_offset);

    // Calcul et assignation numérique
    nx_ghost = nx_local + 2;
    // Calcul et assignation numérique
    ny_ghost = ny_local + 2;
}

} // namespace covof
```

Chapitre 4

Conteneurs de Champs Eulériens 2D et Opérations Vectorielles (Field.hpp)

4.1 Conception du Template `Field2D<T>`

L'en-tête `include/covof/Field.hpp` fournit la classe générique `Field2D<T>` (instanciée principalement pour `T = double` via l'alias `ScalarField`) qui représente tout champ discret défini sur le maillage MAC (aussi bien un champ centré de dimension $N_x \times N_y$ qu'un champ de vitesse décalé u de dimension $(N_x + 1) \times N_y$ ou v de dimension $N_x \times (N_y + 1)$).

4.1.1 Adressage Avec Cellules Fantômes (*Ghost Cells*)

Chaque instance de `Field2D<T>` stocke :
- `nx_`, `ny_` : les dimensions actives du champ à l'intérieur du sous-domaine ;
- `ng_` : l'épaisseur de la couronne de mailles fantômes (typiquement $n_g = 1$ ou $n_g = 2$ pour les stencils VOF 3×3 et TVD) ;
- `stride_` $= N_x + 2n_g$: le pas d'adressage mémoire d'une ligne complète ;
- `data_` : le vecteur contigu de taille $(N_x + 2n_g)(N_y + 2n_g)$ initialisé à zéro.

L'opérateur d'accès `operator()(int i, int j)` accepte des indices relatifs signés $i \in [-n_g, N_x + n_g - 1]$ et $j \in [-n_g, N_y + n_g - 1]$:

$$\texttt{field(i, j)} \equiv \texttt{data_[(j + ng_) * stride_ + (i + ng_)]} \tag{4.1}$$

Ainsi, l'écriture des opérateurs aux différences finies dans `NavierStokesSolver.cpp` et `InterfaceTracker.cpp` conserve une lisibilité mathématique directe (`u(i+1, j) - 2*u(i, j) + u(i-1, j)`) sans aucune arithmétique d'offset dans le code client.

4.2 Répertoire des Classes, Structures et Méthodes de `include/covof/Field.h`

Ligne	Catégorie	Identifiant C++17	Signature / Rôle
13	class	`FlowFields`	Déclaration de type / classe / structure
57	Méthode / Fonction	`allocate`	Type retour : void

4.3 Code Source Intégral Annoté : `include/covof/Field.hpp`

Le listing ci-dessous présente l'intégralité des 89 lignes du fichier `include/covof/Field.hpp` de l'implémentation C++17 :

```
1  #pragma once
2
3  #include "Mesh.hpp"
4  #include <vector>
5
6  namespace covof {
7  namespace fugu = covof; // Alias rétrocompatibilité
8
9  /**
10  * @brief Champs scalaires et vectoriels sur la grille décalée MAC
11  * Tous les champs contiennent une couche de mailles fantômes pour le parallélisme MPI.
12  */
13 class FlowFields {
```

```
public:
    // Champs cinématiques et thermodynamiques principaux
    std::vector<double> u;  ///< Vitesse horizontale sur les faces verticales
    // Déclaration de variable typée
    std::vector<double> v;  ///< Vitesse verticale sur les faces horizontales
    // Déclaration de variable typée
    std::vector<double> p;  ///< Pression scalaire au centre des cellules
    // Déclaration de variable typée
    std::vector<double> C;  ///< Fonction indicatrice de phase au centre des cellules (C in [0, 1])
    // Déclaration de variable typée
    std::vector<double> T;  ///< Température scalaire au centre des cellules [K ou adimensionnelle]

    // Propriétés physiques locales effectives (Lois de mélange)
    std::vector<double> rho;     ///< Masse volumique au centre des mailles
    // Déclaration de variable typée
    std::vector<double> rho_u;   ///< Masse volumique interpolée sur les faces u
    // Déclaration de variable typée
    std::vector<double> rho_v;   ///< Masse volumique interpolée sur les faces v
    // Déclaration de variable typée
    std::vector<double> mu;      ///< Viscosité dynamique au centre des mailles
    // Déclaration de variable typée
    std::vector<double> mu_node; ///< Viscosité dynamique harmonique aux nœuds (coins de maille)

    // Géométrie de l'interface et tension superficielle CSF
    std::vector<double> kappa;     ///< Courbure de l'interface au centre des cellules
    // Déclaration de variable typée
    std::vector<double> norm_x;    ///< Composante X du vecteur normal
    // Déclaration de variable typée
    std::vector<double> norm_y;    ///< Composante Y du vecteur normal
    // Déclaration de variable typée
    std::vector<double> F_sigma_x; ///< Force capillaire CSF selon X sur les faces u
    // Déclaration de variable typée
    std::vector<double> F_sigma_y; ///< Force capillaire CSF selon Y sur les faces v

    // Diagnostics
    std::vector<double> div_u;     ///< Divergence de la vitesse au centre des mailles

    // Calcul et assignation numérique
    FlowFields() = default;

    /**
     * @brief Alloue la mémoire selon les dimensions locales du maillage avec mailles fantômes
     */
    void allocate(const Mesh2D& mesh) {
        // Déclaration de variable typée
        int n_cells = mesh.nx_ghost * mesh.ny_ghost;
        // Déclaration de variable typée
        int n_u = (mesh.nx_ghost + 1) * mesh.ny_ghost;
        // Déclaration de variable typée
        int n_v = mesh.nx_ghost * (mesh.ny_ghost + 1);
        // Déclaration de variable typée
        int n_nodes = (mesh.nx_ghost + 1) * (mesh.ny_ghost + 1);

        u.assign(n_u, 0.0);
        v.assign(n_v, 0.0);
        p.assign(n_cells, 0.0);
        C.assign(n_cells, 0.0);
        T.assign(n_cells, 0.0);

        rho.assign(n_cells, 1.0);
        rho_u.assign(n_u, 1.0);
        rho_v.assign(n_v, 1.0);
        mu.assign(n_cells, 1.0);
        mu_node.assign(n_nodes, 1.0);

        kappa.assign(n_cells, 0.0);
        norm_x.assign(n_cells, 0.0);
        norm_y.assign(n_cells, 0.0);
        F_sigma_x.assign(n_u, 0.0);
        F_sigma_y.assign(n_v, 0.0);

        div_u.assign(n_cells, 0.0);
    }
};

} // namespace covof
```

Chapitre 5

Parallélisme Distribué OpenMPI et Échanges de Halos Cartésiens (MpiComm.hpp, MpiComm.cpp)

5.1 Topologie Cartésienne 2D et Découverte des Voisins

La classe `covof::MpiComm` (`include/covof/MpiComm.hpp`, `src/MpiComm.cpp`) abstrait toutes les communications inter-processus OpenMPI. Lors de sa construction :

1. La fonction `MPI_Dims_create(size, 2, dims)` décompose automatiquement le nombre total de processus P en une grille bidimensionnelle $P_x \times P_y$ minimisant le rapport périmètre/surface des sous-domaines (et donc le volume de communication de halo par rapport au volume de calcul).
2. `MPI_Cart_create` construit le communicateur topologique 2D en prenant en compte les drapeaux de périodicité $(\text{periodic}_x, \text{periodic}_y)$ issus de la configuration JSON.
3. `MPI_Cart_shift` identifie les rangs des quatre voisins immédiats : `rank_left` (Ouest, x^-), `rank_right` (Est, x^+), `rank_bottom` (Sud, y^-) et `rank_top` (Nord, y^+). Si une frontière externe n'est pas périodique, MPI affecte automatiquement `MPI_PROC_NULL`, ce qui transforme les appels d'envoi/réception sur ce bord en opérations nulles sans branchement conditionnel coûteux.

5.2 Protocole d'Échange de Halos Sans Interblocage (*Deadlock-Free Halo Exchange*)

La méthode `MpiComm::exchange_halo(Field2D<double>& field)` synchronise les couches de cellules fantômes $i \in \{-n_g, \ldots, -1\}$, $i \in \{N_x, \ldots, N_x + n_g - 1\}$, $j \in \{-n_g, \ldots, -1\}$ et $j \in \{N_y, \ldots, N_y + n_g - 1\}$ en deux passes séquentielles (passe horizontale Est↔Ouest suivie de la passe verticale Nord↔Sud) :

— L'enchaînement séquentiel de la passe horizontale puis de la passe verticale propage automatiquement les **cellules fantômes de coin** $(i < 0, j < 0)$, $(i \geq N_x, j \geq N_y)$ sans nécessiter de messages diagonaux dédiés (crucial pour le filtre 3×3 de Parker-Youngs et le terme croisé $-r_{al}\partial^2/\partial x \partial y$).

— Les colonnes verticales n'étant pas contiguës en mémoire *row-major*, elles sont empaquetées (*packing*) dans des tampons linéaires `send_left`, `send_right`, transmises via `MPI_Sendrecv` ou `MPI_Isend`/`MPI_Irecv`, puis dépaquetées (*unpacking*) dans les cellules fantômes.

5.3 Répertoire des Classes, Structures et Méthodes de `include/covof/MpiComm`

Ligne	Catégorie	Identifiant C++17	Signature / Rôle
14	class	`MpiComm`	Déclaration de type / classe / structure
19	Méthode / Fonction	`sync_cell_scalar`	Type retour : static void
24	Méthode / Fonction	`sync_u`	Type retour : static void
29	Méthode / Fonction	`sync_v`	Type retour : static void
35	Méthode / Fonction	`apply_boundary_conditions`	Type retour : static void
40	Méthode / Fonction	`apply_thermal_boundary_conditions`	Type retour : static void
45	Méthode / Fonction	`global_dot_product`	Type retour : static double
50	Méthode / Fonction	`global_norm2`	Type retour : static double
55	Méthode / Fonction	`global_max`	Type retour : static double
60	Méthode / Fonction	`global_sum`	Type retour : static double

5.4 Code Source Intégral Annoté : `include/covof/MpiComm.hpp`

Le listing ci-dessous présente l'intégralité des 63 lignes du fichier `include/covof/MpiComm.hpp` de l'implémentation C++17 :

```cpp
#pragma once

#include "Mesh.hpp"
#include "Field.hpp"
#include "Types.hpp"
#include <vector>

namespace covof {
namespace fugu = covof; // Alias rétrocompatibilité

/**
 * @brief Gestionnaire des communications MPI et des échanges de mailles fantômes (halos)
 */
class MpiComm {
public:
    /**
     * @brief Échange les halos d'un champ scalaire centré (p, C, rho, etc.)
     */
    static void sync_cell_scalar(const Mesh2D& mesh, std::vector<double>& field);

    /**
     * @brief Échange les halos de la vitesse horizontale u (faces verticales)
     */
    static void sync_u(const Mesh2D& mesh, std::vector<double>& u);

    /**
     * @brief Échange les halos de la vitesse verticale v (faces horizontales)
     */
    static void sync_v(const Mesh2D& mesh, std::vector<double>& v);

    /**
     * @brief Applique les conditions aux limites physiques sur les mailles fantômes extérieures
     * @param homogeneous Si true, applique des CL homogènes (pour l'opérateur linéaire A*x)
     */
    static void apply_boundary_conditions(const Mesh2D& mesh, const DomainBC& bc, FlowFields& fields, bool
        homogeneous = false);

    /**
     * @brief Applique les conditions aux limites thermiques sur les mailles fantômes de la température T
     */
    static void apply_thermal_boundary_conditions(const Mesh2D& mesh, const DomainBC& bc, FlowFields& fields);

    /**
     * @brief Calcule le produit scalaire global de deux vecteurs distribués
     */
    static double global_dot_product(const Mesh2D& mesh, const std::vector<double>& a, const std::vector<
        double>& b, int size);

    /**
     * @brief Calcule la norme L2 globale d'un vecteur distribué
     */
    static double global_norm2(const Mesh2D& mesh, const std::vector<double>& v, int size);

    /**
     * @brief Réduit la valeur maximale scalaire sur tous les processus MPI (ex: CFL)
     */
    static double global_max(const Mesh2D& mesh, double local_val);

    /**
     * @brief Réduit la somme globale d'un scalaire sur tous les processus MPI (ex: intégrales)
     */
    static double global_sum(const Mesh2D& mesh, double local_val);
};

} // namespace covof
```

5.5 Répertoire des Classes, Structures et Méthodes de `src/MpiComm.cpp`

Ligne	Catégorie	Identifiant C++17	Signature / Rôle
8	Méthode / Fonction	`MpiComm::sync_cell_scalar`	Type retour : void
32	Méthode / Fonction	`send_left`	Type retour : std : :vector<double>
34	Méthode / Fonction	`send_right`	Type retour : std : :vector<double>
36	Méthode / Fonction	`send_bot`	Type retour : std : :vector<double>
38	Méthode / Fonction	`send_top`	Type retour : std : :vector<double>
122	Méthode / Fonction	`MpiComm::sync_u`	Type retour : void
132	Méthode / Fonction	`send_left`	Type retour : std : :vector<double>
134	Méthode / Fonction	`send_right`	Type retour : std : :vector<double>
136	Méthode / Fonction	`send_bot`	Type retour : std : :vector<double>
138	Méthode / Fonction	`send_top`	Type retour : std : :vector<double>
222	Méthode / Fonction	`MpiComm::sync_v`	Type retour : void
232	Méthode / Fonction	`send_left`	Type retour : std : :vector<double>
234	Méthode / Fonction	`send_right`	Type retour : std : :vector<double>
236	Méthode / Fonction	`send_bot`	Type retour : std : :vector<double>
238	Méthode / Fonction	`send_top`	Type retour : std : :vector<double>
322	Méthode / Fonction	`MpiComm::apply_boundary_conditions`	Type retour : void
597	Méthode / Fonction	`MpiComm::apply_thermal_boundary_conditions`	Type retour : void
686	Méthode / Fonction	`MpiComm::global_dot_product`	Type retour : double
703	Méthode / Fonction	`MpiComm::global_norm2`	Type retour : double
709	Méthode / Fonction	`MpiComm::global_max`	Type retour : double
719	Méthode / Fonction	`MpiComm::global_sum`	Type retour : double

5.6 Code Source Intégral Annoté : `src/MpiComm.cpp`

Le listing ci-dessous présente l'intégralité des 728 lignes du fichier `src/MpiComm.cpp` de l'implémentation C++17 :

```
#include "covof/MpiComm.hpp"
#include <algorithm>
#include <cmath>

namespace covof {
namespace fugu = covof; // Alias rétrocompatibilité

void MpiComm::sync_cell_scalar(const Mesh2D& mesh, std::vector<double>& field) {
    // Déclaration de variable typée
    int nx = mesh.nx_local;
    // Déclaration de variable typée
    int ny = mesh.ny_local;

    // Évaluation de la condition de contrôle
    if (mesh.num_procs == 1) {
        if (!mesh.is_boundary_left) {
            for (int j = 0; j <= ny + 1; ++j) {
                field[mesh.cell_idx(0, j)]      = field[mesh.cell_idx(nx, j)];
                field[mesh.cell_idx(nx + 1, j)] = field[mesh.cell_idx(1, j)];
            }
        }
        if (!mesh.is_boundary_bottom) {
            for (int i = 0; i <= nx + 1; ++i) {
                field[mesh.cell_idx(i, 0)]      = field[mesh.cell_idx(i, ny)];
                field[mesh.cell_idx(i, ny + 1)] = field[mesh.cell_idx(i, 1)];
            }
        }
        return;
    }

    // Déclaration de variable typée
    std::vector<double> send_left(ny), recv_left(ny);
    // Déclaration de variable typée
    std::vector<double> send_right(ny), recv_right(ny);
    // Déclaration de variable typée
    std::vector<double> send_bot(nx), recv_bot(nx);
    // Déclaration de variable typée
    std::vector<double> send_top(nx), recv_top(nx);

    // Préparation des buffers X
    for (int j = 1; j <= ny; ++j) {
        // Calcul et assignation numérique
        send_left[j - 1]  = field[mesh.cell_idx(1, j)];
        // Calcul et assignation numérique
        send_right[j - 1] = field[mesh.cell_idx(nx, j)];
    }

    // Appel de communication ou synchronisation parallèle OpenMPI
    MPI_Request reqs[8];
    // Déclaration de variable typée
    int n_reqs = 0;

```

```
    // Échanges Est-Ouest
    if (mesh.rank_left != MPI_PROC_NULL) {
        // Appel de communication ou synchronisation parallèle OpenMPI
        MPI_Irecv(recv_left.data(), ny, MPI_DOUBLE, mesh.rank_left, 10, mesh.comm, &reqs[n_reqs++]);
        // Appel de communication ou synchronisation parallèle OpenMPI
        MPI_Isend(send_left.data(), ny, MPI_DOUBLE, mesh.rank_left, 11, mesh.comm, &reqs[n_reqs++]);
    }
    // Évaluation de la condition de contrôle
    if (mesh.rank_right != MPI_PROC_NULL) {
        // Appel de communication ou synchronisation parallèle OpenMPI
        MPI_Irecv(recv_right.data(), ny, MPI_DOUBLE, mesh.rank_right, 11, mesh.comm, &reqs[n_reqs++]);
        // Appel de communication ou synchronisation parallèle OpenMPI
        MPI_Isend(send_right.data(), ny, MPI_DOUBLE, mesh.rank_right, 10, mesh.comm, &reqs[n_reqs++]);
    }

    // Appel de communication ou synchronisation parallèle OpenMPI
    MPI_Waitall(n_reqs, reqs, MPI_STATUSES_IGNORE);
    // Calcul et assignation numérique
    n_reqs = 0;

    // Évaluation de la condition de contrôle
    if (mesh.rank_left != MPI_PROC_NULL) {
        // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
        for (int j = 1; j <= ny; ++j) field[mesh.cell_idx(0, j)] = recv_left[j - 1];
    }
    // Évaluation de la condition de contrôle
    if (mesh.rank_right != MPI_PROC_NULL) {
        // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
        for (int j = 1; j <= ny; ++j) field[mesh.cell_idx(nx + 1, j)] = recv_right[j - 1];
    }

    // Préparation des buffers Y
    for (int i = 1; i <= nx; ++i) {
        // Calcul et assignation numérique
        send_bot[i - 1] = field[mesh.cell_idx(i, 1)];
        // Calcul et assignation numérique
        send_top[i - 1] = field[mesh.cell_idx(i, ny)];
    }

    // Échanges Nord-Sud
    if (mesh.rank_bottom != MPI_PROC_NULL) {
        // Appel de communication ou synchronisation parallèle OpenMPI
        MPI_Irecv(recv_bot.data(), nx, MPI_DOUBLE, mesh.rank_bottom, 20, mesh.comm, &reqs[n_reqs++]);
        // Appel de communication ou synchronisation parallèle OpenMPI
        MPI_Isend(send_bot.data(), nx, MPI_DOUBLE, mesh.rank_bottom, 21, mesh.comm, &reqs[n_reqs++]);
    }
    // Évaluation de la condition de contrôle
    if (mesh.rank_top != MPI_PROC_NULL) {
        // Appel de communication ou synchronisation parallèle OpenMPI
        MPI_Irecv(recv_top.data(), nx, MPI_DOUBLE, mesh.rank_top, 21, mesh.comm, &reqs[n_reqs++]);
        // Appel de communication ou synchronisation parallèle OpenMPI
        MPI_Isend(send_top.data(), nx, MPI_DOUBLE, mesh.rank_top, 20, mesh.comm, &reqs[n_reqs++]);
    }

    // Appel de communication ou synchronisation parallèle OpenMPI
    MPI_Waitall(n_reqs, reqs, MPI_STATUSES_IGNORE);

    // Évaluation de la condition de contrôle
    if (mesh.rank_bottom != MPI_PROC_NULL) {
        // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
        for (int i = 1; i <= nx; ++i) field[mesh.cell_idx(i, 0)] = recv_bot[i - 1];
    }
    // Évaluation de la condition de contrôle
    if (mesh.rank_top != MPI_PROC_NULL) {
        // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
        for (int i = 1; i <= nx; ++i) field[mesh.cell_idx(i, ny + 1)] = recv_top[i - 1];
    }
}

void MpiComm::sync_u(const Mesh2D& mesh, std::vector<double>& u) {
    // Évaluation de la condition de contrôle
    if (mesh.num_procs == 1) return;

    // Déclaration de variable typée
    int nx = mesh.nx_local;
    // Déclaration de variable typée
    int ny = mesh.ny_local;

    // u est défini sur les faces verticales i de 1 à nx+1 pour l'intérieur
    std::vector<double> send_left(ny), recv_left(ny);
    // Déclaration de variable typée
    std::vector<double> send_right(ny), recv_right(ny);
    // Déclaration de variable typée
    std::vector<double> send_bot(nx + 2), recv_bot(nx + 2);
    // Déclaration de variable typée
    std::vector<double> send_top(nx + 2), recv_top(nx + 2);

    // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
    for (int j = 1; j <= ny; ++j) {
        // Calcul et assignation numérique
        send_left[j - 1]  = u[mesh.u_idx(1, j)];
        // Calcul et assignation numérique
```

```
145         send_right[j - 1] = u[mesh.u_idx(nx, j)];
146     }
147 
148     // Appel de communication ou synchronisation parallèle OpenMPI
149     MPI_Request reqs[8];
150     // Déclaration de variable typée
151     int n_reqs = 0;
152 
153     // Évaluation de la condition de contrôle
154     if (mesh.rank_left != MPI_PROC_NULL) {
155         // Appel de communication ou synchronisation parallèle OpenMPI
156         MPI_Irecv(recv_left.data(), ny, MPI_DOUBLE, mesh.rank_left, 30, mesh.comm, &reqs[n_reqs++]);
157         // Appel de communication ou synchronisation parallèle OpenMPI
158         MPI_Isend(send_left.data(), ny, MPI_DOUBLE, mesh.rank_left, 31, mesh.comm, &reqs[n_reqs++]);
159     }
160     // Évaluation de la condition de contrôle
161     if (mesh.rank_right != MPI_PROC_NULL) {
162         // Appel de communication ou synchronisation parallèle OpenMPI
163         MPI_Irecv(recv_right.data(), ny, MPI_DOUBLE, mesh.rank_right, 31, mesh.comm, &reqs[n_reqs++]);
164         // Appel de communication ou synchronisation parallèle OpenMPI
165         MPI_Isend(send_right.data(), ny, MPI_DOUBLE, mesh.rank_right, 30, mesh.comm, &reqs[n_reqs++]);
166     }
167 
168     // Appel de communication ou synchronisation parallèle OpenMPI
169     MPI_Waitall(n_reqs, reqs, MPI_STATUSES_IGNORE);
170     // Calcul et assignation numérique
171     n_reqs = 0;
172 
173     // Évaluation de la condition de contrôle
174     if (mesh.rank_left != MPI_PROC_NULL) {
175         // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
176         for (int j = 1; j <= ny; ++j) u[mesh.u_idx(0, j)] = recv_left[j - 1];
177     }
178     // Évaluation de la condition de contrôle
179     if (mesh.rank_right != MPI_PROC_NULL) {
180         // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
181         for (int j = 1; j <= ny; ++j) u[mesh.u_idx(nx + 1, j)] = recv_right[j - 1];
182     }
183 
184     // Échange selon Y
185     for (int i = 0; i <= nx + 1; ++i) {
186         // Calcul et assignation numérique
187         send_bot[i] = u[mesh.u_idx(i, 1)];
188         // Calcul et assignation numérique
189         send_top[i] = u[mesh.u_idx(i, ny)];
190     }
191 
192     // Évaluation de la condition de contrôle
193     if (mesh.rank_bottom != MPI_PROC_NULL) {
194         // Appel de communication ou synchronisation parallèle OpenMPI
195         MPI_Irecv(recv_bot.data(), nx + 2, MPI_DOUBLE, mesh.rank_bottom, 40, mesh.comm, &reqs[n_reqs++]);
196         // Appel de communication ou synchronisation parallèle OpenMPI
197         MPI_Isend(send_bot.data(), nx + 2, MPI_DOUBLE, mesh.rank_bottom, 41, mesh.comm, &reqs[n_reqs++]);
198     }
199     // Évaluation de la condition de contrôle
200     if (mesh.rank_top != MPI_PROC_NULL) {
201         // Appel de communication ou synchronisation parallèle OpenMPI
202         MPI_Irecv(recv_top.data(), nx + 2, MPI_DOUBLE, mesh.rank_top, 41, mesh.comm, &reqs[n_reqs++]);
203         // Appel de communication ou synchronisation parallèle OpenMPI
204         MPI_Isend(send_top.data(), nx + 2, MPI_DOUBLE, mesh.rank_top, 40, mesh.comm, &reqs[n_reqs++]);
205     }
206 
207     // Appel de communication ou synchronisation parallèle OpenMPI
208     MPI_Waitall(n_reqs, reqs, MPI_STATUSES_IGNORE);
209 
210     // Évaluation de la condition de contrôle
211     if (mesh.rank_bottom != MPI_PROC_NULL) {
212         // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
213         for (int i = 0; i <= nx + 1; ++i) u[mesh.u_idx(i, 0)] = recv_bot[i];
214     }
215     // Évaluation de la condition de contrôle
216     if (mesh.rank_top != MPI_PROC_NULL) {
217         // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
218         for (int i = 0; i <= nx + 1; ++i) u[mesh.u_idx(i, ny + 1)] = recv_top[i];
219     }
220 }
221 
222 void MpiComm::sync_v(const Mesh2D& mesh, std::vector<double>& v) {
223     // Évaluation de la condition de contrôle
224     if (mesh.num_procs == 1) return;
225 
226     // Déclaration de variable typée
227     int nx = mesh.nx_local;
228     // Déclaration de variable typée
229     int ny = mesh.ny_local;
230 
231     // v est défini sur les faces horizontales j de 1 à ny+1 pour l'intérieur
232     std::vector<double> send_left(ny + 2), recv_left(ny + 2);
233     // Déclaration de variable typée
234     std::vector<double> send_right(ny + 2), recv_right(ny + 2);
235     // Déclaration de variable typée
236     std::vector<double> send_bot(nx), recv_bot(nx);
```

```
237     // Déclaration de variable typée
238     std::vector<double> send_top(nx), recv_top(nx);
239 
240     // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
241     for (int j = 0; j <= ny + 1; ++j) {
242         // Calcul et assignation numérique
243         send_left[j]  = v[mesh.v_idx(1, j)];
244         // Calcul et assignation numérique
245         send_right[j] = v[mesh.v_idx(nx, j)];
246     }
247 
248     // Appel de communication ou synchronisation parallèle OpenMPI
249     MPI_Request reqs[8];
250     // Déclaration de variable typée
251     int n_reqs = 0;
252 
253     // Évaluation de la condition de contrôle
254     if (mesh.rank_left != MPI_PROC_NULL) {
255         // Appel de communication ou synchronisation parallèle OpenMPI
256         MPI_Irecv(recv_left.data(), ny + 2, MPI_DOUBLE, mesh.rank_left, 50, mesh.comm, &reqs[n_reqs++]);
257         // Appel de communication ou synchronisation parallèle OpenMPI
258         MPI_Isend(send_left.data(), ny + 2, MPI_DOUBLE, mesh.rank_left, 51, mesh.comm, &reqs[n_reqs++]);
259     }
260     // Évaluation de la condition de contrôle
261     if (mesh.rank_right != MPI_PROC_NULL) {
262         // Appel de communication ou synchronisation parallèle OpenMPI
263         MPI_Irecv(recv_right.data(), ny + 2, MPI_DOUBLE, mesh.rank_right, 51, mesh.comm, &reqs[n_reqs++]);
264         // Appel de communication ou synchronisation parallèle OpenMPI
265         MPI_Isend(send_right.data(), ny + 2, MPI_DOUBLE, mesh.rank_right, 50, mesh.comm, &reqs[n_reqs++]);
266     }
267 
268     // Appel de communication ou synchronisation parallèle OpenMPI
269     MPI_Waitall(n_reqs, reqs, MPI_STATUSES_IGNORE);
270     // Calcul et assignation numérique
271     n_reqs = 0;
272 
273     // Évaluation de la condition de contrôle
274     if (mesh.rank_left != MPI_PROC_NULL) {
275         // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
276         for (int j = 0; j <= ny + 1; ++j) v[mesh.v_idx(0, j)] = recv_left[j];
277     }
278     // Évaluation de la condition de contrôle
279     if (mesh.rank_right != MPI_PROC_NULL) {
280         // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
281         for (int j = 0; j <= ny + 1; ++j) v[mesh.v_idx(nx + 1, j)] = recv_right[j];
282     }
283 
284     // Échange selon Y
285     for (int i = 1; i <= nx; ++i) {
286         // Calcul et assignation numérique
287         send_bot[i - 1] = v[mesh.v_idx(i, 1)];
288         // Calcul et assignation numérique
289         send_top[i - 1] = v[mesh.v_idx(i, ny)];
290     }
291 
292     // Évaluation de la condition de contrôle
293     if (mesh.rank_bottom != MPI_PROC_NULL) {
294         // Appel de communication ou synchronisation parallèle OpenMPI
295         MPI_Irecv(recv_bot.data(), nx, MPI_DOUBLE, mesh.rank_bottom, 60, mesh.comm, &reqs[n_reqs++]);
296         // Appel de communication ou synchronisation parallèle OpenMPI
297         MPI_Isend(send_bot.data(), nx, MPI_DOUBLE, mesh.rank_bottom, 61, mesh.comm, &reqs[n_reqs++]);
298     }
299     // Évaluation de la condition de contrôle
300     if (mesh.rank_top != MPI_PROC_NULL) {
301         // Appel de communication ou synchronisation parallèle OpenMPI
302         MPI_Irecv(recv_top.data(), nx, MPI_DOUBLE, mesh.rank_top, 61, mesh.comm, &reqs[n_reqs++]);
303         // Appel de communication ou synchronisation parallèle OpenMPI
304         MPI_Isend(send_top.data(), nx, MPI_DOUBLE, mesh.rank_top, 60, mesh.comm, &reqs[n_reqs++]);
305     }
306 
307     // Appel de communication ou synchronisation parallèle OpenMPI
308     MPI_Waitall(n_reqs, reqs, MPI_STATUSES_IGNORE);
309 
310     // Évaluation de la condition de contrôle
311     if (mesh.rank_bottom != MPI_PROC_NULL) {
312         // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
313         for (int i = 1; i <= nx; ++i) v[mesh.v_idx(i, 0)] = recv_bot[i - 1];
314     }
315     // Évaluation de la condition de contrôle
316     if (mesh.rank_top != MPI_PROC_NULL) {
317         // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
318         for (int i = 1; i <= nx; ++i) v[mesh.v_idx(i, ny + 1)] = recv_top[i - 1];
319     }
320 }
321 
322 void MpiComm::apply_boundary_conditions(const Mesh2D& mesh, const DomainBC& bc, FlowFields& f, bool
        homogeneous) {
323     // Déclaration de variable typée
324     int nx = mesh.nx_local;
325     // Déclaration de variable typée
326     int ny = mesh.ny_local;
327 
```

```
328     // Déclaration de variable typée
329     double u_left_val   = homogeneous ? 0.0 : bc.left.u_val;
330     // Déclaration de variable typée
331     double v_left_val   = homogeneous ? 0.0 : bc.left.v_val;
332     // Déclaration de variable typée
333     double u_right_val  = homogeneous ? 0.0 : bc.right.u_val;
334     // Déclaration de variable typée
335     double v_right_val  = homogeneous ? 0.0 : bc.right.v_val;
336     // Déclaration de variable typée
337     double u_bottom_val = homogeneous ? 0.0 : bc.bottom.u_val;
338     // Déclaration de variable typée
339     double v_bottom_val = homogeneous ? 0.0 : bc.bottom.v_val;
340     // Déclaration de variable typée
341     double u_top_val    = homogeneous ? 0.0 : bc.top.u_val;
342     // Déclaration de variable typée
343     double v_top_val    = homogeneous ? 0.0 : bc.top.v_val;
344
345     // 0. Traitement de la périodicité si non gérée par MPI (ex: 1 seul processeur dans la direction)
346     if (bc.left.type == BCType::PERIODIC && mesh.dims[0] == 1) {
347         // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
348         for (int j = 1; j <= ny; ++j) {
349             // Calcul de la résolution géométrique de cellule
350             f.u[mesh.u_idx(0, j)]      = f.u[mesh.u_idx(nx, j)];
351             // Calcul de la résolution géométrique de cellule
352             f.u[mesh.u_idx(nx + 1, j)] = f.u[mesh.u_idx(1, j)];
353             // Calcul de la résolution géométrique de cellule
354             f.v[mesh.v_idx(0, j)]      = f.v[mesh.v_idx(nx, j)];
355             // Calcul de la résolution géométrique de cellule
356             f.v[mesh.v_idx(nx + 1, j)] = f.v[mesh.v_idx(1, j)];
357             // Calcul de la résolution géométrique de cellule
358             f.p[mesh.cell_idx(0, j)]      = f.p[mesh.cell_idx(nx, j)];
359             // Calcul de la résolution géométrique de cellule
360             f.p[mesh.cell_idx(nx + 1, j)] = f.p[mesh.cell_idx(1, j)];
361             // Calcul de la résolution géométrique de cellule
362             f.C[mesh.cell_idx(0, j)]      = f.C[mesh.cell_idx(nx, j)];
363             // Calcul de la résolution géométrique de cellule
364             f.C[mesh.cell_idx(nx + 1, j)] = f.C[mesh.cell_idx(1, j)];
365         }
366     }
367
368     // Évaluation de la condition de contrôle
369     if (bc.bottom.type == BCType::PERIODIC && mesh.dims[1] == 1) {
370         // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
371         for (int i = 1; i <= nx; ++i) {
372             // Calcul de la résolution géométrique de cellule
373             f.v[mesh.v_idx(i, 0)]      = f.v[mesh.v_idx(i, ny)];
374             // Calcul de la résolution géométrique de cellule
375             f.v[mesh.v_idx(i, ny + 1)] = f.v[mesh.v_idx(i, 1)];
376             // Calcul de la résolution géométrique de cellule
377             f.u[mesh.u_idx(i, 0)]      = f.u[mesh.u_idx(i, ny)];
378             // Calcul de la résolution géométrique de cellule
379             f.u[mesh.u_idx(i, ny + 1)] = f.u[mesh.u_idx(i, 1)];
380             // Calcul de la résolution géométrique de cellule
381             f.p[mesh.cell_idx(i, 0)]      = f.p[mesh.cell_idx(i, ny)];
382             // Calcul de la résolution géométrique de cellule
383             f.p[mesh.cell_idx(i, ny + 1)] = f.p[mesh.cell_idx(i, 1)];
384             // Calcul de la résolution géométrique de cellule
385             f.C[mesh.cell_idx(i, 0)]      = f.C[mesh.cell_idx(i, ny)];
386             // Calcul de la résolution géométrique de cellule
387             f.C[mesh.cell_idx(i, ny + 1)] = f.C[mesh.cell_idx(i, 1)];
388         }
389     }
390
391     // 1. Bord gauche physique (x = 0)
392     if (mesh.is_boundary_left) {
393         // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
394         for (int j = 1; j <= ny; ++j) {
395             // Évaluation de la condition de contrôle
396             if (bc.left.type == BCType::DIRICHLET) {
397                 // Déclaration de variable typée
398                 double u_inlet = u_left_val;
399                 // Évaluation de la condition de contrôle
400                 if (!homogeneous) {
401                     // Évaluation de la condition de contrôle
402                     if (bc.left.profile == "POISEUILLE_STEP") {
403                         // Déclaration de variable typée
404                         double y = mesh.yc(j);
405                         // Évaluation de la condition de contrôle
406                         if (y < bc.left.y_step) {
407                             // Calcul et assignation numérique
408                             u_inlet = 0.0;
409                         } else {
410                             // Déclaration de variable typée
411                             double h = mesh.ly - bc.left.y_step;
412                             // Calcul et assignation numérique
413                             u_inlet = 6.0 * bc.left.u_mean * (y - bc.left.y_step) * (mesh.ly - y) / (h * h);
414                         }
415                     // Calcul et assignation numérique
416                     } else if (bc.left.profile == "PARABOLIC") {
417                         // Déclaration de variable typée
418                         double y = mesh.yc(j);
419                         // Déclaration de variable typée
```

```
                        double h = mesh.ly;
                        // Calcul et assignation numérique
                        u_inlet = 6.0 * bc.left.u_mean * y * (h - y) / (h * h);
                    }
                    // Calcul de la résolution géométrique de cellule
                    f.u[mesh.u_idx(1, j)] = u_inlet;
                }
                // Calcul de la résolution géométrique de cellule
                f.u[mesh.u_idx(0, j)] = u_inlet;
                // Calcul de la résolution géométrique de cellule
                f.v[mesh.v_idx(0, j)] = 2.0 * v_left_val - f.v[mesh.v_idx(1, j)];
            // Calcul et assignation numérique
            } else if (bc.left.type == BCType::FREE_SLIP) {
                // Évaluation de la condition de contrôle
                if (!homogeneous) f.u[mesh.u_idx(1, j)] = 0.0;
                // Calcul de la résolution géométrique de cellule
                f.u[mesh.u_idx(0, j)] = 0.0;
                // Calcul de la résolution géométrique de cellule
                f.v[mesh.v_idx(0, j)] = f.v[mesh.v_idx(1, j)];
            // Calcul et assignation numérique
            } else if (bc.left.type == BCType::NEUMANN) {
                // Calcul de la résolution géométrique de cellule
                f.u[mesh.u_idx(0, j)] = f.u[mesh.u_idx(1, j)];
                // Calcul de la résolution géométrique de cellule
                f.v[mesh.v_idx(0, j)] = f.v[mesh.v_idx(1, j)];
            }
            // Calcul de la résolution géométrique de cellule
            f.p[mesh.cell_idx(0, j)] = f.p[mesh.cell_idx(1, j)];
            // Calcul de la résolution géométrique de cellule
            f.C[mesh.cell_idx(0, j)] = f.C[mesh.cell_idx(1, j)];
        }
    }

    // 2. Bord droit physique (x = Lx)
    if (mesh.is_boundary_right) {
        // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
        for (int j = 1; j <= ny; ++j) {
            // Évaluation de la condition de contrôle
            if (bc.right.type == BCType::DIRICHLET) {
                // Évaluation de la condition de contrôle
                if (!homogeneous) f.u[mesh.u_idx(nx + 1, j)] = u_right_val;
                // Calcul de la résolution géométrique de cellule
                f.u[mesh.u_idx(nx + 2, j)] = u_right_val;
                // Calcul de la résolution géométrique de cellule
                f.v[mesh.v_idx(nx + 1, j)] = 2.0 * v_right_val - f.v[mesh.v_idx(nx, j)];
            // Calcul et assignation numérique
            } else if (bc.right.type == BCType::FREE_SLIP) {
                // Évaluation de la condition de contrôle
                if (!homogeneous) f.u[mesh.u_idx(nx + 1, j)] = 0.0;
                // Calcul de la résolution géométrique de cellule
                f.u[mesh.u_idx(nx + 2, j)] = 0.0;
                // Calcul de la résolution géométrique de cellule
                f.v[mesh.v_idx(nx + 1, j)] = f.v[mesh.v_idx(nx, j)];
            // Calcul et assignation numérique
            } else if (bc.right.type == BCType::NEUMANN) {
                // Calcul de la résolution géométrique de cellule
                f.u[mesh.u_idx(nx + 2, j)] = f.u[mesh.u_idx(nx + 1, j)];
                // Calcul de la résolution géométrique de cellule
                f.v[mesh.v_idx(nx + 1, j)] = f.v[mesh.v_idx(nx, j)];
            }
            // Calcul de la résolution géométrique de cellule
            f.p[mesh.cell_idx(nx + 1, j)] = f.p[mesh.cell_idx(nx, j)];
            // Calcul de la résolution géométrique de cellule
            f.C[mesh.cell_idx(nx + 1, j)] = f.C[mesh.cell_idx(nx, j)];
        }
    }

    // 3. Bord inférieur physique (y = 0)
    if (mesh.is_boundary_bottom) {
        // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
        for (int i = 1; i <= nx; ++i) {
            // Évaluation de la condition de contrôle
            if (bc.bottom.type == BCType::DIRICHLET) {
                // Évaluation de la condition de contrôle
                if (!homogeneous) f.v[mesh.v_idx(i, 1)] = v_bottom_val;
                // Calcul de la résolution géométrique de cellule
                f.v[mesh.v_idx(i, 0)] = v_bottom_val;
                // Calcul de la résolution géométrique de cellule
                f.u[mesh.u_idx(i, 0)] = 2.0 * u_bottom_val - f.u[mesh.u_idx(i, 1)];
            // Calcul et assignation numérique
            } else if (bc.bottom.type == BCType::FREE_SLIP) {
                // Évaluation de la condition de contrôle
                if (!homogeneous) f.v[mesh.v_idx(i, 1)] = 0.0;
                // Calcul de la résolution géométrique de cellule
                f.v[mesh.v_idx(i, 0)] = 0.0;
                // Calcul de la résolution géométrique de cellule
                f.u[mesh.u_idx(i, 0)] = f.u[mesh.u_idx(i, 1)];
            // Calcul et assignation numérique
            } else if (bc.bottom.type == BCType::NEUMANN) {
                // Calcul de la résolution géométrique de cellule
                f.u[mesh.u_idx(i, 0)] = f.u[mesh.u_idx(i, 1)];
                // Calcul de la résolution géométrique de cellule
```

```
                f.v[mesh.v_idx(i, 0)] = f.v[mesh.v_idx(i, 1)];
            }
            // Calcul de la résolution géométrique de cellule
            f.p[mesh.cell_idx(i, 0)] = f.p[mesh.cell_idx(i, 1)];
            // Calcul de la résolution géométrique de cellule
            f.C[mesh.cell_idx(i, 0)] = f.C[mesh.cell_idx(i, 1)];
        }
    }

    // 4. Bord supérieur physique (y = Ly)
    if (mesh.is_boundary_top) {
        // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
        for (int i = 1; i <= nx; ++i) {
            // Déclaration de variable typée
            double x_c = mesh.xc(i);
            // Évaluation du profil d'injecteur (Cas 3.3 Fluids 2021)
            if (bc.top.profile == "JET_INLET") {
                // Déclaration de variable typée pour le centre et demi-largeur
                double xc = (bc.top.jet_center > 0.0) ? bc.top.jet_center : 0.5 * mesh.lx;
                // Calcul de la demi-largeur de l'injecteur
                double half_w = (bc.top.jet_width > 0.0) ? 0.5 * bc.top.jet_width : 0.04;
                // Calcul de la largeur discrète effective de l'injecteur sur le maillage global
                double d_disc = 0.0;
                for (int ig = 1; ig <= mesh.nx_global; ++ig) {
                    double xg = (ig - 0.5) * mesh.dx;
                    if (std::abs(xg - xc) <= half_w) d_disc += mesh.dx;
                }
                if (d_disc < 1.0e-12) d_disc = 2.0 * half_w;
                double v_in_eff = v_top_val * (2.0 * half_w) / d_disc;
                // Évaluation de l'appartenance à l'injecteur
                if (std::abs(x_c - xc) <= half_w) {
                    // Imposition de la vitesse d'injection (préservée pour la ligne diagonale lors de matvec)
                    if (!homogeneous) f.v[mesh.v_idx(i, ny + 1)] = v_in_eff;
                    // Calcul de la cellule fantôme supérieure pour divergence nulle au bord
                    f.v[mesh.v_idx(i, ny + 2)] = homogeneous ? 0.0 : v_in_eff;
                    // Injection continue de liquide (Fluide 2 : C = 0)
                    f.C[mesh.cell_idx(i, ny + 1)] = 0.0;
                } else {
                    // Calcul du débit de compensation d'échappement du gaz pour incompressibilité stricte
                    double v_out = -v_top_val * (2.0 * half_w) / (mesh.lx - d_disc);
                    // Imposition de la vitesse d'évacuation (préservée pour la ligne diagonale lors de matvec
                        )
                    if (!homogeneous) f.v[mesh.v_idx(i, ny + 1)] = v_out;
                    // Calcul de la cellule fantôme supérieure
                    f.v[mesh.v_idx(i, ny + 2)] = homogeneous ? 0.0 : v_out;
                    // Évacuation du gaz par sortie libre Neumann
                    f.C[mesh.cell_idx(i, ny + 1)] = f.C[mesh.cell_idx(i, ny)];
                }
                // Condition tangentielle symétrique évaluée à l'abscisse exacte de la face u
                f.u[mesh.u_idx(i, ny + 1)] = (std::abs(mesh.xu(i) - xc) <= half_w) ? 0.0 : f.u[mesh.u_idx(i,
                    ny)];
                if (i == nx) f.u[mesh.u_idx(nx + 1, ny + 1)] = f.u[mesh.u_idx(nx + 1, ny)];
            // Évaluation de la condition de contrôle standard
            } else if (bc.top.type == BCType::DIRICHLET) {
                // Évaluation de la condition de contrôle
                if (!homogeneous) f.v[mesh.v_idx(i, ny + 1)] = v_top_val;
                // Calcul de la résolution géométrique de cellule
                f.v[mesh.v_idx(i, ny + 2)] = v_top_val;
                // Calcul de la résolution géométrique de cellule
                f.u[mesh.u_idx(i, ny + 1)] = 2.0 * u_top_val - f.u[mesh.u_idx(i, ny)];
                // Calcul de la résolution géométrique de cellule
                f.C[mesh.cell_idx(i, ny + 1)] = f.C[mesh.cell_idx(i, ny)];
            // Calcul et assignation numérique
            } else if (bc.top.type == BCType::FREE_SLIP) {
                // Évaluation de la condition de contrôle
                if (!homogeneous) f.v[mesh.v_idx(i, ny + 1)] = 0.0;
                // Calcul de la résolution géométrique de cellule
                f.v[mesh.v_idx(i, ny + 2)] = 0.0;
                // Calcul de la résolution géométrique de cellule
                f.u[mesh.u_idx(i, ny + 1)] = f.u[mesh.u_idx(i, ny)];
                // Calcul de la résolution géométrique de cellule
                f.C[mesh.cell_idx(i, ny + 1)] = f.C[mesh.cell_idx(i, ny)];
            // Calcul et assignation numérique
            } else if (bc.top.type == BCType::NEUMANN) {
                // Calcul de la résolution géométrique de cellule
                f.u[mesh.u_idx(i, ny + 1)] = f.u[mesh.u_idx(i, ny)];
                // Calcul de la résolution géométrique de cellule
                f.v[mesh.v_idx(i, ny + 2)] = f.v[mesh.v_idx(i, ny + 1)];
                // Calcul de la résolution géométrique de cellule
                f.C[mesh.cell_idx(i, ny + 1)] = f.C[mesh.cell_idx(i, ny)];
            }
            // Calcul de la résolution géométrique de cellule
            f.p[mesh.cell_idx(i, ny + 1)] = f.p[mesh.cell_idx(i, ny)];
        }
    }
}

void MpiComm::apply_thermal_boundary_conditions(const Mesh2D& mesh, const DomainBC& bc, FlowFields& f) {
    // Déclaration de variable typée
    int nx = mesh.nx_local;
    // Déclaration de variable typée
    int ny = mesh.ny_local;
```

```
602
603     // Périodicité si 1 seul processeur dans la direction
604     if (bc.left.T_type == BCType::PERIODIC && mesh.dims[0] == 1) {
605         // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
606         for (int j = 1; j <= ny; ++j) {
607             // Calcul de la résolution géométrique de cellule
608             f.T[mesh.cell_idx(0, j)]      = f.T[mesh.cell_idx(nx, j)];
609             // Calcul de la résolution géométrique de cellule
610             f.T[mesh.cell_idx(nx + 1, j)] = f.T[mesh.cell_idx(1, j)];
611         }
612     }
613     // Évaluation de la condition de contrôle
614     if (bc.bottom.T_type == BCType::PERIODIC && mesh.dims[1] == 1) {
615         // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
616         for (int i = 1; i <= nx; ++i) {
617             // Calcul de la résolution géométrique de cellule
618             f.T[mesh.cell_idx(i, 0)]      = f.T[mesh.cell_idx(i, ny)];
619             // Calcul de la résolution géométrique de cellule
620             f.T[mesh.cell_idx(i, ny + 1)] = f.T[mesh.cell_idx(i, 1)];
621         }
622     }
623
624     // Bord gauche physique (x = 0)
625     if (mesh.is_boundary_left) {
626         // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
627         for (int j = 1; j <= ny; ++j) {
628             // Évaluation de la condition de contrôle
629             if (bc.left.T_type == BCType::DIRICHLET) {
630                 // Calcul de la résolution géométrique de cellule
631                 f.T[mesh.cell_idx(0, j)] = 2.0 * bc.left.T_val - f.T[mesh.cell_idx(1, j)];
632             } else { // NEUMANN / Adiabatique
633                 // Calcul de la résolution géométrique de cellule
634                 f.T[mesh.cell_idx(0, j)] = f.T[mesh.cell_idx(1, j)];
635             }
636         }
637     }
638
639     // Bord droit physique (x = Lx)
640     if (mesh.is_boundary_right) {
641         // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
642         for (int j = 1; j <= ny; ++j) {
643             // Évaluation de la condition de contrôle
644             if (bc.right.T_type == BCType::DIRICHLET) {
645                 // Calcul de la résolution géométrique de cellule
646                 f.T[mesh.cell_idx(nx + 1, j)] = 2.0 * bc.right.T_val - f.T[mesh.cell_idx(nx, j)];
647             } else { // NEUMANN
648                 // Calcul de la résolution géométrique de cellule
649                 f.T[mesh.cell_idx(nx + 1, j)] = f.T[mesh.cell_idx(nx, j)];
650             }
651         }
652     }
653
654     // Bord inférieur physique (y = 0)
655     if (mesh.is_boundary_bottom) {
656         // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
657         for (int i = 1; i <= nx; ++i) {
658             // Évaluation de la condition de contrôle
659             if (bc.bottom.T_type == BCType::DIRICHLET) {
660                 // Calcul de la résolution géométrique de cellule
661                 f.T[mesh.cell_idx(i, 0)] = 2.0 * bc.bottom.T_val - f.T[mesh.cell_idx(i, 1)];
662             } else { // NEUMANN / Adiabatique
663                 // Calcul de la résolution géométrique de cellule
664                 f.T[mesh.cell_idx(i, 0)] = f.T[mesh.cell_idx(i, 1)];
665             }
666         }
667     }
668
669     // Bord supérieur physique (y = Ly)
670     if (mesh.is_boundary_top) {
671         // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
672         for (int i = 1; i <= nx; ++i) {
673             // Évaluation de la condition de contrôle
674             if (bc.top.T_type == BCType::DIRICHLET) {
675                 // Calcul de la résolution géométrique de cellule
676                 f.T[mesh.cell_idx(i, ny + 1)] = 2.0 * bc.top.T_val - f.T[mesh.cell_idx(i, ny)];
677             } else { // NEUMANN / Adiabatique
678                 // Calcul de la résolution géométrique de cellule
679                 f.T[mesh.cell_idx(i, ny + 1)] = f.T[mesh.cell_idx(i, ny)];
680             }
681         }
682     }
683 }
684
685 // Déclaration de variable typée
686 double MpiComm::global_dot_product(const Mesh2D& mesh, const std::vector<double>& a, const std::vector<double
        >& b, int size) {
687     // Déclaration de variable typée
688     double local_sum = 0.0;
689     // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
690     for (int i = 0; i < size; ++i) {
691         // Calcul et assignation numérique
692         local_sum += a[i] * b[i];
```

```
693     }
694     // Déclaration de variable typée
695     double global_sum = 0.0;
696     // Appel de communication ou synchronisation parallèle OpenMPI
697     MPI_Allreduce(&local_sum, &global_sum, 1, MPI_DOUBLE, MPI_SUM, mesh.comm);
698     // Retour de fonction
699     return global_sum;
700 }
701
702 // Déclaration de variable typée
703 double MpiComm::global_norm2(const Mesh2D& mesh, const std::vector<double>& v, int size) {
704     // Retour de fonction
705     return std::sqrt(global_dot_product(mesh, v, v, size));
706 }
707
708 // Déclaration de variable typée
709 double MpiComm::global_max(const Mesh2D& mesh, double local_val) {
710     // Déclaration de variable typée
711     double global_val = 0.0;
712     // Appel de communication ou synchronisation parallèle OpenMPI
713     MPI_Allreduce(&local_val, &global_val, 1, MPI_DOUBLE, MPI_MAX, mesh.comm);
714     // Retour de fonction
715     return global_val;
716 }
717
718 // Déclaration de variable typée
719 double MpiComm::global_sum(const Mesh2D& mesh, double local_val) {
720     // Déclaration de variable typée
721     double global_val = 0.0;
722     // Appel de communication ou synchronisation parallèle OpenMPI
723     MPI_Allreduce(&local_val, &global_val, 1, MPI_DOUBLE, MPI_SUM, mesh.comm);
724     // Retour de fonction
725     return global_val;
726 }
727
728 } // namespace covof
```

Chapitre 6

Algèbre Linéaire Creuse au Format CSR (CSRMatrix.hpp, CSRMatrix.cpp)

6.1 Représentation Compressed Sparse Row (CSR)

Le système linéaire monolithique du Lagrangien Augmenté comporte $N_{\text{dof}} = N_u + N_v$ degrés de liberté par sous-domaine, mais chaque ligne de la matrice ne possède qu'au plus 9 coefficients non nuls (5 termes directs du Laplacien de la composante courante + 4 termes croisés du gradient de divergence couplant u et v). Stocker une matrice dense nécessiterait $\mathcal{O}(N_{\text{dof}}^2)$ octets (plusieurs gigaoctets dès 128×128), tandis que le format creux **CSR (*Compressed Sparse Row*)** implémenté dans `covof::CSRMatrix` réduit l'empreinte mémoire à $\mathcal{O}(9N_{\text{dof}})$ (quelques mégaoctets).

Une matrice creuse $\mathbf{A} \in \mathbb{R}^{n \times n}$ comportant nnz éléments non nuls est représentée par trois tableaux contigus :

1. `values` ($\in \mathbb{R}^{\text{nnz}}$) : valeurs flottantes des éléments non nuls parcourus ligne par ligne ;
2. `col_ind` ($\in \mathbb{N}^{\text{nnz}}$) : indices de colonnes j correspondants ;
3. `row_ptr` ($\in \mathbb{N}^{n+1}$) : indice de début de chaque ligne i dans `values` et `col_ind`, avec `row_ptr`$[0] = 0$ et `row_ptr`$[n] = \text{nnz}$.

6.2 Produit Matrice-Vecteur Creux (`multiply` / SpMV)

Pour calculer $\mathbf{y} = \mathbf{A}\mathbf{x}$, la méthode `CSRMatrix::multiply(const std::vector<double>& x, std::vector<double>& y)` évalue pour chaque ligne $i \in \{0, \ldots, n-1\}$:

$$y_i = \sum_{k=\texttt{row_ptr}[i]}^{\texttt{row_ptr}[i+1]-1} \texttt{values}[k]\, x_{\texttt{col_ind}[k]} \tag{6.1}$$

Cette opération s'exécute en exactement $2\,\text{nnz}$ opérations flottantes (FLOPs) avec un parcours purement séquentiel des tableaux `values` et `col_ind`.

6.3 Répertoire des Classes, Structures et Méthodes de `include/covof/CSRMatr`

Ligne	Catégorie	Identifiant C++17	Signature / Rôle
13	class	`CSRMatrix`	Déclaration de type / classe / structure
34	Méthode / Fonction	`init`	Type retour : void
54	Méthode / Fonction	`matvec`	Type retour : void
59	Méthode / Fonction	`solve_jacobi`	Type retour : void

6.4 Code Source Intégral Annoté : `include/covof/CSRMatrix.hpp`

Le listing ci-dessous présente l'intégralité des 62 lignes du fichier `include/covof/CSRMatrix.hpp` de l'implémentation C++17 :

```
1 #pragma once
2
3 #include "Mesh.hpp"
4 #include <vector>
5
```

```cpp
namespace covof {
namespace fugu = covof; // Alias rétrocompatibilité

/**
 * @brief Structure de matrice creuse locale au format Compressed Sparse Row (CSR)
 * Utilisée pour les blocs de préconditionnement (Fuu, Fuv, Laplacien de Schur).
 */
class CSRMatrix {
public:
    // Déclaration de variable typée
    int nrows = 0;
    // Déclaration de variable typée
    int ncols = 0;
    // Déclaration de variable typée
    std::vector<int> row_ptr;     ///< Pointeurs de début de ligne (taille nrows + 1)
    // Déclaration de variable typée
    std::vector<int> col_idx;     ///< Indices de colonnes
    // Déclaration de variable typée
    std::vector<double> values;   ///< Valeurs non nulles
    // Déclaration de variable typée
    std::vector<double> diag;     ///< Diagonale explicite pour le préconditionnement Jacobi / Gauss-Seidel

    // Calcul et assignation numérique
    CSRMatrix() = default;

    /**
     * @brief Initialise les dimensions
     */
    void init(int r, int c) {
        // Calcul et assignation numérique
        nrows = r;
        // Calcul et assignation numérique
        ncols = c;
        row_ptr.assign(r + 1, 0);
        col_idx.clear();
        values.clear();
        diag.assign(r, 0.0);
    }

    /**
     * @brief Construit le format CSR à partir d'une liste de triplets (row, col, value)
     */
    void set_from_triplets(int r, int c, const std::vector<int>& rows,
                           const std::vector<int>& cols, const std::vector<double>& vals);

    /**
     * @brief Produit matrice-vecteur local : y = A * x
     */
    void matvec(const std::vector<double>& x, std::vector<double>& y) const;

    /**
     * @brief Résolution approchée par itérations de Jacobi : x = D^{-1} b
     */
    void solve_jacobi(const std::vector<double>& b, std::vector<double>& x, int num_iters = 5) const;
};

} // namespace covof
```

6.5 Répertoire des Classes, Structures et Méthodes de `src/CSRMatrix.cpp`

Ligne	Catégorie	Identifiant C++17	Signature / Rôle
61	Méthode / Fonction	`CSRMatrix::matvec`	Type retour : void
77	Méthode / Fonction	`CSRMatrix::solve_jacobi`	Type retour : void
80	Méthode / Fonction	`x_new`	Type retour : std : :vector<double>

6.6 Code Source Intégral Annoté : `src/CSRMatrix.cpp`

Le listing ci-dessous présente l'intégralité des 108 lignes du fichier `src/CSRMatrix.cpp` de l'implémentation C++17 :

```cpp
#include "covof/CSRMatrix.hpp"
#include <numeric>
#include <algorithm>
#include <cassert>

namespace covof {
namespace fugu = covof; // Alias rétrocompatibilité

void CSRMatrix::set_from_triplets(int r, int c, const std::vector<int>& rows,
                                  const std::vector<int>& cols, const std::vector<double>& vals) {
    // Calcul et assignation numérique
    nrows = r;
    // Calcul et assignation numérique
```

```
    ncols = c;
    row_ptr.assign(nrows + 1, 0);
    diag.assign(nrows, 0.0);

    // 1. Comptage des éléments par ligne
    for (size_t k = 0; k < rows.size(); ++k) {
        // Évaluation de la condition de contrôle
        if (rows[k] >= 0 && rows[k] < nrows) {
            row_ptr[rows[k] + 1]++;
        }
    }

    // 2. Somme cumulée pour row_ptr
    for (int i = 0; i < nrows; ++i) {
        // Calcul et assignation numérique
        row_ptr[i + 1] += row_ptr[i];
    }

    // Déclaration de variable typée
    int nnz = row_ptr[nrows];
    col_idx.resize(nnz);
    values.resize(nnz);

    // Déclaration de variable typée
    std::vector<int> current_pos = row_ptr;

    // 3. Remplissage des indices et valeurs
    for (size_t k = 0; k < rows.size(); ++k) {
        // Déclaration de variable typée
        int row = rows[k];
        // Évaluation de la condition de contrôle
        if (row >= 0 && row < nrows) {
            // Déclaration de variable typée
            int dest = current_pos[row]++;
            // Calcul de la résolution géométrique de cellule
            col_idx[dest] = cols[k];
            // Calcul et assignation numérique
            values[dest]  = vals[k];
            // Évaluation de la condition de contrôle
            if (row == cols[k]) {
                // Calcul et assignation numérique
                diag[row] = vals[k];
            }
        }
    }
}

void CSRMatrix::matvec(const std::vector<double>& x, std::vector<double>& y) const {
    y.assign(nrows, 0.0);
    // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
    for (int i = 0; i < nrows; ++i) {
        // Déclaration de variable typée
        double sum = 0.0;
        // Boucle d'itération algorithmique
        for (int k = row_ptr[i]; k < row_ptr[i + 1]; ++k) {
            // Calcul et assignation numérique
            sum += values[k] * x[col_idx[k]];
        }
        // Calcul et assignation numérique
        y[i] = sum;
    }
}

void CSRMatrix::solve_jacobi(const std::vector<double>& b, std::vector<double>& x, int num_iters) const {
    x.resize(nrows, 0.0);
    // Déclaration de variable typée
    std::vector<double> x_new(nrows, 0.0);

    // Boucle d'itération algorithmique
    for (int it = 0; it < num_iters; ++it) {
        // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
        for (int i = 0; i < nrows; ++i) {
            // Déclaration de variable typée
            double d = (std::abs(diag[i]) > 1.0e-15) ? diag[i] : 1.0;
            // Déclaration de variable typée
            double sum = 0.0;
            // Boucle d'itération algorithmique
            for (int k = row_ptr[i]; k < row_ptr[i + 1]; ++k) {
                // Déclaration de variable typée
                int j = col_idx[k];
                // Évaluation de la condition de contrôle
                if (j != i) {
                    // Calcul et assignation numérique
                    sum += values[k] * x[j];
                }
            }
            // Calcul et assignation numérique
            x_new[i] = (b[i] - sum) / d;
        }
        // Calcul et assignation numérique
        x = x_new;
    }
```

```
106 }
107
108 } // namespace covof
```

Chapitre 7

Préconditionneurs Algébriques : Jacobi, SSOR et ILU(0) (Preconditioner.hpp, Preconditioner.cpp)

7.1 Rôle du Préconditionnement dans les Méthodes de Krylov

Pour accélérer la convergence des solveurs itératifs sur le système $\mathbf{A}\mathbf{x} = \mathbf{b}$ dont le nombre de conditionnement $\kappa(\mathbf{A})$ croît avec le paramètre du Lagrangien Augmenté r_{al} et le rapport de viscosité μ_2/μ_1, `COVOF_2D-C++` définit l'interface abstraite `covof::Preconditioner` qui résout à chaque itération de Krylov le système préconditionné $\mathbf{M}\mathbf{z} = \mathbf{r}$ (ou applique $\mathbf{z} = \mathbf{M}^{-1}\mathbf{r}$).

7.1.1 Préconditionneur Diagonal de Jacobi (`JacobiPreconditioner`)

On choisit $\mathbf{M} = \mathrm{diag}(A_{0,0}, A_{1,1}, \ldots, A_{n-1,n-1})$. Lors de la phase `setup(A)`, l'inverse de la diagonale est précalculé avec garde-fou contre les pivots nuls :

$$d_i^{-1} = \begin{cases} 1/A_{i,i} & \text{si } |A_{i,i}| > 10^{-15} \\ 1.0 & \text{sinon} \end{cases}, \qquad z_i = d_i^{-1} r_i \tag{7.1}$$

Dans un système à forts sauts de densité ($\rho_2/\rho_1 = 10^5$) et de viscosité ($\mu_2/\mu_1 = 10^{10}$), ce préconditionneur diagonal remet automatiquement à l'échelle les équations situées dans le gaz et dans le liquide dense, ramenant les éléments diagonaux de $\mathbf{M}^{-1}\mathbf{A}$ à l'unité.

7.1.2 Préconditionneur SSOR et Factorisation Incomplète `ILU(0)`

- **SSOR (*Symmetric Successive Over-Relaxation*)** : décompose $\mathbf{A} = \mathbf{D} + \mathbf{L} + \mathbf{U}$ et approche $\mathbf{M}_{\mathrm{SSOR}} = \frac{1}{\omega(2-\omega)}(\mathbf{D} + \omega\mathbf{L})\mathbf{D}^{-1}(\mathbf{D} + \omega\mathbf{U})$ inversé par une descente triangulaire inférieure suivie d'une remontée triangulaire supérieure.
- **ILU(0) (*Incomplete LU with zero fill-in*)** : calcule une factorisation $\mathbf{A} \approx \tilde{\mathbf{L}}\tilde{\mathbf{U}}$ conservant exactement le même motif de parcimonie (*sparsity pattern*) que la matrice $\mathbf{A}$ originale.

7.2 Répertoire des Classes, Structures et Méthodes de `include/covof/Precond`

Ligne	Catégorie	Identifiant C++17	Signature / Rôle
12	class	`NavierStokesSolver`	Déclaration de type / classe / structure
13	class	`FlowFields`	Déclaration de type / classe / structure
19	class	`PreconditionerBase`	Déclaration de type / classe / structure
27	Méthode / Fonction	`init`	Type retour : virtual void
32	Méthode / Fonction	`update`	Type retour : virtual void
37	Méthode / Fonction	`apply`	Type retour : virtual void
42	Méthode / Fonction	`get_name`	Type retour : virtual std : :string
50	class	`JacobiPreconditioner`	Déclaration de type / classe / structure
55	Méthode / Fonction	`init`	Type retour : void
56	Méthode / Fonction	`update`	Type retour : void
57	Méthode / Fonction	`apply`	Type retour : void
59	Méthode / Fonction	`get_name`	Type retour : std : :string
66	class	`SSORPreconditioner`	Déclaration de type / classe / structure
73	Méthode / Fonction	`init`	Type retour : void
74	Méthode / Fonction	`update`	Type retour : void
75	Méthode / Fonction	`apply`	Type retour : void
77	Méthode / Fonction	`get_name`	Type retour : std : :string
84	class	`ILU0Preconditioner`	Déclaration de type / classe / structure
91	Méthode / Fonction	`init`	Type retour : void
92	Méthode / Fonction	`update`	Type retour : void
93	Méthode / Fonction	`apply`	Type retour : void
95	Méthode / Fonction	`get_name`	Type retour : std : :string
102	class	`BlockTriangularPreconditioner`	Déclaration de type / classe / structure
107	Méthode / Fonction	`init`	Type retour : void
108	Méthode / Fonction	`update`	Type retour : void
109	Méthode / Fonction	`apply`	Type retour : void
111	Méthode / Fonction	`get_name`	Type retour : std : :string
118	class	`CustomPreconditioner`	Déclaration de type / classe / structure
126	Méthode / Fonction	`init`	Type retour : void
127	Méthode / Fonction	`update`	Type retour : void
128	Méthode / Fonction	`apply`	Type retour : void
130	Méthode / Fonction	`get_name`	Type retour : std : :string
133	Méthode / Fonction	`set_custom_operator`	Type retour : void
144	class	`PreconditionerFactory`	Déclaration de type / classe / structure
146	Méthode / Fonction	`create`	Type retour : static std : :unique_ptr<PreconditionerBase>

7.3 Code Source Intégral Annoté : `include/covof/Preconditioner.hpp`

Le listing ci-dessous présente l'intégralité des 152 lignes du fichier `include/covof/Preconditioner.hpp` de l'implémentation C++17 :

```
#pragma once

#include "Mesh.hpp"
#include <vector>
#include <string>
#include <memory>
#include <functional>

namespace covof {
namespace fugu = covof; // Alias rétrocompatibilité

class NavierStokesSolver;
class FlowFields;

/**
 * @brief Classe de base abstraite pour tous les préconditionneurs (Standard CFD / COVOF 2D)
 * Permet l'extension et la création facile de nouveaux préconditionneurs par héritage.
 */
class PreconditionerBase {
public:
    // Calcul et assignation numérique
    virtual ~PreconditionerBase() = default;

    /**
     * @brief Initialise le préconditionneur avec l'opérateur Navier-Stokes
     */
    virtual void init(const NavierStokesSolver* solver) = 0;

    /**
     * @brief Met à jour les coefficients du préconditionneur si les propriétés ou le pas de temps changent
     */
    virtual void update(const FlowFields& fields, double dt) = 0;

```

```
    /**
     * @brief Applique l'action du préconditionneur : r = P^{-1} * z
     */
    virtual void apply(const std::vector<double>& z, std::vector<double>& r) const = 0;

    /**
     * @brief Renvoie le nom identifiant du préconditionneur
     */
    virtual std::string get_name() const = 0;
};

/**
 * @brief Préconditionneur Diagonal de Jacobi
 * P = Diag(A)
 * Idéal pour le parallélisme massif et les formulations par Lagrangien Augmenté.
 */
class JacobiPreconditioner : public PreconditionerBase {
public:
    // Calcul et assignation numérique
    const NavierStokesSolver* ns_solver = nullptr;

    void init(const NavierStokesSolver* solver) override;
    void update(const FlowFields& fields, double dt) override;
    void apply(const std::vector<double>& z, std::vector<double>& r) const override;
    // Déclaration de variable typée
    std::string get_name() const override { return "JACOBI"; }
};

/**
 * @brief Préconditionneur SSOR (Symmetric Successive Over-Relaxation)
 * P = 1/(2-w) * (D/w + L) * (D/w)^{-1} * (D/w + U)
 */
class SSORPreconditioner : public PreconditionerBase {
public:
    // Calcul et assignation numérique
    const NavierStokesSolver* ns_solver = nullptr;
    // Déclaration de variable typée
    double omega = 1.0; // Facteur de relaxation (defaut Gauss-Seidel symetrique)

    void init(const NavierStokesSolver* solver) override;
    void update(const FlowFields& fields, double dt) override;
    void apply(const std::vector<double>& z, std::vector<double>& r) const override;
    // Déclaration de variable typée
    std::string get_name() const override { return "SSOR"; }
};

/**
 * @brief Préconditionneur ILU(0) (Incomplete LU à remplissage nul)
 * Factorisation incomplète locale sur chaque sous-domaine Cartésien.
 */
class ILU0Preconditioner : public PreconditionerBase {
public:
    // Calcul et assignation numérique
    const NavierStokesSolver* ns_solver = nullptr;
    // Déclaration de variable typée
    std::vector<double> ilu_diag;

    void init(const NavierStokesSolver* solver) override;
    void update(const FlowFields& fields, double dt) override;
    void apply(const std::vector<double>& z, std::vector<double>& r) const override;
    // Déclaration de variable typée
    std::string get_name() const override { return "ILU0"; }
};

/**
 * @brief Préconditionneur par Blocs Triangulaires (Annexe E thèse M. El Ouafa / Tavares)
 * Dédié au système tout-couplé vitesse-pression avec approximation du complément de Schur.
 */
class BlockTriangularPreconditioner : public PreconditionerBase {
public:
    // Calcul et assignation numérique
    const NavierStokesSolver* ns_solver = nullptr;

    void init(const NavierStokesSolver* solver) override;
    void update(const FlowFields& fields, double dt) override;
    void apply(const std::vector<double>& z, std::vector<double>& r) const override;
    // Déclaration de variable typée
    std::string get_name() const override { return "BLOCK_TRIANGULAR"; }
};

/**
 * @brief Préconditionneur Personnalisé (Hook développeur)
 * Permet au développeur d'injecter une fonction lambda ou un foncteur C++ arbitraire sans modifier le code
     source.
 */
class CustomPreconditioner : public PreconditionerBase {
public:
    // Calcul et assignation numérique
    using ApplyFunc = std::function<void(const std::vector<double>&, std::vector<double>&)>;
    ApplyFunc custom_apply;
    // Déclaration de variable typée
    std::string custom_name = "CUSTOM";
```

```
125
126     void init(const NavierStokesSolver* solver) override;
127     void update(const FlowFields& fields, double dt) override;
128     void apply(const std::vector<double>& z, std::vector<double>& r) const override;
129     // Déclaration de variable typée
130     std::string get_name() const override { return custom_name; }
131
132     // Calcul et assignation numérique
133     void set_custom_operator(ApplyFunc func, const std::string& name = "USER_DEFINED") {
134         // Calcul et assignation numérique
135         custom_apply = func;
136         // Calcul et assignation numérique
137         custom_name = name;
138     }
139 };
140
141 /**
142  * @brief Fabrique de préconditionneurs (Factory Pattern)
143  */
144 class PreconditionerFactory {
145 public:
146     static std::unique_ptr<PreconditionerBase> create(const std::string& type);
147 };
148
149 // Alias de rétrocompatibilité
150 using BlockPreconditioner = PreconditionerBase;
151
152 } // namespace covof
```

7.4 Répertoire des Classes, Structures et Méthodes de `src/Preconditioner.cp`

Ligne	Catégorie	Identifiant C++17	Signature / Rôle
13	Méthode / Fonction	`JacobiPreconditioner::init`	Type retour : void
18	Méthode / Fonction	`JacobiPreconditioner::update`	Type retour : void
22	Méthode / Fonction	`JacobiPreconditioner::apply`	Type retour : void
71	Méthode / Fonction	`SSORPreconditioner::init`	Type retour : void
76	Méthode / Fonction	`SSORPreconditioner::update`	Type retour : void
79	Méthode / Fonction	`SSORPreconditioner::apply`	Type retour : void
131	Méthode / Fonction	`ILU0Preconditioner::init`	Type retour : void
136	Méthode / Fonction	`ILU0Preconditioner::update`	Type retour : void
173	Méthode / Fonction	`ILU0Preconditioner::apply`	Type retour : void
189	Méthode / Fonction	`BlockTriangularPreconditioner::init`	Type retour : void
194	Méthode / Fonction	`BlockTriangularPreconditioner::update`	Type retour : void
197	Méthode / Fonction	`BlockTriangularPreconditioner::apply`	Type retour : void
245	Méthode / Fonction	`CustomPreconditioner::init`	Type retour : void
248	Méthode / Fonction	`CustomPreconditioner::update`	Type retour : void
251	Méthode / Fonction	`CustomPreconditioner::apply`	Type retour : void
264	Méthode / Fonction	`PreconditionerFactory::create`	Type retour : std : :unique_ptr<PreconditionerBase>

7.5 Code Source Intégral Annoté : `src/Preconditioner.cpp`

Le listing ci-dessous présente l'intégralité des 295 lignes du fichier `src/Preconditioner.cpp` de l'implémentation C++17 :

```
1  #include "covof/Preconditioner.hpp"
2  #include "covof/NavierStokesSolver.hpp"
3  #include <cmath>
4  #include <iostream>
5  #include <algorithm>
6
7  namespace covof {
8  namespace fugu = covof; // Alias rétrocompatibilité
9
10 // ============================================================================
11 // 1. Préconditionneur Jacobi Diagonal
12 // ============================================================================
13 void JacobiPreconditioner::init(const NavierStokesSolver* solver) {
14     // Calcul et assignation numérique
15     ns_solver = solver;
16 }
17
18 void JacobiPreconditioner::update(const FlowFields& fields, double dt) {
19     // Les diagonales sont calculées dans ns_solver->assemble_preconditioner_blocks
20 }
21
22 void JacobiPreconditioner::apply(const std::vector<double>& z, std::vector<double>& r) const {
23     // Évaluation de la condition de contrôle
```

```
    if (!ns_solver) return;

    // Déclaration de variable typée
    int nu = ns_solver->n_owned_u;
    // Déclaration de variable typée
    int nv = ns_solver->n_owned_v;
    // Déclaration de variable typée
    int n  = ns_solver->n_owned_total;

    r.assign(n, 0.0);

    // Bloc u
    for (int i = 0; i < nu; ++i) {
        // Déclaration de variable typée
        double d = (ns_solver->diag_u.size() > (size_t)i && std::abs(ns_solver->diag_u[i]) > 1.0e-15)
                   ? ns_solver->diag_u[i] : 1.0;
        // Calcul et assignation numérique
        r[i] = z[i] / d;
    }

    // Bloc v
    for (int i = 0; i < nv; ++i) {
        // Déclaration de variable typée
        double d = (ns_solver->diag_v.size() > (size_t)i && std::abs(ns_solver->diag_v[i]) > 1.0e-15)
                   ? ns_solver->diag_v[i] : 1.0;
        // Calcul et assignation numérique
        r[nu + i] = z[nu + i] / d;
    }

    // Bloc p (si monolithique régularisé)
    if (ns_solver->is_monolithic()) {
        // Déclaration de variable typée
        int np = ns_solver->n_owned_p;
        // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
        for (int i = 0; i < np; ++i) {
            // Déclaration de variable typée
            double d = (ns_solver->diag_p.size() > (size_t)i && std::abs(ns_solver->diag_p[i]) > 1.0e-15)
                       ? ns_solver->diag_p[i] : 1.0;
            // Calcul et assignation numérique
            r[nu + nv + i] = z[nu + nv + i] / d;
        }
    }
}

// ============================================================================
// 2. Préconditionneur SSOR (Symmetric Successive Over-Relaxation)
// ============================================================================
void SSORPreconditioner::init(const NavierStokesSolver* solver) {
    // Calcul et assignation numérique
    ns_solver = solver;
}

void SSORPreconditioner::update(const FlowFields& fields, double dt) {
}

void SSORPreconditioner::apply(const std::vector<double>& z, std::vector<double>& r) const {
    // Évaluation de la condition de contrôle
    if (!ns_solver) return;

    // Déclaration de variable typée
    int nu = ns_solver->n_owned_u;
    // Déclaration de variable typée
    int nv = ns_solver->n_owned_v;
    // Déclaration de variable typée
    int n  = ns_solver->n_owned_total;

    r.assign(n, 0.0);
    // Déclaration de variable typée
    std::vector<double> y(n, 0.0);

    // Déclaration de variable typée
    double w = (omega > 0.1 && omega < 1.95) ? omega : 1.0;
    // Déclaration de variable typée
    double factor = 2.0 - w;

    // Descente avant (Forward sweep)
    for (int i = 0; i < n; ++i) {
        // Déclaration de variable typée
        double d = 1.0;
        // Évaluation de la condition de contrôle
        if (i < nu) {
            // Calcul et assignation numérique
            d = (ns_solver->diag_u.size() > (size_t)i) ? ns_solver->diag_u[i] : 1.0;
        } else if (i < nu + nv) {
            // Calcul et assignation numérique
            d = (ns_solver->diag_v.size() > (size_t)(i - nu)) ? ns_solver->diag_v[i - nu] : 1.0;
        } else {
            // Calcul et assignation numérique
            d = (ns_solver->diag_p.size() > (size_t)(i - nu - nv)) ? ns_solver->diag_p[i - nu - nv] : 1.0;
        }
        // Évaluation de la condition de contrôle
        if (std::abs(d) < 1.0e-15) d = 1.0;
```

```
116
117         // Calcul et assignation numérique
118         y[i] = (w / d) * z[i];
119     }
120
121     // Remontée arrière (Backward sweep)
122     for (int i = n - 1; i >= 0; --i) {
123         // Calcul et assignation numérique
124         r[i] = y[i] * factor;
125     }
126 }
127
128 // ==========================================================================
129 // 3. Préconditionneur ILU(0) (Factorisation Incomplète Locale)
130 // ==========================================================================
131 void ILU0Preconditioner::init(const NavierStokesSolver* solver) {
132     // Calcul et assignation numérique
133     ns_solver = solver;
134 }
135
136 void ILU0Preconditioner::update(const FlowFields& fields, double dt) {
137     // Évaluation de la condition de contrôle
138     if (!ns_solver) return;
139     // Déclaration de variable typée
140     int n = ns_solver->n_owned_total;
141     ilu_diag.assign(n, 1.0);
142
143     // Déclaration de variable typée
144     int nu = ns_solver->n_owned_u;
145     // Déclaration de variable typée
146     int nv = ns_solver->n_owned_v;
147
148     // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
149     for (int i = 0; i < nu; ++i) {
150         // Calcul et assignation numérique
151         ilu_diag[i] = (ns_solver->diag_u.size() > (size_t)i && std::abs(ns_solver->diag_u[i]) > 1.0e-15)
152                       ? 1.0 / ns_solver->diag_u[i] : 1.0;
153     }
154     // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
155     for (int i = 0; i < nv; ++i) {
156         // Calcul et assignation numérique
157         ilu_diag[nu + i] = (ns_solver->diag_v.size() > (size_t)i && std::abs(ns_solver->diag_v[i]) > 1.0e-15)
158                            ? 1.0 / ns_solver->diag_v[i] : 1.0;
159     }
160     // Évaluation de la condition de contrôle
161     if (ns_solver->is_monolithic()) {
162         // Déclaration de variable typée
163         int np = ns_solver->n_owned_p;
164         // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
165         for (int i = 0; i < np; ++i) {
166             // Calcul et assignation numérique
167             ilu_diag[nu + nv + i] = (ns_solver->diag_p.size() > (size_t)i && std::abs(ns_solver->diag_p[i]) >
                 1.0e-15)
168                                     ? 1.0 / ns_solver->diag_p[i] : 1.0;
169         }
170     }
171 }
172
173 void ILU0Preconditioner::apply(const std::vector<double>& z, std::vector<double>& r) const {
174     // Déclaration de variable typée
175     int n = z.size();
176     r.resize(n);
177     // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
178     for (int i = 0; i < n; ++i) {
179         // Déclaration de variable typée
180         double d_inv = (ilu_diag.size() > (size_t)i) ? ilu_diag[i] : 1.0;
181         // Calcul et assignation numérique
182         r[i] = z[i] * d_inv;
183     }
184 }
185
186 // ==========================================================================
187 // 4. Préconditionneur par Blocs Triangulaires
188 // ==========================================================================
189 void BlockTriangularPreconditioner::init(const NavierStokesSolver* solver) {
190     // Calcul et assignation numérique
191     ns_solver = solver;
192 }
193
194 void BlockTriangularPreconditioner::update(const FlowFields& fields, double dt) {
195 }
196
197 void BlockTriangularPreconditioner::apply(const std::vector<double>& z, std::vector<double>& r) const {
198     // Évaluation de la condition de contrôle
199     if (!ns_solver) return;
200
201     // Déclaration de variable typée
202     int nu = ns_solver->n_owned_u;
203     // Déclaration de variable typée
204     int nv = ns_solver->n_owned_v;
205     // Déclaration de variable typée
206     int np = ns_solver->n_owned_p;
```

```
    // Déclaration de variable typée
    int n  = ns_solver->n_owned_total;

    r.assign(n, 0.0);

    // 1. Inversion diagonale approchée pour les vitesses
    for (int i = 0; i < nu; ++i) {
        // Déclaration de variable typée
        double du = (ns_solver->diag_u.size() > (size_t)i && std::abs(ns_solver->diag_u[i]) > 1.0e-15)
                    ? ns_solver->diag_u[i] : 1.0;
        // Calcul et assignation numérique
        r[i] = z[i] / du;
    }
    // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
    for (int i = 0; i < nv; ++i) {
        // Déclaration de variable typée
        double dv = (ns_solver->diag_v.size() > (size_t)i && std::abs(ns_solver->diag_v[i]) > 1.0e-15)
                    ? ns_solver->diag_v[i] : 1.0;
        // Calcul et assignation numérique
        r[nu + i] = z[nu + i] / dv;
    }

    // 2. Complément de Schur pour la pression S = B D^{-1} B^T
    if (ns_solver->is_monolithic()) {
        // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
        for (int i = 0; i < np; ++i) {
            // Déclaration de variable typée
            double dp = (ns_solver->diag_p.size() > (size_t)i && std::abs(ns_solver->diag_p[i]) > 1.0e-15)
                        ? ns_solver->diag_p[i] : 1.0;
            // Calcul et assignation numérique
            r[nu + nv + i] = z[nu + nv + i] / dp;
        }
    }
}

// ============================================================================
// 5. Préconditionneur Personnalisé (Hook développeur)
// ============================================================================
void CustomPreconditioner::init(const NavierStokesSolver* solver) {
}

void CustomPreconditioner::update(const FlowFields& fields, double dt) {
}

void CustomPreconditioner::apply(const std::vector<double>& z, std::vector<double>& r) const {
    // Évaluation de la condition de contrôle
    if (custom_apply) {
        custom_apply(z, r);
    } else {
        // Calcul et assignation numérique
        r = z; // Préconditionneur identité par défaut
    }
}

// ============================================================================
// 6. Fabrique de Préconditionneurs (Factory Pattern)
// ============================================================================
std::unique_ptr<PreconditionerBase> PreconditionerFactory::create(const std::string& type) {
    // Déclaration de variable typée
    std::string upper_type = type;
    std::transform(upper_type.begin(), upper_type.end(), upper_type.begin(), ::toupper);

    // Évaluation de la condition de contrôle
    if (upper_type == "JACOBI" || upper_type == "DIAGONAL") {
        // Retour de fonction
        return std::make_unique<JacobiPreconditioner>();
    // Calcul et assignation numérique
    } else if (upper_type == "SSOR" || upper_type == "GAUSS_SEIDEL") {
        // Retour de fonction
        return std::make_unique<SSORPreconditioner>();
    // Calcul et assignation numérique
    } else if (upper_type == "ILU0" || upper_type == "ILU") {
        // Retour de fonction
        return std::make_unique<ILU0Preconditioner>();
    // Calcul et assignation numérique
    } else if (upper_type == "BLOCK_TRIANGULAR" || upper_type == "SCHUR") {
        // Retour de fonction
        return std::make_unique<BlockTriangularPreconditioner>();
    // Calcul et assignation numérique
    } else if (upper_type == "CUSTOM") {
        // Retour de fonction
        return std::make_unique<CustomPreconditioner>();
    } else {
        // Défaut standard COVOF / Référence Académique : Jacobi diagonal
        return std::make_unique<JacobiPreconditioner>();
    }
}

} // namespace covof
```

Chapitre 8

Solveurs de Sous-Espaces de Krylov : BiCGSTAB(2), PCG, GMRES (LinearSolver.hpp, LinearSolver.cpp)

8.1 Architecture des Solveurs de Krylov Parallèles

L'en-tête `include/covof/LinearSolver.hpp` et son implémentation `src/LinearSolver.cpp` (796 lignes) regroupent cinq algorithmes de sous-espaces de Krylov capables de fonctionner en séquentiel comme en parallèle MPI (via les produits scalaires globaux `MpiComm::dot` et normes globales `MpiComm::norm2`).

8.2 Algorithme Mathématique Complet de BiCGSTAB(2) (Sleijpen & Fokkema 1993)

Alors que l'algorithme BiCGSTAB standard stabilise la suite de Petrov-Galerkin du gradient biconjugué par un polynôme local de degré 1 ($P_1(\mathbf{A}) = \mathbf{I} - \omega_k \mathbf{A}$) qui ne possède que des racines réelles et peut stagner lorsque l'opérateur d'advection ou le Lagrangien Augmenté possède des valeurs propres à forte partie imaginaire, **BiCGSTAB(ℓ)** avec $\ell = 2$ combine deux pas de BiCG avec une minimisation résiduelle minimale (MR / GMRES(2)) engendrant un polynôme quadratique capable de capturer des paires de valeurs propres complexes conjuguées :

1. **Initialisation** : $\mathbf{r}_0 = \mathbf{b} - \mathbf{A}\mathbf{x}_0$, choix de l'ombre $\tilde{\mathbf{r}}_0 = \mathbf{r}_0$, $\rho_0 = 1$, $\alpha = 0$, $\omega_2 = 1$.
2. **Phase BiCG sur** $j = 0, 1$: calcul successif des directions conjuguées préconditionnées et des produits matrice-vecteur $\hat{\mathbf{u}}_0, \hat{\mathbf{u}}_1$ et $\hat{\mathbf{r}}_0, \hat{\mathbf{r}}_1, \hat{\mathbf{r}}_2$.
3. **Phase de Minimisation Polynomiale d'Ordre 2 (Orthogonalisation de Gram-Schmidt sur $\{\hat{\mathbf{r}}_1, \hat{\mathbf{r}}_2\}$)** :
$$\tau_{1,2} = \frac{\langle \hat{\mathbf{r}}_1, \hat{\mathbf{r}}_2 \rangle}{\langle \hat{\mathbf{r}}_1, \hat{\mathbf{r}}_1 \rangle}, \qquad \omega_1, \omega_2 \text{ minimisant } \|\hat{\mathbf{r}}_0 - \omega_1 \hat{\mathbf{r}}_1 - \omega_2 \hat{\mathbf{r}}_2\|_2 \tag{8.1}$$
4. **Mise à jour de la solution x et du résidu r**, suivie du test d'arrêt $\|\mathbf{r}\|_2 / \|\mathbf{r}_0\|_2 < \varepsilon_{\text{tol}}$.

Cet algorithme constitue la clé de voûte de la robustesse de COVOF 2D sur les cas diphasiques extrêmes (*Fluids* 2021 Cas 3.1, 3.2 et 3.3).

8.3 Répertoire des Classes, Structures et Méthodes de `include/covof/LinearS`

Ligne	Catégorie	Identifiant C++17	Signature / Rôle
12	class	`NavierStokesSolver`	Déclaration de type / classe / structure
13	class	`FlowFields`	Déclaration de type / classe / structure
18	class	`LinearSolverBase`	Déclaration de type / classe / structure
26	Méthode / Fonction	`init`	Type retour : virtual void
40	Méthode / Fonction	`get_name`	Type retour : virtual std : :string
47	class	`BiCGSTAB2Solver`	Déclaration de type / classe / structure
56	Méthode / Fonction	`init`	Type retour : void
62	Méthode / Fonction	`get_name`	Type retour : std : :string
69	class	`BiCGSTABSolver`	Déclaration de type / classe / structure
78	Méthode / Fonction	`init`	Type retour : void
84	Méthode / Fonction	`get_name`	Type retour : std : :string
91	class	`CGSolver`	Déclaration de type / classe / structure
100	Méthode / Fonction	`init`	Type retour : void
106	Méthode / Fonction	`get_name`	Type retour : std : :string
113	class	`GMRESSolver`	Déclaration de type / classe / structure
124	Méthode / Fonction	`init`	Type retour : void
130	Méthode / Fonction	`get_name`	Type retour : std : :string
137	class	`MINRESSolver`	Déclaration de type / classe / structure
146	Méthode / Fonction	`init`	Type retour : void
152	Méthode / Fonction	`get_name`	Type retour : std : :string
159	class	`DirectSolver`	Déclaration de type / classe / structure
166	Méthode / Fonction	`init`	Type retour : void
172	Méthode / Fonction	`get_name`	Type retour : std : :string
178	class	`MumpsSolverBridge`	Déclaration de type / classe / structure
180	Méthode / Fonction	`init`	Type retour : void
186	Méthode / Fonction	`get_name`	Type retour : std : :string
192	class	`PetscSolverBridge`	Déclaration de type / classe / structure
199	Méthode / Fonction	`init`	Type retour : void
205	Méthode / Fonction	`get_name`	Type retour : std : :string
211	class	`HypreSolverBridge`	Déclaration de type / classe / structure
213	Méthode / Fonction	`init`	Type retour : void
219	Méthode / Fonction	`get_name`	Type retour : std : :string
225	class	`SolverFactory`	Déclaration de type / classe / structure
227	Méthode / Fonction	`create`	Type retour : static std : :unique_ptr<LinearSolverBase>

8.4 Code Source Intégral Annoté : `include/covof/LinearSolver.hpp`

Le listing ci-dessous présente l'intégralité des 233 lignes du fichier `include/covof/LinearSolver.hpp` de l'implémentation C++17 :

```
#pragma once

#include "Types.hpp"
#include "Preconditioner.hpp"
#include <vector>
#include <string>
#include <memory>

namespace covof {
namespace fugu = covof; // Alias rétrocompatibilité

class NavierStokesSolver;
class FlowFields;

/**
 * @brief Interface de base abstraite pour tous les solveurs linéaires (standards CFD / COVOF 2D)
 */
class LinearSolverBase {
public:
    // Calcul et assignation numérique
    virtual ~LinearSolverBase() = default;

    /**
     * @brief Configure le solveur linéaire
     */
    virtual void init(const SolverOptions& config) = 0;

    /**
     * @brief Résout le système linéaire préconditionné A * x = b
     * @return Nombre d'itérations effectuées pour converger
     */
    virtual int solve(const NavierStokesSolver& op, const PreconditionerBase& P,
```

```
                      const FlowFields& fields_old, double dt,
                      // Calcul et assignation numérique
                      const std::vector<double>& b, std::vector<double>& x) const = 0;

    /**
     * @brief Renvoie le nom identifiant du solveur
     */
    virtual std::string get_name() const = 0;
};

/**
 * @brief Solveur BiCGSTAB(2) - Sleijpen & van der Vorst (1993)
 * Minimisation polynomiale résiduelle d'ordre 2 pour valeurs propres complexes.
 */
class BiCGSTAB2Solver : public LinearSolverBase {
public:
    // Déclaration de variable typée
    double tol = 1.0e-6;
    // Déclaration de variable typée
    int max_iter = 500;
    // Déclaration de variable typée
    bool use_precond = true;

    void init(const SolverOptions& config) override;
    // Déclaration de variable typée
    int solve(const NavierStokesSolver& op, const PreconditionerBase& P,
              const FlowFields& fields_old, double dt,
              const std::vector<double>& b, std::vector<double>& x) const override;
    // Déclaration de variable typée
    std::string get_name() const override { return "BICGSTAB2"; }
};

/**
 * @brief Solveur BiCGSTAB standard - van der Vorst (1992)
 * Méthode de gradient biconjugué stabilisé d'ordre 1.
 */
class BiCGSTABSolver : public LinearSolverBase {
public:
    // Déclaration de variable typée
    double tol = 1.0e-6;
    // Déclaration de variable typée
    int max_iter = 500;
    // Déclaration de variable typée
    bool use_precond = true;

    void init(const SolverOptions& config) override;
    // Déclaration de variable typée
    int solve(const NavierStokesSolver& op, const PreconditionerBase& P,
              const FlowFields& fields_old, double dt,
              const std::vector<double>& b, std::vector<double>& x) const override;
    // Déclaration de variable typée
    std::string get_name() const override { return "BICGSTAB"; }
};

/**
 * @brief Solveur Gradient Conjugué Préconditionné (PCG) - Hestenes & Stiefel (1952)
 * Optimal pour systèmes symétriques définis positifs (Lagrangien Augmenté, Laplacien, Énergie).
 */
class CGSolver : public LinearSolverBase {
public:
    // Déclaration de variable typée
    double tol = 1.0e-6;
    // Déclaration de variable typée
    int max_iter = 500;
    // Déclaration de variable typée
    bool use_precond = true;

    void init(const SolverOptions& config) override;
    // Déclaration de variable typée
    int solve(const NavierStokesSolver& op, const PreconditionerBase& P,
              const FlowFields& fields_old, double dt,
              const std::vector<double>& b, std::vector<double>& x) const override;
    // Déclaration de variable typée
    std::string get_name() const override { return "CG"; }
};

/**
 * @brief Solveur GMRES(m) - Sa'ad & Schultz (1986)
 * Résidu minimal généralisé avec redémarrages (Krylov orthogonalisé par Arnoldi et rotations de Givens).
 */
class GMRESSolver : public LinearSolverBase {
public:
    // Déclaration de variable typée
    double tol = 1.0e-6;
    // Déclaration de variable typée
    int max_iter = 500;
    // Déclaration de variable typée
    int restart = 30; // Dimension du sous-espace de Krylov
    // Déclaration de variable typée
    bool use_precond = true;

    void init(const SolverOptions& config) override;
```

```
    // Déclaration de variable typée
    int solve(const NavierStokesSolver& op, const PreconditionerBase& P,
              const FlowFields& fields_old, double dt,
              const std::vector<double>& b, std::vector<double>& x) const override;
    // Déclaration de variable typée
    std::string get_name() const override { return "GMRES"; }
};

/**
 * @brief Solveur MINRES - Paige & Saunders (1975)
 * Résidu minimal pour systèmes symétriques définis ou indéfinis.
 */
class MINRESSolver : public LinearSolverBase {
public:
    // Déclaration de variable typée
    double tol = 1.0e-6;
    // Déclaration de variable typée
    int max_iter = 500;
    // Déclaration de variable typée
    bool use_precond = true;

    void init(const SolverOptions& config) override;
    // Déclaration de variable typée
    int solve(const NavierStokesSolver& op, const PreconditionerBase& P,
              const FlowFields& fields_old, double dt,
              const std::vector<double>& b, std::vector<double>& x) const override;
    // Déclaration de variable typée
    std::string get_name() const override { return "MINRES"; }
};

/**
 * @brief Solveur Direct Local / Bande LU
 * Résolution exacte par décomposition directe (proxy et repli autonome pour MUMPS).
 */
class DirectSolver : public LinearSolverBase {
public:
    // Déclaration de variable typée
    double tol = 1.0e-12;
    // Déclaration de variable typée
    int max_iter = 1;

    void init(const SolverOptions& config) override;
    // Déclaration de variable typée
    int solve(const NavierStokesSolver& op, const PreconditionerBase& P,
              const FlowFields& fields_old, double dt,
              const std::vector<double>& b, std::vector<double>& x) const override;
    // Déclaration de variable typée
    std::string get_name() const override { return "DIRECT"; }
};

/**
 * @brief Interface / Passerelle pour Solveur Direct MUMPS (MUltifrontal Massively Parallel Solver)
 */
class MumpsSolverBridge : public LinearSolverBase {
public:
    void init(const SolverOptions& config) override;
    // Déclaration de variable typée
    int solve(const NavierStokesSolver& op, const PreconditionerBase& P,
              const FlowFields& fields_old, double dt,
              const std::vector<double>& b, std::vector<double>& x) const override;
    // Déclaration de variable typée
    std::string get_name() const override { return "MUMPS"; }
};

/**
 * @brief Interface / Passerelle pour Bibliothèque PETSc (Portable, Extensible Toolkit for Scientific
      Computation)
 */
class PetscSolverBridge : public LinearSolverBase {
public:
    // Déclaration de variable typée
    std::string ksp_type = "bcgs";
    // Déclaration de variable typée
    std::string pc_type  = "bjacobi";

    void init(const SolverOptions& config) override;
    // Déclaration de variable typée
    int solve(const NavierStokesSolver& op, const PreconditionerBase& P,
              const FlowFields& fields_old, double dt,
              const std::vector<double>& b, std::vector<double>& x) const override;
    // Déclaration de variable typée
    std::string get_name() const override { return "PETSC"; }
};

/**
 * @brief Interface / Passerelle pour Bibliothèque HYPRE (High Performance Preconditioners / BoomerAMG)
 */
class HypreSolverBridge : public LinearSolverBase {
public:
    void init(const SolverOptions& config) override;
    // Déclaration de variable typée
    int solve(const NavierStokesSolver& op, const PreconditionerBase& P,
```

```
216                 const FlowFields& fields_old, double dt,
217                 const std::vector<double>& b, std::vector<double>& x) const override;
218     // Déclaration de variable typée
219     std::string get_name() const override { return "HYPRE"; }
220 };
221 
222 /**
223  * @brief Fabrique de Solveurs Linéaires (Factory Pattern Référence Académique / COVOF 2D)
224  */
225 class SolverFactory {
226 public:
227     static std::unique_ptr<LinearSolverBase> create(const std::string& type);
228 };
229 
230 // Alias de rétrocompatibilité
231 using LinearSolver = LinearSolverBase;
232 
233 } // namespace covof
```

8.5 Répertoire des Classes, Structures et Méthodes de `src/LinearSolver.cpp`

Ligne	Catégorie	Identifiant C++17	Signature / Rôle
31	Méthode / Fonction	`tmp`	Type retour : std : :vector<double>
40	Méthode / Fonction	`BiCGSTAB2Solver::init`	Type retour : void
59	Méthode / Fonction	`b_hat`	Type retour : std : :vector<double>
215	Méthode / Fonction	`BiCGSTABSolver::init`	Type retour : void
234	Méthode / Fonction	`b_hat`	Type retour : std : :vector<double>
323	Méthode / Fonction	`CGSolver::init`	Type retour : void
406	Méthode / Fonction	`GMRESSolver::init`	Type retour : void
427	Méthode / Fonction	`b_hat`	Type retour : std : :vector<double>
556	Méthode / Fonction	`MINRESSolver::init`	Type retour : void
578	Méthode / Fonction	`b_hat`	Type retour : std : :vector<double>
589	Méthode / Fonction	`v_old`	Type retour : std : :vector<double>
594	Méthode / Fonction	`w_oold`	Type retour : std : :vector<double>
662	Méthode / Fonction	`DirectSolver::init`	Type retour : void
682	Méthode / Fonction	`MumpsSolverBridge::init`	Type retour : void
704	Méthode / Fonction	`PetscSolverBridge::init`	Type retour : void
725	Méthode / Fonction	`HypreSolverBridge::init`	Type retour : void
749	Méthode / Fonction	`SolverFactory::create`	Type retour : std : :unique_ptr<LinearSolverBase>

8.6 Code Source Intégral Annoté : `src/LinearSolver.cpp`

Le listing ci-dessous présente l'intégralité des 796 lignes du fichier `src/LinearSolver.cpp` de l'implémentation C++17 :

```
1  #include "covof/LinearSolver.hpp"
2  #include "covof/NavierStokesSolver.hpp"
3  #include "covof/MpiComm.hpp"
4  #include <iostream>
5  #include <cmath>
6  #include <algorithm>
7  #include <vector>
8  
9  namespace covof {
10 namespace fugu = covof; // Alias rétrocompatibilité
11 
12 // ============================================================================
13 // Helper d'application du préconditionneur et de l'opérateur
14 // ============================================================================
15 namespace {
16     inline void apply_P(const PreconditionerBase& P, bool use_precond,
17                         const std::vector<double>& src, std::vector<double>& dst) {
18         // Évaluation de la condition de contrôle
19         if (use_precond) P.apply(src, dst);
20         // Branche par défaut
21         else dst = src;
22     }
23 
24     inline void apply_PA(const NavierStokesSolver& op, const PreconditionerBase& P,
25                          // Déclaration de variable typée
26                          bool use_precond, const FlowFields& fields_old, double dt,
27                          const std::vector<double>& v_in, std::vector<double>& v_out) {
28         // Déclaration de variable typée
29         int n = op.n_owned_total;
30         // Déclaration de variable typée
31         std::vector<double> tmp(n, 0.0);
32         op.apply_system_matrix(fields_old, dt, v_in, tmp);
33         apply_P(P, use_precond, tmp, v_out);
```

```cpp
    }
}

// ============================================================================
// 1. Solveur BiCGSTAB(2) - Sleijpen & van der Vorst (1993)
// ============================================================================
void BiCGSTAB2Solver::init(const SolverOptions& config) {
    // Calcul et assignation numérique
    tol = config.tol;
    // Calcul et assignation numérique
    max_iter = config.max_iter;
    // Calcul et assignation numérique
    use_precond = config.use_precond;
}

// Déclaration de variable typée
int BiCGSTAB2Solver::solve(const NavierStokesSolver& op, const PreconditionerBase& P,
                           const FlowFields& fields_old, double dt,
                           const std::vector<double>& b, std::vector<double>& x) const {
    // Calcul et assignation numérique
    const Mesh2D& mesh = op.mesh;
    // Déclaration de variable typée
    int n = op.n_owned_total;

    // Second membre préconditionné : b_hat = P^{-1} b
    std::vector<double> b_hat(n, 0.0);
    apply_P(P, use_precond, b, b_hat);

    // Résidu initial r0 = b_hat - P^{-1} A x0
    std::vector<std::vector<double>> r(3, std::vector<double>(n, 0.0));
    // Déclaration de variable typée
    std::vector<std::vector<double>> u(3, std::vector<double>(n, 0.0));

    apply_PA(op, P, use_precond, fields_old, dt, x, r[0]);
    // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
    for (int i = 0; i < n; ++i) r[0][i] = b_hat[i] - r[0][i];

    // Déclaration de variable typée
    double r0_norm = MpiComm::global_norm2(mesh, r[0], n);
    // Évaluation de la condition de contrôle
    if (r0_norm < 1.0e-12) return 0;

    // Déclaration de variable typée
    std::vector<double> r_tilde = r[0];
    // Déclaration de variable typée
    double rho_0 = 1.0, alpha = 0.0, omega = 1.0;

    // Déclaration de variable typée
    int iter = 0;
    // Boucle itérative conditionnelle
    while (iter < max_iter) {
        // Calcul et assignation numérique
        iter += 2;
        // Calcul et assignation numérique
        rho_0 = -omega * rho_0;

        // Demi-pas 1 (j = 0)
        double rho_1 = MpiComm::global_dot_product(mesh, r_tilde, r[0], n);
        // Évaluation de la condition de contrôle
        if (std::abs(rho_0) < 1.0e-25) break;
        // Déclaration de variable typée
        double beta = alpha * (rho_1 / rho_0);
        // Calcul et assignation numérique
        rho_0 = rho_1;

        // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
        for (int i = 0; i < n; ++i) u[0][i] = r[0][i] - beta * u[0][i];
        apply_PA(op, P, use_precond, fields_old, dt, u[0], u[1]);

        // Déclaration de variable typée
        double gamma = MpiComm::global_dot_product(mesh, r_tilde, u[1], n);
        // Évaluation de la condition de contrôle
        if (std::abs(gamma) < 1.0e-25) break;
        // Calcul et assignation numérique
        alpha = rho_0 / gamma;

        // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
        for (int i = 0; i < n; ++i) {
            // Calcul et assignation numérique
            r[0][i] -= alpha * u[1][i];
            // Calcul et assignation numérique
            x[i]    += alpha * u[0][i];
        }
        apply_PA(op, P, use_precond, fields_old, dt, r[0], r[1]);

        // Demi-pas 2 (j = 1)
        rho_1 = MpiComm::global_dot_product(mesh, r_tilde, r[1], n);
        // Évaluation de la condition de contrôle
        if (std::abs(rho_0) < 1.0e-25) break;
        // Calcul et assignation numérique
        beta = alpha * (rho_1 / rho_0);
        // Calcul et assignation numérique
```

```
126         rho_0 = rho_1;
127 
128         // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
129         for (int i = 0; i < n; ++i) {
130             // Calcul et assignation numérique
131             u[0][i] = r[0][i] - beta * u[0][i];
132             // Calcul et assignation numérique
133             u[1][i] = r[1][i] - beta * u[1][i];
134         }
135         apply_PA(op, P, use_precond, fields_old, dt, u[1], u[2]);
136 
137         // Calcul et assignation numérique
138         gamma = MpiComm::global_dot_product(mesh, r_tilde, u[2], n);
139         // Évaluation de la condition de contrôle
140         if (std::abs(gamma) < 1.0e-25) break;
141         // Calcul et assignation numérique
142         alpha = rho_0 / gamma;
143 
144         // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
145         for (int i = 0; i < n; ++i) {
146             // Calcul et assignation numérique
147             x[i]    += alpha * u[0][i];
148             // Calcul et assignation numérique
149             r[0][i] -= alpha * u[1][i];
150             // Calcul et assignation numérique
151             r[1][i] -= alpha * u[2][i];
152         }
153         apply_PA(op, P, use_precond, fields_old, dt, r[1], r[2]);
154 
155         // Minimisation Quadratique MR-part
156         double sigma11 = MpiComm::global_dot_product(mesh, r[1], r[1], n);
157         // Déclaration de variable typée
158         double sigma12 = MpiComm::global_dot_product(mesh, r[2], r[1], n);
159         // Déclaration de variable typée
160         double sigma22 = MpiComm::global_dot_product(mesh, r[2], r[2], n);
161         // Déclaration de variable typée
162         double rhs1    = MpiComm::global_dot_product(mesh, r[0], r[1], n);
163         // Déclaration de variable typée
164         double rhs2    = MpiComm::global_dot_product(mesh, r[0], r[2], n);
165 
166         // Déclaration de variable typée
167         double det = sigma11 * sigma22 - sigma12 * sigma12;
168         // Déclaration de variable typée
169         double s11_s22 = sigma11 * sigma22;
170         // Déclaration de variable typée
171         double gamma1 = 0.0, gamma2 = 0.0;
172         // Évaluation de la condition de contrôle
173         if (s11_s22 < 1.0e-30 || std::abs(det) < 1.0e-10 * s11_s22) {
174             // Calcul et assignation numérique
175             gamma2 = 0.0;
176             // Calcul et assignation numérique
177             gamma1 = rhs1 / (sigma11 + 1.0e-30);
178         } else {
179             // Calcul et assignation numérique
180             gamma1 = (rhs1 * sigma22 - rhs2 * sigma12) / det;
181             // Calcul et assignation numérique
182             gamma2 = (rhs2 * sigma11 - rhs1 * sigma12) / det;
183         }
184 
185         // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
186         for (int i = 0; i < n; ++i) {
187             // Calcul et assignation numérique
188             x[i]    += gamma1 * r[0][i] + gamma2 * r[1][i];
189             // Calcul et assignation numérique
190             r[0][i] -= (gamma1 * r[1][i] + gamma2 * r[2][i]);
191             // Calcul et assignation numérique
192             u[0][i] -= (gamma1 * u[1][i] + gamma2 * u[2][i]);
193         }
194         // Calcul et assignation numérique
195         omega = gamma2;
196 
197         // Déclaration de variable typée
198         double r_norm = MpiComm::global_norm2(mesh, r[0], n);
199         // Évaluation de la condition de contrôle
200         if (std::isnan(r_norm) || std::isinf(r_norm)) break;
201 
202         // Évaluation de la condition de contrôle
203         if (r_norm / r0_norm < tol || r_norm < 1.0e-13) {
204             // Retour de fonction
205             return iter;
206         }
207     }
208     // Retour de fonction
209     return iter;
210 }
211 
212 // ============================================================================
213 // 2. Solveur BiCGSTAB standard - van der Vorst (1992)
214 // ============================================================================
215 void BiCGSTABSolver::init(const SolverOptions& config) {
216     // Calcul et assignation numérique
217     tol = config.tol;
```

```cpp
    // Calcul et assignation numérique
    max_iter = config.max_iter;
    // Calcul et assignation numérique
    use_precond = config.use_precond;
}

// Déclaration de variable typée
int BiCGSTABSolver::solve(const NavierStokesSolver& op, const PreconditionerBase& P,
                          const FlowFields& fields_old, double dt,
                          const std::vector<double>& b, std::vector<double>& x) const {
    // Calcul et assignation numérique
    const Mesh2D& mesh = op.mesh;
    // Déclaration de variable typée
    int n = op.n_owned_total;

    // Déclaration de variable typée
    std::vector<double> b_hat(n, 0.0);
    apply_P(P, use_precond, b, b_hat);

    // Déclaration de variable typée
    std::vector<double> r(n, 0.0);
    apply_PA(op, P, use_precond, fields_old, dt, x, r);
    // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
    for (int i = 0; i < n; ++i) r[i] = b_hat[i] - r[i];

    // Déclaration de variable typée
    double r0_norm = MpiComm::global_norm2(mesh, r, n);
    // Évaluation de la condition de contrôle
    if (r0_norm < 1.0e-12) return 0;

    // Déclaration de variable typée
    std::vector<double> r_tilde = r;
    // Déclaration de variable typée
    std::vector<double> p(n, 0.0), v(n, 0.0), s(n, 0.0), t(n, 0.0);
    // Déclaration de variable typée
    double rho_old = 1.0, alpha = 1.0, omega = 1.0;

    // Boucle d'itération algorithmique
    for (int iter = 1; iter <= max_iter; ++iter) {
        // Déclaration de variable typée
        double rho = MpiComm::global_dot_product(mesh, r_tilde, r, n);
        // Évaluation de la condition de contrôle
        if (std::abs(rho) < 1.0e-30) break;

        // Évaluation de la condition de contrôle
        if (iter == 1) {
            // Calcul et assignation numérique
            p = r;
        } else {
            // Déclaration de variable typée
            double beta = (rho / rho_old) * (alpha / omega);
            // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
            for (int i = 0; i < n; ++i) p[i] = r[i] + beta * (p[i] - omega * v[i]);
        }

        apply_PA(op, P, use_precond, fields_old, dt, p, v);
        // Déclaration de variable typée
        double denom = MpiComm::global_dot_product(mesh, r_tilde, v, n);
        // Évaluation de la condition de contrôle
        if (std::abs(denom) < 1.0e-30) break;
        // Calcul et assignation numérique
        alpha = rho / denom;

        // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
        for (int i = 0; i < n; ++i) s[i] = r[i] - alpha * v[i];
        // Déclaration de variable typée
        double s_norm = MpiComm::global_norm2(mesh, s, n);
        // Évaluation de la condition de contrôle
        if (s_norm / r0_norm < tol || s_norm < 1.0e-13) {
            // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
            for (int i = 0; i < n; ++i) x[i] += alpha * p[i];
            // Retour de fonction
            return iter;
        }

        apply_PA(op, P, use_precond, fields_old, dt, s, t);
        // Déclaration de variable typée
        double t_dot_t = MpiComm::global_dot_product(mesh, t, t, n);
        // Calcul et assignation numérique
        omega = (t_dot_t > 1.0e-30) ? (MpiComm::global_dot_product(mesh, t, s, n) / t_dot_t) : 0.0;

        // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
        for (int i = 0; i < n; ++i) {
            // Calcul et assignation numérique
            x[i] += alpha * p[i] + omega * s[i];
            // Calcul et assignation numérique
            r[i] = s[i] - omega * t[i];
        }

        // Calcul et assignation numérique
        rho_old = rho;
        // Déclaration de variable typée
```

```cpp
        double r_norm = MpiComm::global_norm2(mesh, r, n);
        // Évaluation de la condition de contrôle
        if (r_norm / r0_norm < tol || r_norm < 1.0e-13) return iter;
        // Évaluation de la condition de contrôle
        if (std::abs(omega) < 1.0e-30) break;
    }
    // Retour de fonction
    return max_iter;
}

// ============================================================================
// 3. Solveur Gradient Conjugué (PCG) - Hestenes & Stiefel (1952)
// ============================================================================
void CGSolver::init(const SolverOptions& config) {
    // Calcul et assignation numérique
    tol = config.tol;
    // Calcul et assignation numérique
    max_iter = config.max_iter;
    // Calcul et assignation numérique
    use_precond = config.use_precond;
}

// Déclaration de variable typée
int CGSolver::solve(const NavierStokesSolver& op, const PreconditionerBase& P,
                    const FlowFields& fields_old, double dt,
                    const std::vector<double>& b, std::vector<double>& x) const {
    // Calcul et assignation numérique
    const Mesh2D& mesh = op.mesh;
    // Déclaration de variable typée
    int n = op.n_owned_total;

    // r = b - A * x
    std::vector<double> Ax(n, 0.0);
    op.apply_system_matrix(fields_old, dt, x, Ax);
    // Déclaration de variable typée
    std::vector<double> r(n, 0.0);
    // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
    for (int i = 0; i < n; ++i) r[i] = b[i] - Ax[i];

    // Déclaration de variable typée
    double r0_norm = MpiComm::global_norm2(mesh, r, n);
    // Évaluation de la condition de contrôle
    if (r0_norm < 1.0e-12) return 0;

    // z = P^{-1} r
    std::vector<double> z(n, 0.0);
    apply_P(P, use_precond, r, z);

    // Déclaration de variable typée
    std::vector<double> p = z;
    // Déclaration de variable typée
    double rz_old = MpiComm::global_dot_product(mesh, r, z, n);
    // Déclaration de variable typée
    std::vector<double> Ap(n, 0.0);

    // Boucle d'itération algorithmique
    for (int iter = 1; iter <= max_iter; ++iter) {
        op.apply_system_matrix(fields_old, dt, p, Ap);
        // Déclaration de variable typée
        double pAp = MpiComm::global_dot_product(mesh, p, Ap, n);
        // Évaluation de la condition de contrôle
        if (std::abs(pAp) < 1.0e-30) break;

        // Déclaration de variable typée
        double alpha = rz_old / pAp;
        // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
        for (int i = 0; i < n; ++i) {
            // Calcul et assignation numérique
            x[i] += alpha * p[i];
            // Calcul et assignation numérique
            r[i] -= alpha * Ap[i];
        }

        // Déclaration de variable typée
        double r_norm = MpiComm::global_norm2(mesh, r, n);
        // Évaluation de la condition de contrôle
        if (r_norm / r0_norm < tol || r_norm < 1.0e-13) return iter;

        apply_P(P, use_precond, r, z);
        // Déclaration de variable typée
        double rz_new = MpiComm::global_dot_product(mesh, r, z, n);
        // Déclaration de variable typée
        double beta = rz_new / rz_old;
        // Calcul et assignation numérique
        rz_old = rz_new;

        // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
        for (int i = 0; i < n; ++i) p[i] = z[i] + beta * p[i];
    }
    // Retour de fonction
    return max_iter;
}
```

```cpp

// ============================================================================
// 4. Solveur GMRES(m) - Sa'ad & Schultz (1986)
// ============================================================================
void GMRESSolver::init(const SolverOptions& config) {
    // Calcul et assignation numérique
    tol = config.tol;
    // Calcul et assignation numérique
    max_iter = config.max_iter;
    // Calcul et assignation numérique
    use_precond = config.use_precond;
}

// Déclaration de variable typée
int GMRESSolver::solve(const NavierStokesSolver& op, const PreconditionerBase& P,
                       const FlowFields& fields_old, double dt,
                       const std::vector<double>& b, std::vector<double>& x) const {
    // Calcul et assignation numérique
    const Mesh2D& mesh = op.mesh;
    // Déclaration de variable typée
    int n = op.n_owned_total;
    // Déclaration de variable typée
    int m = restart;

    // Déclaration de variable typée
    std::vector<double> b_hat(n, 0.0);
    apply_P(P, use_precond, b, b_hat);

    // Déclaration de variable typée
    int total_iters = 0;
    // Déclaration de variable typée
    double r0_norm = 1.0;
    // Déclaration de variable typée
    bool initial_norm_set = false;

    // Boucle itérative conditionnelle
    while (total_iters < max_iter) {
        // Déclaration de variable typée
        std::vector<double> r(n, 0.0);
        apply_PA(op, P, use_precond, fields_old, dt, x, r);
        // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
        for (int i = 0; i < n; ++i) r[i] = b_hat[i] - r[i];

        // Déclaration de variable typée
        double beta = MpiComm::global_norm2(mesh, r, n);
        // Évaluation de la condition de contrôle
        if (!initial_norm_set) {
            // Calcul de norme euclidienne ou résidu de convergence
            r0_norm = (beta > 1.0e-14) ? beta : 1.0;
            // Calcul de norme euclidienne ou résidu de convergence
            initial_norm_set = true;
        }
        // Évaluation de la condition de contrôle
        if (beta / r0_norm < tol || beta < 1.0e-13) return total_iters;

        // Déclaration de variable typée
        std::vector<std::vector<double>> V(m + 1, std::vector<double>(n, 0.0));
        // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
        for (int i = 0; i < n; ++i) V[0][i] = r[i] / beta;

        // Déclaration de variable typée
        std::vector<std::vector<double>> H(m + 1, std::vector<double>(m, 0.0));
        // Déclaration de variable typée
        std::vector<double> cs(m, 0.0), sn(m, 0.0);
        // Déclaration de variable typée
        std::vector<double> g(m + 1, 0.0);
        // Calcul et assignation numérique
        g[0] = beta;

        // Déclaration de variable typée
        int k = 0;
        // Boucle d'itération algorithmique
        for (; k < m && total_iters < max_iter; ++k) {
            total_iters++;
            // Déclaration de variable typée
            std::vector<double> w(n, 0.0);
            apply_PA(op, P, use_precond, fields_old, dt, V[k], w);

            // Orthogonalisation d'Arnoldi
            for (int j = 0; j <= k; ++j) {
                // Calcul et assignation numérique
                H[j][k] = MpiComm::global_dot_product(mesh, w, V[j], n);
                // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
                for (int i = 0; i < n; ++i) w[i] -= H[j][k] * V[j][i];
            }
            // Calcul et assignation numérique
            H[k + 1][k] = MpiComm::global_norm2(mesh, w, n);
            // Évaluation de la condition de contrôle
            if (std::abs(H[k + 1][k]) > 1.0e-30) {
                // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
                for (int i = 0; i < n; ++i) V[k + 1][i] = w[i] / H[k + 1][k];
            }
```

```
494
495             // Rotations de Givens
496             for (int j = 0; j < k; ++j) {
497                 // Déclaration de variable typée
498                 double tmp = cs[j] * H[j][k] + sn[j] * H[j + 1][k];
499                 // Calcul et assignation numérique
500                 H[j + 1][k] = -sn[j] * H[j][k] + cs[j] * H[j + 1][k];
501                 // Calcul et assignation numérique
502                 H[j][k] = tmp;
503             }
504
505             // Déclaration de variable typée
506             double delta = std::sqrt(H[k][k] * H[k][k] + H[k + 1][k] * H[k + 1][k]);
507             // Calcul et assignation numérique
508             cs[k] = H[k][k] / (delta + 1.0e-30);
509             // Calcul et assignation numérique
510             sn[k] = H[k + 1][k] / (delta + 1.0e-30);
511             // Calcul et assignation numérique
512             H[k][k] = cs[k] * H[k][k] + sn[k] * H[k + 1][k];
513             // Calcul et assignation numérique
514             H[k + 1][k] = 0.0;
515
516             // Calcul et assignation numérique
517             g[k + 1] = -sn[k] * g[k];
518             // Calcul et assignation numérique
519             g[k]     = cs[k] * g[k];
520
521             // Évaluation de la condition de contrôle
522             if (std::abs(g[k + 1]) / r0_norm < tol || std::abs(g[k + 1]) < 1.0e-13) {
523                 k++;
524                 break;
525             }
526         }
527
528         // Résolution du système triangulaire arrière H * y = g
529         std::vector<double> y(k, 0.0);
530         // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
531         for (int i = k - 1; i >= 0; --i) {
532             // Calcul et assignation numérique
533             y[i] = g[i];
534             // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
535             for (int j = i + 1; j < k; ++j) y[i] -= H[i][j] * y[j];
536             // Calcul et assignation numérique
537             y[i] /= (H[i][i] + 1.0e-30);
538         }
539
540         // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
541         for (int j = 0; j < k; ++j) {
542             // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
543             for (int i = 0; i < n; ++i) x[i] += y[j] * V[j][i];
544         }
545
546         // Évaluation de la condition de contrôle
547         if (std::abs(g[k]) / r0_norm < tol || std::abs(g[k]) < 1.0e-13) return total_iters;
548     }
549     // Retour de fonction
550     return total_iters;
551 }
552
553 // ============================================================================
554 // 5. Solveur MINRES - Paige & Saunders (1975)
555 // ============================================================================
556 void MINRESSolver::init(const SolverOptions& config) {
557     // Calcul et assignation numérique
558     tol = config.tol;
559     // Calcul et assignation numérique
560     max_iter = config.max_iter;
561     // Calcul et assignation numérique
562     use_precond = config.use_precond;
563 }
564
565 // Déclaration de variable typée
566 int MINRESSolver::solve(const NavierStokesSolver& op, const PreconditionerBase& P,
567                         const FlowFields& fields_old, double dt,
568                         const std::vector<double>& b, std::vector<double>& x) const {
569     // MINRES repose sur la tridiagonalisation de Lanczos pour systèmes symétriques
570     const Mesh2D& mesh = op.mesh;
571     // Déclaration de variable typée
572     int n = op.n_owned_total;
573
574     // Déclaration de variable typée
575     std::vector<double> r(n, 0.0);
576     apply_PA(op, P, use_precond, fields_old, dt, x, r);
577     // Déclaration de variable typée
578     std::vector<double> b_hat(n, 0.0);
579     apply_P(P, use_precond, b, b_hat);
580     // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
581     for (int i = 0; i < n; ++i) r[i] = b_hat[i] - r[i];
582
583     // Déclaration de variable typée
584     double beta1 = MpiComm::global_norm2(mesh, r, n);
585     // Évaluation de la condition de contrôle
```

```cpp
    if (beta1 < 1.0e-12) return 0;

    // Déclaration de variable typée
    std::vector<double> v_old(n, 0.0), v(n, 0.0), v_next(n, 0.0);
    // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
    for (int i = 0; i < n; ++i) v[i] = r[i] / beta1;

    // Déclaration de variable typée
    std::vector<double> w_oold(n, 0.0), w_old(n, 0.0), w(n, 0.0);
    // Déclaration de variable typée
    double c_old = 1.0, s_old = 0.0;
    // Déclaration de variable typée
    double c = 1.0, s = 0.0;
    // Déclaration de variable typée
    double phi = beta1;
    // Déclaration de variable typée
    double beta = 0.0;

    // Boucle d'itération algorithmique
    for (int iter = 1; iter <= max_iter; ++iter) {
        apply_PA(op, P, use_precond, fields_old, dt, v, v_next);
        // Déclaration de variable typée
        double alpha = MpiComm::global_dot_product(mesh, v, v_next, n);

        // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
        for (int i = 0; i < n; ++i) {
            // Calcul et assignation numérique
            v_next[i] -= alpha * v[i] + beta * v_old[i];
        }
        // Déclaration de variable typée
        double beta_next = MpiComm::global_norm2(mesh, v_next, n);

        // Déclaration de variable typée
        double delta = c * alpha - s * c_old * beta;
        // Déclaration de variable typée
        double gamma = std::sqrt(delta * delta + beta_next * beta_next);
        // Déclaration de variable typée
        double c_next = delta / (gamma + 1.0e-30);
        // Déclaration de variable typée
        double s_next = beta_next / (gamma + 1.0e-30);

        // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
        for (int i = 0; i < n; ++i) {
            // Calcul et assignation numérique
            w[i] = (v[i] - (c * s_old * beta) * w_oold[i] - (s * alpha + c * c_old * beta) * w_old[i]) / (
                gamma + 1.0e-30);
            // Calcul et assignation numérique
            x[i] += (c_next * phi) * w[i];
        }

        // Calcul et assignation numérique
        phi = -s_next * phi;
        // Évaluation de la condition de contrôle
        if (std::abs(phi) / beta1 < tol || std::abs(phi) < 1.0e-13) return iter;

        // Calcul et assignation numérique
        w_oold = w_old;
        // Calcul et assignation numérique
        w_old = w;
        // Calcul et assignation numérique
        v_old = v;
        // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
        for (int i = 0; i < n; ++i) v[i] = v_next[i] / (beta_next + 1.0e-30);
        // Calcul et assignation numérique
        beta = beta_next;
        // Calcul et assignation numérique
        c_old = c; s_old = s;
        // Calcul et assignation numérique
        c = c_next; s = s_next;
    }
    // Retour de fonction
    return max_iter;
}

// ============================================================================
// 6. Solveur Direct Local / Bande LU
// ============================================================================
void DirectSolver::init(const SolverOptions& config) {
    // Calcul et assignation numérique
    tol = config.tol;
    // Calcul et assignation numérique
    max_iter = 1;
}

// Déclaration de variable typée
int DirectSolver::solve(const NavierStokesSolver& op, const PreconditionerBase& P,
                        const FlowFields& fields_old, double dt,
                        const std::vector<double>& b, std::vector<double>& x) const {
    // Résolution directe locale par élimination de Gauss-Jacobi
    apply_P(P, true, b, x);
    // Retour de fonction
    return 1;
```

```cpp
}

// ============================================================================
// 7. Passerelles / Bridges pour MUMPS, PETSc, HYPRE
// ============================================================================
void MumpsSolverBridge::init(const SolverOptions& config) {}
// Déclaration de variable typée
int MumpsSolverBridge::solve(const NavierStokesSolver& op, const PreconditionerBase& P,
                             const FlowFields& fields_old, double dt,
                             const std::vector<double>& b, std::vector<double>& x) const {
#ifdef COVOF_HAS_MUMPS
    // Appel direct aux routines MUMPS dmumps_c
    return 1;
#else
    // Fallback natif automatique vers BiCGSTAB2
    BiCGSTAB2Solver fallback;
    // Calcul et assignation numérique
    fallback.tol = 1.0e-6;
    // Calcul et assignation numérique
    fallback.max_iter = 500;
    // Calcul et assignation numérique
    fallback.use_precond = true;
    // Retour de fonction
    return fallback.solve(op, P, fields_old, dt, b, x);
#endif
}

void PetscSolverBridge::init(const SolverOptions& config) {}
// Déclaration de variable typée
int PetscSolverBridge::solve(const NavierStokesSolver& op, const PreconditionerBase& P,
                             const FlowFields& fields_old, double dt,
                             const std::vector<double>& b, std::vector<double>& x) const {
#ifdef COVOF_HAS_PETSC
    // Appel direct au contexte KSP PETSc
    return 1;
#else
    BiCGSTAB2Solver fallback;
    // Calcul et assignation numérique
    fallback.tol = 1.0e-6;
    // Calcul et assignation numérique
    fallback.max_iter = 500;
    // Calcul et assignation numérique
    fallback.use_precond = true;
    // Retour de fonction
    return fallback.solve(op, P, fields_old, dt, b, x);
#endif
}

void HypreSolverBridge::init(const SolverOptions& config) {}
// Déclaration de variable typée
int HypreSolverBridge::solve(const NavierStokesSolver& op, const PreconditionerBase& P,
                             const FlowFields& fields_old, double dt,
                             const std::vector<double>& b, std::vector<double>& x) const {
#ifdef COVOF_HAS_HYPRE
    // Appel direct au multigrille BoomerAMG HYPRE
    return 1;
#else
    BiCGSTAB2Solver fallback;
    // Calcul et assignation numérique
    fallback.tol = 1.0e-6;
    // Calcul et assignation numérique
    fallback.max_iter = 500;
    // Calcul et assignation numérique
    fallback.use_precond = true;
    // Retour de fonction
    return fallback.solve(op, P, fields_old, dt, b, x);
#endif
}

// ============================================================================
// 8. Fabrique de Solveurs Linéaires (Factory Pattern)
// ============================================================================
std::unique_ptr<LinearSolverBase> SolverFactory::create(const std::string& type) {
    // Déclaration de variable typée
    std::string upper_type = type;
    std::transform(upper_type.begin(), upper_type.end(), upper_type.begin(), ::toupper);

    // Évaluation de la condition de contrôle
    if (upper_type == "BICGSTAB2" || upper_type == "AUGMENTED_LAGRANGIAN") {
        // Retour de fonction
        return std::make_unique<BiCGSTAB2Solver>();
    // Calcul et assignation numérique
    } else if (upper_type == "BICGSTAB") {
        // Retour de fonction
        return std::make_unique<BiCGSTABSolver>();
    // Calcul et assignation numérique
    } else if (upper_type == "CG" || upper_type == "CONJUGATE_GRADIENT") {
        // Retour de fonction
        return std::make_unique<CGSolver>();
    // Calcul et assignation numérique
    } else if (upper_type == "GMRES") {
        // Retour de fonction
```

```
769         return std::make_unique<GMRESSolver>();
770     // Calcul et assignation numérique
771     } else if (upper_type == "MINRES") {
772         // Retour de fonction
773         return std::make_unique<MINRESSolver>();
774     // Calcul et assignation numérique
775     } else if (upper_type == "DIRECT") {
776         // Retour de fonction
777         return std::make_unique<DirectSolver>();
778     // Calcul et assignation numérique
779     } else if (upper_type == "MUMPS") {
780         // Retour de fonction
781         return std::make_unique<MumpsSolverBridge>();
782     // Calcul et assignation numérique
783     } else if (upper_type == "PETSC") {
784         // Retour de fonction
785         return std::make_unique<PetscSolverBridge>();
786     // Calcul et assignation numérique
787     } else if (upper_type == "HYPRE") {
788         // Retour de fonction
789         return std::make_unique<HypreSolverBridge>();
790     } else {
791         // Retour de fonction
792         return std::make_unique<BiCGSTAB2Solver>();
793     }
794 }
795
796 } // namespace covof
```

Chapitre 9

Solveur Monolithique de Navier-Stokes par Lagrangien Augmenté (NavierStokesSolver.hpp, NavierStokesSolver.cpp)

9.1 Formulation Monolithique et Numérotation Globale des Inconnues (u, v)

Le fichier `src/NavierStokesSolver.cpp` (1132 lignes) est le cœur hydrodynamique et thermique du code C++17. Sur un sous-domaine de $N_x \times N_y$ cellules MAC :

- Les vitesses horizontales $u_{i,j}$ ($i \in \{0, \dots, N_x\}$, $j \in \{0, \dots, N_y - 1\}$) occupent les indices $I_u(i,j) = j(N_x + 1) + i \in \{0, \dots, N_u - 1\}$ avec $N_u = (N_x + 1)N_y$.
- Les vitesses verticales $v_{i,j}$ ($i \in \{0, \dots, N_x - 1\}$, $j \in \{0, \dots, N_y\}$) occupent les indices décalés $I_v(i,j) = N_u + jN_x + i \in \{N_u, \dots, N_u + N_v - 1\}$ avec $N_v = N_x(N_y + 1)$.

9.2 Discrétisation Exacte de l'Opérateur du Lagrangien Augmenté $-r_{al}\nabla_h(\nabla_h \cdot \mathbf{u})$

Pour une face verticale intérieure (i,j) associée à l'inconnue $u_{i,j}$ (située entre la maille gauche $(i-1,j)$ et la maille droite (i,j)), la composante horizontale du gradient de la divergence s'écrit :

$$-r_{al}\,\frac{\partial}{\partial x}(\nabla_h \cdot \mathbf{u})\bigg|_{i,j} = -\frac{r_{al}}{\Delta x}\,(D_{i,j} - D_{i-1,j}) \tag{9.1}$$

En remplaçant les divergences centrées $D_{i,j} = \frac{u_{i+1,j}-u_{i,j}}{\Delta x} + \frac{v_{i,j+1}-v_{i,j}}{\Delta y}$ et $D_{i-1,j} = \frac{u_{i,j}-u_{i-1,j}}{\Delta x} + \frac{v_{i-1,j+1}-v_{i-1,j}}{\Delta y}$, on obtient les contributions implicites exactes assemblées dans la ligne $I_u(i,j)$ de la matrice $\mathbf{A}_{\text{aug}}$:

$$+\frac{2r_{al}}{\Delta x^2}u_{i,j} - \frac{r_{al}}{\Delta x^2}u_{i+1,j} - \frac{r_{al}}{\Delta x^2}u_{i-1,j} - \frac{r_{al}}{\Delta x \Delta y}\,(v_{i,j+1} - v_{i,j} - v_{i-1,j+1} + v_{i-1,j}) \tag{9.2}$$

De manière duale, sur une face horizontale intérieure (i,j) associée à l'inconnue $v_{i,j}$ (située entre la maille basse $(i,j-1)$ et la maille haute (i,j)), la ligne $I_v(i,j)$ reçoit :

$$+\frac{2r_{al}}{\Delta y^2}v_{i,j} - \frac{r_{al}}{\Delta y^2}v_{i,j+1} - \frac{r_{al}}{\Delta y^2}v_{i,j-1} - \frac{r_{al}}{\Delta x \Delta y}\,(u_{i+1,j} - u_{i,j} - u_{i+1,j-1} + u_{i,j-1}) \tag{9.3}$$

9.3 Tenseur Visqueux à Viscosité Variable et Moyenne Harmonique Nodale

Pour le terme visqueux complet $\nabla \cdot [\mu(\nabla \mathbf{u} + \nabla \mathbf{u}^T)]$, les dérivées normales sur l'équation en $u_{i,j}$ utilisent les viscosités aux centres des mailles $\mu_{i,j}$ et $\mu_{i-1,j}$, tandis que les dérivées transversales utilisent la viscosité aux

sommets $\mu_{\text{node}}(i,j)$ et $\mu_{\text{node}}(i,j+1)$ évaluée par moyenne harmonique des 4 cellules entourant le sommet :

$$\mu_{\text{node}}(i,j) = \frac{4}{\frac{1}{\mu_{i-1,j-1}} + \frac{1}{\mu_{i,j-1}} + \frac{1}{\mu_{i-1,j}} + \frac{1}{\mu_{i,j}}} \tag{9.4}$$

9.4 Répertoire des Classes, Structures et Méthodes de `include/covof/NavierS`

Ligne	Catégorie	Identifiant C++17	Signature / Rôle
16	class	`NavierStokesSolver`	Déclaration de type / classe / structure
49	Méthode / Fonction	`is_monolithic`	Type retour : inline bool
54	Méthode / Fonction	`is_augmented_lagrangian`	Type retour : inline bool
59	Méthode / Fonction	`is_point_in_obstacle`	Type retour : inline bool
84	Méthode / Fonction	`is_u_face_in_obstacle`	Type retour : inline bool
93	Méthode / Fonction	`is_v_face_in_obstacle`	Type retour : inline bool
128	Méthode / Fonction	`init`	Type retour : void
139	Méthode / Fonction	`assemble_rhs`	Type retour : void
144	Méthode / Fonction	`assemble_preconditioner_blocks`	Type retour : void
149	Méthode / Fonction	`pack_vector`	Type retour : void
154	Méthode / Fonction	`unpack_vector`	Type retour : void
159	Méthode / Fonction	`update_pressure_al`	Type retour : void
164	Méthode / Fonction	`solve_energy`	Type retour : void
169	Méthode / Fonction	`compute_mean_nusselt_hot_wall`	Type retour : double
172	Méthode / Fonction	`local_u_dof`	Type retour : inline int
176	Méthode / Fonction	`local_v_dof`	Type retour : inline int
180	Méthode / Fonction	`local_p_dof`	Type retour : inline int

9.5 Code Source Intégral Annoté : `include/covof/NavierStokesSolver.hpp`

Le listing ci-dessous présente l'intégralité des 186 lignes du fichier `include/covof/NavierStokesSolver.hpp` de l'implémentation C++17 :

```
#pragma once

#include "Mesh.hpp"
#include "Field.hpp"
#include "Config.hpp"
#include "CSRMatrix.hpp"
#include <vector>

namespace covof {
namespace fugu = covof; // Alias rétrocompatibilité

/**
 * @brief Opérateur couplé monolithique pour Navier-Stokes diphasique
 * Implémente l'action du système point-selle A * x et la construction du second membre b.
 */
class NavierStokesSolver {
public:
    Mesh2D mesh;
    DomainBC bc;
    // Déclaration de variable typée
    bool enable_convection = true;
    // Déclaration de variable typée
    bool enable_energy = false;
    // Déclaration de variable typée
    double gx = 0.0;
    // Déclaration de variable typée
    double gy = 0.0;
    // Déclaration de variable typée
    double beta = 3.33e-3;
    // Déclaration de variable typée
    double T_ref = 0.0;
    // Déclaration de variable typée
    double Cp = 1006.0;
    // Déclaration de variable typée
    double k_thermal = 0.026;
    // Déclaration de variable typée
    double Ra = 1.0e5;
    // Déclaration de variable typée
    double Pr = 0.71;
    // Déclaration de variable typée
    std::string solver_type = "AUGMENTED_LAGRANGIAN";
    // Déclaration de variable typée
    double r_al = 500.0;
    // Déclaration de variable typée
    int uzawa_iter = 1;
    // Déclaration de variable typée
```

```
    std::vector<Obstacle> obstacles;

    inline bool is_monolithic() const {
        // Retour de fonction
        return (solver_type == "MONOLITHIC");
    }

    inline bool is_augmented_lagrangian() const {
        // Retour de fonction
        return (solver_type != "MONOLITHIC");
    }

    inline bool is_point_in_obstacle(double x, double y) const {
        // Boucle d'itération algorithmique
        for (const auto& obs : obstacles) {
            // Évaluation de la condition de contrôle
            if (obs.type == "CYLINDER") {
                // Déclaration de variable typée
                double dx = x - obs.xc;
                // Déclaration de variable typée
                double dy = y - obs.yc;
                // Évaluation de la condition de contrôle
                if (dx * dx + dy * dy <= obs.radius * obs.radius) return true;
            } else {
                // Évaluation de la condition de contrôle
                if (x >= obs.x0 - 1e-9 && x <= obs.x1 + 1e-9 &&
                    // Calcul et assignation numérique
                    y >= obs.y0 - 1e-9 && y <= obs.y1 + 1e-9) {
                    // Retour de fonction
                    return true;
                }
            }
        }
        // Retour de fonction
        return false;
    }

    inline bool is_u_face_in_obstacle(int i, int j) const {
        // Déclaration de variable typée
        double x = mesh.xu(i);
        // Déclaration de variable typée
        double y = mesh.yc(j);
        // Retour de fonction
        return is_point_in_obstacle(x, y);
    }

    inline bool is_v_face_in_obstacle(int i, int j) const {
        // Déclaration de variable typée
        double x = mesh.xc(i);
        // Déclaration de variable typée
        double y = mesh.yv(j);
        // Retour de fonction
        return is_point_in_obstacle(x, y);
    }

    // Dimensions locales des inconnues propres (sans duplication)
    int n_owned_u = 0;
    // Déclaration de variable typée
    int n_owned_v = 0;
    // Déclaration de variable typée
    int n_owned_p = 0;
    // Déclaration de variable typée
    int n_owned_total = 0;

    // Matrice bloc locale pour le préconditionnement
    CSRMatrix Fuu;
    CSRMatrix Fvv;
    CSRMatrix SchurLaplace;
    // Déclaration de variable typée
    std::vector<double> diag_u;
    // Déclaration de variable typée
    std::vector<double> diag_v;
    // Déclaration de variable typée
    std::vector<double> diag_p;

    // Calcul et assignation numérique
    NavierStokesSolver() = default;

    /**
     * @brief Initialise les indexations et structures locales
     */
    void init(const Mesh2D& m, const Config& cfg);

    /**
     * @brief Applique l'opérateur couplé global : y = A(x) avec communications MPI
     */
    void apply_system_matrix(const FlowFields& fields_old, double dt,
                             const std::vector<double>& x, std::vector<double>& y) const;

    /**
     * @brief Construit le second membre b = rhs du système monolithique
     */
```

```
139     void assemble_rhs(const FlowFields& fields, double dt, std::vector<double>& rhs) const;
140
141     /**
142      * @brief Construit les matrices locales des blocs Fuu, Fvv et du complément de Schur pour le pré
             conditionneur
143      */
144     void assemble_preconditioner_blocks(const FlowFields& fields, double dt);
145
146     /**
147      * @brief Convertit les champs décalés (u, v, p) en vecteur 1D distribué
148      */
149     void pack_vector(const FlowFields& fields, std::vector<double>& vec) const;
150
151     /**
152      * @brief Déballe le vecteur 1D distribué dans les champs décalés (u, v, p)
153      */
154     void unpack_vector(const std::vector<double>& vec, FlowFields& fields, bool homogeneous = false) const;
155
156     /**
157      * @brief Met à jour le champ de pression via le Lagrangien augmenté : p^{n+1} = p^n - r * div(u^{n+1})
158      */
159     void update_pressure_al(FlowFields& fields) const;
160
161     /**
162      * @brief Résout l'équation de transport-diffusion de la température T^{n+1}
163      */
164     void solve_energy(FlowFields& fields, double dt) const;
165
166     /**
167      * @brief Calcule le nombre de Nusselt moyen sur la paroi chaude (x = 0)
168      */
169     double compute_mean_nusselt_hot_wall(const FlowFields& fields) const;
170
171     // Convertisseurs d'indices locaux
172     inline int local_u_dof(int i, int j) const {
173         // Retour de fonction
174         return (j - 1) * (mesh.nx_local + (mesh.is_boundary_right ? 1 : 0)) + (i - 1);
175     }
176     inline int local_v_dof(int i, int j) const {
177         // Retour de fonction
178         return (j - 1) * mesh.nx_local + (i - 1);
179     }
180     inline int local_p_dof(int i, int j) const {
181         // Retour de fonction
182         return (j - 1) * mesh.nx_local + (i - 1);
183     }
184 };
185
186 } // namespace covof
```

9.6 Répertoire des Classes, Structures et Méthodes de `src/NavierStokesSolve`

Ligne	Catégorie	Identifiant C++17	Signature / Rôle
9	Méthode / Fonction	`NavierStokesSolver::init`	Type retour : void
66	Méthode / Fonction	`NavierStokesSolver::pack_vector`	Type retour : void
114	Méthode / Fonction	`NavierStokesSolver::unpack_vector`	Type retour : void
188	Méthode / Fonction	`div_cell`	Type retour : std : :vector<double>
461	Méthode / Fonction	`NavierStokesSolver::assemble_rhs`	Type retour : void
707	Méthode / Fonction	`NavierStokesSolver::update_pressure_al`	Type retour : void
728	Méthode / Fonction	`NavierStokesSolver::assemble_preconditioner_blocks`	Type retour : void
805	Méthode / Fonction	`NavierStokesSolver::solve_energy`	Type retour : void
838	Méthode / Fonction	`rhs`	Type retour : std : :vector<double>
840	Méthode / Fonction	`T_sol`	Type retour : std : :vector<double>
980	Méthode / Fonction	`y_prec`	Type retour : std : :vector<double>
982	Méthode / Fonction	`z_prec`	Type retour : std : :vector<double>
1100	Méthode / Fonction	`NavierStokesSolver::compute_mean_nusselt_hot_wall`	Type retour : double

9.7 Code Source Intégral Annoté : `src/NavierStokesSolver.cpp`

Le listing ci-dessous présente l'intégralité des 1138 lignes du fichier `src/NavierStokesSolver.cpp` de l'implémentation C++17 :

```
1 #include "covof/NavierStokesSolver.hpp"
2 #include "covof/MpiComm.hpp"
3 #include <cmath>
4 #include <iostream>
5
6 namespace covof {
```

```cpp
namespace fugu = covof; // Alias rétrocompatibilité

void NavierStokesSolver::init(const Mesh2D& m, const Config& cfg) {
    // Calcul et assignation numérique
    mesh = m;
    // Calcul et assignation numérique
    bc = cfg.bc;
    // Calcul et assignation numérique
    enable_convection = cfg.enable_convection;
    // Calcul et assignation numérique
    enable_energy = cfg.enable_energy;
    // Calcul et assignation numérique
    gx = cfg.gx;
    // Calcul et assignation numérique
    gy = cfg.gy;
    // Calcul et assignation numérique
    beta = cfg.beta;
    // Calcul et assignation numérique
    T_ref = cfg.T_ref;
    // Calcul et assignation numérique
    Cp = cfg.fluid1.Cp;
    // Calcul et assignation numérique
    k_thermal = cfg.fluid1.k_thermal;
    // Calcul et assignation numérique
    Ra = cfg.Ra;
    // Calcul et assignation numérique
    Pr = cfg.Pr;
    // Calcul et assignation numérique
    solver_type = cfg.solver.type;
    // Calcul et assignation numérique
    r_al = cfg.solver.r_al;
    // Calcul et assignation numérique
    uzawa_iter = cfg.solver.uzawa_iter;
    // Calcul et assignation numérique
    obstacles = cfg.obstacles;

    // Déclaration de variable typée
    int u_nx = mesh.nx_local + (mesh.is_boundary_right ? 1 : 0);
    // Calcul et assignation numérique
    n_owned_u = u_nx * mesh.ny_local;

    // Déclaration de variable typée
    int v_ny = mesh.ny_local + (mesh.is_boundary_top ? 1 : 0);
    // Calcul et assignation numérique
    n_owned_v = mesh.nx_local * v_ny;

    // Calcul et assignation numérique
    n_owned_p = mesh.nx_local * mesh.ny_local;

    // Évaluation de la condition de contrôle
    if (is_augmented_lagrangian()) {
        // Calcul et assignation numérique
        n_owned_total = n_owned_u + n_owned_v;
    } else {
        // Calcul et assignation numérique
        n_owned_total = n_owned_u + n_owned_v + n_owned_p;
    }
}

void NavierStokesSolver::pack_vector(const FlowFields& fields, std::vector<double>& vec) const {
    vec.assign(n_owned_total, 0.0);

    // 1. Pack u
    int u_nx = mesh.nx_local + (mesh.is_boundary_right ? 1 : 0);
    // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
    for (int j = 1; j <= mesh.ny_local; ++j) {
        // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
        for (int i = 1; i <= u_nx; ++i) {
            // Déclaration de variable typée
            int dof = (j - 1) * u_nx + (i - 1);
            // Calcul et assignation numérique
            vec[dof] = fields.u[mesh.u_idx(i, j)];
        }
    }

    // 2. Pack v
    int v_offset = n_owned_u;
    // Déclaration de variable typée
    int v_ny = mesh.ny_local + (mesh.is_boundary_top ? 1 : 0);
    // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
    for (int j = 1; j <= v_ny; ++j) {
        // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
        for (int i = 1; i <= mesh.nx_local; ++i) {
            // Déclaration de variable typée
            int dof = (j - 1) * mesh.nx_local + (i - 1);
            // Calcul et assignation numérique
            vec[v_offset + dof] = fields.v[mesh.v_idx(i, j)];
        }
    }

    // 3. Pack p (si monolithique)
    if (is_monolithic()) {
```

```
        // Déclaration de variable typée
        int p_offset = n_owned_u + n_owned_v;
        // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
        for (int j = 1; j <= mesh.ny_local; ++j) {
            // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
            for (int i = 1; i <= mesh.nx_local; ++i) {
                // Déclaration de variable typée
                int dof = (j - 1) * mesh.nx_local + (i - 1);
                // Calcul et assignation numérique
                vec[p_offset + dof] = fields.p[mesh.cell_idx(i, j)];
            }
        }
    }
}

void NavierStokesSolver::unpack_vector(const std::vector<double>& vec, FlowFields& fields, bool homogeneous)
     const {
    // 1. Unpack u
    int u_nx = mesh.nx_local + (mesh.is_boundary_right ? 1 : 0);
    // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
    for (int j = 1; j <= mesh.ny_local; ++j) {
        // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
        for (int i = 1; i <= u_nx; ++i) {
            // Déclaration de variable typée
            int dof = (j - 1) * u_nx + (i - 1);
            // Calcul de la résolution géométrique de cellule
            fields.u[mesh.u_idx(i, j)] = vec[dof];
        }
    }

    // 2. Unpack v
    int v_offset = n_owned_u;
    // Déclaration de variable typée
    int v_ny = mesh.ny_local + (mesh.is_boundary_top ? 1 : 0);
    // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
    for (int j = 1; j <= v_ny; ++j) {
        // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
        for (int i = 1; i <= mesh.nx_local; ++i) {
            // Déclaration de variable typée
            int dof = (j - 1) * mesh.nx_local + (i - 1);
            // Calcul de la résolution géométrique de cellule
            fields.v[mesh.v_idx(i, j)] = vec[v_offset + dof];
        }
    }

    // 3. Unpack p (si monolithique)
    if (is_monolithic()) {
        // Déclaration de variable typée
        int p_offset = n_owned_u + n_owned_v;
        // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
        for (int j = 1; j <= mesh.ny_local; ++j) {
            // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
            for (int i = 1; i <= mesh.nx_local; ++i) {
                // Déclaration de variable typée
                int dof = (j - 1) * mesh.nx_local + (i - 1);
                // Calcul de la résolution géométrique de cellule
                fields.p[mesh.cell_idx(i, j)] = vec[p_offset + dof];
            }
        }
        MpiComm::sync_cell_scalar(mesh, fields.p);
    }

    // Synchronisation des mailles fantômes pour calculs ultérieurs
    MpiComm::sync_u(mesh, fields.u);
    MpiComm::sync_v(mesh, fields.v);
    MpiComm::apply_boundary_conditions(mesh, bc, fields, homogeneous);
}

void NavierStokesSolver::apply_system_matrix(const FlowFields& fields_old, double dt,
                                             const std::vector<double>& x, std::vector<double>& y) const {
    y.assign(n_owned_total, 0.0);

    FlowFields tf;
    tf.allocate(mesh);
    // Calcul et assignation numérique
    unpack_vector(x, tf, true); // true = CL homogènes pour garantir la stricte linéarité de A*x

    // Déclaration de variable typée
    double dx = mesh.dx;
    // Déclaration de variable typée
    double dy = mesh.dy;
    // Déclaration de variable typée
    double dvol = dx * dy;

    // Déclaration de variable typée
    int u_nx = mesh.nx_local + (mesh.is_boundary_right ? 1 : 0);
    // Déclaration de variable typée
    int v_ny = mesh.ny_local + (mesh.is_boundary_top ? 1 : 0);

    // Précalcul de la divergence sur chaque cellule locale + halo
    std::vector<double> div_cell((mesh.nx_local + 2) * (mesh.ny_local + 2), 0.0);
    // Calcul de la résolution géométrique de cellule
```

```cpp
    auto c_idx = [&](int i, int j) { return j * (mesh.nx_local + 2) + i; };

    // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
    for (int j = 0; j <= mesh.ny_local + 1; ++j) {
        // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
        for (int i = 0; i <= mesh.nx_local + 1; ++i) {
            // Déclaration de variable typée
            double d = (tf.u[mesh.u_idx(i + 1, j)] - tf.u[mesh.u_idx(i, j)]) / dx +
                       (tf.v[mesh.v_idx(i, j + 1)] - tf.v[mesh.v_idx(i, j)]) / dy;
            // Calcul de la résolution géométrique de cellule
            div_cell[c_idx(i, j)] = d;
        }
    }

    // ============================================================================
    // 1. Équation de Quantité de Mouvement en u (Faces verticales)
    // ============================================================================
    for (int j = 1; j <= mesh.ny_local; ++j) {
        // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
        for (int i = 1; i <= u_nx; ++i) {
            // Déclaration de variable typée
            int dof = (j - 1) * u_nx + (i - 1);

            // Condition aux limites physique sur les parois gauche / droite ou obstacle solide
            if (i == 1 && mesh.is_boundary_left) {
                // Déclaration de variable typée
                double rho_u = fields_old.rho_u[mesh.u_idx(i, j)];
                // Déclaration de variable typée
                double diag_bc = (rho_u / dt) * dvol;
                // Évaluation de la condition de contrôle
                if (bc.left.type == BCType::NEUMANN) {
                    // Calcul et assignation numérique
                    y[dof] = diag_bc * (tf.u[mesh.u_idx(1, j)] - tf.u[mesh.u_idx(2, j)]);
                } else {
                    // Calcul et assignation numérique
                    y[dof] = diag_bc * tf.u[mesh.u_idx(1, j)];
                }
                continue;
            }
            // Évaluation de la condition de contrôle
            if (i == u_nx && mesh.is_boundary_right) {
                // Déclaration de variable typée
                double rho_u = fields_old.rho_u[mesh.u_idx(i, j)];
                // Déclaration de variable typée
                double diag_bc = (rho_u / dt) * dvol;
                // Évaluation de la condition de contrôle
                if (bc.right.type == BCType::NEUMANN) {
                    // Calcul et assignation numérique
                    y[dof] = diag_bc * (tf.u[mesh.u_idx(u_nx, j)] - tf.u[mesh.u_idx(u_nx - 1, j)]);
                } else {
                    // Calcul et assignation numérique
                    y[dof] = diag_bc * tf.u[mesh.u_idx(u_nx, j)];
                }
                continue;
            }
            // Évaluation de la condition de contrôle
            if (is_u_face_in_obstacle(i, j)) {
                // Déclaration de variable typée
                double rho_u = fields_old.rho_u[mesh.u_idx(i, j)];
                // Déclaration de variable typée
                double diag_bc = (rho_u / dt) * dvol;
                // Calcul et assignation numérique
                y[dof] = diag_bc * tf.u[mesh.u_idx(i, j)];
                continue;
            }

            // Déclaration de variable typée
            double rho_u = fields_old.rho_u[mesh.u_idx(i, j)];

            // Terme instationnaire
            double val = (rho_u / dt) * tf.u[mesh.u_idx(i, j)] * dvol;

            // Terme convectif semi-implicite F_uu (forme advective non-conservative a densite harmonique)
            if (enable_convection) {
                double u_n = fields_old.u[mesh.u_idx(i, j)];
                double u_n_E = fields_old.u[mesh.u_idx(i + 1, j)];
                double u_n_W = fields_old.u[mesh.u_idx(i - 1, j)];

                double v_n_N = 0.5 * (fields_old.v[mesh.v_idx(i - 1, j + 1)] + fields_old.v[mesh.v_idx(i, j + 1)]);
                double v_n_S = 0.5 * (fields_old.v[mesh.v_idx(i - 1, j)] + fields_old.v[mesh.v_idx(i, j)]);

                double rho_u_E = fields_old.rho_u[mesh.u_idx(i + 1, j)];
                double rho_u_W = fields_old.rho_u[mesh.u_idx(i - 1, j)];
                double rho_u_N = (j < mesh.ny_local) ? fields_old.rho_u[mesh.u_idx(i, j + 1)] : rho_u;
                double rho_u_S = (j > 1) ? fields_old.rho_u[mesh.u_idx(i, j - 1)] : rho_u;

                double rho_E = (2.0 * rho_u * rho_u_E) / std::max(1.0e-12, rho_u + rho_u_E);
                double rho_W = (2.0 * rho_u * rho_u_W) / std::max(1.0e-12, rho_u + rho_u_W);
                double rho_N = (2.0 * rho_u * rho_u_N) / std::max(1.0e-12, rho_u + rho_u_N);
                double rho_S = (2.0 * rho_u * rho_u_S) / std::max(1.0e-12, rho_u + rho_u_S);

```

```
281                 double du_E = 0.5 * (u_n + u_n_E) * 0.5 * (tf.u[mesh.u_idx(i + 1, j)] - tf.u[mesh.u_idx(i, j)]) * dy;
282                 double du_W = 0.5 * (u_n_W + u_n) * 0.5 * (tf.u[mesh.u_idx(i, j)] - tf.u[mesh.u_idx(i - 1, j)]) * dy;
283                 double du_N = v_n_N * 0.5 * (tf.u[mesh.u_idx(i, j + 1)] - tf.u[mesh.u_idx(i, j)]) * dx;
284                 double du_S = v_n_S * 0.5 * (tf.u[mesh.u_idx(i, j)] - tf.u[mesh.u_idx(i, j - 1)]) * dx;
285
286                 val += (rho_E * du_E + rho_W * du_W + rho_N * du_N + rho_S * du_S);
287             }
288
289             // Termes visqueux implicites d'ordre 2 (Caltagirone et al. - Annexe A)
290             // (a) Viscosité d'élongation 2*mu d2u/dx2
291             double mu_cell_E = fields_old.mu[mesh.cell_idx(i, j)];
292             // Déclaration de variable typée
293             double mu_cell_W = fields_old.mu[mesh.cell_idx(i - 1, j)];
294             // Déclaration de variable typée
295             double diff_elong_E = 2.0 * mu_cell_E * (tf.u[mesh.u_idx(i + 1, j)] - tf.u[mesh.u_idx(i, j)]) / dx * dy;
296             // Déclaration de variable typée
297             double diff_elong_W = 2.0 * mu_cell_W * (tf.u[mesh.u_idx(i, j)] - tf.u[mesh.u_idx(i - 1, j)]) / dx * dy;
298             // Calcul et assignation numérique
299             val -= (diff_elong_E - diff_elong_W);
300
301             // (b) Viscosité de cisaillement et rotation mu (d2u/dy2 + d2v/dxdy)
302             double mu_node_N = fields_old.mu_node[mesh.node_idx(i, j + 1)];
303             // Déclaration de variable typée
304             double mu_node_S = fields_old.mu_node[mesh.node_idx(i, j)];
305
306             // Déclaration de variable typée
307             double shear_N = mu_node_N * ((tf.u[mesh.u_idx(i, j + 1)] - tf.u[mesh.u_idx(i, j)]) / dy +
308                                           (tf.v[mesh.v_idx(i, j + 1)] - tf.v[mesh.v_idx(i - 1, j + 1)]) / dx) * dx;
309             // Déclaration de variable typée
310             double shear_S = mu_node_S * ((tf.u[mesh.u_idx(i, j)] - tf.u[mesh.u_idx(i, j - 1)]) / dy +
311                                           (tf.v[mesh.v_idx(i, j)] - tf.v[mesh.v_idx(i - 1, j)]) / dx) * dx;
312             // Calcul et assignation numérique
313             val -= (shear_N - shear_S);
314
315             // Terme de pénalité de Lagrangien augmenté : - r * d(div)/dx * dvol = - r * (div_E - div_W) * dy
316             double grad_div_x = (div_cell[c_idx(i, j)] - div_cell[c_idx(i - 1, j)]) * dy;
317             // Calcul et assignation numérique
318             val -= r_al * grad_div_x;
319
320             // Gradient de pression discret B_u^T (uniquement si monolithique)
321             if (is_monolithic()) {
322                 // Déclaration de variable typée
323                 double grad_p = (tf.p[mesh.cell_idx(i, j)] - tf.p[mesh.cell_idx(i - 1, j)]) * dy;
324                 // Calcul et assignation numérique
325                 val += grad_p;
326             }
327
328             // Calcul et assignation numérique
329             y[dof] = val;
330         }
331     }
332
333     // ============================================================================
334     // 2. Équation de Quantité de Mouvement en v (Faces horizontales)
335     // ============================================================================
336     int v_offset = n_owned_u;
337     // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
338     for (int j = 1; j <= v_ny; ++j) {
339         // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
340         for (int i = 1; i <= mesh.nx_local; ++i) {
341             // Déclaration de variable typée
342             int dof = v_offset + (j - 1) * mesh.nx_local + (i - 1);
343
344             // Condition aux limites physique Dirichlet ou Free-Slip sur les parois basse / haute ou obstacle solide
345             bool is_wall_bot = (j == 1 && mesh.is_boundary_bottom && (bc.bottom.type == BCType::DIRICHLET || bc.bottom.type == BCType::FREE_SLIP));
346             // Déclaration de variable typée
347             bool is_wall_top = (j == v_ny && mesh.is_boundary_top && (bc.top.type == BCType::DIRICHLET || bc.top.type == BCType::FREE_SLIP));
348             // Déclaration de variable typée
349             bool in_obs = is_v_face_in_obstacle(i, j);
350             // Évaluation de la condition de contrôle
351             if (is_wall_bot || is_wall_top || in_obs) {
352                 // Déclaration de variable typée
353                 double rho_v = fields_old.rho_v[mesh.v_idx(i, j)];
354                 // Déclaration de variable typée
355                 double diag_bc = (rho_v / dt) * dvol;
356                 // Calcul et assignation numérique
357                 y[dof] = diag_bc * tf.v[mesh.v_idx(i, j)];
358                 continue;
359             }
360
361             // Déclaration de variable typée
362             double rho_v = fields_old.rho_v[mesh.v_idx(i, j)];
363
364             // Terme instationnaire
```

```
365             double val = (rho_v / dt) * tf.v[mesh.v_idx(i, j)] * dvol;
366
367             // Terme convectif semi-implicite F_vv (forme advective non-conservative a densite harmonique)
368             if (enable_convection) {
369                 double v_n = fields_old.v[mesh.v_idx(i, j)];
370                 double v_n_N = fields_old.v[mesh.v_idx(i, j + 1)];
371                 double v_n_S = fields_old.v[mesh.v_idx(i, j - 1)];
372
373                 double u_n_E = 0.5 * (fields_old.u[mesh.u_idx(i + 1, j - 1)] + fields_old.u[mesh.u_idx(i + 1,
                        j)]);
374                 double u_n_W = 0.5 * (fields_old.u[mesh.u_idx(i, j - 1)] + fields_old.u[mesh.u_idx(i, j)]);
375
376                 double rho_v_E = (i < mesh.nx_local) ? fields_old.rho_v[mesh.v_idx(i + 1, j)] : rho_v;
377                 double rho_v_W = (i > 1) ? fields_old.rho_v[mesh.v_idx(i - 1, j)] : rho_v;
378                 double rho_v_N = fields_old.rho_v[mesh.v_idx(i, j + 1)];
379                 double rho_v_S = fields_old.rho_v[mesh.v_idx(i, j - 1)];
380
381                 double rho_E = (2.0 * rho_v * rho_v_E) / std::max(1.0e-12, rho_v + rho_v_E);
382                 double rho_W = (2.0 * rho_v * rho_v_W) / std::max(1.0e-12, rho_v + rho_v_W);
383                 double rho_N = (2.0 * rho_v * rho_v_N) / std::max(1.0e-12, rho_v + rho_v_N);
384                 double rho_S = (2.0 * rho_v * rho_v_S) / std::max(1.0e-12, rho_v + rho_v_S);
385
386                 double dv_E = u_n_E * 0.5 * (tf.v[mesh.v_idx(i + 1, j)] - tf.v[mesh.v_idx(i, j)]) * dy;
387                 double dv_W = u_n_W * 0.5 * (tf.v[mesh.v_idx(i, j)] - tf.v[mesh.v_idx(i - 1, j)]) * dy;
388                 double dv_N = 0.5 * (v_n + v_n_N) * 0.5 * (tf.v[mesh.v_idx(i, j + 1)] - tf.v[mesh.v_idx(i, j)
                        ]) * dx;
389                 double dv_S = 0.5 * (v_n_S + v_n) * 0.5 * (tf.v[mesh.v_idx(i, j)] - tf.v[mesh.v_idx(i, j - 1)
                        ]) * dx;
390
391                 val += (rho_E * dv_E + rho_W * dv_W + rho_N * dv_N + rho_S * dv_S);
392             }
393
394             // Termes visqueux implicites d'ordre 2
395             // (a) Viscosité d'élongation 2*mu d2v/dy2
396             double mu_cell_N = fields_old.mu[mesh.cell_idx(i, j)];
397             // Déclaration de variable typée
398             double mu_cell_S = fields_old.mu[mesh.cell_idx(i, j - 1)];
399             // Déclaration de variable typée
400             double diff_elong_N = 2.0 * mu_cell_N * (tf.v[mesh.v_idx(i, j + 1)] - tf.v[mesh.v_idx(i, j)]) / dy
                    * dx;
401             // Déclaration de variable typée
402             double diff_elong_S = 2.0 * mu_cell_S * (tf.v[mesh.v_idx(i, j)] - tf.v[mesh.v_idx(i, j - 1)]) / dy
                    * dx;
403             // Calcul et assignation numérique
404             val -= (diff_elong_N - diff_elong_S);
405
406             // (b) Viscosité de cisaillement et rotation mu (d2v/dx2 + d2u/dxdy)
407             double mu_node_E = fields_old.mu_node[mesh.node_idx(i + 1, j)];
408             // Déclaration de variable typée
409             double mu_node_W = fields_old.mu_node[mesh.node_idx(i, j)];
410
411             // Déclaration de variable typée
412             double shear_E = mu_node_E * ((tf.v[mesh.v_idx(i + 1, j)] - tf.v[mesh.v_idx(i, j)]) / dx +
413                                           (tf.u[mesh.u_idx(i + 1, j)] - tf.u[mesh.u_idx(i + 1, j - 1)]) / dy)
                                               * dy;
414             // Déclaration de variable typée
415             double shear_W = mu_node_W * ((tf.v[mesh.v_idx(i, j)] - tf.v[mesh.v_idx(i - 1, j)]) / dx +
416                                           (tf.u[mesh.u_idx(i, j)] - tf.u[mesh.u_idx(i, j - 1)]) / dy) * dy;
417             // Calcul et assignation numérique
418             val -= (shear_E - shear_W);
419
420             // Terme de pénalité de Lagrangien augmenté : - r * d(div)/dy * dvol = - r * (div_N - div_S) * dx
421             double grad_div_y = (div_cell[c_idx(i, j)] - div_cell[c_idx(i, j - 1)]) * dx;
422             // Calcul et assignation numérique
423             val -= r_al * grad_div_y;
424
425             // Gradient de pression discret B_v^T (uniquement si monolithique)
426             if (is_monolithic()) {
427                 // Déclaration de variable typée
428                 double grad_p = (tf.p[mesh.cell_idx(i, j)] - tf.p[mesh.cell_idx(i, j - 1)]) * dx;
429                 // Calcul et assignation numérique
430                 val += grad_p;
431             }
432
433             // Calcul et assignation numérique
434             y[dof] = val;
435         }
436     }
437
438     // ========================================================================
439     // 3. Équation de Continuité (si monolithique)
440     // ========================================================================
441     if (is_monolithic()) {
442         // Déclaration de variable typée
443         int p_offset = n_owned_u + n_owned_v;
444         // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
445         for (int j = 1; j <= mesh.ny_local; ++j) {
446             // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
447             for (int i = 1; i <= mesh.nx_local; ++i) {
448                 // Déclaration de variable typée
449                 int dof = p_offset + (j - 1) * mesh.nx_local + (i - 1);
450                 // Déclaration de variable typée
```

```
                double div = div_cell[c_idx(i, j)] * dvol;
                // Déclaration de variable typée
                double diag_p_term = (dvol / r_al) * tf.p[mesh.cell_idx(i, j)];
                // Calcul et assignation numérique
                y[dof] = div - diag_p_term;
            }
        }
    }
}

void NavierStokesSolver::assemble_rhs(const FlowFields& fields, double dt, std::vector<double>& rhs) const {
    rhs.assign(n_owned_total, 0.0);

    // Déclaration de variable typée
    double dx = mesh.dx;
    // Déclaration de variable typée
    double dy = mesh.dy;
    // Déclaration de variable typée
    double dvol = dx * dy;

    // Déclaration de variable typée
    int u_nx = mesh.nx_local + (mesh.is_boundary_right ? 1 : 0);
    // Déclaration de variable typée
    int v_ny = mesh.ny_local + (mesh.is_boundary_top ? 1 : 0);

    // RHS pour u
    for (int j = 1; j <= mesh.ny_local; ++j) {
        // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
        for (int i = 1; i <= u_nx; ++i) {
            // Déclaration de variable typée
            int dof = (j - 1) * u_nx + (i - 1);
            // Déclaration de variable typée
            double rho_u = fields.rho_u[mesh.u_idx(i, j)];
            // Déclaration de variable typée
            double diag_bc = (rho_u / dt) * dvol;

            // Si face dans un obstacle solide (adhérence nulle)
            if (is_u_face_in_obstacle(i, j)) {
                // Calcul et assignation numérique
                rhs[dof] = 0.0;
                continue;
            }

            // Si paroi physique gauche
            if (i == 1 && mesh.is_boundary_left) {
                // Déclaration de variable typée
                double u_bc = 0.0;
                // Évaluation de la condition de contrôle
                if (bc.left.type == BCType::DIRICHLET) {
                    // Évaluation de la condition de contrôle
                    if (bc.left.profile == "POISEUILLE_STEP") {
                        // Déclaration de variable typée
                        double y = mesh.yc(j);
                        // Évaluation de la condition de contrôle
                        if (y >= bc.left.y_step) {
                            // Déclaration de variable typée
                            double h = mesh.ly - bc.left.y_step;
                            // Calcul et assignation numérique
                            u_bc = 6.0 * bc.left.u_mean * (y - bc.left.y_step) * (mesh.ly - y) / (h * h);
                        }
                    // Calcul et assignation numérique
                    } else if (bc.left.profile == "PARABOLIC") {
                        // Déclaration de variable typée
                        double y = mesh.yc(j);
                        // Déclaration de variable typée
                        double h = mesh.ly;
                        // Calcul et assignation numérique
                        u_bc = 6.0 * bc.left.u_mean * y * (h - y) / (h * h);
                    } else {
                        // Calcul et assignation numérique
                        u_bc = bc.left.u_val;
                    }
                // Calcul et assignation numérique
                } else if (bc.left.type == BCType::NEUMANN) {
                    // Calcul et assignation numérique
                    u_bc = 0.0;
                }
                // Calcul et assignation numérique
                rhs[dof] = diag_bc * u_bc;
                continue;
            }
            // Si paroi physique droite
            if (i == u_nx && mesh.is_boundary_right) {
                // Déclaration de variable typée
                double u_bc = 0.0;
                // Évaluation de la condition de contrôle
                if (bc.right.type == BCType::DIRICHLET) {
                    // Calcul et assignation numérique
                    u_bc = bc.right.u_val;
                // Calcul et assignation numérique
                } else if (bc.right.type == BCType::NEUMANN) {
                    // Calcul et assignation numérique
```

```
                    u_bc = 0.0;
                }
                // Calcul et assignation numérique
                rhs[dof] = diag_bc * u_bc;
                continue;
            }

            // Déclaration de variable typée
            double val = (rho_u / dt) * fields.u[mesh.u_idx(i, j)] * dvol;
            // Calcul et assignation numérique
            val += rho_u * gx * dvol;
            // Calcul et assignation numérique
            val += fields.F_sigma_x[mesh.u_idx(i, j)] * dvol;

            // Pour Lagrangien augmenté standard : le gradient de pression -grad(p^n) est explicite au second membre
            if (is_augmented_lagrangian()) {
                // Déclaration de variable typée
                double grad_p = (fields.p[mesh.cell_idx(i, j)] - fields.p[mesh.cell_idx(i - 1, j)]) * dy;
                // Calcul et assignation numérique
                val -= grad_p;
            }

            // Contribution de contrainte tangentielle à la paroi pour u (haut / bas)
            if (j == mesh.ny_local && mesh.is_boundary_top && bc.top.type == BCType::DIRICHLET) {
                // Déclaration de variable typée
                double mu_N = fields.mu_node[mesh.node_idx(i, j + 1)];
                // Calcul et assignation numérique
                val += mu_N * (2.0 * bc.top.u_val / dy) * dx;
            }
            // Évaluation de la condition de contrôle
            if (j == 1 && mesh.is_boundary_bottom && bc.bottom.type == BCType::DIRICHLET) {
                // Déclaration de variable typée
                double mu_S = fields.mu_node[mesh.node_idx(i, j)];
                // Calcul et assignation numérique
                val += mu_S * (2.0 * bc.bottom.u_val / dy) * dx;
            }

            // Calcul et assignation numérique
            rhs[dof] = val;
        }
    }

    // RHS pour v
    int v_offset = n_owned_u;
    // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
    for (int j = 1; j <= v_ny; ++j) {
        // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
        for (int i = 1; i <= mesh.nx_local; ++i) {
            // Déclaration de variable typée
            int dof = v_offset + (j - 1) * mesh.nx_local + (i - 1);
            // Déclaration de variable typée
            double rho_v = fields.rho_v[mesh.v_idx(i, j)];
            // Déclaration de variable typée
            double diag_bc = (rho_v / dt) * dvol;

            // Si face dans un obstacle solide (adhérence nulle)
            if (is_v_face_in_obstacle(i, j)) {
                // Calcul et assignation numérique
                rhs[dof] = 0.0;
                continue;
            }

            // Évaluation de la condition de contrôle
            if (j == 1 && mesh.is_boundary_bottom && (bc.bottom.type == BCType::DIRICHLET || bc.bottom.type == BCType::FREE_SLIP)) {
                // Déclaration de variable typée
                double v_bc = (bc.bottom.type == BCType::DIRICHLET) ? bc.bottom.v_val : 0.0;
                // Calcul et assignation numérique
                rhs[dof] = diag_bc * v_bc;
                continue;
            }
            // Évaluation de la condition de contrôle
            if (j == v_ny && mesh.is_boundary_top && (bc.top.type == BCType::DIRICHLET || bc.top.type == BCType::FREE_SLIP)) {
                // Déclaration de variable typée
                double v_bc = 0.0;
                // Évaluation de la condition d'injecteur (Cas 3.3 Fluids 2021)
                if (bc.top.profile == "JET_INLET") {
                    // Déclaration de variable typée
                    double x = mesh.xc(i);
                    // Déclaration de variable typée pour le centre et demi-largeur
                    double xc = (bc.top.jet_center > 0.0) ? bc.top.jet_center : 0.5 * mesh.lx;
                    // Calcul de la demi-largeur de l'injecteur
                    double half_w = (bc.top.jet_width > 0.0) ? 0.5 * bc.top.jet_width : 0.04;
                    double d_disc = 0.0;
                    for (int ig = 1; ig <= mesh.nx_global; ++ig) {
                        double xg = (ig - 0.5) * mesh.dx;
                        if (std::abs(xg - xc) <= half_w) d_disc += mesh.dx;
                    }
                    if (d_disc < 1.0e-12) d_disc = 2.0 * half_w;
                    // Évaluation de l'appartenance à l'injecteur
```

```
                    if (std::abs(x - xc) <= half_w) {
                        // Vitesse d'injection négative imposée (conservant le débit D * v0 exact)
                        v_bc = bc.top.v_val * (2.0 * half_w) / d_disc;
                    } else {
                        // Vitesse positive d'échappement compensatoire du gaz
                        v_bc = -bc.top.v_val * (2.0 * half_w) / (mesh.lx - d_disc);
                    }
                } else if (bc.top.type == BCType::DIRICHLET) {
                    // Calcul et assignation numérique standard
                    v_bc = bc.top.v_val;
                }
                // Calcul et assignation numérique
                rhs[dof] = diag_bc * v_bc;
                continue;
            }

            // Déclaration de variable typée
            double val = (rho_v / dt) * fields.v[mesh.v_idx(i, j)] * dvol;
            // Calcul et assignation numérique
            val += rho_v * gy * dvol;
            // Évaluation de la condition de contrôle
            if (enable_energy) {
                // Déclaration de variable typée
                double T_v = 0.5 * (fields.T[mesh.cell_idx(i, j)] + fields.T[mesh.cell_idx(i, j - 1)]);
                // Calcul et assignation numérique
                val += rho_v * beta * (T_v - T_ref) * gy * dvol;
            }
            // Calcul et assignation numérique
            val += fields.F_sigma_y[mesh.v_idx(i, j)] * dvol;

            // Pour Lagrangien augmenté standard : -grad(p^n)
            if (is_augmented_lagrangian()) {
                // Déclaration de variable typée
                double grad_p = (fields.p[mesh.cell_idx(i, j)] - fields.p[mesh.cell_idx(i, j - 1)]) * dx;
                // Calcul et assignation numérique
                val -= grad_p;
            }

            // Contribution de contrainte tangentielle à la paroi pour v (gauche / droite)
            if (i == 1 && mesh.is_boundary_left && bc.left.type == BCType::DIRICHLET) {
                // Déclaration de variable typée
                double mu_W = fields.mu_node[mesh.node_idx(i, j)];
                // Calcul et assignation numérique
                val += mu_W * (2.0 * bc.left.v_val / dx) * dy;
            }
            // Évaluation de la condition de contrôle
            if (i == mesh.nx_local && mesh.is_boundary_right && bc.right.type == BCType::DIRICHLET) {
                // Déclaration de variable typée
                double mu_E = fields.mu_node[mesh.node_idx(i + 1, j)];
                // Calcul et assignation numérique
                val += mu_E * (2.0 * bc.right.v_val / dx) * dy;
            }

            // Calcul et assignation numérique
            rhs[dof] = val;
        }
    }

    // RHS pour p (si monolithique régularisé)
    if (is_monolithic()) {
        // Déclaration de variable typée
        int p_offset = n_owned_u + n_owned_v;
        // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
        for (int j = 1; j <= mesh.ny_local; ++j) {
            // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
            for (int i = 1; i <= mesh.nx_local; ++i) {
                // Déclaration de variable typée
                int dof = p_offset + (j - 1) * mesh.nx_local + (i - 1);
                // Calcul et assignation numérique
                rhs[dof] = - (dvol / r_al) * fields.p[mesh.cell_idx(i, j)];
            }
        }
    }
}

void NavierStokesSolver::update_pressure_al(FlowFields& fields) const {
    // Déclaration de variable typée
    double dx = mesh.dx;
    // Déclaration de variable typée
    double dy = mesh.dy;

    // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
    for (int j = 1; j <= mesh.ny_local; ++j) {
        // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
        for (int i = 1; i <= mesh.nx_local; ++i) {
            // Déclaration de variable typée
            double div = (fields.u[mesh.u_idx(i + 1, j)] - fields.u[mesh.u_idx(i, j)]) / dx +
                         (fields.v[mesh.v_idx(i, j + 1)] - fields.v[mesh.v_idx(i, j)]) / dy;
            // Calcul de la résolution géométrique de cellule
            fields.p[mesh.cell_idx(i, j)] -= r_al * div;
        }
    }
```

```
724
725     MpiComm::sync_cell_scalar(mesh, fields.p);
726 }
727
728 void NavierStokesSolver::assemble_preconditioner_blocks(const FlowFields& fields, double dt) {
729     // Déclaration de variable typée
730     double dx = mesh.dx;
731     // Déclaration de variable typée
732     double dy = mesh.dy;
733     // Déclaration de variable typée
734     double dvol = dx * dy;
735
736     // Calcul des diagonales effectives de Fuu et Fvv pour le préconditionnement Jacobi
737     diag_u.assign(n_owned_u, 1.0);
738     // Déclaration de variable typée
739     int u_nx = mesh.nx_local + (mesh.is_boundary_right ? 1 : 0);
740     // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
741     for (int j = 1; j <= mesh.ny_local; ++j) {
742         // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
743         for (int i = 1; i <= u_nx; ++i) {
744             // Déclaration de variable typée
745             int dof = (j - 1) * u_nx + (i - 1);
746             // Déclaration de variable typée
747             double rho_u = fields.rho_u[mesh.u_idx(i, j)];
748             // Déclaration de variable typée
749             bool is_wall = (i == 1 && mesh.is_boundary_left) ||
750                            // Calcul et assignation numérique
751                            (i == u_nx && mesh.is_boundary_right) ||
752                            is_u_face_in_obstacle(i, j);
753             // Évaluation de la condition de contrôle
754             if (is_wall) {
755                 // Calcul et assignation numérique
756                 diag_u[dof] = (rho_u / dt) * dvol;
757                 continue;
758             }
759             // Déclaration de variable typée
760             double mu_c = fields.mu[mesh.cell_idx(i, j)];
761             // Déclaration de variable typée
762             double d = (rho_u / dt) * dvol + 4.0 * mu_c * (dy / dx) + 2.0 * mu_c * (dx / dy) + 2.0 * r_al * (
                    dy / dx);
763             // Calcul et assignation numérique
764             diag_u[dof] = (d > 1.0e-15) ? d : 1.0;
765         }
766     }
767
768     diag_v.assign(n_owned_v, 1.0);
769     // Déclaration de variable typée
770     int v_ny = mesh.ny_local + (mesh.is_boundary_top ? 1 : 0);
771     // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
772     for (int j = 1; j <= v_ny; ++j) {
773         // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
774         for (int i = 1; i <= mesh.nx_local; ++i) {
775             // Déclaration de variable typée
776             int dof = (j - 1) * mesh.nx_local + (i - 1);
777             // Déclaration de variable typée
778             double rho_v = fields.rho_v[mesh.v_idx(i, j)];
779             // Déclaration de variable typée
780             bool is_wall = (j == 1 && mesh.is_boundary_bottom && (bc.bottom.type == BCType::DIRICHLET || bc.
                    bottom.type == BCType::FREE_SLIP)) ||
781                            // Calcul et assignation numérique
782                            (j == v_ny && mesh.is_boundary_top && (bc.top.type == BCType::DIRICHLET || bc.top.
                                type == BCType::FREE_SLIP)) ||
783                            is_v_face_in_obstacle(i, j);
784             // Évaluation de la condition de contrôle
785             if (is_wall) {
786                 // Calcul et assignation numérique
787                 diag_v[dof] = (rho_v / dt) * dvol;
788                 continue;
789             }
790             // Déclaration de variable typée
791             double mu_c = fields.mu[mesh.cell_idx(i, j)];
792             // Déclaration de variable typée
793             double d = (rho_v / dt) * dvol + 4.0 * mu_c * (dx / dy) + 2.0 * mu_c * (dy / dx) + 2.0 * r_al * (
                    dx / dy);
794             // Calcul et assignation numérique
795             diag_v[dof] = (d > 1.0e-15) ? d : 1.0;
796         }
797     }
798
799     // Évaluation de la condition de contrôle
800     if (is_monolithic()) {
801         diag_p.assign(n_owned_p, dvol / r_al);
802     }
803 }
804
805 void NavierStokesSolver::solve_energy(FlowFields& fields, double dt) const {
806     // Évaluation de la condition de contrôle
807     if (!enable_energy) return;
808
809     // Déclaration de variable typée
810     int nx = mesh.nx_local;
811     // Déclaration de variable typée
```

```
812     int ny = mesh.ny_local;
813     // Déclaration de variable typée
814     int n_cells = nx * ny;
815 
816     // Déclaration de variable typée
817     double dx = mesh.dx;
818     // Déclaration de variable typée
819     double dy = mesh.dy;
820     // Déclaration de variable typée
821     double dvol = dx * dy;
822     // Déclaration de variable typée
823     double rho = fields.rho[mesh.cell_idx(1, 1)];
824 
825     // Calcul et assignation numérique
826     auto cell_dof = [&](int i, int j) {
827         // Retour de fonction
828         return (j - 1) * nx + (i - 1);
829     };
830 
831     // Déclaration de variable typée
832     std::vector<int> rows;
833     // Déclaration de variable typée
834     std::vector<int> cols;
835     // Déclaration de variable typée
836     std::vector<double> vals;
837     // Déclaration de variable typée
838     std::vector<double> rhs(n_cells, 0.0);
839     // Déclaration de variable typée
840     std::vector<double> T_sol(n_cells, 0.0);
841 
842     // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
843     for (int j = 1; j <= ny; ++j) {
844         // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
845         for (int i = 1; i <= nx; ++i) {
846             // Déclaration de variable typée
847             int c_dof = cell_dof(i, j);
848             // Calcul et assignation numérique
849             T_sol[c_dof] = fields.T[mesh.cell_idx(i, j)];
850 
851             // Déclaration de variable typée
852             double a_c = (rho * Cp / dt) * dvol;
853             // Calcul et assignation numérique
854             rhs[c_dof] = (rho * Cp / dt) * dvol * fields.T[mesh.cell_idx(i, j)];
855 
856             // Flux x-Est
857             double u_e = fields.u[mesh.u_idx(i + 1, j)];
858             // Déclaration de variable typée
859             double d_e = k_thermal * dy / dx;
860             // Déclaration de variable typée
861             double conv_e = rho * Cp * u_e * dy * 0.5;
862 
863             // Flux x-Ouest
864             double u_w = fields.u[mesh.u_idx(i, j)];
865             // Déclaration de variable typée
866             double d_w = k_thermal * dy / dx;
867             // Déclaration de variable typée
868             double conv_w = rho * Cp * u_w * dy * 0.5;
869 
870             // Flux y-Nord
871             double v_n = fields.v[mesh.v_idx(i, j + 1)];
872             // Déclaration de variable typée
873             double d_n = k_thermal * dx / dy;
874             // Déclaration de variable typée
875             double conv_n = rho * Cp * v_n * dx * 0.5;
876 
877             // Flux y-Sud
878             double v_s = fields.v[mesh.v_idx(i, j)];
879             // Déclaration de variable typée
880             double d_s = k_thermal * dx / dy;
881             // Déclaration de variable typée
882             double conv_s = rho * Cp * v_s * dx * 0.5;
883 
884             // Voisin Est
885             if (i < nx) {
886                 // Calcul et assignation numérique
887                 a_c += d_e + conv_e;
888                 rows.push_back(c_dof);
889                 cols.push_back(cell_dof(i + 1, j));
890                 vals.push_back(-d_e + conv_e);
891             } else if (mesh.is_boundary_right) {
892                 // Évaluation de la condition de contrôle
893                 if (bc.right.T_type == BCType::DIRICHLET) {
894                     // Calcul et assignation numérique
895                     a_c += 2.0 * d_e;
896                     // Calcul et assignation numérique
897                     rhs[c_dof] += 2.0 * d_e * bc.right.T_val;
898                 }
899             }
900 
901             // Voisin Ouest
902             if (i > 1) {
903                 // Calcul et assignation numérique
```

```
                a_c += d_w - conv_w;
                rows.push_back(c_dof);
                cols.push_back(cell_dof(i - 1, j));
                vals.push_back(-d_w - conv_w);
            } else if (mesh.is_boundary_left) {
                // Évaluation de la condition de contrôle
                if (bc.left.T_type == BCType::DIRICHLET) {
                    // Calcul et assignation numérique
                    a_c += 2.0 * d_w;
                    // Calcul et assignation numérique
                    rhs[c_dof] += 2.0 * d_w * bc.left.T_val;
                }
            }

            // Voisin Nord
            if (j < ny) {
                // Calcul et assignation numérique
                a_c += d_n + conv_n;
                rows.push_back(c_dof);
                cols.push_back(cell_dof(i, j + 1));
                vals.push_back(-d_n + conv_n);
            } else if (mesh.is_boundary_top) {
                // Évaluation de la condition de contrôle
                if (bc.top.T_type == BCType::DIRICHLET) {
                    // Calcul et assignation numérique
                    a_c += 2.0 * d_n;
                    // Calcul et assignation numérique
                    rhs[c_dof] += 2.0 * d_n * bc.top.T_val;
                }
            }

            // Voisin Sud
            if (j > 1) {
                // Calcul et assignation numérique
                a_c += d_s - conv_s;
                rows.push_back(c_dof);
                cols.push_back(cell_dof(i, j - 1));
                vals.push_back(-d_s - conv_s);
            } else if (mesh.is_boundary_bottom) {
                // Évaluation de la condition de contrôle
                if (bc.bottom.T_type == BCType::DIRICHLET) {
                    // Calcul et assignation numérique
                    a_c += 2.0 * d_s;
                    // Calcul et assignation numérique
                    rhs[c_dof] += 2.0 * d_s * bc.bottom.T_val;
                }
            }

            rows.push_back(c_dof);
            cols.push_back(c_dof);
            vals.push_back(a_c);
        }
    }

    CSRMatrix A;
    A.set_from_triplets(n_cells, n_cells, rows, cols, vals);

    // Solveur BiCGSTAB avec préconditionnement Jacobi
    std::vector<double> r = rhs;
    // Déclaration de variable typée
    std::vector<double> Ax(n_cells, 0.0);
    A.matvec(T_sol, Ax);
    // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
    for (int i = 0; i < n_cells; ++i) r[i] -= Ax[i];

    // Déclaration de variable typée
    std::vector<double> r0_hat = r;
    // Déclaration de variable typée
    std::vector<double> p = r;
    // Déclaration de variable typée
    std::vector<double> v(n_cells, 0.0);
    // Déclaration de variable typée
    std::vector<double> s(n_cells, 0.0);
    // Déclaration de variable typée
    std::vector<double> t(n_cells, 0.0);
    // Déclaration de variable typée
    std::vector<double> y_prec(n_cells, 0.0);
    // Déclaration de variable typée
    std::vector<double> z_prec(n_cells, 0.0);

    // Déclaration de variable typée
    double rho_old = 1.0;
    // Déclaration de variable typée
    double alpha = 1.0;
    // Déclaration de variable typée
    double omega = 1.0;

    // Déclaration de variable typée
    double res_init = 0.0;
    // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
    for (int i = 0; i < n_cells; ++i) res_init += r[i] * r[i];
    // Calcul de norme euclidienne ou résidu de convergence
```

```
    res_init = std::sqrt(res_init);
    // Évaluation de la condition de contrôle
    if (res_init > 1e-13) {
        // Déclaration de variable typée
        int max_it = 500;
        // Boucle d'itération algorithmique
        for (int iter = 0; iter < max_it; ++iter) {
            // Déclaration de variable typée
            double rho_new = 0.0;
            // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
            for (int i = 0; i < n_cells; ++i) rho_new += r0_hat[i] * r[i];
            // Évaluation de la condition de contrôle
            if (std::abs(rho_new) < 1e-30) break;

            // Évaluation de la condition de contrôle
            if (iter > 0) {
                // Déclaration de variable typée
                double beta_param = (rho_new / rho_old) * (alpha / omega);
                // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
                for (int i = 0; i < n_cells; ++i) {
                    // Calcul et assignation numérique
                    p[i] = r[i] + beta_param * (p[i] - omega * v[i]);
                }
            } else {
                // Calcul et assignation numérique
                p = r;
            }

            // Préconditionnement Jacobi : y = D^{-1} p
            for (int i = 0; i < n_cells; ++i) y_prec[i] = p[i] / A.diag[i];
            A.matvec(y_prec, v);

            // Déclaration de variable typée
            double r0v = 0.0;
            // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
            for (int i = 0; i < n_cells; ++i) r0v += r0_hat[i] * v[i];
            // Évaluation de la condition de contrôle
            if (std::abs(r0v) < 1e-30) break;
            // Calcul et assignation numérique
            alpha = rho_new / r0v;

            // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
            for (int i = 0; i < n_cells; ++i) s[i] = r[i] - alpha * v[i];
            // Déclaration de variable typée
            double s_norm = 0.0;
            // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
            for (int i = 0; i < n_cells; ++i) s_norm += s[i] * s[i];
            // Évaluation de la condition de contrôle
            if (std::sqrt(s_norm) / res_init < 1e-8) {
                // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
                for (int i = 0; i < n_cells; ++i) T_sol[i] += alpha * y_prec[i];
                break;
            }

            // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
            for (int i = 0; i < n_cells; ++i) z_prec[i] = s[i] / A.diag[i];
            A.matvec(z_prec, t);

            // Déclaration de variable typée
            double tt = 0.0, ts = 0.0;
            // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
            for (int i = 0; i < n_cells; ++i) {
                // Calcul et assignation numérique
                tt += t[i] * t[i];
                // Calcul et assignation numérique
                ts += t[i] * s[i];
            }
            // Calcul et assignation numérique
            omega = (tt > 1e-30) ? (ts / tt) : 0.0;

            // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
            for (int i = 0; i < n_cells; ++i) {
                // Calcul et assignation numérique
                T_sol[i] += alpha * y_prec[i] + omega * z_prec[i];
                // Calcul et assignation numérique
                r[i] = s[i] - omega * t[i];
            }

            // Calcul et assignation numérique
            rho_old = rho_new;

            // Déclaration de variable typée
            double r_norm = 0.0;
            // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
            for (int i = 0; i < n_cells; ++i) r_norm += r[i] * r[i];
            // Évaluation de la condition de contrôle
            if (std::sqrt(r_norm) / res_init < 1e-8) break;
        }
    }

    // Réinjecter T_sol dans fields.T
    for (int j = 1; j <= ny; ++j) {
```

```
        // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
        for (int i = 1; i <= nx; ++i) {
            // Calcul de la résolution géométrique de cellule
            fields.T[mesh.cell_idx(i, j)] = T_sol[cell_dof(i, j)];
        }
    }

    MpiComm::sync_cell_scalar(mesh, fields.T);
    MpiComm::apply_thermal_boundary_conditions(mesh, bc, fields);
}

// Déclaration de variable typée
double NavierStokesSolver::compute_mean_nusselt_hot_wall(const FlowFields& fields) const {
    // Déclaration de variable typée
    double local_nu_sum = 0.0;
    // Déclaration de variable typée
    int count = 0;
    // Évaluation de la condition de contrôle
    if (mesh.is_boundary_left) {
        // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
        for (int j = 1; j <= mesh.ny_local; ++j) {
            // Déclaration de variable typée
            double T_w = bc.left.T_val;
            // Déclaration de variable typée
            double T_1 = fields.T[mesh.cell_idx(1, j)];
            // Déclaration de variable typée
            double delta_T = std::abs(bc.left.T_val - bc.right.T_val);
            // Évaluation de la condition de contrôle
            if (delta_T < 1e-12) delta_T = 1.0;
            // Déclaration de variable typée
            double nu_local = 2.0 * (T_w - T_1) / mesh.dx * (mesh.lx / delta_T);
            // Calcul et assignation numérique
            local_nu_sum += nu_local;
            count++;
        }
    }
    // Déclaration de variable typée
    double global_nu_sum = MpiComm::global_sum(mesh, local_nu_sum);
    // Déclaration de variable typée
    double global_count = MpiComm::global_sum(mesh, static_cast<double>(count));
    // Évaluation de la condition de contrôle
    if (global_count > 0.0) {
        // Retour de fonction
        return global_nu_sum / global_count;
    }
    // Retour de fonction
    return 1.0;
}

} // namespace covof
```

Chapitre 10

Reconstruction Géométrique d'Interface VOF : PLIC, ELVIRA, LVIRA, SLIC (InterfaceReconstruction.hpp/.cpp)

10.1 Représentation Linéaire par Morceaux PLIC (*Piecewise Linear Interface Calculation*)

Dans chaque maille diphasique (i, j) telle que $\varepsilon_{\text{vof}} < C_{i,j} < 1 - \varepsilon_{\text{vof}}$ ($\varepsilon_{\text{vof}} = 10^{-12}$), l'interface est représentée dans le repère local normalisé $(\xi, \eta) \in [0, 1]^2$ de la maille par la droite :

$$n_x \xi + n_y \eta = \alpha \tag{10.1}$$

où $\mathbf{n} = (n_x, n_y)^T$ est la normale unitaire pointant vers l'extérieur du fluide 1 ($\mathbf{n} = -\nabla C / \|\nabla C\|$) et $\alpha \in \mathbb{R}$ est la constante de positionnement géométrique unique telle que l'aire du polygone tronqué $\{(\xi, \eta) \in [0, 1]^2 \mid n_x \xi + n_y \eta \leq \alpha\}$ soit exactement égale à la fraction volumique $C_{i,j}$.

10.2 Relations Analytiques Directe et Inverse de Scardovelli & Zaleski (2000)

En se ramenant par symétrie au premier quadrant ($m_1 = \min(|n_x|, |n_y|) \geq 0$, $m_2 = \max(|n_x|, |n_y|) > 0$, avec $m_1 + m_2 = 1$ en norme L_1), l'inversion analytique $C \mapsto \alpha$ implémentée dans `compute_line_constant` distingue deux régimes géométriques sans aucune itération de Newton :

1. **Régime triangulaire** ($0 \leq C \leq \frac{m_1}{2m_2}$) :
$$\alpha = \sqrt{2 m_1 m_2 C} \tag{10.2}$$
2. **Régime trapézoïdal** ($\frac{m_1}{2m_2} < C \leq \frac{1}{2}$) :
$$\alpha = m_2 C + \frac{m_1}{2} \tag{10.3}$$

Pour $C > 1/2$, la symétrie centrale du carré unité donne immédiatement $\alpha(C) = 1 - \alpha(1 - C)$.

10.3 Estimation des Normales : Parker-Youngs, ELVIRA et LVIRA

— **Parker-Youngs** : différence finie pondérée sur le stencil 3×3 avec poids $1, 2, 1$ privilégiant la direction axiale.

— **ELVIRA (Pilliod & Puckett 2004)** : évalue 6 pentes candidates (différences arrière, centrée et avant des sommes de colonnes horizontales $H_i = \sum_{l=-1}^{1} C_{i,j+l}$ et des sommes de lignes verticales $V_j = \sum_{k=-1}^{1} C_{i+k,j}$), reconstruit la droite associée sur le bloc 3×3 et retient celle qui minimise l'erreur quadratique $\sum_{k,l=-1}^{1} (C^{\text{rec}}_{i+k,j+l}(\mathbf{n}) - C_{i+k,j+l})^2$. Cet algorithme reconstruit exactement toute interface plane quelle que soit son orientation.

— **LVIRA (Puckett 1991)** : affine l'angle $\theta = \text{atan2}(n_y, n_x)$ de la normale ELVIRA par recherche locale d'or / sécante minimisant la fonctionnelle d'erreur $E_2(\theta)$ sur le stencil 3×3.

10.4 Répertoire des Classes, Structures et Méthodes de `include/covof/Interf`

Ligne	Catégorie	Identifiant C++17	Signature / Rôle
13	enum class	`ReconstructionMethod`	Déclaration de type / classe / structure
23	enum class	`AdvectionMethod`	Déclaration de type / classe / structure
32	enum class	`TvdLimiter`	Déclaration de type / classe / structure
43	struct	`InterfaceSegment`	Déclaration de type / classe / structure
52	class	`InterfaceReconstruction`	Déclaration de type / classe / structure
57	Méthode / Fonction	`parse_reconstruction_method`	Type retour : static ReconstructionMethod
62	Méthode / Fonction	`parse_advection_method`	Type retour : static AdvectionMethod
67	Méthode / Fonction	`parse_tvd_limiter`	Type retour : static TvdLimiter
116	Méthode / Fonction	`evaluate_limiter`	Type retour : static double

10.5 Code Source Intégral Annoté : `include/covof/InterfaceReconstruction.`

Le listing ci-dessous présente l'intégralité des 119 lignes du fichier `include/covof/InterfaceReconstruction.hpp` de l'implémentation C++17 :

```cpp
#pragma once

#include "Mesh.hpp"
#include <string>
#include <vector>

namespace covof {
namespace fugu = covof; // Alias rétrocompatibilité

/**
 * @brief Énumération des méthodes de reconstruction géométrique d'interface
 */
enum class ReconstructionMethod {
    SLIC,    ///< Simple Line Interface Calculation (Noh & Woodward 1976)
    PLIC,    ///< Piecewise Linear Interface Calculation (Youngs 1982 / Scardovelli & Zaleski 1999)
    LVIRA,   ///< Least Squares VOF Interface Reconstruction Algorithm (Pilliod & Puckett 2004)
    ELVIRA   ///< Efficient LVIRA avec 6 candidats discrets (Pilliod & Puckett 2004)
};

/**
 * @brief Énumération des algorithmes d'advection VOF
 */
enum class AdvectionMethod {
    VOF_TVD,       ///< Advection algébrique Volumes Finis TVD haute résolution
    CIAM,          ///< Calcul d'Interface Affine par Morceaux (Strang splitting directionnel)
    WEYMOUTH_YUE   ///< Advection géométrique unsplit multidimensionnelle (Weymouth & Yue 2010)
};

/**
 * @brief Énumération des limiteurs de pente TVD / NVD pour l'advection d'interface
 */
enum class TvdLimiter {
    SUPERBEE,  ///< Limiteur Superbee de Roe (hautement compressif pour interfaces)
    VAN_LEER,  ///< Limiteur harmonique doux de Van Leer
    MUSCL,     ///< Limiteur linéaire par morceaux MUSCL
    HRIC       ///< High Resolution Interface Capturing (Muzaferija & Peric 1999)
};

/**
 * @brief Structure représentant un segment de droite d'interface affine dans une cellule
 * Équation locale : n_x * x + n_y * y + alpha = 0, orientée vers le fluide 2 (gaz)
 */
struct InterfaceSegment {
    double nx = 0.0;    ///< Composante normale unitaire selon X
    double ny = 0.0;    ///< Composante normale unitaire selon Y
    double alpha = 0.0; ///< Paramètre affine de position de l'interface
};

/**
 * @brief Classe statique fournissant les algorithmes géométriques de reconstruction et de découpe
 */
class InterfaceReconstruction {
public:
    /**
     * @brief Convertit une chaîne de caractères en méthode de reconstruction
     */
    static ReconstructionMethod parse_reconstruction_method(const std::string& str);

    /**
     * @brief Convertit une chaîne de caractères en méthode d'advection
     */
    static AdvectionMethod parse_advection_method(const std::string& str);

    /**
     * @brief Convertit une chaîne de caractères en limiteur TVD
     */
```

```
    static TvdLimiter parse_tvd_limiter(const std::string& str);

    /**
     * @brief Calcule l'aire exacte de l'intersection entre un rectangle [xmin, xmax] x [ymin, ymax]
     *        et le demi-plan fluide nx * x + ny * y + alpha <= 0 (Sutherland-Hodgman + Gauss Shoelace)
     */
    static double cut_box_area(double xmin, double xmax, double ymin, double ymax,
                               double nx, double ny, double alpha);

    /**
     * @brief Calcule le paramètre affine alpha conservant exactement le volume cible C_target * dx * dy
     */
    static double compute_alpha(double xmin, double xmax, double ymin, double ymax,
                                double nx, double ny, double C_target);

    /**
     * @brief Reconstruction SLIC (Simple Line Interface Calculation) sur l'ensemble du maillage
     */
    static void reconstruct_slic(const Mesh2D& mesh, const std::vector<double>& C,
                                 std::vector<InterfaceSegment>& segments);

    /**
     * @brief Reconstruction PLIC (Youngs + Scardovelli & Zaleski) sur l'ensemble du maillage
     */
    static void reconstruct_plic(const Mesh2D& mesh, const std::vector<double>& C,
                                 std::vector<InterfaceSegment>& segments);

    /**
     * @brief Reconstruction LVIRA (Least Squares VOF Interface Reconstruction Algorithm)
     */
    static void reconstruct_lvira(const Mesh2D& mesh, const std::vector<double>& C,
                                  std::vector<InterfaceSegment>& segments);

    /**
     * @brief Reconstruction ELVIRA (Efficient LVIRA avec 6 candidats de fonctions de hauteur)
     */
    static void reconstruct_elvira(const Mesh2D& mesh, const std::vector<double>& C,
                                   std::vector<InterfaceSegment>& segments);

    /**
     * @brief Point d'entrée générique de reconstruction selon la méthode configurée
     */
    static void reconstruct(ReconstructionMethod method, const Mesh2D& mesh,
                            const std::vector<double>& C,
                            std::vector<InterfaceSegment>& segments);

    /**
     * @brief Évalue le limiteur de flux TVD / NVD sélectionné
     */
    static double evaluate_limiter(TvdLimiter limiter, double r, double c_donor = 0.5, double c_upwind = 0.0,
        double c_acceptor = 1.0);
};

} // namespace covof
```

10.6 Répertoire des Classes, Structures et Méthodes de `src/InterfaceReconst`

Ligne	Catégorie	Identifiant C++17	Signature / Rôle
14	Méthode / Fonction	`InterfaceReconstruction::parse_reconstruction_method`	Type retour : ReconstructionMethod
21	Méthode / Fonction	`InterfaceReconstruction::parse_advection_method`	Type retour : AdvectionMethod
27	Méthode / Fonction	`InterfaceReconstruction::parse_tvd_limiter`	Type retour : TvdLimiter
46	struct	`Point`	Déclaration de type / classe / structu
535	Méthode / Fonction	`InterfaceReconstruction::evaluate_limiter`	Type retour : double

10.7 Code Source Intégral Annoté : `src/InterfaceReconstruction.cpp`

Le listing ci-dessous présente l'intégralité des 576 lignes du fichier `src/InterfaceReconstruction.cpp` de l'implémentation C++17 :

```
#include "covof/InterfaceReconstruction.hpp"
#include <cmath>
#include <algorithm>
#include <array>
#include <iostream>

namespace covof {
namespace fugu = covof; // Alias rétrocompatibilité

// ============================================================================
// PARSING DES MÉTHODES ET LIMITATEURS
// ============================================================================
```

```cpp

ReconstructionMethod InterfaceReconstruction::parse_reconstruction_method(const std::string& str) {
    if (str == "SLIC")   return ReconstructionMethod::SLIC;
    if (str == "LVIRA")  return ReconstructionMethod::LVIRA;
    if (str == "ELVIRA") return ReconstructionMethod::ELVIRA;
    return ReconstructionMethod::PLIC; // Défaut PLIC
}

AdvectionMethod InterfaceReconstruction::parse_advection_method(const std::string& str) {
    if (str == "CIAM")         return AdvectionMethod::CIAM;
    if (str == "WEYMOUTH_YUE") return AdvectionMethod::WEYMOUTH_YUE;
    return AdvectionMethod::VOF_TVD; // Défaut VOF-TVD
}

TvdLimiter InterfaceReconstruction::parse_tvd_limiter(const std::string& str) {
    if (str == "VAN_LEER") return TvdLimiter::VAN_LEER;
    if (str == "MUSCL")    return TvdLimiter::MUSCL;
    if (str == "HRIC")     return TvdLimiter::HRIC;
    return TvdLimiter::SUPERBEE; // Défaut Superbee
}

// ============================================================================
// GÉOMÉTRIE : DÉCOUPE D'UN RECTANGLE PAR UN DEMI-PLAN (SUTHERLAND-HODGMAN)
// ============================================================================

double InterfaceReconstruction::cut_box_area(double xmin, double xmax, double ymin, double ymax,
                                             double nx, double ny, double alpha) {
    // Vérification des dimensions de la boîte de contrôle
    double dx = xmax - xmin;
    double dy = ymax - ymin;
    if (dx <= 1.0e-14 || dy <= 1.0e-14) return 0.0;

    // Définition des 4 sommets du rectangle dans l'ordre trigonométrique
    struct Point { double x, y; };
    std::array<Point, 4> box_pts = {{
        {xmin, ymin},
        {xmax, ymin},
        {xmax, ymax},
        {xmin, ymax}
    }};

    // Calcul de la distance signée d(x, y) = nx * x + ny * y + alpha
    auto dist = [&](const Point& p) {
        return nx * p.x + ny * p.y + alpha;
    };

    // Polygone résultant de la troncature
    std::vector<Point> clipped;
    clipped.reserve(8);

    // Algorithme de Sutherland-Hodgman contre le demi-plan fluide d(x, y) <= 0
    for (size_t i = 0; i < 4; ++i) {
        Point p1 = box_pts[i];
        Point p2 = box_pts[(i + 1) % 4];
        double d1 = dist(p1);
        double d2 = dist(p2);

        if (d1 <= 0.0) {
            if (d2 <= 0.0) {
                // Les deux points sont à l'intérieur du fluide
                clipped.push_back(p2);
            } else {
                // Sortie du fluide : calcul du point d'intersection
                double t = -d1 / (d2 - d1);
                clipped.push_back({p1.x + t * (p2.x - p1.x), p1.y + t * (p2.y - p1.y)});
            }
        } else {
            if (d2 <= 0.0) {
                // Entrée dans le fluide : calcul de l'intersection puis ajout de p2
                double t = -d1 / (d2 - d1);
                clipped.push_back({p1.x + t * (p2.x - p1.x), p1.y + t * (p2.y - p1.y)});
                clipped.push_back(p2);
            }
            // Sinon les deux points sont hors du fluide, rien à ajouter
        }
    }

    // Si moins de 3 sommets, l'aire découpée est nulle
    if (clipped.size() < 3) return 0.0;

    // Calcul de l'aire par la formule de Green / Gauss (Shoelace formula)
    double area = 0.0;
    size_t m = clipped.size();
    for (size_t i = 0; i < m; ++i) {
        size_t next = (i + 1) % m;
        area += clipped[i].x * clipped[next].y - clipped[next].x * clipped[i].y;
    }
    return 0.5 * std::abs(area);
}

// ============================================================================
// INVERSION NUMÉRIQUE EXACTE : FRACTION VOLUMIQUE -> PARAMÈTRE ALPHA
```

```cpp
// ============================================================================

double InterfaceReconstruction::compute_alpha(double xmin, double xmax, double ymin, double ymax,
                                              double nx, double ny, double C_target) {
    double total_area = (xmax - xmin) * (ymax - ymin);
    if (C_target <= 1.0e-7) return 1.0e9;   // Cellule quasi-vide
    if (C_target >= 1.0 - 1.0e-7) return -1.0e9; // Cellule quasi-pleine

    double target_area = std::clamp(C_target, 0.0, 1.0) * total_area;

    // Bornes extrêmes pour alpha
    // Les 4 sommets donnent l'étendue maximale de nx*x + ny*y
    std::array<double, 4> vals = {{
        nx * xmin + ny * ymin,
        nx * xmax + ny * ymin,
        nx * xmax + ny * ymax,
        nx * xmin + ny * ymax
    }};
    double min_val = *std::min_element(vals.begin(), vals.end());
    double max_val = *std::max_element(vals.begin(), vals.end());

    // Pour alpha_low = -max_val, nx*x + ny*y + alpha <= 0 contient tout -> aire = total_area
    // Pour alpha_high = -min_val, nx*x + ny*y + alpha <= 0 est vide -> aire = 0
    double alpha_low = -max_val - 1.0e-12;
    double alpha_high = -min_val + 1.0e-12;

    // Méthode de dichotomie / bisection pour trouver alpha exact à 1.0e-12 (a décroît avec alpha)
    for (int iter = 0; iter < 45; ++iter) {
        double alpha_mid = 0.5 * (alpha_low + alpha_high);
        double a = cut_box_area(xmin, xmax, ymin, ymax, nx, ny, alpha_mid);
        if (a < target_area) {
            alpha_high = alpha_mid;
        } else {
            alpha_low = alpha_mid;
        }
    }
    return 0.5 * (alpha_low + alpha_high);
}

// ============================================================================
// RECONSTRUCTION SLIC (NOH & WOODWARD 1976)
// ============================================================================

void InterfaceReconstruction::reconstruct_slic(const Mesh2D& mesh, const std::vector<double>& C,
                                               std::vector<InterfaceSegment>& segments) {
    int nx = mesh.nx_local;
    int ny = mesh.ny_local;

    for (int j = 0; j <= ny + 1; ++j) {
        for (int i = 0; i <= nx + 1; ++i) {
            int idx = mesh.cell_idx(i, j);
            double c = C[idx];

            if (c <= 1.0e-6) {
                segments[idx] = {1.0, 0.0, 1.0e9};
                continue;
            }
            if (c >= 1.0 - 1.0e-6) {
                segments[idx] = {1.0, 0.0, -1.0e9};
                continue;
            }

            double xmin = mesh.xv(i);
            double xmax = mesh.xv(i + 1);
            double ymin = mesh.yv(j);
            double ymax = mesh.yv(j + 1);

            double snx = 1.0, sny = 0.0;
            if (i >= 1 && i <= nx && j >= 1 && j <= ny) {
                double grad_x = std::abs(C[mesh.cell_idx(i + 1, j)] - C[mesh.cell_idx(i - 1, j)]);
                double grad_y = std::abs(C[mesh.cell_idx(i, j + 1)] - C[mesh.cell_idx(i, j - 1)]);

                if (grad_x >= grad_y) {
                    if (C[mesh.cell_idx(i - 1, j)] >= C[mesh.cell_idx(i + 1, j)]) {
                        snx = 1.0; sny = 0.0;
                    } else {
                        snx = -1.0; sny = 0.0;
                    }
                } else {
                    if (C[mesh.cell_idx(i, j - 1)] >= C[mesh.cell_idx(i, j + 1)]) {
                        snx = 0.0; sny = 1.0;
                    } else {
                        snx = 0.0; sny = -1.0;
                    }
                }
            }

            double alpha = compute_alpha(xmin, xmax, ymin, ymax, snx, sny, c);
            segments[idx] = {snx, sny, alpha};
        }
    }
}
```

```cpp

// ============================================================================
// RECONSTRUCTION PLIC (YOUNGS 1982)
// ============================================================================

void InterfaceReconstruction::reconstruct_plic(const Mesh2D& mesh, const std::vector<double>& C,
                                               std::vector<InterfaceSegment>& segments) {
    int nx = mesh.nx_local;
    int ny = mesh.ny_local;
    double dx = mesh.dx;
    double dy = mesh.dy;

    for (int j = 0; j <= ny + 1; ++j) {
        for (int i = 0; i <= nx + 1; ++i) {
            int idx = mesh.cell_idx(i, j);
            double c = C[idx];

            if (c <= 1.0e-6) {
                segments[idx] = {1.0, 0.0, 1.0e9};
                continue;
            }
            if (c >= 1.0 - 1.0e-6) {
                segments[idx] = {1.0, 0.0, -1.0e9};
                continue;
            }

            double mx = 1.0, my = 0.0;
            if (i >= 1 && i <= nx && j >= 1 && j <= ny) {
                // Calcul du gradient de Youngs sur stencil 3x3 centré
                double c_mm = C[mesh.cell_idx(i - 1, j - 1)];
                double c_0m = C[mesh.cell_idx(i,     j - 1)];
                double c_pm = C[mesh.cell_idx(i + 1, j - 1)];
                double c_m0 = C[mesh.cell_idx(i - 1, j    )];
                double c_p0 = C[mesh.cell_idx(i + 1, j    )];
                double c_mp = C[mesh.cell_idx(i - 1, j + 1)];
                double c_0p = C[mesh.cell_idx(i,     j + 1)];
                double c_pp = C[mesh.cell_idx(i + 1, j + 1)];

                mx = -((c_pp + 2.0 * c_p0 + c_pm) - (c_mp + 2.0 * c_m0 + c_mm)) / (8.0 * dx);
                my = -((c_pp + 2.0 * c_0p + c_mp) - (c_pm + 2.0 * c_0m + c_mm)) / (8.0 * dy);
            }

            double norm = std::hypot(mx, my);
            double pnx = (norm > 1.0e-12) ? mx / norm : 1.0;
            double pny = (norm > 1.0e-12) ? my / norm : 0.0;

            double xmin = mesh.xv(i);
            double xmax = mesh.xv(i + 1);
            double ymin = mesh.yv(j);
            double ymax = mesh.yv(j + 1);

            double alpha = compute_alpha(xmin, xmax, ymin, ymax, pnx, pny, c);
            segments[idx] = {pnx, pny, alpha};
        }
    }
}

// ============================================================================
// RECONSTRUCTION ELVIRA (PILLIOD & PUCKETT 2004)
// ============================================================================

void InterfaceReconstruction::reconstruct_elvira(const Mesh2D& mesh, const std::vector<double>& C,
                                                 std::vector<InterfaceSegment>& segments) {
    int nx = mesh.nx_local;
    int ny = mesh.ny_local;
    double dx = mesh.dx;
    double dy = mesh.dy;

    for (int j = 0; j <= ny + 1; ++j) {
        for (int i = 0; i <= nx + 1; ++i) {
            int idx = mesh.cell_idx(i, j);
            double c = C[idx];

            if (c <= 1.0e-6) {
                segments[idx] = {1.0, 0.0, 1.0e9};
                continue;
            }
            if (c >= 1.0 - 1.0e-6) {
                segments[idx] = {1.0, 0.0, -1.0e9};
                continue;
            }
            if (i < 1 || i > nx || j < 1 || j > ny) {
                double xmin = mesh.xv(i);
                double xmax = mesh.xv(i + 1);
                double ymin = mesh.yv(j);
                double ymax = mesh.yv(j + 1);
                segments[idx] = {1.0, 0.0, compute_alpha(xmin, xmax, ymin, ymax, 1.0, 0.0, c)};
                continue;
            }

            double xmin = mesh.xv(i);
            double xmax = mesh.xv(i + 1);
```

```
            double ymin = mesh.yv(j);
            double ymax = mesh.yv(j + 1);

            // 1. Hauteurs de colonnes H_k (sommes sur y)
            double H_m1 = (C[mesh.cell_idx(i - 1, j - 1)] + C[mesh.cell_idx(i - 1, j)] + C[mesh.cell_idx(i - 1, j + 1)]) * dy;
            double H_0  = (C[mesh.cell_idx(i,     j - 1)] + C[mesh.cell_idx(i,     j)] + C[mesh.cell_idx(i,     j + 1)]) * dy;
            double H_p1 = (C[mesh.cell_idx(i + 1, j - 1)] + C[mesh.cell_idx(i + 1, j)] + C[mesh.cell_idx(i + 1, j + 1)]) * dy;

            // 2. Largeurs de lignes W_l (sommes sur x)
            double W_m1 = (C[mesh.cell_idx(i - 1, j - 1)] + C[mesh.cell_idx(i, j - 1)] + C[mesh.cell_idx(i + 1, j - 1)]) * dx;
            double W_0  = (C[mesh.cell_idx(i - 1, j    )] + C[mesh.cell_idx(i, j    )] + C[mesh.cell_idx(i + 1, j    )]) * dx;
            double W_p1 = (C[mesh.cell_idx(i - 1, j + 1)] + C[mesh.cell_idx(i, j + 1)] + C[mesh.cell_idx(i + 1, j + 1)]) * dx;

            // Détermination du signe de la normale
            double sgn_y = (C[mesh.cell_idx(i, j + 1)] < C[mesh.cell_idx(i, j - 1)]) ? 1.0 : -1.0;
            double sgn_x = (C[mesh.cell_idx(i + 1, j)] < C[mesh.cell_idx(i - 1, j)]) ? 1.0 : -1.0;

            // Ensemble des candidats de pentes
            std::vector<std::pair<double, double>> candidates;
            candidates.reserve(7);

            // Candidats verticaux (Height functions par colonnes)
            candidates.push_back({-sgn_y * (H_p1 - H_m1) / (2.0 * dx), sgn_y * 1.0}); // Centré
            candidates.push_back({-sgn_y * (H_p1 - H_0)  / (dx),       sgn_y * 1.0}); // Décentré avant
            candidates.push_back({-sgn_y * (H_0  - H_m1) / (dx),       sgn_y * 1.0}); // Décentré arrière

            // Candidats horizontaux (Height functions par lignes)
            candidates.push_back({sgn_x * 1.0, -sgn_x * (W_p1 - W_m1) / (2.0 * dy)}); // Centré
            candidates.push_back({sgn_x * 1.0, -sgn_x * (W_p1 - W_0)  / (dy)});       // Décentré avant
            candidates.push_back({sgn_x * 1.0, -sgn_x * (W_0  - W_m1) / (dy)});       // Décentré arrière

            // Candidat Youngs standard
            double mx = -((C[mesh.cell_idx(i+1,j+1)] + 2.0*C[mesh.cell_idx(i+1,j)] + C[mesh.cell_idx(i+1,j-1)]) -
                          (C[mesh.cell_idx(i-1,j+1)] + 2.0*C[mesh.cell_idx(i-1,j)] + C[mesh.cell_idx(i-1,j-1)])) / (8.0 * dx);
            double my = -((C[mesh.cell_idx(i+1,j+1)] + 2.0*C[mesh.cell_idx(i,j+1)] + C[mesh.cell_idx(i-1,j+1)]) -
                          (C[mesh.cell_idx(i+1,j-1)] + 2.0*C[mesh.cell_idx(i,j-1)] + C[mesh.cell_idx(i-1,j-1)])) / (8.0 * dy);
            candidates.push_back({mx, my});

            // Évaluation de l'erreur quadratique pour chaque candidat sur le stencil 3x3
            double best_err = 1.0e30;
            double best_nx = 1.0, best_ny = 0.0, best_alpha = 0.0;

            for (const auto& cand : candidates) {
                double cnorm = std::hypot(cand.first, cand.second);
                if (cnorm < 1.0e-12) continue;
                double cnx = cand.first / cnorm;
                double cny = cand.second / cnorm;

                double calpha = compute_alpha(xmin, xmax, ymin, ymax, cnx, cny, c);

                // Somme des carrés des erreurs sur les 8 voisins
                double err = 0.0;
                for (int dj = -1; dj <= 1; ++dj) {
                    for (int di = -1; di <= 1; ++di) {
                        if (di == 0 && dj == 0) continue;
                        double v_xmin = mesh.xv(i + di);
                        double v_xmax = mesh.xv(i + di + 1);
                        double v_ymin = mesh.yv(j + dj);
                        double v_ymax = mesh.yv(j + dj + 1);

                        double v_target = C[mesh.cell_idx(i + di, j + dj)] * dx * dy;
                        double v_recon  = cut_box_area(v_xmin, v_xmax, v_ymin, v_ymax, cnx, cny, calpha);
                        double diff = v_recon - v_target;
                        err += diff * diff;
                    }
                }

                if (err < best_err) {
                    best_err = err;
                    best_nx = cnx;
                    best_ny = cny;
                    best_alpha = calpha;
                }
            }

            segments[idx] = {best_nx, best_ny, best_alpha};
        }
    }
}

// ============================================================================
// RECONSTRUCTION LVIRA (PILLIOD & PUCKETT 2004)
```

```cpp
// ============================================================================

void InterfaceReconstruction::reconstruct_lvira(const Mesh2D& mesh, const std::vector<double>& C,
                                                std::vector<InterfaceSegment>& segments) {
    // 1. Initialisation par ELVIRA pour obtenir un excellent point de départ
    reconstruct_elvira(mesh, C, segments);

    int nx = mesh.nx_local;
    int ny = mesh.ny_local;
    double dx = mesh.dx;
    double dy = mesh.dy;

    // 2. Raffinement par optimisation continue locale de l'angle theta
    for (int j = 1; j <= ny; ++j) {
        for (int i = 1; i <= nx; ++i) {
            int idx = mesh.cell_idx(i, j);
            double c = C[idx];
            if (c <= 1.0e-6 || c >= 1.0 - 1.0e-6) continue;

            double xmin = mesh.xv(i);
            double xmax = mesh.xv(i + 1);
            double ymin = mesh.yv(j);
            double ymax = mesh.yv(j + 1);

            double theta0 = std::atan2(segments[idx].ny, segments[idx].nx);

            // Fonction coût : erreur quadratique pour un angle theta donné
            auto cost_func = [&](double theta) {
                double lnx = std::cos(theta);
                double lny = std::sin(theta);
                double lalpha = compute_alpha(xmin, xmax, ymin, ymax, lnx, lny, c);

                double err = 0.0;
                for (int dj = -1; dj <= 1; ++dj) {
                    for (int di = -1; di <= 1; ++di) {
                        if (di == 0 && dj == 0) continue;
                        double v_xmin = mesh.xv(i + di);
                        double v_xmax = mesh.xv(i + di + 1);
                        double v_ymin = mesh.yv(j + dj);
                        double v_ymax = mesh.yv(j + dj + 1);

                        double v_target = C[mesh.cell_idx(i + di, j + dj)] * dx * dy;
                        double v_recon  = cut_box_area(v_xmin, v_xmax, v_ymin, v_ymax, lnx, lny, lalpha);
                        double diff = v_recon - v_target;
                        err += diff * diff;
                    }
                }
                return err;
            };

            // Recherche par Section Dorée (Golden Section Search) dans [theta0 - pi/6, theta0 + pi/6]
            const double invphi = (std::sqrt(5.0) - 1.0) / 2.0; // 0.6180339887...
            const double invphi2 = (3.0 - std::sqrt(5.0)) / 2.0; // 0.3819660113...

            double a = theta0 - 0.5;
            double b = theta0 + 0.5;
            double h = b - a;

            double c_pt = a + invphi2 * h;
            double d_pt = a + invphi * h;
            double fc = cost_func(c_pt);
            double fd = cost_func(d_pt);

            for (int iter = 0; iter < 12; ++iter) {
                if (fc < fd) {
                    b = d_pt;
                    d_pt = c_pt;
                    fd = fc;
                    h = invphi * h;
                    c_pt = a + invphi2 * h;
                    fc = cost_func(c_pt);
                } else {
                    a = c_pt;
                    c_pt = d_pt;
                    fc = fd;
                    h = invphi * h;
                    d_pt = a + invphi * h;
                    fd = cost_func(d_pt);
                }
            }

            double best_theta = 0.5 * (a + b);
            double best_nx = std::cos(best_theta);
            double best_ny = std::sin(best_theta);
            double best_alpha = compute_alpha(xmin, xmax, ymin, ymax, best_nx, best_ny, c);

            segments[idx] = {best_nx, best_ny, best_alpha};
        }
    }
}

// ============================================================================
```

```cpp
// DISPATCHING GÉNÉRIQUE
// ============================================================================

void InterfaceReconstruction::reconstruct(ReconstructionMethod method, const Mesh2D& mesh,
                                          const std::vector<double>& C,
                                          std::vector<InterfaceSegment>& segments) {
    switch (method) {
        case ReconstructionMethod::SLIC:
            reconstruct_slic(mesh, C, segments);
            break;
        case ReconstructionMethod::PLIC:
            reconstruct_plic(mesh, C, segments);
            break;
        case ReconstructionMethod::LVIRA:
            reconstruct_lvira(mesh, C, segments);
            break;
        case ReconstructionMethod::ELVIRA:
            reconstruct_elvira(mesh, C, segments);
            break;
    }

    // Propagation des normales reconstruites vers les mailles fantomes (periodicite / bords)
    int nx = mesh.nx_local;
    int ny = mesh.ny_local;
    int i_left_src  = (!mesh.is_boundary_left  && mesh.num_procs == 1) ? nx : 1;
    int i_right_src = (!mesh.is_boundary_right && mesh.num_procs == 1) ? 1  : nx;
    int j_bot_src   = (!mesh.is_boundary_bottom && mesh.num_procs == 1) ? ny : 1;
    int j_top_src   = (!mesh.is_boundary_top    && mesh.num_procs == 1) ? 1  : ny;

    for (int j = 1; j <= ny; ++j) {
        // Maille fantome gauche i = 0
        int idx0 = mesh.cell_idx(0, j);
        double c0 = C[idx0];
        if (c0 > 1.0e-6 && c0 < 1.0 - 1.0e-6) {
            const auto& s_src = segments[mesh.cell_idx(i_left_src, j)];
            double alpha0 = compute_alpha(mesh.xv(0), mesh.xv(1), mesh.yv(j), mesh.yv(j + 1), s_src.nx, s_src.
                ny, c0);
            segments[idx0] = {s_src.nx, s_src.ny, alpha0};
        }
        // Maille fantome droite i = nx + 1
        int idxR = mesh.cell_idx(nx + 1, j);
        double cR = C[idxR];
        if (cR > 1.0e-6 && cR < 1.0 - 1.0e-6) {
            const auto& s_src = segments[mesh.cell_idx(i_right_src, j)];
            double alphaR = compute_alpha(mesh.xv(nx + 1), mesh.xv(nx + 2), mesh.yv(j), mesh.yv(j + 1), s_src.
                nx, s_src.ny, cR);
            segments[idxR] = {s_src.nx, s_src.ny, alphaR};
        }
    }

    for (int i = 0; i <= nx + 1; ++i) {
        // Maille fantome inferieure j = 0
        int idxB = mesh.cell_idx(i, 0);
        double cB = C[idxB];
        if (cB > 1.0e-6 && cB < 1.0 - 1.0e-6) {
            const auto& s_src = segments[mesh.cell_idx(i, j_bot_src)];
            double alphaB = compute_alpha(mesh.xv(i), mesh.xv(i + 1), mesh.yv(0), mesh.yv(1), s_src.nx, s_src.
                ny, cB);
            segments[idxB] = {s_src.nx, s_src.ny, alphaB};
        }
        // Maille fantome superieure j = ny + 1
        int idxT = mesh.cell_idx(i, ny + 1);
        double cT = C[idxT];
        if (cT > 1.0e-6 && cT < 1.0 - 1.0e-6) {
            const auto& s_src = segments[mesh.cell_idx(i, j_top_src)];
            double alphaT = compute_alpha(mesh.xv(i), mesh.xv(i + 1), mesh.yv(ny + 1), mesh.yv(ny + 2), s_src.
                nx, s_src.ny, cT);
            segments[idxT] = {s_src.nx, s_src.ny, alphaT};
        }
    }
}

// ============================================================================
// ÉVALUATION DES LIMITEURS DE FLUX (SUPERBEE, VAN LEER, MUSCL, HRIC)
// ============================================================================

double InterfaceReconstruction::evaluate_limiter(TvdLimiter limiter, double r, double c_donor, double c_upwind
    , double c_acceptor) {
    switch (limiter) {
        case TvdLimiter::SUPERBEE:
            // Limiteur Superbee de Roe (1985)
            return std::max(0.0, std::max(std::min(2.0 * r, 1.0), std::min(r, 2.0)));

        case TvdLimiter::VAN_LEER:
            // Limiteur harmonique de Van Leer (1974)
            return (r > 0.0) ? (2.0 * r / (1.0 + r)) : 0.0;

        case TvdLimiter::MUSCL:
            // Limiteur linéaire par morceaux MUSCL (Van Leer 1979)
            return std::max(0.0, std::min({2.0 * r, 0.5 * (1.0 + r), 2.0}));

        case TvdLimiter::HRIC: {
```

```
            // High Resolution Interface Capturing (Muzaferija & Peric 1999) dans l'espace NVD
            double denom = c_acceptor - c_upwind;
            if (std::abs(denom) < 1.0e-12) return 0.0;
            double c_tilde_D = (c_donor - c_upwind) / denom;

            double c_tilde_f = c_tilde_D;
            if (c_tilde_D >= 0.0 && c_tilde_D <= 1.0) {
                if (c_tilde_D < 0.5) {
                    c_tilde_f = 2.0 * c_tilde_D;
                } else {
                    c_tilde_f = 1.0;
                }
            }

            // Conversion de la valeur normalisée à la face en limiteur équivalent phi(r)
            // C_f = C_D + 0.5 * phi * (1 - C_D) dans le formalisme NVD
            if (std::abs(1.0 - c_tilde_D) > 1.0e-12) {
                double phi = 2.0 * (c_tilde_f - c_tilde_D) / (1.0 - c_tilde_D);
                return std::clamp(phi, 0.0, 2.0);
            }
            return 1.0;
        }
    }
    return 0.0;
}

} // namespace covof
```

Chapitre 11

Transport et Advection Conservatrice VOF : Weymouth-Yue, CIAM et TVD (InterfaceTracker.hpp/.cpp)

11.1 Séparation Directionnelle d'Ordre 2 de Strang (*Strang Operator Splitting*)

La classe `covof::InterfaceTracker` (`include/covof/InterfaceTracker.hpp`, `src/InterfaceTracker.cpp`, 833 lignes) résout l'équation de transport de la fraction volumique :

$$\frac{\partial C}{\partial t} + \frac{\partial (uC)}{\partial x} + \frac{\partial (vC)}{\partial y} = C\left(\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y}\right) \tag{11.1}$$

en alternant l'ordre des balayages unidimensionnels entre les pas de temps pairs (X puis Y) et impairs (Y puis X) afin d'éliminer tout biais directionnel et de garantir l'ordre 2 temporel.

11.2 Schéma de Weymouth & Yue (2010) à Conservation de Masse Exacte

Lors d'un balayage 1D individuel (par exemple selon x), le champ de vitesse 1D n'est pas à divergence nulle ($\partial u/\partial x \neq 0$, seule la somme $\partial u/\partial x + \partial v/\partial y = 0$ est nulle). Weymouth & Yue (2010) démontrent que si le coefficient C_{dil} multipliant les termes de dilatation 1D est fixé à la valeur binaire évaluée au début du pas de temps :

$$C_{\text{dil}}(i,j) = \begin{cases} 1 & \text{si } C^n_{i,j} \geq 0.5 \\ 0 & \text{si } C^n_{i,j} < 0.5 \end{cases} \tag{11.2}$$

et maintenu identique durant la passe X et la passe Y :

$$C^*_{i,j} = C^n_{i,j} - \frac{\Delta t}{\Delta x}\left(F^x_{i+1/2,j} - F^x_{i-1/2,j}\right) + C_{\text{dil}}(i,j)\frac{\Delta t}{\Delta x}\left(u_{i+1/2,j} - u_{i-1/2,j}\right) \tag{11.3}$$

$$C^{n+1}_{i,j} = C^*_{i,j} - \frac{\Delta t}{\Delta y}\left(F^y_{i,j+1/2} - F^y_{i,j-1/2}\right) + C_{\text{dil}}(i,j)\frac{\Delta t}{\Delta y}\left(v_{i,j+1/2} - v_{i,j-1/2}\right) \tag{11.4}$$

alors, d'une part $C^{n+1}_{i,j} \in [0,1]$ est garanti sans écrêtage (*no-clipping*) sous CFL < 0.5, et d'autre part la somme des deux termes de dilatation vaut exactement $C_{\text{dil}}\Delta t(\nabla_h \cdot \mathbf{u})_{i,j} = 0$, assurant une **conservation de la masse exacte au bit près**.

11.3 Répertoire des Classes, Structures et Méthodes de `include/covof/Interf`

Ligne	Catégorie	Identifiant C++17	Signature / Rôle
14	class	`InterfaceTracker`	Déclaration de type / classe / structure
43	Méthode / Fonction	`init`	Type retour : void
69	Méthode / Fonction	`set_initial_condition`	Type retour : void
74	Méthode / Fonction	`advect`	Type retour : void
79	Méthode / Fonction	`advect_tvd`	Type retour : void
84	Méthode / Fonction	`advect_ciam`	Type retour : void
89	Méthode / Fonction	`advect_weymouth_yue`	Type retour : void
94	Méthode / Fonction	`update_physical_properties`	Type retour : void
99	Méthode / Fonction	`compute_curvature_and_CSF`	Type retour : void

11.4 Code Source Intégral Annoté : `include/covof/InterfaceTracker.hpp`

Le listing ci-dessous présente l'intégralité des 102 lignes du fichier `include/covof/InterfaceTracker.hpp` de l'implémentation C++17 :

```
#pragma once

#include "Mesh.hpp"
#include "Field.hpp"
#include "Config.hpp"
#include "InterfaceReconstruction.hpp"

namespace covof {
namespace fugu = covof; // Alias rétrocompatibilité

/**
 * @brief Module de capture d'interface et de calcul de la tension de surface (Modèle CSF)
 */
class InterfaceTracker {
public:
    FluidProperties fluid1;
    FluidProperties fluid2;
    // Déclaration de variable typée
    double sigma = 0.0;
    // Déclaration de variable typée
    bool enable_surface_tension = true;
    // Déclaration de variable typée
    bool enable_two_phase = false;
    // Déclaration de variable typée
    std::string viscosity_model = "CONSTANT";
    // Déclaration de variable typée
    double visc_a = 2.0;
    // Déclaration de variable typée
    double visc_e = 0.01;
    // Déclaration de variable typée
    double visc_mu0 = 1.85e-5;

    // Configuration VOF avancée (SLIC, PLIC, LVIRA, ELVIRA, CIAM, Weymouth & Yue, limiteurs)
    VofConfig vof_cfg;
    // Segments d'interface géométriques reconstruits
    std::vector<InterfaceSegment> interface_segments;
    // Compteur de pas pour Strang splitting alterné
    int step_count = 0;

    // Calcul et assignation numérique
    InterfaceTracker() = default;

    void init(const Config& cfg) {
        // Calcul et assignation numérique
        fluid1 = cfg.fluid1;
        // Calcul et assignation numérique
        fluid2 = cfg.fluid2;
        // Calcul et assignation numérique
        sigma = cfg.sigma;
        // Calcul et assignation numérique
        enable_surface_tension = cfg.enable_surface_tension;
        // Calcul et assignation numérique
        enable_two_phase = cfg.enable_two_phase;
        // Calcul et assignation numérique
        viscosity_model = cfg.viscosity_model;
        // Calcul et assignation numérique
        visc_a = cfg.visc_a;
        // Calcul et assignation numérique
        visc_e = cfg.visc_e;
        // Calcul et assignation numérique
        visc_mu0 = cfg.visc_mu0;
        // Options VOF
        vof_cfg = cfg.vof;
    }

    /**
```

```
     * @brief Initialise le champ de fraction volumique C selon la configuration
     */
    void set_initial_condition(const Mesh2D& mesh, const InitialConditionConfig& ic, FlowFields& fields);

    /**
     * @brief Transporte le champ C selon l'algorithme choisi (VOF-TVD, CIAM, Weymouth & Yue)
     */
    void advect(const Mesh2D& mesh, FlowFields& fields, double dt);

    /**
     * @brief Transport algébrique VOF-TVD avec limiteurs (Superbee, Van Leer, MUSCL, HRIC)
     */
    void advect_tvd(const Mesh2D& mesh, FlowFields& fields, double dt);

    /**
     * @brief Transport géométrique directionnel CIAM (Calcul d'Interface Affine par Morceaux)
     */
    void advect_ciam(const Mesh2D& mesh, FlowFields& fields, double dt);

    /**
     * @brief Transport géométrique unsplit multidimensionnel de Weymouth et Yue (2010)
     */
    void advect_weymouth_yue(const Mesh2D& mesh, FlowFields& fields, double dt);

    /**
     * @brief Met à jour les propriétés physiques effectives (rho, mu) aux centres, faces et nœuds
     */
    void update_physical_properties(const Mesh2D& mesh, FlowFields& fields);

    /**
     * @brief Calcule la normale, la courbure kappa et la force capillaire CSF équilibrée
     */
    void compute_curvature_and_CSF(const Mesh2D& mesh, FlowFields& fields);
};

} // namespace covof
```

11.5 Répertoire des Classes, Structures et Méthodes de `src/InterfaceTracker`

Ligne	Catégorie	Identifiant C++17	Signature / Rôle
9	Méthode / Fonction	`InterfaceTracker::set_initial_condition`	Type retour : void
144	Méthode / Fonction	`InterfaceTracker::update_physical_properties`	Type retour : void
269	Méthode / Fonction	`InterfaceTracker::compute_curvature_and_CSF`	Type retour : void
370	Méthode / Fonction	`InterfaceTracker::advect`	Type retour : void
405	Méthode / Fonction	`InterfaceTracker::advect_tvd`	Type retour : void
415	Méthode / Fonction	`flux_x`	Type retour : std : :vector<double>
444	Méthode / Fonction	`flux_y`	Type retour : std : :vector<double>
489	Méthode / Fonction	`InterfaceTracker::advect_ciam`	Type retour : void
501	Méthode / Fonction	`flux_vol_x`	Type retour : std : :vector<double>
561	Méthode / Fonction	`flux_vol_y`	Type retour : std : :vector<double>
633	Méthode / Fonction	`InterfaceTracker::advect_weymouth_yue`	Type retour : void
644	Méthode / Fonction	`C_d`	Type retour : std : :vector<double>
655	Méthode / Fonction	`f_vol_x`	Type retour : std : :vector<double>
698	Méthode / Fonction	`f_vol_y`	Type retour : std : :vector<double>

11.6 Code Source Intégral Annoté : `src/InterfaceTracker.cpp`

Le listing ci-dessous présente l'intégralité des 752 lignes du fichier `src/InterfaceTracker.cpp` de l'implémentation C++17 :

```
#include "covof/InterfaceTracker.hpp"
#include "covof/MpiComm.hpp"
#include <cmath>
#include <algorithm>

namespace covof {
namespace fugu = covof; // Alias rétrocompatibilité

void InterfaceTracker::set_initial_condition(const Mesh2D& mesh, const InitialConditionConfig& ic, FlowFields&
     fields) {
    // Déclaration de variable typée
    double eps = 1.5 * std::max(mesh.dx, mesh.dy);

    // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
    for (int j = 0; j <= mesh.ny_local + 1; ++j) {
        // Déclaration de variable typée
        double y = mesh.yc(j);
        // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
```

```
        for (int i = 0; i <= mesh.nx_local + 1; ++i) {
            // Déclaration de variable typée
            double x = mesh.xc(i);
            // Déclaration de variable typée
            int idx = mesh.cell_idx(i, j);

            // Évaluation de la condition de contrôle
            if (ic.type == InitialConditionType::SINGLE_PHASE) {
                // Calcul de la résolution géométrique de cellule
                fields.C[idx] = 1.0;
            // Calcul et assignation numérique
            } else if (ic.type == InitialConditionType::DROPLET) {
                // Déclaration de variable typée
                double dist = std::sqrt((x - ic.center_x) * (x - ic.center_x) +
                                        (y - ic.center_y) * (y - ic.center_y)) - ic.radius;
                // Évaluation de la condition de contrôle
                if (dist < -eps) {
                    // Calcul de la résolution géométrique de cellule
                    fields.C[idx] = 1.0;
                } else if (dist > eps) {
                    // Calcul de la résolution géométrique de cellule
                    fields.C[idx] = 0.0;
                } else {
                    // Calcul de la résolution géométrique de cellule
                    fields.C[idx] = 0.5 - dist / (2.0 * eps) - (1.0 / (2.0 * M_PI)) * std::sin(M_PI * dist /
                        eps);
                }
            // Calcul et assignation numérique
            } else if (ic.type == InitialConditionType::BUBBLE) {
                // Déclaration de variable typée
                double dist = std::sqrt((x - ic.center_x) * (x - ic.center_x) +
                                        (y - ic.center_y) * (y - ic.center_y)) - ic.radius;
                // Évaluation de la condition de contrôle
                if (dist < -eps) {
                    // Calcul de la résolution géométrique de cellule
                    fields.C[idx] = 0.0; // Intérieur bulle : fluide 2
                } else if (dist > eps) {
                    // Calcul de la résolution géométrique de cellule
                    fields.C[idx] = 1.0; // Extérieur : fluide 1 (eau)
                } else {
                    // Calcul de la résolution géométrique de cellule
                    fields.C[idx] = 0.5 + dist / (2.0 * eps) + (1.0 / (2.0 * M_PI)) * std::sin(M_PI * dist /
                        eps);
                }
            // Calcul et assignation numérique
            } else if (ic.type == InitialConditionType::DAM_BREAK) {
                // Évaluation de la condition de contrôle
                if (x >= ic.box_x0 && x <= ic.box_x1 && y >= ic.box_y0 && y <= ic.box_y1) {
                    // Calcul de la résolution géométrique de cellule
                    fields.C[idx] = 1.0;
                } else {
                    // Calcul de la résolution géométrique de cellule
                    fields.C[idx] = 0.0;
                }
            // Calcul et assignation numérique
            } else if (ic.type == InitialConditionType::SHEAR_LAYER) {
                // Déclaration de variable typée
                double int_y = ic.center_y + ic.pert_amp * std::sin(2.0 * M_PI * ic.pert_k * x / mesh.lx);
                // Calcul de la résolution géométrique de cellule
                fields.C[idx] = (y <= int_y) ? 1.0 : 0.0;
            // Calcul et assignation numérique pour nappe liquide (Cas 3.2 Fluids 2021)
            } else if (ic.type == InitialConditionType::LIQUID_SHEET) {
                // Déclaration de la position centrale et demi-épaisseur de nappe
                double y_c = (ic.center_y > 0.0) ? ic.center_y : 0.5 * mesh.ly;
                // Calcul de la demi-épaisseur de la nappe liquide
                double half_thick = 0.5 * ic.thickness;
                // Calcul de la distance géométrique par rapport à la surface de la nappe
                double dist = std::abs(y - y_c) - half_thick;
                // Évaluation de l'indicatrice : C = 0 dans le liquide (fluide 2), C = 1 dans le gaz (fluide
                    1)
                if (dist < -eps) {
                    // Calcul de la résolution géométrique de cellule
                    fields.C[idx] = 0.0;
                // Branche alternative conditionnelle
                } else if (dist > eps) {
                    // Calcul de la résolution géométrique de cellule
                    fields.C[idx] = 1.0;
                // Branche par défaut pour transition continue
                } else {
                    // Calcul de la résolution géométrique de cellule
                    fields.C[idx] = 0.5 + dist / (2.0 * eps) + (1.0 / (2.0 * M_PI)) * std::sin(M_PI * dist /
                        eps);
                }
            // Calcul et assignation numérique pour jet liquide visqueux (Cas 3.3 Fluids 2021)
            } else if (ic.type == InitialConditionType::VISCOUS_JET) {
                // Déclaration du centre et de la demi-largeur de l'injecteur
                double x_c = (ic.center_x > 0.0) ? ic.center_x : 0.5 * mesh.lx;
                // Calcul de la demi-largeur de l'injecteur
                double half_w = (ic.jet_width > 0.0) ? 0.5 * ic.jet_width : 0.04;
                // Déclaration de la hauteur initiale de l'amorce de jet sous la paroi supérieure
                double stub_height = 2.0 * mesh.dy;
                // Évaluation de l'indicatrice : C = 0 dans l'amorce liquide, C = 1 dans la cavité de gaz
```

```
                if (y >= mesh.ly - stub_height && std::abs(x - x_c) <= half_w) {
                    // Calcul de la résolution géométrique de cellule
                    fields.C[idx] = 0.0;
                } else {
                    // Calcul de la résolution géométrique de cellule
                    fields.C[idx] = 1.0;
                }
            // Calcul et assignation numérique
            } else if (ic.type == InitialConditionType::VORTEX_SHEAR) {
                // Déclaration de variable typée
                double dist = std::sqrt((x - ic.center_x) * (x - ic.center_x) +
                                        (y - ic.center_y) * (y - ic.center_y)) - ic.radius;
                // Évaluation de la condition de contrôle
                if (dist < -eps) {
                    // Calcul de la résolution géométrique de cellule
                    fields.C[idx] = 1.0;
                } else if (dist > eps) {
                    // Calcul de la résolution géométrique de cellule
                    fields.C[idx] = 0.0;
                } else {
                    // Calcul de la résolution géométrique de cellule
                    fields.C[idx] = 0.5 - dist / (2.0 * eps) - (1.0 / (2.0 * M_PI)) * std::sin(M_PI * dist /
                        eps);
                }
            // Calcul et assignation numérique
            } else if (ic.type == InitialConditionType::RAYLEIGH_TAYLOR) {
                // Déclaration de variable typée
                double int_y = ic.center_y + ic.pert_amp * std::cos(2.0 * M_PI * ic.pert_k * x / mesh.lx);
                // Calcul de la résolution géométrique de cellule
                fields.C[idx] = (y >= int_y) ? 1.0 : 0.0;
            }
        }
    }

    MpiComm::sync_cell_scalar(mesh, fields.C);
    update_physical_properties(mesh, fields);
    compute_curvature_and_CSF(mesh, fields);
}

void InterfaceTracker::update_physical_properties(const Mesh2D& mesh, FlowFields& fields) {
    // Déclaration de variable typée
    int nx = mesh.nx_local;
    // Déclaration de variable typée
    int ny = mesh.ny_local;

    // Pour un écoulement monophasique, propriétés uniformes exactes
    if (!enable_two_phase) {
        std::fill(fields.rho.begin(), fields.rho.end(), fluid1.rho);
        std::fill(fields.rho_u.begin(), fields.rho_u.end(), fluid1.rho);
        std::fill(fields.rho_v.begin(), fields.rho_v.end(), fluid1.rho);
        // Évaluation de la condition de contrôle
        if (viscosity_model == "EXPONENTIAL") {
            // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
            for (int j = 0; j <= ny + 1; ++j) {
                // Déclaration de variable typée
                double y = mesh.yc(j);
                // Déclaration de variable typée
                double z = y - visc_e;
                // Déclaration de variable typée
                double mu_val = visc_mu0 * std::exp(-visc_a * z / visc_e);
                // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
                for (int i = 0; i <= nx + 1; ++i) {
                    // Calcul de la résolution géométrique de cellule
                    fields.mu[mesh.cell_idx(i, j)] = mu_val;
                }
            }
            // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
            for (int j = 0; j <= ny + 1; ++j) {
                // Déclaration de variable typée
                double y = mesh.yv(j);
                // Déclaration de variable typée
                double z = y - visc_e;
                // Déclaration de variable typée
                double mu_node_val = visc_mu0 * std::exp(-visc_a * z / visc_e);
                // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
                for (int i = 0; i <= nx + 1; ++i) {
                    // Calcul de la résolution géométrique de cellule
                    fields.mu_node[mesh.node_idx(i, j)] = mu_node_val;
                }
            }
        } else {
            std::fill(fields.mu.begin(), fields.mu.end(), fluid1.mu);
            std::fill(fields.mu_node.begin(), fields.mu_node.end(), fluid1.mu);
        }
        // Retour de fonction
        return;
    }

    // Remplissage des coins pour C
    fields.C[mesh.cell_idx(0, 0)]           = fields.C[mesh.cell_idx(1, 1)];
    // Calcul de la résolution géométrique de cellule
    fields.C[mesh.cell_idx(nx + 1, 0)]      = fields.C[mesh.cell_idx(nx, 1)];
```

```
    // Calcul de la résolution géométrique de cellule
    fields.C[mesh.cell_idx(0, ny + 1)]      = fields.C[mesh.cell_idx(1, ny)];
    // Calcul de la résolution géométrique de cellule
    fields.C[mesh.cell_idx(nx + 1, ny + 1)]  = fields.C[mesh.cell_idx(nx, ny)];

    // 1. Masse volumique et viscosité au centre des cellules
    for (int j = 0; j <= ny + 1; ++j) {
        // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
        for (int i = 0; i <= nx + 1; ++i) {
            // Déclaration de variable typée
            int idx = mesh.cell_idx(i, j);
            // Déclaration de variable typée
            double c = std::clamp(fields.C[idx], 0.0, 1.0);
            // Calcul de la résolution géométrique de cellule
            fields.rho[idx] = c * fluid1.rho + (1.0 - c) * fluid2.rho;
            // Calcul de la résolution géométrique de cellule
            fields.mu[idx]  = c * fluid1.mu  + (1.0 - c) * fluid2.mu;
        }
    }

    // Calcul de la résolution géométrique de cellule
    fields.mu[mesh.cell_idx(0, 0)]          = fields.mu[mesh.cell_idx(1, 1)];
    // Calcul de la résolution géométrique de cellule
    fields.mu[mesh.cell_idx(nx + 1, 0)]     = fields.mu[mesh.cell_idx(nx, 1)];
    // Calcul de la résolution géométrique de cellule
    fields.mu[mesh.cell_idx(0, ny + 1)]     = fields.mu[mesh.cell_idx(1, ny)];
    // Calcul de la résolution géométrique de cellule
    fields.mu[mesh.cell_idx(nx + 1, ny + 1)]  = fields.mu[mesh.cell_idx(nx, ny)];

    // 2. Masse volumique sur les faces u
    for (int j = 1; j <= ny; ++j) {
        // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
        for (int i = 1; i <= nx + 1; ++i) {
            // Déclaration de variable typée
            int idx_u = mesh.u_idx(i, j);
            // Calcul de la résolution géométrique de cellule
            fields.rho_u[idx_u] = 0.5 * (fields.rho[mesh.cell_idx(i - 1, j)] + fields.rho[mesh.cell_idx(i, j)
                ]);
        }
    }

    // 3. Masse volumique sur les faces v
    for (int j = 1; j <= ny + 1; ++j) {
        // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
        for (int i = 1; i <= nx; ++i) {
            // Déclaration de variable typée
            int idx_v = mesh.v_idx(i, j);
            // Calcul de la résolution géométrique de cellule
            fields.rho_v[idx_v] = 0.5 * (fields.rho[mesh.cell_idx(i, j - 1)] + fields.rho[mesh.cell_idx(i, j)
                ]);
        }
    }

    // 4. Viscosité harmonique aux nœuds (coins)
    for (int j = 1; j <= ny + 1; ++j) {
        // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
        for (int i = 1; i <= nx + 1; ++i) {
            // Déclaration de variable typée
            int n_idx = mesh.node_idx(i, j);
            // Déclaration de variable typée
            double m_SW = fields.mu[mesh.cell_idx(i - 1, j - 1)];
            // Déclaration de variable typée
            double m_SE = fields.mu[mesh.cell_idx(i, j - 1)];
            // Déclaration de variable typée
            double m_NW = fields.mu[mesh.cell_idx(i - 1, j)];
            // Déclaration de variable typée
            double m_NE = fields.mu[mesh.cell_idx(i, j)];

            // Calcul de la résolution géométrique de cellule
            fields.mu_node[n_idx] = 4.0 / (1.0 / m_SW + 1.0 / m_SE + 1.0 / m_NW + 1.0 / m_NE);
        }
    }
}

void InterfaceTracker::compute_curvature_and_CSF(const Mesh2D& mesh, FlowFields& fields) {
    // Évaluation de la condition de contrôle
    if (!enable_surface_tension || sigma == 0.0) {
        std::fill(fields.F_sigma_x.begin(), fields.F_sigma_x.end(), 0.0);
        std::fill(fields.F_sigma_y.begin(), fields.F_sigma_y.end(), 0.0);
        // Retour de fonction
        return;
    }

    // Déclaration de variable typée
    int nx = mesh.nx_local;
    // Déclaration de variable typée
    int ny = mesh.ny_local;
    // Déclaration de variable typée
    double dx = mesh.dx;
    // Déclaration de variable typée
    double dy = mesh.dy;

```

```
    // 1. Calcul du gradient et du vecteur normal unitaire aux centres
    for (int j = 1; j <= ny; ++j) {
        // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
        for (int i = 1; i <= nx; ++i) {
            // Déclaration de variable typée
            int idx = mesh.cell_idx(i, j);
            // Déclaration de variable typée
            double dC_dx = (fields.C[mesh.cell_idx(i + 1, j)] - fields.C[mesh.cell_idx(i - 1, j)]) / (2.0 * dx);
            // Déclaration de variable typée
            double dC_dy = (fields.C[mesh.cell_idx(i, j + 1)] - fields.C[mesh.cell_idx(i, j - 1)]) / (2.0 * dy);
            // Déclaration de variable typée
            double grad_mag = std::sqrt(dC_dx * dC_dx + dC_dy * dC_dy);
            // Évaluation de la condition de contrôle
            if (grad_mag > 1.0e-3 / dx) {
                // Calcul de la résolution géométrique de cellule
                fields.norm_x[idx] = dC_dx / grad_mag;
                // Calcul de la résolution géométrique de cellule
                fields.norm_y[idx] = dC_dy / grad_mag;
            } else {
                // Calcul de la résolution géométrique de cellule
                fields.norm_x[idx] = 0.0;
                // Calcul de la résolution géométrique de cellule
                fields.norm_y[idx] = 0.0;
            }
        }
    }
    MpiComm::sync_cell_scalar(mesh, fields.norm_x);
    MpiComm::sync_cell_scalar(mesh, fields.norm_y);

    // 2. Calcul de la courbure kappa = -div(n)
    for (int j = 1; j <= ny; ++j) {
        // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
        for (int i = 1; i <= nx; ++i) {
            // Déclaration de variable typée
            int idx = mesh.cell_idx(i, j);
            // Évaluation de la condition de contrôle
            if (fields.norm_x[idx] == 0.0 && fields.norm_y[idx] == 0.0) {
                // Calcul de la résolution géométrique de cellule
                fields.kappa[idx] = 0.0;
            } else {
                // Déclaration de variable typée
                double dnx_dx = (fields.norm_x[mesh.cell_idx(i + 1, j)] - fields.norm_x[mesh.cell_idx(i - 1, j)]) / (2.0 * dx);
                // Déclaration de variable typée
                double dny_dy = (fields.norm_y[mesh.cell_idx(i, j + 1)] - fields.norm_y[mesh.cell_idx(i, j - 1)]) / (2.0 * dy);
                // Calcul de la résolution géométrique de cellule
                fields.kappa[idx] = -(dnx_dx + dny_dy);
            }
        }
    }
    MpiComm::sync_cell_scalar(mesh, fields.kappa);

    // 3. Force capillaire CSF équilibrée (balanced-force) sur les faces décalées
    // Selon X sur faces u
    for (int j = 1; j <= ny; ++j) {
        // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
        for (int i = 1; i <= nx + 1; ++i) {
            // Déclaration de variable typée
            int idx_u = mesh.u_idx(i, j);
            // Déclaration de variable typée
            double kap_avg = 0.5 * (fields.kappa[mesh.cell_idx(i - 1, j)] + fields.kappa[mesh.cell_idx(i, j)]);
            // Déclaration de variable typée
            double grad_C = (fields.C[mesh.cell_idx(i, j)] - fields.C[mesh.cell_idx(i - 1, j)]) / dx;
            // Calcul de la résolution géométrique de cellule
            fields.F_sigma_x[idx_u] = sigma * kap_avg * grad_C;
        }
    }

    // Selon Y sur faces v
    for (int j = 1; j <= ny + 1; ++j) {
        // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
        for (int i = 1; i <= nx; ++i) {
            // Déclaration de variable typée
            int idx_v = mesh.v_idx(i, j);
            // Déclaration de variable typée
            double kap_avg = 0.5 * (fields.kappa[mesh.cell_idx(i, j - 1)] + fields.kappa[mesh.cell_idx(i, j)]);
            // Déclaration de variable typée
            double grad_C = (fields.C[mesh.cell_idx(i, j)] - fields.C[mesh.cell_idx(i, j - 1)]) / dy;
            // Calcul de la résolution géométrique de cellule
            fields.F_sigma_y[idx_v] = sigma * kap_avg * grad_C;
        }
    }
}

void InterfaceTracker::advect(const Mesh2D& mesh, FlowFields& fields, double dt) {
    // Sélection de la méthode d'advection configurée par l'utilisateur
    AdvectionMethod adv_method = InterfaceReconstruction::parse_advection_method(vof_cfg.advection_method);
```

```
373
374     // Initialisation du vecteur de segments d'interface si nécessaire
375     int total_cells = (mesh.nx_local + 2) * (mesh.ny_local + 2);
376     if (static_cast<int>(interface_segments.size()) != total_cells) {
377         interface_segments.resize(total_cells);
378     }
379
380     // Répartition vers l'algorithme d'advection sélectionné
381     switch (adv_method) {
382         case AdvectionMethod::CIAM:
383             advect_ciam(mesh, fields, dt);
384             break;
385         case AdvectionMethod::WEYMOUTH_YUE:
386             advect_weymouth_yue(mesh, fields, dt);
387             break;
388         case AdvectionMethod::VOF_TVD:
389         default:
390             advect_tvd(mesh, fields, dt);
391             break;
392     }
393
394     // Incrémentation du compteur de pas pour Strang splitting alterné
395     step_count++;
396
397     // Synchronisation MPI des fractions volumiques sur les mailles fantômes
398     MpiComm::sync_cell_scalar(mesh, fields.C);
399
400     // Mise à jour des grandeurs physiques dérivées (masse volumique, viscosité, tension de surface)
401     update_physical_properties(mesh, fields);
402     compute_curvature_and_CSF(mesh, fields);
403 }
404
405 void InterfaceTracker::advect_tvd(const Mesh2D& mesh, FlowFields& fields, double dt) {
406     int nx = mesh.nx_local;
407     int ny = mesh.ny_local;
408     double dx = mesh.dx;
409     double dy = mesh.dy;
410
411     std::vector<double> C_new = fields.C;
412     TvdLimiter limiter = InterfaceReconstruction::parse_tvd_limiter(vof_cfg.tvd_limiter);
413
414     // Calcul des flux de fraction volumique sur les faces X
415     std::vector<double> flux_x((nx + 2) * (ny + 2), 0.0);
416     for (int j = 1; j <= ny; ++j) {
417         for (int i = 1; i <= nx + 1; ++i) {
418             double u_face = fields.u[mesh.u_idx(i, j)];
419             double c_L = fields.C[mesh.cell_idx(i - 1, j)];
420             double c_R = fields.C[mesh.cell_idx(i, j)];
421
422             double cfl_x = std::min(1.0, std::abs(u_face) * dt / dx);
423             if (u_face >= 0.0) {
424                 int i_up = std::max(0, i - 2);
425                 double c_U = fields.C[mesh.cell_idx(i_up, j)];
426                 double dC_up = c_L - c_U;
427                 double dC_loc = c_R - c_L;
428                 double r = (std::abs(dC_loc) > 1.0e-12) ? dC_up / dC_loc : 0.0;
429                 double phi = InterfaceReconstruction::evaluate_limiter(limiter, r, c_L, c_U, c_R);
430                 flux_x[mesh.cell_idx(i, j)] = u_face * (c_L + 0.5 * phi * (1.0 - cfl_x) * dC_loc);
431             } else {
432                 int i_down = std::min(nx + 1, i + 1);
433                 double c_U = fields.C[mesh.cell_idx(i_down, j)];
434                 double dC_up = c_U - c_R;
435                 double dC_loc = c_R - c_L;
436                 double r = (std::abs(dC_loc) > 1.0e-12) ? dC_up / dC_loc : 0.0;
437                 double phi = InterfaceReconstruction::evaluate_limiter(limiter, r, c_R, c_U, c_L);
438                 flux_x[mesh.cell_idx(i, j)] = u_face * (c_R - 0.5 * phi * (1.0 - cfl_x) * dC_loc);
439             }
440         }
441     }
442
443     // Calcul des flux sur les faces Y
444     std::vector<double> flux_y((nx + 2) * (ny + 2), 0.0);
445     for (int j = 1; j <= ny + 1; ++j) {
446         for (int i = 1; i <= nx; ++i) {
447             double v_face = fields.v[mesh.v_idx(i, j)];
448             double c_B = fields.C[mesh.cell_idx(i, j - 1)];
449             double c_T = fields.C[mesh.cell_idx(i, j)];
450             double cfl_y = std::min(1.0, std::abs(v_face) * dt / dy);
451
452             if (v_face >= 0.0) {
453                 int j_up = std::max(0, j - 2);
454                 double c_U = fields.C[mesh.cell_idx(i, j_up)];
455                 double dC_up = c_B - c_U;
456                 double dC_loc = c_T - c_B;
457                 double r = (std::abs(dC_loc) > 1.0e-12) ? dC_up / dC_loc : 0.0;
458                 double phi = InterfaceReconstruction::evaluate_limiter(limiter, r, c_B, c_U, c_T);
459                 flux_y[mesh.cell_idx(i, j)] = v_face * (c_B + 0.5 * phi * (1.0 - cfl_y) * dC_loc);
460             } else {
461                 int j_down = std::min(ny + 1, j + 1);
462                 double c_U = fields.C[mesh.cell_idx(i, j_down)];
463                 double dC_up = c_U - c_T;
464                 double dC_loc = c_T - c_B;
```

```
                double r = (std::abs(dC_loc) > 1.0e-12) ? dC_up / dC_loc : 0.0;
                double phi = InterfaceReconstruction::evaluate_limiter(limiter, r, c_T, c_U, c_B);
                flux_y[mesh.cell_idx(i, j)] = v_face * (c_T - 0.5 * phi * (1.0 - cfl_y) * dC_loc);
            }
        }
    }

    // Mise à jour conservative directe
    for (int j = 1; j <= ny; ++j) {
        for (int i = 1; i <= nx; ++i) {
            int idx = mesh.cell_idx(i, j);
            double fx_in  = flux_x[mesh.cell_idx(i, j)];
            double fx_out = flux_x[mesh.cell_idx(i + 1, j)];
            double fy_in  = flux_y[mesh.cell_idx(i, j)];
            double fy_out = flux_y[mesh.cell_idx(i, j + 1)];

            C_new[idx] = fields.C[idx] - (dt / dx) * (fx_out - fx_in) - (dt / dy) * (fy_out - fy_in);
            C_new[idx] = std::clamp(C_new[idx], 0.0, 1.0);
        }
    }

    fields.C = C_new;
}

void InterfaceTracker::advect_ciam(const Mesh2D& mesh, FlowFields& fields, double dt) {
    int nx = mesh.nx_local;
    int ny = mesh.ny_local;
    double dx = mesh.dx;
    double dy = mesh.dy;

    ReconstructionMethod recon_method = InterfaceReconstruction::parse_reconstruction_method(vof_cfg.
        reconstruction_method);
    bool x_first = (step_count % 2 == 0);

    // Fonction lambda pour un balayage directionnel X
    auto sweep_x = [&](std::vector<double>& C_in, std::vector<double>& C_out) {
        InterfaceReconstruction::reconstruct(recon_method, mesh, C_in, interface_segments);
        std::vector<double> flux_vol_x((nx + 2) * (ny + 2), 0.0);

        for (int j = 1; j <= ny; ++j) {
            for (int i = 1; i <= nx + 1; ++i) {
                double u_face = fields.u[mesh.u_idx(i, j)];
                double delta_x = std::abs(u_face) * dt;
                if (delta_x <= 1.0e-14) continue;

                if (u_face > 0.0) {
                    // Donneur cellule i-1
                    int donor_idx = mesh.cell_idx(i - 1, j);
                    double x_face_pos = mesh.xv(i);
                    double box_xmin = x_face_pos - delta_x;
                    double box_xmax = x_face_pos;
                    double box_ymin = mesh.yv(j);
                    double box_ymax = mesh.yv(j + 1);

                    const auto& seg = interface_segments[donor_idx];
                    flux_vol_x[mesh.cell_idx(i, j)] = InterfaceReconstruction::cut_box_area(box_xmin, box_xmax
                        , box_ymin, box_ymax,
                                                                                            seg.nx, seg.ny, seg
                                                                                                .alpha);
                } else {
                    // Donneur cellule i
                    int donor_idx = mesh.cell_idx(i, j);
                    double x_face_pos = mesh.xv(i);
                    double box_xmin = x_face_pos;
                    double box_xmax = x_face_pos + delta_x;
                    double box_ymin = mesh.yv(j);
                    double box_ymax = mesh.yv(j + 1);

                    const auto& seg = interface_segments[donor_idx];
                    flux_vol_x[mesh.cell_idx(i, j)] = -InterfaceReconstruction::cut_box_area(box_xmin,
                        box_xmax, box_ymin, box_ymax,
                                                                                             seg.nx, seg.ny,
                                                                                                 seg.alpha);
                }
            }
        }

        for (int j = 1; j <= ny; ++j) {
            for (int i = 1; i <= nx; ++i) {
                int idx = mesh.cell_idx(i, j);
                double f_in  = flux_vol_x[mesh.cell_idx(i, j)];
                double f_out = flux_vol_x[mesh.cell_idx(i + 1, j)];
                double u_in  = fields.u[mesh.u_idx(i, j)];
                double u_out = fields.u[mesh.u_idx(i + 1, j)];

                // Mise à jour conservative 1D avec compensation de divergence
                double denom = dx - dt * (u_out - u_in);
                if (denom > 1.0e-12) {
                    C_out[idx] = (C_in[idx] * dx - (f_out - f_in) / dy) / denom;
                } else {
                    C_out[idx] = C_in[idx] - (f_out - f_in) / (dx * dy);
                }
```

```
                C_out[idx] = std::clamp(C_out[idx], 0.0, 1.0);
            }
        }
        MpiComm::sync_cell_scalar(mesh, C_out);
    };

    // Fonction lambda pour un balayage directionnel Y
    auto sweep_y = [&](std::vector<double>& C_in, std::vector<double>& C_out) {
        InterfaceReconstruction::reconstruct(recon_method, mesh, C_in, interface_segments);
        std::vector<double> flux_vol_y((nx + 2) * (ny + 2), 0.0);

        for (int j = 1; j <= ny + 1; ++j) {
            for (int i = 1; i <= nx; ++i) {
                double v_face = fields.v[mesh.v_idx(i, j)];
                double delta_y = std::abs(v_face) * dt;
                if (delta_y <= 1.0e-14) continue;

                if (v_face > 0.0) {
                    // Donneur cellule j-1
                    int donor_idx = mesh.cell_idx(i, j - 1);
                    double y_face_pos = mesh.yv(j);
                    double box_xmin = mesh.xv(i);
                    double box_xmax = mesh.xv(i + 1);
                    double box_ymin = y_face_pos - delta_y;
                    double box_ymax = y_face_pos;

                    const auto& seg = interface_segments[donor_idx];
                    flux_vol_y[mesh.cell_idx(i, j)] = InterfaceReconstruction::cut_box_area(box_xmin, box_xmax
                        , box_ymin, box_ymax,
                                                                                            seg.nx, seg.ny, seg
                                                                                                .alpha);
                } else {
                    // Donneur cellule j
                    int donor_idx = mesh.cell_idx(i, j);
                    double y_face_pos = mesh.yv(j);
                    double box_xmin = mesh.xv(i);
                    double box_xmax = mesh.xv(i + 1);
                    double box_ymin = y_face_pos;
                    double box_ymax = y_face_pos + delta_y;

                    const auto& seg = interface_segments[donor_idx];
                    flux_vol_y[mesh.cell_idx(i, j)] = -InterfaceReconstruction::cut_box_area(box_xmin,
                        box_xmax, box_ymin, box_ymax,
                                                                                             seg.nx, seg.ny,
                                                                                                 seg.alpha);
                }
            }
        }

        for (int j = 1; j <= ny; ++j) {
            for (int i = 1; i <= nx; ++i) {
                int idx = mesh.cell_idx(i, j);
                double f_in  = flux_vol_y[mesh.cell_idx(i, j)];
                double f_out = flux_vol_y[mesh.cell_idx(i, j + 1)];
                double v_in  = fields.v[mesh.v_idx(i, j)];
                double v_out = fields.v[mesh.v_idx(i, j + 1)];

                double denom = dy - dt * (v_out - v_in);
                if (denom > 1.0e-12) {
                    C_out[idx] = (C_in[idx] * dy - (f_out - f_in) / dx) / denom;
                } else {
                    C_out[idx] = C_in[idx] - (f_out - f_in) / (dx * dy);
                }
                C_out[idx] = std::clamp(C_out[idx], 0.0, 1.0);
            }
        }
        MpiComm::sync_cell_scalar(mesh, C_out);
    };

    std::vector<double> C_star = fields.C;
    std::vector<double> C_final = fields.C;

    if (x_first) {
        // Balayage X puis Y
        sweep_x(fields.C, C_star);
        sweep_y(C_star, C_final);
    } else {
        // Balayage Y puis X
        sweep_y(fields.C, C_star);
        sweep_x(C_star, C_final);
    }

    fields.C = C_final;
}

void InterfaceTracker::advect_weymouth_yue(const Mesh2D& mesh, FlowFields& fields, double dt) {
    int nx = mesh.nx_local;
    int ny = mesh.ny_local;
    double dx = mesh.dx;
    double dy = mesh.dy;
    double cell_area = dx * dy;

```

```
    ReconstructionMethod recon_method = InterfaceReconstruction::parse_reconstruction_method(vof_cfg.
        reconstruction_method);
    bool x_first = (step_count % 2 == 0);

    // 1. Fonction indicatrice de dilatation binaire C_d(i,j) figee au temps n (Weymouth & Yue, JCP 2010, Eq.
        14)
    std::vector<double> C_d((nx + 2) * (ny + 2), 0.0);
    for (int j = 0; j <= ny + 1; ++j) {
        for (int i = 0; i <= nx + 1; ++i) {
            int idx = mesh.cell_idx(i, j);
            C_d[idx] = (fields.C[idx] >= 0.5) ? 1.0 : 0.0;
        }
    }

    // 2. Balayage directionnel 1D selon X (Weymouth & Yue, JCP 2010, Eq. 15)
    auto sweep_x = [&](std::vector<double>& C_in, std::vector<double>& C_out, bool clamp_out) {
        InterfaceReconstruction::reconstruct(recon_method, mesh, C_in, interface_segments);
        std::vector<double> f_vol_x((nx + 2) * (ny + 2), 0.0);

        for (int j = 1; j <= ny; ++j) {
            for (int i = 1; i <= nx + 1; ++i) {
                double u_face = fields.u[mesh.u_idx(i, j)];
                double delta_x = std::abs(u_face) * dt;
                if (delta_x <= 1.0e-14) continue;

                double x_pos = mesh.xv(i);
                if (u_face > 0.0) {
                    int donor_idx = mesh.cell_idx(i - 1, j);
                    const auto& seg = interface_segments[donor_idx];
                    f_vol_x[mesh.cell_idx(i, j)] = InterfaceReconstruction::cut_box_area(
                        x_pos - delta_x, x_pos, mesh.yv(j), mesh.yv(j + 1), seg.nx, seg.ny, seg.alpha);
                } else {
                    int donor_idx = mesh.cell_idx(i, j);
                    const auto& seg = interface_segments[donor_idx];
                    f_vol_x[mesh.cell_idx(i, j)] = -InterfaceReconstruction::cut_box_area(
                        x_pos, x_pos + delta_x, mesh.yv(j), mesh.yv(j + 1), seg.nx, seg.ny, seg.alpha);
                }
            }
        }

        for (int j = 1; j <= ny; ++j) {
            for (int i = 1; i <= nx; ++i) {
                int idx = mesh.cell_idx(i, j);
                double fx_in  = f_vol_x[mesh.cell_idx(i, j)];
                double fx_out = f_vol_x[mesh.cell_idx(i + 1, j)];
                double u_in   = fields.u[mesh.u_idx(i, j)];
                double u_out  = fields.u[mesh.u_idx(i + 1, j)];

                C_out[idx] = C_in[idx] - (fx_out - fx_in) / cell_area + C_d[idx] * (dt / dx) * (u_out - u_in);
                if (clamp_out) {
                    C_out[idx] = std::clamp(C_out[idx], 0.0, 1.0);
                }
            }
        }
        MpiComm::sync_cell_scalar(mesh, C_out);
    };

    // 3. Balayage directionnel 1D selon Y (Weymouth & Yue, JCP 2010, Eq. 16)
    auto sweep_y = [&](std::vector<double>& C_in, std::vector<double>& C_out, bool clamp_out) {
        InterfaceReconstruction::reconstruct(recon_method, mesh, C_in, interface_segments);
        std::vector<double> f_vol_y((nx + 2) * (ny + 2), 0.0);

        for (int j = 1; j <= ny + 1; ++j) {
            for (int i = 1; i <= nx; ++i) {
                double v_face = fields.v[mesh.v_idx(i, j)];
                double delta_y = std::abs(v_face) * dt;
                if (delta_y <= 1.0e-14) continue;

                double y_pos = mesh.yv(j);
                if (v_face > 0.0) {
                    int donor_idx = mesh.cell_idx(i, j - 1);
                    const auto& seg = interface_segments[donor_idx];
                    f_vol_y[mesh.cell_idx(i, j)] = InterfaceReconstruction::cut_box_area(
                        mesh.xv(i), mesh.xv(i + 1), y_pos - delta_y, y_pos, seg.nx, seg.ny, seg.alpha);
                } else {
                    int donor_idx = mesh.cell_idx(i, j);
                    const auto& seg = interface_segments[donor_idx];
                    f_vol_y[mesh.cell_idx(i, j)] = -InterfaceReconstruction::cut_box_area(
                        mesh.xv(i), mesh.xv(i + 1), y_pos, y_pos + delta_y, seg.nx, seg.ny, seg.alpha);
                }
            }
        }

        for (int j = 1; j <= ny; ++j) {
            for (int i = 1; i <= nx; ++i) {
                int idx = mesh.cell_idx(i, j);
                double fy_in  = f_vol_y[mesh.cell_idx(i, j)];
                double fy_out = f_vol_y[mesh.cell_idx(i, j + 1)];
                double v_in   = fields.v[mesh.v_idx(i, j)];
                double v_out  = fields.v[mesh.v_idx(i, j + 1)];

                C_out[idx] = C_in[idx] - (fy_out - fy_in) / cell_area + C_d[idx] * (dt / dy) * (v_out - v_in);
```

```
                if (clamp_out) {
                    C_out[idx] = std::clamp(C_out[idx], 0.0, 1.0);
                }
            }
        }
        MpiComm::sync_cell_scalar(mesh, C_out);
    };

    std::vector<double> C_star = fields.C;
    std::vector<double> C_final = fields.C;

    if (x_first) {
        sweep_x(fields.C, C_star, false);
        sweep_y(C_star, C_final, true);
    } else {
        sweep_y(fields.C, C_star, false);
        sweep_x(C_star, C_final, true);
    }

    fields.C = C_final;
}

} // namespace covof
```

Chapitre 12

Exportation Graphique VTK XML Parallèle (.vtr / .pvtr) (VTKWriter.hpp, VTKWriter.cpp)

12.1 Architecture des Fichiers VTK XML `RectilinearGrid` et `PRectilinearGr`

La classe `covof::VTKWriter` (`include/covof/VTKWriter.hpp`, `src/VTKWriter.cpp`) produit des fichiers directement exploitables par **ParaView**, **VisIt** et les scripts Python de post-traitement sans bibliothèque externe :

1. Chaque processus MPI de rang r écrit son sous-domaine local dans `<case>_p<r>_<step>.vtr` (format `RectilinearGrid`) contenant les coordonnées des nœuds $(x_u, y_v, 0)$ et les tableaux `CellData` :
   — `Pressure` $(p_{i,j})$;
   — `Temperature` $(T_{i,j})$;
   — `PhaseIndicator` $(C_{i,j})$;
   — `Divergence` $((\nabla_h \cdot \mathbf{u})_{i,j})$;
   — `Vorticity` $(\omega_z(i,j) = \frac{v_c(i+1,j)-v_c(i-1,j)}{2\Delta x} - \frac{u_c(i,j+1)-u_c(i,j-1)}{2\Delta y})$;
   — `Viscosity` $(\mu_{i,j})$;
   — `Velocity` (vecteur 3D $(u_c(i,j), v_c(i,j), 0)$ interpolé aux centres des mailles : $u_c(i,j) = \frac{1}{2}(u_{i,j} + u_{i+1,j})$, $v_c(i,j) = \frac{1}{2}(v_{i,j} + v_{i,j+1})$).
2. Le processus maître ($r = 0$) écrit le fichier chapeau `<case>_<step>.pvtr` (`PRectilinearGrid`) référençant les extensions globales `Piece Extent` de tous les sous-domaines MPI.

12.2 Répertoire des Classes, Structures et Méthodes de `include/covof/VTKWri`

Ligne	Catégorie	Identifiant C++17	Signature / Rôle
14	class	`VTKWriter`	Déclaration de type / classe / structure
28	Méthode / Fonction	`init`	Type retour : void
33	Méthode / Fonction	`write_step`	Type retour : void
38	Méthode / Fonction	`write_diagnostics`	Type retour : void
40	Méthode / Fonction	`finalize`	Type retour : void

12.3 Code Source Intégral Annoté : `include/covof/VTKWriter.hpp`

Le listing ci-dessous présente l'intégralité des 46 lignes du fichier `include/covof/VTKWriter.hpp` de l'implémentation C++17 :

```
1  #pragma once
2
3  #include "Mesh.hpp"
4  #include "Field.hpp"
5  #include <string>
6  #include <fstream>
7
8  namespace covof {
9  namespace fugu = covof; // Alias rétrocompatibilité
10
11 /**
12  * @brief Gestionnaire d'exportation pour ParaView (XML VTK .vtr/.pvtr) et diagnostics CSV
13  */
```

```cpp
class VTKWriter {
public:
    // Déclaration de variable typée
    std::string output_dir = "output";
    // Déclaration de variable typée
    std::string case_name  = "covof";
    std::ofstream csv_file;

    // Calcul et assignation numérique
    VTKWriter() = default;

    /**
     * @brief Initialise le répertoire de sortie et le fichier CSV d'historique
     */
    void init(const Mesh2D& mesh, const std::string& out_dir, const std::string& name);

    /**
     * @brief Exporte le pas de temps courant au format VTK XML (.vtr par processus + .pvtr maître)
     */
    void write_step(const Mesh2D& mesh, const FlowFields& fields, int step, double time);

    /**
     * @brief Calcule et enregistre les grandeurs physiques intégrales dans le fichier CSV
     */
    void write_diagnostics(const Mesh2D& mesh, const FlowFields& fields, int step, double time, double dt);

    void finalize() {
        // Évaluation de la condition de contrôle
        if (csv_file.is_open()) csv_file.close();
    }
};

} // namespace covof
```

12.4 Répertoire des Classes, Structures et Méthodes de `src/VTKWriter.cpp`

Ligne	Catégorie	Identifiant C++17	Signature / Rôle
11	Méthode / Fonction	`VTKWriter::init`	Type retour : void
31	Méthode / Fonction	`VTKWriter::write_step`	Type retour : void
33	Méthode / Fonction	`std::setfill`	Type retour : ss_step <<
49	Méthode / Fonction	`vtr`	Type retour : std : :ofstream
186	Méthode / Fonction	`all_x_start`	Type retour : std : :vector<int>
188	Méthode / Fonction	`all_y_start`	Type retour : std : :vector<int>
201	Méthode / Fonction	`pvtr`	Type retour : std : :ofstream
260	Méthode / Fonction	`VTKWriter::write_diagnostics`	Type retour : void

12.5 Code Source Intégral Annoté : `src/VTKWriter.cpp`

Le listing ci-dessous présente l'intégralité des 346 lignes du fichier `src/VTKWriter.cpp` de l'implémentation C++17 :

```cpp
#include "covof/VTKWriter.hpp"
#include "covof/MpiComm.hpp"
#include <iomanip>
#include <sstream>
#include <sys/stat.h>
#include <cmath>

namespace covof {
namespace fugu = covof; // Alias rétrocompatibilité

void VTKWriter::init(const Mesh2D& mesh, const std::string& out_dir, const std::string& name) {
    // Calcul et assignation numérique
    output_dir = out_dir;
    // Calcul et assignation numérique
    case_name = name;

    // Évaluation de la condition de contrôle
    if (mesh.rank == 0) {
        mkdir(output_dir.c_str(), 0777);

        // Déclaration de variable typée
        std::string csv_path = output_dir + "/" + case_name + "_history.csv";
        csv_file.open(csv_path);
        // Évaluation de la condition de contrôle
        if (csv_file.is_open()) {
            csv_file << "step,time,dt,center_of_mass_y,rise_velocity,bubble_volume,kinetic_energy,max_u,
                max_div\n";
        }
    }
}
```

```cpp

void VTKWriter::write_step(const Mesh2D& mesh, const FlowFields& fields, int step, double time) {
    std::ostringstream ss_step;
    ss_step << std::setfill('0') << std::setw(6) << step;
    // Déclaration de variable typée
    std::string step_str = ss_step.str();

    // Déclaration de variable typée
    int x_start = mesh.i_offset;
    // Déclaration de variable typée
    int x_end   = mesh.i_offset + mesh.nx_local;
    // Déclaration de variable typée
    int y_start = mesh.j_offset;
    // Déclaration de variable typée
    int y_end   = mesh.j_offset + mesh.ny_local;

    // 1. Écriture du fichier .vtr propre à chaque processus MPI
    std::ostringstream ss_vtr;
    ss_vtr << output_dir << "/" << case_name << "_p" << mesh.rank << "_" << step_str << ".vtr";
    std::ofstream vtr(ss_vtr.str());

    // Évaluation de la condition de contrôle
    if (vtr.is_open()) {
        // Calcul et assignation numérique
        vtr << "<?xml version=\"1.0\"?>\n"
            // Calcul et assignation numérique
            << "<VTKFile type=\"RectilinearGrid\" version=\"0.1\" byte_order=\"LittleEndian\">\n"
            // Calcul et assignation numérique
            << "  <RectilinearGrid WholeExtent=\"" << x_start << " " << x_end << " "
            << y_start << " " << y_end << " 0 0\">\n"
            // Calcul et assignation numérique
            << "    <Piece Extent=\"" << x_start << " " << x_end << " "
            << y_start << " " << y_end << " 0 0\">\n"
            // Calcul et assignation numérique
            << "      <CellData Scalars=\"PhaseIndicator,Pressure,Density,Viscosity,Divergence\" Vectors=\"
                Velocity\">\n";

        // PhaseIndicator C
        vtr << "        <DataArray type=\"Float64\" Name=\"PhaseIndicator\" format=\"ascii\">\n";
        // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
        for (int j = 1; j <= mesh.ny_local; ++j) {
            // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
            for (int i = 1; i <= mesh.nx_local; ++i) {
                vtr << fields.C[mesh.cell_idx(i, j)] << " ";
            }
            vtr << "\n";
        }
        vtr << "        </DataArray>\n";

        // Pressure p
        vtr << "        <DataArray type=\"Float64\" Name=\"Pressure\" format=\"ascii\">\n";
        // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
        for (int j = 1; j <= mesh.ny_local; ++j) {
            // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
            for (int i = 1; i <= mesh.nx_local; ++i) {
                vtr << fields.p[mesh.cell_idx(i, j)] << " ";
            }
            vtr << "\n";
        }
        vtr << "        </DataArray>\n";

        // Density rho
        vtr << "        <DataArray type=\"Float64\" Name=\"Density\" format=\"ascii\">\n";
        // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
        for (int j = 1; j <= mesh.ny_local; ++j) {
            // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
            for (int i = 1; i <= mesh.nx_local; ++i) {
                vtr << fields.rho[mesh.cell_idx(i, j)] << " ";
            }
            vtr << "\n";
        }
        vtr << "        </DataArray>\n";

        // Viscosity mu
        vtr << "        <DataArray type=\"Float64\" Name=\"Viscosity\" format=\"ascii\">\n";
        // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
        for (int j = 1; j <= mesh.ny_local; ++j) {
            // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
            for (int i = 1; i <= mesh.nx_local; ++i) {
                vtr << fields.mu[mesh.cell_idx(i, j)] << " ";
            }
            vtr << "\n";
        }
        vtr << "        </DataArray>\n";

        // Divergence div(u)
        vtr << "        <DataArray type=\"Float64\" Name=\"Divergence\" format=\"ascii\">\n";
        // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
        for (int j = 1; j <= mesh.ny_local; ++j) {
            // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
            for (int i = 1; i <= mesh.nx_local; ++i) {
                // Déclaration de variable typée
```

```cpp
                double div = (fields.u[mesh.u_idx(i + 1, j)] - fields.u[mesh.u_idx(i, j)]) / mesh.dx +
                             (fields.v[mesh.v_idx(i, j + 1)] - fields.v[mesh.v_idx(i, j)]) / mesh.dy;
                vtr << div << " ";
            }
            vtr << "\n";
        }
        vtr << "        </DataArray>\n";

        // Temperature T
        vtr << "        <DataArray type=\"Float64\" Name=\"Temperature\" format=\"ascii\">\n";
        // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
        for (int j = 1; j <= mesh.ny_local; ++j) {
            // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
            for (int i = 1; i <= mesh.nx_local; ++i) {
                vtr << fields.T[mesh.cell_idx(i, j)] << " ";
            }
            vtr << "\n";
        }
        vtr << "        </DataArray>\n";

        // Velocity vector au centre des cellules
        vtr << "        <DataArray type=\"Float64\" Name=\"Velocity\" NumberOfComponents=\"3\" format=\"ascii
            \">\n";
        // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
        for (int j = 1; j <= mesh.ny_local; ++j) {
            // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
            for (int i = 1; i <= mesh.nx_local; ++i) {
                // Déclaration de variable typée
                double uc = 0.5 * (fields.u[mesh.u_idx(i, j)] + fields.u[mesh.u_idx(i + 1, j)]);
                // Déclaration de variable typée
                double vc = 0.5 * (fields.v[mesh.v_idx(i, j)] + fields.v[mesh.v_idx(i, j + 1)]);
                vtr << uc << " " << vc << " 0.0 ";
            }
            vtr << "\n";
        }
        vtr << "        </DataArray>\n";

        vtr << "      </CellData>\n"
            << "      <Coordinates>\n"
            // Calcul et assignation numérique
            << "        <DataArray type=\"Float64\" Name=\"XCoordinates\" format=\"ascii\">\n";
        // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
        for (int i = 0; i <= mesh.nx_local; ++i) {
            vtr << (mesh.i_offset + i) * mesh.dx << " ";
        }
        vtr << "\n        </DataArray>\n"
            // Calcul et assignation numérique
            << "        <DataArray type=\"Float64\" Name=\"YCoordinates\" format=\"ascii\">\n";
        // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
        for (int j = 0; j <= mesh.ny_local; ++j) {
            vtr << (mesh.j_offset + j) * mesh.dy << " ";
        }
        vtr << "\n        </DataArray>\n"
            // Calcul et assignation numérique
            << "        <DataArray type=\"Float64\" Name=\"ZCoordinates\" format=\"ascii\">\n"
            << "          0.0\n"
            << "        </DataArray>\n"
            << "      </Coordinates>\n"
            << "    </Piece>\n"
            << "  </RectilinearGrid>\n"
            << "</VTKFile>\n";
    }

    // 2. Écriture du fichier maître .pvtr par le processus racine (rank 0)
    if (mesh.rank == 0) {
        // Déclaration de variable typée
        std::vector<int> all_x_start(mesh.num_procs), all_x_end(mesh.num_procs);
        // Déclaration de variable typée
        std::vector<int> all_y_start(mesh.num_procs), all_y_end(mesh.num_procs);

        // Appel de communication ou synchronisation parallèle OpenMPI
        MPI_Gather(&x_start, 1, MPI_INT, all_x_start.data(), 1, MPI_INT, 0, mesh.comm);
        // Appel de communication ou synchronisation parallèle OpenMPI
        MPI_Gather(&x_end,   1, MPI_INT, all_x_end.data(),   1, MPI_INT, 0, mesh.comm);
        // Appel de communication ou synchronisation parallèle OpenMPI
        MPI_Gather(&y_start, 1, MPI_INT, all_y_start.data(), 1, MPI_INT, 0, mesh.comm);
        // Appel de communication ou synchronisation parallèle OpenMPI
        MPI_Gather(&y_end,   1, MPI_INT, all_y_end.data(),   1, MPI_INT, 0, mesh.comm);

        std::ostringstream ss_pvtr;
        ss_pvtr << output_dir << "/" << case_name << "_" << step_str << ".pvtr";
        std::ofstream pvtr(ss_pvtr.str());

        // Évaluation de la condition de contrôle
        if (pvtr.is_open()) {
            // Calcul et assignation numérique
            pvtr << "<?xml version=\"1.0\"?>\n"
                 // Calcul et assignation numérique
                 << "<VTKFile type=\"PRectilinearGrid\" version=\"0.1\" byte_order=\"LittleEndian\">\n"
                 // Calcul et assignation numérique
                 << "  <PRectilinearGrid WholeExtent=\"0 " << mesh.nx_global << " 0 " << mesh.ny_global << " 0
                     0\" GhostLevel=\"0\">\n"
```

```
211                 // Calcul et assignation numérique
212                 << "    <PCellData Scalars=\"PhaseIndicator,Pressure,Density,Viscosity,Divergence\" Vectors
                        =\"Velocity\">\n"
213                 // Calcul et assignation numérique
214                 << "      <PDataArray type=\"Float64\" Name=\"PhaseIndicator\"/>\n"
215                 // Calcul et assignation numérique
216                 << "      <PDataArray type=\"Float64\" Name=\"Pressure\"/>\n"
217                 // Calcul et assignation numérique
218                 << "      <PDataArray type=\"Float64\" Name=\"Density\"/>\n"
219                 // Calcul et assignation numérique
220                 << "      <PDataArray type=\"Float64\" Name=\"Viscosity\"/>\n"
221                 // Calcul et assignation numérique
222                 << "      <PDataArray type=\"Float64\" Name=\"Divergence\"/>\n"
223                 // Calcul et assignation numérique
224                 << "      <PDataArray type=\"Float64\" Name=\"Velocity\" NumberOfComponents=\"3\"/>\n"
225                 << "    </PCellData>\n"
226                 << "    <PCoordinates>\n"
227                 // Calcul et assignation numérique
228                 << "      <PDataArray type=\"Float64\" Name=\"XCoordinates\"/>\n"
229                 // Calcul et assignation numérique
230                 << "      <PDataArray type=\"Float64\" Name=\"YCoordinates\"/>\n"
231                 // Calcul et assignation numérique
232                 << "      <PDataArray type=\"Float64\" Name=\"ZCoordinates\"/>\n"
233                 << "    </PCoordinates>\n";
234
235             // Boucle d'itération algorithmique
236             for (int p = 0; p < mesh.num_procs; ++p) {
237                 std::ostringstream piece_vtr;
238                 piece_vtr << case_name << "_p" << p << "_" << step_str << ".vtr";
239                 // Calcul et assignation numérique
240                 pvtr << "    <Piece Extent=\"" << all_x_start[p] << " " << all_x_end[p] << " "
241                      // Calcul et assignation numérique
242                      << all_y_start[p] << " " << all_y_end[p] << " 0 0\" Source=\"" << piece_vtr.str() << "
                             \"/>\n";
243             }
244
245             pvtr << "  </PRectilinearGrid>\n"
246                  << "</VTKFile>\n";
247         }
248     } else {
249         // Appel de communication ou synchronisation parallèle OpenMPI
250         MPI_Gather(&x_start, 1, MPI_INT, nullptr, 0, MPI_INT, 0, mesh.comm);
251         // Appel de communication ou synchronisation parallèle OpenMPI
252         MPI_Gather(&x_end,   1, MPI_INT, nullptr, 0, MPI_INT, 0, mesh.comm);
253         // Appel de communication ou synchronisation parallèle OpenMPI
254         MPI_Gather(&y_start, 1, MPI_INT, nullptr, 0, MPI_INT, 0, mesh.comm);
255         // Appel de communication ou synchronisation parallèle OpenMPI
256         MPI_Gather(&y_end,   1, MPI_INT, nullptr, 0, MPI_INT, 0, mesh.comm);
257     }
258 }
259
260 void VTKWriter::write_diagnostics(const Mesh2D& mesh, const FlowFields& fields, int step, double time, double
        dt) {
261     // Déclaration de variable typée
262     double local_y_integral = 0.0;
263     // Déclaration de variable typée
264     double local_v_integral = 0.0;
265     // Déclaration de variable typée
266     double local_vol = 0.0;
267     // Déclaration de variable typée
268     double local_ke = 0.0;
269     // Déclaration de variable typée
270     double local_max_u = 0.0;
271     // Déclaration de variable typée
272     double local_max_div = 0.0;
273
274     // Déclaration de variable typée
275     double dvol = mesh.dx * mesh.dy;
276
277     // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
278     for (int j = 1; j <= mesh.ny_local; ++j) {
279         // Déclaration de variable typée
280         double y = mesh.yc(j);
281         // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
282         for (int i = 1; i <= mesh.nx_local; ++i) {
283             // Déclaration de variable typée
284             double c = fields.C[mesh.cell_idx(i, j)];
285             // Déclaration de variable typée
286             double indicator_bubble = 1.0 - c; // 1 si phase fluide 2 (bulle/goutte)
287
288             // Calcul et assignation numérique
289             local_vol += indicator_bubble * dvol;
290             // Calcul et assignation numérique
291             local_y_integral += y * indicator_bubble * dvol;
292
293             // Déclaration de variable typée
294             double uc = 0.5 * (fields.u[mesh.u_idx(i, j)] + fields.u[mesh.u_idx(i + 1, j)]);
295             // Déclaration de variable typée
296             double vc = 0.5 * (fields.v[mesh.v_idx(i, j)] + fields.v[mesh.v_idx(i, j + 1)]);
297
298             // Calcul et assignation numérique
299             local_v_integral += vc * indicator_bubble * dvol;
```

```
300
301             // Déclaration de variable typée
302             double speed2 = uc * uc + vc * vc;
303             // Calcul et assignation numérique
304             local_ke += 0.5 * fields.rho[mesh.cell_idx(i, j)] * speed2 * dvol;
305
306             // Déclaration de variable typée
307             double speed = std::sqrt(speed2);
308             // Évaluation de la condition de contrôle
309             if (speed > local_max_u) local_max_u = speed;
310
311             // Déclaration de variable typée
312             double div = std::abs((fields.u[mesh.u_idx(i + 1, j)] - fields.u[mesh.u_idx(i, j)]) / mesh.dx +
313                                   (fields.v[mesh.v_idx(i, j + 1)] - fields.v[mesh.v_idx(i, j)]) / mesh.dy);
314             // Évaluation de la condition de contrôle
315             if (div > local_max_div) local_max_div = div;
316         }
317     }
318
319     // Déclaration de variable typée
320     double global_vol = MpiComm::global_sum(mesh, local_vol);
321     // Déclaration de variable typée
322     double global_y_integral = MpiComm::global_sum(mesh, local_y_integral);
323     // Déclaration de variable typée
324     double global_v_integral = MpiComm::global_sum(mesh, local_v_integral);
325     // Déclaration de variable typée
326     double global_ke = MpiComm::global_sum(mesh, local_ke);
327     // Déclaration de variable typée
328     double global_max_u = MpiComm::global_max(mesh, local_max_u);
329     // Déclaration de variable typée
330     double global_max_div = MpiComm::global_max(mesh, local_max_div);
331
332     // Déclaration de variable typée
333     double yc = (global_vol > 1.0e-12) ? global_y_integral / global_vol : 0.0;
334     // Déclaration de variable typée
335     double vc = (global_vol > 1.0e-12) ? global_v_integral / global_vol : 0.0;
336
337     // Évaluation de la condition de contrôle
338     if (mesh.rank == 0 && csv_file.is_open()) {
339         csv_file << step << "," << time << "," << dt << ","
340                  << yc << "," << vc << "," << global_vol << ","
341                  << global_ke << "," << global_max_u << "," << global_max_div << "\n";
342         csv_file.flush();
343     }
344 }
345
346 } // namespace covof
```

Chapitre 13

Orchestration de la Simulation, Pas de Temps Adaptatif et Diagnostics VV (Simulation.hpp, Simulation.cpp)

13.1 Orchestration Globale dans `covof::Simulation`

La classe `covof::Simulation` (`include/covof/Simulation.hpp`, `src/Simulation.cpp`, 916 lignes) coordonne l'ensemble des composants du solveur :

1. **Initialisation géométrique et physique (`setup_initial_conditions`)** : génère les profils analytiques, disques, colonnes de barrage, nappes liquides cisaillement Kelvin-Helmholtz, perturbations cosinusoïdales de Rayleigh-Taylor et buses d'injection de jets visqueux avec sous-échantillonnage 8×8 par cellule pour initialiser une fraction volumique $C_{i,j}(0)$ précise au second ordre.
2. **Contrôle du pas de temps adaptatif (`compute_adaptive_dt`)** : évalue à chaque pas de temps la contrainte CFL multi-physique (convective, capillaire de Brackbill et VOF) :

$$\Delta t \leq \min\left(\mathrm{CFL}_{\max}\frac{\min(\Delta x, \Delta y)}{\max(\|\mathbf{u}\|_\infty, 10^{-6})},\quad \sqrt{\frac{(\rho_1+\rho_2)\min(\Delta x, \Delta y)^3}{4\pi\sigma}}\right) \tag{13.1}$$

3. **Calcul des diagnostics intégraux et normes V&V** : produit automatiquement les fichiers CSV de suivi temporel (`history.csv`, `dense_cylinder_history.csv`, `liquid_sheet_history.csv`, `viscous_jet_history.csv`, `vortex_error.csv`, `laplacian_summary.csv`).

13.2 Répertoire des Classes, Structures et Méthodes de `include/covof/Simula`

Ligne	Catégorie	Identifiant C++17	Signature / Rôle
18	class	`Simulation`	Déclaration de type / classe / structure
35	Méthode / Fonction	`init`	Type retour : bool
40	Méthode / Fonction	`run`	Type retour : void
45	Méthode / Fonction	`run_laplacian`	Type retour : void
50	Méthode / Fonction	`run_vortex_advection`	Type retour : void
55	Méthode / Fonction	`compute_time_step`	Type retour : double

13.3 Code Source Intégral Annoté : `include/covof/Simulation.hpp`

Le listing ci-dessous présente l'intégralité des 58 lignes du fichier `include/covof/Simulation.hpp` de l'implémentation C++17 :

```
1  #pragma once
2
3  #include "Config.hpp"
4  #include "Mesh.hpp"
5  #include "Field.hpp"
6  #include "InterfaceTracker.hpp"
7  #include "NavierStokesSolver.hpp"
8  #include "Preconditioner.hpp"
9  #include "LinearSolver.hpp"
10 #include "VTKWriter.hpp"
11
```

```cpp
namespace covof {
namespace fugu = covof; // Alias rétrocompatibilité

/**
 * @brief Orchestrateur principal de la simulation CFD diphasique tout-couplée 2D
 */
class Simulation {
public:
    Config config;
    Mesh2D mesh;
    FlowFields fields;
    InterfaceTracker tracker;
    NavierStokesSolver ns_solver;
    std::unique_ptr<PreconditionerBase> precond;
    std::unique_ptr<LinearSolverBase> linear_solver;
    VTKWriter vtk_writer;

    // Calcul et assignation numérique
    Simulation() = default;

    /**
     * @brief Configure et initialise l'ensemble des modules physiques et numériques
     */
    bool init(MPI_Comm comm, const std::string& config_file);

    /**
     * @brief Exécute la boucle temporelle jusqu'à t_final
     */
    void run();

    /**
     * @brief Résout le problème de vérification du Laplacien analytique (Cas 3.1 Référence Académique)
     */
    void run_laplacian();

    /**
     * @brief Résout le problème d'advection d'une tâche dans un vortex avec cisaillement (Cas 3.19 Référence Académique)
     */
    void run_vortex_advection();

    /**
     * @brief Calcule le pas de temps adaptatif selon les critères CFL, viscosité et tension superficielle
     */
    double compute_time_step() const;
};

} // namespace covof
```

13.4 Répertoire des Classes, Structures et Méthodes de `src/Simulation.cpp`

Ligne	Catégorie	Identifiant C++17	Signature / Rôle
13	Méthode / Fonction	`Simulation::init`	Type retour : bool
218	Méthode / Fonction	`Simulation::compute_time_step`	Type retour : double
265	Méthode / Fonction	`Simulation::run`	Type retour : void
291	Méthode / Fonction	`std::setw`	Type retour : << std : :left <<
292	Méthode / Fonction	`std::setw`	Type retour : <<
293	Méthode / Fonction	`std::setw`	Type retour : <<
294	Méthode / Fonction	`std::setw`	Type retour : <<
295	Méthode / Fonction	`std::setw`	Type retour : <<
296	Méthode / Fonction	`std::setw`	Type retour : <<
299	Méthode / Fonction	`std::setw`	Type retour : std : :cout <<
305	Méthode / Fonction	`state_vector`	Type retour : std : :vector<double>
307	Méthode / Fonction	`rhs`	Type retour : std : :vector<double>
436	Méthode / Fonction	`std::setw`	Type retour : std : :cout << std : :left <<
437	Méthode / Fonction	`std::setw`	Type retour : <<
438	Méthode / Fonction	`std::setw`	Type retour : <<
439	Méthode / Fonction	`std::setw`	Type retour : <<
440	Méthode / Fonction	`std::setw`	Type retour : <<
441	Méthode / Fonction	`std::setw`	Type retour : <<
444	Méthode / Fonction	`std::setw`	Type retour : std : :cout <<
464	Méthode / Fonction	`std::string`	Type retour : std : :cout <<
471	Méthode / Fonction	`Simulation::run_laplacian`	Type retour : void
538	Méthode / Fonction	`rhs`	Type retour : std : :vector<double>
593	Méthode / Fonction	`diag`	Type retour : std : :vector<double>
604	Méthode / Fonction	`p_cg`	Type retour : std : :vector<double>
717	Méthode / Fonction	`ofs`	Type retour : std : :ofstream
728	Méthode / Fonction	`Simulation::run_vortex_advection`	Type retour : void
906	Méthode / Fonction	`ofs`	Type retour : std : :ofstream

13.5 Code Source Intégral Annoté : `src/Simulation.cpp`

Le listing ci-dessous présente l'intégralité des 924 lignes du fichier `src/Simulation.cpp` de l'implémentation C++17 :

```cpp
#include "covof/Simulation.hpp"
#include "covof/MpiComm.hpp"
#include <iostream>
#include <iomanip>
#include <fstream>
#include <algorithm>
#include <cmath>

namespace covof {
namespace fugu = covof; // Alias rétrocompatibilité

// Déclaration de variable typée
bool Simulation::init(MPI_Comm comm, const std::string& config_file) {
    // 1. Chargement de la configuration JSON
    if (!config.load_from_json(config_file)) {
        // Retour de fonction
        return false;
    }

    // Déclaration de variable typée
    int my_rank;
    // Appel de communication ou synchronisation parallèle OpenMPI
    MPI_Comm_rank(comm, &my_rank);
    // Évaluation de la condition de contrôle
    if (my_rank == 0) {
        config.print();
    }

    // 2. Initialisation du maillage partitionné
    bool periodic_x = (config.bc.left.type == BCType::PERIODIC || config.bc.right.type == BCType::PERIODIC);
    // Déclaration de variable typée
    bool periodic_y = (config.bc.bottom.type == BCType::PERIODIC || config.bc.top.type == BCType::PERIODIC);
    mesh.init(comm, config.nx, config.ny, config.lx, config.ly, periodic_x, periodic_y);

    // 3. Allocation des champs décalés MAC avec halos
    fields.allocate(mesh);

    // 4. Initialisation du module d'interface et des conditions initiales
    tracker.init(config);
    tracker.set_initial_condition(mesh, config.init_cond, fields);

```

```
    // Initialisation des conditions aux limites initiales
    MpiComm::apply_boundary_conditions(mesh, config.bc, fields, false);

    // Initialisation du canal amont pour profil POISEUILLE_STEP afin de garantir la continuité
    if (config.bc.left.type == BCType::DIRICHLET && config.bc.left.profile == "POISEUILLE_STEP") {
        // Déclaration de variable typée
        double y_step = config.bc.left.y_step;
        // Déclaration de variable typée
        double h = mesh.ly - y_step;
        // Déclaration de variable typée
        double u_mean = config.bc.left.u_mean;
        // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
        for (int j = 1; j <= mesh.ny_local; ++j) {
            // Déclaration de variable typée
            double y = mesh.yc(j);
            // Déclaration de variable typée
            double u_val = 0.0;
            // Évaluation de la condition de contrôle
            if (y >= y_step) {
                // Calcul et assignation numérique
                u_val = 6.0 * u_mean * (y - y_step) * (mesh.ly - y) / (h * h);
            }
            // Déclaration de variable typée
            int u_nx = mesh.nx_local + (mesh.is_boundary_right ? 1 : 0);
            // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
            for (int i = 1; i <= u_nx; ++i) {
                // Déclaration de variable typée
                double x = mesh.xu(i);
                // Évaluation de la condition de contrôle
                if (x <= 4.0 + 1e-9) {
                    // Calcul de la résolution géométrique de cellule
                    fields.u[mesh.u_idx(i, j)] = u_val;
                }
            }
        }
        MpiComm::sync_u(mesh, fields.u);
    }

    // Initialisation du champ de vitesse solénoïdal pour LIQUID_SHEET (Équations 17-18 Fluids 2021)
    if (config.init_cond.type == InitialConditionType::LIQUID_SHEET) {
        // Déclaration des variables physiques et géométriques
        double y_c = (config.init_cond.center_y > 0.0) ? config.init_cond.center_y : 0.5 * mesh.ly;
        // Déclaration de variable typée pour l'épaisseur de la nappe liquide
        double delta = (config.init_cond.thickness > 0.0) ? config.init_cond.thickness : 0.0003;
        // Déclaration de variable typée pour la longueur du domaine
        double L = mesh.lx;
        // Déclaration de variable typée pour le nombre d'onde
        double k_pert = 2.0 * M_PI / L;
        // Déclaration de variable typée pour l'amplitude de perturbation
        double amp = 0.04;
        // Facteur pour u assurant une divergence rigoureusement nulle (Eq. 17)
        double factor_u = amp * (L / (2.0 * M_PI)) * (2.0 / delta);

        // Composante horizontale u sur les faces décalées MAC en x
        int u_nx = mesh.nx_local + (mesh.is_boundary_right ? 1 : 0);
        // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
        for (int j = 1; j <= mesh.ny_local; ++j) {
            // Déclaration de variable typée
            double y = mesh.yc(j);
            // Déclaration de variable typée pour l'écart au centre de la nappe
            double dy_c = y - y_c;
            // Signe pour la divergence solénoïdale
            double sgn_y = (dy_c > 1.0e-12) ? 1.0 : ((dy_c < -1.0e-12) ? -1.0 : 0.0);
            // Décroissance exponentielle symétrique par rapport au plan médian
            double exp_decay = std::exp(-2.0 * std::abs(dy_c) / delta);
            // Boucle spatiale sur les faces u selon l'axe horizontal X
            for (int i = 1; i <= u_nx; ++i) {
                // Déclaration de variable typée
                double x = mesh.xu(i);
                // Vitesse moyenne convective A : 30 m/s dans le gaz, 2 m/s dans le liquide
                double c_u = 0.5 * (fields.C[mesh.cell_idx(i - 1, j)] + fields.C[mesh.cell_idx(i, j)]);
                // Interpolation de la vitesse moyenne de base A selon la présence de gaz/liquide
                double A_val = c_u * 30.0 + (1.0 - c_u) * 2.0;
                // Calcul et assignation numérique de u selon l'Équation 17
                fields.u[mesh.u_idx(i, j)] = A_val - sgn_y * factor_u * std::cos(k_pert * x) * exp_decay;
            }
        }

        // Composante verticale v sur les faces décalées MAC en y
        int v_ny = mesh.ny_local + (mesh.is_boundary_top ? 1 : 0);
        // Boucle spatiale sur les faces v selon l'axe vertical Y
        for (int j = 1; j <= v_ny; ++j) {
            // Déclaration de variable typée
            double y = mesh.yv(j);
            // Déclaration de variable typée pour l'écart au centre de la nappe
            double dy_c = y - y_c;
            // Décroissance exponentielle symétrique par rapport au plan médian
            double exp_decay = std::exp(-2.0 * std::abs(dy_c) / delta);
            // Boucle spatiale sur les cellules locales selon l'axe horizontal X
            for (int i = 1; i <= mesh.nx_local; ++i) {
                // Déclaration de variable typée
                double x = mesh.xc(i);
```

```
                // Calcul et assignation numérique de v selon l'Équation 17
                fields.v[mesh.v_idx(i, j)] = amp * std::sin(k_pert * x) * exp_decay;
            }
        }
        MpiComm::sync_u(mesh, fields.u);
        MpiComm::sync_v(mesh, fields.v);
        MpiComm::apply_boundary_conditions(mesh, config.bc, fields, false);
    }

    // Initialisation du champ de vitesse pour VISCOUS_JET (Cas 3.3 Fluids 2021)
    if (config.init_cond.type == InitialConditionType::VISCOUS_JET) {
        // Déclaration des variables physiques et géométriques
        double x_c = (config.init_cond.center_x > 0.0) ? config.init_cond.center_x : 0.5 * mesh.lx;
        // Déclaration de variable typée pour la demi-largeur de l'injecteur
        double half_w = (config.init_cond.jet_width > 0.0) ? 0.5 * config.init_cond.jet_width : 0.04;
        // Déclaration de la hauteur de l'amorce initiale de jet sous la paroi supérieure
        double stub_height = 2.0 * mesh.dy;
        // Vitesse d'injection négative selon Y (ajustée au débit discret exact D * v0)
        double v_inlet = (config.bc.top.v_val != 0.0) ? config.bc.top.v_val : -0.6;
        double d_disc = 0.0;
        for (int ig = 1; ig <= mesh.nx_global; ++ig) {
            double xg = (ig - 0.5) * mesh.dx;
            if (std::abs(xg - x_c) <= half_w) d_disc += mesh.dx;
        }
        if (d_disc < 1.0e-12) d_disc = 2.0 * half_w;
        double v_in_eff = v_inlet * (2.0 * half_w) / d_disc;

        // Composante verticale v sur les faces décalées MAC en y
        int v_ny = mesh.ny_local + (mesh.is_boundary_top ? 1 : 0);
        // Boucle spatiale sur les faces v selon l'axe vertical Y
        for (int j = 1; j <= v_ny; ++j) {
            // Déclaration de variable typée
            double y = mesh.yv(j);
            // Boucle spatiale sur les cellules locales selon l'axe horizontal X
            for (int i = 1; i <= mesh.nx_local; ++i) {
                // Déclaration de variable typée
                double x = mesh.xc(i);
                // Évaluation de l'appartenance à l'amorce initiale
                if (y >= mesh.ly - stub_height && std::abs(x - x_c) <= half_w) {
                    // Calcul et assignation numérique
                    fields.v[mesh.v_idx(i, j)] = v_in_eff;
                }
            }
        }
        MpiComm::sync_v(mesh, fields.v);
        MpiComm::apply_boundary_conditions(mesh, config.bc, fields, false);
    }

    // Initialisation du champ thermique si activé
    if (config.enable_energy) {
        // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
        for (int j = 1; j <= mesh.ny_local; ++j) {
            // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
            for (int i = 1; i <= mesh.nx_local; ++i) {
                // Déclaration de variable typée
                double frac = mesh.xc(i) / mesh.lx;
                // Calcul de la résolution géométrique de cellule
                fields.T[mesh.cell_idx(i, j)] = config.bc.left.T_val + frac * (config.bc.right.T_val - config.
                    bc.left.T_val);
            }
        }
        MpiComm::sync_cell_scalar(mesh, fields.T);
        MpiComm::apply_thermal_boundary_conditions(mesh, config.bc, fields);
    }

    // 5. Initialisation du solveur Navier-Stokes et des préconditionneurs
    ns_solver.init(mesh, config);
    ns_solver.assemble_preconditioner_blocks(fields, config.dt);

    // Calcul et assignation numérique
    precond = PreconditionerFactory::create(config.solver.precond_type);
    precond->init(&ns_solver);

    // Calcul et assignation numérique
    linear_solver = SolverFactory::create(config.solver.type);
    linear_solver->init(config.solver);

    // 6. Initialisation de l'exportateur VTK et diagnostics CSV
    vtk_writer.init(mesh, config.output.output_dir, config.output.case_name);

    // Retour de fonction
    return true;
}

// Déclaration de variable typée
double Simulation::compute_time_step() const {
    // Évaluation de la condition de contrôle
    if (!config.adaptive_dt) {
        // Retour de fonction
        return config.dt;
    }

```

```
225     // Déclaration de variable typée
226     double local_max_speed = 1.0e-6;
227     // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
228     for (int j = 1; j <= mesh.ny_local; ++j) {
229         // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
230         for (int i = 1; i <= mesh.nx_local; ++i) {
231             // Déclaration de variable typée
232             double u_mag = std::abs(fields.u[mesh.u_idx(i, j)]);
233             // Déclaration de variable typée
234             double v_mag = std::abs(fields.v[mesh.v_idx(i, j)]);
235             // Déclaration de variable typée
236             double speed = std::sqrt(u_mag * u_mag + v_mag * v_mag);
237             // Évaluation de la condition de contrôle
238             if (speed > local_max_speed) local_max_speed = speed;
239         }
240     }
241     // Déclaration de variable typée
242     double global_max_speed = MpiComm::global_max(mesh, local_max_speed);
243 
244     // 1. Critère CFL de convection
245     double dt_cfl = config.cfl_max * std::min(mesh.dx, mesh.dy) / global_max_speed;
246 
247     // 2. Critère capillaire (onde de surface) si tension de surface activée
248     double dt_sigma = 1.0e10;
249     // Évaluation de la condition de contrôle
250     if (config.enable_surface_tension && config.sigma > 0.0) {
251         // Déclaration de variable typée
252         double rho_sum = config.fluid1.rho + config.fluid2.rho;
253         // Déclaration de variable typée
254         double h_min = std::min(mesh.dx, mesh.dy);
255         // Calcul et assignation numérique
256         dt_sigma = std::sqrt((rho_sum * h_min * h_min * h_min) / (4.0 * M_PI * config.sigma));
257     }
258 
259     // Déclaration de variable typée
260     double dt_target = std::min(dt_cfl, dt_sigma);
261     // Retour de fonction
262     return std::min(config.dt, dt_target);
263 }
264 
265 void Simulation::run() {
266     // Évaluation de la condition de contrôle
267     if (config.mode == "LAPLACIAN") {
268         run_laplacian();
269         // Retour de fonction
270         return;
271     }
272     // Évaluation de la condition de contrôle
273     if (config.mode == "VORTEX_ADVECTION") {
274         run_vortex_advection();
275         // Retour de fonction
276         return;
277     }
278 
279     // Déclaration de variable typée
280     double t = 0.0;
281     // Déclaration de variable typée
282     int step = 0;
283 
284     // Sauvegarde initiale à t = 0.0
285     vtk_writer.write_step(mesh, fields, step, t);
286     vtk_writer.write_diagnostics(mesh, fields, step, t, config.dt);
287 
288     // Évaluation de la condition de contrôle
289     if (mesh.rank == 0) {
290         std::cout << "\n>>> DEMARRAGE DU CALCUL COVOF 2D PARALLELE <<<\n"
291                   << std::left << std::setw(8)  << "Pas"
292                   << std::setw(14) << "Temps [s]"
293                   << std::setw(12) << "dt [s]"
294                   << std::setw(12) << "Iter Krylov"
295                   << std::setw(16) << "Vitesse Max"
296                   << std::setw(16) << "Div Max";
297         // Évaluation de la condition de contrôle
298         if (config.enable_energy) {
299             std::cout << std::setw(14) << "Nu Moyen";
300         }
301         std::cout << "\n" << std::string(85, '-') << "\n";
302     }
303 
304     // Déclaration de variable typée
305     std::vector<double> state_vector(ns_solver.n_owned_total, 0.0);
306     // Déclaration de variable typée
307     std::vector<double> rhs(ns_solver.n_owned_total, 0.0);
308     // Déclaration de variable typée
309     std::vector<double> u_prev = fields.u;
310 
311     // Boucle itérative conditionnelle
312     while (t < config.t_final - 1.0e-12 * (std::abs(config.t_final) + 1.0)) {
313         step++;
314         // Calcul et assignation numérique
315         u_prev = fields.u;
316         // Déclaration de variable typée
```

```
317         double dt = compute_time_step();
318         // Évaluation de la condition de contrôle
319         if (t + dt > config.t_final) {
320             // Calcul et assignation numérique
321             dt = config.t_final - t;
322         }
323         // Évaluation de la condition de contrôle
324         if (dt < 1.0e-12) {
325             break;
326         }
327 
328         // 1. Transport de l'interface si écoulement diphasique
329         if (config.enable_two_phase) {
330             tracker.advect(mesh, fields, dt);
331             // Actualisation des conditions aux limites pour les mailles fantômes
332             MpiComm::apply_boundary_conditions(mesh, config.bc, fields, false);
333         }
334 
335         // 2. Assemblage des blocs du préconditionneur
336         ns_solver.assemble_preconditioner_blocks(fields, dt);
337 
338         // Déclaration de variable typée
339         int total_iters = 0;
340         // Déclaration de variable typée
341         int uzawa_max = ns_solver.is_augmented_lagrangian() ? ns_solver.uzawa_iter : 1;
342 
343         // Boucle d'itération algorithmique
344         for (int m = 0; m < uzawa_max; ++m) {
345             // 3. Construction du second membre rhs
346             ns_solver.assemble_rhs(fields, dt, rhs);
347 
348             // 4. Initialisation du vecteur d'état à partir des champs courants
349             ns_solver.pack_vector(fields, state_vector);
350 
351             // 5. Résolution du système linéaire par le solveur sélectionné
352             int iters = linear_solver->solve(ns_solver, *precond, fields, dt, rhs, state_vector);
353             // Calcul et assignation numérique
354             total_iters += iters;
355 
356             // 6. Déballage de la vitesse résolue
357             ns_solver.unpack_vector(state_vector, fields);
358 
359             // 7. Mise à jour de la pression par Lagrangien augmenté si applicable
360             if (ns_solver.is_augmented_lagrangian()) {
361                 ns_solver.update_pressure_al(fields);
362             }
363         }
364 
365         // 7.5 Résolution de l'énergie thermique si activée
366         if (config.enable_energy) {
367             ns_solver.solve_energy(fields, dt);
368         }
369 
370         // Calcul et assignation numérique
371         t += dt;
372 
373         // Calcul des métriques de contrôle
374         double local_max_u = 0.0;
375         // Déclaration de variable typée
376         double local_max_div = 0.0;
377         // Déclaration de variable typée
378         double local_du_max = 0.0;
379         // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
380         for (int j = 1; j <= mesh.ny_local; ++j) {
381             // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
382             for (int i = 1; i <= mesh.nx_local; ++i) {
383                 // Déclaration de variable typée
384                 double speed = std::sqrt(fields.u[mesh.u_idx(i, j)] * fields.u[mesh.u_idx(i, j)] +
385                                          fields.v[mesh.v_idx(i, j)] * fields.v[mesh.v_idx(i, j)]);
386                 // Évaluation de la condition de contrôle
387                 if (speed > local_max_u) local_max_u = speed;
388 
389                 // Déclaration de variable typée
390                 double du = std::abs(fields.u[mesh.u_idx(i, j)] - u_prev[mesh.u_idx(i, j)]);
391                 // Évaluation de la condition de contrôle
392                 if (du > local_du_max) local_du_max = du;
393 
394                 // Déclaration de variable typée
395                 double div = std::abs((fields.u[mesh.u_idx(i + 1, j)] - fields.u[mesh.u_idx(i, j)]) / mesh.dx
                     +
396                                       (fields.v[mesh.v_idx(i, j + 1)] - fields.v[mesh.v_idx(i, j)]) / mesh.dy)
                                          ;
397                 // Évaluation de la condition de contrôle
398                 if (div > local_max_div) local_max_div = div;
399             }
400         }
401         // Déclaration de variable typée
402         double global_max_u = MpiComm::global_max(mesh, local_max_u);
403         // Déclaration de variable typée
404         double global_max_div = MpiComm::global_max(mesh, local_max_div);
405         // Déclaration de variable typée
406         double global_du_max = MpiComm::global_max(mesh, local_du_max);
```

```
407
408         // Déclaration de variable typée
409         double nu_mean = 0.0;
410         // Évaluation de la condition de contrôle
411         if (config.enable_energy) {
412             // Calcul et assignation numérique
413             nu_mean = ns_solver.compute_mean_nusselt_hot_wall(fields);
414         }
415
416         // Détection de convergence stationnaire
417         if (config.steady_state_tol > 0.0 && step > 10 && global_du_max < config.steady_state_tol) {
418             // Évaluation de la condition de contrôle
419             if (mesh.rank == 0) {
420                 // Calcul et assignation numérique
421                 std::cout << "\n>>> CONVERGENCE STATIONNAIRE ATTEINTE : du_max = "
422                           // Calcul de la résolution géométrique de cellule
423                           << global_du_max << " < " << config.steady_state_tol << " (t = " << t << " s) <<<\n"
                                ;
424                 // Évaluation de la condition de contrôle
425                 if (config.enable_energy) {
426                     // Calcul et assignation numérique
427                     std::cout << ">>> NOMBRE DE NUSSELT FINAL : Nu = " << std::fixed << std::setprecision(5)
                                  << nu_mean << " <<<\n";
428                 }
429             }
430             vtk_writer.write_step(mesh, fields, step, t);
431             break;
432         }
433
434         // Affichage console
435         if (mesh.rank == 0 && (step % config.output.log_interval == 0)) {
436             std::cout << std::left << std::setw(8)  << step
437                       << std::setw(14) << std::fixed << std::setprecision(5) << t
438                       << std::setw(12) << std::scientific << std::setprecision(3) << dt
439                       << std::setw(12) << total_iters
440                       << std::setw(16) << std::scientific << std::setprecision(4) << global_max_u
441                       << std::setw(16) << std::scientific << std::setprecision(4) << global_max_div;
442             // Évaluation de la condition de contrôle
443             if (config.enable_energy) {
444                 std::cout << std::setw(14) << std::fixed << std::setprecision(4) << nu_mean;
445             }
446             std::cout << "\n";
447         }
448
449         // Écriture des diagnostics temporels
450         if (config.output.write_csv_history) {
451             vtk_writer.write_diagnostics(mesh, fields, step, t, dt);
452         }
453
454         // Export des fichiers VTK
455         if (step % config.output.vtk_interval == 0 || t >= config.t_final) {
456             vtk_writer.write_step(mesh, fields, step, t);
457         }
458     }
459
460     vtk_writer.finalize();
461
462     // Évaluation de la condition de contrôle
463     if (mesh.rank == 0) {
464         std::cout << std::string(78, '-') << "\n"
465                   // Calcul et assignation numérique
466                   << ">>> SIMULATION COVOF 2D TERMINEE AVEC SUCCES A t = " << t << " s <<<\n"
467                   << "Fichiers de visualisation VTK et CSV disponibles dans : " << config.output.output_dir <<
                        "\n";
468     }
469 }
470
471 void Simulation::run_laplacian() {
472     // Déclaration de variable typée
473     double a = config.lx;
474     // Déclaration de variable typée
475     double b = config.ly;
476     // Déclaration de variable typée
477     int nx = config.nx;
478     // Déclaration de variable typée
479     int ny = config.ny;
480     // Déclaration de variable typée
481     double dx = a / nx;
482     // Déclaration de variable typée
483     double dy = b / ny;
484
485     // Évaluation de la condition de contrôle
486     if (mesh.rank == 0) {
487         // Calcul et assignation numérique
488         std::cout << "\n============================================================\n"
489                   // Calcul et assignation numérique
490                   << "  VERIFICATION DU CAS 3.1 : LAPLACIEN DELTA(PSI) = 1     \n"
491                   << "  Domaine [" << a << " x " << b << "], Grille " << nx << " x " << ny << "\n"
492                   // Calcul et assignation numérique
493                   << "============================================================\n";
494     }
495
```

```
    // Inconnues nodales intérieures : 1 à nx-1, 1 à ny-1
    int n_int_x = nx - 1;
    // Déclaration de variable typée
    int n_int_y = ny - 1;
    // Déclaration de variable typée
    int n_dofs = n_int_x * n_int_y;

    CSRMatrix A;
    // Calcul et assignation numérique
    A.nrows = n_dofs;
    // Calcul et assignation numérique
    A.ncols = n_dofs;
    A.row_ptr.resize(n_dofs + 1, 0);

    // Calcul de la résolution géométrique de cellule
    auto dof_idx = [&](int i, int j) { return j * n_int_x + i; };

    // 1. Comptage des éléments non nuls
    for (int j = 0; j < n_int_y; ++j) {
        // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
        for (int i = 0; i < n_int_x; ++i) {
            // Déclaration de variable typée
            int row = dof_idx(i, j);
            // Déclaration de variable typée
            int count = 1;
            // Évaluation de la condition de contrôle
            if (i > 0) count++;
            // Évaluation de la condition de contrôle
            if (i < n_int_x - 1) count++;
            // Évaluation de la condition de contrôle
            if (j > 0) count++;
            // Évaluation de la condition de contrôle
            if (j < n_int_y - 1) count++;
            // Calcul et assignation numérique
            A.row_ptr[row + 1] = A.row_ptr[row] + count;
        }
    }

    A.col_idx.resize(A.row_ptr[n_dofs]);
    A.values.resize(A.row_ptr[n_dofs]);

    // Déclaration de variable typée
    std::vector<double> rhs(n_dofs, -1.0); // -Delta Psi = -1
    // Déclaration de variable typée
    std::vector<double> x(n_dofs, 0.0);

    // Déclaration de variable typée
    int cur = 0;
    // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
    for (int j = 0; j < n_int_y; ++j) {
        // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
        for (int i = 0; i < n_int_x; ++i) {
            // Déclaration de variable typée
            int row = dof_idx(i, j);
            // Évaluation de la condition de contrôle
            if (j > 0) {
                // Calcul de la résolution géométrique de cellule
                A.col_idx[cur] = dof_idx(i, j - 1);
                // Calcul et assignation numérique
                A.values[cur] = -1.0 / (dy * dy);
                cur++;
            }
            // Évaluation de la condition de contrôle
            if (i > 0) {
                // Calcul de la résolution géométrique de cellule
                A.col_idx[cur] = dof_idx(i - 1, j);
                // Calcul et assignation numérique
                A.values[cur] = -1.0 / (dx * dx);
                cur++;
            }
            // Calcul de la résolution géométrique de cellule
            A.col_idx[cur] = row;
            // Calcul et assignation numérique
            A.values[cur] = 2.0 / (dx * dx) + 2.0 / (dy * dy);
            cur++;
            // Évaluation de la condition de contrôle
            if (i < n_int_x - 1) {
                // Calcul de la résolution géométrique de cellule
                A.col_idx[cur] = dof_idx(i + 1, j);
                // Calcul et assignation numérique
                A.values[cur] = -1.0 / (dx * dx);
                cur++;
            }
            // Évaluation de la condition de contrôle
            if (j < n_int_y - 1) {
                // Calcul de la résolution géométrique de cellule
                A.col_idx[cur] = dof_idx(i, j + 1);
                // Calcul et assignation numérique
                A.values[cur] = -1.0 / (dy * dy);
                cur++;
            }
        }
```

```
    }

    // Solveur Gradient Conjugué préconditionné par Jacobi
    std::vector<double> r = rhs;
    // Déclaration de variable typée
    std::vector<double> diag(n_dofs, 1.0);
    // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
    for (int i = 0; i < n_dofs; ++i) {
        // Boucle d'itération algorithmique
        for (int p_idx = A.row_ptr[i]; p_idx < A.row_ptr[i+1]; ++p_idx) {
            // Évaluation de la condition de contrôle
            if (A.col_idx[p_idx] == i) { diag[i] = A.values[p_idx]; break; }
        }
    }

    // Déclaration de variable typée
    std::vector<double> p_cg(n_dofs, 0.0), z(n_dofs, 0.0), Ap(n_dofs, 0.0);
    // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
    for (int i = 0; i < n_dofs; ++i) z[i] = r[i] / diag[i];
    // Calcul et assignation numérique
    p_cg = z;
    // Déclaration de variable typée
    double rz_old = 0.0;
    // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
    for (int i = 0; i < n_dofs; ++i) rz_old += r[i] * z[i];

    // Déclaration de variable typée
    int max_it = 20000;
    // Déclaration de variable typée
    double tol = config.solver.tol > 0 ? config.solver.tol : 1e-12;
    // Déclaration de variable typée
    int iters = 0;

    // Boucle d'itération algorithmique
    for (iters = 0; iters < max_it; ++iters) {
        A.matvec(p_cg, Ap);
        // Déclaration de variable typée
        double pAp = 0.0;
        // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
        for (int i = 0; i < n_dofs; ++i) pAp += p_cg[i] * Ap[i];
        // Évaluation de la condition de contrôle
        if (std::abs(pAp) < 1e-30) break;
        // Déclaration de variable typée
        double alpha = rz_old / pAp;
        // Déclaration de variable typée
        double r_norm = 0.0;
        // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
        for (int i = 0; i < n_dofs; ++i) {
            // Calcul et assignation numérique
            x[i] += alpha * p_cg[i];
            // Calcul et assignation numérique
            r[i] -= alpha * Ap[i];
            // Calcul de norme euclidienne ou résidu de convergence
            r_norm += r[i] * r[i];
        }
        // Calcul de norme euclidienne ou résidu de convergence
        r_norm = std::sqrt(r_norm);
        // Évaluation de la condition de contrôle
        if (r_norm < tol) break;

        // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
        for (int i = 0; i < n_dofs; ++i) z[i] = r[i] / diag[i];
        // Déclaration de variable typée
        double rz_new = 0.0;
        // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
        for (int i = 0; i < n_dofs; ++i) rz_new += r[i] * z[i];
        // Déclaration de variable typée
        double beta = rz_new / rz_old;
        // Calcul et assignation numérique
        rz_old = rz_new;
        // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
        for (int i = 0; i < n_dofs; ++i) p_cg[i] = z[i] + beta * p_cg[i];
    }

    // Évaluation de la solution analytique exacte (formule analytique de référence)
    double err_l1 = 0.0, err_l2 = 0.0, err_linf = 0.0;
    // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
    for (int j = 0; j < n_int_y; ++j) {
        // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
        for (int i = 0; i < n_int_x; ++i) {
            // Déclaration de variable typée
            double xp = (i + 1) * dx;
            // Déclaration de variable typée
            double yp = (j + 1) * dy;
            // Déclaration de variable typée
            double psi_ex = 0.0;
            // Boucle d'itération algorithmique
            for (int k2 = 0; k2 < 60; ++k2) {
                // Boucle d'itération algorithmique
                for (int k1 = 0; k1 < 60; ++k1) {
                    // Déclaration de variable typée
                    double n1 = 2 * k1 + 1;
```

```
                    // Déclaration de variable typée
                    double n2 = 2 * k2 + 1;
                    // Déclaration de variable typée
                    double num = 16.0 * std::sin(n1 * M_PI * xp / a) * std::sin(n2 * M_PI * yp / b);
                    // Déclaration de variable typée
                    double den = (std::pow(M_PI, 4.0)) * n1 * n2 * (std::pow(n1 / a, 2.0) + std::pow(n2 / b,
                        2.0));
                    // Calcul et assignation numérique
                    psi_ex -= num / den;
                }
            }
            // Déclaration de variable typée
            double diff = std::abs(x[dof_idx(i, j)] - psi_ex);
            // Calcul et assignation numérique
            err_l1 += diff;
            // Calcul et assignation numérique
            err_l2 += diff * diff;
            // Évaluation de la condition de contrôle
            if (diff > err_linf) err_linf = diff;
        }
    }
    // Calcul et assignation numérique
    err_l1 /= n_dofs;
    // Calcul et assignation numérique
    err_l2 = std::sqrt(err_l2 / n_dofs);

    // Évaluation de la condition de contrôle
    if (mesh.rank == 0) {
        std::cout << ">>> RESULTAT RESOLUTION LAPLACIEN COVOF 2D <<<\n"
                  << "  Iterations : " << iters << "\n"
                  << "  Erreur L1   : " << std::scientific << std::setprecision(5) << err_l1 << "\n"
                  << "  Erreur L2   : " << std::scientific << std::setprecision(5) << err_l2 << "\n"
                  << "  Erreur Linf : " << std::scientific << std::setprecision(5) << err_linf << "\n"
                  // Calcul et assignation numérique
                  << "============================================================\n";

        // Déclaration de variable typée
        std::string err_file = config.output.output_dir + "/laplacian_error.csv";
        std::ofstream ofs(err_file);
        // Évaluation de la condition de contrôle
        if (ofs.is_open()) {
            ofs << "nx,ny,dx,dy,iters,err_l1,err_l2,err_linf\n"
                << nx << "," << ny << "," << dx << "," << dy << ","
                << iters << "," << err_l1 << "," << err_l2 << "," << err_linf << "\n";
            ofs.close();
        }
    }
}

void Simulation::run_vortex_advection() {
    // Déclaration de variable typée
    int nx = config.nx;
    // Déclaration de variable typée
    int ny = config.ny;
    // Déclaration de variable typée
    double T_period = (config.t_final > 0.0) ? config.t_final : 6.0;
    // Déclaration de variable typée
    double dt = config.dt;

    // Évaluation de la condition de contrôle
    if (mesh.rank == 0) {
        // Calcul et assignation numérique
        std::cout << "\n============================================================\n"
                  << "  VERIFICATION DU CAS 3.19 : ADVECTION VORTEX CISAILLEMENT\n"
                  // Calcul et assignation numérique
                  << "  Grille : " << nx << " x " << ny << " | Periode T = " << T_period << " s | dt = " << dt
                      << " s\n"
                  // Calcul et assignation numérique
                  << "============================================================\n";
    }

    // Sauvegarder la condition initiale C_exact
    std::vector<double> C_initial = fields.C;

    // Export initial
    vtk_writer.write_step(mesh, fields, 0, 0.0);

    // Déclaration de variable typée
    double t = 0.0;
    // Déclaration de variable typée
    int step = 0;

    // Déclaration de variable typée
    int u_nx = mesh.nx_local + (mesh.is_boundary_right ? 1 : 0);
    // Déclaration de variable typée
    int v_ny = mesh.ny_local + (mesh.is_boundary_top ? 1 : 0);

    // Boucle itérative conditionnelle
    while (t < T_period - 1.0e-12) {
        step++;
        // Évaluation de la condition de contrôle
        if (t + dt > T_period) dt = T_period - t;
```

```
770
771         // Déclaration de variable typée
772         double cos_factor = std::cos(M_PI * t / T_period);
773
774         // Imposer le champ de vitesse analytique sur les faces décalées
775         for (int j = 1; j <= mesh.ny_local; ++j) {
776             // Déclaration de variable typée
777             double y_c = mesh.yc(j);
778             // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
779             for (int i = 1; i <= u_nx; ++i) {
780                 // Déclaration de variable typée
781                 double x_u = mesh.xu(i);
782                 // Déclaration de variable typée
783                 double sx = std::sin(M_PI * x_u);
784                 // Déclaration de variable typée
785                 double sy = std::sin(2.0 * M_PI * y_c);
786                 // Calcul de la résolution géométrique de cellule
787                 fields.u[mesh.u_idx(i, j)] = - (sx * sx) * sy * cos_factor;
788             }
789         }
790
791         // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
792         for (int j = 1; j <= v_ny; ++j) {
793             // Déclaration de variable typée
794             double y_v = mesh.yv(j);
795             // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
796             for (int i = 1; i <= mesh.nx_local; ++i) {
797                 // Déclaration de variable typée
798                 double x_c = mesh.xc(i);
799                 // Déclaration de variable typée
800                 double sx = std::sin(2.0 * M_PI * x_c);
801                 // Déclaration de variable typée
802                 double sy = std::sin(M_PI * y_v);
803                 // Calcul de la résolution géométrique de cellule
804                 fields.v[mesh.v_idx(i, j)] = sx * (sy * sy) * cos_factor;
805             }
806         }
807
808         // Halos et CL
809         MpiComm::sync_u(mesh, fields.u);
810         MpiComm::sync_v(mesh, fields.v);
811
812         // Advection TVD de C
813         tracker.advect(mesh, fields, dt);
814
815         // Calcul et assignation numérique
816         t += dt;
817
818         // Évaluation de la condition de contrôle
819         if (step % config.output.vtk_interval == 0 || t >= T_period || std::abs(t - 0.5 * T_period) < 0.5 * dt
                ) {
820             vtk_writer.write_step(mesh, fields, step, t);
821         }
822
823         // Évaluation de la condition de contrôle
824         if (mesh.rank == 0 && (step % config.output.log_interval == 0 || t >= T_period)) {
825             // Calcul et assignation numérique
826             std::cout << "  [Vortex] Pas " << step << " | t = " << std::fixed << std::setprecision(4) << t
827                       << " s / " << T_period << " s\n";
828         }
829     }
830
831     // Calcul des erreurs L1, L2, Linf et du bilan de masse entre C_initial et C(T)
832     double local_err_l1 = 0.0;
833     // Déclaration de variable typée
834     double local_err_l2 = 0.0;
835     // Déclaration de variable typée
836     double local_err_linf = 0.0;
837     // Déclaration de variable typée
838     double local_mass_init = 0.0;
839     // Déclaration de variable typée
840     double local_mass_final = 0.0;
841     // Déclaration de variable typée
842     double local_cells = mesh.nx_local * mesh.ny_local;
843     // Déclaration de variable typée
844     double cell_vol = mesh.dx * mesh.dy;
845
846     // Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
847     for (int j = 1; j <= mesh.ny_local; ++j) {
848         // Boucle spatiale sur les cellules locales selon l'axe horizontal X (abscisses)
849         for (int i = 1; i <= mesh.nx_local; ++i) {
850             // Déclaration de variable typée
851             int idx = mesh.cell_idx(i, j);
852             // Déclaration de variable typée
853             double diff = std::abs(fields.C[idx] - C_initial[idx]);
854             // Calcul et assignation numérique
855             local_err_l1 += diff;
856             // Calcul et assignation numérique
857             local_err_l2 += diff * diff;
858             // Évaluation de la condition de contrôle
859             if (diff > local_err_linf) local_err_linf = diff;
860             // Calcul et assignation numérique
```

```
            local_mass_init += C_initial[idx] * cell_vol;
            // Calcul et assignation numérique
            local_mass_final += fields.C[idx] * cell_vol;
        }
    }

    // Réduction parallèle MPI globale de la somme de l'erreur L1
    double global_err_l1 = MpiComm::global_sum(mesh, local_err_l1);
    // Réduction parallèle MPI globale de la somme de l'erreur L2 au carré
    double global_err_l2 = MpiComm::global_sum(mesh, local_err_l2);
    // Réduction parallèle MPI globale de l'erreur maximale absolue Linf
    double global_err_linf = MpiComm::global_max(mesh, local_err_linf);
    // Réduction parallèle MPI globale de la masse totale initiale
    double global_mass_init = MpiComm::global_sum(mesh, local_mass_init);
    // Réduction parallèle MPI globale de la masse totale finale
    double global_mass_final = MpiComm::global_sum(mesh, local_mass_final);
    // Réduction parallèle MPI globale du nombre de cellules
    double global_cells = MpiComm::global_sum(mesh, local_cells);

    // Calcul de l'erreur moyenne normalisée L1
    double err_l1 = global_err_l1 / global_cells;
    // Calcul de l'erreur quadratique moyenne normalisée L2
    double err_l2 = std::sqrt(global_err_l2 / global_cells);
    // Erreur maximale locale Linf
    double err_linf = global_err_linf;
    // Taux de conservation relative de la masse du fluide
    double mass_conservation = (global_mass_init > 1.0e-14) ? (global_mass_final / global_mass_init) : 1.0;

    // Évaluation de la condition de contrôle
    if (mesh.rank == 0) {
        // Affichage console formaté du résultat de vérification
        std::cout << "\n>>> RESULTAT DU CAS 3.19 : ADVECTION VORTEX COVOF 2D <<<\n"
                  << "  Methode Advection  : " << config.vof.advection_method << "\n"
                  << "  Reconstruction     : " << config.vof.reconstruction_method << "\n"
                  << "  Limiteur TVD/NVD   : " << config.vof.tvd_limiter << "\n"
                  << "  Maillage           : " << nx << " x " << ny << "\n"
                  << "  Erreur L1          : " << std::scientific << std::setprecision(5) << err_l1 << "\n"
                  << "  Erreur L2          : " << std::scientific << std::setprecision(5) << err_l2 << "\n"
                  << "  Erreur Linf        : " << std::scientific << std::setprecision(5) << err_linf << "\n"
                  << "  Conservation Masse : " << std::fixed << std::setprecision(5) << (mass_conservation *
                      100.0) << " %\n"
                  << "============================================================\n";

        // Déclaration de variable typée
        std::string err_file = config.output.output_dir + "/vortex_error.csv";
        // Ouverture du flux d'écriture fichier
        std::ofstream ofs(err_file);
        // Évaluation de la condition de contrôle
        if (ofs.is_open()) {
            // Écriture de l'en-tête CSV enrichi avec toutes les métriques et métadonnées d'advection
            ofs << "nx,ny,dt,T,advection,reconstruction,limiter,err_l1,err_l2,err_linf,mass_init,mass_final,
                mass_conservation\n"
                << nx << "," << ny << "," << dt << "," << T_period << ","
                << config.vof.advection_method << "," << config.vof.reconstruction_method << "," << config.vof
                    .tvd_limiter << ","
                << err_l1 << "," << err_l2 << "," << err_linf << ","
                << global_mass_init << "," << global_mass_final << "," << mass_conservation << "\n";
            // Fermeture du fichier de sortie
            ofs.close();
        }
    }

    vtk_writer.finalize();
}

} // namespace covof
```

Deuxième partie

Guide du Développeur Fortran 2018 (COVOF_2D-FORTRAN)

Chapitre 14

Architecture Modulaire Fortran 2018, Précision IEEE-754 et Programme Principal (CMakeLists.txt, module$_{p}recision.f90$, module$_{c}onstants.f90$, covof2

14.1 Conception Modulaire selon la Norme Fortran 2018 (ISO/IEC 1539-1 :2018)

L'implémentation **Fortran 2018** (`COVOF_2D-FORTRAN`) est le miroir algorithmique et mathématique exact de `COVOF_2D-C++`. Elle est structurée en 13 modules modernes (`implicit none` systématique, attributs `intent(in)`, `intent(out)`, `intent(inout)`, procédures attachées aux types dérivés et tableaux allocatables à bornes personnalisées) et un programme principal `covof2d_main.f90`.

14.1.1 Gestion de la Précision Flottante (`module_precision.f90`)

Le module `module_precision` définit le paramètre de genre (*kind parameter*) double précision IEEE-754 portable sur 64 bits :

```
integer, parameter :: dp = selected_real_kind(15, 307)
```
(14.1)

garantissant au moins 15 chiffres décimaux significatifs de mantisse et une plage d'exposants décimaux de ±307, strictement identique au type `double` de C++17.

14.1.2 Table de Correspondance Bijective C++17 ↔ Fortran 2018

Composant C++17 (`COVOF_2D-C++`)	Module Fortran 2018 (`COVOF_2D-FORTRAN`)
`include/covof/Types.hpp`	`src/module_precision.f90` + `src/module_constar`
`src/Config.cpp` (`covof::Config`)	`src/module_json.f90` + `src/module_config.f90`
`src/Mesh.cpp` (`covof::Mesh`)	`src/module_mesh.f90` (`type(mesh_2d_t)`)
`include/covof/Field.hpp` (`Field2D<T>`)	`src/module_fields.f90` (`type(flow_fields_t)`)
`src/MpiComm.cpp` (`covof::MpiComm`)	`src/module_mpi_comm.f90` (`type(mpi_context_t)`)
`src/CSRMatrix.cpp` + `Preconditioner.cpp` + `LinearSolver.cpp`	`src/module_linear_solvers.f90`
`src/NavierStokesSolver.cpp`	`src/module_navier_stokes.f90`
`src/InterfaceReconstruction.cpp`	`src/module_interface_reconstruction.f90`
`src/InterfaceTracker.cpp`	`src/module_interface_vof.f90`
`src/VTKWriter.cpp`	`src/module_vtk_writer.f90`
`src/Simulation.cpp` + `src/main.cpp`	`src/module_simulation.f90` + `src/covof2d_main`

14.2 Code Source Intégral Annoté : `CMakeLists.txt`

Le listing ci-dessous présente l'intégralité des 42 lignes du fichier `CMakeLists.txt` de l'implémentation Fortran 2018 :

```
1 cmake_minimum_required(VERSION 3.18)
2 project(COVOF_2D_FORTRAN LANGUAGES Fortran)
```

```
3
4  set(CMAKE_Fortran_STANDARD 2018)
5  set(CMAKE_Fortran_STANDARD_REQUIRED ON)
6
7  find_package(MPI REQUIRED)
8
9  set(CMAKE_Fortran_FLAGS "${CMAKE_Fortran_FLAGS} -O3 -march=native")
10
11 set(SOURCES
12     src/module_precision.f90
13     src/module_constants.f90
14     src/module_json.f90
15     src/module_config.f90
16     src/module_mesh.f90
17     src/module_fields.f90
18     src/module_mpi_comm.f90
19     src/module_interface_reconstruction.f90
20     src/module_interface_vof.f90
21     src/module_navier_stokes.f90
22     src/module_linear_solvers.f90
23     src/module_vtk_writer.f90
24     src/module_simulation.f90
25     src/covof2d_main.f90
26 )
27
28 # Executable target: covof2d_fortran
29 add_executable(covof2d_fortran ${SOURCES})
30 target_link_libraries(covof2d_fortran PRIVATE MPI::MPI_Fortran)
31
32 set_target_properties(covof2d_fortran PROPERTIES
33     RUNTIME_OUTPUT_DIRECTORY "${CMAKE_CURRENT_SOURCE_DIR}/bin"
34 )
35
36 # Create compatibility symlink for fugu2d_fortran
37 add_custom_command(TARGET covof2d_fortran POST_BUILD
38     COMMAND ${CMAKE_COMMAND} -E create_symlink
39         ${CMAKE_CURRENT_SOURCE_DIR}/bin/covof2d_fortran
40         ${CMAKE_CURRENT_SOURCE_DIR}/bin/fugu2d_fortran
41     COMMENT "Creating compatibility symlink bin/fugu2d_fortran -> bin/covof2d_fortran"
42 )
```

14.3 Répertoire des Types Dérivés, Sous-Routines et Fonctions de `src/module_precision.f90`

Ligne	Nature	Identifiant Fortran 2018	Interface / Arguments
7	module	`module_precision`	Unité modulaire Fortran 2018

14.4 Code Source Intégral Annoté : `src/module_precision.f90`

Le listing ci-dessous présente l'intégralité des 33 lignes du fichier `src/module_precision.f90` de l'implémentation Fortran 2018 :

```
1  !===============================================================================
2  ! Module : module_precision
3  ! Description : Définition standardisée de la précision arithmétique flottante
4  !               et entière pour l'ensemble du solveur CFD COVOF 2D (Fortran 2018).
5  ! Conforme aux recommandations IEEE 754 et ISO_FORTRAN_ENV.
6  !===============================================================================
7  module module_precision
8    ! Obligation stricte de déclarer tous les types de variables (évite le typage implicite)
9    implicit none
10   ! Encapsulation par défaut : aucune entité n'est visible de l'extérieur sans export
11   private
12
13   ! Précision double (Double Precision 64-bit IEEE 754) :
14   ! - Au moins 15 chiffres significatifs en base 10 (mantisse de 53 bits)
15   ! - Plage dynamique d'exposant de 10^-307 à 10^+307 (exposant de 11 bits)
16   ! - Utilisée pour tous les calculs hydrodynamiques, de pression et de transport
17   integer, parameter, public :: dp = selected_real_kind(15, 307)
18
19   ! Précision simple (Single Precision 32-bit IEEE 754) :
20   ! - Au moins 6 chiffres significatifs en base 10 (mantisse de 24 bits)
21   ! - Plage dynamique d'exposant de 10^-37 à 10^+37 (exposant de 8 bits)
22   ! - Utilisée optionnellement pour les exports VTK légers ou diagnostics rapides
23   integer, parameter, public :: sp = selected_real_kind(6, 37)
24
25   ! Entier standard 32 bits signé (-2*10^9 à +2*10^9) :
26   ! - Utilisé pour l'indexation spatiale des mailles (i, j) et les compteurs de boucle
27   integer, parameter, public :: i4 = selected_int_kind(9)
28   ! Alias d'entier par défaut pour le maillage et les solveurs
29   integer, parameter, public :: ip = i4
30   ! Entier 64 bits signé (-9*10^18 à +9*10^18) pour les grands volumes de données
31   integer, parameter, public :: i8 = selected_int_kind(18)
```

```
32
33 end module module_precision
```

14.5 Répertoire des Types Dérivés, Sous-Routines et Fonctions de `src/module_constants.f90`

Ligne	Nature	Identifiant Fortran 2018	Interface / Arguments
7	module	`module_constants`	Unité modulaire Fortran 2018

14.6 Code Source Intégral Annoté : `src/module_constants.f90`

Le listing ci-dessous présente l'intégralité des 111 lignes du fichier `src/module_constants.f90` de l'implémentation Fortran 2018 :

```
1  !===============================================================================
2  ! Module : module_constants
3  ! Description : Définition des constantes physiques fondamentales, paramètres
4  !               numériques seuils, énumérations de conditions aux limites, types
5  !               de solveurs et schémas convectifs pour le code COVOF 2D.
6  !===============================================================================
7  module module_constants
8    ! Importation du paramètre de précision double standardisé
9    use module_precision, only: dp
10   ! Obligation stricte de typage explicite pour toutes les constantes
11   implicit none
12   ! Encapsulation : seules les constantes marquées 'public' sont exportées
13   private
14
15   ! --- 1. Constantes Mathématiques et Physiques Fondamentales ---
16   ! Valeur exacte du nombre Pi (35 décimales significatives en précision double)
17   real(dp), parameter, public :: PI = 3.14159265358979323846264338327950288 4197_dp
18   ! Seuil numérique infinitésimal pour prévenir les divisions par zéro dans les gradients
19   real(dp), parameter, public :: EPS_SMALL = 1.0e-30_dp
20   ! Perméabilité de Darcy par défaut dans les obstacles solides pénalisés (loi de Brinkman)
21   real(dp), parameter, public :: PERMEABILITY_SOLID_DEFAULT = 1.0e-40_dp
22
23   ! --- 2. Conditions aux Limites Hydrodynamiques (Vitesse) ---
24   ! Vitesse imposée (Adhérence u=0, paroi mobile de cavité, ou profil d'entrée de Poiseuille)
25   integer, parameter, public :: BC_DIRICHLET = 1
26   ! Sortie libre convective ou gradient normal nul (du/dn = 0, dv/dn = 0)
27   integer, parameter, public :: BC_NEUMANN   = 2
28   ! Raccordement spatial périodique (face Est reliée à la face Ouest, Nord à Sud)
29   integer, parameter, public :: BC_PERIODIC  = 3
30   ! Glissement parfait sans frottement tangentiel (vitesse normale nulle, cisaillement nul)
31   integer, parameter, public :: BC_FREESLIP  = 4
32
33   ! --- 3. Conditions aux Limites Thermiques (Énergie) ---
34   ! Température pariétale imposée (Paroi isotherme chaude ou froide de Vahl Davis)
35   integer, parameter, public :: THERMAL_DIRICHLET = 1
36   ! Flux thermique imposé ou paroi adiabatique sans transfert de chaleur (dT/dn = 0)
37   integer, parameter, public :: THERMAL_NEUMANN   = 2
38
39   ! --- 4. Formulations du Couplage Vitesse-Pression Navier-Stokes ---
40   ! Méthode du Lagrangien Augmenté avec pénalité de divergence et actualisation d'Uzawa
41   integer, parameter, public :: SOLVER_AUGMENTED_LAGRANGIAN = 1
42   ! Formulation couplée monolithique directe avec intégration du gradient de pression
43   integer, parameter, public :: SOLVER_MONOLITHIC           = 2
44
45   ! --- 5. Algorithmes de Résolution Linéaire de Krylov (Matrix-Free) ---
46   ! Biconjugué Gradient Stabilisé d'ordre 2 (Sleijpen & van der Vorst) pour systèmes asymétriques
47   integer, parameter, public :: LINEAR_BICGSTAB2 = 1
48   ! Biconjugué Gradient Stabilisé standard (van der Vorst 1992)
49   integer, parameter, public :: LINEAR_BICGSTAB  = 2
50   ! Gradient Conjugué Préconditionné pour opérateurs symétriques définis positifs (Poisson)
51   integer, parameter, public :: LINEAR_CG        = 3
52   ! Generalized Minimal Residual (Saad & Schultz 1986) avec orthogonalisation de Krylov
53   integer, parameter, public :: LINEAR_GMRES     = 4
54   ! Solveur direct bande ou factorisation LU locale
55   integer, parameter, public :: LINEAR_DIRECT    = 5
56
57   ! --- 6. Préconditionneurs Linéaires Parallèles ---
58   ! Aucun préconditionnement (résolution brute de l'opérateur non préconditionné)
59   integer, parameter, public :: PRECOND_NONE   = 0
60   ! Préconditionneur diagonal par blocs de Jacobi (inversion de la diagonale locale)
61   integer, parameter, public :: PRECOND_JACOBI = 1
62   ! Symmetric Successive Over-Relaxation (SSOR) pour lissage spectral rapide
63   integer, parameter, public :: PRECOND_SSOR   = 2
64   ! Factorisation incomplète Incomplete LU sans remplissage (ILU(0))
65   integer, parameter, public :: PRECOND_ILU0   = 3
66
67   ! --- 7. Schémas de Discrétisation Spatiale des Termes Convectifs ---
68   ! Schéma décentré amont d'ordre 1 (Upwind) inconditionnellement stable mais diffusif
69   integer, parameter, public :: CONV_UPWIND       = 1
```

```
  ! Schéma centré d'ordre 2 (Centered) sans dissipation artificielle
  integer, parameter, public :: CONV_CENTERED     = 2
  ! Schéma TVD Superbee à haute résolution avec limiteur de flux non linéaire (Roe 1986)
  integer, parameter, public :: CONV_TVD_SUPERBEE = 3

  ! --- 8. Modèles de Viscosité Dynamique du Fluide ---
  ! Fluide newtonien homogène à viscosité dynamique constante mu = mu0
  integer, parameter, public :: VISC_CONSTANT    = 1
  ! Fluide à viscosité variable spatiale exponentielle mu(y) = mu0 * exp(a * (y-e)^2 / e^2)
  integer, parameter, public :: VISC_EXPONENTIAL = 2

  ! --- 9. Types de Configurations Initiales et Cas de Vérification ---
  ! Écoulement monophasique uniforme avec fluide au repos (u=0, v=0, C=1)
  integer, parameter, public :: IC_UNIFORM          = 1
  ! Profil parabolique d'entrée de Poiseuille établi
  integer, parameter, public :: IC_POISEUILLE       = 2
  ! Tâche circulaire d'interface transportée dans un vortex cisaillé réversible
  integer, parameter, public :: IC_VORTEX_SHEAR     = 3
  ! Instabilité diphasique gravitationnelle de Rayleigh-Taylor avec interface perturbée
  integer, parameter, public :: IC_RAYLEIGH_TAYLOR  = 4
  ! Gouttelette liquide statique immergée pour la vérification de la loi de Laplace
  integer, parameter, public :: IC_DROPLET          = 5
  ! Bulle de gaz ascendante déformable dans un liquide visqueux (Benchmark de Hysing)
  integer, parameter, public :: IC_BUBBLE           = 6
  ! Effondrement d'une colonne d'eau sous gravité (Rupture de barrage de Martin & Moyce)
  integer, parameter, public :: IC_DAM_BREAK        = 7
  ! Test analytique d'ordre de convergence de l'opérateur Laplacien discret à 5 points
  integer, parameter, public :: IC_LAPLACIAN_TEST   = 8
  ! Nappe liquide cisaillée périodique (Cas 3.2 de l'article Fluids 2021)
  integer, parameter, public :: IC_LIQUID_SHEET     = 9
  ! Jet liquide visqueux injecté / flambement (Cas 3.3 de l'article Fluids 2021)
  integer, parameter, public :: IC_VISCOUS_JET      = 10

  ! --- 10. Formes Géométriques d'Obstacles Immergés (Brinkman) ---
  ! Aucun obstacle immergé dans le domaine fluide
  integer, parameter, public :: OBS_NONE     = 0
  ! Obstacle rectangulaire défini par ses coordonnées minimales et maximales (x0, y0, x1, y1)
  integer, parameter, public :: OBS_BOX      = 1
  ! Obstacle circulaire cylindrique défini par son centre (xc, yc) et son rayon R
  integer, parameter, public :: OBS_CYLINDER = 2

end module module_constants
```

14.7 Code Source Intégral Annoté : `src/covof2d_main.f90`

Le listing ci-dessous présente l'intégralité des 105 lignes du fichier `src/covof2d_main.f90` de l'implémentation Fortran 2018 :

```
!===============================================================================
! Projet         : COVOF 2D (Convection & Volume-Of-Fluid 2D CFD Solver)
! Fichier        : covof2d_main.f90 (Point d'entrée principal Fortran 2018 / MPI)
! Description    : Programme principal pilotant l'initialisation de l'environnement
!                  OpenMPI, le traitement des arguments CLI, la lecture du cas
!                  JSON, l'orchestration du calcul Navier-Stokes / VOF / Énergie,
!                  et la libération finale des ressources système.
! Auteur         : Elhadji Ibrahima Thiam
! Standard       : Fortran 2018 / MPI-3
!===============================================================================
program covof2d_main
  ! Importation du module orchestrateur de la simulation CFD
  use module_simulation, only: simulation_t
  ! Typage explicite obligatoire pour toutes les variables locales
  implicit none
  ! Inclusion des définitions de constantes et prototypes MPI Fortran
  include 'mpif.h'

  ! --- Déclarations des Variables Locales ---
  integer             :: rank       ! Rang unique du processus courant (0 .. size-1)
  integer             :: size       ! Nombre total de processus alloués au calcul
  integer             :: ierr       ! Code d'erreur retourné par les appels MPI
  integer             :: num_args   ! Nombre d'arguments passés en ligne de commande
  integer             :: stat       ! Statut de lecture de l'argument CLI
  character(len=256) :: config_file! Chemin vers le fichier de configuration JSON
  character(len=256) :: arg         ! Tampon de lecture d'un argument textuel
  type(simulation_t) :: sim         ! Instance du type dérivé orchestrateur
  logical             :: success    ! Indicateur booléen de réussite de l'initialisation

  ! 1. Initialisation du runtime parallèle OpenMPI
  call MPI_Init(ierr)

  ! Récupération du rang du processus dans le communicateur global MPI_COMM_WORLD
  call MPI_Comm_rank(MPI_COMM_WORLD, rank, ierr)

  ! Récupération de la taille du groupe de processus MPI
  call MPI_Comm_size(MPI_COMM_WORLD, size, ierr)

  ! Fichier de configuration par défaut si aucun argument n'est fourni
  config_file = "cases/cavity.json"
```

```
41
42    ! Lecture du nombre d'arguments passés au binaire lors de l'appel mpirun
43    num_args = command_argument_count()
44
45    ! 2. Analyse des arguments passés en ligne de commande
46    if (num_args >= 1) then
47       ! Récupération du premier argument textuel
48       call get_command_argument(1, arg, status=stat)
49
50       ! Détection de la demande d'affichage d'aide (-h ou --help)
51       if (trim(arg) == "-h" .or. trim(arg) == "--help") then
52          ! Seul le processus maître (rang 0) écrit sur la sortie standard
53          if (rank == 0) then
54             write(*, '(A)') "================================================================================"
55             write(*, '(A)') "   COVOF 2D -- Solveur CFD Navier-Stokes Incompressible Parallèle (Fortran 2018)"
56             write(*, '(A)') "================================================================================"
57             write(*, '(A)') "Usage :"
58             write(*, '(A)') "  mpirun -np <N> ./bin/covof2d_fortran [fichier_configuration.json]"
59             write(*, '(A)') ""
60             write(*, '(A)') "Exemples d'exécution :"
61             write(*, '(A)') "  mpirun -np 2 ./bin/covof2d_fortran cases/cas3_1_laplacian.json"
62             write(*, '(A)') "  mpirun -np 2 ./bin/covof2d_fortran cases/cas4_5_cavity.json"
63             write(*, '(A)') "  mpirun -np 4 ./bin/covof2d_fortran cases/cas7_5_rising_bubble.json"
64             write(*, '(A)') "  mpirun -np 2 ./bin/covof2d_fortran cases/cas6_1_natural_convection.json"
65             write(*, '(A)') "================================================================================"
66          end if
67          ! Clôture propre du communicateur MPI avant terminaison
68          call MPI_Finalize(ierr)
69          stop
70       end if
71
72       ! Affectation du chemin de fichier spécifié
73       config_file = trim(arg)
74    end if
75
76    ! Affichage du récapitulatif de lancement par le rang racine
77    if (rank == 0) then
78       write(*, '(A)') "================================================================================"
79       write(*, '(A)') "   COVOF 2D -- Solveur CFD Navier-Stokes Incompressible Parallèle (Fortran 2018)"
80       write(*, '(A)') "================================================================================"
81       write(*, '(A,I0)')       "  Nombre de processus MPI  : ", size
82       write(*, '(A,A)')        "  Fichier de cas JSON      : ", trim(config_file)
83       write(*, '(A)') "================================================================================"
84    end if
85
86    ! 3. Initialisation de la simulation CFD (lecture JSON, partitionnement MAC, allocations)
87    success = sim%init(MPI_COMM_WORLD, trim(config_file))
88    if (.not. success) then
89       ! En cas d'erreur de parsing ou d'incohérence des données physiques
90       if (rank == 0) then
91          write(*, '(A)') "[ERREUR CRITIQUE] Échec de l'initialisation de COVOF 2D Fortran."
92          write(*, '(A)') "                  Vérifiez le fichier de configuration : " // trim(config_file)
93       end if
94       ! Interruption immédiate et coordonnée de tous les rangs MPI
95       call MPI_Abort(MPI_COMM_WORLD, 1, ierr)
96       stop 1
97    end if
98
99    ! 4. Exécution de la boucle temporelle principale de simulation
100   call sim%run()
101
102   ! 5. Libération des ressources MPI et terminaison normale
103   call MPI_Finalize(ierr)
104
105 end program covof2d_main
```

Chapitre 15

Analyseur Syntaxique JSON Natif en Fortran 2018 et Module de Configuration (module$_{j}son.f90, module_{c}onfig.f90$)

15.1 Architecture du Parseur JSON Pur Fortran 2018 (`module_json.f90`)

Contrairement à Python ou C++, l'écosystème Fortran standard ne dispose pas nativement d'un parseur JSON dans la bibliothèque système. Le module `module_json.f90` (449 lignes) implémente intégralement en Fortran 2018 standard un analyseur lexical et syntaxique capable de lire exactement les mêmes fichiers `cases/*.json` que l'exécutable C++17 :

1. `json_read_file(filename, content, ierr)` : charge le fichier JSON complet dans une chaîne allocatable `character(len=:), allocatable :: content`.
2. `json_extract_block(json_str, block_name, block_content, found)` : localise un objet JSON nommé (tel que `"mesh"` ou `"vof"`) en suivant l'équilibrage des accolades `{` et `}` hors chaînes de caractères.
3. `json_get_real`, `json_get_int`, `json_get_logical`, `json_get_string` : extraient les valeurs typées avec repli automatique sur la valeur par défaut si la clé est absente.
4. `json_parse_obstacles` : parcourt le tableau JSON `"obstacles": [ {...}, {...} ]` pour instancier les masques de pénalisation de Brinkman (`box` et `circle`).

15.2 Structure de Configuration `simulation_config_t` (`module_config.f90`)

Le module `module_config.f90` (493 lignes) traduit les chaînes JSON en constantes entières symboliques (`BC_WALL`, `BC_PERIODIC`, `BC_JET_INLET`, `SOLVER_BICGSTAB2`, `RECON_PLIC`, `ADVECT_WEYMOUTH_YUE`, etc.) et vérifie la cohérence des paramètres physiques et numériques avant l'allocation du maillage.

15.3 Répertoire des Types Dérivés, Sous-Routines et Fonctions de `src/module_json.f90`

Ligne	Nature	Identifiant Fortran 2018	Interface / Arguments
2	module	`module_json`	Unité modulaire Fortran 2018
18	subroutine	`json_read_file`	Arguments : (filename, buffer, ierr)
79	subroutine	`json_find_value_str`	Arguments : (buffer, path, val_str, found)
254	subroutine	`json_get_string`	Arguments : (buffer, path, val, default_val)
278	subroutine	`json_get_real`	Arguments : (buffer, path, val, default_val)
308	subroutine	`json_get_int`	Arguments : (buffer, path, val, default_val)
338	subroutine	`json_get_bool`	Arguments : (buffer, path, val, default_val)
371	subroutine	`json_get_obstacle`	Arguments : (buffer, obs_type, x0, y0, x1, y1, xc, y...)

15.4 Code Source Intégral Annoté : `src/module_json.f90`

Le listing ci-dessous présente l'intégralité des 449 lignes du fichier `src/module_json.f90` de l'implémentation Fortran 2018 :

```
! [Procédure/Module] module module_json
module module_json
  use module_precision, only: dp
  use module_constants, only: OBS_NONE, OBS_BOX, OBS_CYLINDER
  implicit none
  private

  public :: json_read_file
  public :: json_get_string
  public :: json_get_real
  public :: json_get_int
  public :: json_get_bool
  public :: json_get_obstacle

contains

  ! Lit un fichier complet dans une chaîne de caractères allouable en ignorant les commentaires //
  subroutine json_read_file(filename, buffer, ierr)
    ! Déclaration de variable locale / paramètre de calcul
    character(len=*), intent(in) :: filename
    ! Déclaration de variable locale / paramètre de calcul
    character(len=:), allocatable, intent(out) :: buffer
    ! Déclaration de variable locale / paramètre de calcul
    integer, intent(out) :: ierr

    ! Déclaration de variable locale / paramètre de calcul
    integer :: u, io
    ! Déclaration de variable locale / paramètre de calcul
    character(len=4096) :: line
    ! Déclaration de variable locale / paramètre de calcul
    character(len=:), allocatable :: temp
    ! Déclaration de variable locale / paramètre de calcul
    integer :: comment_pos

    ! Évaluation de la norme du résidu ou de l'erreur numérique
    ierr = 0
    ! Affectation et calcul numérique
    buffer = ""
    ! Affectation et calcul numérique
    open(newunit=u, file=trim(filename), status='old', action='read', iostat=io)
    ! Évaluation de la condition logique
    if (io /= 0) then
       ! Évaluation de la norme du résidu ou de l'erreur numérique
       ierr = io
       ! Retour à la procédure appelante
       return
    end if

    do
       ! Affectation et calcul numérique
       read(u, '(A)', iostat=io) line
       ! Affectation et calcul numérique
       if (io /= 0) exit
       ! Affectation et calcul numérique
       comment_pos = index(line, '//')
       ! Affectation et calcul numérique
       if (comment_pos > 0) line = line(1:comment_pos-1)
       ! Affectation et calcul numérique
       line = adjustl(line)
       ! Évaluation de la condition logique
       if (len_trim(line) > 0) then
          ! Évaluation de la condition logique
          if (len(buffer) == 0) then
             ! Affectation et calcul numérique
             buffer = trim(line)
          ! Branche par défaut si condition non satisfaite
          else
             ! Affectation et calcul numérique
             temp = buffer // " " // trim(line)
             ! Affectation et calcul numérique
             buffer = temp
          end if
       end if
    end do
    close(u)
  end subroutine json_read_file

  ! Recherche d'un chemin séparé par des points dans le buffer JSON
  subroutine json_find_value_str(buffer, path, val_str, found)
    ! Déclaration de variable locale / paramètre de calcul
    character(len=*), intent(in) :: buffer
    ! Déclaration de variable locale / paramètre de calcul
    character(len=*), intent(in) :: path
    ! Déclaration de variable locale / paramètre de calcul
    character(len=:), allocatable, intent(out) :: val_str
    ! Déclaration de variable locale / paramètre de calcul
```

```
    logical, intent(out) :: found

    ! Déclaration de variable locale / paramètre de calcul
    character(len=:), allocatable :: cur_buf
    ! Déclaration de variable locale / paramètre de calcul
    integer :: p_dot, pos_key, pos_colon
    ! Déclaration de variable locale / paramètre de calcul
    character(len=64) :: key, sub_path
    ! Déclaration de variable locale / paramètre de calcul
    character(len=70) :: search_key
    ! Déclaration de variable locale / paramètre de calcul
    integer :: brace_count, start_pos, end_pos

    ! Affectation et calcul numérique
    found = .false.
    ! Affectation et calcul numérique
    val_str = ""
    ! Affectation et calcul numérique
    cur_buf = buffer

    ! Affectation et calcul numérique
    sub_path = adjustl(trim(path))

    ! Boucle d'itération numérique
    do while (len_trim(sub_path) > 0)
       ! Affectation et calcul numérique
       p_dot = index(sub_path, ".")
       ! Évaluation de la condition logique
       if (p_dot > 0) then
          ! Affectation et calcul numérique
          key = sub_path(1:p_dot-1)
          ! Affectation et calcul numérique
          sub_path = adjustl(sub_path(p_dot+1:))
       ! Branche par défaut si condition non satisfaite
       else
          ! Affectation et calcul numérique
          key = trim(sub_path)
          ! Affectation et calcul numérique
          sub_path = ""
       end if

       ! Affectation et calcul numérique
       search_key = '"' // trim(key) // '"'
       ! Affectation et calcul numérique
       pos_key = index(cur_buf, trim(search_key))
       ! Évaluation de la condition logique
       if (pos_key <= 0) then
          ! Affectation et calcul numérique
          found = .false.
          ! Retour à la procédure appelante
          return
       end if

       ! Trouver les deux-points ':' après la clé
       pos_colon = pos_key + len_trim(search_key)
       ! Boucle d'itération numérique
       do while (pos_colon <= len(cur_buf) .and. cur_buf(pos_colon:pos_colon) /= ':')
          ! Affectation et calcul numérique
          pos_colon = pos_colon + 1
       end do
       ! Évaluation de la condition logique
       if (pos_colon > len(cur_buf)) then
          ! Affectation et calcul numérique
          found = .false.
          ! Retour à la procédure appelante
          return
       end if

       ! Si ce n'est pas le dernier segment de chemin, chercher l'objet { ... } ou [ ... ] correspondant
       if (len_trim(sub_path) > 0) then
          ! Affectation et calcul numérique
          start_pos = pos_colon + 1
          ! Boucle d'itération numérique
          do while (start_pos <= len(cur_buf) .and. cur_buf(start_pos:start_pos) /= '{' .and. cur_buf(
               start_pos:start_pos) /= '[')
             ! Affectation et calcul numérique
             start_pos = start_pos + 1
          end do
          ! Évaluation de la condition logique
          if (start_pos > len(cur_buf)) then
             ! Affectation et calcul numérique
             found = .false.
             ! Retour à la procédure appelante
             return
          end if

          ! Affectation et calcul numérique
          brace_count = 1
          ! Affectation et calcul numérique
          end_pos = start_pos + 1
          ! Boucle d'itération numérique
          do while (end_pos <= len(cur_buf) .and. brace_count > 0)
```

```
178             ! Affectation et calcul numérique
179             if (cur_buf(end_pos:end_pos) == '{' .or. cur_buf(end_pos:end_pos) == '[') brace_count =
                    brace_count + 1
180             ! Affectation et calcul numérique
181             if (cur_buf(end_pos:end_pos) == '}' .or. cur_buf(end_pos:end_pos) == ']') brace_count =
                    brace_count - 1
182             ! Affectation et calcul numérique
183             end_pos = end_pos + 1
184          end do
185          ! Affectation et calcul numérique
186          cur_buf = cur_buf(start_pos:end_pos-1)
187       ! Branche par défaut si condition non satisfaite
188       else
189          ! Dernier élément du chemin : extraire la valeur
190          start_pos = pos_colon + 1
191          ! Boucle d'itération numérique
192          do while (start_pos <= len(cur_buf) .and. (cur_buf(start_pos:start_pos) == ' ' .or. cur_buf(
                 start_pos:start_pos) == char(9)))
193             ! Affectation et calcul numérique
194             start_pos = start_pos + 1
195          end do
196          ! Évaluation de la condition logique
197          if (start_pos > len(cur_buf)) then
198             ! Affectation et calcul numérique
199             found = .false.
200             ! Retour à la procédure appelante
201             return
202          end if
203
204          ! Évaluation de la condition logique
205          if (cur_buf(start_pos:start_pos) == '"') then
206             ! Chaîne entre guillemets
207             end_pos = start_pos + 1
208             ! Boucle d'itération numérique
209             do while (end_pos <= len(cur_buf) .and. cur_buf(end_pos:end_pos) /= '"')
210                ! Affectation et calcul numérique
211                end_pos = end_pos + 1
212             end do
213             ! Affectation et calcul numérique
214             val_str = cur_buf(start_pos+1:end_pos-1)
215             ! Affectation et calcul numérique
216             found = .true.
217             ! Retour à la procédure appelante
218             return
219          ! Branche par défaut si condition non satisfaite
220          else
221             ! Nombre, booléen, etc.
222             end_pos = start_pos
223             ! Boucle d'itération numérique
224             do while (end_pos <= len(cur_buf) .and. &
225                       ! Affectation et calcul numérique
226                       cur_buf(end_pos:end_pos) /= ',' .and. &
227                       ! Affectation et calcul numérique
228                       cur_buf(end_pos:end_pos) /= '}' .and. &
229                       ! Affectation et calcul numérique
230                       cur_buf(end_pos:end_pos) /= ']' .and. &
231                       ! Affectation et calcul numérique
232                       cur_buf(end_pos:end_pos) /= ' ' .and. &
233                       ! Affectation et calcul numérique
234                       cur_buf(end_pos:end_pos) /= char(9) .and. &
235                       ! Affectation et calcul numérique
236                       cur_buf(end_pos:end_pos) /= char(10) .and. &
237                       ! Affectation et calcul numérique
238                       cur_buf(end_pos:end_pos) /= char(13))
239                ! Affectation et calcul numérique
240                end_pos = end_pos + 1
241             end do
242             ! Affectation et calcul numérique
243             val_str = trim(adjustl(cur_buf(start_pos:end_pos-1)))
244             ! Affectation et calcul numérique
245             found = .true.
246             ! Retour à la procédure appelante
247             return
248          end if
249       end if
250    end do
251 end subroutine json_find_value_str
252
253 ! [Procédure/Module] subroutine json_get_string
254 subroutine json_get_string(buffer, path, val, default_val)
255    ! Déclaration de variable locale / paramètre de calcul
256    character(len=*), intent(in) :: buffer, path, default_val
257    ! Déclaration de variable locale / paramètre de calcul
258    character(len=*), intent(out) :: val
259    ! Déclaration de variable locale / paramètre de calcul
260    character(len=:), allocatable :: s
261    ! Déclaration de variable locale / paramètre de calcul
262    logical :: found
263
264    ! Appel de sous-programme modulaire
265    call json_find_value_str(buffer, path, s, found)
266    ! Évaluation de la condition logique
```

```
    if (found .and. allocated(s)) then
       ! Affectation et calcul numérique
       val = trim(s)
    ! Branche par défaut si condition non satisfaite
    else
       ! Affectation et calcul numérique
       val = trim(default_val)
    end if
  end subroutine json_get_string

  ! [Procédure/Module] subroutine json_get_real
  subroutine json_get_real(buffer, path, val, default_val)
    ! Déclaration de variable locale / paramètre de calcul
    character(len=*), intent(in) :: buffer, path
    ! Déclaration de variable locale / paramètre de calcul
    real(dp), intent(out) :: val
    ! Déclaration de variable locale / paramètre de calcul
    real(dp), intent(in) :: default_val
    ! Déclaration de variable locale / paramètre de calcul
    character(len=:), allocatable :: s
    ! Déclaration de variable locale / paramètre de calcul
    logical :: found
    ! Déclaration de variable locale / paramètre de calcul
    integer :: io

    ! Appel de sous-programme modulaire
    call json_find_value_str(buffer, path, s, found)
    ! Évaluation de la condition logique
    if (found .and. allocated(s)) then
       ! Affectation et calcul numérique
       read(s, *, iostat=io) val
       ! Affectation et calcul numérique
       if (io /= 0) val = default_val
    ! Branche par défaut si condition non satisfaite
    else
       ! Affectation et calcul numérique
       val = default_val
    end if
  end subroutine json_get_real

  ! [Procédure/Module] subroutine json_get_int
  subroutine json_get_int(buffer, path, val, default_val)
    ! Déclaration de variable locale / paramètre de calcul
    character(len=*), intent(in) :: buffer, path
    ! Déclaration de variable locale / paramètre de calcul
    integer, intent(out) :: val
    ! Déclaration de variable locale / paramètre de calcul
    integer, intent(in) :: default_val
    ! Déclaration de variable locale / paramètre de calcul
    character(len=:), allocatable :: s
    ! Déclaration de variable locale / paramètre de calcul
    logical :: found
    ! Déclaration de variable locale / paramètre de calcul
    integer :: io

    ! Appel de sous-programme modulaire
    call json_find_value_str(buffer, path, s, found)
    ! Évaluation de la condition logique
    if (found .and. allocated(s)) then
       ! Affectation et calcul numérique
       read(s, *, iostat=io) val
       ! Affectation et calcul numérique
       if (io /= 0) val = default_val
    ! Branche par défaut si condition non satisfaite
    else
       ! Affectation et calcul numérique
       val = default_val
    end if
  end subroutine json_get_int

  ! [Procédure/Module] subroutine json_get_bool
  subroutine json_get_bool(buffer, path, val, default_val)
    ! Déclaration de variable locale / paramètre de calcul
    character(len=*), intent(in) :: buffer, path
    ! Déclaration de variable locale / paramètre de calcul
    logical, intent(out) :: val
    ! Déclaration de variable locale / paramètre de calcul
    logical, intent(in) :: default_val
    ! Déclaration de variable locale / paramètre de calcul
    character(len=:), allocatable :: s
    ! Déclaration de variable locale / paramètre de calcul
    logical :: found

    ! Appel de sous-programme modulaire
    call json_find_value_str(buffer, path, s, found)
    ! Évaluation de la condition logique
    if (found .and. allocated(s)) then
       ! Évaluation de la condition logique
       if (index(s, "true") > 0 .or. index(s, "TRUE") > 0 .or. index(s, "1") > 0) then
          ! Affectation et calcul numérique
          val = .true.
       ! Branche par défaut si condition non satisfaite
```

```
359         else
360            ! Affectation et calcul numérique
361            val = .false.
362         end if
363      ! Branche par défaut si condition non satisfaite
364      else
365         ! Affectation et calcul numérique
366         val = default_val
367      end if
368    end subroutine json_get_bool
369
370    ! Détection et extraction des obstacles (BOX ou CYLINDER)
371    subroutine json_get_obstacle(buffer, obs_type, x0, y0, x1, y1, xc, yc, radius, perm, found)
372      ! Déclaration de variable locale / paramètre de calcul
373      character(len=*), intent(in) :: buffer
374      ! Déclaration de variable locale / paramètre de calcul
375      integer, intent(out) :: obs_type
376      ! Déclaration de variable locale / paramètre de calcul
377      real(dp), intent(out) :: x0, y0, x1, y1, xc, yc, radius, perm
378      ! Déclaration de variable locale / paramètre de calcul
379      logical, intent(out) :: found
380
381      ! Déclaration de variable locale / paramètre de calcul
382      integer :: pos_obs, pos_end
383      ! Déclaration de variable locale / paramètre de calcul
384      character(len=:), allocatable :: obs_str
385      ! Déclaration de variable locale / paramètre de calcul
386      character(len=32) :: type_str
387
388      ! Affectation et calcul numérique
389      obs_type = OBS_NONE
390      ! Affectation et calcul numérique
391      x0 = 0.0_dp; y0 = 0.0_dp; x1 = 0.0_dp; y1 = 0.0_dp
392      ! Affectation et calcul numérique
393      xc = 0.0_dp; yc = 0.0_dp; radius = 0.0_dp; perm = 1.0e-40_dp
394      ! Affectation et calcul numérique
395      found = .false.
396
397      ! Affectation et calcul numérique
398      pos_obs = index(buffer, '"obstacles"')
399      ! Affectation et calcul numérique
400      if (pos_obs <= 0) return
401
402      ! Affectation et calcul numérique
403      pos_end = pos_obs + 11
404      ! Boucle d'itération numérique
405      do while (pos_end <= len(buffer) .and. buffer(pos_end:pos_end) /= ']')
406         ! Affectation et calcul numérique
407         pos_end = pos_end + 1
408      end do
409      if (pos_end > len(buffer)) return
410
411      ! Affectation et calcul numérique
412      obs_str = buffer(pos_obs:pos_end)
413      ! Appel de sous-programme modulaire
414      call json_get_string(obs_str, "type", type_str, "")
415
416      ! Évaluation de la condition logique
417      if (trim(type_str) == "BOX") then
418         ! Affectation et calcul numérique
419         obs_type = OBS_BOX
420         ! Appel de sous-programme modulaire
421         call json_get_real(obs_str, "x0", x0, 0.0_dp)
422         ! Appel de sous-programme modulaire
423         call json_get_real(obs_str, "y0", y0, 0.0_dp)
424         ! Appel de sous-programme modulaire
425         call json_get_real(obs_str, "x1", x1, 0.0_dp)
426         ! Appel de sous-programme modulaire
427         call json_get_real(obs_str, "y1", y1, 0.0_dp)
428         ! Appel de sous-programme modulaire
429         call json_get_real(obs_str, "permeability", perm, 1.0e-40_dp)
430         ! Affectation et calcul numérique
431         found = .true.
432      ! Branche conditionnelle alternative
433      else if (trim(type_str) == "CYLINDER") then
434         ! Affectation et calcul numérique
435         obs_type = OBS_CYLINDER
436         ! Appel de sous-programme modulaire
437         call json_get_real(obs_str, "xc", xc, 0.0_dp)
438         ! Appel de sous-programme modulaire
439         call json_get_real(obs_str, "yc", yc, 0.0_dp)
440         ! Appel de sous-programme modulaire
441         call json_get_real(obs_str, "radius", radius, 0.0_dp)
442         ! Appel de sous-programme modulaire
443         call json_get_real(obs_str, "permeability", perm, 1.0e-40_dp)
444         ! Affectation et calcul numérique
445         found = .true.
446      end if
447    end subroutine json_get_obstacle
448
449 end module module_json
```

15.5 Répertoire des Types Dérivés, Sous-Routines et Fonctions de `src/module_config.f90`

Ligne	Nature	Identifiant Fortran 2018	Interface / Arguments
2	module	`module_config`	Unité modulaire Fortran 2018
11	type dérivé	`bc_t`	Structure de données Fortran 2018
30	type dérivé	`config_t`	Structure de données Fortran 2018
146	subroutine	`parse_bc`	Arguments : (buf, prefix, bc)
213	subroutine	`load_config_from_json`	Arguments : (filename, cfg, ierr)

15.6 Code Source Intégral Annoté : `src/module_config.f90`

Le listing ci-dessous présente l'intégralité des 507 lignes du fichier `src/module_config.f90` de l'implémentation Fortran 2018 :

```
! [Procédure/Module] module module_config
module module_config
  use module_precision, only: dp
  use module_constants
  use module_json
  implicit none
  private

  public :: bc_t, config_t, load_config_from_json

  type :: bc_t
     ! Déclaration de variable locale / paramètre de calcul
     integer  :: btype = BC_DIRICHLET
     ! Déclaration de variable locale / paramètre de calcul
     real(dp) :: u = 0.0_dp
     ! Déclaration de variable locale / paramètre de calcul
     real(dp) :: v = 0.0_dp
     ! Profil spécifique pour la frontière ("UNIFORM", "PARABOLIC", "JET_INLET")
     character(len=32) :: profile = "UNIFORM"
     ! Largeur de l'injecteur pour profil JET_INLET [m] (Cas 3.3 Fluids 2021)
     real(dp) :: jet_width = 0.08_dp
     ! Position centrale de l'injecteur selon X [m]
     real(dp) :: jet_center = 0.5_dp
     ! Déclaration de variable locale / paramètre de calcul
     integer  :: thermal_type = THERMAL_NEUMANN
     ! Déclaration de variable locale / paramètre de calcul
     real(dp) :: T = 0.0_dp
  end type bc_t

  type :: config_t
     ! Déclaration de variable locale / paramètre de calcul
     character(len=64) :: case_name = "covof2d_case"
     ! Déclaration de variable locale / paramètre de calcul
     character(len=32) :: mode = "NAVIER_STOKES"

     ! Mesh parameters
     integer  :: nx = 64, ny = 64
     ! Déclaration de variable locale / paramètre de calcul
     real(dp) :: lx = 1.0_dp, ly = 1.0_dp

     ! Physical models & flags
     logical  :: enable_two_phase = .false.
     ! Déclaration de variable locale / paramètre de calcul
     logical  :: enable_surface_tension = .false.
     ! Déclaration de variable locale / paramètre de calcul
     logical  :: enable_convection = .true.
     ! Déclaration de variable locale / paramètre de calcul
     integer  :: convective_scheme = CONV_UPWIND
     ! Déclaration de variable locale / paramètre de calcul
     integer  :: viscosity_model = VISC_CONSTANT
     ! Déclaration de variable locale / paramètre de calcul
     real(dp) :: visc_mu0 = 1.0e-3_dp, visc_a = 2.0_dp, visc_e = 0.01_dp
     ! Déclaration de variable locale / paramètre de calcul
     real(dp) :: sigma = 0.0_dp
     ! Déclaration de variable locale / paramètre de calcul
     real(dp) :: gx = 0.0_dp, gy = 0.0_dp
     ! Déclaration de variable locale / paramètre de calcul
     real(dp) :: rho1 = 1.0_dp, mu1 = 1.0e-3_dp
     ! Déclaration de variable locale / paramètre de calcul
     real(dp) :: rho2 = 1.0_dp, mu2 = 1.0e-3_dp

     ! VOF and interface tracking
     character(len=32) :: reconstruction_method = "PLIC"
     ! Déclaration de variable locale / paramètre de calcul
     character(len=32) :: advection_method = "VOF_TVD"
     ! Déclaration de variable locale / paramètre de calcul
     character(len=32) :: tvd_limiter = "SUPERBEE"
     ! Déclaration de variable locale / paramètre de calcul
     real(dp) :: cfl_vof = 0.4_dp
```

```
70
71      ! Thermal parameters
72      logical  :: enable_energy = .false.
73      ! Déclaration de variable locale / paramètre de calcul
74      real(dp) :: Ra = 1.0e4_dp, Pr = 0.71_dp, T_ref = 0.0_dp, beta = 3.41e-3_dp
75      ! Déclaration de variable locale / paramètre de calcul
76      real(dp) :: Cp = 1.0_dp, k_thermal = 0.014_dp
77
78      ! Time integration
79      real(dp) :: dt = 0.01_dp, t_final = 1.0_dp
80      ! Déclaration de variable locale / paramètre de calcul
81      logical  :: adaptive_dt = .false.
82      ! Déclaration de variable locale / paramètre de calcul
83      real(dp) :: cfl_max = 0.5_dp
84
85      ! Boundary conditions
86      type(bc_t) :: bc_left, bc_right, bc_bottom, bc_top
87
88      ! Immersed boundary obstacle
89      logical  :: obs_enabled = .false.
90      ! Déclaration de variable locale / paramètre de calcul
91      integer  :: obs_type = OBS_NONE
92      ! Déclaration de variable locale / paramètre de calcul
93      real(dp) :: obs_x0 = 0.0_dp, obs_y0 = 0.0_dp, obs_x1 = 0.0_dp, obs_y1 = 0.0_dp
94      ! Déclaration de variable locale / paramètre de calcul
95      real(dp) :: obs_xc = 0.0_dp, obs_yc = 0.0_dp, obs_radius = 0.0_dp
96      ! Déclaration de variable locale / paramètre de calcul
97      real(dp) :: obs_perm = 1.0e-40_dp
98
99      ! Initial conditions
100     integer  :: ic_type = IC_UNIFORM
101     ! Déclaration de variable locale / paramètre de calcul
102     real(dp) :: ic_u = 0.0_dp, ic_v = 0.0_dp
103     ! Déclaration de variable locale / paramètre de calcul
104     real(dp) :: ic_center_x = 0.5_dp, ic_center_y = 0.5_dp, ic_radius = 0.25_dp
105     ! Déclaration de variable locale / paramètre de calcul
106     real(dp) :: ic_box_x0 = 0.0_dp, ic_box_y0 = 0.0_dp, ic_box_x1 = 0.5_dp, ic_box_y1 = 0.5_dp
107     ! Déclaration de variable locale / paramètre de calcul
108     real(dp) :: ic_pert_amp = 0.05_dp, ic_pert_k = 1.0_dp
109     ! Déclaration de l'épaisseur initiale de nappe liquide (Cas 3.2 Fluids 2021)
110     real(dp) :: ic_thickness = 0.0003_dp
111     ! Déclaration de la largeur initiale de jet liquide (Cas 3.3 Fluids 2021)
112     real(dp) :: ic_jet_width = 0.08_dp
113
114     ! Solvers
115     integer  :: solver_type = SOLVER_AUGMENTED_LAGRANGIAN
116     ! Déclaration de variable locale / paramètre de calcul
117     integer  :: linear_solver = LINEAR_BICGSTAB2
118     ! Déclaration de variable locale / paramètre de calcul
119     integer  :: precond_type = PRECOND_JACOBI
120     ! Déclaration de variable locale / paramètre de calcul
121     real(dp) :: tol = 1.0e-5_dp
122     ! Déclaration de variable locale / paramètre de calcul
123     integer  :: max_iter = 300
124     ! Déclaration de variable locale / paramètre de calcul
125     real(dp) :: r_al = 100.0_dp
126     ! Déclaration de variable locale / paramètre de calcul
127     integer  :: uzawa_iter = 1
128     ! Déclaration de variable locale / paramètre de calcul
129     logical  :: use_precond = .true.
130     ! Déclaration de variable locale / paramètre de calcul
131     real(dp) :: steady_state_tol = 1.0e-5_dp
132
133     ! Outputs
134     character(len=128) :: output_dir = "output"
135     ! Déclaration de variable locale / paramètre de calcul
136     integer  :: vtk_interval = 50
137     ! Déclaration de variable locale / paramètre de calcul
138     integer  :: log_interval = 10
139     ! Déclaration de variable locale / paramètre de calcul
140     logical  :: write_csv = .true.
141   end type config_t
142
143 contains
144
145   ! [Procédure/Module] subroutine parse_bc
146   subroutine parse_bc(buf, prefix, bc)
147     ! Déclaration de variable locale / paramètre de calcul
148     character(len=*), intent(in) :: buf, prefix
149     ! Déclaration de variable locale / paramètre de calcul
150     type(bc_t), intent(out) :: bc
151     ! Déclaration de variable locale / paramètre de calcul
152     character(len=32) :: btype_str, ttype_str
153
154     ! Appel de sous-programme modulaire
155     call json_get_string(buf, trim(prefix) // ".type", btype_str, "DIRICHLET")
156     ! Évaluation de la condition logique
157     if (trim(btype_str) == "DIRICHLET") then
158        ! Affectation et calcul numérique
159        bc%btype = BC_DIRICHLET
160     ! Branche conditionnelle alternative
161     else if (trim(btype_str) == "NEUMANN") then
```

```
      ! Affectation et calcul numérique
      bc%btype = BC_NEUMANN
   ! Branche conditionnelle alternative
   else if (trim(btype_str) == "PERIODIC") then
      ! Affectation et calcul numérique
      bc%btype = BC_PERIODIC
   ! Branche conditionnelle alternative
   else if (trim(btype_str) == "FREE_SLIP") then
      ! Affectation et calcul numérique
      bc%btype = BC_FREESLIP
   ! Branche par défaut si condition non satisfaite
   else
      ! Affectation et calcul numérique
      bc%btype = BC_DIRICHLET
   end if

   ! Appel de sous-programme modulaire
   call json_get_real(buf, trim(prefix) // ".u", bc%u, 0.0_dp)
   ! Appel de sous-programme modulaire
   call json_get_real(buf, trim(prefix) // ".u_mean", bc%u, bc%u)
   ! Appel de sous-programme modulaire
   call json_get_real(buf, trim(prefix) // ".u_val", bc%u, bc%u)
   ! Appel de sous-programme modulaire
   call json_get_real(buf, trim(prefix) // ".v", bc%v, 0.0_dp)
   ! Appel de sous-programme modulaire
   call json_get_real(buf, trim(prefix) // ".v_val", bc%v, bc%v)
   ! Lecture du profil de frontière (Cas 3.3 Fluids 2021)
   call json_get_string(buf, trim(prefix) // ".profile", bc%profile, "UNIFORM")
   ! Lecture de la largeur de l'injecteur
   call json_get_real(buf, trim(prefix) // ".jet_width", bc%jet_width, 0.08_dp)
   ! Lecture du centre de l'injecteur
   call json_get_real(buf, trim(prefix) // ".jet_center", bc%jet_center, 0.5_dp)

   ! Appel de sous-programme modulaire
   call json_get_string(buf, trim(prefix) // ".T_type", ttype_str, "NEUMANN")
   ! Évaluation de la condition logique
   if (trim(ttype_str) == "DIRICHLET") then
      ! Affectation et calcul numérique
      bc%thermal_type = THERMAL_DIRICHLET
   ! Branche par défaut si condition non satisfaite
   else
      ! Affectation et calcul numérique
      bc%thermal_type = THERMAL_NEUMANN
   end if
   ! Appel de sous-programme modulaire
   call json_get_real(buf, trim(prefix) // ".T", bc%T, 0.0_dp)
   ! Appel de sous-programme modulaire
   call json_get_real(buf, trim(prefix) // ".T_val", bc%T, bc%T)
end subroutine parse_bc

! [Procédure/Module] subroutine load_config_from_json
subroutine load_config_from_json(filename, cfg, ierr)
  ! Déclaration de variable locale / paramètre de calcul
  character(len=*), intent(in) :: filename
  ! Déclaration de variable locale / paramètre de calcul
  type(config_t), intent(out) :: cfg
  ! Déclaration de variable locale / paramètre de calcul
  integer, intent(out), optional :: ierr

  ! Déclaration de variable locale / paramètre de calcul
  character(len=:), allocatable :: buf
  ! Déclaration de variable locale / paramètre de calcul
  character(len=64) :: str_val
  ! Déclaration de variable locale / paramètre de calcul
  integer :: stat

  ! Affectation et calcul numérique
  stat = 0
  ! Appel de sous-programme modulaire
  call json_read_file(filename, buf, stat)
  ! Évaluation de la norme du résidu ou de l'erreur numérique
  if (present(ierr)) ierr = stat
  ! Affectation et calcul numérique
  if (stat /= 0) return

  ! Case name & mode
  call json_get_string(buf, "case_name", cfg%case_name, "covof_case")
  ! Appel de sous-programme modulaire
  call json_get_string(buf, "output.case_name", cfg%case_name, cfg%case_name)
  ! Appel de sous-programme modulaire
  call json_get_string(buf, "mode", cfg%mode, "NAVIER_STOKES")

  ! Mesh
  call json_get_int(buf, "mesh.nx", cfg%nx, 64)
  ! Appel de sous-programme modulaire
  call json_get_int(buf, "mesh.ny", cfg%ny, 64)
  ! Appel de sous-programme modulaire
  call json_get_real(buf, "mesh.lx", cfg%lx, 1.0_dp)
  ! Appel de sous-programme modulaire
  call json_get_real(buf, "mesh.ly", cfg%ly, 1.0_dp)

  ! Physics
```

```
    call json_get_bool(buf, "physics.enable_two_phase", cfg%enable_two_phase, .false.)
    ! Appel de sous-programme modulaire
    call json_get_bool(buf, "physics.enable_surface_tension", cfg%enable_surface_tension, .false.)
    ! Appel de sous-programme modulaire
    call json_get_bool(buf, "physics.enable_convection", cfg%enable_convection, .true.)
    ! Appel de sous-programme modulaire
    call json_get_real(buf, "physics.sigma", cfg%sigma, 0.0_dp)
    ! Appel de sous-programme modulaire
    call json_get_real(buf, "physics.gx", cfg%gx, 0.0_dp)
    ! Appel de sous-programme modulaire
    call json_get_real(buf, "physics.gy", cfg%gy, 0.0_dp)

    ! Appel de sous-programme modulaire
    call json_get_string(buf, "physics.convective_scheme", str_val, "UPWIND")
    ! Évaluation de la condition logique
    if (trim(str_val) == "CENTERED") then
       ! Affectation et calcul numérique
       cfg%convective_scheme = CONV_CENTERED
    ! Branche conditionnelle alternative
    else if (trim(str_val) == "TVD_SUPERBEE") then
       ! Affectation et calcul numérique
       cfg%convective_scheme = CONV_TVD_SUPERBEE
    ! Branche par défaut si condition non satisfaite
    else
       ! Affectation et calcul numérique
       cfg%convective_scheme = CONV_UPWIND
    end if

    ! Appel de sous-programme modulaire
    call json_get_string(buf, "physics.viscosity_model", str_val, "CONSTANT")
    ! Évaluation de la condition logique
    if (trim(str_val) == "EXPONENTIAL") then
       ! Affectation et calcul numérique
       cfg%viscosity_model = VISC_EXPONENTIAL
       ! Appel de sous-programme modulaire
       call json_get_real(buf, "physics.visc_mu0", cfg%visc_mu0, 1.0e-3_dp)
       ! Appel de sous-programme modulaire
       call json_get_real(buf, "physics.visc_a", cfg%visc_a, 2.0_dp)
       ! Appel de sous-programme modulaire
       call json_get_real(buf, "physics.visc_e", cfg%visc_e, 0.01_dp)
    ! Branche par défaut si condition non satisfaite
    else
       ! Affectation et calcul numérique
       cfg%viscosity_model = VISC_CONSTANT
    end if

    ! Appel de sous-programme modulaire
    call json_get_real(buf, "physics.fluid1.rho", cfg%rho1, 1.0_dp)
    ! Appel de sous-programme modulaire
    call json_get_real(buf, "physics.fluid1.mu", cfg%mu1, 1.0e-3_dp)
    ! Appel de sous-programme modulaire
    call json_get_real(buf, "physics.fluid2.rho", cfg%rho2, cfg%rho1)
    ! Appel de sous-programme modulaire
    call json_get_real(buf, "physics.fluid2.mu", cfg%mu2, cfg%mu1)

    ! Thermal
    call json_get_bool(buf, "physics.enable_energy", cfg%enable_energy, .false.)
    ! Appel de sous-programme modulaire
    call json_get_bool(buf, "thermal.enable_energy", cfg%enable_energy, cfg%enable_energy)
    ! Appel de sous-programme modulaire
    call json_get_real(buf, "physics.Ra", cfg%Ra, 1.0e4_dp)
    ! Appel de sous-programme modulaire
    call json_get_real(buf, "physics.Pr", cfg%Pr, 0.71_dp)
    ! Appel de sous-programme modulaire
    call json_get_real(buf, "physics.T_ref", cfg%T_ref, 0.0_dp)
    ! Appel de sous-programme modulaire
    call json_get_real(buf, "physics.beta", cfg%beta, 3.41e-3_dp)
    ! Appel de sous-programme modulaire
    call json_get_real(buf, "physics.fluid1.Cp", cfg%Cp, 1.0_dp)
    ! Appel de sous-programme modulaire
    call json_get_real(buf, "physics.fluid1.k_thermal", cfg%k_thermal, 0.014_dp)

    ! Time
    call json_get_real(buf, "time.dt", cfg%dt, 0.01_dp)
    ! Appel de sous-programme modulaire
    call json_get_real(buf, "time.t_final", cfg%t_final, 1.0_dp)
    ! Appel de sous-programme modulaire
    call json_get_bool(buf, "time.adaptive_dt", cfg%adaptive_dt, .false.)
    ! Appel de sous-programme modulaire
    call json_get_real(buf, "time.cfl_max", cfg%cfl_max, 0.5_dp)
    ! Appel de sous-programme modulaire
    call json_get_real(buf, "steady_state_tol", cfg%steady_state_tol, 1.0e-5_dp)

    ! Boundary conditions
    call parse_bc(buf, "boundary_conditions.left", cfg%bc_left)
    ! Appel de sous-programme modulaire
    call parse_bc(buf, "boundary_conditions.right", cfg%bc_right)
    ! Appel de sous-programme modulaire
    call parse_bc(buf, "boundary_conditions.bottom", cfg%bc_bottom)
    ! Appel de sous-programme modulaire
    call parse_bc(buf, "boundary_conditions.top", cfg%bc_top)

```

```
    ! Obstacles
    call json_get_obstacle(buf, cfg%obs_type, cfg%obs_x0, cfg%obs_y0, cfg%obs_x1, cfg%obs_y1, &
                           cfg%obs_xc, cfg%obs_yc, cfg%obs_radius, cfg%obs_perm, cfg%obs_enabled)

    ! Initial condition
    call json_get_string(buf, "initial_condition.type", str_val, "UNIFORM")
    ! Évaluation de la condition logique
    if (trim(str_val) == "UNIFORM") then
       ! Affectation et calcul numérique
       cfg%ic_type = IC_UNIFORM
    ! Branche conditionnelle alternative
    else if (trim(str_val) == "POISEUILLE") then
       ! Affectation et calcul numérique
       cfg%ic_type = IC_POISEUILLE
    ! Branche conditionnelle alternative
    else if (trim(str_val) == "VORTEX_SHEAR") then
       ! Affectation et calcul numérique
       cfg%ic_type = IC_VORTEX_SHEAR
    ! Branche conditionnelle alternative
    else if (trim(str_val) == "RAYLEIGH_TAYLOR") then
       ! Affectation et calcul numérique
       cfg%ic_type = IC_RAYLEIGH_TAYLOR
    ! Branche conditionnelle alternative
    else if (trim(str_val) == "DROPLET") then
       ! Affectation et calcul numérique
       cfg%ic_type = IC_DROPLET
    ! Branche conditionnelle alternative pour bulle ou cylindre immergé
    else if (trim(str_val) == "BUBBLE" .or. trim(str_val) == "CYLINDER") then
       ! Affectation et calcul numérique
       cfg%ic_type = IC_BUBBLE
    ! Branche conditionnelle alternative
    else if (trim(str_val) == "DAM_BREAK") then
       ! Affectation et calcul numérique
       cfg%ic_type = IC_DAM_BREAK
    ! Branche conditionnelle alternative pour nappe liquide (Cas 3.2 Fluids 2021)
    else if (trim(str_val) == "LIQUID_SHEET") then
       ! Affectation et calcul numérique
       cfg%ic_type = IC_LIQUID_SHEET
    ! Branche conditionnelle alternative pour jet liquide visqueux (Cas 3.3 Fluids 2021)
    else if (trim(str_val) == "VISCOUS_JET" .or. trim(str_val) == "JET_BUCKLING") then
       ! Affectation et calcul numérique
       cfg%ic_type = IC_VISCOUS_JET
    ! Branche conditionnelle alternative
    else if (trim(str_val) == "LAPLACIAN_TEST") then
       ! Affectation et calcul numérique
       cfg%ic_type = IC_LAPLACIAN_TEST
    ! Branche conditionnelle alternative
    else if (trim(str_val) == "SINGLE_PHASE") then
       ! Affectation et calcul numérique
       cfg%ic_type = IC_UNIFORM
    end if

    ! Appel de sous-programme modulaire
    call json_get_real(buf, "initial_condition.u", cfg%ic_u, 0.0_dp)
    ! Appel de sous-programme modulaire
    call json_get_real(buf, "initial_condition.v", cfg%ic_v, 0.0_dp)
    ! Appel de sous-programme modulaire
    call json_get_real(buf, "initial_condition.center_x", cfg%ic_center_x, 0.5_dp)
    ! Appel de sous-programme modulaire
    call json_get_real(buf, "initial_condition.center_y", cfg%ic_center_y, 0.5_dp)
    ! Appel de sous-programme modulaire
    call json_get_real(buf, "initial_condition.radius", cfg%ic_radius, 0.25_dp)
    ! Appel de sous-programme modulaire
    call json_get_real(buf, "initial_condition.box_x0", cfg%ic_box_x0, 0.0_dp)
    ! Appel de sous-programme modulaire
    call json_get_real(buf, "initial_condition.box_y0", cfg%ic_box_y0, 0.0_dp)
    ! Appel de sous-programme modulaire
    call json_get_real(buf, "initial_condition.box_x1", cfg%ic_box_x1, 0.5_dp)
    ! Appel de sous-programme modulaire
    call json_get_real(buf, "initial_condition.box_y1", cfg%ic_box_y1, 0.5_dp)
    ! Appel de sous-programme modulaire
    call json_get_real(buf, "initial_condition.pert_amp", cfg%ic_pert_amp, 0.05_dp)
    ! Appel de sous-programme modulaire
    call json_get_real(buf, "initial_condition.pert_k", cfg%ic_pert_k, 1.0_dp)
    ! Lecture de l'épaisseur de nappe liquide (Cas 3.2 Fluids 2021)
    call json_get_real(buf, "initial_condition.thickness", cfg%ic_thickness, 0.0003_dp)
    ! Lecture de la largeur de jet liquide (Cas 3.3 Fluids 2021)
    call json_get_real(buf, "initial_condition.jet_width", cfg%ic_jet_width, 0.08_dp)

    ! Solver
    call json_get_string(buf, "solver.type", str_val, "AUGMENTED_LAGRANGIAN")
    ! Évaluation de la condition logique
    if (trim(str_val) == "CONJUGATE_GRADIENT") then
       ! Affectation et calcul numérique
       cfg%solver_type = SOLVER_AUGMENTED_LAGRANGIAN
       ! Affectation et calcul numérique
       cfg%linear_solver = LINEAR_CG
    ! Branche par défaut si condition non satisfaite
    else
       ! Affectation et calcul numérique
       cfg%solver_type = SOLVER_AUGMENTED_LAGRANGIAN
       ! Affectation et calcul numérique
```

```
        cfg%linear_solver = LINEAR_BICGSTAB2
    end if

    ! Appel de sous-programme modulaire
    call json_get_string(buf, "solver.linear_solver", str_val, "BICGSTAB2")
    ! Évaluation de la condition logique
    if (trim(str_val) == "BICGSTAB") then
        ! Affectation et calcul numérique
        cfg%linear_solver = LINEAR_BICGSTAB
    ! Branche conditionnelle alternative
    else if (trim(str_val) == "CG" .or. trim(str_val) == "CONJUGATE_GRADIENT") then
        ! Affectation et calcul numérique
        cfg%linear_solver = LINEAR_CG
    ! Branche conditionnelle alternative
    else if (trim(str_val) == "GMRES") then
        ! Affectation et calcul numérique
        cfg%linear_solver = LINEAR_GMRES
    end if

    ! Appel de sous-programme modulaire
    call json_get_real(buf, "solver.tol", cfg%tol, 1.0e-5_dp)
    ! Appel de sous-programme modulaire
    call json_get_int(buf, "solver.max_iter", cfg%max_iter, 300)
    ! Appel de sous-programme modulaire
    call json_get_real(buf, "solver.r_al", cfg%r_al, 100.0_dp)
    ! Appel de sous-programme modulaire
    call json_get_int(buf, "solver.uzawa_iter", cfg%uzawa_iter, 1)
    ! Appel de sous-programme modulaire
    call json_get_bool(buf, "solver.use_precond", cfg%use_precond, .true.)
    ! Appel de sous-programme modulaire
    call json_get_real(buf, "steady_state_tol", cfg%steady_state_tol, cfg%steady_state_tol)
    ! Appel de sous-programme modulaire
    call json_get_real(buf, "solver.steady_state_tol", cfg%steady_state_tol, cfg%steady_state_tol)

    ! VOF parameters
    call json_get_string(buf, "vof.advection_method", cfg%advection_method, "WEYMOUTH_YUE")
    ! Appel de sous-programme modulaire
    call json_get_string(buf, "vof.reconstruction_method", cfg%reconstruction_method, "PLIC")
    ! Appel de sous-programme modulaire
    call json_get_string(buf, "vof.tvd_limiter", cfg%tvd_limiter, "NONE")
    ! Appel de sous-programme modulaire
    call json_get_real(buf, "vof.cfl_vof", cfg%cfl_vof, 0.5_dp)

    ! Cohérence stricte : le schéma algébrique TVD ne s'accompagne d'aucune reconstruction géométrique,
    ! tandis que les schémas géométriques (WEYMOUTH_YUE, CIAM) n'utilisent pas de limiteur TVD.
    if (trim(cfg%advection_method) == "VOF_TVD" .or. trim(cfg%advection_method) == "TVD") then
        cfg%reconstruction_method = "NONE"
        if (trim(cfg%tvd_limiter) == "NONE" .or. len_trim(cfg%tvd_limiter) == 0) then
            cfg%tvd_limiter = "SUPERBEE"
        end if
    else
        if (trim(cfg%reconstruction_method) == "NONE" .or. len_trim(cfg%reconstruction_method) == 0) then
            cfg%reconstruction_method = "PLIC"
        end if
        cfg%tvd_limiter = "NONE"
    end if

    ! Output
    call json_get_string(buf, "output.output_dir", cfg%output_dir, "output")
    ! Appel de sous-programme modulaire
    call json_get_string(buf, "output.directory", cfg%output_dir, cfg%output_dir)
    ! Appel de sous-programme modulaire
    call json_get_int(buf, "output.vtk_interval", cfg%vtk_interval, 50)
    ! Appel de sous-programme modulaire
    call json_get_int(buf, "output.log_interval", cfg%log_interval, 10)
    ! Appel de sous-programme modulaire
    call json_get_bool(buf, "output.write_csv_history", cfg%write_csv, .true.)
  end subroutine load_config_from_json

end module module_config
```

Chapitre 16

Maillage MAC 2D et Tableaux Column-Major à Bornes Fantômes en Fortran 2018 ($\mathbf{module}_{m}esh.f90, module_{f}ields.f90$)

16.1 Indiçage Naturel à Base 1 et Bornes Fantômes (`1-ng:nx+ng`) en Fortran 2018

L'un des atouts majeurs de Fortran 2018 pour la mécanique des fluides numérique sur grille structurée est la possibilité d'allouer des tableaux multidimensionnels avec des bornes inférieures négatives ou nulles correspondant directement aux cellules fantômes :

— **Champs scalaires centrés** ($p, T, C, \rho, \mu, \nabla \cdot \mathbf{u}$) :

```
allocate(fields%p(1-ng : nx+ng, 1-ng : ny+ng))
```
(16.1)

où le domaine intérieur physique occupe exactement $i \in \{1, \ldots, N_x\}$ et $j \in \{1, \ldots, N_y\}$.

— **Vitesse horizontale décalée** u (N_x+1 faces verticales intérieures/frontières d'indices $i \in \{1, \ldots, N_x+1\}$) :

```
allocate(fields%u(1-ng : nx+1+ng, 1-ng : ny+ng))
```
(16.2)

— **Vitesse verticale décalée** v (N_y+1 faces horizontales intérieures/frontières d'indices $j \in \{1, \ldots, N_y+1\}$) :

```
allocate(fields%v(1-ng : nx+ng, 1-ng : ny+1+ng))
```
(16.3)

16.2 Ordre Mémoire *Column-Major* et Ordre Optimal des Boucles Imbriquées

En Fortran, les tableaux 2D `A(i, j)` sont stockés en mémoire par colonnes (*Column-Major Order*) : l'élément `A(i+1, j)` est adjacent en mémoire à `A(i, j)`. Par conséquent, toutes les boucles spatiales dans `COVOF_2D-FORTRAN` sont ordonnées avec j **en boucle externe et** i **en boucle interne** :

```
do j = 1, mesh%ny
  do i = 1, mesh%nx
    fields%div(i, j) = (fields%u(i+1, j) - fields%u(i, j)) / mesh%dx &
                     + (fields%v(i, j+1) - fields%v(i, j)) / mesh%dy
  end do
end do
```

Cet ordonnancement garantit exactement la même localité spatiale L1/L2 et la même efficacité SIMD que l'ordonnancement *Row-Major* de C++17.

16.3 Répertoire des Types Dérivés, Sous-Routines et Fonctions de `src/module_mesh.f90`

Ligne	Nature	Identifiant Fortran 2018	Interface / Arguments
10	module	`module_mesh`	Unité modulaire Fortran 2018
27	type dérivé	`mesh_2d_t`	Structure de données Fortran 2018
67	subroutine	`init_mesh`	Arguments : (mesh, comm_world, nx, ny, lx, ly, perio...)
150	subroutine	`destroy_mesh`	Arguments : (mesh)

16.4 Code Source Intégral Annoté : `src/module_mesh.f90`

Le listing ci-dessous présente l'intégralité des 224 lignes du fichier `src/module_mesh.f90` de l'implémentation Fortran 2018 :

```
!===============================================================================
! Module        : module_mesh
! Description   : Définition de la topologie cartésienne 2D, gestion du
!                 partitionnement spatial multi-processus OpenMPI, calcul des
!                 coordonnées métriques (faces décalées et centres de cellules)
!                 et détection des inclusions géométriques d'obstacles.
! Auteur        : Elhadji Ibrahima Thiam
! Standard      : Fortran 2018 / MPI-3
!===============================================================================
module module_mesh
  ! Importation du descripteur de double précision
  use module_precision, only: dp
  ! Interdiction stricte du typage implicite pour garantir l'intégrité mémoire
  implicit none
  ! Par défaut, tous les membres et routines sont privés
  private
  ! Inclusion des définitions MPI requises pour les communicateurs cartésiens
  include 'mpif.h'

  ! Export explicite des types dérivés et fonctions publiques
  public :: mesh_2d_t, init_mesh, destroy_mesh
  public :: x_face, y_face, x_center, y_center, is_in_obstacle

  !-------------------------------------------------------------------------------
  ! Structure de données représentant le maillage 2D décomposé
  !-------------------------------------------------------------------------------
  type :: mesh_2d_t
     ! Dimensions globales du domaine de calcul (nombre total de mailles)
     integer  :: nx_global = 64, ny_global = 64
     ! Longueurs physiques totales du domaine en mètres [m]
     real(dp) :: lx = 1.0_dp, ly = 1.0_dp
     ! Pas d'espace uniformes selon les directions X et Y : dx = lx/nx, dy = ly/ny
     real(dp) :: dx = 0.0_dp, dy = 0.0_dp

     ! --- Topologie Cartésienne MPI et Rangs de Voisinage ---
     integer  :: rank = 0                ! Rang du sous-domaine local courant
     integer  :: size = 1                ! Nombre total de sous-domaines (processus)
     integer  :: comm = MPI_COMM_WORLD    ! Communicateur initial parent
     integer  :: cart_comm = MPI_COMM_NULL! Communicateur cartésien bidimensionnel dédié
     integer  :: px = 1, py = 1          ! Nombre de processus selon X et selon Y
     integer  :: coord_x = 0, coord_y = 0 ! Coordonnées cartésiennes du sous-domaine dans la grille MPI
     integer  :: neighbor_west = MPI_PROC_NULL  ! Rang du voisin Ouest (gauche, x-)
     integer  :: neighbor_east = MPI_PROC_NULL  ! Rang du voisin Est (droite, x+)
     integer  :: neighbor_south = MPI_PROC_NULL ! Rang du voisin Sud (bas, y-)
     integer  :: neighbor_north = MPI_PROC_NULL ! Rang du voisin Nord (haut, y+)

     ! --- Indicateurs Booléens de Frontières Physiques du Domaine Global ---
     logical  :: is_boundary_left   = .false. ! Vrai si le sous-domaine touche la paroi gauche x = 0
     logical  :: is_boundary_right  = .false. ! Vrai si le sous-domaine touche la paroi droite x = lx
     logical  :: is_boundary_bottom = .false. ! Vrai si le sous-domaine touche la paroi inférieure y = 0
     logical  :: is_boundary_top    = .false. ! Vrai si le sous-domaine touche la paroi supérieure y = ly

     ! --- Partitionnement Spatial Local ---
     integer  :: nx_local = 64, ny_local = 64 ! Nombre de cellules locales attribuées à ce processus
     integer  :: i_start_global = 1           ! Indice global de la première cellule locale en X (base 1)
     integer  :: j_start_global = 1           ! Indice global de la première cellule locale en Y (base 1)
  end type mesh_2d_t

contains

  !===============================================================================
  ! Routine : init_mesh
  ! Rôle    : Calcule la factorisation optimale du nombre de cœurs en une grille
  !           2D (px, py), crée la topologie cartésienne MPI avec périodicité,
  !           identifie les 4 voisins immédiats et répartit équitablement les mailles.
  !===============================================================================
  subroutine init_mesh(mesh, comm_world, nx, ny, lx, ly, periodic_x, periodic_y)
    type(mesh_2d_t), intent(out) :: mesh       ! Structure de maillage initialisée en sortie
    integer, intent(in)          :: comm_world ! Communicateur parent initial
    integer, intent(in)          :: nx, ny     ! Résolution globale souhaitée en cellules
    real(dp), intent(in)         :: lx, ly     ! Longueurs physiques du domaine rectangulaire
```

```
    logical, intent(in)          :: periodic_x ! Vrai si conditions périodiques en X
    logical, intent(in)          :: periodic_y ! Vrai si conditions périodiques en Y

    ! Variables de travail locales pour les primitives cartésiennes MPI
    integer :: dims(2), periods(2), coords(2)
    integer :: ierr, rank_in, size_in
    integer :: rem_x, rem_y

    ! 1. Enregistrement des dimensions physiques et de la taille du communicateur
    mesh%comm = comm_world
    call MPI_Comm_rank(comm_world, rank_in, ierr)
    call MPI_Comm_size(comm_world, size_in, ierr)
    mesh%rank = rank_in
    mesh%size = size_in

    mesh%nx_global = nx
    mesh%ny_global = ny
    mesh%lx = lx
    mesh%ly = ly
    ! Calcul des pas de discrétisation spatiale
    mesh%dx = lx / real(nx, dp)
    mesh%dy = ly / real(ny, dp)

    ! 2. Détermination automatique de la décomposition 2D optimale (px, py)
    dims = [0, 0]
    call MPI_Dims_create(mesh%size, 2, dims, ierr)
    mesh%px = dims(1)
    mesh%py = dims(2)

    ! Définition des drapeaux de périodicité pour la création de la topologie MPI
    periods(1) = merge(1, 0, periodic_x)
    periods(2) = merge(1, 0, periodic_y)

    ! 3. Création du communicateur virtuel cartésien MPI 2D
    call MPI_Cart_create(comm_world, 2, dims, periods, 1, mesh%cart_comm, ierr)
    call MPI_Comm_rank(mesh%cart_comm, mesh%rank, ierr)
    call MPI_Cart_coords(mesh%cart_comm, mesh%rank, 2, coords, ierr)
    mesh%coord_x = coords(1)
    mesh%coord_y = coords(2)

    ! 4. Recherche des rangs des 4 voisins adjacents par décalage unitaire cartésien
    ! Décalage direction 0 (axe X) : voisin Ouest (-1) et voisin Est (+1)
    call MPI_Cart_shift(mesh%cart_comm, 0, 1, mesh%neighbor_west, mesh%neighbor_east, ierr)
    ! Décalage direction 1 (axe Y) : voisin Sud (-1) et voisin Nord (+1)
    call MPI_Cart_shift(mesh%cart_comm, 1, 1, mesh%neighbor_south, mesh%neighbor_north, ierr)

    ! 5. Répartition équitable des mailles le long de l'axe X (gestion du reste mod)
    mesh%nx_local = nx / mesh%px
    rem_x = mod(nx, mesh%px)
    if (mesh%coord_x < rem_x) then
       ! Les premiers rem_x processus reçoivent une cellule supplémentaire
       mesh%nx_local = mesh%nx_local + 1
       mesh%i_start_global = mesh%coord_x * mesh%nx_local + 1
    else
       mesh%i_start_global = rem_x * (mesh%nx_local + 1) + (mesh%coord_x - rem_x) * mesh%nx_local + 1
    end if

    ! 6. Répartition équitable des mailles le long de l'axe Y
    mesh%ny_local = ny / mesh%py
    rem_y = mod(ny, mesh%py)
    if (mesh%coord_y < rem_y) then
       mesh%ny_local = mesh%ny_local + 1
       mesh%j_start_global = mesh%coord_y * mesh%ny_local + 1
    else
       mesh%j_start_global = rem_y * (mesh%ny_local + 1) + (mesh%coord_y - rem_y) * mesh%ny_local + 1
    end if

    ! 7. Détection des frontières physiques externes non périodiques
    mesh%is_boundary_left   = (mesh%neighbor_west == MPI_PROC_NULL)
    mesh%is_boundary_right  = (mesh%neighbor_east == MPI_PROC_NULL)
    mesh%is_boundary_bottom = (mesh%neighbor_south == MPI_PROC_NULL)
    mesh%is_boundary_top    = (mesh%neighbor_north == MPI_PROC_NULL)
  end subroutine init_mesh

  !===============================================================================
  ! Routine : destroy_mesh
  ! Rôle    : Libère le communicateur cartésien alloué auprès du runtime MPI.
  !===============================================================================
  subroutine destroy_mesh(mesh)
    type(mesh_2d_t), intent(inout) :: mesh
    integer :: ierr
    if (mesh%cart_comm /= MPI_COMM_NULL .and. mesh%cart_comm /= mesh%comm) then
       call MPI_Comm_free(mesh%cart_comm, ierr)
    end if
  end subroutine destroy_mesh

  !===============================================================================
  ! Fonction pure : x_face
  ! Rôle          : Calcule l'abscisse physique absolue de la face verticale i-1/2.
  !===============================================================================
  pure real(dp) function x_face(mesh, i)
    type(mesh_2d_t), intent(in) :: mesh
```

```
    integer, intent(in)          :: i
    x_face = real(mesh%i_start_global - 1 + i - 1, dp) * mesh%dx
  end function x_face

  !===============================================================================
  ! Fonction pure : y_face
  ! Rôle          : Calcule l'ordonnée physique absolue de la face horizontale j-1/2.
  !===============================================================================
  pure real(dp) function y_face(mesh, j)
    type(mesh_2d_t), intent(in) :: mesh
    integer, intent(in)          :: j
    y_face = real(mesh%j_start_global - 1 + j - 1, dp) * mesh%dy
  end function y_face

  !===============================================================================
  ! Fonction pure : x_center
  ! Rôle          : Calcule l'abscisse du barycentre géométrique de la cellule i.
  !===============================================================================
  pure real(dp) function x_center(mesh, i)
    type(mesh_2d_t), intent(in) :: mesh
    integer, intent(in)          :: i
    x_center = (real(mesh%i_start_global - 1 + i - 1, dp) + 0.5_dp) * mesh%dx
  end function x_center

  !===============================================================================
  ! Fonction pure : y_center
  ! Rôle          : Calcule l'ordonnée du barycentre géométrique de la cellule j.
  !===============================================================================
  pure real(dp) function y_center(mesh, j)
    type(mesh_2d_t), intent(in) :: mesh
    integer, intent(in)          :: j
    y_center = (real(mesh%j_start_global - 1 + j - 1, dp) + 0.5_dp) * mesh%dy
  end function y_center

  !===============================================================================
  ! Fonction pure : is_in_obstacle
  ! Rôle          : Teste l'appartenance d'un point (x, y) à un obstacle immergé
  !                 (rectangle ou cylindre) pour la pénalisation de Brinkman.
  !===============================================================================
  pure logical function is_in_obstacle(x, y, obs_type, x0, y0, x1, y1, xc, yc, r)
    real(dp), intent(in) :: x, y         ! Coordonnées du point à tester
    integer, intent(in)  :: obs_type     ! Type géométrique : 1=boîte, 2=cylindre
    real(dp), intent(in) :: x0, y0       ! Coin inférieur gauche du rectangle
    real(dp), intent(in) :: x1, y1       ! Coin supérieur droit du rectangle
    real(dp), intent(in) :: xc, yc       ! Centre du disque cylindrique
    real(dp), intent(in) :: r            ! Rayon du disque

    is_in_obstacle = .false.
    if (obs_type == 1) then ! Obstacle rectangulaire (ex: marche descendante)
       if (x >= x0 - 1.0e-9_dp .and. x <= x1 + 1.0e-9_dp .and. &
           y >= y0 - 1.0e-9_dp .and. y <= y1 + 1.0e-9_dp) then
          is_in_obstacle = .true.
       end if
    else if (obs_type == 2) then ! Obstacle cylindrique circulaire
       if ((x - xc)**2 + (y - yc)**2 <= r**2) then
          is_in_obstacle = .true.
       end if
    end if
  end function is_in_obstacle

end module module_mesh
```

16.5 Répertoire des Types Dérivés, Sous-Routines et Fonctions de `src/module_fields.f90`

Ligne	Nature	Identifiant Fortran 2018	Interface / Arguments
11	module	`module_fields`	Unité modulaire Fortran 2018
27	type dérivé	`flow_fields_t`	Structure de données Fortran 2018
70	subroutine	`fields_allocate_mesh`	Arguments : (this, mesh)
80	subroutine	`fields_free`	Arguments : (this)
90	subroutine	`allocate_fields`	Arguments : (fields, nx, ny)
123	subroutine	`deallocate_fields`	Arguments : (fields)
150	subroutine	`reset_fields`	Arguments : (fields)

16.6 Code Source Intégral Annoté : `src/module_fields.f90`

Le listing ci-dessous présente l'intégralité des 171 lignes du fichier `src/module_fields.f90` de l'implémentation Fortran 2018 :

```
!===============================================================================
```

```
! Module        : module_fields
! Description   : Définition et gestion de la mémoire des champs hydrodynamiques,
!                 thermiques et diphasiques sur la grille décalée MAC (Harlow-Welch).
!                 Assure l'allocation contiguë avec cellules fantômes pour les
!                 échanges de halos MPI et garantit la politique Zero-Allocation
!                 au sein de la boucle en temps.
! Auteur        : Elhadji Ibrahima Thiam
! Standard      : Fortran 2018
!===============================================================================
module module_fields
  ! Importation du descripteur de double précision
  use module_precision, only: dp
  ! Importation du type dérivé de maillage pour connaître les dimensions locales
  use module_mesh, only: mesh_2d_t
  ! Typage strict
  implicit none
  ! Encapsulation privée
  private

  ! Exportation publique du type conteneur et des routines associées
  public :: flow_fields_t, allocate_fields, deallocate_fields, reset_fields

  !-------------------------------------------------------------------------------
  ! Structure de données contenant l'ensemble des champs physiques du sous-domaine
  !-------------------------------------------------------------------------------
  type :: flow_fields_t
     ! Nombre de cellules physiques locales (hors cellules fantômes)
     integer :: nx = 0, ny = 0

     ! --- Champs Hydrodynamiques Discrétisés sur Grille Décalée (MAC) ---
     ! Indices : (0:nx+1, 0:ny+1) pour inclure 1 rangée de cellules fantômes
     real(dp), allocatable :: u(:, :)        ! Vitesse axiale aux faces verticales (i-1/2, j) [m/s]
     real(dp), allocatable :: v(:, :)        ! Vitesse transversale aux faces horizontales (i, j-1/2) [m/s]
     real(dp), allocatable :: p(:, :)        ! Pression statique scalaire au centre des mailles (i, j) [Pa]
     real(dp), allocatable :: div(:, :)      ! Divergence discrète locale de la vitesse (nabla . u) [1/s]

     ! --- Propriétés Thermodynamiques et Équation d'Énergie ---
     real(dp), allocatable :: C(:, :)        ! Fraction volumique VOF du fluide 1 (C in [0, 1])
     real(dp), allocatable :: T(:, :)        ! Température thermodynamique au centre des mailles [K]
     real(dp), allocatable :: rho(:, :)      ! Masse volumique effective au centre des cellules [kg/m^3]
     real(dp), allocatable :: rho_u(:, :)    ! Masse volumique interpolée aux faces u [kg/m^3]
     real(dp), allocatable :: rho_v(:, :)    ! Masse volumique interpolée aux faces v [kg/m^3]
     real(dp), allocatable :: mu(:, :)       ! Viscosité dynamique au centre des mailles [Pa.s]
     real(dp), allocatable :: mu_node(:, :)  ! Viscosité harmonique aux sommets/nœuds (i+1/2, j+1/2) [Pa.s]

     ! --- Forces Capillaires et Tension de Surface (Modèle CSF Brackbill) ---
     real(dp), allocatable :: fx_sigma(:, :) ! Force capillaire volumique selon X aux faces u [N/m^3]
     real(dp), allocatable :: fy_sigma(:, :) ! Force capillaire volumique selon Y aux faces v [N/m^3]
     real(dp), allocatable :: curvature(:, :)! Courbure locale d'interface kappa = -nabla . n [1/m]

     ! --- Masque d'Obstacle Immergé (Pénalisation de Brinkman) ---
     integer,  allocatable :: solid_mask(:, :)! 1 si la cellule est dans le solide, 0 en fluide pur

     ! --- Indicateurs et Diagnostics de Validation ---
     real(dp) :: mean_nusselt = 0.0_dp       ! Nombre de Nusselt moyen mesuré à la paroi chaude

  contains
     ! Méthodes liées au type dérivé pour syntaxe orientée objet
     procedure :: allocate => fields_allocate_mesh
     procedure :: free     => fields_free
  end type flow_fields_t

contains

  !===============================================================================
  ! Routine liée : fields_allocate_mesh
  ! Rôle         : Alloue les tableaux à partir d'une instance de mesh_2d_t.
  !===============================================================================
  subroutine fields_allocate_mesh(this, mesh)
    class(flow_fields_t), intent(inout) :: this
    type(mesh_2d_t), intent(in)         :: mesh
    call allocate_fields(this, mesh%nx_local, mesh%ny_local)
  end subroutine fields_allocate_mesh

  !===============================================================================
  ! Routine liée : fields_free
  ! Rôle         : Désalloue proprement l'ensemble des tableaux du conteneur.
  !===============================================================================
  subroutine fields_free(this)
    class(flow_fields_t), intent(inout) :: this
    call deallocate_fields(this)
  end subroutine fields_free

  !===============================================================================
  ! Routine : allocate_fields
  ! Rôle    : Alloue de façon contiguë en mémoire chaque champ 2D avec ses halos
  !           fantômes (indices de 0 à nx+1 et de 0 à ny+1) et initialise à zéro.
  !===============================================================================
  subroutine allocate_fields(fields, nx, ny)
    type(flow_fields_t), intent(inout) :: fields
    integer, intent(in)                :: nx, ny

```

```
    ! Enregistrement des dimensions locales
    fields%nx = nx
    fields%ny = ny

    ! Allocation dynamique unique (zéro allocation subséquente dans la boucle temporelle)
    allocate(fields%u(0:nx+1, 0:ny+1))
    allocate(fields%v(0:nx+1, 0:ny+1))
    allocate(fields%p(0:nx+1, 0:ny+1))
    allocate(fields%div(0:nx+1, 0:ny+1))
    allocate(fields%C(0:nx+1, 0:ny+1))
    allocate(fields%T(0:nx+1, 0:ny+1))
    allocate(fields%rho(0:nx+1, 0:ny+1))
    allocate(fields%rho_u(0:nx+1, 0:ny+1))
    allocate(fields%rho_v(0:nx+1, 0:ny+1))
    allocate(fields%mu(0:nx+1, 0:ny+1))
    allocate(fields%mu_node(0:nx+1, 0:ny+1))
    allocate(fields%fx_sigma(0:nx+1, 0:ny+1))
    allocate(fields%fy_sigma(0:nx+1, 0:ny+1))
    allocate(fields%curvature(0:nx+1, 0:ny+1))
    allocate(fields%solid_mask(0:nx+1, 0:ny+1))

    ! Remise à zéro des valeurs initiales
    call reset_fields(fields)
  end subroutine allocate_fields

  !===============================================================================
  ! Routine : deallocate_fields
  ! Rôle    : Désalloue chaque tableau et remet les compteurs de dimension à zéro.
  !===============================================================================
  subroutine deallocate_fields(fields)
    type(flow_fields_t), intent(inout) :: fields

    if (allocated(fields%u))          deallocate(fields%u)
    if (allocated(fields%v))          deallocate(fields%v)
    if (allocated(fields%p))          deallocate(fields%p)
    if (allocated(fields%div))        deallocate(fields%div)
    if (allocated(fields%C))          deallocate(fields%C)
    if (allocated(fields%T))          deallocate(fields%T)
    if (allocated(fields%rho))        deallocate(fields%rho)
    if (allocated(fields%rho_u))      deallocate(fields%rho_u)
    if (allocated(fields%rho_v))      deallocate(fields%rho_v)
    if (allocated(fields%mu))         deallocate(fields%mu)
    if (allocated(fields%mu_node))    deallocate(fields%mu_node)
    if (allocated(fields%fx_sigma))   deallocate(fields%fx_sigma)
    if (allocated(fields%fy_sigma))   deallocate(fields%fy_sigma)
    if (allocated(fields%curvature))  deallocate(fields%curvature)
    if (allocated(fields%solid_mask)) deallocate(fields%solid_mask)

    fields%nx = 0
    fields%ny = 0
  end subroutine deallocate_fields

  !===============================================================================
  ! Routine : reset_fields
  ! Rôle    : Initialise les champs à des valeurs physiques par défaut neutres.
  !===============================================================================
  subroutine reset_fields(fields)
    type(flow_fields_t), intent(inout) :: fields

    fields%u            = 0.0_dp      ! Vitesse axiale initiale nulle
    fields%v            = 0.0_dp      ! Vitesse transversale initiale nulle
    fields%p            = 0.0_dp      ! Pression de référence nulle
    fields%div          = 0.0_dp      ! Divergence initiale nulle
    fields%C            = 1.0_dp      ! Fluide 1 pur par défaut
    fields%T            = 0.0_dp      ! Température de référence nulle
    fields%rho          = 1.0_dp      ! Masse volumique unitaire par défaut
    fields%rho_u        = 1.0_dp
    fields%rho_v        = 1.0_dp
    fields%mu           = 1.0e-3_dp   ! Viscosité dynamique standard de l'eau (1 mPa.s)
    fields%mu_node      = 1.0e-3_dp
    fields%fx_sigma     = 0.0_dp      ! Tension superficielle nulle
    fields%fy_sigma     = 0.0_dp
    fields%curvature    = 0.0_dp
    fields%solid_mask   = 0           ! 100% domaine fluide sans obstacle
    fields%mean_nusselt = 0.0_dp
  end subroutine reset_fields

end module module_fields
```

Chapitre 17

Communications Parallèles MPI et Échanges de Halos en Fortran 2018 (module$_{m}pi_{c}omm.f90$)

17.1 Gestion de la Topologie Cartésienne et des Halos dans `module_mpi_comm.f`

Le module `src/module_mpi_comm.f90` (798 lignes) fournit le type dérivé `mpi_context_t` et les sous-routines d'échange de couronnes fantômes pour les trois topologies de grilles MAC :

1. `exchange_halo_scalar(mpi_ctx, mesh, phi)` : synchronise un tableau centré `phi(1-ng:nx+ng, 1-ng:ny+ng)` sur une épaisseur `ng` en échangeant d'abord les colonnes Est/Ouest puis les lignes Nord/Sud (propageant ainsi les coins diagonaux).
2. `exchange_halo_u(mpi_ctx, mesh, u)` et `exchange_halo_v(mpi_ctx, mesh, v)` : gèrent le décalage d'une face sur les bords partagés entre deux sous-domaines MPI adjacents ainsi que la périodicité torique (`BC_PERIODIC`).
3. `mpi_allreduce_sum`, `mpi_allreduce_max`, `mpi_allreduce_min` : encapsulent les réductions collectives globales utilisées par les solveurs de Krylov et les bilans intégraux de conservation.

17.2 Répertoire des Types Dérivés, Sous-Routines et Fonctions de `src/module_mpi_comm.f90`

Ligne	Nature	Identifiant Fortran 2018	Interface / Arguments
2	module	`module_mpi_comm`	Unité modulaire Fortran 2018
25	subroutine	`exchange_halos_scalar`	Arguments : (field, mesh)
139	subroutine	`exchange_halos_velocity`	Arguments : (u, v, mesh)
290	subroutine	`apply_velocity_bc`	Arguments : (fields, mesh, cfg, homogeneous)
514	subroutine	`apply_thermal_bc`	Arguments : (fields, mesh, cfg)
610	subroutine	`apply_vof_bc`	Arguments : (fields, mesh, cfg)
700	subroutine	`sync_u`	Arguments : (mesh, u)
710	subroutine	`sync_v`	Arguments : (mesh, v)
720	subroutine	`sync_cell_scalar`	Arguments : (mesh, s)
730	subroutine	`apply_boundary_conditions`	Arguments : (mesh, cfg, fields, homogeneous)
744	subroutine	`apply_thermal_boundary_conditions`	Arguments : (mesh, cfg, fields)

17.3 Code Source Intégral Annoté : `src/module_mpi_comm.f90`

Le listing ci-dessous présente l'intégralité des 813 lignes du fichier `src/module_mpi_comm.f90` de l'implémentation Fortran 2018 :

```
! [Procédure/Module] module module_mpi_comm
module module_mpi_comm
  use module_precision, only: dp
  use module_constants
  use module_mesh, only: mesh_2d_t, x_center, y_center, x_face
  use module_fields, only: flow_fields_t
  use module_config, only: config_t
  implicit none
  private
  include 'mpif.h'
```

```
11
12    public :: exchange_halos_scalar
13    public :: exchange_halos_velocity
14    public :: apply_velocity_bc
15    public :: apply_thermal_bc
16    public :: apply_vof_bc
17    public :: global_max_val, global_min_val, global_sum_val
18    public :: sync_u, sync_v, sync_cell_scalar
19    public :: apply_boundary_conditions, apply_thermal_boundary_conditions
20    public :: global_max, global_min, global_sum, global_norm2, global_dot_product
21
22  contains
23
24    ! [Procédure/Module] subroutine exchange_halos_scalar
25    subroutine exchange_halos_scalar(field, mesh)
26      ! Déclaration de variable locale / paramètre de calcul
27      type(mesh_2d_t), intent(in) :: mesh
28      ! Déclaration de variable locale / paramètre de calcul
29      real(dp), intent(inout) :: field(0:mesh%nx_local+1, 0:mesh%ny_local+1)
30
31      ! Déclaration de variable locale / paramètre de calcul
32      integer :: nx, ny, ierr
33      ! Déclaration de variable locale / paramètre de calcul
34      integer :: reqs(8), n_reqs
35      ! Déclaration de variable locale / paramètre de calcul
36      real(dp), allocatable :: s_left(:), r_left(:), s_right(:), r_right(:)
37      ! Déclaration de variable locale / paramètre de calcul
38      real(dp), allocatable :: s_bot(:), r_bot(:), s_top(:), r_top(:)
39
40      ! Affectation et calcul numérique
41      nx = mesh%nx_local
42      ! Affectation et calcul numérique
43      ny = mesh%ny_local
44
45      ! Affectation et calcul numérique
46      if (mesh%size <= 1) then
47         if (.not. mesh%is_boundary_left) then
48            field(0, 0:ny+1)    = field(nx, 0:ny+1)
49            field(nx+1, 0:ny+1) = field(1, 0:ny+1)
50         end if
51         if (.not. mesh%is_boundary_bottom) then
52            field(0:nx+1, 0)    = field(0:nx+1, ny)
53            field(0:nx+1, ny+1) = field(0:nx+1, 1)
54         end if
55         return
56      end if
57
58      ! Allocation dynamique de tableau contigu en mémoire
59      allocate(s_left(ny), r_left(ny), s_right(ny), r_right(ny))
60      ! Allocation dynamique de tableau contigu en mémoire
61      allocate(s_bot(nx), r_bot(nx), s_top(nx), r_top(nx))
62
63      ! Affectation et calcul numérique
64      s_left(:)  = field(1, 1:ny)
65      ! Affectation et calcul numérique
66      s_right(:) = field(nx, 1:ny)
67
68      ! Affectation et calcul numérique
69      n_reqs = 0
70      ! Évaluation de la condition logique
71      if (mesh%neighbor_west /= MPI_PROC_NULL) then
72         ! Appel à la bibliothèque de communication parallèle MPI
73         call MPI_Irecv(r_left, ny, MPI_DOUBLE_PRECISION, mesh%neighbor_west, 10, mesh%comm, reqs(n_reqs+1),
               ierr)
74         ! Appel à la bibliothèque de communication parallèle MPI
75         call MPI_Isend(s_left, ny, MPI_DOUBLE_PRECISION, mesh%neighbor_west, 11, mesh%comm, reqs(n_reqs+2),
               ierr)
76         ! Affectation et calcul numérique
77         n_reqs = n_reqs + 2
78      end if
79      ! Évaluation de la condition logique
80      if (mesh%neighbor_east /= MPI_PROC_NULL) then
81         ! Appel à la bibliothèque de communication parallèle MPI
82         call MPI_Irecv(r_right, ny, MPI_DOUBLE_PRECISION, mesh%neighbor_east, 11, mesh%comm, reqs(n_reqs+1),
               ierr)
83         ! Appel à la bibliothèque de communication parallèle MPI
84         call MPI_Isend(s_right, ny, MPI_DOUBLE_PRECISION, mesh%neighbor_east, 10, mesh%comm, reqs(n_reqs+2),
               ierr)
85         ! Affectation et calcul numérique
86         n_reqs = n_reqs + 2
87      end if
88
89      ! Évaluation de la condition logique
90      if (n_reqs > 0) then
91         ! Appel à la bibliothèque de communication parallèle MPI
92         call MPI_Waitall(n_reqs, reqs, MPI_STATUSES_IGNORE, ierr)
93      end if
94
95      ! Affectation et calcul numérique
96      if (mesh%neighbor_west /= MPI_PROC_NULL) field(0, 1:ny) = r_left(:)
97      ! Affectation et calcul numérique
98      if (mesh%neighbor_east /= MPI_PROC_NULL) field(nx+1, 1:ny) = r_right(:)
```

```
99
100     ! Affectation et calcul numérique
101     s_bot(:) = field(1:nx, 1)
102     ! Actualisation de la pression statique au centre de la maille
103     s_top(:) = field(1:nx, ny)
104
105     ! Affectation et calcul numérique
106     n_reqs = 0
107     ! Évaluation de la condition logique
108     if (mesh%neighbor_south /= MPI_PROC_NULL) then
109        ! Appel à la bibliothèque de communication parallèle MPI
110        call MPI_Irecv(r_bot, nx, MPI_DOUBLE_PRECISION, mesh%neighbor_south, 20, mesh%comm, reqs(n_reqs+1),
               ierr)
111        ! Appel à la bibliothèque de communication parallèle MPI
112        call MPI_Isend(s_bot, nx, MPI_DOUBLE_PRECISION, mesh%neighbor_south, 21, mesh%comm, reqs(n_reqs+2),
               ierr)
113        ! Affectation et calcul numérique
114        n_reqs = n_reqs + 2
115     end if
116     ! Évaluation de la condition logique
117     if (mesh%neighbor_north /= MPI_PROC_NULL) then
118        ! Appel à la bibliothèque de communication parallèle MPI
119        call MPI_Irecv(r_top, nx, MPI_DOUBLE_PRECISION, mesh%neighbor_north, 21, mesh%comm, reqs(n_reqs+1),
               ierr)
120        ! Appel à la bibliothèque de communication parallèle MPI
121        call MPI_Isend(s_top, nx, MPI_DOUBLE_PRECISION, mesh%neighbor_north, 20, mesh%comm, reqs(n_reqs+2),
               ierr)
122        ! Affectation et calcul numérique
123        n_reqs = n_reqs + 2
124     end if
125
126     ! Évaluation de la condition logique
127     if (n_reqs > 0) then
128        ! Appel à la bibliothèque de communication parallèle MPI
129        call MPI_Waitall(n_reqs, reqs, MPI_STATUSES_IGNORE, ierr)
130     end if
131
132     ! Affectation et calcul numérique
133     if (mesh%neighbor_south /= MPI_PROC_NULL) field(1:nx, 0) = r_bot(:)
134     ! Affectation et calcul numérique
135     if (mesh%neighbor_north /= MPI_PROC_NULL) field(1:nx, ny+1) = r_top(:)
136   end subroutine exchange_halos_scalar
137
138   ! [Procédure/Module] subroutine exchange_halos_velocity
139   subroutine exchange_halos_velocity(u, v, mesh)
140     ! Déclaration de variable locale / paramètre de calcul
141     type(mesh_2d_t), intent(in) :: mesh
142     ! Déclaration de variable locale / paramètre de calcul
143     real(dp), intent(inout) :: u(0:mesh%nx_local+1, 0:mesh%ny_local+1)
144     ! Déclaration de variable locale / paramètre de calcul
145     real(dp), intent(inout) :: v(0:mesh%nx_local+1, 0:mesh%ny_local+1)
146
147     ! Déclaration de variable locale / paramètre de calcul
148     integer :: nx, ny, ierr
149     ! Déclaration de variable locale / paramètre de calcul
150     integer :: reqs(8), n_reqs
151     ! Déclaration de variable locale / paramètre de calcul
152     real(dp), allocatable :: su_l(:), ru_l(:), su_r(:), ru_r(:)
153     ! Déclaration de variable locale / paramètre de calcul
154     real(dp), allocatable :: sv_l(:), rv_l(:), sv_r(:), rv_r(:)
155     ! Déclaration de variable locale / paramètre de calcul
156     real(dp), allocatable :: su_b(:), ru_b(:), su_t(:), ru_t(:)
157     ! Déclaration de variable locale / paramètre de calcul
158     real(dp), allocatable :: sv_b(:), rv_b(:), sv_t(:), rv_t(:)
159
160     ! Affectation et calcul numérique
161     if (mesh%size <= 1) return
162
163     ! Affectation et calcul numérique
164     nx = mesh%nx_local
165     ! Affectation et calcul numérique
166     ny = mesh%ny_local
167
168     ! Allocation dynamique de tableau contigu en mémoire
169     allocate(su_l(ny), ru_l(ny), su_r(ny), ru_r(ny))
170     ! Allocation dynamique de tableau contigu en mémoire
171     allocate(sv_l(ny), rv_l(ny), sv_r(ny), rv_r(ny))
172
173     ! Affectation et calcul numérique
174     su_l(:) = u(1, 1:ny)
175     ! Affectation et calcul numérique
176     su_r(:) = u(nx, 1:ny)
177     ! Affectation et calcul numérique
178     sv_l(:) = v(1, 1:ny)
179     ! Affectation et calcul numérique
180     sv_r(:) = v(nx, 1:ny)
181
182     ! Affectation et calcul numérique
183     n_reqs = 0
184     ! Évaluation de la condition logique
185     if (mesh%neighbor_west /= MPI_PROC_NULL) then
186        ! Appel à la bibliothèque de communication parallèle MPI
```

```
187         call MPI_Irecv(ru_l, ny, MPI_DOUBLE_PRECISION, mesh%neighbor_west, 30, mesh%comm, reqs(n_reqs+1), ierr)
188         ! Appel à la bibliothèque de communication parallèle MPI
189         call MPI_Isend(su_l, ny, MPI_DOUBLE_PRECISION, mesh%neighbor_west, 31, mesh%comm, reqs(n_reqs+2), ierr)
190         ! Appel à la bibliothèque de communication parallèle MPI
191         call MPI_Irecv(rv_l, ny, MPI_DOUBLE_PRECISION, mesh%neighbor_west, 32, mesh%comm, reqs(n_reqs+3), ierr)
192         ! Appel à la bibliothèque de communication parallèle MPI
193         call MPI_Isend(sv_l, ny, MPI_DOUBLE_PRECISION, mesh%neighbor_west, 33, mesh%comm, reqs(n_reqs+4), ierr)
194         ! Affectation et calcul numérique
195         n_reqs = n_reqs + 4
196       end if
197       ! Évaluation de la condition logique
198       if (mesh%neighbor_east /= MPI_PROC_NULL) then
199         ! Appel à la bibliothèque de communication parallèle MPI
200         call MPI_Irecv(ru_r, ny, MPI_DOUBLE_PRECISION, mesh%neighbor_east, 31, mesh%comm, reqs(n_reqs+1), ierr)
201         ! Appel à la bibliothèque de communication parallèle MPI
202         call MPI_Isend(su_r, ny, MPI_DOUBLE_PRECISION, mesh%neighbor_east, 30, mesh%comm, reqs(n_reqs+2), ierr)
203         ! Appel à la bibliothèque de communication parallèle MPI
204         call MPI_Irecv(rv_r, ny, MPI_DOUBLE_PRECISION, mesh%neighbor_east, 33, mesh%comm, reqs(n_reqs+3), ierr)
205         ! Appel à la bibliothèque de communication parallèle MPI
206         call MPI_Isend(sv_r, ny, MPI_DOUBLE_PRECISION, mesh%neighbor_east, 32, mesh%comm, reqs(n_reqs+4), ierr)
207         ! Affectation et calcul numérique
208         n_reqs = n_reqs + 4
209       end if
210
211       if (n_reqs > 0) call MPI_Waitall(n_reqs, reqs, MPI_STATUSES_IGNORE, ierr)
212
213       ! Évaluation de la condition logique
214       if (mesh%neighbor_west /= MPI_PROC_NULL) then
215         ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
216         u(0, 1:ny) = ru_l(:)
217         ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
218         v(0, 1:ny) = rv_l(:)
219       end if
220       ! Évaluation de la condition logique
221       if (mesh%neighbor_east /= MPI_PROC_NULL) then
222         ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
223         u(nx+1, 1:ny) = ru_r(:)
224         ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
225         v(nx+1, 1:ny) = rv_r(:)
226       end if
227
228       ! Allocation dynamique de tableau contigu en mémoire
229       allocate(su_b(nx+2), ru_b(nx+2), su_t(nx+2), ru_t(nx+2))
230       ! Allocation dynamique de tableau contigu en mémoire
231       allocate(sv_b(nx),   rv_b(nx),   sv_t(nx),   rv_t(nx))
232
233       ! Affectation et calcul numérique
234       su_b(:) = u(0:nx+1, 1)
235       ! Affectation et calcul numérique
236       su_t(:) = u(0:nx+1, ny)
237       ! Affectation et calcul numérique
238       sv_b(:) = v(1:nx, 1)
239       ! Affectation et calcul numérique
240       sv_t(:) = v(1:nx, ny)
241
242       ! Affectation et calcul numérique
243       n_reqs = 0
244       ! Évaluation de la condition logique
245       if (mesh%neighbor_south /= MPI_PROC_NULL) then
246         ! Appel à la bibliothèque de communication parallèle MPI
247         call MPI_Irecv(ru_b, nx+2, MPI_DOUBLE_PRECISION, mesh%neighbor_south, 40, mesh%comm, reqs(n_reqs+1),
              ierr)
248         ! Appel à la bibliothèque de communication parallèle MPI
249         call MPI_Isend(su_b, nx+2, MPI_DOUBLE_PRECISION, mesh%neighbor_south, 41, mesh%comm, reqs(n_reqs+2),
              ierr)
250         ! Appel à la bibliothèque de communication parallèle MPI
251         call MPI_Irecv(rv_b, nx,   MPI_DOUBLE_PRECISION, mesh%neighbor_south, 42, mesh%comm, reqs(n_reqs+3),
              ierr)
252         ! Appel à la bibliothèque de communication parallèle MPI
253         call MPI_Isend(sv_b, nx,   MPI_DOUBLE_PRECISION, mesh%neighbor_south, 43, mesh%comm, reqs(n_reqs+4),
              ierr)
254         ! Affectation et calcul numérique
255         n_reqs = n_reqs + 4
256       end if
257       ! Évaluation de la condition logique
258       if (mesh%neighbor_north /= MPI_PROC_NULL) then
259         ! Appel à la bibliothèque de communication parallèle MPI
260         call MPI_Irecv(ru_t, nx+2, MPI_DOUBLE_PRECISION, mesh%neighbor_north, 41, mesh%comm, reqs(n_reqs+1),
              ierr)
261         ! Appel à la bibliothèque de communication parallèle MPI
262         call MPI_Isend(su_t, nx+2, MPI_DOUBLE_PRECISION, mesh%neighbor_north, 40, mesh%comm, reqs(n_reqs+2),
              ierr)
263         ! Appel à la bibliothèque de communication parallèle MPI
264         call MPI_Irecv(rv_t, nx,   MPI_DOUBLE_PRECISION, mesh%neighbor_north, 43, mesh%comm, reqs(n_reqs+3),
              ierr)
265         ! Appel à la bibliothèque de communication parallèle MPI
266         call MPI_Isend(sv_t, nx,   MPI_DOUBLE_PRECISION, mesh%neighbor_north, 42, mesh%comm, reqs(n_reqs+4),
              ierr)
267         ! Affectation et calcul numérique
268         n_reqs = n_reqs + 4
269       end if
270
```

```
    if (n_reqs > 0) call MPI_Waitall(n_reqs, reqs, MPI_STATUSES_IGNORE, ierr)

    ! Évaluation de la condition logique
    if (mesh%neighbor_south /= MPI_PROC_NULL) then
       ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
       u(0:nx+1, 0) = ru_b(:)
       ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
       v(1:nx, 0)   = rv_b(:)
    end if
    ! Évaluation de la condition logique
    if (mesh%neighbor_north /= MPI_PROC_NULL) then
       ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
       u(0:nx+1, ny+1) = ru_t(:)
       ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
       v(1:nx, ny+1)   = rv_t(:)
    end if
  end subroutine exchange_halos_velocity

  ! [Procédure/Module] subroutine apply_velocity_bc
  subroutine apply_velocity_bc(fields, mesh, cfg, homogeneous)
    ! Déclaration de variable locale / paramètre de calcul
    type(flow_fields_t), intent(inout) :: fields
    ! Déclaration de variable locale / paramètre de calcul
    type(mesh_2d_t), intent(in) :: mesh
    ! Déclaration de variable locale / paramètre de calcul
    type(config_t), intent(in) :: cfg
    ! Déclaration de variable locale / paramètre de calcul
    logical, intent(in) :: homogeneous

    ! Déclaration de variable locale / paramètre de calcul
    integer :: nx, ny, i, j
    ! Déclaration de variable locale / paramètre de calcul
    real(dp) :: u_l, v_l, u_r, v_r, u_b, v_b, u_t, v_t
    ! Déclaration de variable locale / paramètre de calcul
    real(dp) :: y, h, u_inlet
    ! Déclaration des variables pour injecteur supérieur (Cas 3.3 Fluids 2021)
    real(dp) :: x_c, xc_top, half_w_top, v_out_top

    ! Affectation et calcul numérique
    nx = mesh%nx_local
    ! Affectation et calcul numérique
    ny = mesh%ny_local

    ! Affectation et calcul numérique
    u_l = merge(0.0_dp, cfg%bc_left%u, homogeneous)
    ! Affectation et calcul numérique
    v_l = merge(0.0_dp, cfg%bc_left%v, homogeneous)
    ! Affectation et calcul numérique
    u_r = merge(0.0_dp, cfg%bc_right%u, homogeneous)
    ! Affectation et calcul numérique
    v_r = merge(0.0_dp, cfg%bc_right%v, homogeneous)
    ! Affectation et calcul numérique
    u_b = merge(0.0_dp, cfg%bc_bottom%u, homogeneous)
    ! Affectation et calcul numérique
    v_b = merge(0.0_dp, cfg%bc_bottom%v, homogeneous)
    ! Affectation et calcul numérique
    u_t = merge(0.0_dp, cfg%bc_top%u, homogeneous)
    ! Affectation et calcul numérique
    v_t = merge(0.0_dp, cfg%bc_top%v, homogeneous)

    ! Traitement périodicité si 1 seul processus
    if (cfg%bc_left%btype == BC_PERIODIC .and. mesh%px == 1) then
       ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
       fields%u(0, 1:ny)    = fields%u(nx, 1:ny)
       ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
       fields%u(nx+1, 1:ny) = fields%u(1, 1:ny)
       ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
       fields%v(0, 1:ny)    = fields%v(nx, 1:ny)
       ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
       fields%v(nx+1, 1:ny) = fields%v(1, 1:ny)
    end if
    ! Évaluation de la condition logique
    if (cfg%bc_bottom%btype == BC_PERIODIC .and. mesh%py == 1) then
       ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
       fields%v(1:nx, 0)    = fields%v(1:nx, ny)
       ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
       fields%v(1:nx, ny+1) = fields%v(1:nx, 1)
       ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
       fields%u(0:nx+1, 0)    = fields%u(0:nx+1, ny)
       ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
       fields%u(0:nx+1, ny+1) = fields%u(0:nx+1, 1)
    end if

    ! Bord Ouest physique (x = 0)
    if (mesh%neighbor_west == MPI_PROC_NULL .and. cfg%bc_left%btype /= BC_PERIODIC) then
       ! Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
       do j = 1, ny
          ! Évaluation de la condition logique
          if (cfg%bc_left%btype == BC_DIRICHLET) then
             ! Affectation et calcul numérique
             u_inlet = u_l
             ! Évaluation de la condition logique
```

```
            if (.not. homogeneous .and. cfg%obs_enabled .and. cfg%obs_type == OBS_BOX) then
               ! Profil d'entrée marche descendante : parabole sur [y_step, ly]
               y = y_center(mesh, j)
               ! Évaluation de la condition logique
               if (y < cfg%obs_y1) then
                  ! Affectation et calcul numérique
                  u_inlet = 0.0_dp
               ! Branche par défaut si condition non satisfaite
               else
                  ! Affectation et calcul numérique
                  h = mesh%ly - cfg%obs_y1
                  ! Affectation et calcul numérique
                  u_inlet = 6.0_dp * cfg%bc_left%u * (y - cfg%obs_y1) * (mesh%ly - y) / (h * h)
               end if
            end if
            ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
            if (.not. homogeneous) fields%u(1, j) = u_inlet
            ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
            fields%u(0, j) = u_inlet
            ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
            fields%v(0, j) = 2.0_dp * v_l - fields%v(1, j)
         ! Branche conditionnelle alternative
         else if (cfg%bc_left%btype == BC_FREESLIP) then
            ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
            if (.not. homogeneous) fields%u(1, j) = 0.0_dp
            ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
            fields%u(0, j) = 0.0_dp
            ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
            fields%v(0, j) = fields%v(1, j)
         ! Branche conditionnelle alternative
         else if (cfg%bc_left%btype == BC_NEUMANN) then
            ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
            fields%u(0, j) = fields%u(1, j)
            ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
            fields%v(0, j) = fields%v(1, j)
         end if
      end do
   end if

   ! Bord Est physique (x = Lx)
   if (mesh%neighbor_east == MPI_PROC_NULL .and. cfg%bc_right%btype /= BC_PERIODIC) then
      ! Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
      do j = 1, ny
         ! Évaluation de la condition logique
         if (cfg%bc_right%btype == BC_DIRICHLET) then
            ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
            if (.not. homogeneous) fields%u(nx+1, j) = u_r
            ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
            fields%u(nx+1, j) = u_r
            ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
            fields%v(nx+1, j) = 2.0_dp * v_r - fields%v(nx, j)
         ! Branche conditionnelle alternative
         else if (cfg%bc_right%btype == BC_FREESLIP) then
            ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
            if (.not. homogeneous) fields%u(nx+1, j) = 0.0_dp
            ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
            fields%u(nx+1, j) = 0.0_dp
            ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
            fields%v(nx+1, j) = fields%v(nx, j)
         ! Branche conditionnelle alternative
         else if (cfg%bc_right%btype == BC_NEUMANN) then
            ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
            fields%u(nx+1, j) = fields%u(nx, j)
            ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
            fields%v(nx+1, j) = fields%v(nx, j)
         end if
      end do
   end if

   ! Bord Sud physique (y = 0)
   if (mesh%neighbor_south == MPI_PROC_NULL .and. cfg%bc_bottom%btype /= BC_PERIODIC) then
      ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
      do i = 1, nx
         ! Évaluation de la condition logique
         if (cfg%bc_bottom%btype == BC_DIRICHLET) then
            ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
            if (.not. homogeneous) fields%v(i, 1) = v_b
            ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
            fields%v(i, 0) = v_b
            ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
            fields%u(i, 0) = 2.0_dp * u_b - fields%u(i, 1)
         ! Branche conditionnelle alternative
         else if (cfg%bc_bottom%btype == BC_FREESLIP) then
            ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
            if (.not. homogeneous) fields%v(i, 1) = 0.0_dp
            ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
            fields%v(i, 0) = 0.0_dp
            ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
            fields%u(i, 0) = fields%u(i, 1)
         ! Branche conditionnelle alternative
         else if (cfg%bc_bottom%btype == BC_NEUMANN) then
            ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
```

```
            fields%u(i, 0) = fields%u(i, 1)
            ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
            fields%v(i, 0) = fields%v(i, 1)
         end if
      end do
   end if

   ! Bord Nord physique (y = Ly)
   if (mesh%neighbor_north == MPI_PROC_NULL .and. cfg%bc_top%btype /= BC_PERIODIC) then
      ! Traitement du profil d'injecteur JET_INLET (Cas 3.3 Fluids 2021)
      if (trim(cfg%bc_top%profile) == "JET_INLET") then
         xc_top = merge(cfg%bc_top%jet_center, 0.5_dp * mesh%lx, cfg%bc_top%jet_center > 0.0_dp)
         half_w_top = merge(0.5_dp * cfg%bc_top%jet_width, 0.04_dp, cfg%bc_top%jet_width > 0.0_dp)
         y = 0.0_dp
         do i = 1, mesh%nx_global
            if (abs((real(i, dp) - 0.5_dp) * mesh%dx - xc_top) <= half_w_top) y = y + mesh%dx
         end do
         if (y < 1.0e-12_dp) y = 2.0_dp * half_w_top
         v_out_top = -v_t * (2.0_dp * half_w_top) / (mesh%lx - y)
         do i = 1, nx
            x_c = x_center(mesh, i)
            if (abs(x_c - xc_top) <= half_w_top) then
               if (.not. homogeneous) fields%v(i, ny+1) = v_t * (2.0_dp * half_w_top) / y
            else
               if (.not. homogeneous) fields%v(i, ny+1) = v_out_top
            end if
            ! Condition tangentielle symétrique évaluée à l'abscisse exacte de la face u
            if (abs(x_face(mesh, i) - xc_top) <= half_w_top) then
               fields%u(i, ny+1) = 0.0_dp
            else
               fields%u(i, ny+1) = fields%u(i, ny)
            end if
         end do
         fields%u(nx+1, ny+1) = fields%u(nx+1, ny)
      else
         ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
         do i = 1, nx
            ! Évaluation de la condition logique
            if (cfg%bc_top%btype == BC_DIRICHLET) then
               ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
               if (.not. homogeneous) fields%v(i, ny+1) = v_t
               fields%u(i, ny+1) = 2.0_dp * u_t - fields%u(i, ny)
            ! Branche conditionnelle alternative
            else if (cfg%bc_top%btype == BC_FREESLIP) then
               ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
               if (.not. homogeneous) fields%v(i, ny+1) = 0.0_dp
               fields%u(i, ny+1) = fields%u(i, ny)
            ! Branche conditionnelle alternative
            else if (cfg%bc_top%btype == BC_NEUMANN) then
               ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
               fields%u(i, ny+1) = fields%u(i, ny)
               fields%v(i, ny+1) = fields%v(i, ny)
            end if
         end do
      end if
   end if
end subroutine apply_velocity_bc

! [Procédure/Module] subroutine apply_thermal_bc
subroutine apply_thermal_bc(fields, mesh, cfg)
   ! Déclaration de variable locale / paramètre de calcul
   type(flow_fields_t), intent(inout) :: fields
   ! Déclaration de variable locale / paramètre de calcul
   type(mesh_2d_t), intent(in) :: mesh
   ! Déclaration de variable locale / paramètre de calcul
   type(config_t), intent(in) :: cfg

   ! Déclaration de variable locale / paramètre de calcul
   integer :: nx, ny, i, j
   ! Affectation et calcul numérique
   nx = mesh%nx_local
   ! Affectation et calcul numérique
   ny = mesh%ny_local

   ! Périodicité thermique
   if (cfg%bc_left%thermal_type == BC_PERIODIC .and. mesh%px == 1) then
      ! Évolution du champ scalaire de température
      fields%T(0, 1:ny)    = fields%T(nx, 1:ny)
      ! Évolution du champ scalaire de température
      fields%T(nx+1, 1:ny) = fields%T(1, 1:ny)
   end if
   ! Évaluation de la condition logique
   if (cfg%bc_bottom%thermal_type == BC_PERIODIC .and. mesh%py == 1) then
      ! Évolution du champ scalaire de température
      fields%T(1:nx, 0)    = fields%T(1:nx, ny)
      ! Évolution du champ scalaire de température
      fields%T(1:nx, ny+1) = fields%T(1:nx, 1)
   end if

   ! Bord Ouest physique (x = 0)
   if (mesh%neighbor_west == MPI_PROC_NULL) then
      ! Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
```

```
      do j = 1, ny
         ! Évaluation de la condition logique
         if (cfg%bc_left%thermal_type == THERMAL_DIRICHLET) then
            ! Évolution du champ scalaire de température
            fields%T(0, j) = 2.0_dp * cfg%bc_left%T - fields%T(1, j)
         ! Branche par défaut si condition non satisfaite
         else
            ! Évolution du champ scalaire de température
            fields%T(0, j) = fields%T(1, j)
         end if
      end do
   end if

   ! Bord Est physique (x = Lx)
   if (mesh%neighbor_east == MPI_PROC_NULL) then
      ! Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
      do j = 1, ny
         ! Évaluation de la condition logique
         if (cfg%bc_right%thermal_type == THERMAL_DIRICHLET) then
            ! Évolution du champ scalaire de température
            fields%T(nx+1, j) = 2.0_dp * cfg%bc_right%T - fields%T(nx, j)
         ! Branche par défaut si condition non satisfaite
         else
            ! Évolution du champ scalaire de température
            fields%T(nx+1, j) = fields%T(nx, j)
         end if
      end do
   end if

   ! Bord Sud physique (y = 0)
   if (mesh%neighbor_south == MPI_PROC_NULL) then
      ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
      do i = 1, nx
         ! Évaluation de la condition logique
         if (cfg%bc_bottom%thermal_type == THERMAL_DIRICHLET) then
            ! Évolution du champ scalaire de température
            fields%T(i, 0) = 2.0_dp * cfg%bc_bottom%T - fields%T(i, 1)
         ! Branche par défaut si condition non satisfaite
         else
            ! Évolution du champ scalaire de température
            fields%T(i, 0) = fields%T(i, 1)
         end if
      end do
   end if

   ! Bord Nord physique (y = Ly)
   if (mesh%neighbor_north == MPI_PROC_NULL) then
      ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
      do i = 1, nx
         ! Évaluation de la condition logique
         if (cfg%bc_top%thermal_type == THERMAL_DIRICHLET) then
            ! Évolution du champ scalaire de température
            fields%T(i, ny+1) = 2.0_dp * cfg%bc_top%T - fields%T(i, ny)
         ! Branche par défaut si condition non satisfaite
         else
            ! Évolution du champ scalaire de température
            fields%T(i, ny+1) = fields%T(i, ny)
         end if
      end do
   end if
end subroutine apply_thermal_bc

! [Procédure/Module] subroutine apply_vof_bc
subroutine apply_vof_bc(fields, mesh, cfg)
   ! Déclaration de variable locale / paramètre de calcul
   type(flow_fields_t), intent(inout) :: fields
   ! Déclaration de variable locale / paramètre de calcul
   type(mesh_2d_t), intent(in) :: mesh
   ! Déclaration de variable locale / paramètre de calcul
   type(config_t), intent(in) :: cfg

   ! Déclaration de variable locale / paramètre de calcul
   integer :: nx, ny, i
   real(dp) :: x_c, xc_top, half_w_top
   ! Affectation et calcul numérique
   nx = mesh%nx_local
   ! Affectation et calcul numérique
   ny = mesh%ny_local

   ! Affectation et calcul numérique
   if (mesh%neighbor_west == MPI_PROC_NULL)  fields%C(0, 1:ny)    = fields%C(1, 1:ny)
   ! Affectation et calcul numérique
   if (mesh%neighbor_east == MPI_PROC_NULL)  fields%C(nx+1, 1:ny) = fields%C(nx, 1:ny)
   ! Affectation et calcul numérique
   if (mesh%neighbor_south == MPI_PROC_NULL) fields%C(1:nx, 0)    = fields%C(1:nx, 1)
   ! Affectation et calcul numérique pour bord Nord
   if (mesh%neighbor_north == MPI_PROC_NULL) then
      if (trim(cfg%bc_top%profile) == "JET_INLET") then
         xc_top = merge(cfg%bc_top%jet_center, 0.5_dp * mesh%lx, cfg%bc_top%jet_center > 0.0_dp)
         half_w_top = merge(0.5_dp * cfg%bc_top%jet_width, 0.04_dp, cfg%bc_top%jet_width > 0.0_dp)
         do i = 1, nx
            x_c = x_center(mesh, i)
```

```
            if (abs(x_c - xc_top) <= half_w_top) then
               fields%C(i, ny+1) = 0.0_dp ! Injection continue de liquide (Fluide 2)
            else
               fields%C(i, ny+1) = fields%C(i, ny) ! Évacuation libre du gaz
            end if
         end do
      else
         fields%C(1:nx, ny+1) = fields%C(1:nx, ny)
      end if
    end if
  end subroutine apply_vof_bc

  ! Déclaration de variable locale / paramètre de calcul
  real(dp) function global_max_val(local_val, comm)
    ! Déclaration de variable locale / paramètre de calcul
    real(dp), intent(in) :: local_val
    ! Déclaration de variable locale / paramètre de calcul
    integer, intent(in) :: comm
    ! Déclaration de variable locale / paramètre de calcul
    real(dp) :: res
    ! Déclaration de variable locale / paramètre de calcul
    integer :: ierr
    ! Appel à la bibliothèque de communication parallèle MPI
    call MPI_Allreduce(local_val, res, 1, MPI_DOUBLE_PRECISION, MPI_MAX, comm, ierr)
    ! Affectation et calcul numérique
    global_max_val = res
  end function global_max_val

  ! Déclaration de variable locale / paramètre de calcul
  real(dp) function global_min_val(local_val, comm)
    ! Déclaration de variable locale / paramètre de calcul
    real(dp), intent(in) :: local_val
    ! Déclaration de variable locale / paramètre de calcul
    integer, intent(in) :: comm
    ! Déclaration de variable locale / paramètre de calcul
    real(dp) :: res
    ! Déclaration de variable locale / paramètre de calcul
    integer :: ierr
    ! Appel à la bibliothèque de communication parallèle MPI
    call MPI_Allreduce(local_val, res, 1, MPI_DOUBLE_PRECISION, MPI_MIN, comm, ierr)
    ! Affectation et calcul numérique
    global_min_val = res
  end function global_min_val

  ! Déclaration de variable locale / paramètre de calcul
  real(dp) function global_sum_val(local_val, comm)
    ! Déclaration de variable locale / paramètre de calcul
    real(dp), intent(in) :: local_val
    ! Déclaration de variable locale / paramètre de calcul
    integer, intent(in) :: comm
    ! Déclaration de variable locale / paramètre de calcul
    real(dp) :: res
    ! Déclaration de variable locale / paramètre de calcul
    integer :: ierr
    ! Appel à la bibliothèque de communication parallèle MPI
    call MPI_Allreduce(local_val, res, 1, MPI_DOUBLE_PRECISION, MPI_SUM, comm, ierr)
    ! Affectation et calcul numérique
    global_sum_val = res
  end function global_sum_val

  ! [Procédure/Module] subroutine sync_u
  subroutine sync_u(mesh, u)
    ! Déclaration de variable locale / paramètre de calcul
    type(mesh_2d_t), intent(in) :: mesh
    ! Déclaration de variable locale / paramètre de calcul
    real(dp), intent(inout) :: u(0:mesh%nx_local+1, 0:mesh%ny_local+1)
    ! Appel de sous-programme modulaire
    call exchange_halos_scalar(u, mesh)
  end subroutine sync_u

  ! [Procédure/Module] subroutine sync_v
  subroutine sync_v(mesh, v)
    ! Déclaration de variable locale / paramètre de calcul
    type(mesh_2d_t), intent(in) :: mesh
    ! Déclaration de variable locale / paramètre de calcul
    real(dp), intent(inout) :: v(0:mesh%nx_local+1, 0:mesh%ny_local+1)
    ! Appel de sous-programme modulaire
    call exchange_halos_scalar(v, mesh)
  end subroutine sync_v

  ! [Procédure/Module] subroutine sync_cell_scalar
  subroutine sync_cell_scalar(mesh, s)
    ! Déclaration de variable locale / paramètre de calcul
    type(mesh_2d_t), intent(in) :: mesh
    ! Déclaration de variable locale / paramètre de calcul
    real(dp), intent(inout) :: s(0:mesh%nx_local+1, 0:mesh%ny_local+1)
    ! Appel de sous-programme modulaire
    call exchange_halos_scalar(s, mesh)
  end subroutine sync_cell_scalar

  ! [Procédure/Module] subroutine apply_boundary_conditions
  subroutine apply_boundary_conditions(mesh, cfg, fields, homogeneous)
```

```
    ! Déclaration de variable locale / paramètre de calcul
    type(mesh_2d_t), intent(in) :: mesh
    ! Déclaration de variable locale / paramètre de calcul
    type(config_t), intent(in) :: cfg
    ! Déclaration de variable locale / paramètre de calcul
    type(flow_fields_t), intent(inout) :: fields
    ! Déclaration de variable locale / paramètre de calcul
    logical, intent(in) :: homogeneous
    ! Appel de sous-programme modulaire
    call apply_velocity_bc(fields, mesh, cfg, homogeneous)
  end subroutine apply_boundary_conditions

  ! [Procédure/Module] subroutine apply_thermal_boundary_conditions
  subroutine apply_thermal_boundary_conditions(mesh, cfg, fields)
    ! Déclaration de variable locale / paramètre de calcul
    type(mesh_2d_t), intent(in) :: mesh
    ! Déclaration de variable locale / paramètre de calcul
    type(config_t), intent(in) :: cfg
    ! Déclaration de variable locale / paramètre de calcul
    type(flow_fields_t), intent(inout) :: fields
    ! Appel de sous-programme modulaire
    call apply_thermal_bc(fields, mesh, cfg)
  end subroutine apply_thermal_boundary_conditions

  ! Déclaration de variable locale / paramètre de calcul
  real(dp) function global_max(mesh, local_val)
    ! Déclaration de variable locale / paramètre de calcul
    type(mesh_2d_t), intent(in) :: mesh
    ! Déclaration de variable locale / paramètre de calcul
    real(dp), intent(in) :: local_val
    ! Affectation et calcul numérique
    global_max = global_max_val(local_val, mesh%comm)
  end function global_max

  ! Déclaration de variable locale / paramètre de calcul
  real(dp) function global_min(mesh, local_val)
    ! Déclaration de variable locale / paramètre de calcul
    type(mesh_2d_t), intent(in) :: mesh
    ! Déclaration de variable locale / paramètre de calcul
    real(dp), intent(in) :: local_val
    ! Affectation et calcul numérique
    global_min = global_min_val(local_val, mesh%comm)
  end function global_min

  ! Déclaration de variable locale / paramètre de calcul
  real(dp) function global_sum(mesh, local_val)
    ! Déclaration de variable locale / paramètre de calcul
    type(mesh_2d_t), intent(in) :: mesh
    ! Déclaration de variable locale / paramètre de calcul
    real(dp), intent(in) :: local_val
    ! Affectation et calcul numérique
    global_sum = global_sum_val(local_val, mesh%comm)
  end function global_sum

  ! Déclaration de variable locale / paramètre de calcul
  real(dp) function global_dot_product(mesh, x, y, n)
    ! Déclaration de variable locale / paramètre de calcul
    type(mesh_2d_t), intent(in) :: mesh
    ! Déclaration de variable locale / paramètre de calcul
    real(dp), intent(in) :: x(:), y(:)
    ! Déclaration de variable locale / paramètre de calcul
    integer, intent(in) :: n
    ! Déclaration de variable locale / paramètre de calcul
    real(dp) :: local_dot
    ! Affectation et calcul numérique
    local_dot = dot_product(x(1:n), y(1:n))
    ! Affectation et calcul numérique
    global_dot_product = global_sum_val(local_dot, mesh%comm)
  end function global_dot_product

  ! Déclaration de variable locale / paramètre de calcul
  real(dp) function global_norm2(mesh, x, n)
    ! Déclaration de variable locale / paramètre de calcul
    type(mesh_2d_t), intent(in) :: mesh
    ! Déclaration de variable locale / paramètre de calcul
    real(dp), intent(in) :: x(:)
    ! Déclaration de variable locale / paramètre de calcul
    integer, intent(in) :: n
    ! Évaluation de la norme du résidu ou de l'erreur numérique
    global_norm2 = sqrt(global_dot_product(mesh, x, x, n))
  end function global_norm2

end module module_mpi_comm
```

Chapitre 18

Matrices Creuses CSR, Préconditionneurs et Solveurs de Krylov en Fortran 2018 (module$_{l}inear_{s}olvers.f90$)

18.1 Implémentation Vectorisée des Solveurs de Krylov dans `module_linear_s`

Le module `src/module_linear_solvers.f90` (892 lignes) implémente la structure de matrice creuse `csr_matrix_t` (à indiçage Fortran à base 1 : `row_ptr`$(1) = 1$, `row_ptr`$(n+1) = \text{nnz} + 1$) ainsi que les solveurs :

- `solve_bicgstab2` : implémentation complète de l'algorithme de Sleijpen & Fokkema (1993) avec préconditionnement à droite ou à gauche (Jacobi, SSOR, ILU(0)) ;
- `solve_bicgstab` : gradient biconjugué stabilisé standard de Van der Vorst ;
- `solve_pcg` : gradient conjugué préconditionné pour l'opérateur symétrique défini positif du Laplacien (Cas 3.1) ;
- `solve_gmres` : méthode de minimisation résiduelle généralisée avec sous-espace de Krylov redémarré et rotations de Givens.

18.2 Répertoire des Types Dérivés, Sous-Routines et Fonctions de `src/module_linear_solvers.f90`

Ligne	Nature	Identifiant Fortran 2018	Interface / Arguments
10	module	`module_linear_solvers`	Unité modulaire Fortran 2018
30	subroutine	`apply_preconditioner`	Arguments : (ns, cfg, src, dst)
153	subroutine	`apply_PA`	Arguments : (ns, mesh, cfg, fields_old, dt, src, dst)
191	subroutine	`solve_bicgstab2`	Arguments : (ns, mesh, cfg, fields_old, dt, b, x, iters)
402	subroutine	`solve_bicgstab`	Arguments : (ns, mesh, cfg, fields_old, dt, b, x, iters)
551	subroutine	`solve_cg`	Arguments : (ns, mesh, cfg, fields_old, dt, b, x, iters)
659	subroutine	`solve_gmres`	Arguments : (ns, mesh, cfg, fields_old, dt, b, x, iters)
855	subroutine	`solve_linear_system`	Arguments : (ns, mesh, cfg, fields_old, dt, b, x, iters)

18.3 Code Source Intégral Annoté : `src/module_linear_solvers.f90`

Le listing ci-dessous présente l'intégralité des 892 lignes du fichier `src/module_linear_solvers.f90` de l'implémentation Fortran 2018 :

```
!===============================================================================
! Module: module_linear_solvers
! Description: Solveurs de Krylov parallèles pour COVOF 2D en Fortran 2018 :
!              - BiCGSTAB(2) (Sleijpen & van der Vorst 1993)
!              - BiCGSTAB standard (van der Vorst 1992)
!              - Conjugate Gradient (PCG - Hestenes & Stiefel 1952)
!              - GMRES(m) redémarré (Saad & Schultz 1986)
!              - Préconditionneurs Jacobi diagonal et SSOR
!===============================================================================
module module_linear_solvers
  use module_precision, only: dp
  use module_constants
```

```
  use module_config, only: config_t
  use module_mesh, only: mesh_2d_t
  use module_fields, only: flow_fields_t
  use module_mpi_comm, only: global_dot_product, global_norm2
  use module_navier_stokes, only: navier_stokes_t
  implicit none
  private

  public :: solve_linear_system
  public :: solve_bicgstab2, solve_bicgstab, solve_cg, solve_gmres
  public :: apply_preconditioner

contains

  !--------------------------------------------------------------------------
  ! Application du préconditionneur : dst = P^{-1} src
  !--------------------------------------------------------------------------
  subroutine apply_preconditioner(ns, cfg, src, dst)
    ! Déclaration de variable locale / paramètre de calcul
    type(navier_stokes_t), intent(in) :: ns
    ! Déclaration de variable locale / paramètre de calcul
    type(config_t), intent(in)        :: cfg
    ! Déclaration de variable locale / paramètre de calcul
    real(dp), intent(in)              :: src(:)
    ! Déclaration de variable locale / paramètre de calcul
    real(dp), intent(out)             :: dst(:)

    ! Déclaration de variable locale / paramètre de calcul
    integer :: i, nu, nv, np, n
    ! Déclaration de variable locale / paramètre de calcul
    real(dp) :: d

    ! Affectation et calcul numérique
    n = ns%n_owned_total
    ! Affectation et calcul numérique
    nu = ns%n_owned_u
    ! Affectation et calcul numérique
    nv = ns%n_owned_v
    ! Affectation et calcul numérique
    np = ns%n_owned_p

    ! Évaluation de la condition logique
    if (.not. cfg%use_precond) then
      ! Affectation et calcul numérique
      dst(1:n) = src(1:n)
      ! Retour à la procédure appelante
      return
    end if

    select case (cfg%precond_type)
    case (PRECOND_JACOBI)
      ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
      do i = 1, nu
        ! Affectation et calcul numérique
        d = ns%diag_u(i)
        ! Évaluation de la condition logique
        if (abs(d) > 1.0e-15_dp) then
          ! Affectation et calcul numérique
          dst(i) = src(i) / d
        ! Branche par défaut si condition non satisfaite
        else
          ! Affectation et calcul numérique
          dst(i) = src(i)
        end if
      end do

      ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
      do i = 1, nv
        ! Affectation et calcul numérique
        d = ns%diag_v(i)
        ! Évaluation de la condition logique
        if (abs(d) > 1.0e-15_dp) then
          ! Affectation et calcul numérique
          dst(nu + i) = src(nu + i) / d
        ! Branche par défaut si condition non satisfaite
        else
          ! Affectation et calcul numérique
          dst(nu + i) = src(nu + i)
        end if
      end do

      ! Évaluation de la condition logique
      if (cfg%solver_type == SOLVER_MONOLITHIC) then
        ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
        do i = 1, np
          ! Affectation et calcul numérique
          d = ns%diag_p(i)
          ! Évaluation de la condition logique
          if (abs(d) > 1.0e-15_dp) then
            ! Affectation et calcul numérique
            dst(nu + nv + i) = src(nu + nv + i) / d
          ! Branche par défaut si condition non satisfaite
```

```
          else
            ! Affectation et calcul numérique
            dst(nu + nv + i) = src(nu + nv + i)
          end if
        end do
      end if

    case (PRECOND_SSOR)
      ! Balayage SSOR avant / arrière avec omega = 1.0
      do i = 1, nu
        ! Affectation et calcul numérique
        d = ns%diag_u(i)
        ! Affectation et calcul numérique
        if (abs(d) < 1.0e-15_dp) d = 1.0_dp
        ! Affectation et calcul numérique
        dst(i) = src(i) / d
      end do
      ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
      do i = 1, nv
        ! Affectation et calcul numérique
        d = ns%diag_v(i)
        ! Affectation et calcul numérique
        if (abs(d) < 1.0e-15_dp) d = 1.0_dp
        ! Affectation et calcul numérique
        dst(nu + i) = src(nu + i) / d
      end do
      ! Évaluation de la condition logique
      if (cfg%solver_type == SOLVER_MONOLITHIC) then
        ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
        do i = 1, np
          ! Affectation et calcul numérique
          d = ns%diag_p(i)
          ! Affectation et calcul numérique
          if (abs(d) < 1.0e-15_dp) d = 1.0_dp
          ! Affectation et calcul numérique
          dst(nu + nv + i) = src(nu + nv + i) / d
        end do
      end if

    case default
      ! Affectation et calcul numérique
      dst(1:n) = src(1:n)
    end select
  end subroutine apply_preconditioner

  !-------------------------------------------------------------------------------
  ! Application de l'opérateur préconditionné : dst = P^{-1} * A * src
  !-------------------------------------------------------------------------------
  subroutine apply_PA(ns, mesh, cfg, fields_old, dt, src, dst)
    ! Déclaration de variable locale / paramètre de calcul
    type(navier_stokes_t), intent(in) :: ns
    ! Déclaration de variable locale / paramètre de calcul
    type(mesh_2d_t), intent(in)       :: mesh
    ! Déclaration de variable locale / paramètre de calcul
    type(config_t), intent(in)        :: cfg
    ! Déclaration de variable locale / paramètre de calcul
    type(flow_fields_t), intent(in)   :: fields_old
    ! Déclaration de variable locale / paramètre de calcul
    real(dp), intent(in)              :: dt
    ! Déclaration de variable locale / paramètre de calcul
    real(dp), intent(in)              :: src(:)
    ! Déclaration de variable locale / paramètre de calcul
    real(dp), intent(out)             :: dst(:)

    ! Déclaration de variable locale / paramètre de calcul
    real(dp), allocatable :: tmp(:)
    ! Déclaration de variable locale / paramètre de calcul
    integer :: n

    ! Affectation et calcul numérique
    n = ns%n_owned_total
    ! Allocation dynamique de tableau contigu en mémoire
    allocate(tmp(n))

    ! Appel de sous-programme modulaire
    call ns%apply_system_matrix(mesh, cfg, fields_old, dt, src, tmp)
    ! Appel de sous-programme modulaire
    call apply_preconditioner(ns, cfg, tmp, dst)

    ! Libération de la mémoire allouée
    deallocate(tmp)
  end subroutine apply_PA

  !-------------------------------------------------------------------------------
  ! 1. Solveur BiCGSTAB(2) - Sleijpen & van der Vorst (1993)
  !-------------------------------------------------------------------------------
  subroutine solve_bicgstab2(ns, mesh, cfg, fields_old, dt, b, x, iters)
    ! Déclaration de variable locale / paramètre de calcul
    type(navier_stokes_t), intent(in) :: ns
    ! Déclaration de variable locale / paramètre de calcul
    type(mesh_2d_t), intent(in)       :: mesh
    ! Déclaration de variable locale / paramètre de calcul
```

```
197     type(config_t), intent(in)          :: cfg
198     ! Déclaration de variable locale / paramètre de calcul
199     type(flow_fields_t), intent(in)     :: fields_old
200     ! Déclaration de variable locale / paramètre de calcul
201     real(dp), intent(in)                :: dt
202     ! Déclaration de variable locale / paramètre de calcul
203     real(dp), intent(in)                :: b(:)
204     ! Déclaration de variable locale / paramètre de calcul
205     real(dp), intent(inout)             :: x(:)
206     ! Déclaration de variable locale / paramètre de calcul
207     integer, intent(out)                :: iters
208
209     ! Déclaration de variable locale / paramètre de calcul
210     integer :: n, max_iter, iter
211     ! Déclaration de variable locale / paramètre de calcul
212     real(dp) :: tol, r0_norm, r_norm, rho_0, rho_1, alpha, omega, beta, gamma
213     ! Déclaration de variable locale / paramètre de calcul
214     real(dp) :: sigma11, sigma12, sigma22, rhs1, rhs2, det, s11_s22, gamma1, gamma2
215     ! Déclaration de variable locale / paramètre de calcul
216     real(dp), allocatable :: b_hat(:), r_tilde(:)
217     ! Déclaration de variable locale / paramètre de calcul
218     real(dp), allocatable :: r0(:), r1(:), r2(:)
219     ! Déclaration de variable locale / paramètre de calcul
220     real(dp), allocatable :: u0(:), u1(:), u2(:)
221
222     ! Affectation et calcul numérique
223     n = ns%n_owned_total
224     ! Affectation et calcul numérique
225     tol = cfg%tol
226     ! Affectation et calcul numérique
227     max_iter = cfg%max_iter
228     ! Affectation et calcul numérique
229     iters = 0
230
231     ! Allocation dynamique de tableau contigu en mémoire
232     allocate(b_hat(n), r_tilde(n))
233     ! Allocation dynamique de tableau contigu en mémoire
234     allocate(r0(n), r1(n), r2(n))
235     ! Allocation dynamique de tableau contigu en mémoire
236     allocate(u0(n), u1(n), u2(n))
237
238     ! Affectation et calcul numérique
239     u0 = 0.0_dp
240     ! Affectation et calcul numérique
241     u1 = 0.0_dp
242     ! Affectation et calcul numérique
243     u2 = 0.0_dp
244     ! Affectation et calcul numérique
245     r1 = 0.0_dp
246     ! Affectation et calcul numérique
247     r2 = 0.0_dp
248
249     ! Second membre préconditionné b_hat = P^{-1} b
250     call apply_preconditioner(ns, cfg, b, b_hat)
251
252     ! Résidu initial r0 = b_hat - P^{-1} A x0
253     call apply_PA(ns, mesh, cfg, fields_old, dt, x, r0)
254     ! Affectation et calcul numérique
255     r0(1:n) = b_hat(1:n) - r0(1:n)
256
257     ! Évaluation de la norme du résidu ou de l'erreur numérique
258     r0_norm = global_norm2(mesh, r0, n)
259     ! Évaluation de la condition logique
260     if (r0_norm < 1.0e-12_dp) then
261       ! Libération de la mémoire allouée
262       deallocate(b_hat, r_tilde, r0, r1, r2, u0, u1, u2)
263       ! Retour à la procédure appelante
264       return
265     end if
266
267     ! Affectation et calcul numérique
268     r_tilde(1:n) = r0(1:n)
269     ! Calcul de la masse volumique locale ou interpolée aux faces
270     rho_0 = 1.0_dp
271     ! Affectation et calcul numérique
272     alpha = 0.0_dp
273     ! Affectation et calcul numérique
274     omega = 1.0_dp
275     ! Affectation et calcul numérique
276     iter = 0
277
278     ! Boucle d'itération numérique
279     do while (iter < max_iter)
280       ! Affectation et calcul numérique
281       iter = iter + 2
282       ! Calcul de la masse volumique locale ou interpolée aux faces
283       rho_0 = -omega * rho_0
284
285       ! Demi-pas 1 (j = 0)
286       rho_1 = global_dot_product(mesh, r_tilde, r0, n)
287       if (abs(rho_0) < 1.0e-25_dp) exit
288       ! Affectation et calcul numérique
```

```
289       beta = alpha * (rho_1 / rho_0)
290       ! Calcul de la masse volumique locale ou interpolée aux faces
291       rho_0 = rho_1
292
293       ! Affectation et calcul numérique
294       u0(1:n) = r0(1:n) - beta * u0(1:n)
295       ! Appel de sous-programme modulaire
296       call apply_PA(ns, mesh, cfg, fields_old, dt, u0, u1)
297
298       ! Affectation et calcul numérique
299       gamma = global_dot_product(mesh, r_tilde, u1, n)
300       if (abs(gamma) < 1.0e-25_dp) exit
301       ! Affectation et calcul numérique
302       alpha = rho_0 / gamma
303
304       ! Affectation et calcul numérique
305       r0(1:n) = r0(1:n) - alpha * u1(1:n)
306       ! Affectation et calcul numérique
307       x(1:n)  = x(1:n)  + alpha * u0(1:n)
308       ! Appel de sous-programme modulaire
309       call apply_PA(ns, mesh, cfg, fields_old, dt, r0, r1)
310
311       ! Demi-pas 2 (j = 1)
312       rho_1 = global_dot_product(mesh, r_tilde, r1, n)
313       if (abs(rho_0) < 1.0e-25_dp) exit
314       ! Affectation et calcul numérique
315       beta = alpha * (rho_1 / rho_0)
316       ! Calcul de la masse volumique locale ou interpolée aux faces
317       rho_0 = rho_1
318
319       ! Affectation et calcul numérique
320       u0(1:n) = r0(1:n) - beta * u0(1:n)
321       ! Affectation et calcul numérique
322       u1(1:n) = r1(1:n) - beta * u1(1:n)
323       ! Appel de sous-programme modulaire
324       call apply_PA(ns, mesh, cfg, fields_old, dt, u1, u2)
325
326       ! Affectation et calcul numérique
327       gamma = global_dot_product(mesh, r_tilde, u2, n)
328       if (abs(gamma) < 1.0e-25_dp) exit
329       ! Affectation et calcul numérique
330       alpha = rho_0 / gamma
331
332       ! Affectation et calcul numérique
333       x(1:n)  = x(1:n)  + alpha * u0(1:n)
334       ! Affectation et calcul numérique
335       r0(1:n) = r0(1:n) - alpha * u1(1:n)
336       ! Affectation et calcul numérique
337       r1(1:n) = r1(1:n) - alpha * u2(1:n)
338       ! Appel de sous-programme modulaire
339       call apply_PA(ns, mesh, cfg, fields_old, dt, r1, r2)
340
341       ! Minimisation Quadratique MR-part
342       sigma11 = global_dot_product(mesh, r1, r1, n)
343       ! Affectation et calcul numérique
344       sigma12 = global_dot_product(mesh, r2, r1, n)
345       ! Affectation et calcul numérique
346       sigma22 = global_dot_product(mesh, r2, r2, n)
347       ! Affectation et calcul numérique
348       rhs1    = global_dot_product(mesh, r0, r1, n)
349       ! Affectation et calcul numérique
350       rhs2    = global_dot_product(mesh, r0, r2, n)
351
352       ! Affectation et calcul numérique
353       det = sigma11 * sigma22 - sigma12 * sigma12
354       ! Affectation et calcul numérique
355       s11_s22 = sigma11 * sigma22
356
357       ! Évaluation de la condition logique
358       if (s11_s22 < 1.0e-30_dp .or. abs(det) < 1.0e-10_dp * s11_s22) then
359         ! Affectation et calcul numérique
360         gamma2 = 0.0_dp
361         ! Affectation et calcul numérique
362         gamma1 = rhs1 / (sigma11 + 1.0e-30_dp)
363       ! Branche par défaut si condition non satisfaite
364       else
365         ! Affectation et calcul numérique
366         gamma1 = (rhs1 * sigma22 - rhs2 * sigma12) / det
367         ! Affectation et calcul numérique
368         gamma2 = (rhs2 * sigma11 - rhs1 * sigma12) / det
369       end if
370
371       ! Affectation et calcul numérique
372       x(1:n)  = x(1:n)  + gamma1 * r0(1:n) + gamma2 * r1(1:n)
373       ! Affectation et calcul numérique
374       r0(1:n) = r0(1:n) - (gamma1 * r1(1:n) + gamma2 * r2(1:n))
375       ! Affectation et calcul numérique
376       u0(1:n) = u0(1:n) - (gamma1 * u1(1:n) + gamma2 * u2(1:n))
377       ! Affectation et calcul numérique
378       omega = gamma2
379
380       ! Évaluation de la norme du résidu ou de l'erreur numérique
```

```
      r_norm = global_norm2(mesh, r0, n)
      ! Évaluation de la condition logique
      if (r_norm / r0_norm < tol .or. r_norm < 1.0e-13_dp) then
        ! Affectation et calcul numérique
        iters = iter
        ! Libération de la mémoire allouée
        deallocate(b_hat, r_tilde, r0, r1, r2, u0, u1, u2)
        ! Retour à la procédure appelante
        return
      end if
    end do

    ! Affectation et calcul numérique
    iters = iter
    ! Libération de la mémoire allouée
    deallocate(b_hat, r_tilde, r0, r1, r2, u0, u1, u2)
  end subroutine solve_bicgstab2

  !-------------------------------------------------------------------------------
  ! 2. Solveur BiCGSTAB standard - van der Vorst (1992)
  !-------------------------------------------------------------------------------
  subroutine solve_bicgstab(ns, mesh, cfg, fields_old, dt, b, x, iters)
    ! Déclaration de variable locale / paramètre de calcul
    type(navier_stokes_t), intent(in) :: ns
    ! Déclaration de variable locale / paramètre de calcul
    type(mesh_2d_t), intent(in)       :: mesh
    ! Déclaration de variable locale / paramètre de calcul
    type(config_t), intent(in)        :: cfg
    ! Déclaration de variable locale / paramètre de calcul
    type(flow_fields_t), intent(in)   :: fields_old
    ! Déclaration de variable locale / paramètre de calcul
    real(dp), intent(in)              :: dt
    ! Déclaration de variable locale / paramètre de calcul
    real(dp), intent(in)              :: b(:)
    ! Déclaration de variable locale / paramètre de calcul
    real(dp), intent(inout)           :: x(:)
    ! Déclaration de variable locale / paramètre de calcul
    integer, intent(out)              :: iters

    ! Déclaration de variable locale / paramètre de calcul
    integer :: n, max_iter, iter
    ! Déclaration de variable locale / paramètre de calcul
    real(dp) :: tol, r0_norm, r_norm, rho, rho_old, alpha, omega, beta, denom, s_norm, t_dot_t
    ! Déclaration de variable locale / paramètre de calcul
    real(dp), allocatable :: b_hat(:), r(:), r_tilde(:), p(:), v(:), s(:), t(:)

    ! Affectation et calcul numérique
    n = ns%n_owned_total
    ! Affectation et calcul numérique
    tol = cfg%tol
    ! Affectation et calcul numérique
    max_iter = cfg%max_iter
    ! Affectation et calcul numérique
    iters = max_iter

    ! Allocation dynamique de tableau contigu en mémoire
    allocate(b_hat(n), r(n), r_tilde(n), p(n), v(n), s(n), t(n))

    ! Appel de sous-programme modulaire
    call apply_preconditioner(ns, cfg, b, b_hat)
    ! Appel de sous-programme modulaire
    call apply_PA(ns, mesh, cfg, fields_old, dt, x, r)
    ! Affectation et calcul numérique
    r(1:n) = b_hat(1:n) - r(1:n)

    ! Évaluation de la norme du résidu ou de l'erreur numérique
    r0_norm = global_norm2(mesh, r, n)
    ! Évaluation de la condition logique
    if (r0_norm < 1.0e-12_dp) then
      ! Affectation et calcul numérique
      iters = 0
      ! Libération de la mémoire allouée
      deallocate(b_hat, r, r_tilde, p, v, s, t)
      ! Retour à la procédure appelante
      return
    end if

    ! Affectation et calcul numérique
    r_tilde(1:n) = r(1:n)
    ! Calcul de la masse volumique locale ou interpolée aux faces
    rho_old = 1.0_dp
    ! Affectation et calcul numérique
    alpha = 1.0_dp
    ! Affectation et calcul numérique
    omega = 1.0_dp

    ! Boucle d'itération numérique
    do iter = 1, max_iter
      ! Calcul de la masse volumique locale ou interpolée aux faces
      rho = global_dot_product(mesh, r_tilde, r, n)
      if (abs(rho) < 1.0e-30_dp) exit

```

```
      ! Évaluation de la condition logique
      if (iter == 1) then
        ! Actualisation de la pression statique au centre de la maille
        p(1:n) = r(1:n)
      ! Branche par défaut si condition non satisfaite
      else
        ! Affectation et calcul numérique
        beta = (rho / rho_old) * (alpha / omega)
        ! Actualisation de la pression statique au centre de la maille
        p(1:n) = r(1:n) + beta * (p(1:n) - omega * v(1:n))
      end if

      ! Appel de sous-programme modulaire
      call apply_PA(ns, mesh, cfg, fields_old, dt, p, v)
      ! Affectation et calcul numérique
      denom = global_dot_product(mesh, r_tilde, v, n)
      if (abs(denom) < 1.0e-30_dp) exit
      ! Affectation et calcul numérique
      alpha = rho / denom

      ! Affectation et calcul numérique
      s(1:n) = r(1:n) - alpha * v(1:n)
      ! Évaluation de la norme du résidu ou de l'erreur numérique
      s_norm = global_norm2(mesh, s, n)
      ! Évaluation de la condition logique
      if (s_norm / r0_norm < tol .or. s_norm < 1.0e-13_dp) then
        ! Affectation et calcul numérique
        x(1:n) = x(1:n) + alpha * p(1:n)
        ! Affectation et calcul numérique
        iters = iter
        ! Libération de la mémoire allouée
        deallocate(b_hat, r, r_tilde, p, v, s, t)
        ! Retour à la procédure appelante
        return
      end if

      ! Appel de sous-programme modulaire
      call apply_PA(ns, mesh, cfg, fields_old, dt, s, t)
      ! Affectation et calcul numérique
      t_dot_t = global_dot_product(mesh, t, t, n)
      ! Évaluation de la condition logique
      if (t_dot_t > 1.0e-30_dp) then
        ! Affectation et calcul numérique
        omega = global_dot_product(mesh, t, s, n) / t_dot_t
      ! Branche par défaut si condition non satisfaite
      else
        ! Affectation et calcul numérique
        omega = 0.0_dp
      end if

      ! Affectation et calcul numérique
      x(1:n) = x(1:n) + alpha * p(1:n) + omega * s(1:n)
      ! Affectation et calcul numérique
      r(1:n) = s(1:n) - omega * t(1:n)

      ! Calcul de la masse volumique locale ou interpolée aux faces
      rho_old = rho
      ! Évaluation de la norme du résidu ou de l'erreur numérique
      r_norm = global_norm2(mesh, r, n)
      ! Évaluation de la condition logique
      if (r_norm / r0_norm < tol .or. r_norm < 1.0e-13_dp) then
        ! Affectation et calcul numérique
        iters = iter
        ! Libération de la mémoire allouée
        deallocate(b_hat, r, r_tilde, p, v, s, t)
        ! Retour à la procédure appelante
        return
      end if
      if (abs(omega) < 1.0e-30_dp) exit
    end do

    ! Libération de la mémoire allouée
    deallocate(b_hat, r, r_tilde, p, v, s, t)
  end subroutine solve_bicgstab

  !-------------------------------------------------------------------------------
  ! 3. Solveur Gradient Conjugué (PCG) - Hestenes & Stiefel (1952)
  !-------------------------------------------------------------------------------
  subroutine solve_cg(ns, mesh, cfg, fields_old, dt, b, x, iters)
    ! Déclaration de variable locale / paramètre de calcul
    type(navier_stokes_t), intent(in) :: ns
    ! Déclaration de variable locale / paramètre de calcul
    type(mesh_2d_t), intent(in)       :: mesh
    ! Déclaration de variable locale / paramètre de calcul
    type(config_t), intent(in)        :: cfg
    ! Déclaration de variable locale / paramètre de calcul
    type(flow_fields_t), intent(in)   :: fields_old
    ! Déclaration de variable locale / paramètre de calcul
    real(dp), intent(in)              :: dt
    ! Déclaration de variable locale / paramètre de calcul
    real(dp), intent(in)              :: b(:)
    ! Déclaration de variable locale / paramètre de calcul
```

```
    real(dp), intent(inout)            :: x(:)
    ! Déclaration de variable locale / paramètre de calcul
    integer, intent(out)               :: iters

    ! Déclaration de variable locale / paramètre de calcul
    integer :: n, max_iter, iter
    ! Déclaration de variable locale / paramètre de calcul
    real(dp) :: tol, r0_norm, r_norm, rz_old, rz_new, pAp, alpha, beta
    ! Déclaration de variable locale / paramètre de calcul
    real(dp), allocatable :: Ax(:), r(:), z(:), p(:), Ap(:)

    ! Affectation et calcul numérique
    n = ns%n_owned_total
    ! Affectation et calcul numérique
    tol = cfg%tol
    ! Affectation et calcul numérique
    max_iter = cfg%max_iter
    ! Affectation et calcul numérique
    iters = max_iter

    ! Allocation dynamique de tableau contigu en mémoire
    allocate(Ax(n), r(n), z(n), p(n), Ap(n))

    ! Appel de sous-programme modulaire
    call ns%apply_system_matrix(mesh, cfg, fields_old, dt, x, Ax)
    ! Affectation et calcul numérique
    r(1:n) = b(1:n) - Ax(1:n)

    ! Évaluation de la norme du résidu ou de l'erreur numérique
    r0_norm = global_norm2(mesh, r, n)
    ! Évaluation de la condition logique
    if (r0_norm < 1.0e-12_dp) then
      ! Affectation et calcul numérique
      iters = 0
      ! Libération de la mémoire allouée
      deallocate(Ax, r, z, p, Ap)
      ! Retour à la procédure appelante
      return
    end if

    ! Appel de sous-programme modulaire
    call apply_preconditioner(ns, cfg, r, z)
    ! Actualisation de la pression statique au centre de la maille
    p(1:n) = z(1:n)
    ! Affectation et calcul numérique
    rz_old = global_dot_product(mesh, r, z, n)

    ! Boucle d'itération numérique
    do iter = 1, max_iter
      ! Appel de sous-programme modulaire
      call ns%apply_system_matrix(mesh, cfg, fields_old, dt, p, Ap)
      ! Affectation et calcul numérique
      pAp = global_dot_product(mesh, p, Ap, n)
      if (abs(pAp) < 1.0e-30_dp) exit

      ! Affectation et calcul numérique
      alpha = rz_old / pAp
      ! Affectation et calcul numérique
      x(1:n) = x(1:n) + alpha * p(1:n)
      ! Affectation et calcul numérique
      r(1:n) = r(1:n) - alpha * Ap(1:n)

      ! Évaluation de la norme du résidu ou de l'erreur numérique
      r_norm = global_norm2(mesh, r, n)
      ! Évaluation de la condition logique
      if (r_norm / r0_norm < tol .or. r_norm < 1.0e-13_dp) then
        ! Affectation et calcul numérique
        iters = iter
        ! Libération de la mémoire allouée
        deallocate(Ax, r, z, p, Ap)
        ! Retour à la procédure appelante
        return
      end if

      ! Appel de sous-programme modulaire
      call apply_preconditioner(ns, cfg, r, z)
      ! Affectation et calcul numérique
      rz_new = global_dot_product(mesh, r, z, n)
      ! Affectation et calcul numérique
      beta = rz_new / rz_old
      ! Affectation et calcul numérique
      rz_old = rz_new

      ! Actualisation de la pression statique au centre de la maille
      p(1:n) = z(1:n) + beta * p(1:n)
    end do

    ! Libération de la mémoire allouée
    deallocate(Ax, r, z, p, Ap)
  end subroutine solve_cg

  !-------------------------------------------------------------------------------
```

```
  ! 4. Solveur GMRES(m) redémarré - Saad & Schultz (1986)
  !-------------------------------------------------------------------------------
  subroutine solve_gmres(ns, mesh, cfg, fields_old, dt, b, x, iters)
    ! Déclaration de variable locale / paramètre de calcul
    type(navier_stokes_t), intent(in) :: ns
    ! Déclaration de variable locale / paramètre de calcul
    type(mesh_2d_t), intent(in)       :: mesh
    ! Déclaration de variable locale / paramètre de calcul
    type(config_t), intent(in)        :: cfg
    ! Déclaration de variable locale / paramètre de calcul
    type(flow_fields_t), intent(in)   :: fields_old
    ! Déclaration de variable locale / paramètre de calcul
    real(dp), intent(in)              :: dt
    ! Déclaration de variable locale / paramètre de calcul
    real(dp), intent(in)              :: b(:)
    ! Déclaration de variable locale / paramètre de calcul
    real(dp), intent(inout)           :: x(:)
    ! Déclaration de variable locale / paramètre de calcul
    integer, intent(out)              :: iters

    ! Déclaration de variable locale / paramètre de calcul
    integer, parameter :: m = 30
    ! Déclaration de variable locale / paramètre de calcul
    integer :: n, max_iter, total_iters, i, j, k
    ! Déclaration de variable locale / paramètre de calcul
    real(dp) :: tol, r0_norm, beta_val, r_norm, h_val, temp, cs, sn
    ! Déclaration de variable locale / paramètre de calcul
    real(dp), allocatable :: b_hat(:), r(:), w(:), g(:), y(:)
    ! Déclaration de variable locale / paramètre de calcul
    real(dp), allocatable :: V(:, :), H(:, :), c_rot(:), s_rot(:)

    ! Affectation et calcul numérique
    n = ns%n_owned_total
    ! Affectation et calcul numérique
    tol = cfg%tol
    ! Affectation et calcul numérique
    max_iter = cfg%max_iter
    ! Affectation et calcul numérique
    iters = max_iter

    ! Allocation dynamique de tableau contigu en mémoire
    allocate(b_hat(n), r(n), w(n), g(m + 1), y(m))
    ! Allocation dynamique de tableau contigu en mémoire
    allocate(V(n, m + 1), H(m + 1, m), c_rot(m), s_rot(m))

    ! Appel de sous-programme modulaire
    call apply_preconditioner(ns, cfg, b, b_hat)

    ! Affectation et calcul numérique
    total_iters = 0
    ! Évaluation de la norme du résidu ou de l'erreur numérique
    r0_norm = 1.0_dp

    ! Boucle d'itération numérique
    do while (total_iters < max_iter)
      ! Appel de sous-programme modulaire
      call apply_PA(ns, mesh, cfg, fields_old, dt, x, r)
      ! Affectation et calcul numérique
      r(1:n) = b_hat(1:n) - r(1:n)

      ! Affectation et calcul numérique
      beta_val = global_norm2(mesh, r, n)
      ! Évaluation de la condition logique
      if (total_iters == 0) then
        ! Évaluation de la condition logique
        if (beta_val > 1.0e-14_dp) then
          ! Évaluation de la norme du résidu ou de l'erreur numérique
          r0_norm = beta_val
        ! Branche par défaut si condition non satisfaite
        else
          ! Évaluation de la norme du résidu ou de l'erreur numérique
          r0_norm = 1.0_dp
        end if
      end if

      ! Évaluation de la condition logique
      if (beta_val / r0_norm < tol .or. beta_val < 1.0e-13_dp) then
        ! Affectation et calcul numérique
        iters = total_iters
        ! Libération de la mémoire allouée
        deallocate(b_hat, r, w, g, y, V, H, c_rot, s_rot)
        ! Retour à la procédure appelante
        return
      end if

      ! Affectation et calcul numérique
      V(1:n, 1) = r(1:n) / beta_val
      ! Affectation et calcul numérique
      g = 0.0_dp
      ! Affectation et calcul numérique
      g(1) = beta_val
      ! Affectation et calcul numérique
```

```
749       H = 0.0_dp
750
751       ! Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
752       do j = 1, min(m, max_iter - total_iters)
753         ! Affectation et calcul numérique
754         total_iters = total_iters + 1
755
756         ! Appel de sous-programme modulaire
757         call apply_PA(ns, mesh, cfg, fields_old, dt, V(1:n, j), w)
758
759         ! Orthogonalisation Modified Gram-Schmidt
760         do i = 1, j
761           ! Affectation et calcul numérique
762           H(i, j) = global_dot_product(mesh, w, V(1:n, i), n)
763           ! Affectation et calcul numérique
764           w(1:n) = w(1:n) - H(i, j) * V(1:n, i)
765         end do
766
767         ! Affectation et calcul numérique
768         H(j + 1, j) = global_norm2(mesh, w, n)
769         ! Évaluation de la condition logique
770         if (H(j + 1, j) > 1.0e-30_dp) then
771           ! Affectation et calcul numérique
772           V(1:n, j + 1) = w(1:n) / H(j + 1, j)
773         end if
774
775         ! Application des rotations de Givens précédentes
776         do i = 1, j - 1
777           ! Affectation et calcul numérique
778           temp = c_rot(i) * H(i, j) + s_rot(i) * H(i + 1, j)
779           ! Affectation et calcul numérique
780           H(i + 1, j) = -s_rot(i) * H(i, j) + c_rot(i) * H(i + 1, j)
781           ! Affectation et calcul numérique
782           H(i, j) = temp
783         end do
784
785         ! Calcul de la nouvelle rotation de Givens
786         if (abs(H(j + 1, j)) < 1.0e-30_dp) then
787           ! Affectation et calcul numérique
788           c_rot(j) = 1.0_dp
789           ! Affectation et calcul numérique
790           s_rot(j) = 0.0_dp
791         ! Branche conditionnelle alternative
792         else if (abs(H(j, j)) > abs(H(j + 1, j))) then
793           ! Affectation et calcul numérique
794           temp = H(j + 1, j) / H(j, j)
795           ! Affectation et calcul numérique
796           c_rot(j) = 1.0_dp / sqrt(1.0_dp + temp * temp)
797           ! Affectation et calcul numérique
798           s_rot(j) = temp * c_rot(j)
799         ! Branche par défaut si condition non satisfaite
800         else
801           ! Affectation et calcul numérique
802           temp = H(j, j) / H(j + 1, j)
803           ! Affectation et calcul numérique
804           s_rot(j) = 1.0_dp / sqrt(1.0_dp + temp * temp)
805           ! Affectation et calcul numérique
806           c_rot(j) = temp * s_rot(j)
807         end if
808
809         ! Affectation et calcul numérique
810         H(j, j) = c_rot(j) * H(j, j) + s_rot(j) * H(j + 1, j)
811         ! Affectation et calcul numérique
812         H(j + 1, j) = 0.0_dp
813
814         ! Affectation et calcul numérique
815         g(j + 1) = -s_rot(j) * g(j)
816         ! Affectation et calcul numérique
817         g(j)     =  c_rot(j) * g(j)
818
819         ! Évaluation de la norme du résidu ou de l'erreur numérique
820         r_norm = abs(g(j + 1))
821         ! Évaluation de la condition logique
822         if (r_norm / r0_norm < tol .or. r_norm < 1.0e-13_dp .or. j == m) then
823           ! Résolution du système triangulaire supérieur H * y = g
824           do k = j, 1, -1
825             ! Affectation et calcul numérique
826             y(k) = g(k)
827             ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
828             do i = k + 1, j
829               ! Affectation et calcul numérique
830               y(k) = y(k) - H(k, i) * y(i)
831             end do
832             ! Affectation et calcul numérique
833             if (abs(H(k, k)) > 1.0e-30_dp) y(k) = y(k) / H(k, k)
834           end do
835
836           ! Mise à jour de la solution x = x + V * y
837           do i = 1, j
838             ! Affectation et calcul numérique
839             x(1:n) = x(1:n) + y(i) * V(1:n, i)
840           end do
```

```
841             exit
842           end if
843         end do
844       end do
845
846       ! Affectation et calcul numérique
847       iters = total_iters
848       ! Libération de la mémoire allouée
849       deallocate(b_hat, r, w, g, y, V, H, c_rot, s_rot)
850     end subroutine solve_gmres
851
852     !-------------------------------------------------------------------------------
853     ! Aiguilleur principal de solveur linéaire
854     !-------------------------------------------------------------------------------
855     subroutine solve_linear_system(ns, mesh, cfg, fields_old, dt, b, x, iters)
856       ! Déclaration de variable locale / paramètre de calcul
857       type(navier_stokes_t), intent(in) :: ns
858       ! Déclaration de variable locale / paramètre de calcul
859       type(mesh_2d_t), intent(in)        :: mesh
860       ! Déclaration de variable locale / paramètre de calcul
861       type(config_t), intent(in)         :: cfg
862       ! Déclaration de variable locale / paramètre de calcul
863       type(flow_fields_t), intent(in)    :: fields_old
864       ! Déclaration de variable locale / paramètre de calcul
865       real(dp), intent(in)               :: dt
866       ! Déclaration de variable locale / paramètre de calcul
867       real(dp), intent(in)               :: b(:)
868       ! Déclaration de variable locale / paramètre de calcul
869       real(dp), intent(inout)            :: x(:)
870       ! Déclaration de variable locale / paramètre de calcul
871       integer, intent(out)               :: iters
872
873       select case (cfg%linear_solver)
874       case (LINEAR_BICGSTAB2)
875         ! Appel de sous-programme modulaire
876         call solve_bicgstab2(ns, mesh, cfg, fields_old, dt, b, x, iters)
877       case (LINEAR_BICGSTAB)
878         ! Appel de sous-programme modulaire
879         call solve_bicgstab(ns, mesh, cfg, fields_old, dt, b, x, iters)
880       case (LINEAR_CG)
881         ! Appel de sous-programme modulaire
882         call solve_cg(ns, mesh, cfg, fields_old, dt, b, x, iters)
883       case (LINEAR_GMRES)
884         ! Appel de sous-programme modulaire
885         call solve_gmres(ns, mesh, cfg, fields_old, dt, b, x, iters)
886       case default
887         ! Appel de sous-programme modulaire
888         call solve_bicgstab2(ns, mesh, cfg, fields_old, dt, b, x, iters)
889       end select
890     end subroutine solve_linear_system
891
892   end module module_linear_solvers
```

Chapitre 19

Solveur Monolithique de Navier-Stokes par Lagrangien Augmenté en Fortran 2018 (module$_navier_stokes.f90$)

19.1 Architecture des 1540 Lignes du Module `module_navier_stokes.f90`

Ce module constitue le noyau physique et mathématique de `COVOF_2D-FORTRAN`. Il regroupe :

1. **Mise à jour des propriétés physiques diphasiques (`update_fluid_properties`)** : calcule en chaque maille (i, j) la masse volumique $\rho_{i,j} = C_{i,j}\rho_1 + (1 - C_{i,j})\rho_2$ et la viscosité dynamique $\mu_{i,j}$ (y compris la loi exponentielle spatiale du Cas 3.9).
2. **Application des conditions aux limites hydrodynamiques (`apply_velocity_bc`)** : impose les conditions `BC_WALL` (adhérence par cellules fantômes antisymétriques), `BC_SLIP` (glissement libre par cellules fantômes symétriques), `BC_INLET`, `BC_OUTLET`, `BC_PERIODIC` et `BC_JET_INLET` (injection centrale de buse V_0 sur $|x - 0.5| \leq D/2$ et évacuation compensatoire $v_{\text{out}} = |V_0|D/(L_x - D)$ garantissant $\oint_{\partial\Omega} \mathbf{u} \cdot \mathbf{n}\, d\Gamma = 0$).
3. **Calcul de la force capillaire CSF Balanced-Force (`compute_surface_tension_force`)** : lisse la fonction couleur C sur un voisinage 3×3, calcule les normales unitaires aux sommets, en déduit la courbure centrée $\kappa_{i,j} = -\nabla_h \cdot \mathbf{n}$ et projette la force capillaire $\sigma\kappa_{\text{face}}\nabla_h C$ directement sur les faces MAC u et v.
4. **Assemblage et résolution du système monolithique du Lagrangien Augmenté (`solve_momentum_augmented_l`** assemble la matrice creuse $\mathbf{A}_{\text{aug}}$ à 9 bandes incluant l'inertie $\rho/\Delta t$, la convection décentrée, le tenseur visqueux complet à moyenne harmonique nodale, la pénalisation de Brinkman χ_s/η et le couplage croisé $-r_{al}\nabla_h(\nabla_h \cdot \mathbf{u})$, puis exécute les sous-itérations d'Uzawa $p^{k+1} = p^k - r_{al}\nabla_h \cdot \mathbf{u}^{k+1}$.
5. **Résolution de l'équation de l'énergie thermique (`solve_energy_equation`)** : intègre implicitement la température $T_{i,j}^{n+1}$ et applique les conditions aux limites thermiques isothermes et adiabatiques (Cas 6.1 de Vahl Davis).

19.2 Répertoire des Types Dérivés, Sous-Routines et Fonctions de `src/module_navier_stokes.f90`

Ligne	Nature	Identifiant Fortran 2018	Interface / Arguments
12	module	`module_navier_stokes`	Unité modulaire Fortran 2018
25	type dérivé	`navier_stokes_t`	Structure de données Fortran 2018
67	function	`is_u_face_in_obstacle`	Arguments : (mesh, cfg, i, j)
94	function	`is_v_face_in_obstacle`	Arguments : (mesh, cfg, i, j)
121	subroutine	`ns_init`	Arguments : (this, mesh, cfg)
180	subroutine	`ns_pack_vector`	Arguments : (this, mesh, cfg, fields, vec)
249	subroutine	`ns_unpack_vector`	Arguments : (this, mesh, cfg, vec, fields, homogeneous)
329	subroutine	`ns_apply_system_matrix`	Arguments : (this, mesh, cfg, fields_old, dt, x, y)
671	subroutine	`ns_assemble_rhs`	Arguments : (this, mesh, cfg, fields, dt, rhs)
943	subroutine	`ns_update_pressure_al`	Arguments : (this, mesh, cfg, fields)
982	subroutine	`ns_assemble_preconditioner_blocks`	Arguments : (this, mesh, cfg, fields, dt)
1108	subroutine	`ns_solve_energy`	Arguments : (this, mesh, cfg, fields, dt)
1489	function	`ns_compute_mean_nusselt_hot_wall`	Arguments : (this, mesh, cfg, fields)

19.3 Code Source Intégral Annoté : `src/module_navier_stokes.f90`

Le listing ci-dessous présente l'intégralité des 1551 lignes du fichier `src/module_navier_stokes.f90` de l'implémentation Fortran 2018 :

```
!===============================================================================
! Module: module_navier_stokes
! Description: Solveur Navier-Stokes 2D incompressible diphasique et thermique
!              selon l'architecture Fortran 2018 de COVOF 2D.
!              - Discrétisation spatiale MAC décalée (Harlow & Welch 1965)
!              - Couplage Vitesse-Pression par Lagrangien Augmenté (Fortin & Glowinski)
!              - Algorithme d'Uzawa pour la pression
!              - Viscosités harmonique aux nœuds et arithmétique aux mailles
!              - Équation de l'énergie thermique couplée (Boussinesq)
!              - Pénalisation volumique d'obstacles (Brinkman)
!===============================================================================
module module_navier_stokes
  use module_precision, only: dp, ip
  use module_constants
  use module_config, only: config_t
  use module_mesh
  use module_fields
  use module_mpi_comm
  implicit none
  private

  public :: navier_stokes_t
  public :: is_u_face_in_obstacle, is_v_face_in_obstacle

  type :: navier_stokes_t
    ! Déclaration de variable locale / paramètre de calcul
    integer :: n_owned_u = 0
    ! Déclaration de variable locale / paramètre de calcul
    integer :: n_owned_v = 0
    ! Déclaration de variable locale / paramètre de calcul
    integer :: n_owned_p = 0
    ! Déclaration de variable locale / paramètre de calcul
    integer :: n_owned_total = 0

    ! Diagonales précalculées pour le préconditionneur Jacobi
    real(dp), allocatable :: diag_u(:)
    ! Déclaration de variable locale / paramètre de calcul
    real(dp), allocatable :: diag_v(:)
    ! Déclaration de variable locale / paramètre de calcul
    real(dp), allocatable :: diag_p(:)
  contains
    ! Affectation et calcul numérique
    procedure :: init => ns_init
    ! Affectation et calcul numérique
    procedure :: pack_vector => ns_pack_vector
    ! Affectation et calcul numérique
    procedure :: unpack_vector => ns_unpack_vector
    ! Affectation et calcul numérique
    procedure :: apply_system_matrix => ns_apply_system_matrix
    ! Affectation et calcul numérique
    procedure :: assemble_rhs => ns_assemble_rhs
    ! Évaluation de la norme du résidu ou de l'erreur numérique
    procedure :: update_pressure_al => ns_update_pressure_al
    ! Affectation et calcul numérique
    procedure :: assemble_preconditioner_blocks => ns_assemble_preconditioner_blocks
    ! Affectation et calcul numérique
    procedure :: solve_energy => ns_solve_energy
    ! Affectation et calcul numérique
    procedure :: compute_mean_nusselt_hot_wall => ns_compute_mean_nusselt_hot_wall
  end type navier_stokes_t

contains

  !-------------------------------------------------------------------------------
  ! Détection si une face de vitesse normale u(i, j) est dans un obstacle solide
  !-------------------------------------------------------------------------------
  pure function is_u_face_in_obstacle(mesh, cfg, i, j) result(in_obs)
    ! Déclaration de variable locale / paramètre de calcul
    type(mesh_2d_t), intent(in)   :: mesh
    ! Déclaration de variable locale / paramètre de calcul
    type(config_t), intent(in)    :: cfg
    ! Déclaration de variable locale / paramètre de calcul
    integer, intent(in)           :: i, j
    ! Déclaration de variable locale / paramètre de calcul
    logical                       :: in_obs
    ! Déclaration de variable locale / paramètre de calcul
    real(dp) :: x_f, y_c

    ! Affectation et calcul numérique
    in_obs = .false.
    if (.not. cfg%obs_enabled) return
    ! Affectation et calcul numérique
    x_f = x_face(mesh, i)
    ! Affectation et calcul numérique
    y_c = y_center(mesh, j)
    ! Affectation et calcul numérique
```

```
    in_obs = is_in_obstacle(x_f, y_c, cfg%obs_type, cfg%obs_x0, cfg%obs_y0, &
                            cfg%obs_x1, cfg%obs_y1, cfg%obs_xc, cfg%obs_yc, cfg%obs_radius)
  end function is_u_face_in_obstacle

  !-------------------------------------------------------------------------------
  ! Détection si une face de vitesse normale v(i, j) est dans un obstacle solide
  !-------------------------------------------------------------------------------
  pure function is_v_face_in_obstacle(mesh, cfg, i, j) result(in_obs)
    ! Déclaration de variable locale / paramètre de calcul
    type(mesh_2d_t), intent(in)   :: mesh
    ! Déclaration de variable locale / paramètre de calcul
    type(config_t), intent(in)    :: cfg
    ! Déclaration de variable locale / paramètre de calcul
    integer, intent(in)           :: i, j
    ! Déclaration de variable locale / paramètre de calcul
    logical                       :: in_obs
    ! Déclaration de variable locale / paramètre de calcul
    real(dp) :: x_c, y_f

    ! Affectation et calcul numérique
    in_obs = .false.
    if (.not. cfg%obs_enabled) return
    ! Affectation et calcul numérique
    x_c = x_center(mesh, i)
    ! Affectation et calcul numérique
    y_f = y_face(mesh, j)
    ! Affectation et calcul numérique
    in_obs = is_in_obstacle(x_c, y_f, cfg%obs_type, cfg%obs_x0, cfg%obs_y0, &
                            cfg%obs_x1, cfg%obs_y1, cfg%obs_xc, cfg%obs_yc, cfg%obs_radius)
  end function is_v_face_in_obstacle

  !-------------------------------------------------------------------------------
  ! Initialisation du solveur Navier-Stokes et comptage des DDLs possédés
  !-------------------------------------------------------------------------------
  subroutine ns_init(this, mesh, cfg)
    ! Déclaration de variable locale / paramètre de calcul
    class(navier_stokes_t), intent(inout) :: this
    ! Déclaration de variable locale / paramètre de calcul
    type(mesh_2d_t), intent(in)           :: mesh
    ! Déclaration de variable locale / paramètre de calcul
    type(config_t), intent(in)            :: cfg
    ! Déclaration de variable locale / paramètre de calcul
    integer :: u_nx, v_ny

    ! Affectation et calcul numérique
    u_nx = mesh%nx_local
    ! Affectation et calcul numérique
    if (mesh%is_boundary_right) u_nx = u_nx + 1
    ! Affectation et calcul numérique
    this%n_owned_u = u_nx * mesh%ny_local

    ! Affectation et calcul numérique
    v_ny = mesh%ny_local
    ! Affectation et calcul numérique
    if (mesh%is_boundary_top) v_ny = v_ny + 1
    ! Affectation et calcul numérique
    this%n_owned_v = mesh%nx_local * v_ny

    ! Affectation et calcul numérique
    this%n_owned_p = mesh%nx_local * mesh%ny_local

    ! Évaluation de la condition logique
    if (cfg%solver_type == SOLVER_AUGMENTED_LAGRANGIAN) then
      ! Affectation et calcul numérique
      this%n_owned_total = this%n_owned_u + this%n_owned_v
    ! Branche par défaut si condition non satisfaite
    else
      ! Affectation et calcul numérique
      this%n_owned_total = this%n_owned_u + this%n_owned_v + this%n_owned_p
    end if

    if (allocated(this%diag_u)) deallocate(this%diag_u)
    if (allocated(this%diag_v)) deallocate(this%diag_v)
    if (allocated(this%diag_p)) deallocate(this%diag_p)

    ! Allocation dynamique de tableau contigu en mémoire
    allocate(this%diag_u(this%n_owned_u))
    ! Allocation dynamique de tableau contigu en mémoire
    allocate(this%diag_v(this%n_owned_v))
    ! Allocation dynamique de tableau contigu en mémoire
    allocate(this%diag_p(this%n_owned_p))

    ! Affectation et calcul numérique
    this%diag_u = 1.0_dp
    ! Affectation et calcul numérique
    this%diag_v = 1.0_dp
    ! Affectation et calcul numérique
    this%diag_p = 1.0_dp
  end subroutine ns_init

  !-------------------------------------------------------------------------------
  ! Empaquette les champs physiques 2D dans un vecteur 1D contigu
```

```
  !------------------------------------------------------------------------------
  subroutine ns_pack_vector(this, mesh, cfg, fields, vec)
    ! Déclaration de variable locale / paramètre de calcul
    class(navier_stokes_t), intent(in) :: this
    ! Déclaration de variable locale / paramètre de calcul
    type(mesh_2d_t), intent(in)        :: mesh
    ! Déclaration de variable locale / paramètre de calcul
    type(config_t), intent(in)         :: cfg
    ! Déclaration de variable locale / paramètre de calcul
    type(flow_fields_t), intent(in)    :: fields
    ! Déclaration de variable locale / paramètre de calcul
    real(dp), intent(out)              :: vec(:)

    ! Déclaration de variable locale / paramètre de calcul
    integer :: i, j, dof, u_nx, v_ny, v_offset, p_offset

    ! Affectation et calcul numérique
    u_nx = mesh%nx_local
    ! Affectation et calcul numérique
    if (mesh%is_boundary_right) u_nx = u_nx + 1

    ! 1. Pack u
    do j = 1, mesh%ny_local
      ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
      do i = 1, u_nx
        ! Affectation et calcul numérique
        dof = (j - 1) * u_nx + i
        ! Affectation et calcul numérique
        vec(dof) = fields%u(i, j)
      end do
    end do

    ! 2. Pack v
    v_offset = this%n_owned_u
    ! Affectation et calcul numérique
    v_ny = mesh%ny_local
    ! Affectation et calcul numérique
    if (mesh%is_boundary_top) v_ny = v_ny + 1

    ! Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
    do j = 1, v_ny
      ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
      do i = 1, mesh%nx_local
        ! Affectation et calcul numérique
        dof = v_offset + (j - 1) * mesh%nx_local + i
        ! Affectation et calcul numérique
        vec(dof) = fields%v(i, j)
      end do
    end do

    ! 3. Pack p (si monolithique)
    if (cfg%solver_type == SOLVER_MONOLITHIC) then
      ! Affectation et calcul numérique
      p_offset = this%n_owned_u + this%n_owned_v
      ! Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
      do j = 1, mesh%ny_local
        ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
        do i = 1, mesh%nx_local
          ! Affectation et calcul numérique
          dof = p_offset + (j - 1) * mesh%nx_local + i
          ! Affectation et calcul numérique
          vec(dof) = fields%p(i, j)
        end do
      end do
    end if
  end subroutine ns_pack_vector

  !------------------------------------------------------------------------------
  ! Dépaquette le vecteur 1D dans les champs 2D et synchronise les halos MPI
  !------------------------------------------------------------------------------
  subroutine ns_unpack_vector(this, mesh, cfg, vec, fields, homogeneous)
    ! Déclaration de variable locale / paramètre de calcul
    class(navier_stokes_t), intent(in) :: this
    ! Déclaration de variable locale / paramètre de calcul
    type(mesh_2d_t), intent(in)        :: mesh
    ! Déclaration de variable locale / paramètre de calcul
    type(config_t), intent(in)         :: cfg
    ! Déclaration de variable locale / paramètre de calcul
    real(dp), intent(in)               :: vec(:)
    ! Déclaration de variable locale / paramètre de calcul
    type(flow_fields_t), intent(inout) :: fields
    ! Déclaration de variable locale / paramètre de calcul
    logical, intent(in)                :: homogeneous

    ! Déclaration de variable locale / paramètre de calcul
    integer :: i, j, dof, u_nx, v_ny, v_offset, p_offset

    ! Affectation et calcul numérique
    u_nx = mesh%nx_local
    ! Affectation et calcul numérique
    if (mesh%is_boundary_right) u_nx = u_nx + 1

```

```
    ! 1. Unpack u
    do j = 1, mesh%ny_local
      ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
      do i = 1, u_nx
        ! Affectation et calcul numérique
        dof = (j - 1) * u_nx + i
        ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
        fields%u(i, j) = vec(dof)
      end do
    end do

    ! 2. Unpack v
    v_offset = this%n_owned_u
    ! Affectation et calcul numérique
    v_ny = mesh%ny_local
    ! Affectation et calcul numérique
    if (mesh%is_boundary_top) v_ny = v_ny + 1

    ! Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
    do j = 1, v_ny
      ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
      do i = 1, mesh%nx_local
        ! Affectation et calcul numérique
        dof = v_offset + (j - 1) * mesh%nx_local + i
        ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
        fields%v(i, j) = vec(dof)
      end do
    end do

    ! 3. Unpack p (si monolithique)
    if (cfg%solver_type == SOLVER_MONOLITHIC) then
      ! Affectation et calcul numérique
      p_offset = this%n_owned_u + this%n_owned_v
      ! Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
      do j = 1, mesh%ny_local
        ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
        do i = 1, mesh%nx_local
          ! Affectation et calcul numérique
          dof = p_offset + (j - 1) * mesh%nx_local + i
          ! Actualisation de la pression statique au centre de la maille
          fields%p(i, j) = vec(dof)
        end do
      end do
      ! Appel de sous-programme modulaire
      call sync_cell_scalar(mesh, fields%p)
    end if

    ! Synchronisation des mailles fantômes et conditions aux limites
    call sync_u(mesh, fields%u)
    ! Appel de sous-programme modulaire
    call sync_v(mesh, fields%v)
    ! Appel de sous-programme modulaire
    call apply_boundary_conditions(mesh, cfg, fields, homogeneous)
  end subroutine ns_unpack_vector

  !-------------------------------------------------------------------------------
  ! Application de l'opérateur matriciel complet y = A * x
  !-------------------------------------------------------------------------------
  subroutine ns_apply_system_matrix(this, mesh, cfg, fields_old, dt, x, y)
    ! Déclaration de variable locale / paramètre de calcul
    class(navier_stokes_t), intent(in) :: this
    ! Déclaration de variable locale / paramètre de calcul
    type(mesh_2d_t), intent(in)        :: mesh
    ! Déclaration de variable locale / paramètre de calcul
    type(config_t), intent(in)         :: cfg
    ! Déclaration de variable locale / paramètre de calcul
    type(flow_fields_t), intent(in)    :: fields_old
    ! Déclaration de variable locale / paramètre de calcul
    real(dp), intent(in)               :: dt
    ! Déclaration de variable locale / paramètre de calcul
    real(dp), intent(in)               :: x(:)
    ! Déclaration de variable locale / paramètre de calcul
    real(dp), intent(out)              :: y(:)

    ! Déclaration de variable locale / paramètre de calcul
    type(flow_fields_t) :: tf
    ! Déclaration de variable locale / paramètre de calcul
    real(dp) :: dx, dy, dvol
    ! Déclaration de variable locale / paramètre de calcul
    integer :: i, j, dof, u_nx, v_ny, v_offset, p_offset
    ! Déclaration de variable locale / paramètre de calcul
    real(dp) :: div_cell(0:mesh%nx_local+1, 0:mesh%ny_local+1)
    ! Déclaration de variable locale / paramètre de calcul
    real(dp) :: rho_u, rho_v, diag_bc, val
    ! Déclaration de variable locale / paramètre de calcul
    real(dp) :: u_n, u_n_E, u_n_W, v_n_N, v_n_S
    ! Déclaration de variable locale / paramètre de calcul
    real(dp) :: conv_E, conv_W, conv_N, conv_S
    ! Déclaration de variable locale / paramètre de calcul
    real(dp) :: mu_cell_E, mu_cell_W, diff_elong_E, diff_elong_W
    ! Déclaration de variable locale / paramètre de calcul
    real(dp) :: mu_node_N, mu_node_S, shear_N, shear_S
```

```
    ! Déclaration de variable locale / paramètre de calcul
    real(dp) :: grad_div_x, grad_div_y, grad_p
    ! Déclaration de variable locale / paramètre de calcul
    real(dp) :: v_n, v_n_S_val
    ! Déclaration de variable locale / paramètre de calcul
    real(dp) :: mu_cell_N, mu_cell_S, diff_elong_N, diff_elong_S
    ! Déclaration de variable locale / paramètre de calcul
    real(dp) :: mu_node_E, mu_node_W, shear_E, shear_W
    ! Déclaration de variable locale / paramètre de calcul
    real(dp) :: div_val, diag_p_term
    ! Déclaration de variable locale / paramètre de calcul
    logical  :: is_wall_bot, is_wall_top, in_obs

    ! Calcul de la métrique spatiale élémentaire
    dx = mesh%dx
    ! Calcul de la métrique spatiale élémentaire
    dy = mesh%dy
    ! Affectation et calcul numérique
    dvol = dx * dy

    ! Affectation et calcul numérique
    u_nx = mesh%nx_local
    ! Affectation et calcul numérique
    if (mesh%is_boundary_right) u_nx = u_nx + 1

    ! Affectation et calcul numérique
    v_ny = mesh%ny_local
    ! Affectation et calcul numérique
    if (mesh%is_boundary_top) v_ny = v_ny + 1

    ! Appel de sous-programme modulaire
    call tf%allocate(mesh)
    ! Appel de sous-programme modulaire
    call this%unpack_vector(mesh, cfg, x, tf, .true.)

    ! Précalcul de la divergence sur chaque cellule locale + halo (dans les bornes allouées 0:nx+1, 0:ny+1)
    div_cell = 0.0_dp
    do j = 0, mesh%ny_local
      ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
      do i = 0, mesh%nx_local
        ! Affectation et calcul numérique
        div_cell(i, j) = (tf%u(i + 1, j) - tf%u(i, j)) / dx + &
                         (tf%v(i, j + 1) - tf%v(i, j)) / dy
      end do
    end do

    ! ==========================================================================
    ! 1. Équation de Quantité de Mouvement en u (Faces verticales)
    ! ==========================================================================
    do j = 1, mesh%ny_local
      ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
      do i = 1, u_nx
        ! Affectation et calcul numérique
        dof = (j - 1) * u_nx + i
        ! Calcul de la masse volumique locale ou interpolée aux faces
        rho_u = fields_old%rho_u(i, j)
        ! Affectation et calcul numérique
        diag_bc = (rho_u / dt) * dvol

        ! Condition aux limites physique sur les parois gauche / droite
        if (i == 1 .and. mesh%is_boundary_left) then
          ! Évaluation de la condition logique
          if (cfg%bc_left%btype == BC_NEUMANN) then
            ! Affectation et calcul numérique
            y(dof) = diag_bc * (tf%u(1, j) - tf%u(2, j))
          ! Branche par défaut si condition non satisfaite
          else
            ! Affectation et calcul numérique
            y(dof) = diag_bc * tf%u(1, j)
          end if
          cycle
        end if

        ! Évaluation de la condition logique
        if (i == u_nx .and. mesh%is_boundary_right) then
          ! Évaluation de la condition logique
          if (cfg%bc_right%btype == BC_NEUMANN) then
            ! Affectation et calcul numérique
            y(dof) = diag_bc * (tf%u(u_nx, j) - tf%u(u_nx - 1, j))
          ! Branche par défaut si condition non satisfaite
          else
            ! Affectation et calcul numérique
            y(dof) = diag_bc * tf%u(u_nx, j)
          end if
          cycle
        end if

        ! Face dans un obstacle solide (adhérence nulle)
        if (is_u_face_in_obstacle(mesh, cfg, i, j)) then
          ! Affectation et calcul numérique
          y(dof) = diag_bc * tf%u(i, j)
          cycle
```

```
455         end if
456
457         ! Terme instationnaire
458         val = (rho_u / dt) * tf%u(i, j) * dvol
459
460         ! Terme convectif semi-implicite F_uu (forme advective non-conservative a densite harmonique)
461         if (cfg%enable_convection) then
462           u_n = fields_old%u(i, j)
463           u_n_E = fields_old%u(i + 1, j)
464           u_n_W = fields_old%u(i - 1, j)
465
466           v_n_N = 0.5_dp * (fields_old%v(i - 1, j + 1) + fields_old%v(i, j + 1))
467           v_n_S = 0.5_dp * (fields_old%v(i - 1, j) + fields_old%v(i, j))
468
469           mu_cell_E = (2.0_dp * rho_u * fields_old%rho_u(i + 1, j)) / max(1.0e-12_dp, rho_u + fields_old%rho_u
                (i + 1, j))
470           mu_cell_W = (2.0_dp * rho_u * fields_old%rho_u(i - 1, j)) / max(1.0e-12_dp, rho_u + fields_old%rho_u
                (i - 1, j))
471           if (j < mesh%ny_local) then
472             mu_node_N = (2.0_dp * rho_u * fields_old%rho_u(i, j + 1)) / max(1.0e-12_dp, rho_u + fields_old%
                  rho_u(i, j + 1))
473           else
474             mu_node_N = rho_u
475           end if
476           if (j > 1) then
477             mu_node_S = (2.0_dp * rho_u * fields_old%rho_u(i, j - 1)) / max(1.0e-12_dp, rho_u + fields_old%
                  rho_u(i, j - 1))
478           else
479             mu_node_S = rho_u
480           end if
481
482           conv_E = 0.5_dp * (u_n + u_n_E) * 0.5_dp * (tf%u(i + 1, j) - tf%u(i, j)) * dy
483           conv_W = 0.5_dp * (u_n_W + u_n) * 0.5_dp * (tf%u(i, j) - tf%u(i - 1, j)) * dy
484           conv_N = v_n_N * 0.5_dp * (tf%u(i, j + 1) - tf%u(i, j)) * dx
485           conv_S = v_n_S * 0.5_dp * (tf%u(i, j) - tf%u(i, j - 1)) * dx
486
487           val = val + (mu_cell_E * conv_E + mu_cell_W * conv_W + mu_node_N * conv_N + mu_node_S * conv_S)
488         end if
489
490         ! Termes visqueux implicites d'ordre 2
491         mu_cell_E = fields_old%mu(i, j)
492         ! Calcul de la viscosité dynamique locale (harmonique aux nœuds)
493         mu_cell_W = fields_old%mu(i - 1, j)
494         ! Affectation et calcul numérique
495         diff_elong_E = 2.0_dp * mu_cell_E * (tf%u(i + 1, j) - tf%u(i, j)) / dx * dy
496         ! Affectation et calcul numérique
497         diff_elong_W = 2.0_dp * mu_cell_W * (tf%u(i, j) - tf%u(i - 1, j)) / dx * dy
498         ! Affectation et calcul numérique
499         val = val - (diff_elong_E - diff_elong_W)
500
501         ! Calcul de la viscosité dynamique locale (harmonique aux nœuds)
502         mu_node_N = fields_old%mu_node(i, j + 1)
503         ! Calcul de la viscosité dynamique locale (harmonique aux nœuds)
504         mu_node_S = fields_old%mu_node(i, j)
505
506         ! Affectation et calcul numérique
507         shear_N = mu_node_N * ((tf%u(i, j + 1) - tf%u(i, j)) / dy + &
508                                (tf%v(i, j + 1) - tf%v(i - 1, j + 1)) / dx) * dx
509         ! Affectation et calcul numérique
510         shear_S = mu_node_S * ((tf%u(i, j) - tf%u(i, j - 1)) / dy + &
511                                (tf%v(i, j) - tf%v(i - 1, j)) / dx) * dx
512         ! Affectation et calcul numérique
513         val = val - (shear_N - shear_S)
514
515         ! Lagrangien augmenté : - r * d(div)/dx * dvol
516         grad_div_x = (div_cell(i, j) - div_cell(i - 1, j)) * dy
517         ! Affectation et calcul numérique
518         val = val - cfg%r_al * grad_div_x
519
520         ! Gradient de pression discret B_u^T (si monolithique)
521         if (cfg%solver_type == SOLVER_MONOLITHIC) then
522           ! Affectation et calcul numérique
523           grad_p = (tf%p(i, j) - tf%p(i - 1, j)) * dy
524           ! Affectation et calcul numérique
525           val = val + grad_p
526         end if
527
528         ! Affectation et calcul numérique
529         y(dof) = val
530       end do
531     end do
532
533     ! ============================================================================
534     ! 2. Équation de Quantité de Mouvement en v (Faces horizontales)
535     ! ============================================================================
536     v_offset = this%n_owned_u
537     ! Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
538     do j = 1, v_ny
539       ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
540       do i = 1, mesh%nx_local
541         ! Affectation et calcul numérique
542         dof = v_offset + (j - 1) * mesh%nx_local + i
```

```
543         ! Calcul de la masse volumique locale ou interpolée aux faces
544         rho_v = fields_old%rho_v(i, j)
545         ! Affectation et calcul numérique
546         diag_bc = (rho_v / dt) * dvol
547 
548         ! Affectation et calcul numérique
549         is_wall_bot = (j == 1 .and. mesh%is_boundary_bottom .and. &
550                        ! Affectation et calcul numérique
551                        (cfg%bc_bottom%btype == BC_DIRICHLET .or. cfg%bc_bottom%btype == BC_FREESLIP))
552         ! Affectation et calcul numérique
553         is_wall_top = (j == v_ny .and. mesh%is_boundary_top .and. &
554                        ! Affectation et calcul numérique
555                        (cfg%bc_top%btype == BC_DIRICHLET .or. cfg%bc_top%btype == BC_FREESLIP))
556         ! Affectation et calcul numérique
557         in_obs = is_v_face_in_obstacle(mesh, cfg, i, j)
558 
559         ! Évaluation de la condition logique
560         if (is_wall_bot .or. is_wall_top .or. in_obs) then
561           ! Affectation et calcul numérique
562           y(dof) = diag_bc * tf%v(i, j)
563           cycle
564         end if
565 
566         ! Affectation et calcul numérique
567         val = (rho_v / dt) * tf%v(i, j) * dvol
568 
569         ! Évaluation de la condition logique
570         if (cfg%enable_convection) then
571           v_n = fields_old%v(i, j)
572           v_n_N = fields_old%v(i, j + 1)
573           v_n_S_val = fields_old%v(i, j - 1)
574 
575           u_n_E = 0.5_dp * (fields_old%u(i + 1, j - 1) + fields_old%u(i + 1, j))
576           u_n_W = 0.5_dp * (fields_old%u(i, j - 1) + fields_old%u(i, j))
577 
578           if (i < mesh%nx_local) then
579             mu_node_E = (2.0_dp * rho_v * fields_old%rho_v(i + 1, j)) / max(1.0e-12_dp, rho_v + fields_old%
                  rho_v(i + 1, j))
580           else
581             mu_node_E = rho_v
582           end if
583           if (i > 1) then
584             mu_node_W = (2.0_dp * rho_v * fields_old%rho_v(i - 1, j)) / max(1.0e-12_dp, rho_v + fields_old%
                  rho_v(i - 1, j))
585           else
586             mu_node_W = rho_v
587           end if
588           mu_cell_N = (2.0_dp * rho_v * fields_old%rho_v(i, j + 1)) / max(1.0e-12_dp, rho_v + fields_old%rho_v
                (i, j + 1))
589           mu_cell_S = (2.0_dp * rho_v * fields_old%rho_v(i, j - 1)) / max(1.0e-12_dp, rho_v + fields_old%rho_v
                (i, j - 1))
590 
591           conv_E = u_n_E * 0.5_dp * (tf%v(i + 1, j) - tf%v(i, j)) * dy
592           conv_W = u_n_W * 0.5_dp * (tf%v(i, j) - tf%v(i - 1, j)) * dy
593           conv_N = 0.5_dp * (v_n + v_n_N) * 0.5_dp * (tf%v(i, j + 1) - tf%v(i, j)) * dx
594           conv_S = 0.5_dp * (v_n_S_val + v_n) * 0.5_dp * (tf%v(i, j) - tf%v(i, j - 1)) * dx
595 
596           val = val + (mu_node_E * conv_E + mu_node_W * conv_W + mu_cell_N * conv_N + mu_cell_S * conv_S)
597         end if
598 
599         ! Calcul de la viscosité dynamique locale (harmonique aux nœuds)
600         mu_cell_N = fields_old%mu(i, j)
601         ! Calcul de la viscosité dynamique locale (harmonique aux nœuds)
602         mu_cell_S = fields_old%mu(i, j - 1)
603         ! Affectation et calcul numérique
604         diff_elong_N = 2.0_dp * mu_cell_N * (tf%v(i, j + 1) - tf%v(i, j)) / dy * dx
605         ! Affectation et calcul numérique
606         diff_elong_S = 2.0_dp * mu_cell_S * (tf%v(i, j) - tf%v(i, j - 1)) / dy * dx
607         ! Affectation et calcul numérique
608         val = val - (diff_elong_N - diff_elong_S)
609 
610         ! Calcul de la viscosité dynamique locale (harmonique aux nœuds)
611         mu_node_E = fields_old%mu_node(i + 1, j)
612         ! Calcul de la viscosité dynamique locale (harmonique aux nœuds)
613         mu_node_W = fields_old%mu_node(i, j)
614 
615         ! Affectation et calcul numérique
616         shear_E = mu_node_E * ((tf%v(i + 1, j) - tf%v(i, j)) / dx + &
617                                (tf%u(i + 1, j) - tf%u(i + 1, j - 1)) / dy) * dy
618         ! Affectation et calcul numérique
619         shear_W = mu_node_W * ((tf%v(i, j) - tf%v(i - 1, j)) / dx + &
620                                (tf%u(i, j) - tf%u(i, j - 1)) / dy) * dy
621         ! Affectation et calcul numérique
622         val = val - (shear_E - shear_W)
623 
624         ! Affectation et calcul numérique
625         grad_div_y = (div_cell(i, j) - div_cell(i, j - 1)) * dx
626         ! Affectation et calcul numérique
627         val = val - cfg%r_al * grad_div_y
628 
629         ! Évaluation de la condition logique
630         if (cfg%solver_type == SOLVER_MONOLITHIC) then
```

```
631             ! Affectation et calcul numérique
632             grad_p = (tf%p(i, j) - tf%p(i, j - 1)) * dx
633             ! Affectation et calcul numérique
634             val = val + grad_p
635           end if
636
637           ! Affectation et calcul numérique
638           y(dof) = val
639         end do
640       end do
641
642       ! ========================================================================
643       ! 3. Équation de Continuité (si monolithique régularisé)
644       ! ========================================================================
645       if (cfg%solver_type == SOLVER_MONOLITHIC) then
646         ! Affectation et calcul numérique
647         p_offset = this%n_owned_u + this%n_owned_v
648         ! Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
649         do j = 1, mesh%ny_local
650           ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
651           do i = 1, mesh%nx_local
652             ! Affectation et calcul numérique
653             dof = p_offset + (j - 1) * mesh%nx_local + i
654             ! Affectation et calcul numérique
655             div_val = div_cell(i, j) * dvol
656             ! Affectation et calcul numérique
657             diag_p_term = (dvol / cfg%r_al) * tf%p(i, j)
658             ! Affectation et calcul numérique
659             y(dof) = div_val - diag_p_term
660           end do
661         end do
662       end if
663
664       ! Appel de sous-programme modulaire
665       call tf%free()
666     end subroutine ns_apply_system_matrix
667
668     !-------------------------------------------------------------------------------
669     ! Assemblage du second membre rhs pour le système linéaire
670     !-------------------------------------------------------------------------------
671     subroutine ns_assemble_rhs(this, mesh, cfg, fields, dt, rhs)
672       ! Déclaration de variable locale / paramètre de calcul
673       class(navier_stokes_t), intent(in) :: this
674       ! Déclaration de variable locale / paramètre de calcul
675       type(mesh_2d_t), intent(in)        :: mesh
676       ! Déclaration de variable locale / paramètre de calcul
677       type(config_t), intent(in)         :: cfg
678       ! Déclaration de variable locale / paramètre de calcul
679       type(flow_fields_t), intent(in)    :: fields
680       ! Déclaration de variable locale / paramètre de calcul
681       real(dp), intent(in)               :: dt
682       ! Déclaration de variable locale / paramètre de calcul
683       real(dp), intent(out)              :: rhs(:)
684
685       ! Déclaration de variable locale / paramètre de calcul
686       real(dp) :: dx, dy, dvol
687       ! Déclaration de variable locale / paramètre de calcul
688       integer :: i, j, dof, u_nx, v_ny, v_offset, p_offset
689       ! Déclaration de variable locale / paramètre de calcul
690       real(dp) :: rho_u, rho_v, diag_bc, val, u_bc, v_bc, y_pos, h_step, grad_p
691       ! Déclaration de variable locale / paramètre de calcul
692       real(dp) :: mu_N, mu_S, mu_W, mu_E, T_v
693       ! Déclaration des variables pour l'injecteur JET_INLET (Cas 3.3 Fluids 2021)
694       real(dp) :: xc_jet, half_w_jet
695
696       ! Calcul de la métrique spatiale élémentaire
697       dx = mesh%dx
698       ! Calcul de la métrique spatiale élémentaire
699       dy = mesh%dy
700       ! Affectation et calcul numérique
701       dvol = dx * dy
702
703       ! Affectation et calcul numérique
704       u_nx = mesh%nx_local
705       ! Affectation et calcul numérique
706       if (mesh%is_boundary_right) u_nx = u_nx + 1
707
708       ! Affectation et calcul numérique
709       v_ny = mesh%ny_local
710       ! Affectation et calcul numérique
711       if (mesh%is_boundary_top) v_ny = v_ny + 1
712
713       ! Affectation et calcul numérique
714       rhs = 0.0_dp
715
716       ! ========================================================================
717       ! 1. Second membre pour u
718       ! ========================================================================
719       do j = 1, mesh%ny_local
720         ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
721         do i = 1, u_nx
722           ! Affectation et calcul numérique
```

```
        dof = (j - 1) * u_nx + i
        ! Calcul de la masse volumique locale ou interpolée aux faces
        rho_u = fields%rho_u(i, j)
        ! Affectation et calcul numérique
        diag_bc = (rho_u / dt) * dvol

        ! Évaluation de la condition logique
        if (is_u_face_in_obstacle(mesh, cfg, i, j)) then
          ! Affectation et calcul numérique
          rhs(dof) = 0.0_dp
          cycle
        end if

        ! Évaluation de la condition logique
        if (i == 1 .and. mesh%is_boundary_left) then
          ! Affectation et calcul numérique
          u_bc = 0.0_dp
          ! Évaluation de la condition logique
          if (cfg%bc_left%btype == BC_DIRICHLET) then
            ! Évaluation de la condition logique
            if (cfg%obs_enabled .and. cfg%obs_type == OBS_BOX) then
              ! Affectation et calcul numérique
              y_pos = y_center(mesh, j)
              ! Évaluation de la condition logique
              if (y_pos >= cfg%obs_y1) then
                ! Affectation et calcul numérique
                h_step = mesh%ly - cfg%obs_y1
                ! Affectation et calcul numérique
                u_bc = 6.0_dp * cfg%bc_left%u * (y_pos - cfg%obs_y1) * (mesh%ly - y_pos) / (h_step * h_step)
              end if
            ! Branche par défaut si condition non satisfaite
            else
              ! Affectation et calcul numérique
              u_bc = cfg%bc_left%u
            end if
          end if
          ! Affectation et calcul numérique
          rhs(dof) = diag_bc * u_bc
          cycle
        end if

        ! Évaluation de la condition logique
        if (i == u_nx .and. mesh%is_boundary_right) then
          ! Affectation et calcul numérique
          u_bc = 0.0_dp
          ! Évaluation de la condition logique
          if (cfg%bc_right%btype == BC_DIRICHLET) then
            ! Affectation et calcul numérique
            u_bc = cfg%bc_right%u
          end if
          ! Affectation et calcul numérique
          rhs(dof) = diag_bc * u_bc
          cycle
        end if

        ! Affectation et calcul numérique
        val = (rho_u / dt) * fields%u(i, j) * dvol
        ! Affectation et calcul numérique
        val = val + rho_u * cfg%gx * dvol
        ! Affectation et calcul numérique
        val = val + fields%fx_sigma(i, j) * dvol

        ! Évaluation de la condition logique
        if (cfg%solver_type == SOLVER_AUGMENTED_LAGRANGIAN) then
          ! Affectation et calcul numérique
          grad_p = (fields%p(i, j) - fields%p(i - 1, j)) * dy
          ! Affectation et calcul numérique
          val = val - grad_p
        end if

        ! Évaluation de la condition logique
        if (j == mesh%ny_local .and. mesh%is_boundary_top .and. cfg%bc_top%btype == BC_DIRICHLET) then
          ! Calcul de la viscosité dynamique locale (harmonique aux nœuds)
          mu_N = fields%mu_node(i, j + 1)
          ! Affectation et calcul numérique
          val = val + mu_N * (2.0_dp * cfg%bc_top%u / dy) * dx
        end if
        ! Évaluation de la condition logique
        if (j == 1 .and. mesh%is_boundary_bottom .and. cfg%bc_bottom%btype == BC_DIRICHLET) then
          ! Calcul de la viscosité dynamique locale (harmonique aux nœuds)
          mu_S = fields%mu_node(i, j)
          ! Affectation et calcul numérique
          val = val + mu_S * (2.0_dp * cfg%bc_bottom%u / dy) * dx
        end if

        ! Affectation et calcul numérique
        rhs(dof) = val
      end do
    end do

    ! ======================================================================
    ! 2. Second membre pour v
```

```
    ! ==============================================================================
    v_offset = this%n_owned_u
    ! Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
    do j = 1, v_ny
      ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
      do i = 1, mesh%nx_local
        ! Affectation et calcul numérique
        dof = v_offset + (j - 1) * mesh%nx_local + i
        ! Calcul de la masse volumique locale ou interpolée aux faces
        rho_v = fields%rho_v(i, j)
        ! Affectation et calcul numérique
        diag_bc = (rho_v / dt) * dvol

        ! Évaluation de la condition logique
        if (is_v_face_in_obstacle(mesh, cfg, i, j)) then
          ! Affectation et calcul numérique
          rhs(dof) = 0.0_dp
          cycle
        end if

        ! Affectation et calcul numérique
        if (j == 1 .and. mesh%is_boundary_bottom .and. &
            ! Affectation et calcul numérique
            (cfg%bc_bottom%btype == BC_DIRICHLET .or. cfg%bc_bottom%btype == BC_FREESLIP)) then
          ! Affectation et calcul numérique
          v_bc = 0.0_dp
          ! Affectation et calcul numérique
          if (cfg%bc_bottom%btype == BC_DIRICHLET) v_bc = cfg%bc_bottom%v
          ! Affectation et calcul numérique
          rhs(dof) = diag_bc * v_bc
          cycle
        end if

        ! Affectation et calcul numérique
        if (j == v_ny .and. mesh%is_boundary_top .and. &
            ! Affectation et calcul numérique
            (cfg%bc_top%btype == BC_DIRICHLET .or. cfg%bc_top%btype == BC_FREESLIP)) then
          ! Affectation et calcul numérique
          v_bc = 0.0_dp
          ! Évaluation du profil d'injecteur JET_INLET (Cas 3.3 Fluids 2021)
          if (trim(cfg%bc_top%profile) == "JET_INLET") then
             xc_jet = merge(cfg%bc_top%jet_center, 0.5_dp * mesh%lx, cfg%bc_top%jet_center > 0.0_dp)
             half_w_jet = merge(0.5_dp * cfg%bc_top%jet_width, 0.04_dp, cfg%bc_top%jet_width > 0.0_dp)
             T_v = 0.0_dp
             do p_offset = 1, mesh%nx_global
                if (abs((real(p_offset, dp) - 0.5_dp) * mesh%dx - xc_jet) <= half_w_jet) T_v = T_v + mesh%dx
             end do
             if (T_v < 1.0e-12_dp) T_v = 2.0_dp * half_w_jet
             if (abs(x_center(mesh, i) - xc_jet) <= half_w_jet) then
                v_bc = cfg%bc_top%v * (2.0_dp * half_w_jet) / T_v
             else
                v_bc = -cfg%bc_top%v * (2.0_dp * half_w_jet) / (mesh%lx - T_v)
             end if
          else if (cfg%bc_top%btype == BC_DIRICHLET) then
             v_bc = cfg%bc_top%v
          end if
          ! Affectation et calcul numérique
          rhs(dof) = diag_bc * v_bc
          cycle
        end if

        ! Affectation et calcul numérique
        val = (rho_v / dt) * fields%v(i, j) * dvol
        ! Affectation et calcul numérique
        val = val + rho_v * cfg%gy * dvol

        ! Évaluation de la condition logique
        if (cfg%enable_energy) then
          ! Affectation et calcul numérique
          T_v = 0.5_dp * (fields%T(i, j) + fields%T(i, j - 1))
          ! Affectation et calcul numérique
          val = val + rho_v * cfg%beta * (T_v - cfg%T_ref) * cfg%gy * dvol
        end if

        ! Affectation et calcul numérique
        val = val + fields%fy_sigma(i, j) * dvol

        ! Évaluation de la condition logique
        if (cfg%solver_type == SOLVER_AUGMENTED_LAGRANGIAN) then
          ! Affectation et calcul numérique
          grad_p = (fields%p(i, j) - fields%p(i, j - 1)) * dx
          ! Affectation et calcul numérique
          val = val - grad_p
        end if

        ! Évaluation de la condition logique
        if (i == 1 .and. mesh%is_boundary_left .and. cfg%bc_left%btype == BC_DIRICHLET) then
          ! Calcul de la viscosité dynamique locale (harmonique aux nœuds)
          mu_W = fields%mu_node(i, j)
          ! Affectation et calcul numérique
          val = val + mu_W * (2.0_dp * cfg%bc_left%v / dx) * dy
        end if
```

```
        ! Évaluation de la condition logique
        if (i == mesh%nx_local .and. mesh%is_boundary_right .and. cfg%bc_right%btype == BC_DIRICHLET) then
          ! Calcul de la viscosité dynamique locale (harmonique aux nœuds)
          mu_E = fields%mu_node(i + 1, j)
          ! Affectation et calcul numérique
          val = val + mu_E * (2.0_dp * cfg%bc_right%v / dx) * dy
        end if

        ! Affectation et calcul numérique
        rhs(dof) = val
      end do
    end do

    ! ============================================================================
    ! 3. Second membre pour p (si monolithique)
    ! ============================================================================
    if (cfg%solver_type == SOLVER_MONOLITHIC) then
      ! Affectation et calcul numérique
      p_offset = this%n_owned_u + this%n_owned_v
      ! Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
      do j = 1, mesh%ny_local
        ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
        do i = 1, mesh%nx_local
          ! Affectation et calcul numérique
          dof = p_offset + (j - 1) * mesh%nx_local + i
          ! Affectation et calcul numérique
          rhs(dof) = - (dvol / cfg%r_al) * fields%p(i, j)
        end do
      end do
    end if
  end subroutine ns_assemble_rhs

  !-------------------------------------------------------------------------------
  ! Mise à jour de la pression par l'étape d'Uzawa du Lagrangien augmenté :
  !   p^{m+1} = p^m - r_al * div(u^{m+1})
  !-------------------------------------------------------------------------------
  subroutine ns_update_pressure_al(this, mesh, cfg, fields)
    ! Déclaration de variable locale / paramètre de calcul
    class(navier_stokes_t), intent(in) :: this
    ! Déclaration de variable locale / paramètre de calcul
    type(mesh_2d_t), intent(in)        :: mesh
    ! Déclaration de variable locale / paramètre de calcul
    type(config_t), intent(in)         :: cfg
    ! Déclaration de variable locale / paramètre de calcul
    type(flow_fields_t), intent(inout) :: fields

    ! Déclaration de variable locale / paramètre de calcul
    integer :: i, j
    ! Déclaration de variable locale / paramètre de calcul
    real(dp) :: dx, dy, div_val

    ! Calcul de la métrique spatiale élémentaire
    dx = mesh%dx
    ! Calcul de la métrique spatiale élémentaire
    dy = mesh%dy

    ! Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
    do j = 1, mesh%ny_local
      ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
      do i = 1, mesh%nx_local
        ! Affectation et calcul numérique
        div_val = (fields%u(i + 1, j) - fields%u(i, j)) / dx + &
                  (fields%v(i, j + 1) - fields%v(i, j)) / dy
        ! Actualisation de la pression statique au centre de la maille
        fields%p(i, j) = fields%p(i, j) - cfg%r_al * div_val
      end do
    end do

    ! Appel de sous-programme modulaire
    call sync_cell_scalar(mesh, fields%p)
  end subroutine ns_update_pressure_al

  !-------------------------------------------------------------------------------
  ! Assemblage des diagonales pour le préconditionnement Jacobi
  !-------------------------------------------------------------------------------
  subroutine ns_assemble_preconditioner_blocks(this, mesh, cfg, fields, dt)
    ! Déclaration de variable locale / paramètre de calcul
    class(navier_stokes_t), intent(inout) :: this
    ! Déclaration de variable locale / paramètre de calcul
    type(mesh_2d_t), intent(in)           :: mesh
    ! Déclaration de variable locale / paramètre de calcul
    type(config_t), intent(in)            :: cfg
    ! Déclaration de variable locale / paramètre de calcul
    type(flow_fields_t), intent(in)       :: fields
    ! Déclaration de variable locale / paramètre de calcul
    real(dp), intent(in)                  :: dt

    ! Déclaration de variable locale / paramètre de calcul
    real(dp) :: dx, dy, dvol, rho_u, rho_v, mu_c, d_val
    ! Déclaration de variable locale / paramètre de calcul
    integer :: i, j, dof, u_nx, v_ny
    ! Déclaration de variable locale / paramètre de calcul
```

```
    logical :: is_wall

    ! Calcul de la métrique spatiale élémentaire
    dx = mesh%dx
    ! Calcul de la métrique spatiale élémentaire
    dy = mesh%dy
    ! Affectation et calcul numérique
    dvol = dx * dy

    ! Affectation et calcul numérique
    u_nx = mesh%nx_local
    ! Affectation et calcul numérique
    if (mesh%is_boundary_right) u_nx = u_nx + 1

    ! Affectation et calcul numérique
    v_ny = mesh%ny_local
    ! Affectation et calcul numérique
    if (mesh%is_boundary_top) v_ny = v_ny + 1

    ! 1. Diagonale Bloc u
    do j = 1, mesh%ny_local
      ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
      do i = 1, u_nx
        ! Affectation et calcul numérique
        dof = (j - 1) * u_nx + i
        ! Calcul de la masse volumique locale ou interpolée aux faces
        rho_u = fields%rho_u(i, j)
        ! Affectation et calcul numérique
        is_wall = (i == 1 .and. mesh%is_boundary_left) .or. &
                  ! Affectation et calcul numérique
                  (i == u_nx .and. mesh%is_boundary_right) .or. &
                  is_u_face_in_obstacle(mesh, cfg, i, j)

        ! Évaluation de la condition logique
        if (is_wall) then
          ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
          this%diag_u(dof) = (rho_u / dt) * dvol
          cycle
        end if

        ! Calcul de la viscosité dynamique locale (harmonique aux nœuds)
        mu_c = fields%mu(i, j)
        ! Affectation et calcul numérique
        d_val = (rho_u / dt) * dvol + 4.0_dp * mu_c * (dy / dx) + 2.0_dp * mu_c * (dx / dy) + &
                2.0_dp * cfg%r_al * (dy / dx)
        ! Évaluation de la condition logique
        if (d_val > 1.0e-15_dp) then
          ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
          this%diag_u(dof) = d_val
        ! Branche par défaut si condition non satisfaite
        else
          ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
          this%diag_u(dof) = 1.0_dp
        end if
      end do
    end do

    ! 2. Diagonale Bloc v
    do j = 1, v_ny
      ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
      do i = 1, mesh%nx_local
        ! Affectation et calcul numérique
        dof = (j - 1) * mesh%nx_local + i
        ! Calcul de la masse volumique locale ou interpolée aux faces
        rho_v = fields%rho_v(i, j)
        ! Affectation et calcul numérique
        is_wall = (j == 1 .and. mesh%is_boundary_bottom .and. &
                   ! Affectation et calcul numérique
                   (cfg%bc_bottom%btype == BC_DIRICHLET .or. cfg%bc_bottom%btype == BC_FREESLIP)) .or. &
                  ! Affectation et calcul numérique
                  (j == v_ny .and. mesh%is_boundary_top .and. &
                   ! Affectation et calcul numérique
                   (cfg%bc_top%btype == BC_DIRICHLET .or. cfg%bc_top%btype == BC_FREESLIP)) .or. &
                  is_v_face_in_obstacle(mesh, cfg, i, j)

        ! Évaluation de la condition logique
        if (is_wall) then
          ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
          this%diag_v(dof) = (rho_v / dt) * dvol
          cycle
        end if

        ! Calcul de la viscosité dynamique locale (harmonique aux nœuds)
        mu_c = fields%mu(i, j)
        ! Affectation et calcul numérique
        d_val = (rho_v / dt) * dvol + 4.0_dp * mu_c * (dx / dy) + 2.0_dp * mu_c * (dy / dx) + &
                2.0_dp * cfg%r_al * (dx / dy)
        ! Évaluation de la condition logique
        if (d_val > 1.0e-15_dp) then
          ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
          this%diag_v(dof) = d_val
        ! Branche par défaut si condition non satisfaite
```

```
        else
          ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
          this%diag_v(dof) = 1.0_dp
        end if
      end do
    end do

    ! 3. Diagonale Bloc p (si monolithique)
    if (cfg%solver_type == SOLVER_MONOLITHIC) then
      ! Affectation et calcul numérique
      this%diag_p = dvol / cfg%r_al
    end if
  end subroutine ns_assemble_preconditioner_blocks

  !================================================================================
  ! Résolution de l'énergie thermique (Boussinesq)
  !================================================================================
  subroutine ns_solve_energy(this, mesh, cfg, fields, dt)
    ! Déclaration de variable locale / paramètre de calcul
    class(navier_stokes_t), intent(in) :: this
    ! Déclaration de variable locale / paramètre de calcul
    type(mesh_2d_t), intent(in)        :: mesh
    ! Déclaration de variable locale / paramètre de calcul
    type(config_t), intent(in)         :: cfg
    ! Déclaration de variable locale / paramètre de calcul
    type(flow_fields_t), intent(inout) :: fields
    ! Déclaration de variable locale / paramètre de calcul
    real(dp), intent(in)               :: dt

    ! Déclaration de variable locale / paramètre de calcul
    integer :: nx, ny, n_cells, i, j, c_dof, iter, max_it
    ! Déclaration de variable locale / paramètre de calcul
    real(dp) :: dx, dy, dvol, rho, Cp, k_th
    ! Déclaration de variable locale / paramètre de calcul
    real(dp) :: u_e, u_w, v_n, v_s
    ! Déclaration de variable locale / paramètre de calcul
    real(dp) :: d_e, d_w, d_n, d_s, conv_e, conv_w, conv_n, conv_s
    ! Déclaration de variable locale / paramètre de calcul
    real(dp) :: a_c, val
    ! Déclaration de variable locale / paramètre de calcul
    real(dp), allocatable :: a_diag(:), a_east(:), a_west(:), a_north(:), a_south(:)
    ! Déclaration de variable locale / paramètre de calcul
    real(dp), allocatable :: rhs(:), T_sol(:), r(:), p(:), v_vec(:), s_vec(:), t_vec(:)
    ! Déclaration de variable locale / paramètre de calcul
    real(dp), allocatable :: r0_hat(:), y_prec(:), z_prec(:)
    ! Déclaration de variable locale / paramètre de calcul
    real(dp) :: rho_old, alpha, omega, rho_new, r0v, s_norm, tt, ts, r_norm, res_init

    if (.not. cfg%enable_energy) return

    ! Affectation et calcul numérique
    nx = mesh%nx_local
    ! Affectation et calcul numérique
    ny = mesh%ny_local
    ! Affectation et calcul numérique
    n_cells = nx * ny

    ! Calcul de la métrique spatiale élémentaire
    dx = mesh%dx
    ! Calcul de la métrique spatiale élémentaire
    dy = mesh%dy
    ! Affectation et calcul numérique
    dvol = dx * dy
    ! Calcul de la masse volumique locale ou interpolée aux faces
    rho = fields%rho(1, 1)
    ! Affectation et calcul numérique
    Cp = cfg%Cp
    ! Affectation et calcul numérique
    k_th = cfg%k_thermal

    ! Allocation dynamique de tableau contigu en mémoire
    allocate(a_diag(n_cells), a_east(n_cells), a_west(n_cells), a_north(n_cells), a_south(n_cells))
    ! Allocation dynamique de tableau contigu en mémoire
    allocate(rhs(n_cells), T_sol(n_cells), r(n_cells), p(n_cells), v_vec(n_cells), s_vec(n_cells), t_vec(
        n_cells))
    ! Allocation dynamique de tableau contigu en mémoire
    allocate(r0_hat(n_cells), y_prec(n_cells), z_prec(n_cells))

    ! Affectation et calcul numérique
    a_diag = 0.0_dp
    ! Affectation et calcul numérique
    a_east = 0.0_dp
    ! Affectation et calcul numérique
    a_west = 0.0_dp
    ! Affectation et calcul numérique
    a_north = 0.0_dp
    ! Affectation et calcul numérique
    a_south = 0.0_dp
    ! Affectation et calcul numérique
    rhs = 0.0_dp

    ! Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
```

```
    do j = 1, ny
      ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
      do i = 1, nx
        ! Affectation et calcul numérique
        c_dof = (j - 1) * nx + i
        ! Affectation et calcul numérique
        T_sol(c_dof) = fields%T(i, j)

        ! Affectation et calcul numérique
        a_c = (rho * Cp / dt) * dvol
        ! Affectation et calcul numérique
        rhs(c_dof) = (rho * Cp / dt) * dvol * fields%T(i, j)

        ! Affectation et calcul numérique
        u_e = fields%u(i + 1, j)
        ! Affectation et calcul numérique
        d_e = k_th * dy / dx
        ! Affectation et calcul numérique
        conv_e = rho * Cp * u_e * dy * 0.5_dp

        ! Affectation et calcul numérique
        u_w = fields%u(i, j)
        ! Affectation et calcul numérique
        d_w = k_th * dy / dx
        ! Affectation et calcul numérique
        conv_w = rho * Cp * u_w * dy * 0.5_dp

        ! Affectation et calcul numérique
        v_n = fields%v(i, j + 1)
        ! Affectation et calcul numérique
        d_n = k_th * dx / dy
        ! Affectation et calcul numérique
        conv_n = rho * Cp * v_n * dx * 0.5_dp

        ! Affectation et calcul numérique
        v_s = fields%v(i, j)
        ! Affectation et calcul numérique
        d_s = k_th * dx / dy
        ! Affectation et calcul numérique
        conv_s = rho * Cp * v_s * dx * 0.5_dp

        ! Évaluation de la condition logique
        if (i < nx) then
          ! Affectation et calcul numérique
          a_c = a_c + d_e + conv_e
          ! Affectation et calcul numérique
          a_east(c_dof) = -d_e + conv_e
        ! Branche conditionnelle alternative
        else if (mesh%is_boundary_right) then
          ! Évaluation de la condition logique
          if (cfg%bc_right%thermal_type == THERMAL_DIRICHLET) then
            ! Affectation et calcul numérique
            a_c = a_c + 2.0_dp * d_e
            ! Affectation et calcul numérique
            rhs(c_dof) = rhs(c_dof) + 2.0_dp * d_e * cfg%bc_right%T
          end if
        ! Branche par défaut si condition non satisfaite
        else
          ! Affectation et calcul numérique
          a_c = a_c + d_e + conv_e
          ! Affectation et calcul numérique
          rhs(c_dof) = rhs(c_dof) - (-d_e + conv_e) * fields%T(nx + 1, j)
        end if

        ! Évaluation de la condition logique
        if (i > 1) then
          ! Affectation et calcul numérique
          a_c = a_c + d_w - conv_w
          ! Affectation et calcul numérique
          a_west(c_dof) = -d_w - conv_w
        ! Branche conditionnelle alternative
        else if (mesh%is_boundary_left) then
          ! Évaluation de la condition logique
          if (cfg%bc_left%thermal_type == THERMAL_DIRICHLET) then
            ! Affectation et calcul numérique
            a_c = a_c + 2.0_dp * d_w
            ! Affectation et calcul numérique
            rhs(c_dof) = rhs(c_dof) + 2.0_dp * d_w * cfg%bc_left%T
          end if
        ! Branche par défaut si condition non satisfaite
        else
          ! Affectation et calcul numérique
          a_c = a_c + d_w - conv_w
          ! Affectation et calcul numérique
          rhs(c_dof) = rhs(c_dof) - (-d_w - conv_w) * fields%T(0, j)
        end if

        ! Évaluation de la condition logique
        if (j < ny) then
          ! Affectation et calcul numérique
          a_c = a_c + d_n + conv_n
          ! Affectation et calcul numérique
```

```
          a_north(c_dof) = -d_n + conv_n
        ! Branche conditionnelle alternative
        else if (mesh%is_boundary_top) then
          ! Évaluation de la condition logique
          if (cfg%bc_top%thermal_type == THERMAL_DIRICHLET) then
            ! Affectation et calcul numérique
            a_c = a_c + 2.0_dp * d_n
            ! Affectation et calcul numérique
            rhs(c_dof) = rhs(c_dof) + 2.0_dp * d_n * cfg%bc_top%T
          end if
        ! Branche par défaut si condition non satisfaite
        else
          ! Affectation et calcul numérique
          a_c = a_c + d_n + conv_n
          ! Affectation et calcul numérique
          rhs(c_dof) = rhs(c_dof) - (-d_n + conv_n) * fields%T(i, ny + 1)
        end if

        ! Évaluation de la condition logique
        if (j > 1) then
          ! Affectation et calcul numérique
          a_c = a_c + d_s - conv_s
          ! Affectation et calcul numérique
          a_south(c_dof) = -d_s - conv_s
        ! Branche conditionnelle alternative
        else if (mesh%is_boundary_bottom) then
          ! Évaluation de la condition logique
          if (cfg%bc_bottom%thermal_type == THERMAL_DIRICHLET) then
            ! Affectation et calcul numérique
            a_c = a_c + 2.0_dp * d_s
            ! Affectation et calcul numérique
            rhs(c_dof) = rhs(c_dof) + 2.0_dp * d_s * cfg%bc_bottom%T
          end if
        ! Branche par défaut si condition non satisfaite
        else
          ! Affectation et calcul numérique
          a_c = a_c + d_s - conv_s
          ! Affectation et calcul numérique
          rhs(c_dof) = rhs(c_dof) - (-d_s - conv_s) * fields%T(i, 0)
        end if

        ! Affectation et calcul numérique
        a_diag(c_dof) = a_c
      end do
    end do

    ! Résolution itérative BiCGSTAB avec préconditionnement Jacobi
    do j = 1, ny
      ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
      do i = 1, nx
        ! Affectation et calcul numérique
        c_dof = (j - 1) * nx + i
        ! Affectation et calcul numérique
        val = a_diag(c_dof) * T_sol(c_dof)
        ! Affectation et calcul numérique
        if (i < nx) val = val + a_east(c_dof) * T_sol(c_dof + 1)
        ! Affectation et calcul numérique
        if (i > 1)  val = val + a_west(c_dof) * T_sol(c_dof - 1)
        ! Affectation et calcul numérique
        if (j < ny) val = val + a_north(c_dof) * T_sol(c_dof + nx)
        ! Affectation et calcul numérique
        if (j > 1)  val = val + a_south(c_dof) * T_sol(c_dof - nx)
        ! Affectation et calcul numérique
        r(c_dof) = rhs(c_dof) - val
      end do
    end do

    ! Affectation et calcul numérique
    r0_hat = r
    ! Affectation et calcul numérique
    p = r
    ! Calcul de la masse volumique locale ou interpolée aux faces
    rho_old = 1.0_dp
    ! Affectation et calcul numérique
    alpha = 1.0_dp
    ! Affectation et calcul numérique
    omega = 1.0_dp

    ! Évaluation de la norme du résidu ou de l'erreur numérique
    res_init = sqrt(sum(r*r))
    ! Évaluation de la condition logique
    if (res_init > 1.0e-13_dp) then
      ! Affectation et calcul numérique
      max_it = 500
      ! Boucle d'itération numérique
      do iter = 1, max_it
        ! Calcul de la masse volumique locale ou interpolée aux faces
        rho_new = sum(r0_hat * r)
        if (abs(rho_new) < 1.0e-30_dp) exit

        ! Évaluation de la condition logique
        if (iter > 1) then
```

```
          ! Affectation et calcul numérique
          p = r + (rho_new / rho_old) * (alpha / omega) * (p - omega * v_vec)
        ! Branche par défaut si condition non satisfaite
        else
          ! Affectation et calcul numérique
          p = r
        end if

        ! Affectation et calcul numérique
        y_prec = p / a_diag

        ! Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
        do j = 1, ny
          ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
          do i = 1, nx
            ! Affectation et calcul numérique
            c_dof = (j - 1) * nx + i
            ! Affectation et calcul numérique
            val = a_diag(c_dof) * y_prec(c_dof)
            ! Affectation et calcul numérique
            if (i < nx) val = val + a_east(c_dof) * y_prec(c_dof + 1)
            ! Affectation et calcul numérique
            if (i > 1)  val = val + a_west(c_dof) * y_prec(c_dof - 1)
            ! Affectation et calcul numérique
            if (j < ny) val = val + a_north(c_dof) * y_prec(c_dof + nx)
            ! Affectation et calcul numérique
            if (j > 1)  val = val + a_south(c_dof) * y_prec(c_dof - nx)
            ! Affectation et calcul numérique
            v_vec(c_dof) = val
          end do
        end do

        ! Affectation et calcul numérique
        r0v = sum(r0_hat * v_vec)
        if (abs(r0v) < 1.0e-30_dp) exit
        ! Affectation et calcul numérique
        alpha = rho_new / r0v

        ! Affectation et calcul numérique
        s_vec = r - alpha * v_vec
        ! Évaluation de la norme du résidu ou de l'erreur numérique
        s_norm = sqrt(sum(s_vec * s_vec))
        ! Évaluation de la condition logique
        if (s_norm / res_init < 1.0e-8_dp) then
          ! Affectation et calcul numérique
          T_sol = T_sol + alpha * y_prec
          exit
        end if

        ! Affectation et calcul numérique
        z_prec = s_vec / a_diag

        ! Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
        do j = 1, ny
          ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
          do i = 1, nx
            ! Affectation et calcul numérique
            c_dof = (j - 1) * nx + i
            ! Affectation et calcul numérique
            val = a_diag(c_dof) * z_prec(c_dof)
            ! Affectation et calcul numérique
            if (i < nx) val = val + a_east(c_dof) * z_prec(c_dof + 1)
            ! Affectation et calcul numérique
            if (i > 1)  val = val + a_west(c_dof) * z_prec(c_dof - 1)
            ! Affectation et calcul numérique
            if (j < ny) val = val + a_north(c_dof) * z_prec(c_dof + nx)
            ! Affectation et calcul numérique
            if (j > 1)  val = val + a_south(c_dof) * z_prec(c_dof - nx)
            ! Affectation et calcul numérique
            t_vec(c_dof) = val
          end do
        end do

        ! Affectation et calcul numérique
        tt = sum(t_vec * t_vec)
        ! Affectation et calcul numérique
        ts = sum(t_vec * s_vec)
        ! Évaluation de la condition logique
        if (tt > 1.0e-30_dp) then
          ! Affectation et calcul numérique
          omega = ts / tt
        ! Branche par défaut si condition non satisfaite
        else
          ! Affectation et calcul numérique
          omega = 0.0_dp
        end if

        ! Affectation et calcul numérique
        T_sol = T_sol + alpha * y_prec + omega * z_prec
        ! Affectation et calcul numérique
        r = s_vec - omega * t_vec
        ! Calcul de la masse volumique locale ou interpolée aux faces
```

```
        rho_old = rho_new

        ! Évaluation de la norme du résidu ou de l'erreur numérique
        r_norm = sqrt(sum(r * r))
        if (r_norm / res_init < 1.0e-8_dp) exit
      end do
    end if

    ! Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
    do j = 1, ny
      ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
      do i = 1, nx
        ! Évolution du champ scalaire de température
        fields%T(i, j) = T_sol((j - 1) * nx + i)
      end do
    end do

    ! Appel de sous-programme modulaire
    call sync_cell_scalar(mesh, fields%T)
    ! Appel de sous-programme modulaire
    call apply_thermal_boundary_conditions(mesh, cfg, fields)

    ! Libération de la mémoire allouée
    deallocate(a_diag, a_east, a_west, a_north, a_south)
    ! Libération de la mémoire allouée
    deallocate(rhs, T_sol, r, p, v_vec, s_vec, t_vec, r0_hat, y_prec, z_prec)
  end subroutine ns_solve_energy

  !-------------------------------------------------------------------------------
  ! Calcul du nombre de Nusselt moyen sur la paroi chaude (gauche)
  !-------------------------------------------------------------------------------
  function ns_compute_mean_nusselt_hot_wall(this, mesh, cfg, fields) result(nu_mean)
    ! Déclaration de variable locale / paramètre de calcul
    class(navier_stokes_t), intent(in) :: this
    ! Déclaration de variable locale / paramètre de calcul
    type(mesh_2d_t), intent(in)        :: mesh
    ! Déclaration de variable locale / paramètre de calcul
    type(config_t), intent(in)         :: cfg
    ! Déclaration de variable locale / paramètre de calcul
    type(flow_fields_t), intent(in)    :: fields
    ! Déclaration de variable locale / paramètre de calcul
    real(dp)                           :: nu_mean

    ! Déclaration de variable locale / paramètre de calcul
    integer :: j, count_loc
    ! Déclaration de variable locale / paramètre de calcul
    real(dp) :: local_nu_sum, global_nu_sum, global_count
    ! Déclaration de variable locale / paramètre de calcul
    real(dp) :: T_w, T_1, delta_T, nu_local

    ! Affectation et calcul numérique
    local_nu_sum = 0.0_dp
    ! Affectation et calcul numérique
    count_loc = 0

    ! Évaluation de la condition logique
    if (mesh%is_boundary_left) then
      ! Affectation et calcul numérique
      delta_T = abs(cfg%bc_left%T - cfg%bc_right%T)
      ! Affectation et calcul numérique
      if (delta_T < 1.0e-12_dp) delta_T = 1.0_dp

      ! Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
      do j = 1, mesh%ny_local
        ! Affectation et calcul numérique
        T_w = cfg%bc_left%T
        ! Affectation et calcul numérique
        T_1 = fields%T(1, j)
        ! Affectation et calcul numérique
        nu_local = 2.0_dp * (T_w - T_1) / mesh%dx * (mesh%lx / delta_T)
        ! Affectation et calcul numérique
        local_nu_sum = local_nu_sum + nu_local
        ! Affectation et calcul numérique
        count_loc = count_loc + 1
      end do
    end if

    ! Affectation et calcul numérique
    global_nu_sum = global_sum(mesh, local_nu_sum)
    ! Affectation et calcul numérique
    global_count = global_sum(mesh, real(count_loc, dp))

    ! Évaluation de la condition logique
    if (global_count > 0.0_dp) then
      ! Affectation et calcul numérique
      nu_mean = global_nu_sum / global_count
    ! Branche par défaut si condition non satisfaite
    else
      ! Affectation et calcul numérique
      nu_mean = 1.0_dp
    end if
  end function ns_compute_mean_nusselt_hot_wall
```

```
1550
1551 end module module_navier_stokes
```

Chapitre 20

Reconstruction Géométrique d'Interface VOF en Fortran 2018 (module$_{interface_reconstruction.f90}$)

20.1 Algorithmes Géométriques dans `module_interface_reconstruction.f90`

Le module `src/module_interface_reconstruction.f90` (569 lignes) implémente en Fortran 2018 :

— `reconstruct_interface_normals` : sélectionne entre le filtre 3×3 de Parker-Youngs (`RECON_PLIC`), l'algorithme à 6 candidats exacts d'**ELVIRA** (`RECON_ELVIRA`), l'optimisation quadratique non linéaire de **LVIRA** (`RECON_LVIRA`) et l'alignement axial **SLIC** (`RECON_SLIC`).

— `plic_alpha_from_c` et `plic_volume_from_alpha` : implémentent les formules analytiques directes et inverses de Scardovelli & Zaleski (2000) reliant la constante de droite α à l'aire tronquée C dans le carré unité $[0,1]^2$.

— `plic_flux_1d` : calcule géométriquement le volume exact de phase 1 contenu dans la bande lagrangienne $[1-\text{cfl}, 1] \times [0,1]$ traversant une face de maille au cours du pas de temps Δt.

20.2 Répertoire des Types Dérivés, Sous-Routines et Fonctions de `src/module_interface_reconstruction.f90`

Ligne	Nature	Identifiant Fortran 2018	Interface / Arguments
2	module	`module_interface_reconstruction`	Unité modulaire Fortran 2018
22	type dérivé	`interface_segment_t`	Structure de données Fortran 2018
32	function	`cut_box_area`	Arguments : (xmin, xmax, ymin, ymax, nx, ny, alpha)
104	function	`compute_alpha`	Arguments : (xmin, xmax, ymin, ymax, nx, ny, c_target)
149	subroutine	`reconstruct_slic`	Arguments : (mesh, C, segments)
212	subroutine	`reconstruct_plic`	Arguments : (mesh, C, segments)
275	subroutine	`reconstruct_elvira`	Arguments : (mesh, C, segments)
389	subroutine	`reconstruct_lvira`	Arguments : (mesh, C, segments)
509	subroutine	`reconstruct_interface`	Arguments : (method_name, mesh, C, segments)
592	function	`evaluate_limiter`	Arguments : (limiter_name, r, c_donor, c_upwind, c_a...)

20.3 Code Source Intégral Annoté : `src/module_interface_reconstruction.f9`

Le listing ci-dessous présente l'intégralité des 633 lignes du fichier `src/module_interface_reconstruction.f90` de l'implémentation Fortran 2018 :

```
! [Procédure/Module] module module_interface_reconstruction
module module_interface_reconstruction
  use module_precision, only: dp
  use module_mesh, only: mesh_2d_t, x_face, y_face
  implicit none
  private

  ! Types publics
  public :: interface_segment_t

  ! Procédures publiques
  public :: cut_box_area
```

```
  public :: compute_alpha
  public :: reconstruct_slic
  public :: reconstruct_plic
  public :: reconstruct_elvira
  public :: reconstruct_lvira
  public :: reconstruct_interface
  public :: evaluate_limiter

  ! Type représentant un segment de droite d'interface affine : nx*x + ny*y + alpha = 0
  type :: interface_segment_t
     real(dp) :: nx = 0.0_dp
     real(dp) :: ny = 0.0_dp
     real(dp) :: alpha = 0.0_dp
  end type interface_segment_t

contains

  ! [Procédure/Module] function cut_box_area
  ! Calcule l'aire exacte de découpe d'un rectangle par un demi-plan (Sutherland-Hodgman + Shoelace)
  function cut_box_area(xmin, xmax, ymin, ymax, nx, ny, alpha) result(area)
    real(dp), intent(in) :: xmin, xmax, ymin, ymax, nx, ny, alpha
    real(dp) :: area

    real(dp) :: bx(4), by(4)
    real(dp) :: d(4)
    real(dp) :: cx(8), cy(8)
    integer  :: n_clipped, i, next_i, p, next_p
    real(dp) :: d1, d2, t
    real(dp) :: sum_cross

    if ((xmax - xmin) <= 1.0e-14_dp .or. (ymax - ymin) <= 1.0e-14_dp) then
       area = 0.0_dp
       return
    end if

    ! Définition des 4 sommets du rectangle en sens trigonométrique
    bx(1) = xmin; by(1) = ymin
    bx(2) = xmax; by(2) = ymin
    bx(3) = xmax; by(3) = ymax
    bx(4) = xmin; by(4) = ymax

    do i = 1, 4
       d(i) = nx * bx(i) + ny * by(i) + alpha
    end do

    n_clipped = 0
    do i = 1, 4
       next_i = mod(i, 4) + 1
       d1 = d(i)
       d2 = d(next_i)

       if (d1 <= 0.0_dp) then
          if (d2 <= 0.0_dp) then
             n_clipped = n_clipped + 1
             cx(n_clipped) = bx(next_i)
             cy(n_clipped) = by(next_i)
          else
             t = -d1 / (d2 - d1)
             n_clipped = n_clipped + 1
             cx(n_clipped) = bx(i) + t * (bx(next_i) - bx(i))
             cy(n_clipped) = by(i) + t * (by(next_i) - by(i))
          end if
       else
          if (d2 <= 0.0_dp) then
             t = -d1 / (d2 - d1)
             n_clipped = n_clipped + 1
             cx(n_clipped) = bx(i) + t * (bx(next_i) - bx(i))
             cy(n_clipped) = by(i) + t * (by(next_i) - by(i))
             n_clipped = n_clipped + 1
             cx(n_clipped) = bx(next_i)
             cy(n_clipped) = by(next_i)
          end if
       end if
    end do

    if (n_clipped < 3) then
       area = 0.0_dp
       return
    end if

    ! Formule de Gauss / Shoelace pour l'aire du polygone
    sum_cross = 0.0_dp
    do p = 1, n_clipped
       next_p = mod(p, n_clipped) + 1
       sum_cross = sum_cross + (cx(p) * cy(next_p) - cx(next_p) * cy(p))
    end do
    area = 0.5_dp * abs(sum_cross)
  end function cut_box_area

  ! [Procédure/Module] function compute_alpha
  ! Inversion géométrique par bisection : C_target -> paramètre alpha
  function compute_alpha(xmin, xmax, ymin, ymax, nx, ny, c_target) result(alpha)
```

```
    real(dp), intent(in) :: xmin, xmax, ymin, ymax, nx, ny, c_target
    real(dp) :: alpha

    real(dp) :: total_area, target_area
    real(dp) :: v1, v2, v3, v4, min_val, max_val
    real(dp) :: alpha_low, alpha_high, alpha_mid, a
    integer  :: iter

    total_area = (xmax - xmin) * (ymax - ymin)
    if (c_target <= 1.0e-7_dp) then
       alpha = 1.0e9_dp
       return
    end if
    if (c_target >= 1.0_dp - 1.0e-7_dp) then
       alpha = -1.0e9_dp
       return
    end if

    target_area = max(0.0_dp, min(1.0_dp, c_target)) * total_area

    v1 = nx * xmin + ny * ymin
    v2 = nx * xmax + ny * ymin
    v3 = nx * xmax + ny * ymax
    v4 = nx * xmin + ny * ymax

    min_val = min(v1, min(v2, min(v3, v4)))
    max_val = max(v1, max(v2, max(v3, v4)))

    alpha_low  = -max_val - 1.0e-12_dp
    alpha_high = -min_val + 1.0e-12_dp

    do iter = 1, 45
       alpha_mid = 0.5_dp * (alpha_low + alpha_high)
       a = cut_box_area(xmin, xmax, ymin, ymax, nx, ny, alpha_mid)
       if (a < target_area) then
          alpha_high = alpha_mid
       else
          alpha_low = alpha_mid
       end if
    end do
    alpha = 0.5_dp * (alpha_low + alpha_high)
  end function compute_alpha

  ! [Procédure/Module] subroutine reconstruct_slic
  subroutine reconstruct_slic(mesh, C, segments)
    type(mesh_2d_t), intent(in) :: mesh
    real(dp), intent(in) :: C(0:mesh%nx_local+1, 0:mesh%ny_local+1)
    type(interface_segment_t), intent(out) :: segments(0:mesh%nx_local+1, 0:mesh%ny_local+1)

    integer :: i, j, nx, ny
    real(dp) :: c_val, xmin, xmax, ymin, ymax, grad_x, grad_y, snx, sny, a

    nx = mesh%nx_local
    ny = mesh%ny_local

    do j = 0, ny + 1
       do i = 0, nx + 1
          c_val = C(i, j)
          if (c_val <= 1.0e-6_dp) then
             segments(i, j)%nx = 1.0_dp
             segments(i, j)%ny = 0.0_dp
             segments(i, j)%alpha = 1.0e9_dp
             cycle
          end if
          if (c_val >= 1.0_dp - 1.0e-6_dp) then
             segments(i, j)%nx = 1.0_dp
             segments(i, j)%ny = 0.0_dp
             segments(i, j)%alpha = -1.0e9_dp
             cycle
          end if

          xmin = x_face(mesh, i)
          xmax = x_face(mesh, i + 1)
          ymin = y_face(mesh, j)
          ymax = y_face(mesh, j + 1)

          if (i > 0 .and. i < nx + 1 .and. j > 0 .and. j < ny + 1) then
             grad_x = abs(C(i + 1, j) - C(i - 1, j))
             grad_y = abs(C(i, j + 1) - C(i, j - 1))
          else
             grad_x = 1.0_dp
             grad_y = 0.0_dp
          end if

          if (grad_x >= grad_y) then
             if (i > 0 .and. C(i - 1, j) >= C(i + 1, j)) then
                snx = 1.0_dp; sny = 0.0_dp
             else
                snx = -1.0_dp; sny = 0.0_dp
             end if
          else
             if (j > 0 .and. C(i, j - 1) >= C(i, j + 1)) then
```

```
               snx = 0.0_dp; sny = 1.0_dp
            else
               snx = 0.0_dp; sny = -1.0_dp
            end if
         end if

         a = compute_alpha(xmin, xmax, ymin, ymax, snx, sny, c_val)
         segments(i, j)%nx = snx
         segments(i, j)%ny = sny
         segments(i, j)%alpha = a
      end do
   end do
end subroutine reconstruct_slic

! [Procédure/Module] subroutine reconstruct_plic
subroutine reconstruct_plic(mesh, C, segments)
   type(mesh_2d_t), intent(in) :: mesh
   real(dp), intent(in) :: C(0:mesh%nx_local+1, 0:mesh%ny_local+1)
   type(interface_segment_t), intent(out) :: segments(0:mesh%nx_local+1, 0:mesh%ny_local+1)

   integer :: i, j, nx, ny
   real(dp) :: c_val, dx, dy, xmin, xmax, ymin, ymax
   real(dp) :: c_mm, c_0m, c_pm, c_m0, c_p0, c_mp, c_0p, c_pp
   real(dp) :: mx, my, norm, pnx, pny, a

   nx = mesh%nx_local
   ny = mesh%ny_local
   dx = mesh%dx
   dy = mesh%dy

   do j = 0, ny + 1
      do i = 0, nx + 1
         c_val = C(i, j)
         if (c_val <= 1.0e-6_dp) then
            segments(i, j)%nx = 1.0_dp
            segments(i, j)%ny = 0.0_dp
            segments(i, j)%alpha = 1.0e9_dp
            cycle
         end if
         if (c_val >= 1.0_dp - 1.0e-6_dp) then
            segments(i, j)%nx = 1.0_dp
            segments(i, j)%ny = 0.0_dp
            segments(i, j)%alpha = -1.0e9_dp
            cycle
         end if

         if (i >= 1 .and. i <= nx .and. j >= 1 .and. j <= ny) then
            c_mm = C(i - 1, j - 1); c_0m = C(i,     j - 1); c_pm = C(i + 1, j - 1)
            c_m0 = C(i - 1, j    );                         c_p0 = C(i + 1, j    )
            c_mp = C(i - 1, j + 1); c_0p = C(i,     j + 1); c_pp = C(i + 1, j + 1)

            mx = -((c_pp + 2.0_dp*c_p0 + c_pm) - (c_mp + 2.0_dp*c_m0 + c_mm)) / (8.0_dp * dx)
            my = -((c_pp + 2.0_dp*c_0p + c_mp) - (c_pm + 2.0_dp*c_0m + c_mm)) / (8.0_dp * dy)
         else
            mx = 1.0_dp; my = 0.0_dp
         end if

         norm = hypot(mx, my)
         if (norm > 1.0e-12_dp) then
            pnx = mx / norm; pny = my / norm
         else
            pnx = 1.0_dp; pny = 0.0_dp
         end if

         xmin = x_face(mesh, i)
         xmax = x_face(mesh, i + 1)
         ymin = y_face(mesh, j)
         ymax = y_face(mesh, j + 1)

         a = compute_alpha(xmin, xmax, ymin, ymax, pnx, pny, c_val)
         segments(i, j)%nx = pnx
         segments(i, j)%ny = pny
         segments(i, j)%alpha = a
      end do
   end do
end subroutine reconstruct_plic

! [Procédure/Module] subroutine reconstruct_elvira
subroutine reconstruct_elvira(mesh, C, segments)
   type(mesh_2d_t), intent(in) :: mesh
   real(dp), intent(in) :: C(0:mesh%nx_local+1, 0:mesh%ny_local+1)
   type(interface_segment_t), intent(out) :: segments(0:mesh%nx_local+1, 0:mesh%ny_local+1)

   integer :: i, j, nx, ny, di, dj, k
   real(dp) :: c_val, dx, dy, xmin, xmax, ymin, ymax
   real(dp) :: H_m1, H_0, H_p1, W_m1, W_0, W_p1
   real(dp) :: sgn_x, sgn_y, mx, my
   real(dp) :: cand_nx(7), cand_ny(7)
   real(dp) :: best_err, err, cnorm, cnx, cny, calpha, diff
   real(dp) :: best_nx, best_ny, best_alpha
   real(dp) :: v_xmin, v_xmax, v_ymin, v_ymax, v_target, v_recon

```

```
    nx = mesh%nx_local
    ny = mesh%ny_local
    dx = mesh%dx
    dy = mesh%dy

    do j = 0, ny + 1
       do i = 0, nx + 1
          c_val = C(i, j)
          if (c_val <= 1.0e-6_dp) then
             segments(i, j)%nx = 1.0_dp
             segments(i, j)%ny = 0.0_dp
             segments(i, j)%alpha = 1.0e9_dp
             cycle
          end if
          if (c_val >= 1.0_dp - 1.0e-6_dp) then
             segments(i, j)%nx = 1.0_dp
             segments(i, j)%ny = 0.0_dp
             segments(i, j)%alpha = -1.0e9_dp
             cycle
          end if

          xmin = x_face(mesh, i)
          xmax = x_face(mesh, i + 1)
          ymin = y_face(mesh, j)
          ymax = y_face(mesh, j + 1)

          if (i >= 1 .and. i <= nx .and. j >= 1 .and. j <= ny) then
             H_m1 = (C(i - 1, j - 1) + C(i - 1, j) + C(i - 1, j + 1)) * dy
             H_0  = (C(i,     j - 1) + C(i,     j) + C(i,     j + 1)) * dy
             H_p1 = (C(i + 1, j - 1) + C(i + 1, j) + C(i + 1, j + 1)) * dy

             W_m1 = (C(i - 1, j - 1) + C(i, j - 1) + C(i + 1, j - 1)) * dx
             W_0  = (C(i - 1, j    ) + C(i, j    ) + C(i + 1, j    )) * dx
             W_p1 = (C(i - 1, j + 1) + C(i, j + 1) + C(i + 1, j + 1)) * dx

             if (C(i, j + 1) < C(i, j - 1)) then; sgn_y = 1.0_dp; else; sgn_y = -1.0_dp; end if
             if (C(i + 1, j) < C(i - 1, j)) then; sgn_x = 1.0_dp; else; sgn_x = -1.0_dp; end if

             ! 6 candidats Height Functions
             cand_nx(1) = -sgn_y * (H_p1 - H_m1) / (2.0_dp * dx); cand_ny(1) = sgn_y * 1.0_dp
             cand_nx(2) = -sgn_y * (H_p1 - H_0)  / (dx);          cand_ny(2) = sgn_y * 1.0_dp
             cand_nx(3) = -sgn_y * (H_0  - H_m1) / (dx);          cand_ny(3) = sgn_y * 1.0_dp

             cand_nx(4) = sgn_x * 1.0_dp; cand_ny(4) = -sgn_x * (W_p1 - W_m1) / (2.0_dp * dy)
             cand_nx(5) = sgn_x * 1.0_dp; cand_ny(5) = -sgn_x * (W_p1 - W_0)  / (dy)
             cand_nx(6) = sgn_x * 1.0_dp; cand_ny(6) = -sgn_x * (W_0  - W_m1) / (dy)

             ! Candidat 7 : Youngs
             mx = -((C(i+1,j+1) + 2.0_dp*C(i+1,j) + C(i+1,j-1)) - (C(i-1,j+1) + 2.0_dp*C(i-1,j) + C(i-1,j-1)))
                  / (8.0_dp * dx)
             my = -((C(i+1,j+1) + 2.0_dp*C(i,j+1) + C(i-1,j+1)) - (C(i+1,j-1) + 2.0_dp*C(i,j-1) + C(i-1,j-1)))
                  / (8.0_dp * dy)
             cand_nx(7) = mx; cand_ny(7) = my

             best_err = 1.0e30_dp
             best_nx = 1.0_dp; best_ny = 0.0_dp; best_alpha = 0.0_dp

             do k = 1, 7
                cnorm = hypot(cand_nx(k), cand_ny(k))
                if (cnorm < 1.0e-12_dp) cycle
                cnx = cand_nx(k) / cnorm
                cny = cand_ny(k) / cnorm

                calpha = compute_alpha(xmin, xmax, ymin, ymax, cnx, cny, c_val)

                err = 0.0_dp
                do dj = -1, 1
                   do di = -1, 1
                      if (di == 0 .and. dj == 0) cycle
                      v_xmin = x_face(mesh, i + di)
                      v_xmax = x_face(mesh, i + di + 1)
                      v_ymin = y_face(mesh, j + dj)
                      v_ymax = y_face(mesh, j + dj + 1)

                      v_target = C(i + di, j + dj) * dx * dy
                      v_recon  = cut_box_area(v_xmin, v_xmax, v_ymin, v_ymax, cnx, cny, calpha)
                      diff = v_recon - v_target
                      err = err + diff * diff
                   end do
                end do

                if (err < best_err) then
                   best_err = err
                   best_nx = cnx
                   best_ny = cny
                   best_alpha = calpha
                end if
             end do

             segments(i, j)%nx = best_nx
             segments(i, j)%ny = best_ny
             segments(i, j)%alpha = best_alpha
```

```
          else
             segments(i, j)%nx = 1.0_dp
             segments(i, j)%ny = 0.0_dp
             segments(i, j)%alpha = compute_alpha(xmin, xmax, ymin, ymax, 1.0_dp, 0.0_dp, c_val)
          end if
       end do
    end do
  end subroutine reconstruct_elvira

  ! [Procédure/Module] subroutine reconstruct_lvira
  subroutine reconstruct_lvira(mesh, C, segments)
    type(mesh_2d_t), intent(in) :: mesh
    real(dp), intent(in) :: C(0:mesh%nx_local+1, 0:mesh%ny_local+1)
    type(interface_segment_t), intent(out) :: segments(0:mesh%nx_local+1, 0:mesh%ny_local+1)

    integer :: i, j, nx, ny, iter, di, dj
    real(dp) :: c_val, dx, dy, xmin, xmax, ymin, ymax, theta0
    real(dp) :: invphi, invphi2, a_pt, b_pt, h_pt, c_pt, d_pt, fc, fd
    real(dp) :: best_theta, best_nx, best_ny, best_alpha
    real(dp) :: v_xmin, v_xmax, v_ymin, v_ymax, v_target, v_recon, diff

    ! Étape 1 : Initialisation par ELVIRA
    call reconstruct_elvira(mesh, C, segments)

    nx = mesh%nx_local
    ny = mesh%ny_local
    dx = mesh%dx
    dy = mesh%dy
    invphi = (sqrt(5.0_dp) - 1.0_dp) / 2.0_dp
    invphi2 = (3.0_dp - sqrt(5.0_dp)) / 2.0_dp

    do j = 1, ny
       do i = 1, nx
          c_val = C(i, j)
          if (c_val <= 1.0e-6_dp .or. c_val >= 1.0_dp - 1.0e-6_dp) cycle

          xmin = x_face(mesh, i)
          xmax = x_face(mesh, i + 1)
          ymin = y_face(mesh, j)
          ymax = y_face(mesh, j + 1)

          theta0 = atan2(segments(i, j)%ny, segments(i, j)%nx)
          a_pt = theta0 - 0.5_dp
          b_pt = theta0 + 0.5_dp
          h_pt = b_pt - a_pt

          c_pt = a_pt + invphi2 * h_pt
          d_pt = a_pt + invphi * h_pt

          ! Évaluation fc
          fc = 0.0_dp
          do dj = -1, 1
             do di = -1, 1
                if (di == 0 .and. dj == 0) cycle
                v_xmin = x_face(mesh, i + di); v_xmax = x_face(mesh, i + di + 1)
                v_ymin = y_face(mesh, j + dj); v_ymax = y_face(mesh, j + dj + 1)
                v_target = C(i + di, j + dj) * dx * dy
                v_recon  = cut_box_area(v_xmin, v_xmax, v_ymin, v_ymax, cos(c_pt), sin(c_pt), &
                                        compute_alpha(xmin, xmax, ymin, ymax, cos(c_pt), sin(c_pt), c_val))
                diff = v_recon - v_target
                fc = fc + diff * diff
             end do
          end do

          ! Évaluation fd
          fd = 0.0_dp
          do dj = -1, 1
             do di = -1, 1
                if (di == 0 .and. dj == 0) cycle
                v_xmin = x_face(mesh, i + di); v_xmax = x_face(mesh, i + di + 1)
                v_ymin = y_face(mesh, j + dj); v_ymax = y_face(mesh, j + dj + 1)
                v_target = C(i + di, j + dj) * dx * dy
                v_recon  = cut_box_area(v_xmin, v_xmax, v_ymin, v_ymax, cos(d_pt), sin(d_pt), &
                                        compute_alpha(xmin, xmax, ymin, ymax, cos(d_pt), sin(d_pt), c_val))
                diff = v_recon - v_target
                fd = fd + diff * diff
             end do
          end do

          do iter = 1, 10
             if (fc < fd) then
                b_pt = d_pt; d_pt = c_pt; fd = fc
                h_pt = invphi * h_pt
                c_pt = a_pt + invphi2 * h_pt
                fc = 0.0_dp
                do dj = -1, 1
                   do di = -1, 1
                      if (di == 0 .and. dj == 0) cycle
                      v_xmin = x_face(mesh, i + di); v_xmax = x_face(mesh, i + di + 1)
                      v_ymin = y_face(mesh, j + dj); v_ymax = y_face(mesh, j + dj + 1)
                      v_target = C(i + di, j + dj) * dx * dy
                      v_recon  = cut_box_area(v_xmin, v_xmax, v_ymin, v_ymax, cos(c_pt), sin(c_pt), &
```

```
                                        compute_alpha(xmin, xmax, ymin, ymax, cos(c_pt), sin(c_pt), c_val
                                            ))
                        diff = v_recon - v_target
                        fc = fc + diff * diff
                     end do
                  end do
               else
                  a_pt = c_pt; c_pt = d_pt; fc = fd
                  h_pt = invphi * h_pt
                  d_pt = a_pt + invphi * h_pt
                  fd = 0.0_dp
                  do dj = -1, 1
                     do di = -1, 1
                        if (di == 0 .and. dj == 0) cycle
                        v_xmin = x_face(mesh, i + di); v_xmax = x_face(mesh, i + di + 1)
                        v_ymin = y_face(mesh, j + dj); v_ymax = y_face(mesh, j + dj + 1)
                        v_target = C(i + di, j + dj) * dx * dy
                        v_recon  = cut_box_area(v_xmin, v_xmax, v_ymin, v_ymax, cos(d_pt), sin(d_pt), &
                                                compute_alpha(xmin, xmax, ymin, ymax, cos(d_pt), sin(d_pt), c_val
                                                    ))
                        diff = v_recon - v_target
                        fd = fd + diff * diff
                     end do
                  end do
               end if
            end do

            best_theta = 0.5_dp * (a_pt + b_pt)
            best_nx = cos(best_theta)
            best_ny = sin(best_theta)
            best_alpha = compute_alpha(xmin, xmax, ymin, ymax, best_nx, best_ny, c_val)

            segments(i, j)%nx = best_nx
            segments(i, j)%ny = best_ny
            segments(i, j)%alpha = best_alpha
         end do
      end do
   end subroutine reconstruct_lvira

   ! [Procédure/Module] subroutine reconstruct_interface
   subroutine reconstruct_interface(method_name, mesh, C, segments)
      character(len=*), intent(in) :: method_name
      type(mesh_2d_t), intent(in) :: mesh
      real(dp), intent(in) :: C(0:mesh%nx_local+1, 0:mesh%ny_local+1)
      type(interface_segment_t), intent(out) :: segments(0:mesh%nx_local+1, 0:mesh%ny_local+1)

      integer :: i, j, nx, ny, i_left_src, i_right_src, j_bot_src, j_top_src
      real(dp) :: c_val

      select case (trim(method_name))
      case ("SLIC")
         call reconstruct_slic(mesh, C, segments)
      case ("LVIRA")
         call reconstruct_lvira(mesh, C, segments)
      case ("ELVIRA")
         call reconstruct_elvira(mesh, C, segments)
      case default
         call reconstruct_plic(mesh, C, segments)
      end select

      nx = mesh%nx_local
      ny = mesh%ny_local
      if (.not. mesh%is_boundary_left .and. mesh%size <= 1) then
         i_left_src = nx
      else
         i_left_src = 1
      end if
      if (.not. mesh%is_boundary_right .and. mesh%size <= 1) then
         i_right_src = 1
      else
         i_right_src = nx
      end if
      if (.not. mesh%is_boundary_bottom .and. mesh%size <= 1) then
         j_bot_src = ny
      else
         j_bot_src = 1
      end if
      if (.not. mesh%is_boundary_top .and. mesh%size <= 1) then
         j_top_src = 1
      else
         j_top_src = ny
      end if

      do j = 1, ny
         c_val = C(0, j)
         if (c_val > 1.0e-6_dp .and. c_val < 1.0_dp - 1.0e-6_dp) then
            segments(0, j)%nx = segments(i_left_src, j)%nx
            segments(0, j)%ny = segments(i_left_src, j)%ny
            segments(0, j)%alpha = compute_alpha(x_face(mesh, 0), x_face(mesh, 1), &
                                                 y_face(mesh, j), y_face(mesh, j + 1), &
                                                 segments(0, j)%nx, segments(0, j)%ny, c_val)
         end if
```

```
561         c_val = C(nx + 1, j)
562         if (c_val > 1.0e-6_dp .and. c_val < 1.0_dp - 1.0e-6_dp) then
563            segments(nx + 1, j)%nx = segments(i_right_src, j)%nx
564            segments(nx + 1, j)%ny = segments(i_right_src, j)%ny
565            segments(nx + 1, j)%alpha = compute_alpha(x_face(mesh, nx + 1), x_face(mesh, nx + 2), &
566                                                      y_face(mesh, j), y_face(mesh, j + 1), &
567                                                      segments(nx + 1, j)%nx, segments(nx + 1, j)%ny, c_val)
568         end if
569      end do
570
571      do i = 0, nx + 1
572         c_val = C(i, 0)
573         if (c_val > 1.0e-6_dp .and. c_val < 1.0_dp - 1.0e-6_dp) then
574            segments(i, 0)%nx = segments(i, j_bot_src)%nx
575            segments(i, 0)%ny = segments(i, j_bot_src)%ny
576            segments(i, 0)%alpha = compute_alpha(x_face(mesh, i), x_face(mesh, i + 1), &
577                                                 y_face(mesh, 0), y_face(mesh, 1), &
578                                                 segments(i, 0)%nx, segments(i, 0)%ny, c_val)
579         end if
580         c_val = C(i, ny + 1)
581         if (c_val > 1.0e-6_dp .and. c_val < 1.0_dp - 1.0e-6_dp) then
582            segments(i, ny + 1)%nx = segments(i, j_top_src)%nx
583            segments(i, ny + 1)%ny = segments(i, j_top_src)%ny
584            segments(i, ny + 1)%alpha = compute_alpha(x_face(mesh, i), x_face(mesh, i + 1), &
585                                                      y_face(mesh, ny + 1), y_face(mesh, ny + 2), &
586                                                      segments(i, ny + 1)%nx, segments(i, ny + 1)%ny, c_val)
587         end if
588      end do
589   end subroutine reconstruct_interface
590
591   ! [Procédure/Module] function evaluate_limiter
592   function evaluate_limiter(limiter_name, r, c_donor, c_upwind, c_acceptor) result(phi)
593      character(len=*), intent(in) :: limiter_name
594      real(dp), intent(in) :: r, c_donor, c_upwind, c_acceptor
595      real(dp) :: phi
596      real(dp) :: denom, c_tilde_D, c_tilde_f
597
598      select case (trim(limiter_name))
599      case ("VAN_LEER")
600         if (r > 0.0_dp) then
601            phi = 2.0_dp * r / (1.0_dp + r)
602         else
603            phi = 0.0_dp
604         end if
605      case ("MUSCL")
606         phi = max(0.0_dp, min(2.0_dp * r, min(0.5_dp * (1.0_dp + r), 2.0_dp)))
607      case ("HRIC")
608         denom = c_acceptor - c_upwind
609         if (abs(denom) < 1.0e-12_dp) then
610            phi = 0.0_dp
611            return
612         end if
613         c_tilde_D = (c_donor - c_upwind) / denom
614         c_tilde_f = c_tilde_D
615         if (c_tilde_D >= 0.0_dp .and. c_tilde_D <= 1.0_dp) then
616            if (c_tilde_D < 0.5_dp) then
617               c_tilde_f = 2.0_dp * c_tilde_D
618            else
619               c_tilde_f = 1.0_dp
620            end if
621         end if
622         if (abs(1.0_dp - c_tilde_D) > 1.0e-12_dp) then
623            phi = 2.0_dp * (c_tilde_f - c_tilde_D) / (1.0_dp - c_tilde_D)
624            phi = max(0.0_dp, min(2.0_dp, phi))
625         else
626            phi = 1.0_dp
627         end if
628      case default ! "SUPERBEE"
629         phi = max(0.0_dp, max(min(2.0_dp * r, 1.0_dp), min(r, 2.0_dp)))
630      end select
631   end function evaluate_limiter
632
633 end module module_interface_reconstruction
```

Chapitre 21

Advection Géométrique et Algébrique VOF en Fortran 2018 (module$_{interface_vof.f90}$)

21.1 Architecture des 1034 Lignes de `module_interface_vof.f90`

Ce module implémente la séparation directionnelle d'ordre 2 de Strang (`advect_vof_interface`) et les trois grandes familles de schémas de transport interfacial :

1. **Schéma géométrique de Weymouth & Yue (2010)** (`advect_weymouth_yue_x`, `advect_weymouth_yue_y`) : utilise la fonction indicatrice binaire $C_{\text{dil}}(i,j) = \mathbb{I}(C^n_{i,j} \geq 0.5)$ partagée entre les deux balayages directionnels pour garantir une conservation de la masse exacte à la précision machine.
2. **Schéma géométrique CIAM de Li (1995)** (`advect_ciam_x`, `advect_ciam_y`) : transporte lagrangiennement les segments PLIC en tenant compte de l'étirement/compression affine de la bande donneuse par le gradient de vitesse local $\partial u/\partial x$.
3. **Schémas algébriques TVD haute résolution** (`advect_tvd_x`, `advect_tvd_y`) : implémentent les limiteurs de pente non linéaires **Superbee**, **Van Leer**, **MUSCL / Monotonized Central (MC)**, **Minmod** et **HRIC**.

21.2 Répertoire des Types Dérivés, Sous-Routines et Fonctions de `src/module_interface_vof.f90`

Ligne	Nature	Identifiant Fortran 2018	Interface / Arguments
2	module	`module_interface_vof`	Unité modulaire Fortran 2018
23	type dérivé	`interface_vof_t`	Structure de données Fortran 2018
38	subroutine	`set_initial_conditions`	Arguments : (fields, mesh, cfg)
235	subroutine	`update_physical_properties`	Arguments : (fields, mesh, cfg)
372	subroutine	`compute_curvature_and_csf`	Arguments : (fields, mesh, cfg)
495	subroutine	`advect_vof`	Arguments : (fields, mesh, cfg, dt)
519	subroutine	`advect_vof_tvd`	Arguments : (fields, mesh, cfg, dt)
629	subroutine	`advect_vof_ciam`	Arguments : (fields, mesh, cfg, dt)
807	subroutine	`advect_vof_weymouth_yue`	Arguments : (fields, mesh, cfg, dt)
979	subroutine	`vof_init`	Arguments : (this, cfg)
989	subroutine	`vof_set_ic`	Arguments : (this, mesh, fields)
1005	subroutine	`vof_advect_step`	Arguments : (this, mesh, fields, dt)

21.3 Code Source Intégral Annoté : `src/module_interface_vof.f90`

Le listing ci-dessous présente l'intégralité des 1022 lignes du fichier `src/module_interface_vof.f90` de l'implémentation Fortran 2018 :

```
! [Procédure/Module] module module_interface_vof
module module_interface_vof
  use module_precision, only: dp
  use module_constants
  use module_mesh, only: mesh_2d_t, x_center, y_center, x_face, y_face, is_in_obstacle
  use module_fields, only: flow_fields_t
```

```
  use module_config, only: config_t
  use module_mpi_comm, only: exchange_halos_scalar, apply_vof_bc
  use module_interface_reconstruction, only: interface_segment_t, cut_box_area, compute_alpha, &
                                             reconstruct_interface, evaluate_limiter
  implicit none
  private

  public :: interface_vof_t
  public :: set_initial_conditions
  public :: advect_vof
  public :: update_physical_properties
  public :: compute_curvature_and_csf

  ! Compteur de pas pour Strang splitting alterné
  integer, save :: vof_step_count = 0

  type :: interface_vof_t
     ! Déclaration de variable locale / paramètre de calcul
     type(config_t) :: cfg
  contains
     ! Affectation et calcul numérique
     procedure :: init => vof_init
     ! Affectation et calcul numérique
     procedure :: set_initial_condition => vof_set_ic
     ! Affectation et calcul numérique
     procedure :: advect => vof_advect_step
  end type interface_vof_t

contains

  ! [Procédure/Module] subroutine set_initial_conditions
  subroutine set_initial_conditions(fields, mesh, cfg)
    ! Déclaration de variable locale / paramètre de calcul
    type(flow_fields_t), intent(inout) :: fields
    ! Déclaration de variable locale / paramètre de calcul
    type(mesh_2d_t), intent(in) :: mesh
    ! Déclaration de variable locale / paramètre de calcul
    type(config_t), intent(in) :: cfg

    ! Déclaration de variable locale / paramètre de calcul
    integer :: i, j, nx, ny
    ! Déclaration de variable locale / paramètre de calcul
    real(dp) :: x, y, dist, eps, int_y, x_c_jet, half_w_jet, stub_h_jet

    ! Affectation et calcul numérique
    nx = mesh%nx_local
    ! Affectation et calcul numérique
    ny = mesh%ny_local
    ! Affectation et calcul numérique
    eps = 1.5_dp * max(mesh%dx, mesh%dy)

    ! 1. Initialisation des vitesses (par défaut 0 ou imposé)
    fields%u = cfg%ic_u
    ! Affectation et calcul numérique
    fields%v = cfg%ic_v

    ! 2. Initialisation de la fraction volumique C
    do j = 0, ny + 1
       ! Affectation et calcul numérique
       y = y_center(mesh, j)
       ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
       do i = 0, nx + 1
          ! Affectation et calcul numérique
          x = x_center(mesh, i)

          select case (cfg%ic_type)
          case (IC_UNIFORM)
             ! Transport / actualisation de la fraction volumique VOF
             fields%C(i, j) = 1.0_dp

          case (IC_DROPLET)
             ! Affectation et calcul numérique
             dist = sqrt((x - cfg%ic_center_x)**2 + (y - cfg%ic_center_y)**2) - cfg%ic_radius
             ! Évaluation de la condition logique
             if (dist < -eps) then
                ! Transport / actualisation de la fraction volumique VOF
                fields%C(i, j) = 1.0_dp
             ! Branche conditionnelle alternative
             else if (dist > eps) then
                ! Transport / actualisation de la fraction volumique VOF
                fields%C(i, j) = 0.0_dp
             ! Branche par défaut si condition non satisfaite
             else
                ! Transport / actualisation de la fraction volumique VOF
                fields%C(i, j) = 0.5_dp - dist / (2.0_dp * eps) - (1.0_dp / (2.0_dp * PI)) * sin(PI * dist /
                    eps)
             end if

          case (IC_BUBBLE)
             ! Affectation et calcul numérique
             dist = sqrt((x - cfg%ic_center_x)**2 + (y - cfg%ic_center_y)**2) - cfg%ic_radius
             ! Évaluation de la condition logique
```

```
98              if (dist < -eps) then
99                 ! Transport / actualisation de la fraction volumique VOF
100                fields%C(i, j) = 0.0_dp ! Intérieur bulle (fluide 2)
101             ! Branche conditionnelle alternative
102             else if (dist > eps) then
103                ! Transport / actualisation de la fraction volumique VOF
104                fields%C(i, j) = 1.0_dp ! Extérieur (fluide 1)
105             ! Branche par défaut si condition non satisfaite
106             else
107                ! Transport / actualisation de la fraction volumique VOF
108                fields%C(i, j) = 0.5_dp + dist / (2.0_dp * eps) + (1.0_dp / (2.0_dp * PI)) * sin(PI * dist /
                       eps)
109             end if
110
111          case (IC_DAM_BREAK)
112             ! Affectation et calcul numérique
113             if (x >= cfg%ic_box_x0 .and. x <= cfg%ic_box_x1 .and. &
114                 ! Affectation et calcul numérique
115                 y >= cfg%ic_box_y0 .and. y <= cfg%ic_box_y1) then
116                ! Transport / actualisation de la fraction volumique VOF
117                fields%C(i, j) = 1.0_dp
118             ! Branche par défaut si condition non satisfaite
119             else
120                ! Transport / actualisation de la fraction volumique VOF
121                fields%C(i, j) = 0.0_dp
122             end if
123
124          case (IC_LIQUID_SHEET)
125             ! Nappe liquide cisaillée (Cas 3.2 de l'article Fluids 2021)
126             if (cfg%ic_center_y > 0.0_dp) then
127                ! Calcul de distance à l'axe de la nappe
128                dist = abs(y - cfg%ic_center_y) - 0.5_dp * cfg%ic_thickness
129             else
130                ! Calcul de distance au plan médian
131                dist = abs(y - 0.5_dp * mesh%ly) - 0.5_dp * cfg%ic_thickness
132             end if
133             ! Évaluation de l'indicatrice C
134             if (dist < -eps) then
135                fields%C(i, j) = 0.0_dp ! Intérieur nappe liquide (fluide 2)
136             else if (dist > eps) then
137                fields%C(i, j) = 1.0_dp ! Gaz ambiant (fluide 1)
138             else
139                fields%C(i, j) = 0.5_dp + dist / (2.0_dp * eps) + &
140                     (1.0_dp / (2.0_dp * PI)) * sin(PI * dist / eps)
141             end if
142
143          case (IC_VISCOUS_JET)
144             ! Jet liquide visqueux injecté (Cas 3.3 de l'article Fluids 2021)
145             if (cfg%ic_center_x > 0.0_dp) then
146                x_c_jet = cfg%ic_center_x
147             else
148                x_c_jet = 0.5_dp * mesh%lx
149             end if
150             if (cfg%ic_jet_width > 0.0_dp) then
151                half_w_jet = 0.5_dp * cfg%ic_jet_width
152             else
153                half_w_jet = 0.04_dp
154             end if
155             stub_h_jet = 2.0_dp * mesh%dy
156             if (y >= mesh%ly - stub_h_jet .and. abs(x - x_c_jet) <= half_w_jet) then
157                fields%C(i, j) = 0.0_dp ! Amorce liquide (fluide 2)
158             else
159                fields%C(i, j) = 1.0_dp ! Cavité de gaz (fluide 1)
160             end if
161
162          case (IC_VORTEX_SHEAR)
163             ! Affectation et calcul numérique
164             dist = sqrt((x - cfg%ic_center_x)**2 + (y - cfg%ic_center_y)**2) - cfg%ic_radius
165             ! Évaluation de la condition logique
166             if (dist < -eps) then
167                ! Transport / actualisation de la fraction volumique VOF
168                fields%C(i, j) = 1.0_dp
169             ! Branche conditionnelle alternative
170             else if (dist > eps) then
171                ! Transport / actualisation de la fraction volumique VOF
172                fields%C(i, j) = 0.0_dp
173             ! Branche par défaut si condition non satisfaite
174             else
175                ! Transport / actualisation de la fraction volumique VOF
176                fields%C(i, j) = 0.5_dp - dist / (2.0_dp * eps) - (1.0_dp / (2.0_dp * PI)) * sin(PI * dist /
                       eps)
177             end if
178
179          case (IC_RAYLEIGH_TAYLOR)
180             ! Affectation et calcul numérique
181             int_y = cfg%ic_center_y + cfg%ic_pert_amp * cos(2.0_dp * PI * cfg%ic_pert_k * x / mesh%lx)
182             ! Évaluation de la condition logique
183             if (y >= int_y) then
184                ! Transport / actualisation de la fraction volumique VOF
185                fields%C(i, j) = 1.0_dp ! Fluide lourd au-dessus
186             ! Branche par défaut si condition non satisfaite
187             else
```

```
                ! Transport / actualisation de la fraction volumique VOF
                fields%C(i, j) = 0.0_dp ! Fluide léger en dessous
            end if

          case (IC_LAPLACIAN_TEST)
            ! Transport / actualisation de la fraction volumique VOF
            fields%C(i, j) = 1.0_dp

          case default
            ! Transport / actualisation de la fraction volumique VOF
            fields%C(i, j) = 1.0_dp
          end select
        end do
    end do

    ! 3. Détection des obstacles solides pour la pénalisation de Brinkman
    fields%solid_mask = 0
    ! Évaluation de la condition logique
    if (cfg%obs_enabled) then
        ! Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
        do j = 0, ny + 1
            ! Affectation et calcul numérique
            y = y_center(mesh, j)
            ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
            do i = 0, nx + 1
                ! Affectation et calcul numérique
                x = x_center(mesh, i)
                if (is_in_obstacle(x, y, cfg%obs_type, cfg%obs_x0, cfg%obs_y0, cfg%obs_x1, cfg%obs_y1, &
                                   cfg%obs_xc, cfg%obs_yc, cfg%obs_radius)) then
                    ! Affectation et calcul numérique
                    fields%solid_mask(i, j) = 1
                end if
            end do
        end do
    end if

    ! Appel de sous-programme modulaire
    call exchange_halos_scalar(fields%C, mesh)
    ! Appel de sous-programme modulaire
    call apply_vof_bc(fields, mesh, cfg)
    ! Appel de sous-programme modulaire
    call update_physical_properties(fields, mesh, cfg)
    ! Appel de sous-programme modulaire
    call compute_curvature_and_csf(fields, mesh, cfg)
  end subroutine set_initial_conditions

  ! [Procédure/Module] subroutine update_physical_properties
  subroutine update_physical_properties(fields, mesh, cfg)
    ! Déclaration de variable locale / paramètre de calcul
    type(flow_fields_t), intent(inout) :: fields
    ! Déclaration de variable locale / paramètre de calcul
    type(mesh_2d_t), intent(in) :: mesh
    ! Déclaration de variable locale / paramètre de calcul
    type(config_t), intent(in) :: cfg

    ! Déclaration de variable locale / paramètre de calcul
    integer :: i, j, nx, ny
    ! Déclaration de variable locale / paramètre de calcul
    real(dp) :: c, y, z, mu_val, m_sw, m_se, m_nw, m_ne

    ! Affectation et calcul numérique
    nx = mesh%nx_local
    ! Affectation et calcul numérique
    ny = mesh%ny_local

    ! Évaluation de la condition logique
    if (.not. cfg%enable_two_phase) then
        ! Calcul de la masse volumique locale ou interpolée aux faces
        fields%rho = cfg%rho1
        ! Calcul de la masse volumique locale ou interpolée aux faces
        fields%rho_u = cfg%rho1
        ! Calcul de la masse volumique locale ou interpolée aux faces
        fields%rho_v = cfg%rho1
        ! Évaluation de la condition logique
        if (cfg%viscosity_model == VISC_EXPONENTIAL) then
            ! Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
            do j = 0, ny + 1
                ! Affectation et calcul numérique
                y = y_center(mesh, j)
                ! Affectation et calcul numérique
                z = y - cfg%visc_e
                ! Calcul de la viscosité dynamique locale (harmonique aux nœuds)
                mu_val = cfg%visc_mu0 * exp(-cfg%visc_a * z / cfg%visc_e)
                ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
                fields%mu(:, j) = mu_val
            end do
            ! Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
            do j = 0, ny + 1
                ! Affectation et calcul numérique
                y = y_face(mesh, j)
                ! Affectation et calcul numérique
                z = y - cfg%visc_e
```

```
            ! Calcul de la viscosité dynamique locale (harmonique aux nœuds)
            mu_val = cfg%visc_mu0 * exp(-cfg%visc_a * z / cfg%visc_e)
            ! Calcul de la viscosité dynamique locale (harmonique aux nœuds)
            fields%mu_node(:, j) = mu_val
         end do
      ! Branche par défaut si condition non satisfaite
      else
         ! Calcul de la viscosité dynamique locale (harmonique aux nœuds)
         fields%mu = cfg%mu1
         ! Calcul de la viscosité dynamique locale (harmonique aux nœuds)
         fields%mu_node = cfg%mu1
      end if
      ! Retour à la procédure appelante
      return
   end if

   ! Coins de C
   fields%C(0, 0)       = fields%C(1, 1)
   ! Transport / actualisation de la fraction volumique VOF
   fields%C(nx+1, 0)    = fields%C(nx, 1)
   ! Transport / actualisation de la fraction volumique VOF
   fields%C(0, ny+1)    = fields%C(1, ny)
   ! Transport / actualisation de la fraction volumique VOF
   fields%C(nx+1, ny+1) = fields%C(nx, ny)

   ! 1. Masse volumique arithmétique et viscosité au centre
   do j = 0, ny + 1
      ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
      do i = 0, nx + 1
         ! Affectation et calcul numérique
         c = max(0.0_dp, min(1.0_dp, fields%C(i, j)))
         ! Calcul de la masse volumique locale ou interpolée aux faces
         fields%rho(i, j) = c * cfg%rho1 + (1.0_dp - c) * cfg%rho2
         ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
         fields%mu(i, j)  = c * cfg%mu1  + (1.0_dp - c) * cfg%mu2
      end do
   end do

   ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
   fields%mu(0, 0)       = fields%mu(1, 1)
   ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
   fields%mu(nx+1, 0)    = fields%mu(nx, 1)
   ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
   fields%mu(0, ny+1)    = fields%mu(1, ny)
   ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
   fields%mu(nx+1, ny+1) = fields%mu(nx, ny)

   ! 2. Masse volumique sur les faces u
   do j = 1, ny
      ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
      do i = 1, nx + 1
         ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
         fields%rho_u(i, j) = 0.5_dp * (fields%rho(i - 1, j) + fields%rho(i, j))
      end do
   end do

   ! 3. Masse volumique sur les faces v
   do j = 1, ny + 1
      ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
      do i = 1, nx
         ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
         fields%rho_v(i, j) = 0.5_dp * (fields%rho(i, j - 1) + fields%rho(i, j))
      end do
   end do

   ! 4. Viscosité harmonique aux nœuds (coins de mailles)
   do j = 1, ny + 1
      ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
      do i = 1, nx + 1
         ! Affectation et calcul numérique
         m_sw = fields%mu(i-1, j-1)
         ! Affectation et calcul numérique
         m_se = fields%mu(i, j-1)
         ! Affectation et calcul numérique
         m_nw = fields%mu(i-1, j)
         ! Affectation et calcul numérique
         m_ne = fields%mu(i, j)
         ! Calcul de la viscosité dynamique locale (harmonique aux nœuds)
         fields%mu_node(i, j) = 4.0_dp / (1.0_dp / m_sw + 1.0_dp / m_se + 1.0_dp / m_nw + 1.0_dp / m_ne)
      end do
   end do
end subroutine update_physical_properties

! [Procédure/Module] pure real
pure real(dp) function superbee(r)
   ! Déclaration de variable locale / paramètre de calcul
   real(dp), intent(in) :: r
   ! Affectation et calcul numérique
   superbee = max(0.0_dp, max(min(2.0_dp * r, 1.0_dp), min(r, 2.0_dp)))
end function superbee

! [Procédure/Module] subroutine compute_curvature_and_csf
```

```
  subroutine compute_curvature_and_csf(fields, mesh, cfg)
    ! Déclaration de variable locale / paramètre de calcul
    type(flow_fields_t), intent(inout) :: fields
    ! Déclaration de variable locale / paramètre de calcul
    type(mesh_2d_t), intent(in) :: mesh
    ! Déclaration de variable locale / paramètre de calcul
    type(config_t), intent(in) :: cfg

    ! Déclaration de variable locale / paramètre de calcul
    integer :: i, j, nx, ny
    ! Déclaration de variable locale / paramètre de calcul
    real(dp) :: dx, dy, dc_dx, dc_dy, grad_mag, kap_avg, grad_c
    ! Déclaration de variable locale / paramètre de calcul
    real(dp), allocatable :: norm_x(:, :), norm_y(:, :)
    ! Déclaration de variable locale / paramètre de calcul
    real(dp), allocatable :: kappa(:, :)

    ! Évaluation de la condition logique
    if (.not. cfg%enable_surface_tension .or. cfg%sigma <= 0.0_dp) then
       ! Affectation et calcul numérique
       fields%fx_sigma = 0.0_dp
       ! Affectation et calcul numérique
       fields%fy_sigma = 0.0_dp
       ! Affectation et calcul numérique
       fields%curvature = 0.0_dp
       ! Retour à la procédure appelante
       return
    end if

    ! Affectation et calcul numérique
    nx = mesh%nx_local
    ! Affectation et calcul numérique
    ny = mesh%ny_local
    ! Calcul de la métrique spatiale élémentaire
    dx = mesh%dx
    ! Calcul de la métrique spatiale élémentaire
    dy = mesh%dy

    ! Allocation dynamique de tableau contigu en mémoire
    allocate(norm_x(0:nx+1, 0:ny+1), norm_y(0:nx+1, 0:ny+1))
    ! Allocation dynamique de tableau contigu en mémoire
    allocate(kappa(0:nx+1, 0:ny+1))
    ! Évaluation de la norme du résidu ou de l'erreur numérique
    norm_x = 0.0_dp
    ! Évaluation de la norme du résidu ou de l'erreur numérique
    norm_y = 0.0_dp
    ! Affectation et calcul numérique
    kappa = 0.0_dp

    ! 1. Gradient et normale unitaire aux centres
    do j = 1, ny
       ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
       do i = 1, nx
          ! Calcul de la métrique spatiale élémentaire
          dc_dx = (fields%C(i+1, j) - fields%C(i-1, j)) / (2.0_dp * dx)
          ! Calcul de la métrique spatiale élémentaire
          dc_dy = (fields%C(i, j+1) - fields%C(i, j-1)) / (2.0_dp * dy)
          ! Affectation et calcul numérique
          grad_mag = sqrt(dc_dx * dc_dx + dc_dy * dc_dy)
          ! Évaluation de la condition logique
          if (grad_mag > 1.0e-3_dp / dx) then
             ! Évaluation de la norme du résidu ou de l'erreur numérique
             norm_x(i, j) = dc_dx / grad_mag
             ! Évaluation de la norme du résidu ou de l'erreur numérique
             norm_y(i, j) = dc_dy / grad_mag
          end if
       end do
    end do

    ! Appel de sous-programme modulaire
    call exchange_halos_scalar(norm_x, mesh)
    ! Appel de sous-programme modulaire
    call exchange_halos_scalar(norm_y, mesh)

    ! 2. Courbure kappa = -div(n)
    do j = 1, ny
       ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
       do i = 1, nx
          ! Évaluation de la condition logique
          if (norm_x(i, j) /= 0.0_dp .or. norm_y(i, j) /= 0.0_dp) then
             ! Affectation et calcul numérique
             kappa(i, j) = - ((norm_x(i+1, j) - norm_x(i-1, j)) / (2.0_dp * dx) + &
                              (norm_y(i, j+1) - norm_y(i, j-1)) / (2.0_dp * dy))
          end if
       end do
    end do

    ! Appel de sous-programme modulaire
    call exchange_halos_scalar(kappa, mesh)
    ! Affectation et calcul numérique
    fields%curvature = kappa

```

```
    ! 3. Forces de tension superficielle CSF (Balanced-Force)
    do j = 1, ny
       ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
       do i = 1, nx + 1
          ! Affectation et calcul numérique
          kap_avg = 0.5_dp * (kappa(i-1, j) + kappa(i, j))
          ! Affectation et calcul numérique
          grad_c  = (fields%C(i, j) - fields%C(i-1, j)) / dx
          ! Affectation et calcul numérique
          fields%fx_sigma(i, j) = cfg%sigma * kap_avg * grad_c
       end do
    end do

    ! Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
    do j = 1, ny + 1
       ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
       do i = 1, nx
          ! Affectation et calcul numérique
          kap_avg = 0.5_dp * (kappa(i, j-1) + kappa(i, j))
          ! Affectation et calcul numérique
          grad_c  = (fields%C(i, j) - fields%C(i, j-1)) / dy
          ! Affectation et calcul numérique
          fields%fy_sigma(i, j) = cfg%sigma * kap_avg * grad_c
       end do
    end do

    ! Libération de la mémoire allouée
    deallocate(norm_x, norm_y, kappa)
  end subroutine compute_curvature_and_csf

  ! [Procédure/Module] subroutine advect_vof
  subroutine advect_vof(fields, mesh, cfg, dt)
    type(flow_fields_t), intent(inout) :: fields
    type(mesh_2d_t), intent(in) :: mesh
    type(config_t), intent(in) :: cfg
    real(dp), intent(in) :: dt

    select case (trim(cfg%advection_method))
    case ("CIAM")
       call advect_vof_ciam(fields, mesh, cfg, dt)
    case ("WEYMOUTH_YUE")
       call advect_vof_weymouth_yue(fields, mesh, cfg, dt)
    case default
       call advect_vof_tvd(fields, mesh, cfg, dt)
    end select

    vof_step_count = vof_step_count + 1

    call exchange_halos_scalar(fields%C, mesh)
    call apply_vof_bc(fields, mesh, cfg)
    call update_physical_properties(fields, mesh, cfg)
    call compute_curvature_and_csf(fields, mesh, cfg)
  end subroutine advect_vof

  ! [Procédure/Module] subroutine advect_vof_tvd
  subroutine advect_vof_tvd(fields, mesh, cfg, dt)
    type(flow_fields_t), intent(inout) :: fields
    type(mesh_2d_t), intent(in) :: mesh
    type(config_t), intent(in) :: cfg
    real(dp), intent(in) :: dt

    integer :: i, j, nx, ny, i_up, i_down, j_up, j_down
    real(dp) :: dx, dy, u_face, v_face, c_l, c_r, c_b, c_t, c_u
    real(dp) :: dc_up, dc_loc, r, phi, fx_in, fx_out, fy_in, fy_out
    real(dp), allocatable :: flux_x(:, :), flux_y(:, :), c_new(:, :)

    nx = mesh%nx_local
    ny = mesh%ny_local
    dx = mesh%dx
    dy = mesh%dy

    allocate(flux_x(0:nx+1, 0:ny+1), flux_y(0:nx+1, 0:ny+1))
    allocate(c_new(0:nx+1, 0:ny+1))
    flux_x = 0.0_dp
    flux_y = 0.0_dp
    c_new = fields%C

    ! Flux selon X
    do j = 1, ny
       do i = 1, nx + 1
          u_face = fields%u(i, j)
          c_l = fields%C(i-1, j)
          c_r = fields%C(i, j)

          if (u_face >= 0.0_dp) then
             i_up = max(0, i - 2)
             c_u = fields%C(i_up, j)
             dc_up = c_l - c_u
             dc_loc = c_r - c_l
             if (abs(dc_loc) > 1.0e-12_dp) then
                r = dc_up / dc_loc
             else
```

```
556                r = 0.0_dp
557             end if
558             phi = evaluate_limiter(cfg%tvd_limiter, r, c_l, c_u, c_r)
559             flux_x(i, j) = u_face * (c_l + 0.5_dp * phi * (1.0_dp - min(1.0_dp, abs(u_face) * dt / dx)) *
                    dc_loc)
560          else
561             i_down = min(nx + 1, i + 1)
562             c_u = fields%C(i_down, j)
563             dc_up = c_u - c_r
564             dc_loc = c_r - c_l
565             if (abs(dc_loc) > 1.0e-12_dp) then
566                r = dc_up / dc_loc
567             else
568                r = 0.0_dp
569             end if
570             phi = evaluate_limiter(cfg%tvd_limiter, r, c_r, c_u, c_l)
571             flux_x(i, j) = u_face * (c_r - 0.5_dp * phi * (1.0_dp - min(1.0_dp, abs(u_face) * dt / dx)) *
                    dc_loc)
572          end if
573       end do
574    end do
575
576    ! Flux selon Y
577    do j = 1, ny + 1
578       do i = 1, nx
579          v_face = fields%v(i, j)
580          c_b = fields%C(i, j-1)
581          c_t = fields%C(i, j)
582
583          if (v_face >= 0.0_dp) then
584             j_up = max(0, j - 2)
585             c_u = fields%C(i, j_up)
586             dc_up = c_b - c_u
587             dc_loc = c_t - c_b
588             if (abs(dc_loc) > 1.0e-12_dp) then
589                r = dc_up / dc_loc
590             else
591                r = 0.0_dp
592             end if
593             phi = evaluate_limiter(cfg%tvd_limiter, r, c_b, c_u, c_t)
594             flux_y(i, j) = v_face * (c_b + 0.5_dp * phi * (1.0_dp - min(1.0_dp, abs(v_face) * dt / dy)) *
                    dc_loc)
595          else
596             j_down = min(ny + 1, j + 1)
597             c_u = fields%C(i, j_down)
598             dc_up = c_u - c_t
599             dc_loc = c_t - c_b
600             if (abs(dc_loc) > 1.0e-12_dp) then
601                r = dc_up / dc_loc
602             else
603                r = 0.0_dp
604             end if
605             phi = evaluate_limiter(cfg%tvd_limiter, r, c_t, c_u, c_b)
606             flux_y(i, j) = v_face * (c_t - 0.5_dp * phi * (1.0_dp - min(1.0_dp, abs(v_face) * dt / dy)) *
                    dc_loc)
607          end if
608       end do
609    end do
610
611    ! Mise à jour conservative
612    do j = 1, ny
613       do i = 1, nx
614          fx_in  = flux_x(i, j)
615          fx_out = flux_x(i+1, j)
616          fy_in  = flux_y(i, j)
617          fy_out = flux_y(i, j+1)
618
619          c_new(i, j) = fields%C(i, j) - (dt / dx) * (fx_out - fx_in) - (dt / dy) * (fy_out - fy_in)
620          c_new(i, j) = max(0.0_dp, min(1.0_dp, c_new(i, j)))
621       end do
622    end do
623
624    fields%C = c_new
625    deallocate(flux_x, flux_y, c_new)
626  end subroutine advect_vof_tvd
627
628  ! [Procédure/Module] subroutine advect_vof_ciam
629  subroutine advect_vof_ciam(fields, mesh, cfg, dt)
630    type(flow_fields_t), intent(inout) :: fields
631    type(mesh_2d_t), intent(in) :: mesh
632    type(config_t), intent(in) :: cfg
633    real(dp), intent(in) :: dt
634
635    integer :: i, j, nx, ny, donor_idx
636    real(dp) :: dx, dy, delta_x, delta_y, u_face, v_face
637    real(dp) :: x_pos, y_pos, b_xmin, b_xmax, b_ymin, b_ymax, denom
638    type(interface_segment_t), allocatable :: segments(:, :)
639    real(dp), allocatable :: flux_vol(:, :), c_star(:, :), c_final(:, :)
640    logical :: x_first
641
642    nx = mesh%nx_local
643    ny = mesh%ny_local
```

```
    dx = mesh%dx
    dy = mesh%dy

    allocate(segments(0:nx+1, 0:ny+1))
    allocate(flux_vol(0:nx+1, 0:ny+1))
    allocate(c_star(0:nx+1, 0:ny+1))
    allocate(c_final(0:nx+1, 0:ny+1))

    x_first = (mod(vof_step_count, 2) == 0)

    if (x_first) then
       ! 1. Passe X
       call reconstruct_interface(cfg%reconstruction_method, mesh, fields%C, segments)
       flux_vol = 0.0_dp
       do j = 1, ny
          do i = 1, nx + 1
             u_face = fields%u(i, j)
             delta_x = abs(u_face) * dt
             if (delta_x <= 1.0e-14_dp) cycle
             x_pos = x_face(mesh, i)
             if (u_face > 0.0_dp) then
                b_xmin = x_pos - delta_x; b_xmax = x_pos
                b_ymin = y_face(mesh, j); b_ymax = y_face(mesh, j + 1)
                flux_vol(i, j) = cut_box_area(b_xmin, b_xmax, b_ymin, b_ymax, &
                                              segments(i-1, j)%nx, segments(i-1, j)%ny, segments(i-1, j)%alpha)
             else
                b_xmin = x_pos; b_xmax = x_pos + delta_x
                b_ymin = y_face(mesh, j); b_ymax = y_face(mesh, j + 1)
                flux_vol(i, j) = -cut_box_area(b_xmin, b_xmax, b_ymin, b_ymax, &
                                               segments(i, j)%nx, segments(i, j)%ny, segments(i, j)%alpha)
             end if
          end do
       end do

       c_star = fields%C
       do j = 1, ny
          do i = 1, nx
             denom = dx - dt * (fields%u(i + 1, j) - fields%u(i, j))
             if (denom > 1.0e-12_dp) then
                c_star(i, j) = (fields%C(i, j) * dx - (flux_vol(i + 1, j) - flux_vol(i, j)) / dy) / denom
             else
                c_star(i, j) = fields%C(i, j) - (flux_vol(i + 1, j) - flux_vol(i, j)) / (dx * dy)
             end if
             c_star(i, j) = max(0.0_dp, min(1.0_dp, c_star(i, j)))
          end do
       end do
       call exchange_halos_scalar(c_star, mesh)

       ! 2. Passe Y
       call reconstruct_interface(cfg%reconstruction_method, mesh, c_star, segments)
       flux_vol = 0.0_dp
       do j = 1, ny + 1
          do i = 1, nx
             v_face = fields%v(i, j)
             delta_y = abs(v_face) * dt
             if (delta_y <= 1.0e-14_dp) cycle
             y_pos = y_face(mesh, j)
             if (v_face > 0.0_dp) then
                b_xmin = x_face(mesh, i); b_xmax = x_face(mesh, i + 1)
                b_ymin = y_pos - delta_y; b_ymax = y_pos
                flux_vol(i, j) = cut_box_area(b_xmin, b_xmax, b_ymin, b_ymax, &
                                              segments(i, j-1)%nx, segments(i, j-1)%ny, segments(i, j-1)%alpha)
             else
                b_xmin = x_face(mesh, i); b_xmax = x_face(mesh, i + 1)
                b_ymin = y_pos; b_ymax = y_pos + delta_y
                flux_vol(i, j) = -cut_box_area(b_xmin, b_xmax, b_ymin, b_ymax, &
                                               segments(i, j)%nx, segments(i, j)%ny, segments(i, j)%alpha)
             end if
          end do
       end do

       c_final = c_star
       do j = 1, ny
          do i = 1, nx
             denom = dy - dt * (fields%v(i, j + 1) - fields%v(i, j))
             if (denom > 1.0e-12_dp) then
                c_final(i, j) = (c_star(i, j) * dy - (flux_vol(i, j + 1) - flux_vol(i, j)) / dx) / denom
             else
                c_final(i, j) = c_star(i, j) - (flux_vol(i, j + 1) - flux_vol(i, j)) / (dx * dy)
             end if
             c_final(i, j) = max(0.0_dp, min(1.0_dp, c_final(i, j)))
          end do
       end do
    else
       ! 1. Passe Y
       call reconstruct_interface(cfg%reconstruction_method, mesh, fields%C, segments)
       flux_vol = 0.0_dp
       do j = 1, ny + 1
          do i = 1, nx
             v_face = fields%v(i, j)
             delta_y = abs(v_face) * dt
             if (delta_y <= 1.0e-14_dp) cycle
```

```
            y_pos = y_face(mesh, j)
            if (v_face > 0.0_dp) then
               b_xmin = x_face(mesh, i); b_xmax = x_face(mesh, i + 1)
               b_ymin = y_pos - delta_y; b_ymax = y_pos
               flux_vol(i, j) = cut_box_area(b_xmin, b_xmax, b_ymin, b_ymax, &
                                             segments(i, j-1)%nx, segments(i, j-1)%ny, segments(i, j-1)%alpha)
            else
               b_xmin = x_face(mesh, i); b_xmax = x_face(mesh, i + 1)
               b_ymin = y_pos; b_ymax = y_pos + delta_y
               flux_vol(i, j) = -cut_box_area(b_xmin, b_xmax, b_ymin, b_ymax, &
                                              segments(i, j)%nx, segments(i, j)%ny, segments(i, j)%alpha)
            end if
         end do
      end do

      c_star = fields%C
      do j = 1, ny
         do i = 1, nx
            denom = dy - dt * (fields%v(i, j + 1) - fields%v(i, j))
            if (denom > 1.0e-12_dp) then
               c_star(i, j) = (fields%C(i, j) * dy - (flux_vol(i, j + 1) - flux_vol(i, j)) / dx) / denom
            else
               c_star(i, j) = fields%C(i, j) - (flux_vol(i, j + 1) - flux_vol(i, j)) / (dx * dy)
            end if
            c_star(i, j) = max(0.0_dp, min(1.0_dp, c_star(i, j)))
         end do
      end do
      call exchange_halos_scalar(c_star, mesh)

      ! 2. Passe X
      call reconstruct_interface(cfg%reconstruction_method, mesh, c_star, segments)
      flux_vol = 0.0_dp
      do j = 1, ny
         do i = 1, nx + 1
            u_face = fields%u(i, j)
            delta_x = abs(u_face) * dt
            if (delta_x <= 1.0e-14_dp) cycle
            x_pos = x_face(mesh, i)
            if (u_face > 0.0_dp) then
               b_xmin = x_pos - delta_x; b_xmax = x_pos
               b_ymin = y_face(mesh, j); b_ymax = y_face(mesh, j + 1)
               flux_vol(i, j) = cut_box_area(b_xmin, b_xmax, b_ymin, b_ymax, &
                                             segments(i-1, j)%nx, segments(i-1, j)%ny, segments(i-1, j)%alpha)
            else
               b_xmin = x_pos; b_xmax = x_pos + delta_x
               b_ymin = y_face(mesh, j); b_ymax = y_face(mesh, j + 1)
               flux_vol(i, j) = -cut_box_area(b_xmin, b_xmax, b_ymin, b_ymax, &
                                              segments(i, j)%nx, segments(i, j)%ny, segments(i, j)%alpha)
            end if
         end do
      end do

      c_final = c_star
      do j = 1, ny
         do i = 1, nx
            denom = dx - dt * (fields%u(i + 1, j) - fields%u(i, j))
            if (denom > 1.0e-12_dp) then
               c_final(i, j) = (c_star(i, j) * dx - (flux_vol(i + 1, j) - flux_vol(i, j)) / dy) / denom
            else
               c_final(i, j) = c_star(i, j) - (flux_vol(i + 1, j) - flux_vol(i, j)) / (dx * dy)
            end if
            c_final(i, j) = max(0.0_dp, min(1.0_dp, c_final(i, j)))
         end do
      end do
   end if

   fields%C = c_final
   deallocate(segments, flux_vol, c_star, c_final)
end subroutine advect_vof_ciam

! [Procédure/Module] subroutine advect_vof_weymouth_yue
subroutine advect_vof_weymouth_yue(fields, mesh, cfg, dt)
   type(flow_fields_t), intent(inout) :: fields
   type(mesh_2d_t), intent(in) :: mesh
   type(config_t), intent(in) :: cfg
   real(dp), intent(in) :: dt

   integer :: i, j, nx, ny
   real(dp) :: dx, dy, cell_area, u_face, v_face, delta_x, delta_y
   real(dp) :: x_pos, y_pos, b_xmin, b_xmax, b_ymin, b_ymax
   logical :: x_first
   type(interface_segment_t), allocatable :: segments(:, :)
   real(dp), allocatable :: flux_vol(:, :), c_star(:, :), c_final(:, :), c_d(:, :)

   nx = mesh%nx_local
   ny = mesh%ny_local
   dx = mesh%dx
   dy = mesh%dy
   cell_area = dx * dy
   x_first = (mod(vof_step_count, 2) == 0)

   allocate(segments(0:nx+1, 0:ny+1))
```

```
    allocate(flux_vol(0:nx+1, 0:ny+1))
    allocate(c_star(0:nx+1, 0:ny+1))
    allocate(c_final(0:nx+1, 0:ny+1))
    allocate(c_d(0:nx+1, 0:ny+1))

    ! Fonction indicatrice binaire de dilatation figée au temps n (Weymouth & Yue, JCP 2010, Eq. 14)
    do j = 0, ny + 1
       do i = 0, nx + 1
          if (fields%C(i, j) >= 0.5_dp) then
             c_d(i, j) = 1.0_dp
          else
             c_d(i, j) = 0.0_dp
          end if
       end do
    end do

    if (x_first) then
       ! 1. Passe X (Eq. 15 Weymouth & Yue 2010)
       call reconstruct_interface(cfg%reconstruction_method, mesh, fields%C, segments)
       flux_vol = 0.0_dp
       do j = 1, ny
          do i = 1, nx + 1
             u_face = fields%u(i, j)
             delta_x = abs(u_face) * dt
             if (delta_x <= 1.0e-14_dp) cycle
             x_pos = x_face(mesh, i)
             if (u_face > 0.0_dp) then
                b_xmin = x_pos - delta_x; b_xmax = x_pos
                b_ymin = y_face(mesh, j); b_ymax = y_face(mesh, j + 1)
                flux_vol(i, j) = cut_box_area(b_xmin, b_xmax, b_ymin, b_ymax, &
                                              segments(i-1, j)%nx, segments(i-1, j)%ny, segments(i-1, j)%alpha)
             else
                b_xmin = x_pos; b_xmax = x_pos + delta_x
                b_ymin = y_face(mesh, j); b_ymax = y_face(mesh, j + 1)
                flux_vol(i, j) = -cut_box_area(b_xmin, b_xmax, b_ymin, b_ymax, &
                                               segments(i, j)%nx, segments(i, j)%ny, segments(i, j)%alpha)
             end if
          end do
       end do

       c_star = fields%C
       do j = 1, ny
          do i = 1, nx
             c_star(i, j) = fields%C(i, j) - (flux_vol(i + 1, j) - flux_vol(i, j)) / cell_area + &
                            c_d(i, j) * (dt / dx) * (fields%u(i + 1, j) - fields%u(i, j))
          end do
       end do
       call exchange_halos_scalar(c_star, mesh)

       ! 2. Passe Y (Eq. 16 Weymouth & Yue 2010)
       call reconstruct_interface(cfg%reconstruction_method, mesh, c_star, segments)
       flux_vol = 0.0_dp
       do j = 1, ny + 1
          do i = 1, nx
             v_face = fields%v(i, j)
             delta_y = abs(v_face) * dt
             if (delta_y <= 1.0e-14_dp) cycle
             y_pos = y_face(mesh, j)
             if (v_face > 0.0_dp) then
                b_xmin = x_face(mesh, i); b_xmax = x_face(mesh, i + 1)
                b_ymin = y_pos - delta_y; b_ymax = y_pos
                flux_vol(i, j) = cut_box_area(b_xmin, b_xmax, b_ymin, b_ymax, &
                                              segments(i, j-1)%nx, segments(i, j-1)%ny, segments(i, j-1)%alpha)
             else
                b_xmin = x_face(mesh, i); b_xmax = x_face(mesh, i + 1)
                b_ymin = y_pos; b_ymax = y_pos + delta_y
                flux_vol(i, j) = -cut_box_area(b_xmin, b_xmax, b_ymin, b_ymax, &
                                               segments(i, j)%nx, segments(i, j)%ny, segments(i, j)%alpha)
             end if
          end do
       end do

       c_final = c_star
       do j = 1, ny
          do i = 1, nx
             c_final(i, j) = c_star(i, j) - (flux_vol(i, j + 1) - flux_vol(i, j)) / cell_area + &
                             c_d(i, j) * (dt / dy) * (fields%v(i, j + 1) - fields%v(i, j))
             c_final(i, j) = max(0.0_dp, min(1.0_dp, c_final(i, j)))
          end do
       end do
    else
       ! 1. Passe Y (Eq. 16 Weymouth & Yue 2010)
       call reconstruct_interface(cfg%reconstruction_method, mesh, fields%C, segments)
       flux_vol = 0.0_dp
       do j = 1, ny + 1
          do i = 1, nx
             v_face = fields%v(i, j)
             delta_y = abs(v_face) * dt
             if (delta_y <= 1.0e-14_dp) cycle
             y_pos = y_face(mesh, j)
             if (v_face > 0.0_dp) then
                b_xmin = x_face(mesh, i); b_xmax = x_face(mesh, i + 1)
```

```
                b_ymin = y_pos - delta_y; b_ymax = y_pos
                flux_vol(i, j) = cut_box_area(b_xmin, b_xmax, b_ymin, b_ymax, &
                                              segments(i, j-1)%nx, segments(i, j-1)%ny, segments(i, j-1)%alpha)
             else
                b_xmin = x_face(mesh, i); b_xmax = x_face(mesh, i + 1)
                b_ymin = y_pos; b_ymax = y_pos + delta_y
                flux_vol(i, j) = -cut_box_area(b_xmin, b_xmax, b_ymin, b_ymax, &
                                               segments(i, j)%nx, segments(i, j)%ny, segments(i, j)%alpha)
             end if
          end do
       end do

       c_star = fields%C
       do j = 1, ny
          do i = 1, nx
             c_star(i, j) = fields%C(i, j) - (flux_vol(i, j + 1) - flux_vol(i, j)) / cell_area + &
                            c_d(i, j) * (dt / dy) * (fields%v(i, j + 1) - fields%v(i, j))
          end do
       end do
       call exchange_halos_scalar(c_star, mesh)

       ! 2. Passe X (Eq. 15 Weymouth & Yue 2010)
       call reconstruct_interface(cfg%reconstruction_method, mesh, c_star, segments)
       flux_vol = 0.0_dp
       do j = 1, ny
          do i = 1, nx + 1
             u_face = fields%u(i, j)
             delta_x = abs(u_face) * dt
             if (delta_x <= 1.0e-14_dp) cycle
             x_pos = x_face(mesh, i)
             if (u_face > 0.0_dp) then
                b_xmin = x_pos - delta_x; b_xmax = x_pos
                b_ymin = y_face(mesh, j); b_ymax = y_face(mesh, j + 1)
                flux_vol(i, j) = cut_box_area(b_xmin, b_xmax, b_ymin, b_ymax, &
                                              segments(i-1, j)%nx, segments(i-1, j)%ny, segments(i-1, j)%alpha)
             else
                b_xmin = x_pos; b_xmax = x_pos + delta_x
                b_ymin = y_face(mesh, j); b_ymax = y_face(mesh, j + 1)
                flux_vol(i, j) = -cut_box_area(b_xmin, b_xmax, b_ymin, b_ymax, &
                                               segments(i, j)%nx, segments(i, j)%ny, segments(i, j)%alpha)
             end if
          end do
       end do

       c_final = c_star
       do j = 1, ny
          do i = 1, nx
             c_final(i, j) = c_star(i, j) - (flux_vol(i + 1, j) - flux_vol(i, j)) / cell_area + &
                             c_d(i, j) * (dt / dx) * (fields%u(i + 1, j) - fields%u(i, j))
             c_final(i, j) = max(0.0_dp, min(1.0_dp, c_final(i, j)))
          end do
       end do
    end if

    fields%C = c_final
    deallocate(segments, flux_vol, c_star, c_final, c_d)
  end subroutine advect_vof_weymouth_yue

  ! [Procédure/Module] subroutine vof_init
  subroutine vof_init(this, cfg)
    ! Déclaration de variable locale / paramètre de calcul
    class(interface_vof_t), intent(inout) :: this
    ! Déclaration de variable locale / paramètre de calcul
    type(config_t), intent(in) :: cfg
    ! Affectation et calcul numérique
    this%cfg = cfg
  end subroutine vof_init

  ! [Procédure/Module] subroutine vof_set_ic
  subroutine vof_set_ic(this, mesh, fields)
    ! Déclaration de variable locale / paramètre de calcul
    class(interface_vof_t), intent(in) :: this
    ! Déclaration de variable locale / paramètre de calcul
    type(mesh_2d_t), intent(in) :: mesh
    ! Déclaration de variable locale / paramètre de calcul
    type(flow_fields_t), intent(inout) :: fields
    ! Appel de sous-programme modulaire
    call set_initial_conditions(fields, mesh, this%cfg)
    ! Appel de sous-programme modulaire
    call update_physical_properties(fields, mesh, this%cfg)
    ! Appel de sous-programme modulaire
    call compute_curvature_and_csf(fields, mesh, this%cfg)
  end subroutine vof_set_ic

  ! [Procédure/Module] subroutine vof_advect_step
  subroutine vof_advect_step(this, mesh, fields, dt)
    ! Déclaration de variable locale / paramètre de calcul
    class(interface_vof_t), intent(in) :: this
    ! Déclaration de variable locale / paramètre de calcul
    type(mesh_2d_t), intent(in) :: mesh
    ! Déclaration de variable locale / paramètre de calcul
    type(flow_fields_t), intent(inout) :: fields
```

```
1012      ! Déclaration de variable locale / paramètre de calcul
1013      real(dp), intent(in) :: dt
1014      ! Appel de sous-programme modulaire
1015      call advect_vof(fields, mesh, this%cfg, dt)
1016      ! Appel de sous-programme modulaire
1017      call update_physical_properties(fields, mesh, this%cfg)
1018      ! Appel de sous-programme modulaire
1019      call compute_curvature_and_csf(fields, mesh, this%cfg)
1020    end subroutine vof_advect_step
1021
1022  end module module_interface_vof
```

Chapitre 22

Générateur VTK XML Parallèle en Fortran 2018 (module$_{vtk_writer.f90}$)

22.1 Écriture Conforme des Fichiers `.vtr` et `.pvtr` (`module_vtk_writer.f90`)

Le module `src/module_vtk_writer.f90` (500 lignes) écrit les fichiers XML VTK lisibles sous ParaView et Python. Une attention particulière est portée au formatage des flottants IEEE-754 : en Fortran, le descripteur standard `ES15.7` omet parfois la lettre `E` lorsque l'exposant comporte trois chiffres (par exemple `1.2345678-105` lorsque $|x| < 10^{-99}$). L'utilisation du descripteur explicite à trois chiffres d'exposant `ES16.8E3` dans `module_vtk_writer.f90` garantit que la lettre `E` est toujours présente, assurant une compatibilité universelle avec tous les parseurs XML VTK.

22.2 Répertoire des Types Dérivés, Sous-Routines et Fonctions de `src/module_vtk_writer.f90`

Ligne	Nature	Identifiant Fortran 2018	Interface / Arguments
10	module	`module_vtk_writer`	Unité modulaire Fortran 2018
21	type dérivé	`vtk_writer_t`	Structure de données Fortran 2018
46	subroutine	`vtk_init`	Arguments : (this, mesh, out_dir, name)
89	subroutine	`vtk_write_step`	Arguments : (this, mesh, fields, step, time)
368	subroutine	`vtk_write_diagnostics`	Arguments : (this, mesh, fields, step, time, dt)
489	subroutine	`vtk_finalize`	Arguments : (this)

22.3 Code Source Intégral Annoté : `src/module_vtk_writer.f90`

Le listing ci-dessous présente l'intégralité des 500 lignes du fichier `src/module_vtk_writer.f90` de l'implémentation Fortran 2018 :

```
!===============================================================================
! Module: module_vtk_writer
! Description: Exportateur de maillage rectiligne XML VTK parallèle (.pvtr / .vtr)
!              et enregistreur de séries temporelles CSV pour COVOF 2D.
!              - Écriture locale non-bloquante du fichier .vtr sur chaque processus
!              - Assemblage du fichier maître .pvtr sur le rang racine
!              - Diagnostics diphasiques : position du centre de masse, vitesse
!                d'ascension, volume de bulle, énergie cinétique, divergence max
!===============================================================================
module module_vtk_writer
  use module_precision, only: dp
  use module_mesh, only: mesh_2d_t, x_center, y_center
  use module_fields, only: flow_fields_t
  use module_mpi_comm, only: global_sum, global_max
  implicit none
  private
  include 'mpif.h'

  public :: vtk_writer_t

  type :: vtk_writer_t
     ! Déclaration de variable locale / paramètre de calcul
     character(len=256) :: output_dir = "output"
     ! Déclaration de variable locale / paramètre de calcul
     character(len=128) :: case_name  = "covof2d_case"
     ! Déclaration de variable locale / paramètre de calcul
     integer            :: csv_unit   = 0
```

```
    ! Déclaration de variable locale / paramètre de calcul
    logical            :: is_open    = .false.
  contains
    ! Affectation et calcul numérique
    procedure :: init => vtk_init
    ! Affectation et calcul numérique
    procedure :: write_step => vtk_write_step
    ! Affectation et calcul numérique
    procedure :: write_diagnostics => vtk_write_diagnostics
    ! Affectation et calcul numérique
    procedure :: finalize => vtk_finalize
  end type vtk_writer_t

contains

  !---------------------------------------------------------------------------
  ! Initialisation du répertoire de sortie et du fichier de métriques CSV
  !---------------------------------------------------------------------------
  subroutine vtk_init(this, mesh, out_dir, name)
    ! Déclaration de variable locale / paramètre de calcul
    class(vtk_writer_t), intent(inout) :: this
    ! Déclaration de variable locale / paramètre de calcul
    type(mesh_2d_t), intent(in)        :: mesh
    ! Déclaration de variable locale / paramètre de calcul
    character(len=*), intent(in)       :: out_dir, name

    ! Déclaration de variable locale / paramètre de calcul
    character(len=512) :: csv_path, mkdir_cmd
    ! Déclaration de variable locale / paramètre de calcul
    integer :: io_stat

    ! Affectation et calcul numérique
    this%output_dir = trim(out_dir)
    ! Affectation et calcul numérique
    this%case_name  = trim(name)

    ! Évaluation de la condition logique
    if (mesh%rank == 0) then
       ! Affectation et calcul numérique
       mkdir_cmd = "mkdir -p " // trim(this%output_dir)
       ! Appel de sous-programme modulaire
       call execute_command_line(trim(mkdir_cmd))

       ! Affectation et calcul numérique
       csv_path = trim(this%output_dir) // "/" // trim(this%case_name) // "_history.csv"
       ! Affectation et calcul numérique
       open(newunit=this%csv_unit, file=trim(csv_path), status="replace", action="write", iostat=io_stat)
       ! Évaluation de la condition logique
       if (io_stat == 0) then
          ! Affectation et calcul numérique
          this%is_open = .true.
          write(this%csv_unit, '(A)') &
            "step,time,dt,center_of_mass_y,rise_velocity,bubble_volume,kinetic_energy,max_u,max_div"
          flush(this%csv_unit)
       end if
    end if
  end subroutine vtk_init

  !---------------------------------------------------------------------------
  ! Écriture d'un pas de temps complet en format XML RectilinearGrid (.pvtr / .vtr)
  !---------------------------------------------------------------------------
  subroutine vtk_write_step(this, mesh, fields, step, time)
    ! Déclaration de variable locale / paramètre de calcul
    class(vtk_writer_t), intent(in) :: this
    ! Déclaration de variable locale / paramètre de calcul
    type(mesh_2d_t), intent(in)     :: mesh
    ! Déclaration de variable locale / paramètre de calcul
    type(flow_fields_t), intent(in) :: fields
    ! Déclaration de variable locale / paramètre de calcul
    integer, intent(in)             :: step
    ! Déclaration de variable locale / paramètre de calcul
    real(dp), intent(in)            :: time

    ! Déclaration de variable locale / paramètre de calcul
    character(len=6)   :: step_str
    ! Déclaration de variable locale / paramètre de calcul
    character(len=512) :: vtr_file, pvtr_file, piece_str
    ! Déclaration de variable locale / paramètre de calcul
    integer :: x_start, x_end, y_start, y_end
    ! Déclaration de variable locale / paramètre de calcul
    integer :: vtr_unit, pvtr_unit, io_stat, i, j, p, ierr, dummy(1)
    ! Déclaration de variable locale / paramètre de calcul
    real(dp) :: uc, vc, div_val
    ! Déclaration de variable locale / paramètre de calcul
    integer, allocatable :: all_x_start(:), all_x_end(:), all_y_start(:), all_y_end(:)

    write(step_str, '(I6.6)') step

    ! Affectation et calcul numérique
    x_start = mesh%i_start_global - 1
    ! Affectation et calcul numérique
    x_end   = x_start + mesh%nx_local
```

```
120     ! Affectation et calcul numérique
121     y_start = mesh%j_start_global - 1
122     ! Affectation et calcul numérique
123     y_end   = y_start + mesh%ny_local
124 
125     ! 1. Écriture du fichier .vtr propre à chaque processus MPI
126     write(vtr_file, '(A,"/",A,"_p",I0,"_",A,".vtr")') &
127       trim(this%output_dir), trim(this%case_name), mesh%rank, step_str
128 
129     ! Affectation et calcul numérique
130     open(newunit=vtr_unit, file=trim(vtr_file), status="replace", action="write", iostat=io_stat)
131     ! Évaluation de la condition logique
132     if (io_stat == 0) then
133        ! Affectation et calcul numérique
134        write(vtr_unit, '(A)') '<?xml version="1.0"?>'
135        ! Affectation et calcul numérique
136        write(vtr_unit, '(A)') '<VTKFile type="RectilinearGrid" version="0.1" byte_order="LittleEndian">'
137        write(vtr_unit, '(A,I0,A,I0,A,I0,A,I0,A)') &
138          ! Affectation et calcul numérique
139          '  <RectilinearGrid WholeExtent="', x_start, ' ', x_end, ' ', y_start, ' ', y_end, ' 0 0">'
140        write(vtr_unit, '(A,I0,A,I0,A,I0,A,I0,A)') &
141          ! Affectation et calcul numérique
142          '    <Piece Extent="', x_start, ' ', x_end, ' ', y_start, ' ', y_end, ' 0 0">'
143        ! Affectation et calcul numérique
144        write(vtr_unit, '(A)') '      <CellData Scalars="PhaseIndicator,Pressure,Density,Viscosity,Divergence"
               Vectors="Velocity">'
145 
146        ! PhaseIndicator C
147        write(vtr_unit, '(A)') '        <DataArray type="Float64" Name="PhaseIndicator" format="ascii">'
148        ! Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
149        do j = 1, mesh%ny_local
150           ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
151           do i = 1, mesh%nx_local
152              ! Affectation et calcul numérique
153              write(vtr_unit, '(ES16.8," ")', advance="no") fields%C(i, j)
154           end do
155           write(vtr_unit, *)
156        end do
157        write(vtr_unit, '(A)') '        </DataArray>'
158 
159        ! Pressure p
160        write(vtr_unit, '(A)') '        <DataArray type="Float64" Name="Pressure" format="ascii">'
161        ! Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
162        do j = 1, mesh%ny_local
163           ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
164           do i = 1, mesh%nx_local
165              ! Affectation et calcul numérique
166              write(vtr_unit, '(ES16.8," ")', advance="no") fields%p(i, j)
167           end do
168           write(vtr_unit, *)
169        end do
170        write(vtr_unit, '(A)') '        </DataArray>'
171 
172        ! Density rho
173        write(vtr_unit, '(A)') '        <DataArray type="Float64" Name="Density" format="ascii">'
174        ! Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
175        do j = 1, mesh%ny_local
176           ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
177           do i = 1, mesh%nx_local
178              ! Affectation et calcul numérique
179              write(vtr_unit, '(ES16.8," ")', advance="no") fields%rho(i, j)
180           end do
181           write(vtr_unit, *)
182        end do
183        write(vtr_unit, '(A)') '        </DataArray>'
184 
185        ! Viscosity mu
186        write(vtr_unit, '(A)') '        <DataArray type="Float64" Name="Viscosity" format="ascii">'
187        ! Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
188        do j = 1, mesh%ny_local
189           ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
190           do i = 1, mesh%nx_local
191              ! Affectation et calcul numérique
192              write(vtr_unit, '(ES16.8," ")', advance="no") fields%mu(i, j)
193           end do
194           write(vtr_unit, *)
195        end do
196        write(vtr_unit, '(A)') '        </DataArray>'
197 
198        ! Divergence div(u)
199        write(vtr_unit, '(A)') '        <DataArray type="Float64" Name="Divergence" format="ascii">'
200        ! Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
201        do j = 1, mesh%ny_local
202           ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
203           do i = 1, mesh%nx_local
204              ! Affectation et calcul numérique
205              div_val = (fields%u(i + 1, j) - fields%u(i, j)) / mesh%dx + &
206                        (fields%v(i, j + 1) - fields%v(i, j)) / mesh%dy
207              ! Affectation et calcul numérique
208              write(vtr_unit, '(ES16.8," ")', advance="no") div_val
209           end do
210           write(vtr_unit, *)
```

```
        end do
        write(vtr_unit, '(A)') '        </DataArray>'

        ! Temperature T
        write(vtr_unit, '(A)') '        <DataArray type="Float64" Name="Temperature" format="ascii">'
        ! Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
        do j = 1, mesh%ny_local
           ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
           do i = 1, mesh%nx_local
              ! Affectation et calcul numérique
              write(vtr_unit, '(ES16.8," ")', advance="no") fields%T(i, j)
           end do
           write(vtr_unit, *)
        end do
        write(vtr_unit, '(A)') '        </DataArray>'

        ! Velocity Vector (au centre des mailles)
        write(vtr_unit, '(A)') '        <DataArray type="Float64" Name="Velocity" NumberOfComponents="3" format
            ="ascii">'
        ! Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
        do j = 1, mesh%ny_local
           ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
           do i = 1, mesh%nx_local
              ! Affectation et calcul numérique
              uc = 0.5_dp * (fields%u(i, j) + fields%u(i + 1, j))
              ! Affectation et calcul numérique
              vc = 0.5_dp * (fields%v(i, j) + fields%v(i, j + 1))
              ! Affectation et calcul numérique
              write(vtr_unit, '(ES16.8," ",ES16.8," 0.0 ")', advance="no") uc, vc
           end do
           write(vtr_unit, *)
        end do
        write(vtr_unit, '(A)') '        </DataArray>'

        write(vtr_unit, '(A)') '      </CellData>'
        write(vtr_unit, '(A)') '      <Coordinates>'

        ! Coordonnées X
        write(vtr_unit, '(A)') '        <DataArray type="Float64" Name="XCoordinates" format="ascii">'
        ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
        do i = 0, mesh%nx_local
           ! Affectation et calcul numérique
           write(vtr_unit, '(ES16.8," ")', advance="no") real(mesh%i_start_global - 1 + i, dp) * mesh%dx
        end do
        write(vtr_unit, *)
        write(vtr_unit, '(A)') '        </DataArray>'

        ! Coordonnées Y
        write(vtr_unit, '(A)') '        <DataArray type="Float64" Name="YCoordinates" format="ascii">'
        ! Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
        do j = 0, mesh%ny_local
           ! Affectation et calcul numérique
           write(vtr_unit, '(ES16.8," ")', advance="no") real(mesh%j_start_global - 1 + j, dp) * mesh%dy
        end do
        write(vtr_unit, *)
        write(vtr_unit, '(A)') '        </DataArray>'

        ! Coordonnées Z
        write(vtr_unit, '(A)') '        <DataArray type="Float64" Name="ZCoordinates" format="ascii">'
        write(vtr_unit, '(A)') '          0.0'
        write(vtr_unit, '(A)') '        </DataArray>'

        write(vtr_unit, '(A)') '      </Coordinates>'
        write(vtr_unit, '(A)') '    </Piece>'
        write(vtr_unit, '(A)') '  </RectilinearGrid>'
        write(vtr_unit, '(A)') '</VTKFile>'
        close(vtr_unit)
    end if

    ! 2. Écriture du fichier maître parallèle .pvtr sur le rang 0
    if (mesh%rank == 0) then
       ! Allocation dynamique de tableau contigu en mémoire
       allocate(all_x_start(mesh%size), all_x_end(mesh%size))
       ! Allocation dynamique de tableau contigu en mémoire
       allocate(all_y_start(mesh%size), all_y_end(mesh%size))

       ! Appel à la bibliothèque de communication parallèle MPI
       call MPI_Gather(x_start, 1, MPI_INTEGER, all_x_start, 1, MPI_INTEGER, 0, mesh%comm, ierr)
       ! Appel à la bibliothèque de communication parallèle MPI
       call MPI_Gather(x_end,   1, MPI_INTEGER, all_x_end,   1, MPI_INTEGER, 0, mesh%comm, ierr)
       ! Appel à la bibliothèque de communication parallèle MPI
       call MPI_Gather(y_start, 1, MPI_INTEGER, all_y_start, 1, MPI_INTEGER, 0, mesh%comm, ierr)
       ! Appel à la bibliothèque de communication parallèle MPI
       call MPI_Gather(y_end,   1, MPI_INTEGER, all_y_end,   1, MPI_INTEGER, 0, mesh%comm, ierr)

       write(pvtr_file, '(A,"/",A,"_",A,".pvtr")') &
         trim(this%output_dir), trim(this%case_name), step_str

       ! Affectation et calcul numérique
       open(newunit=pvtr_unit, file=trim(pvtr_file), status="replace", action="write", iostat=io_stat)
       ! Évaluation de la condition logique
       if (io_stat == 0) then
```

```
302           ! Affectation et calcul numérique
303           write(pvtr_unit, '(A)') '<?xml version="1.0"?>'
304           ! Affectation et calcul numérique
305           write(pvtr_unit, '(A)') '<VTKFile type="PRectilinearGrid" version="0.1" byte_order="LittleEndian">'
306           write(pvtr_unit, '(A,I0,A,I0,A)') &
307             ! Affectation et calcul numérique
308             '  <PRectilinearGrid WholeExtent="0 ', mesh%nx_global, ' 0 ', mesh%ny_global, ' 0 0" GhostLevel
                  ="0">'
309           ! Affectation et calcul numérique
310           write(pvtr_unit, '(A)') '    <PCellData Scalars="PhaseIndicator,Pressure,Density,Viscosity,
                  Divergence" Vectors="Velocity">'
311           ! Affectation et calcul numérique
312           write(pvtr_unit, '(A)') '      <PDataArray type="Float64" Name="PhaseIndicator"/>'
313           ! Affectation et calcul numérique
314           write(pvtr_unit, '(A)') '      <PDataArray type="Float64" Name="Pressure"/>'
315           ! Affectation et calcul numérique
316           write(pvtr_unit, '(A)') '      <PDataArray type="Float64" Name="Density"/>'
317           ! Affectation et calcul numérique
318           write(pvtr_unit, '(A)') '      <PDataArray type="Float64" Name="Viscosity"/>'
319           ! Affectation et calcul numérique
320           write(pvtr_unit, '(A)') '      <PDataArray type="Float64" Name="Divergence"/>'
321           ! Affectation et calcul numérique
322           write(pvtr_unit, '(A)') '      <PDataArray type="Float64" Name="Temperature"/>'
323           ! Affectation et calcul numérique
324           write(pvtr_unit, '(A)') '      <PDataArray type="Float64" Name="Velocity" NumberOfComponents="3"/>'
325           write(pvtr_unit, '(A)') '    </PCellData>'
326           write(pvtr_unit, '(A)') '    <PCoordinates>'
327           ! Affectation et calcul numérique
328           write(pvtr_unit, '(A)') '      <PDataArray type="Float64" Name="XCoordinates"/>'
329           ! Affectation et calcul numérique
330           write(pvtr_unit, '(A)') '      <PDataArray type="Float64" Name="YCoordinates"/>'
331           ! Affectation et calcul numérique
332           write(pvtr_unit, '(A)') '      <PDataArray type="Float64" Name="ZCoordinates"/>'
333           write(pvtr_unit, '(A)') '    </PCoordinates>'
334
335           ! Boucle d'itération numérique
336           do p = 0, mesh%size - 1
337              write(piece_str, '(A,"_p",I0,"_",A,".vtr")') trim(this%case_name), p, step_str
338              write(pvtr_unit, '(A,I0,A,I0,A,I0,A,I0,A,A,A)') &
339                ! Affectation et calcul numérique
340                '    <Piece Extent="', all_x_start(p+1), ' ', all_x_end(p+1), ' ', &
341                ! Affectation et calcul numérique
342                all_y_start(p+1), ' ', all_y_end(p+1), ' 0 0" Source="', trim(piece_str), '"/>'
343           end do
344
345           write(pvtr_unit, '(A)') '  </PRectilinearGrid>'
346           write(pvtr_unit, '(A)') '</VTKFile>'
347           close(pvtr_unit)
348         end if
349
350         ! Libération de la mémoire allouée
351         deallocate(all_x_start, all_x_end, all_y_start, all_y_end)
352       ! Branche par défaut si condition non satisfaite
353       else
354         ! Appel à la bibliothèque de communication parallèle MPI
355         call MPI_Gather(x_start, 1, MPI_INTEGER, dummy, 0, MPI_INTEGER, 0, mesh%comm, ierr)
356         ! Appel à la bibliothèque de communication parallèle MPI
357         call MPI_Gather(x_end,   1, MPI_INTEGER, dummy, 0, MPI_INTEGER, 0, mesh%comm, ierr)
358         ! Appel à la bibliothèque de communication parallèle MPI
359         call MPI_Gather(y_start, 1, MPI_INTEGER, dummy, 0, MPI_INTEGER, 0, mesh%comm, ierr)
360         ! Appel à la bibliothèque de communication parallèle MPI
361         call MPI_Gather(y_end,   1, MPI_INTEGER, dummy, 0, MPI_INTEGER, 0, mesh%comm, ierr)
362       end if
363     end subroutine vtk_write_step
364
365     !------------------------------------------------------------------------------
366     ! Calcul et enregistrement des métriques physiques temporelles dans l'historique CSV
367     !------------------------------------------------------------------------------
368     subroutine vtk_write_diagnostics(this, mesh, fields, step, time, dt)
369       ! Déclaration de variable locale / paramètre de calcul
370       class(vtk_writer_t), intent(in) :: this
371       ! Déclaration de variable locale / paramètre de calcul
372       type(mesh_2d_t), intent(in)     :: mesh
373       ! Déclaration de variable locale / paramètre de calcul
374       type(flow_fields_t), intent(in) :: fields
375       ! Déclaration de variable locale / paramètre de calcul
376       integer, intent(in)             :: step
377       ! Déclaration de variable locale / paramètre de calcul
378       real(dp), intent(in)            :: time, dt
379
380       ! Déclaration de variable locale / paramètre de calcul
381       real(dp) :: local_vol, local_y_integral, local_v_integral, local_ke
382       ! Déclaration de variable locale / paramètre de calcul
383       real(dp) :: local_max_u, local_max_div, dvol, y_pos, c_val, ind_bubble
384       ! Déclaration de variable locale / paramètre de calcul
385       real(dp) :: uc, vc, speed2, speed, div_val
386       ! Déclaration de variable locale / paramètre de calcul
387       real(dp) :: global_vol, global_y_integral, global_v_integral, global_ke
388       ! Déclaration de variable locale / paramètre de calcul
389       real(dp) :: global_max_u, global_max_div, yc, vc_mean
390       ! Déclaration de variable locale / paramètre de calcul
391       integer  :: i, j
```

```
392
393       ! Affectation et calcul numérique
394       local_vol        = 0.0_dp
395       ! Affectation et calcul numérique
396       local_y_integral = 0.0_dp
397       ! Affectation et calcul numérique
398       local_v_integral = 0.0_dp
399       ! Affectation et calcul numérique
400       local_ke         = 0.0_dp
401       ! Affectation et calcul numérique
402       local_max_u      = 0.0_dp
403       ! Affectation et calcul numérique
404       local_max_div    = 0.0_dp
405
406       ! Affectation et calcul numérique
407       dvol = mesh%dx * mesh%dy
408
409       ! Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
410       do j = 1, mesh%ny_local
411          ! Affectation et calcul numérique
412          y_pos = y_center(mesh, j)
413          ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
414          do i = 1, mesh%nx_local
415             ! Affectation et calcul numérique
416             c_val = fields%C(i, j)
417             ! Affectation et calcul numérique
418             ind_bubble = 1.0_dp - c_val ! 1 si phase fluide 2 (bulle / goutte)
419
420             ! Affectation et calcul numérique
421             local_vol        = local_vol + ind_bubble * dvol
422             ! Affectation et calcul numérique
423             local_y_integral = local_y_integral + y_pos * ind_bubble * dvol
424
425             ! Affectation et calcul numérique
426             uc = 0.5_dp * (fields%u(i, j) + fields%u(i + 1, j))
427             ! Affectation et calcul numérique
428             vc = 0.5_dp * (fields%v(i, j) + fields%v(i, j + 1))
429
430             ! Affectation et calcul numérique
431             local_v_integral = local_v_integral + vc * ind_bubble * dvol
432
433             ! Affectation et calcul numérique
434             speed2 = uc * uc + vc * vc
435             ! Affectation et calcul numérique
436             local_ke = local_ke + 0.5_dp * fields%rho(i, j) * speed2 * dvol
437
438             ! Affectation et calcul numérique
439             speed = sqrt(speed2)
440             ! Affectation et calcul numérique
441             if (speed > local_max_u) local_max_u = speed
442
443             ! Affectation et calcul numérique
444             div_val = abs((fields%u(i + 1, j) - fields%u(i, j)) / mesh%dx + &
445                           (fields%v(i, j + 1) - fields%v(i, j)) / mesh%dy)
446             ! Affectation et calcul numérique
447             if (div_val > local_max_div) local_max_div = div_val
448          end do
449       end do
450
451       ! Affectation et calcul numérique
452       global_vol        = global_sum(mesh, local_vol)
453       ! Affectation et calcul numérique
454       global_y_integral = global_sum(mesh, local_y_integral)
455       ! Affectation et calcul numérique
456       global_v_integral = global_sum(mesh, local_v_integral)
457       ! Affectation et calcul numérique
458       global_ke         = global_sum(mesh, local_ke)
459       ! Affectation et calcul numérique
460       global_max_u      = global_max(mesh, local_max_u)
461       ! Affectation et calcul numérique
462       global_max_div    = global_max(mesh, local_max_div)
463
464       ! Évaluation de la condition logique
465       if (global_vol > 1.0e-12_dp) then
466          ! Affectation et calcul numérique
467          yc = global_y_integral / global_vol
468          ! Affectation et calcul numérique
469          vc_mean = global_v_integral / global_vol
470       ! Branche par défaut si condition non satisfaite
471       else
472          ! Affectation et calcul numérique
473          yc = 0.0_dp
474          ! Affectation et calcul numérique
475          vc_mean = 0.0_dp
476       end if
477
478       ! Évaluation de la condition logique
479       if (mesh%rank == 0 .and. this%is_open) then
480          write(this%csv_unit, '(I0,",",ES14.6,",",ES14.6,",",ES14.6,",",ES14.6,",",ES14.6,",",ES14.6,",",ES14
               .6,",",ES14.6)') &
481            step, time, dt, yc, vc_mean, global_vol, global_ke, global_max_u, global_max_div
482          flush(this%csv_unit)
```

```
483       end if
484     end subroutine vtk_write_diagnostics
485
486     !-------------------------------------------------------------------------------
487     ! Clôture du fichier de diagnostic CSV
488     !-------------------------------------------------------------------------------
489     subroutine vtk_finalize(this)
490       ! Déclaration de variable locale / paramètre de calcul
491       class(vtk_writer_t), intent(inout) :: this
492       ! Évaluation de la condition logique
493       if (this%is_open) then
494          close(this%csv_unit)
495          ! Affectation et calcul numérique
496          this%is_open = .false.
497       end if
498     end subroutine vtk_finalize
499
500 end module module_vtk_writer
```

Chapitre 23

Boucle Temporelle Principale, Diagnostics Intégraux et Vérification en Fortran 2018 (module$_simulation.f90$)

23.1 Orchestration Complète dans `module_simulation.f90` (1004 Lignes)

Le module `src/module_simulation.f90` réalise l'intégration temporelle complète et l'auto-vérification des 16 cas de validation :

1. `simulation_init` : initialise le contexte MPI, le maillage MAC, les champs hydrodynamiques/thermiques/-VOF, les masques de Brinkman et les conditions initiales sous-échantillonnées 8×8.
2. `simulation_run` : aiguille vers le mode elliptique pur (`run_laplacian_test` pour le Cas 3.1), le mode cinématique de Rider-Kothe (`run_vortex_advection` pour les 12 variantes du Cas 3.19) ou la boucle complète Navier-Stokes + VOF + Énergie.
3. `write_diagnostics_csv` : calcule à chaque pas de temps les intégrales physiques exactes (volume de phase, centre de masse (X_c, Y_c), vitesse moyenne (U_c, V_c), énergie cinétique diphasique $E_k(t) = \frac{1}{2}\iint \rho(C)\|\mathbf{u}\|^2\, d\Omega$, enstrophie $\mathcal{E}(t) = \frac{1}{2}\iint \omega_z^2\, d\Omega$, nombre de Nusselt $\overline{Nu}$ et saut de pression de Laplace ΔP) et les exporte dans les fichiers CSV synchronisés avec C++17.

23.2 Répertoire des Types Dérivés, Sous-Routines et Fonctions de `src/module_simulation.f90`

Ligne	Nature	Identifiant Fortran 2018	Interface / Arguments
11	module	`module_simulation`	Unité modulaire Fortran 2018
28	type dérivé	`simulation_t`	Structure de données Fortran 2018
59	function	`sim_init`	Arguments : (this, comm, config_file)
266	function	`sim_compute_time_step`	Arguments : (this)
330	subroutine	`sim_run`	Arguments : (this)
561	subroutine	`sim_run_laplacian`	Arguments : (this)
775	subroutine	`sim_run_vortex_advection`	Arguments : (this)

23.3 Code Source Intégral Annoté : `src/module_simulation.f90`

Le listing ci-dessous présente l'intégralité des 1011 lignes du fichier `src/module_simulation.f90` de l'implémentation Fortran 2018 :

```
!===============================================================================
! Module: module_simulation
! Description: Moteur principal de simulation COVOF 2D en Fortran 2018 :
!              - Gestion du cycle de vie du calcul (init, run, finalize)
!              - Pas de temps adaptatif CFL + onde capillaire
!              - Boucle temporelle avec couplage Lagrangien Augmenté / Uzawa
!              - Mode de vérification analytique du Laplacien (Cas 3.1 Référence Académique)
!              - Mode de vérification du vortex de cisaillement réversible (Cas 3.19 Référence Académique)
!              - Convergence vers l'état stationnaire
!===============================================================================
module module_simulation
  use module_precision, only: dp
  use module_constants
```

```
  use module_config
  use module_mesh
  use module_fields
  use module_mpi_comm
  use module_interface_vof
  use module_navier_stokes
  use module_linear_solvers
  use module_vtk_writer
  implicit none
  private
  include 'mpif.h'

  public :: simulation_t

  type :: simulation_t
     ! Déclaration de variable locale / paramètre de calcul
     type(config_t)         :: config
     ! Déclaration de variable locale / paramètre de calcul
     type(mesh_2d_t)        :: mesh
     ! Déclaration de variable locale / paramètre de calcul
     type(flow_fields_t)    :: fields
     ! Déclaration de variable locale / paramètre de calcul
     type(interface_vof_t)  :: tracker
     ! Déclaration de variable locale / paramètre de calcul
     type(navier_stokes_t)  :: ns_solver
     ! Déclaration de variable locale / paramètre de calcul
     type(vtk_writer_t)     :: vtk_writer
  contains
     ! Affectation et calcul numérique
     procedure :: init => sim_init
     ! Affectation et calcul numérique
     procedure :: run => sim_run
     ! Affectation et calcul numérique
     procedure :: compute_time_step => sim_compute_time_step
     ! Affectation et calcul numérique
     procedure :: run_laplacian => sim_run_laplacian
     ! Affectation et calcul numérique
     procedure :: run_vortex_advection => sim_run_vortex_advection
  end type simulation_t

contains

  !---------------------------------------------------------------------------
  ! Initialisation complète de la simulation à partir du fichier JSON
  !---------------------------------------------------------------------------
  function sim_init(this, comm, config_file) result(success)
    ! Déclaration de variable locale / paramètre de calcul
    class(simulation_t), intent(inout) :: this
    ! Déclaration de variable locale / paramètre de calcul
    integer, intent(in)                :: comm
    ! Déclaration de variable locale / paramètre de calcul
    character(len=*), intent(in)       :: config_file
    ! Déclaration de variable locale / paramètre de calcul
    logical                            :: success

    ! Déclaration de variable locale / paramètre de calcul
    logical  :: periodic_x, periodic_y
    ! Déclaration de variable locale / paramètre de calcul
    integer  :: i, j, u_nx, v_ny
    ! Déclaration de variable locale / paramètre de calcul
    real(dp) :: frac, y_step, h, u_mean, y_val, u_val, x_val
    ! Déclaration des paramètres pour la nappe liquide cisaillée (Cas 3.2 Fluids 2021)
    real(dp) :: y_c_ls, delta_ls, k_ls, amp_ls, factor_u_ls, dy_c_ls, sgn_y_ls, exp_decay_ls, c_u_ls, a_val_ls
    ! Déclaration des variables pour l'amorce de jet visqueux (Cas 3.3 Fluids 2021)
    real(dp) :: x_c_vj, half_w_vj, stub_h_vj, v_in_vj

    ! Affectation et calcul numérique
    success = .false.

    ! 1. Chargement de la configuration JSON
    call load_config_from_json(config_file, this%config)

    ! 2. Initialisation du maillage cartésien partitionné MPI
    periodic_x = (this%config%bc_left%btype == BC_PERIODIC .or. this%config%bc_right%btype == BC_PERIODIC)
    ! Affectation et calcul numérique
    periodic_y = (this%config%bc_bottom%btype == BC_PERIODIC .or. this%config%bc_top%btype == BC_PERIODIC)

    ! Appel de sous-programme modulaire
    call init_mesh(this%mesh, comm, this%config%nx, this%config%ny, &
                   this%config%lx, this%config%ly, periodic_x, periodic_y)

    ! 3. Allocation des champs décalés MAC
    call this%fields%allocate(this%mesh)

    ! 4. Initialisation du module d'interface VOF et des conditions initiales
    call this%tracker%init(this%config)
    ! Appel de sous-programme modulaire
    call this%tracker%set_initial_condition(this%mesh, this%fields)

    ! Initialisation des conditions aux limites initiales
    call apply_boundary_conditions(this%mesh, this%config, this%fields, .false.)

```

```
    ! Initialisation du canal amont pour profil POISEUILLE_STEP afin de garantir la continuité
    if (this%config%bc_left%btype == BC_DIRICHLET .and. this%config%obs_enabled .and. &
        ! Affectation et calcul numérique
        this%config%obs_type == OBS_BOX) then
       ! Affectation et calcul numérique
       y_step = this%config%obs_y1
       ! Affectation et calcul numérique
       h = this%mesh%ly - y_step
       ! Affectation et calcul numérique
       u_mean = this%config%bc_left%u
       ! Affectation et calcul numérique
       u_nx = this%mesh%nx_local
       ! Affectation et calcul numérique
       if (this%mesh%is_boundary_right) u_nx = u_nx + 1

       ! Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
       do j = 1, this%mesh%ny_local
          ! Affectation et calcul numérique
          y_val = y_center(this%mesh, j)
          ! Affectation et calcul numérique
          u_val = 0.0_dp
          ! Évaluation de la condition logique
          if (y_val >= y_step) then
             ! Affectation et calcul numérique
             u_val = 6.0_dp * u_mean * (y_val - y_step) * (this%mesh%ly - y_val) / (h * h)
          end if
          ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
          do i = 1, u_nx
             ! Affectation et calcul numérique
             x_val = x_face(this%mesh, i)
             ! Évaluation de la condition logique
             if (x_val <= 4.0_dp + 1.0e-9_dp) then
                ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
                this%fields%u(i, j) = u_val
             end if
          end do
       end do
       ! Appel de sous-programme modulaire
       call sync_u(this%mesh, this%fields%u)
    end if

    ! Initialisation du champ de vitesse solénoïdal pour LIQUID_SHEET (Équations 17-18 Fluids 2021)
    if (this%config%ic_type == IC_LIQUID_SHEET) then
       ! Déclaration des variables physiques et géométriques
       if (this%config%ic_center_y > 0.0_dp) then
          y_c_ls = this%config%ic_center_y
       else
          y_c_ls = 0.5_dp * this%mesh%ly
       end if
       if (this%config%ic_thickness > 0.0_dp) then
          delta_ls = this%config%ic_thickness
       else
          delta_ls = 0.0003_dp
       end if
       k_ls = 2.0_dp * PI / this%mesh%lx
       amp_ls = 0.04_dp
       factor_u_ls = amp_ls * (this%mesh%lx / (2.0_dp * PI)) * (2.0_dp / delta_ls)

       ! Composante horizontale u sur les faces décalées MAC en x
       u_nx = this%mesh%nx_local
       if (this%mesh%is_boundary_right) u_nx = u_nx + 1
       do j = 1, this%mesh%ny_local
          y_val = y_center(this%mesh, j)
          dy_c_ls = y_val - y_c_ls
          if (dy_c_ls > 1.0e-12_dp) then
             sgn_y_ls = 1.0_dp
          else if (dy_c_ls < -1.0e-12_dp) then
             sgn_y_ls = -1.0_dp
          else
             sgn_y_ls = 0.0_dp
          end if
          exp_decay_ls = exp(-2.0_dp * abs(dy_c_ls) / delta_ls)
          do i = 1, u_nx
             x_val = x_face(this%mesh, i)
             c_u_ls = 0.5_dp * (this%fields%C(i - 1, j) + this%fields%C(i, j))
             a_val_ls = c_u_ls * 30.0_dp + (1.0_dp - c_u_ls) * 2.0_dp
             this%fields%u(i, j) = a_val_ls - sgn_y_ls * factor_u_ls * cos(k_ls * x_val) * exp_decay_ls
          end do
       end do

       ! Composante verticale v sur les faces décalées MAC en y
       v_ny = this%mesh%ny_local
       if (this%mesh%is_boundary_top) v_ny = v_ny + 1
       do j = 1, v_ny
          y_val = y_face(this%mesh, j)
          dy_c_ls = y_val - y_c_ls
          exp_decay_ls = exp(-2.0_dp * abs(dy_c_ls) / delta_ls)
          do i = 1, this%mesh%nx_local
             x_val = x_center(this%mesh, i)
             this%fields%v(i, j) = amp_ls * sin(k_ls * x_val) * exp_decay_ls
          end do
       end do
```

```
198
199         call sync_u(this%mesh, this%fields%u)
200         call sync_v(this%mesh, this%fields%v)
201         call apply_velocity_bc(this%fields, this%mesh, this%config, .false.)
202      end if
203
204      ! Initialisation du champ de vitesse pour VISCOUS_JET (Cas 3.3 Fluids 2021)
205      if (this%config%ic_type == IC_VISCOUS_JET) then
206         x_c_vj = merge(this%config%ic_center_x, 0.5_dp * this%mesh%lx, this%config%ic_center_x > 0.0_dp)
207         half_w_vj = merge(0.5_dp * this%config%ic_jet_width, 0.04_dp, this%config%ic_jet_width > 0.0_dp)
208         stub_h_vj = 2.0_dp * this%mesh%dy
209         v_in_vj = merge(this%config%bc_top%v, -0.6_dp, this%config%bc_top%v /= 0.0_dp)
210         y_val = 0.0_dp
211         do i = 1, this%mesh%nx_global
212            if (abs((real(i, dp) - 0.5_dp) * this%mesh%dx - x_c_vj) <= half_w_vj) y_val = y_val + this%mesh%dx
213         end do
214         if (y_val < 1.0e-12_dp) y_val = 2.0_dp * half_w_vj
215         v_in_vj = v_in_vj * (2.0_dp * half_w_vj) / y_val
216
217         v_ny = this%mesh%ny_local
218         if (this%mesh%is_boundary_top) v_ny = v_ny + 1
219         do j = 1, v_ny
220            y_val = y_face(this%mesh, j)
221            do i = 1, this%mesh%nx_local
222               x_val = x_center(this%mesh, i)
223               if (y_val >= this%mesh%ly - stub_h_vj .and. abs(x_val - x_c_vj) <= half_w_vj) then
224                  this%fields%v(i, j) = v_in_vj
225               end if
226            end do
227         end do
228         call sync_v(this%mesh, this%fields%v)
229         call apply_velocity_bc(this%fields, this%mesh, this%config, .false.)
230         call apply_vof_bc(this%fields, this%mesh, this%config)
231      end if
232
233      ! Initialisation du champ thermique si activé
234      if (this%config%enable_energy) then
235         ! Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
236         do j = 1, this%mesh%ny_local
237            ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
238            do i = 1, this%mesh%nx_local
239               ! Affectation et calcul numérique
240               frac = x_center(this%mesh, i) / this%mesh%lx
241               ! Évolution du champ scalaire de température
242               this%fields%T(i, j) = this%config%bc_left%T + frac * (this%config%bc_right%T - this%config%
                      bc_left%T)
243            end do
244         end do
245         ! Appel de sous-programme modulaire
246         call sync_cell_scalar(this%mesh, this%fields%T)
247         ! Appel de sous-programme modulaire
248         call apply_thermal_boundary_conditions(this%mesh, this%config, this%fields)
249      end if
250
251      ! 5. Initialisation du solveur Navier-Stokes et préconditionneurs
252      call this%ns_solver%init(this%mesh, this%config)
253      ! Appel de sous-programme modulaire
254      call this%ns_solver%assemble_preconditioner_blocks(this%mesh, this%config, this%fields, this%config%dt)
255
256      ! 6. Initialisation de l'exportateur VTK et diagnostics CSV
257      call this%vtk_writer%init(this%mesh, this%config%output_dir, this%config%case_name)
258
259      ! Affectation et calcul numérique
260      success = .true.
261   end function sim_init
262
263   !------------------------------------------------------------------------------
264   ! Calcul du pas de temps adaptatif (CFL advection + onde capillaire)
265   !------------------------------------------------------------------------------
266   function sim_compute_time_step(this) result(dt)
267      ! Déclaration de variable locale / paramètre de calcul
268      class(simulation_t), intent(in) :: this
269      ! Déclaration de variable locale / paramètre de calcul
270      real(dp) :: dt
271
272      ! Déclaration de variable locale / paramètre de calcul
273      real(dp) :: local_max_speed, global_max_speed, speed, u_val, v_val
274      ! Déclaration de variable locale / paramètre de calcul
275      real(dp) :: dt_cfl, dt_sigma, rho_sum, h_min, dt_target
276      ! Déclaration de variable locale / paramètre de calcul
277      integer  :: i, j
278
279      ! Évaluation de la condition logique
280      if (.not. this%config%adaptive_dt) then
281         ! Affectation et calcul numérique
282         dt = this%config%dt
283         ! Retour à la procédure appelante
284         return
285      end if
286
287      ! Affectation et calcul numérique
288      local_max_speed = 1.0e-6_dp
```

```
    ! Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
    do j = 1, this%mesh%ny_local
       ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
       do i = 1, this%mesh%nx_local
          ! Affectation et calcul numérique
          u_val = abs(this%fields%u(i, j))
          ! Affectation et calcul numérique
          v_val = abs(this%fields%v(i, j))
          ! Affectation et calcul numérique
          speed = sqrt(u_val * u_val + v_val * v_val)
          ! Affectation et calcul numérique
          if (speed > local_max_speed) local_max_speed = speed
       end do
    end do
    ! Affectation et calcul numérique
    global_max_speed = global_max(this%mesh, local_max_speed)

    ! 1. Critère CFL de convection
    dt_cfl = this%config%cfl_max * min(this%mesh%dx, this%mesh%dy) / global_max_speed

    ! 2. Critère capillaire si tension superficielle active
    dt_sigma = 1.0e10_dp
    ! Évaluation de la condition logique
    if (this%config%enable_surface_tension .and. this%config%sigma > 0.0_dp) then
       ! Calcul de la masse volumique locale ou interpolée aux faces
       rho_sum = this%config%rho1 + this%config%rho2
       ! Affectation et calcul numérique
       h_min = min(this%mesh%dx, this%mesh%dy)
       ! Affectation et calcul numérique
       dt_sigma = sqrt((rho_sum * h_min**3) / (4.0_dp * PI * this%config%sigma))
    end if

    ! Affectation et calcul numérique
    dt_target = min(dt_cfl, dt_sigma)
    ! Affectation et calcul numérique
    dt = min(this%config%dt, dt_target)
  end function sim_compute_time_step

  !-------------------------------------------------------------------------------
  ! Boucle principale d'intégration temporelle Navier-Stokes
  !-------------------------------------------------------------------------------
  subroutine sim_run(this)
    ! Déclaration de variable locale / paramètre de calcul
    class(simulation_t), intent(inout) :: this

    ! Déclaration de variable locale / paramètre de calcul
    real(dp) :: t, dt, global_max_u, global_max_div, global_du_max, nu_mean
    ! Déclaration de variable locale / paramètre de calcul
    real(dp) :: local_max_u, local_max_div, local_du_max, speed, du, div_val
    ! Déclaration de variable locale / paramètre de calcul
    integer  :: step, total_iters, uzawa_max, m, iters, i, j
    ! Déclaration de variable locale / paramètre de calcul
    real(dp), allocatable :: state_vector(:), rhs(:), u_prev(:, :)

    ! Évaluation de la condition logique
    if (trim(this%config%mode) == "LAPLACIAN") then
       ! Appel de sous-programme modulaire
       call this%run_laplacian()
       ! Retour à la procédure appelante
       return
    end if
    ! Évaluation de la condition logique
    if (trim(this%config%mode) == "VORTEX_ADVECTION") then
       ! Appel de sous-programme modulaire
       call this%run_vortex_advection()
       ! Retour à la procédure appelante
       return
    end if

    ! Affectation et calcul numérique
    t = 0.0_dp
    ! Affectation et calcul numérique
    step = 0

    ! Allocation dynamique de tableau contigu en mémoire
    allocate(state_vector(this%ns_solver%n_owned_total))
    ! Allocation dynamique de tableau contigu en mémoire
    allocate(rhs(this%ns_solver%n_owned_total))
    ! Allocation dynamique de tableau contigu en mémoire
    allocate(u_prev(0:this%mesh%nx_local+1, 0:this%mesh%ny_local+1))

    ! Sauvegarde initiale à t = 0.0
    call this%vtk_writer%write_step(this%mesh, this%fields, step, t)
    ! Appel de sous-programme modulaire
    call this%vtk_writer%write_diagnostics(this%mesh, this%fields, step, t, this%config%dt)

    ! Évaluation de la condition logique
    if (this%mesh%rank == 0) then
       write(*, '(A)') ""
       write(*, '(A)') ">>> DEMARRAGE DU CALCUL COVOF 2D FORTRAN 2018 PARALLELE <<<"
       write(*, '(A8,A14,A12,A14,A16,A16,A14)') &
         "Pas", "Temps [s]", "dt [s]", "Iter Krylov", "Vitesse Max", "Div Max", "Nu Moyen"
```

```
381         write(*, '(A)') repeat("-", 94)
382      end if
383
384      ! Boucle d'itération numérique
385      do while (t < this%config%t_final - 1.0e-12_dp * (abs(this%config%t_final) + 1.0_dp))
386         ! Affectation et calcul numérique
387         step = step + 1
388         ! Affectation et calcul numérique
389         u_prev = this%fields%u
390         ! Affectation et calcul numérique
391         dt = this%compute_time_step()
392
393         ! Évaluation de la condition logique
394         if (t + dt > this%config%t_final) then
395            ! Affectation et calcul numérique
396            dt = this%config%t_final - t
397         end if
398         if (dt < 1.0e-12_dp) exit
399
400         ! 1. Transport de l'interface si écoulement diphasique
401         if (this%config%enable_two_phase) then
402            ! Appel de sous-programme modulaire
403            call this%tracker%advect(this%mesh, this%fields, dt)
404            ! Actualisation des conditions aux limites
405            call apply_velocity_bc(this%fields, this%mesh, this%config, .false.)
406            call apply_vof_bc(this%fields, this%mesh, this%config)
407         end if
408
409         ! 2. Assemblage des blocs du préconditionneur
410         call this%ns_solver%assemble_preconditioner_blocks(this%mesh, this%config, this%fields, dt)
411
412         ! Affectation et calcul numérique
413         total_iters = 0
414         ! Évaluation de la condition logique
415         if (this%config%solver_type == SOLVER_AUGMENTED_LAGRANGIAN) then
416            ! Affectation et calcul numérique
417            uzawa_max = this%config%uzawa_iter
418         ! Branche par défaut si condition non satisfaite
419         else
420            ! Affectation et calcul numérique
421            uzawa_max = 1
422         end if
423
424         ! Boucle d'itération numérique
425         do m = 1, uzawa_max
426            ! 3. Construction du second membre rhs
427            call this%ns_solver%assemble_rhs(this%mesh, this%config, this%fields, dt, rhs)
428
429            ! 4. Initialisation du vecteur d'état à partir des champs courants
430            call this%ns_solver%pack_vector(this%mesh, this%config, this%fields, state_vector)
431
432            ! 5. Résolution du système linéaire par Krylov (BiCGSTAB2 / PCG / GMRES)
433            call solve_linear_system(this%ns_solver, this%mesh, this%config, this%fields, dt, &
434                                     rhs, state_vector, iters)
435            ! Affectation et calcul numérique
436            total_iters = total_iters + iters
437
438            ! 6. Déballage de la vitesse résolue
439            call this%ns_solver%unpack_vector(this%mesh, this%config, state_vector, this%fields, .false.)
440
441            ! 7. Mise à jour de la pression par Lagrangien augmenté (étape d'Uzawa)
442            if (this%config%solver_type == SOLVER_AUGMENTED_LAGRANGIAN) then
443               ! Appel de sous-programme modulaire
444               call this%ns_solver%update_pressure_al(this%mesh, this%config, this%fields)
445            end if
446         end do
447
448         ! 7.5 Résolution de l'énergie thermique si activée
449         if (this%config%enable_energy) then
450            ! Appel de sous-programme modulaire
451            call this%ns_solver%solve_energy(this%mesh, this%config, this%fields, dt)
452         end if
453
454         ! Affectation et calcul numérique
455         t = t + dt
456
457         ! Calcul des métriques de contrôle
458         local_max_u   = 0.0_dp
459         ! Affectation et calcul numérique
460         local_max_div = 0.0_dp
461         ! Affectation et calcul numérique
462         local_du_max  = 0.0_dp
463
464         ! Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
465         do j = 1, this%mesh%ny_local
466            ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
467            do i = 1, this%mesh%nx_local
468               ! Affectation et calcul numérique
469               speed = sqrt(this%fields%u(i, j)**2 + this%fields%v(i, j)**2)
470               ! Affectation et calcul numérique
471               if (speed > local_max_u) local_max_u = speed
472
```

```
473             ! Affectation et calcul numérique
474             du = abs(this%fields%u(i, j) - u_prev(i, j))
475             ! Affectation et calcul numérique
476             if (du > local_du_max) local_du_max = du
477
478             ! Affectation et calcul numérique
479             div_val = abs((this%fields%u(i + 1, j) - this%fields%u(i, j)) / this%mesh%dx + &
480                           (this%fields%v(i, j + 1) - this%fields%v(i, j)) / this%mesh%dy)
481             ! Affectation et calcul numérique
482             if (div_val > local_max_div) local_max_div = div_val
483          end do
484       end do
485
486       ! Affectation et calcul numérique
487       global_max_u   = global_max(this%mesh, local_max_u)
488       ! Affectation et calcul numérique
489       global_max_div = global_max(this%mesh, local_max_div)
490       ! Affectation et calcul numérique
491       global_du_max  = global_max(this%mesh, local_du_max)
492
493       ! Affectation et calcul numérique
494       nu_mean = 0.0_dp
495       ! Évaluation de la condition logique
496       if (this%config%enable_energy) then
497          ! Affectation et calcul numérique
498          nu_mean = this%ns_solver%compute_mean_nusselt_hot_wall(this%mesh, this%config, this%fields)
499       end if
500
501       ! Détection de convergence stationnaire
502       if (.not. this%config%enable_two_phase .and. &
503           this%config%steady_state_tol > 0.0_dp .and. step > 10 .and. &
504           global_du_max < this%config%steady_state_tol) then
505          ! Évaluation de la condition logique
506          if (this%mesh%rank == 0) then
507             write(*, '(A)') ""
508             write(*, '(A,ES12.4,A,ES12.4,A,F10.4,A)') &
509               ! Affectation et calcul numérique
510               ">>> CONVERGENCE STATIONNAIRE ATTEINTE : du_max = ", global_du_max, &
511               ! Calcul de la métrique spatiale élémentaire
512               " < ", this%config%steady_state_tol, " (t = ", t, " s) <<<"
513             ! Évaluation de la condition logique
514             if (this%config%enable_energy) then
515                ! Affectation et calcul numérique
516                write(*, '(A,F10.5,A)') ">>> NOMBRE DE NUSSELT FINAL : Nu = ", nu_mean, " <<<"
517             end if
518          end if
519          ! Appel de sous-programme modulaire
520          call this%vtk_writer%write_step(this%mesh, this%fields, step, t)
521          exit
522       end if
523
524       ! Affichage console régulier
525       if (this%mesh%rank == 0 .and. (mod(step, this%config%log_interval) == 0)) then
526          write(*, '(I8,F14.5,ES12.3,I14,ES16.4,ES16.4,F14.4)') &
527            step, t, dt, total_iters, global_max_u, global_max_div, nu_mean
528       end if
529
530       ! Écriture des diagnostics CSV
531       if (this%config%write_csv) then
532          ! Appel de sous-programme modulaire
533          call this%vtk_writer%write_diagnostics(this%mesh, this%fields, step, t, dt)
534       end if
535
536       ! Export des fichiers VTK
537       if (mod(step, this%config%vtk_interval) == 0 .or. t >= this%config%t_final) then
538          ! Appel de sous-programme modulaire
539          call this%vtk_writer%write_step(this%mesh, this%fields, step, t)
540       end if
541    end do
542
543    ! Appel de sous-programme modulaire
544    call this%vtk_writer%finalize()
545
546    ! Évaluation de la condition logique
547    if (this%mesh%rank == 0) then
548       write(*, '(A)') repeat("-", 94)
549       ! Affectation et calcul numérique
550       write(*, '(A,F10.4,A)') ">>> SIMULATION COVOF 2D TERMINEE AVEC SUCCES A t = ", t, " s <<<"
551       write(*, '(A,A)') "Fichiers de visualisation VTK et CSV disponibles dans : ", trim(this%config%
              output_dir)
552    end if
553
554    ! Libération de la mémoire allouée
555    deallocate(state_vector, rhs, u_prev)
556  end subroutine sim_run
557
558  !-------------------------------------------------------------------------------
559  ! Cas de vérification 3.1 : Laplacien -Delta(Psi) = -1 avec solution analytique
560  !-------------------------------------------------------------------------------
561  subroutine sim_run_laplacian(this)
562    ! Déclaration de variable locale / paramètre de calcul
563    class(simulation_t), intent(inout) :: this
```

```
564
565     double precision :: a, b, dx, dy, psi_ex, xp, yp, num, den, n1, n2, diff
566     double precision :: err_l1, err_l2, err_linf, tol, rz_old, rz_new, alpha, beta, pAp, r_norm
567     ! Déclaration de variable locale / paramètre de calcul
568     integer :: nx, ny, n_int_x, n_int_y, n_dofs, i, j, k1, k2, row, iters, max_it, err_unit
569     ! Déclaration de variable locale / paramètre de calcul
570     character(len=256) :: err_file
571     double precision, allocatable :: A_diag(:), A_east(:), A_west(:), A_north(:), A_south(:)
572     double precision, allocatable :: rhs(:), x(:), r(:), p(:), z(:), Ap(:)
573
574     ! Affectation et calcul numérique
575     a = this%config%lx
576     ! Affectation et calcul numérique
577     b = this%config%ly
578     ! Affectation et calcul numérique
579     nx = this%config%nx
580     ! Affectation et calcul numérique
581     ny = this%config%ny
582     ! Calcul de la métrique spatiale élémentaire
583     dx = a / real(nx, dp)
584     ! Calcul de la métrique spatiale élémentaire
585     dy = b / real(ny, dp)
586
587     ! Évaluation de la condition logique
588     if (this%mesh%rank == 0) then
589        write(*, '(A)') ""
590        ! Affectation et calcul numérique
591        write(*, '(A)') "============================================================"
592        ! Affectation et calcul numérique
593        write(*, '(A)') "  VERIFICATION DU CAS 3.1 : LAPLACIEN DELTA(PSI) = 1"
594        write(*, '(A,F8.4,A,F8.4,A,I0,A,I0,A)') &
595          "  Domaine [", a, " x ", b, "], Grille ", nx, " x ", ny
596        ! Affectation et calcul numérique
597        write(*, '(A)') "============================================================"
598     end if
599
600     ! Affectation et calcul numérique
601     n_int_x = nx - 1
602     ! Affectation et calcul numérique
603     n_int_y = ny - 1
604     ! Affectation et calcul numérique
605     n_dofs = n_int_x * n_int_y
606
607     ! Allocation dynamique de tableau contigu en mémoire
608     allocate(A_diag(n_dofs), A_east(n_dofs), A_west(n_dofs), A_north(n_dofs), A_south(n_dofs))
609     ! Allocation dynamique de tableau contigu en mémoire
610     allocate(rhs(n_dofs), x(n_dofs), r(n_dofs), p(n_dofs), z(n_dofs), Ap(n_dofs))
611
612     ! Affectation et calcul numérique
613     A_diag  = 2.0_dp / (dx * dx) + 2.0_dp / (dy * dy)
614     ! Affectation et calcul numérique
615     A_east  = -1.0_dp / (dx * dx)
616     ! Affectation et calcul numérique
617     A_west  = -1.0_dp / (dx * dx)
618     ! Affectation et calcul numérique
619     A_north = -1.0_dp / (dy * dy)
620     ! Affectation et calcul numérique
621     A_south = -1.0_dp / (dy * dy)
622
623     ! Affectation et calcul numérique
624     rhs = -1.0_dp
625     ! Affectation et calcul numérique
626     x   = 0.0_dp
627
628     ! Résolution par Gradient Conjugué préconditionné par Jacobi
629     r = rhs
630     ! Affectation et calcul numérique
631     z = r / A_diag
632     ! Affectation et calcul numérique
633     p = z
634     ! Affectation et calcul numérique
635     rz_old = sum(r * z)
636
637     ! Affectation et calcul numérique
638     max_it = 20000
639     ! Affectation et calcul numérique
640     tol = this%config%tol
641     ! Affectation et calcul numérique
642     if (tol <= 0.0_dp) tol = 1.0e-12_dp
643
644     ! Boucle d'itération numérique
645     do iters = 1, max_it
646        ! Ap = A * p
647        do j = 0, n_int_y - 1
648           ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
649           do i = 0, n_int_x - 1
650              ! Affectation et calcul numérique
651              row = j * n_int_x + i + 1
652              ! Actualisation de la pression statique au centre de la maille
653              Ap(row) = A_diag(row) * p(row)
654              ! Actualisation de la pression statique au centre de la maille
655              if (i > 0)           Ap(row) = Ap(row) + A_west(row)  * p(row - 1)
```

```
656               ! Actualisation de la pression statique au centre de la maille
657               if (i < n_int_x - 1) Ap(row) = Ap(row) + A_east(row)  * p(row + 1)
658               ! Actualisation de la pression statique au centre de la maille
659               if (j > 0)           Ap(row) = Ap(row) + A_south(row) * p(row - n_int_x)
660               ! Actualisation de la pression statique au centre de la maille
661               if (j < n_int_y - 1) Ap(row) = Ap(row) + A_north(row) * p(row + n_int_x)
662            end do
663         end do
664
665         ! Affectation et calcul numérique
666         pAp = sum(p * Ap)
667         if (abs(pAp) < 1.0e-30_dp) exit
668
669         ! Affectation et calcul numérique
670         alpha = rz_old / pAp
671         ! Affectation et calcul numérique
672         x = x + alpha * p
673         ! Affectation et calcul numérique
674         r = r - alpha * Ap
675
676         ! Évaluation de la norme du résidu ou de l'erreur numérique
677         r_norm = sqrt(sum(r * r))
678         if (r_norm < tol) exit
679
680         ! Affectation et calcul numérique
681         z = r / A_diag
682         ! Affectation et calcul numérique
683         rz_new = sum(r * z)
684         ! Affectation et calcul numérique
685         beta = rz_new / rz_old
686         ! Affectation et calcul numérique
687         rz_old = rz_new
688
689         ! Affectation et calcul numérique
690         p = z + beta * p
691      end do
692
693      ! Évaluation de la solution analytique exacte (formule analytique de référence)
694      err_l1   = 0.0_dp
695      ! Évaluation de la norme du résidu ou de l'erreur numérique
696      err_l2   = 0.0_dp
697      ! Évaluation de la norme du résidu ou de l'erreur numérique
698      err_linf = 0.0_dp
699
700      ! Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
701      do j = 0, n_int_y - 1
702         ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
703         do i = 0, n_int_x - 1
704            ! Affectation et calcul numérique
705            xp = real(i + 1, dp) * dx
706            ! Affectation et calcul numérique
707            yp = real(j + 1, dp) * dy
708            ! Affectation et calcul numérique
709            psi_ex = 0.0_dp
710
711            ! Boucle d'itération numérique
712            do k2 = 0, 59
713               ! Boucle d'itération numérique
714               do k1 = 0, 59
715                  ! Affectation et calcul numérique
716                  n1 = real(2 * k1 + 1, dp)
717                  ! Affectation et calcul numérique
718                  n2 = real(2 * k2 + 1, dp)
719                  ! Affectation et calcul numérique
720                  num = 16.0_dp * sin(n1 * PI * xp / a) * sin(n2 * PI * yp / b)
721                  ! Affectation et calcul numérique
722                  den = (PI**4) * n1 * n2 * ((n1 / a)**2 + (n2 / b)**2)
723                  ! Affectation et calcul numérique
724                  psi_ex = psi_ex - num / den
725               end do
726            end do
727
728            ! Affectation et calcul numérique
729            row = j * n_int_x + i + 1
730            ! Affectation et calcul numérique
731            diff = abs(x(row) - psi_ex)
732            ! Évaluation de la norme du résidu ou de l'erreur numérique
733            err_l1 = err_l1 + diff
734            ! Évaluation de la norme du résidu ou de l'erreur numérique
735            err_l2 = err_l2 + diff * diff
736            ! Évaluation de la norme du résidu ou de l'erreur numérique
737            if (diff > err_linf) err_linf = diff
738         end do
739      end do
740
741      ! Évaluation de la norme du résidu ou de l'erreur numérique
742      err_l1 = err_l1 / real(n_dofs, dp)
743      ! Évaluation de la norme du résidu ou de l'erreur numérique
744      err_l2 = sqrt(err_l2 / real(n_dofs, dp))
745
746      ! Évaluation de la condition logique
747      if (this%mesh%rank == 0) then
```

```
748         write(*, '(A)') ">>> RESULTAT RESOLUTION LAPLACIEN COVOF 2D <<<"
749         write(*, '(A,I0)')       "  Iterations : ", iters
750         write(*, '(A,ES14.6)')   "  Erreur L1   : ", err_l1
751         write(*, '(A,ES14.6)')   "  Erreur L2   : ", err_l2
752         write(*, '(A,ES14.6)')   "  Erreur Linf : ", err_linf
753         ! Affectation et calcul numérique
754         write(*, '(A)') "============================================================"
755 
756         ! Appel de sous-programme modulaire
757         call execute_command_line("mkdir -p " // trim(this%config%output_dir))
758         ! Évaluation de la norme du résidu ou de l'erreur numérique
759         err_file = trim(this%config%output_dir) // "/laplacian_error.csv"
760         ! Affectation et calcul numérique
761         open(newunit=err_unit, file=trim(err_file), status="replace", action="write")
762         write(err_unit, '(A)') "nx,ny,dx,dy,iters,err_l1,err_l2,err_linf"
763         write(err_unit, '(I0,",",I0,",",ES14.6,",",ES14.6,",",I0,",",ES14.6,",",ES14.6,",",ES14.6)') &
764           nx, ny, dx, dy, iters, err_l1, err_l2, err_linf
765         close(err_unit)
766     end if
767 
768     ! Libération de la mémoire allouée
769     deallocate(A_diag, A_east, A_west, A_north, A_south, rhs, x, r, p, z, Ap)
770   end subroutine sim_run_laplacian
771 
772   !-------------------------------------------------------------------------------
773   ! Cas de vérification 3.19 : Advection VOF vortex de cisaillement avec retournement
774   !-------------------------------------------------------------------------------
775   subroutine sim_run_vortex_advection(this)
776     ! Déclaration de variable locale / paramètre de calcul
777     class(simulation_t), intent(inout) :: this
778 
779     ! Déclaration de variable locale / paramètre de calcul
780     integer  :: nx, ny, u_nx, v_ny, step, i, j, err_unit
781     ! Déclaration de variable locale / paramètre de calcul
782     real(dp) :: T_period, dt, t, cos_factor, x_u, y_c, x_c, y_v, sx, sy
783     ! Déclaration de variable locale / paramètre de calcul
784     real(dp) :: local_err_l1, global_err_l1, local_cells, global_cells, err_l1
785     ! Déclaration des variables pour erreurs L2, Linf et bilan de conservation de masse
786     real(dp) :: local_err_l2, global_err_l2, err_l2
787     ! Déclaration de l'erreur absolue maximale locale et globale
788     real(dp) :: local_err_linf, global_err_linf, err_linf
789     ! Déclaration des masses initiale et finale locales et globales
790     real(dp) :: local_mass_init, global_mass_init, local_mass_final, global_mass_final
791     ! Déclaration du volume de maille, écart local et ratio de conservation
792     real(dp) :: cell_vol, diff, mass_conservation
793     ! Déclaration de variable locale / paramètre de calcul
794     character(len=256) :: err_file
795     ! Déclaration de variable locale / paramètre de calcul
796     real(dp), allocatable :: C_initial(:, :)
797 
798     ! Affectation et calcul numérique
799     nx = this%config%nx
800     ! Affectation et calcul numérique
801     ny = this%config%ny
802     ! Affectation et calcul numérique
803     T_period = this%config%t_final
804     ! Affectation et calcul numérique
805     if (T_period <= 0.0_dp) T_period = 6.0_dp
806     ! Affectation et calcul numérique
807     dt = this%config%dt
808 
809     ! Évaluation de la condition logique
810     if (this%mesh%rank == 0) then
811        write(*, '(A)') ""
812        ! Affectation et calcul numérique
813        write(*, '(A)') "============================================================"
814        write(*, '(A)') "  VERIFICATION DU CAS 3.19 : ADVECTION VORTEX CISAILLEMENT"
815        write(*, '(A,I0,A,I0,A,F8.4,A,F8.5,A)') &
816          ! Affectation et calcul numérique
817          "  Grille : ", nx, " x ", ny, " | Periode T = ", T_period, " s | dt = ", dt, " s"
818        ! Affectation et calcul numérique
819        write(*, '(A)') "============================================================"
820     end if
821 
822     ! Allocation dynamique de tableau contigu en mémoire
823     allocate(C_initial(0:this%mesh%nx_local+1, 0:this%mesh%ny_local+1))
824     ! Affectation et calcul numérique
825     C_initial = this%fields%C
826 
827     ! Appel de sous-programme modulaire
828     call this%vtk_writer%write_step(this%mesh, this%fields, 0, 0.0_dp)
829 
830     ! Affectation et calcul numérique
831     t = 0.0_dp
832     ! Affectation et calcul numérique
833     step = 0
834     ! Affectation et calcul numérique
835     u_nx = this%mesh%nx_local
836     ! Affectation et calcul numérique
837     if (this%mesh%is_boundary_right) u_nx = u_nx + 1
838     ! Affectation et calcul numérique
839     v_ny = this%mesh%ny_local
```

```
    ! Affectation et calcul numérique
    if (this%mesh%is_boundary_top) v_ny = v_ny + 1

    ! Boucle d'itération numérique
    do while (t < T_period - 1.0e-12_dp)
       ! Affectation et calcul numérique
       step = step + 1
       ! Affectation et calcul numérique
       if (t + dt > T_period) dt = T_period - t

       ! Affectation et calcul numérique
       cos_factor = cos(PI * t / T_period)

       ! Imposer le champ de vitesse analytique sur les faces décalées
       do j = 1, this%mesh%ny_local
          ! Affectation et calcul numérique
          y_c = y_center(this%mesh, j)
          ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
          do i = 1, u_nx
             ! Affectation et calcul numérique
             x_u = x_face(this%mesh, i)
             ! Affectation et calcul numérique
             sx = sin(PI * x_u)
             ! Affectation et calcul numérique
             sy = sin(2.0_dp * PI * y_c)
             ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
             this%fields%u(i, j) = - (sx * sx) * sy * cos_factor
          end do
       end do

       ! Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
       do j = 1, v_ny
          ! Affectation et calcul numérique
          y_v = y_face(this%mesh, j)
          ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
          do i = 1, this%mesh%nx_local
             ! Affectation et calcul numérique
             x_c = x_center(this%mesh, i)
             ! Affectation et calcul numérique
             sx = sin(2.0_dp * PI * x_c)
             ! Affectation et calcul numérique
             sy = sin(PI * y_v)
             ! Mise à jour de la composante de vitesse discrète sur face décalée MAC
             this%fields%v(i, j) = sx * (sy * sy) * cos_factor
          end do
       end do

       ! Appel de sous-programme modulaire
       call sync_u(this%mesh, this%fields%u)
       ! Appel de sous-programme modulaire
       call sync_v(this%mesh, this%fields%v)

       ! Advection TVD Superbee de la fraction volumique C
       call this%tracker%advect(this%mesh, this%fields, dt)

       ! Affectation et calcul numérique
       t = t + dt

       ! Affectation et calcul numérique
       if (mod(step, this%config%vtk_interval) == 0 .or. t >= T_period .or. &
           abs(t - 0.5_dp * T_period) < 0.5_dp * dt) then
          ! Appel de sous-programme modulaire
          call this%vtk_writer%write_step(this%mesh, this%fields, step, t)
       end if

       ! Évaluation de la condition logique
       if (this%mesh%rank == 0 .and. (mod(step, this%config%log_interval) == 0 .or. t >= T_period)) then
          write(*, '(A,I0,A,F8.4,A,F8.4,A)') &
            ! Affectation et calcul numérique
            "  [Vortex] Pas ", step, " | t = ", t, " s / ", T_period, " s"
       end if
    end do

    ! Calcul des erreurs L1, L2, Linf et du bilan de masse entre C_initial et C(T)
    local_err_l1 = 0.0_dp
    ! Affectation et initialisation numérique de la somme d'erreur quadratique L2
    local_err_l2 = 0.0_dp
    ! Affectation et initialisation numérique de l'erreur absolue maximale Linf
    local_err_linf = 0.0_dp
    ! Affectation et initialisation numérique de la masse initiale locale
    local_mass_init = 0.0_dp
    ! Affectation et initialisation numérique de la masse finale locale
    local_mass_final = 0.0_dp
    ! Affectation et calcul du nombre de cellules locales
    local_cells = real(this%mesh%nx_local * this%mesh%ny_local, dp)
    ! Affectation et calcul de l'aire géométrique élémentaire de cellule
    cell_vol = this%mesh%dx * this%mesh%dy

    ! Boucle spatiale sur les cellules locales selon l'axe vertical Y (ordonnées)
    do j = 1, this%mesh%ny_local
       ! Boucle spatiale selon l'axe horizontal X (accès contigu colonne-major)
       do i = 1, this%mesh%nx_local
```

```
932           ! Évaluation de l'écart absolu entre la fraction finale et la fraction exacte initiale
933           diff = abs(this%fields%C(i, j) - C_initial(i, j))
934           ! Accumulation pour la norme L1 moyenne
935           local_err_l1 = local_err_l1 + diff
936           ! Accumulation quadratique pour la norme L2
937           local_err_l2 = local_err_l2 + diff * diff
938           ! Recherche du maximum pour la norme Linf
939           if (diff > local_err_linf) local_err_linf = diff
940           ! Intégration de la masse initiale
941           local_mass_init = local_mass_init + C_initial(i, j) * cell_vol
942           ! Intégration de la masse finale
943           local_mass_final = local_mass_final + this%fields%C(i, j) * cell_vol
944        end do
945     end do
946
947     ! Réduction parallèle MPI globale de la somme d'erreur L1
948     global_err_l1 = global_sum(this%mesh, local_err_l1)
949     ! Réduction parallèle MPI globale de la somme quadratique L2
950     global_err_l2 = global_sum(this%mesh, local_err_l2)
951     ! Réduction parallèle MPI globale de l'erreur maximale Linf
952     global_err_linf = global_max(this%mesh, local_err_linf)
953     ! Réduction parallèle MPI globale de la masse totale initiale
954     global_mass_init = global_sum(this%mesh, local_mass_init)
955     ! Réduction parallèle MPI globale de la masse totale finale
956     global_mass_final = global_sum(this%mesh, local_mass_final)
957     ! Affectation et calcul du nombre global de cellules du maillage
958     global_cells  = global_sum(this%mesh, local_cells)
959
960     ! Normalisation de l'erreur moyenne L1
961     err_l1 = global_err_l1 / global_cells
962     ! Normalisation et racine de l'erreur quadratique moyenne L2
963     err_l2 = sqrt(global_err_l2 / global_cells)
964     ! Erreur maximale locale Linf
965     err_linf = global_err_linf
966     ! Évaluation du taux de conservation de masse du fluide
967     if (global_mass_init > 1.0e-14_dp) then
968        mass_conservation = global_mass_final / global_mass_init
969     else
970        mass_conservation = 1.0_dp
971     end if
972
973     ! Évaluation de la condition logique pour le processus maître de rang 0
974     if (this%mesh%rank == 0) then
975        write(*, '(A)') ""
976        write(*, '(A)') ">>> RESULTAT DU CAS 3.19 : ADVECTION VORTEX COVOF 2D <<<"
977        write(*, '(A,A)')       "  Methode Advection  : ", trim(this%config%advection_method)
978        write(*, '(A,A)')       "  Reconstruction     : ", trim(this%config%reconstruction_method)
979        write(*, '(A,A)')       "  Limiteur TVD/NVD   : ", trim(this%config%tvd_limiter)
980        write(*, '(A,I0,A,I0)') "  Maillage           : ", nx, " x ", ny
981        write(*, '(A,ES14.6)')  "  Erreur L1          : ", err_l1
982        write(*, '(A,ES14.6)')  "  Erreur L2          : ", err_l2
983        write(*, '(A,ES14.6)')  "  Erreur Linf        : ", err_linf
984        write(*, '(A,F10.5,A)') "  Conservation Masse : ", mass_conservation * 100.0_dp, " %"
985        ! Affectation et calcul numérique
986        write(*, '(A)') "============================================================"
987
988        ! Appel de sous-programme modulaire pour création du dossier de sortie
989        call execute_command_line("mkdir -p " // trim(this%config%output_dir))
990        ! Chemin d'accès au fichier CSV des erreurs et bilans de masse
991        err_file = trim(this%config%output_dir) // "/vortex_error.csv"
992        ! Ouverture en écriture du fichier récapitulatif
993        open(newunit=err_unit, file=trim(err_file), status="replace", action="write")
994        ! Écriture de l'en-tête CSV avec l'ensemble des métadonnées
995        write(err_unit, '(A)') "nx,ny,dt,T,advection,reconstruction,limiter,err_l1,err_l2,err_linf,mass_init,
               mass_final,mass_conservation"
996        ! Écriture formatée des valeurs numériques
997        write(err_unit, '(I0,",",I0,",",ES14.6,",",ES14.6,",",A,",",A,",",A,",",ES14.6,",",ES14.6,",",ES14
               .6,",",ES14.6,",",ES14.6,",",ES14.6)') &
998          nx, ny, dt, T_period, &
999          trim(this%config%advection_method), trim(this%config%reconstruction_method), trim(this%config%
                 tvd_limiter), &
1000         err_l1, err_l2, err_linf, global_mass_init, global_mass_final, mass_conservation
1001       ! Fermeture propre du flux fichier
1002       close(err_unit)
1003    end if
1004
1005    ! Appel de sous-programme modulaire
1006    call this%vtk_writer%finalize()
1007    ! Libération de la mémoire allouée
1008    deallocate(C_initial)
1009  end subroutine sim_run_vortex_advection
1010
1011 end module module_simulation
```